

1. Estado del Arte y Contextualización.....	11
1.1. La neumonía, un problema de salud global.....	11
1.2. La radiografía de tórax como imagen digital.....	13
1.2.1. ¿Qué es una radiografía de tórax y por qué es la prueba de referencia?.....	13
1.2.2. ¿Cómo ve el ordenador una radiografía?.....	13
1.2.3. Conjuntos de datos utilizados.....	14
1.3. Inteligencia Artificial, aprendizaje profundo y visión artificial.....	15
1.3.1. De la IA al aprendizaje profundo.....	15
1.3.2. La visión artificial.....	16
1.3.3. CheXNet y el estudio de Kermayn.....	16
1.4. Las redes neuronales convolucionales (CNN).....	17
1.4.1. Qué es una red neuronal convolucional.....	17
1.4.2. Las arquitecturas convolucionales del banco de pruebas.....	18
1.5. Los Transformers en visión artificial.....	21
1.5.1. El mecanismo de atención.....	21
1.5.2. De las palabras a los píxeles.....	22
1.5.3. Las arquitecturas Transformer del proyecto.....	23
1.6. La explicabilidad: inteligencia artificial explicable.....	24
1.6.1. El problema de la caja negra.....	24
1.6.2. Técnicas de explicabilidad.....	25
1.6.3. La evaluación cuantitativa.....	26
1.6.4. La calibración.....	27
1.7. Las barreras para la adopción clínica de la IA.....	28
1.8. Los asistentes conversacionales y los grandes modelos de lenguaje.....	30
1.8.1. ¿Qué son los grandes modelos de lenguaje?.....	30
1.8.2. El panorama de los modelos de lenguaje y la elección de Llama 3.....	31
1.9. Metodología experimental.....	32
1.9.1. Fase 1: Entrenamiento con validación cruzada.....	32
1.9.2. Fase 2: Comparación estadística entre modelos.....	36
1.9.3. Fase 3: Validación externa y test de DeLong.....	38
1.9.4. Fase 4: Análisis de explicabilidad.....	40
1.10. Tecnologías y arquitectura del sistema.....	43
2. Metas y Propósitos del Proyecto.....	44
3. Estructura organizativa y partes interesadas del proyecto.....	48
3.1. Estructura organizativa del proyecto.....	48
3.2. Partes interesadas del proyecto.....	50
3.3. Responsabilidades, funciones y participación de las partes interesadas.....	51
4. Marco Metodológico de Gestión.....	55
4.1. Justificación del enfoque metodológico.....	55
4.2. Roles y recursos del proyecto.....	56

4.2.1. Roles dentro del marco de trabajo.....	56
4.2.2. Dedicación y recursos materiales.....	56
4.3. Ciclo de trabajo iterativo.....	57
4.3.1. Los sprints y la cadencia del trabajo.....	57
4.3.2. Ceremonias y reuniones de seguimiento.....	57
4.3.3. Artefactos del proceso de gestión.....	58
4.4. Seguimiento visual del trabajo con Kanban.....	58
4.5. Alcance de la metodología.....	59
5. Cronograma y Plan de Trabajo.....	60
5.1. Marco general y objetivos de la planificación.....	60
5.2. Desglose de tareas por Sprint.....	61
5.2.1. Sprint 0 – Planificación del proyecto.....	63
5.2.2. Sprint 1 – Infraestructura y seguridad.....	65
5.2.3. Sprint 2 – Motor de diagnóstico.....	68
5.2.4. Sprint 3 – Explicabilidad.....	70
5.2.5. Sprint 4 – Entrenamiento CNN.....	73
5.2.6. Sprint 5 – Transformers y análisis XAI.....	76
5.2.7. Sprint 6 – Validación externa.....	79
5.2.8. Sprint 7 – Laboratorio MLOps.....	81
5.2.9. Sprint 8 – Documentación y cierre.....	84
5.3. Recursos y costes del proyecto.....	87
5.4. Reparto de responsabilidades.....	89
6. Identificación y Mitigación de Riesgos.....	90
6.1. Escala de valoración de riesgos.....	91
6.2. Catálogo de riesgos identificados.....	93
6.3. Plan de mitigación (medidas preventivas).....	95
6.4. Plan de contingencia (medidas correctivas).....	97
7. Planes de apoyo a la gestión del proyecto.....	98
7.1. Plan de Comunicación.....	99
7.2. Plan de Calidad.....	99
7.3. Plan de Gestión de la Configuración.....	100
8. Cuestiones abiertas y decisiones adoptadas.....	101
9. Consideraciones complementarias del proyecto.....	105
10. Planteamiento del análisis del sistema.....	107
10.1. Propósito del análisis.....	107
10.2. Dimensiones del análisis.....	107
11. Definición del ámbito del sistema.....	109
11.1. Objetivos del sistema.....	111
11.2. Contexto tecnológico y restricciones de entorno.....	124
11.3. Normativa y estándares de referencia.....	127
11.4. Perfiles de usuario del sistema.....	130

12. Especificación de requisitos del sistema.....	131
12.1. Obtención y organización de los requisitos.....	132
12.1.1. Requisitos funcionales del sistema.....	133
12.1.1.1. Módulo de autenticación y cuenta.....	133
12.1.1.2. Módulo de diagnóstico clínico.....	140
12.1.1.3. Módulo de historial de consultas.....	146
12.1.1.4. Módulo de laboratorio MLOps.....	150
12.1.1.5. Módulo de administración.....	166
12.1.1.6. Módulo transversal.....	169
12.1.1.7. Matriz de trazabilidad entre requisitos funcionales y objetivos del sistema....	175
12.1.2. Requisitos no funcionales del sistema.....	179
12.1.2.1. Seguridad de la plataforma y del acceso.....	179
12.1.2.2. Confidencialidad y protección de los datos.....	191
12.1.2.3. Rendimiento y capacidad de respuesta.....	198
12.1.2.4. Sencillez de uso y accesibilidad.....	202
12.1.2.5. Robustez y disponibilidad del servicio.....	206
12.1.2.6. Persistencia y salvaguarda de los datos.....	210
12.1.2.7. Reproducibilidad y sostenibilidad del desarrollo.....	212
12.1.2.8. Compromisos del proceso de desarrollo y entregables del proyecto.....	214
12.1.2.9. Matriz de trazabilidad entre requisitos no funcionales y objetivos del sistema.....	218
12.2. Especificación de los casos de uso.....	221
12.2.1. Diagramas de casos de uso por módulo.....	222
12.2.1.1. Gestión del acceso y de la cuenta.....	223
12.2.1.2. Interfaz de diagnóstico asistido.....	224
12.2.1.3. Laboratorio de experimentación MLOps.....	226
12.2.1.4. Supervisión y administración de la plataforma.....	228
12.2.1.5. Capacidades transversales de la plataforma.....	230
12.2.2. Actores del sistema y su ámbito de interacción.....	231
12.2.3. Actores del sistema y su ámbito de interacción.....	233
12.2.3.1. Módulo de gestión del acceso y de la cuenta.....	235
12.2.3.2. Módulo de interfaz de diagnóstico asistido.....	240
12.2.3.3. Módulo de laboratorio de experimentación MLOps.....	251
12.2.3.4. Módulo de supervisión y administración de la plataforma.....	268
12.2.3.5. Módulo de capacidades transversales de la plataforma.....	272
12.2.3.6. Matriz de trazabilidad requisitos – casos de uso.....	279
13. Análisis de subsistemas.....	286
13.1. SS-001 — Subsistema de acceso y gestión de cuentas.....	287
13.2. SS-002 — Subsistema de diagnóstico asistido.....	288
13.3. SS-003 — Subsistema de gestión del historial de consultas.....	289
13.4. SS-004 — Subsistema de laboratorio de experimentación MLOps.....	290

13.5. SS-005 — Subsistema de supervisión y administración de la plataforma.....	291
13.6. SS-006 — Subsistema de capacidades transversales de la plataforma.....	292
13.7. Síntesis de la descomposición en subsistemas.....	293
14. Modelo de dominio y entidades del sistema.....	294
14.1. Estructura estática de las entidades del sistema.....	295
14.2. Comportamiento dinámico del sistema y DSS.....	298
14.2.1. SS-001 – Subsistema de Acceso y Gestión de Cuentas.....	299
14.2.2. SS-002 – Subsistema de Diagnóstico Asistido.....	303
14.2.3. SS-003 – Subsistema de Gestión del Historial.....	310
14.2.4. SS-004 – Subsistema de Laboratorio MLOps.....	314
14.2.5. SS-005 – Subsistema de Supervisión y Administración.....	331
14.2.6. SS-006 – Subsistema de Capacidades Transversales.....	335
15. Verificación de la consistencia del análisis.....	338
15.1. Coherencia entre los objetivos del sistema y los requisitos.....	338
15.1.1. Cobertura de los objetivos por los requisitos funcionales.....	339
15.1.2. Cobertura de los objetivos por los requisitos no funcionales.....	341
15.2. Cobertura de los requisitos por los casos de uso y los subsistemas.....	342
15.2.1. Correspondencia entre requisitos funcionales y casos de uso.....	342
15.2.2. Correspondencia entre casos de uso y subsistemas de análisis.....	343
15.2.3. Cobertura de los requisitos no funcionales por los subsistemas.....	344
15.3. Decisiones estratégicas del análisis.....	345
16. Pruebas.....	347
16.1. Verificación de componentes individuales.....	348
16.1.1. Componentes del subsistema de acceso y gestión de cuentas.....	348
16.1.2. Componentes del subsistema de diagnóstico asistido.....	350
16.1.3. Componentes del subsistema de laboratorio MLOps.....	351
16.1.4. Componentes de los subsistemas de historial, administración y transversales.....	353
16.2. Verificación de los flujos entre subsistemas.....	355
16.3. Pruebas de protección y control de acceso.....	357
16.4. Pruebas de rendimiento y capacidad de respuesta.....	359
16.5. Verificación integral de los flujos completos de uso.....	360
17. Diseño y arquitectura de vitalXAI.....	363
17.1. Organización arquitectónica de la solución.....	364
17.1.1. Capa de presentación y acceso desde el navegador.....	364
17.1.2. Capa HTTP y coordinación de los subsistemas funcionales.....	365
17.1.3. Servicios de aplicación y motores de inteligencia artificial.....	365
17.1.4. Persistencia relacional y ejecución asíncrona.....	366
17.1.5. Vista global de la arquitectura.....	367
17.2. Condiciones de calidad, normativa y restricciones del diseño.....	369
17.2.1. Requisitos no funcionales y respuesta arquitectónica.....	369
17.2.1.1. Seguridad, identidad y control del acceso.....	371

17.2.1.2. Confidencialidad y tratamiento de la información.....	372
17.2.1.3. Rendimiento, concurrencia y ejecución sin bloqueo.....	372
17.2.1.4. Usabilidad, internacionalización y presentación.....	373
17.2.1.5. Robustez, persistencia y recuperación.....	373
17.2.2. Estándares y normas aplicables.....	374
17.2.3. Restricciones técnicas y límites del entorno.....	375
17.3. Subsistemas de Diseño.....	376
17.3.1. SD-001: Acceso, identidad y gestión de sesiones.....	377
17.3.2. SD-002: Diagnóstico asistido y generación de resultados.....	378
17.3.3. SD-003: Historial y gestión de consultas.....	380
17.3.4. SD-004: Laboratorio de experimentación MLOps.....	381
17.3.5. SD-005: Supervisión y administración.....	383
17.3.6. SD-006: Cola de trabajos y capacidades transversales.....	384
17.3.7. Relaciones de integración entre los subsistemas de diseño.....	386
17.4. Operación segura y controles de protección.....	388
17.4.1. Control de identidad, sesión y autorización.....	389
17.4.2. Protección de los datos y de los artefactos.....	390
17.4.3. Comunicaciones, configuración y secretos.....	391
17.4.4. Seguridad de la ejecución asíncrona.....	392
17.4.5. Persistencia, arranque y recuperación operativa.....	393
17.4.6. Límites actuales y responsabilidades del operador.....	394
18. Arquitectura de soporte y mecanismos transversales.....	394
18.1. Subsistemas de Soporte.....	396
18.1.1. SSOP-001: Persistencia relacional y conexiones de base de datos.....	397
18.1.2. SSOP-002: Almacenamiento de ficheros y recursos de aplicación.....	398
18.1.3. SSOP-003: Entorno de ejecución e integraciones externas.....	399
18.1.4. SSOP-003: Entorno de ejecución e integraciones externas.....	400
18.2. Mecanismos de diseño transversales.....	401
18.2.1. Configuración externa y separación de secretos.....	401
18.2.2. Validación en las fronteras de entrada.....	401
18.2.3. Autenticación, autorización y propiedad de recursos.....	402
18.2.4. Persistencia de estado y transacciones simples.....	402
18.2.5. Ejecución asíncrona y aislamiento de trabajos.....	403
18.2.6. Errores, estados y comunicación con el usuario.....	403
18.2.7. Modularidad, pruebas y evolución controlada.....	404
18.2.8. Ciclo de vida y sustitución controlada de componentes.....	404
19. Persistencia relacional y esquema de datos de vitalXAI.....	405
19.1. Diseño del esquema de persistencia.....	407
19.2. Acceso a la capa de persistencia.....	418
20. Diseño de los casos de uso.....	420
20.1. Subsistema de diseño SD-001: Acceso, identidad y gestión de sesiones.....	422

20.1.1. Diagrama de casos de uso del subsistema.....	422
20.1.2. Casos de Uso Reales.....	424
20.1.3. Diagramas de interacción entre objetos.....	427
20.2. Subsistema de diseño SD-002: Diagnóstico asistido y generación de resultados.....	436
20.2.1. Diagrama de casos de uso del subsistema.....	436
20.2.2. Casos de uso reales del subsistema.....	438
20.2.3. Diagramas de interacción entre objetos.....	442
20.3. Subsistema de diseño SD-003: Historial y gestión de consultas.....	451
20.3.1. Diagrama de casos de uso del subsistema.....	451
20.3.2. Casos de uso reales del subsistema.....	453
20.3.3. Diagramas de interacción entre objetos.....	455
20.4. Subsistema de diseño SD-004: Laboratorio de experimentación MLOps.....	464
20.4.1. Diagrama de casos de uso del subsistema.....	465
20.4.2. Casos de uso reales del subsistema.....	467
20.4.3. Diagramas de interacción entre objetos.....	474
20.5. Subsistema de diseño SD-005: Supervisión y administración.....	490
20.5.1. Diagrama de casos de uso del subsistema.....	491
20.5.2. Casos de uso reales del subsistema.....	493
20.5.3. Diagramas de interacción entre objetos.....	495
20.6. Subsistema de diseño SD-006: Cola de trabajos y capacidades transversales.....	501
20.6.1. Diagrama de casos de uso del subsistema.....	502
20.6.2. Casos de uso reales del subsistema.....	503
20.6.3. Diagramas de interacción entre objetos.....	505
21. Estructura de clases del diseño.....	509
21.1. Subsistema SD-001: Acceso, identidad y gestión de sesiones.....	510
21.1.1. Diagrama de clases del subsistema.....	511
21.1.2. Especificación de las clases.....	512
21.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados.....	516
21.2.1. Diagrama de clases del subsistema.....	516
21.2.2. Especificación de las clases.....	518
21.3. Subsistema SD-003: Historial y gestión de consultas.....	523
21.3.1. Diagrama de clases del subsistema.....	523
21.3.2. Especificación de las clases.....	525
21.4. Subsistema SD-004: Laboratorio de experimentación MLOps.....	526
21.4.1. Diagrama de clases del subsistema.....	526
21.4.2. Especificación de las clases.....	528
21.5. Subsistema SD-005: Supervisión y administración.....	535
21.5.1. Diagrama de clases del subsistema.....	535
21.5.2. Especificación de las clases.....	537
21.6. Subsistema SD-006: Cola de trabajos y capacidades transversales.....	538
21.6.1. Diagrama de clases del subsistema.....	538

21.6.2. Especificación de las clases.....	540
22. Diseño de interfaces de los subsistemas.....	543
22.1. Subsistema SD-001: Acceso, identidad y gestión de sesiones.....	546
22.1.1. Flujo de navegación.....	546
22.1.2. Especificación de las interfaces.....	547
22.1.3. Informes del subsistema.....	549
22.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados.....	550
22.2.1. Flujo de navegación.....	550
22.2.2. Especificación de las interfaces.....	551
22.2.3. Informes del subsistema.....	553
22.3. Subsistema SD-003: Historial y gestión de consultas.....	554
22.3.1. Flujo de navegación.....	554
22.3.2. Especificación de las interfaces.....	555
22.3.3. Informes del subsistema.....	556
22.4. Subsistema SD-004: Laboratorio de experimentación MLOps.....	557
22.4.1. Flujo de navegación.....	557
22.4.2. Especificación de las interfaces.....	559
22.4.3. Informes del subsistema.....	561
22.5. Subsistema SD-005: Supervisión y administración.....	563
22.5.1. Flujo de navegación.....	563
22.5.2. Especificación de las interfaces.....	565
22.5.3. Informes del subsistema.....	566
22.6. Subsistema SD-006: Cola de trabajos y capacidades transversales.....	567
22.6.1. Flujo de navegación.....	567
22.6.2. Especificación de las interfaces.....	568
22.6.3. Informes del subsistema.....	569
23. Especificación de la construcción del sistema.....	570
23.1. Entorno tecnológico de construcción.....	571
23.2. Paquetes y componentes de construcción.....	575
23.3. Proceso de construcción del software.....	578
23.4. Elaboración de Especificaciones del Modelo Físico de Datos.....	581
24. Preparación inicial de los datos del sistema.....	582
24.1. Entorno de Carga Inicial o Migración.....	583
24.2. Procedimientos de Carga Inicial o Migración.....	586
25. Estrategia de verificación del sistema.....	589
25.1. Entornos de verificación.....	590
25.2. Niveles de verificación y criterios de aceptación.....	593
25.2.1. Nivel unitario.....	594
25.2.2. Nivel de integración.....	595
25.2.3. Nivel de sistema.....	596
26. Implantación del sistema.....	596

26.1. Documentación necesaria para la operación.....	598
26.2. Requisitos de Implantación.....	600
27. Introducción a la codificación.....	604
27.1. Visión general de la implementación.....	605
27.2. Estructura del repositorio.....	606
27.3. Convenciones de código y decisiones transversales.....	608
27.4. Herramientas y calidad del código.....	608
28. Codificación del backend.....	609
28.1. Arranque y configuración de la aplicación.....	609
28.2. Capa HTTP: los routers.....	611
28.3. Servicios de aplicación.....	613
28.4. Persistencia y acceso a los datos.....	615
28.5. Seguridad transversal.....	617
29. Codificación de la ejecución asíncrona.....	619
29.1. La cola de trabajos y su máquina de estados.....	619
29.2. El bucle del worker.....	621
29.3. Reclamación, despacho y procesamiento de los trabajos.....	623
29.4. Tolerancia a fallos y recuperación.....	625
29.5. La cola desde la API.....	627
30. Codificación del motor de diagnóstico y XAI.....	629
30.1. Carga y caché de los modelos.....	629
30.2. Preprocesado y predicción.....	631
30.3. Generación de los mapas de explicabilidad.....	633
30.4. Generación del informe PDF.....	635
31. Codificación del laboratorio MLOps.....	637
31.1. Orquestación del pipeline de entrenamiento.....	637
31.2. Validación externa y comparación estadística.....	639
31.3. Gestión de las sesiones de experimentación.....	641
31.4. Lectura de los resultados.....	643
31.5. Generación del informe de la sesión.....	645
32. Codificación del frontend.....	647
32.1. Las plantillas de presentación.....	647
32.2. Los recursos JavaScript por ámbito.....	649
32.3. Comunicación asíncrona con la API.....	649
32.4. Internacionalización y tema visual.....	651
33. Estado del Arte y Contextualización.....	654
33.1. Alcance de la verificación.....	655
33.2. Entornos y herramientas de verificación.....	656
33.3. Organización de la parte.....	657
34. Pruebas automatizadas.....	658
34.1. Las pruebas unitarias.....	658

34.2. Las pruebas de integración.....	660
34.3. Ejecución y resultados de la batería.....	660
34.4. Cobertura de los módulos.....	661
35. Pruebas de seguridad.....	663
35.1. Protección CSRF.....	663
35.2. Cabeceras de seguridad.....	664
35.3. Limitación de peticiones.....	664
35.4. Gestión de los tokens de sesión.....	665
35.5. Resultados de la verificación de seguridad.....	666
36. Pruebas de rendimiento.....	667
36.1. Rendimiento de la inferencia.....	667
36.2. Ejecución sin bloqueo de la interfaz.....	668
36.3. Acceso concurrente.....	669
36.4. Condiciones de medición.....	669
37. Pruebas de sistema y validación funcional.....	670
37.1. PE-001: Flujo clínico completo.....	671
37.2. PE-002: Gestión del historial.....	674
37.3. PE-003: Flujo del laboratorio.....	677
37.4. PE-004: Ejecución asíncrona de la cola.....	681
37.5. PE-005: Supervisión administrativa.....	684
37.6. PE-006: Capacidades transversales.....	687
37.7. Resultados de la validación funcional.....	690

[OBJ]
[OBJ]
[OBJ]

Parte I.

Plan de Proyecto

1. Estado del Arte y Contextualización

El presente documento constituye el Plan de Proyecto para el Trabajo Fin de Grado titulado "vitalXAI: Plataforma MLOps con inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax", realizado por el alumno Luis Carmona Berdugo y tutorizado por el profesor Aurelio López Fernández. En él se detallan los objetivos identificados para el proyecto, la organización y planificación de las tareas, los riesgos evaluados y gestionados, así como los planes de gestión auxiliares necesarios para una ejecución correcta del mismo.

Este primer capítulo tiene una doble misión. Por una parte, sitúa al lector en el problema que motiva el proyecto: la neumonía, su diagnóstico mediante radiografías de tórax y las limitaciones de los métodos actuales. Por otra, recorre el estado del arte de las tecnologías implicadas —inteligencia artificial, redes neuronales convolucionales, Transformers, inteligencia artificial explicable y asistentes conversacionales basados en grandes modelos de lenguaje—, explicando cada concepto desde sus fundamentos para que el texto resulte comprensible para cualquier persona, tenga o no formación técnica. El capítulo concluye con una descripción de la metodología experimental empleada y de las tecnologías sobre las que se construye el sistema.

1.1. *La neumonía, un problema de salud global*

Para comprender la motivación de este trabajo es necesario empezar por el principio: qué es exactamente la neumonía. La neumonía es una infección que afecta a los pulmones, más concretamente a los alvéolos, que son las pequeñas bolsas de aire situadas al final de las vías respiratorias en las que se produce el intercambio de gases, es decir, el proceso por el cual la sangre recoge el oxígeno que respiramos y elimina el dióxido de carbono. Cuando una persona sufre una neumonía, los alvéolos se llenan de líquido o de pus, lo que dificulta gravemente esa función y provoca que el oxígeno no llegue con normalidad a la sangre. Esta es la razón por la que los síntomas típicos de la enfermedad incluyen tos, fiebre, dificultad para respirar y dolor en el pecho (Torres & al., 2021).

La neumonía puede estar causada por diferentes microorganismos, siendo los más frecuentes las bacterias y los virus, aunque también existen neumonías producidas por hongos. La causa concreta condiciona el tratamiento: las neumonías bacterianas se tratan habitualmente con antibióticos, mientras que las virales no responden a estos medicamentos y requieren, en general, cuidados de soporte. Distinguir el tipo de neumonía no siempre es sencillo, y las radiografías de tórax, junto con los síntomas y las pruebas de laboratorio, son herramientas clave para orientar el diagnóstico.

La relevancia clínica de la neumonía es enorme. Según la Organización Mundial de la Salud, la neumonía fue responsable de aproximadamente 2,5 millones de fallecimientos en el año 2019, lo que la convierte en la principal causa de muerte por infección en todo el mundo (World Health Organization, 2024). La enfermedad afecta de manera desproporcionada a los grupos más vulnerables de la población: los niños menores de cinco años y los adultos mayores de sesenta y cinco. En los países de ingresos bajos y medios, donde el acceso a la atención sanitaria es más limitado, la neumonía infantil es una de las primeras causas de mortalidad.

La detección temprana y el diagnóstico correcto son, por tanto, cuestiones de vida o muerte. Un diagnóstico tardío retrasa el inicio del tratamiento adecuado y puede derivar en complicaciones graves. Ahora bien, diagnosticar neumonía no siempre es una tarea trivial, ni siquiera para especialistas con mucha experiencia. Los estudios muestran una variabilidad significativa en la interpretación de las radiografías de tórax entre distintos observadores: dos radiólogos pueden no coincidir sobre si una imagen muestra o no signos de neumonía, y esta variabilidad puede conducir a diagnósticos erróneos o retrasados con consecuencias clínicas importantes (Hopstaken & al., 2004). Esta es precisamente una de las grietas sobre las que se asienta la motivación de este proyecto: la posibilidad de construir herramientas informáticas que ayuden a los profesionales sanitarios a tomar decisiones mejor fundamentadas.

1.2. La radiografía de tórax como imagen digital

1.2.1. ¿Qué es una radiografía de tórax y por qué es la prueba de referencia?

La radiografía de tórax es una prueba de imagen que utiliza rayos X para obtener una representación del interior del tórax. Los rayos X son una forma de radiación electromagnética de alta energía que atraviesa los tejidos del cuerpo en mayor o menor medida según su densidad. Los huesos, que son muy densos, absorben la mayor parte de la radiación y aparecen en la imagen de color blanco; los tejidos blandos, como el músculo o el corazón, la absorben en menor grado y se muestran en tonos grises; y las zonas llenas de aire, como los pulmones sanos, apenas la absorben y aparecen prácticamente en negro (Gonzalez & Woods, 2008).

Esta propiedad es la que permite al radiólogo detectar una neumonía: cuando los alvéolos se llenan de líquido o de pus, la zona afectada del pulmón deja de ser transparente a los rayos X y se muestra como una mancha más blanca y densa de lo normal, lo que se conoce como opacidad o infiltrado (Franquet, 2001). La radiografía de tórax es la técnica de imagen de primera línea para el diagnóstico de la neumonía porque combina tres ventajas fundamentales: es muy accesible (existe en prácticamente todos los centros sanitarios), tiene un coste bajo en comparación con otras técnicas como el TAC, y es muy rápida de adquirir. Por estos motivos, sigue siendo la herramienta estándar que se utiliza en primer lugar ante la sospecha de neumonía (Mandell & al., 2007).

1.2.2. ¿Cómo ve el ordenador una radiografía?

Una comprensión importante para el resto del capítulo es cómo se representa una radiografía en términos informáticos, porque es la información con la que trabajarán los algoritmos. Una imagen digital no es más que una cuadrícula bidimensional de puntos llamados píxeles. En el caso de una radiografía en escala de grises, cada píxel guarda un único número que indica el nivel de brillo en ese punto, que normalmente va de 0 (negro puro, es decir, zona muy transparente a los rayos X) a 255 (blanco puro, zona muy densa). Así, una radiografía de, por ejemplo, 512 píxeles de ancho por 512 de alto no es, para un ordenador, una "foto de un tórax", sino una tabla de $512 \times 512 = 262.144$ números (Szeliski, 2010).

Este punto es crucial para entender todo lo que viene después: los algoritmos de visión artificial no "ven" imágenes como los humanos, sino que trabajan con grandes cantidades de números organizados en estructuras matemáticas llamadas tensores. Las redes neuronales profundas aprenden a extraer patrones a partir de esos números, como se explicará en las siguientes secciones.

1.2.3. Conjuntos de datos utilizados

Para entrenar y evaluar los modelos de inteligencia artificial es imprescindible disponer de un conjunto de imágenes etiquetadas, es decir, imágenes de las que se sabe con certeza si corresponden o no a un paciente con neumonía. En este proyecto se emplean dos conjuntos de datos públicos y anonimizados, siguiendo la metodología establecida en el estudio de referencia (Kermany & al., 2018).

El conjunto de datos de entrenamiento es el publicado por Kermany y colaboradores, procedente del Guangzhou Women and Children's Medical Center y disponible en la plataforma Kaggle. Este dataset contiene 5.856 radiografías de tórax pediátricas (de pacientes niños) en formato JPEG, clasificadas en dos categorías: NORMAL, con 1.583 imágenes, y PNEUMONIA, con 4.273 imágenes. La categoría PNEUMONIA se subdivide a su vez en neumonía bacteriana y neumonía viral. El hecho de que el dataset contenga muchas más imágenes de neumonía que de pacientes sanos tiene un nombre técnico, desbalanceo de clases, y constituye un desafío para el entrenamiento de los modelos que se abordará más adelante en la metodología. Este conjunto se selecciona por ser el mismo utilizado en el estudio de referencia, lo que permite establecer una comparación directa con los resultados publicados por Kermany y colaboradores.

El segundo conjunto de datos se emplea para la validación externa, un concepto que se desarrollará en profundidad más adelante y que, de forma resumida, consiste en comprobar que los modelos funcionan también con imágenes que nunca han visto y que proceden de un entorno distinto al del entrenamiento. Este segundo conjunto es el COVID-19 Radiography Database, desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka (Chowdhury & al., 2020; Rahman & al., 2021). Se trata de radiografías de tórax de pacientes adultos, adquiridas con equipos y protocolos diferentes a los del dataset pediátrico de entrenamiento. De este conjunto se extraen quinientas imágenes de pacientes sanos y quinientas de pacientes con neumonía viral, garantizando así un balance perfecto entre las dos clases. La población adulta, el equipo de adquisición y los protocolos de imagen diferentes convierten esta prueba en un escenario potencialmente exigente para evaluar la capacidad de generalización de los modelos. No obstante, el conjunto combina imágenes normales y patológicas procedentes de fuentes y repositorios distintos, por lo que existe el riesgo de que un modelo aprenda señales relacionadas con el origen de la imagen en lugar de la patología (DeGrave, Janizek, & Lee, 2021).

Es importante señalar una cuestión de honestidad metodológica: durante la fase de desarrollo y depuración del código, y para poder ejecutar pruebas de forma ágil y con tiempos de computación razonables, se han utilizado subconjuntos reducidos de ambos conjuntos de datos. Estos subconjuntos permiten validar que el código funciona correctamente sin necesidad de procesar las miles de imágenes completas en cada iteración de desarrollo. Sin embargo, la metodología del proyecto está concebida y configurada para trabajar con los conjuntos de datos completos originales: las 5.856 imágenes de Kermany para el entrenamiento y las mil imágenes (500 normales y 500 con neumonía) del COVID-19 Radiography Database para la validación externa.

1.3. *Inteligencia Artificial, aprendizaje profundo y visión artificial*

1.3.1. De la IA al aprendizaje profundo

El término inteligencia artificial (IA) engloba todas las técnicas que pretenden dotar a las máquinas de capacidades consideradas propias de la inteligencia humana, como el razonamiento, el aprendizaje o el reconocimiento de patrones. Dentro de la IA existe una rama denominada aprendizaje automático (machine learning), cuyo principio es que el ordenador no es programado con reglas explícitas para resolver una tarea, sino que aprende esas reglas a partir de ejemplos (Bishop, 2006). En lugar de decirle a la máquina "una neumonía se ve como una mancha blanca con tal forma y tal posición", se le muestran miles de radiografías con su diagnóstico correcto y se le permite deducir por sí misma cuáles son los patrones que distinguen una imagen con neumonía de una sana.

El aprendizaje profundo (deep learning) es una subdisciplina del aprendizaje automático que se basa en redes neuronales artificiales con muchas capas (de ahí lo de "profundo"). Una red neuronal es un modelo matemático inspirado, de forma muy simplificada, en el funcionamiento del cerebro. Está formada por unidades de procesamiento llamadas neuronas, organizadas en capas. Cada neurona recibe varias entradas, las multiplica por unos valores numéricos llamados pesos (que representan la "importancia" de cada entrada), las suma y aplica una función matemática denominada función de activación que decide si la neurona se "activa" o no. La primera capa recibe los datos de entrada (por ejemplo, los píxeles de una radiografía), la última produce el resultado (por ejemplo, la probabilidad de que haya neumonía), y las capas intermedias, llamadas capas ocultas, transforman progresivamente la información (LeCun, Bengio, & Hinton, 2015).

El proceso por el cual la red aprende se denomina entrenamiento, y su idea es sorprendentemente sencilla aunque matemáticamente sofisticada (Goodfellow, Bengio, & Courville, 2016):

1. **Propagación hacia delante:** se introduce una imagen en la red y se calcula su salida con los pesos actuales.
2. **Cálculo del error:** se compara la salida de la red con la respuesta correcta mediante una función matemática denominada función de coste o pérdida (loss), que cuantifica cuánto se ha equivocado la red.
3. **Propagación hacia atrás:** se calcula cómo afectaría una pequeña variación de cada peso al error final, mediante un algoritmo llamado retropropagación del error.
4. **Actualización de los pesos:** se modifican ligeramente los pesos en la dirección que reduce el error, mediante un algoritmo de optimización como el descenso del gradiente.

Este ciclo se repite con todas las imágenes del conjunto de entrenamiento, y el resultado es que, progresivamente, la red va ajustando sus pesos hasta producir salidas cada vez más cercanas a las correctas. En el caso concreto de este proyecto, las redes se entrenan para resolver un problema de clasificación binaria: dada una radiografía, decidir si pertenece a la clase PNEUMONIA o a la clase NORMAL. La red no devuelve una etiqueta rígida, sino una probabilidad entre 0 y 1 (la salida de una neurona con activación sigmoide), que puede interpretarse como el nivel de confianza del modelo.

1.3.2. La visión artificial

La visión artificial (computer vision) es la rama de la inteligencia artificial que se ocupa de que las máquinas puedan interpretar el mundo visual: imágenes, vídeos, fotografías y, en el caso que nos ocupa, radiografías (Szeliski, 2010). La tarea concreta que aborda este proyecto es la clasificación de imágenes: asignar una categoría (en este caso, "neumonía" o "sano") a cada imagen de entrada.

Aplicar visión artificial a la radiografía de tórax es especialmente atractivo porque convierte un proceso que requiere años de formación médica en una tarea automatizable: dada una imagen, producir una decisión. Pero automatizar no significa simplificar; la interpretación de radiografías es sutil y los algoritmos deben ser capaces de captar diferencias muy finas en las texturas y densidades del pulmón.

1.3.3. CheXNet y el estudio de Kermany

Para contextualizar el estado del arte es útil repasar los dos hitos científicos que sustentan directamente este proyecto.

El primero es CheXNet, presentado por Rajpurkar y colaboradores en 2017 (Rajpurkar & al., 2017). CheXNet es una red neuronal convolucional de 121 capas (basada en la arquitectura DenseNet, que se explicará más adelante) entrenada sobre el conjunto ChestX-ray14, una base de datos que contiene más de 112.000 radiografías de tórax de unos 30.000 pacientes, con anotaciones para catorce enfermedades torácicas diferentes. El logro principal del estudio fue demostrar, de manera empírica, que una red neuronal podía igualar e incluso superar el rendimiento de un panel de radiólogos expertos en la detección de neumonía a partir de radiografías de tórax. Este resultado supuso un punto de inflexión porque mostró que la tecnología ya no era solo prometedora, sino que alcanzaba niveles de rendimiento comparables a los humanos en una tarea clínica real.

El segundo hito es el estudio de Kermany y colaboradores, publicado en la revista Cell en 2018 (Kermany & al., 2018), que constituye una referencia metodológica y de datos para este proyecto. El trabajo presenta un marco de aprendizaje profundo basado en transferencia de conocimiento y demuestra su aplicación a la identificación de neumonía pediátrica mediante radiografías de tórax, utilizando el conjunto de 5.856 imágenes descrito en la sección anterior. Su aportación no consiste en comparar sistemáticamente varias arquitecturas convolucionales, sino en mostrar cómo un sistema de aprendizaje profundo puede utilizarse para apoyar el diagnóstico por imagen y acompañar sus predicciones de elementos interpretables. La comparación con el rendimiento de expertos se refiere principalmente a las tareas de diagnóstico oftalmológico analizadas en el estudio, por lo que no debe trasladarse directamente a una afirmación de equivalencia entre el modelo de neumonía y los radiólogos. El artículo resulta relevante para este proyecto por el uso del dataset pediátrico, la transferencia de conocimiento y la preocupación por la interpretabilidad, pero no constituye un benchmark de cinco arquitecturas de neumonía.

El presente trabajo toma esas ideas como punto de partida y desarrolla una propuesta experimental propia en tres direcciones: compara diecinueve arquitecturas —incluyendo redes convolucionales y Vision Transformers— bajo un mismo pipeline, incorpora una evaluación cuantitativa de la explicabilidad y de la calibración de las predicciones, y envuelve toda la metodología en una plataforma web MLOps accesible para profesionales sanitarios sin formación técnica. Por tanto, la novedad no se formula como la ampliación de un supuesto banco de cinco modelos de Kermany, sino como la construcción de un entorno comparativo y operativo que aplica el dataset y la motivación metodológica del trabajo de referencia a una evaluación más amplia.

1.4. Las redes neuronales convolucionales (CNN)

1.4.1. Qué es una red neuronal convolucional

Las redes neuronales convolucionales (del inglés convolutional neural networks, CNN) son el tipo de arquitectura que ha dominado históricamente la visión artificial. Su nombre procede de la operación matemática central que realizan: la convolución.

Para entender qué es una convolución conviene partir de una idea intuitiva. La visión de una CNN se puede comparar con la de un detective con una lupa que recorre la imagen sistemáticamente. La red utiliza pequeños filtros o kernels, que no son más que cuadrículas de números (por ejemplo, de 3×3 o 5×5) que se deslizan por toda la imagen y detectan patrones locales. Cada filtro está especializado en buscar un patrón concreto: algunos detectan bordes verticales, otros horizontales, otros texturas determinadas. El resultado de aplicar un filtro sobre la imagen se denomina mapa de características o feature map, y la particularidad fundamental de la convolución es que el mismo filtro se aplica sobre toda la imagen, lo que se denomina compartición de parámetros. Esto reduce enormemente el número de parámetros del modelo y hace que la respuesta del filtro se desplace de forma coherente cuando también se desplaza el patrón detectado; esta propiedad se conoce como equivanianza a la traslación. Posteriormente, las capas de agrupación (pooling) reducen la resolución espacial y condensan la información de regiones próximas, favoreciendo una mayor invariancia local a pequeños desplazamientos (Krizhevsky, Sutskever, & Hinton, 2012).

Las CNN se estructuran en capas que extraen progresivamente características cada vez más abstractas. Las primeras capas detectan elementos sencillos como bordes y cambios de brillo; las capas intermedias combinan estos elementos en texturas y formas; y las capas finales integran el todo en conceptos de alto nivel relevantes para la tarea, en nuestro caso, la presencia o ausencia de las opacidades características de la neumonía. Entre las capas convolucionales se intercalan capas de agrupación (pooling) que reducen el tamaño de los mapas de características, condensando la información y ayudando a la red a ser más robusta y eficiente. Al final de la red, las características se aplanan y se conectan a una o varias capas densas (como las de cualquier red neuronal clásica) que producen la clasificación final.

En el contexto de la neumonía, lo que una CNN bien entrenada aprende a hacer es identificar en la radiografía las regiones de densidad anómala en los campos pulmonares que los radiólogos asocian con la infección, sin que se le haya indicado explícitamente qué aspecto tiene una opacidad. Aprende por sí misma a partir de los ejemplos.

1.4.2. Las arquitecturas convolucionales del banco de pruebas

Este proyecto evalúa dieciséis arquitecturas convolucionales. Para no abrumar al lector con una descripción técnica exhaustiva de cada una, se agrupan a continuación por familias, explicando la idea esencial de cada familia y la particularidad de cada modelo, siempre en el contexto de la detección de neumonía. Todas ellas están preentrenadas sobre el conjunto de imágenes naturales ImageNet, lo que se explicará con más detalle en la sección de metodología.

Familia ResNet (He, Zhang, Ren, & Sun, 2016): el problema que ResNet vino a resolver es el de la degradación de las redes muy profundas. Cuando una red tiene decenas o cientos de capas, entrenarla de forma directa se vuelve extremadamente difícil, y el rendimiento, lejos de mejorar, empeora. La solución propuesta son las conexiones residuales o saltos: cada bloque de la red aprende no una transformación completa, sino la diferencia (el residuo) entre su entrada y su salida, sumando la entrada original. Esta idea, aparentemente sencilla, permitió entrenar redes de cientos de capas y es una de las arquitecturas más influyentes de la década. En este proyecto se evalúan:

- ResNet50: la versión de 50 capas, la más popular y equilibrada de la familia.
- ResNet101: versión más profunda (101 capas), que puede captar patrones más complejos a costa de mayor coste computacional.
- ResNet152: la más profunda de la serie (152 capas), con mayor capacidad pero también mayor demanda de recursos.
- ResNet50V2: una revisión de ResNet50 que reorganiza el orden de las operaciones dentro de cada bloque (normalizando antes de la convolución), lo que mejora el flujo de gradientes y, en general, la convergencia del entrenamiento (He, Zhang, Ren, & Sun, 2016).

Familia DenseNet (Huang, Liu, Van Der Maaten, & Weinberger, 2017): DenseNet lleva la idea de las conexiones a un extremo: en lugar de saltos puntuales, cada capa recibe como entrada las características de todas las capas anteriores, concatenándolas. Esto se denomina conectividad densa y favorece la reutilización de las características, el flujo de información y la regularización natural. DenseNet es especialmente relevante para este proyecto porque CheXNet, el sistema que igualó a los radiólogos en la detección de neumonía, se basaba precisamente en esta arquitectura (Rajpurkar & al., 2017). Se evalúan:

- DenseNet121: la versión de 121 capas, la más ligera y utilizada de la familia.
- DenseNet201: la versión más profunda (201 capas), con mayor capacidad de representación.

Familia EfficientNet (Tan & Le, 2019): EfficientNet plantea la pregunta de cómo escalar una red de forma óptima. En lugar de decidir a mano cuánto agrandar la profundidad, la anchura o la resolución de la imagen, propone un método de escalado compuesto que equilibra las tres dimensiones mediante un coeficiente único. El resultado es una familia de modelos que ofrece una excelente relación entre precisión y coste computacional. Se evalúan:

- EfficientNetB0: el modelo base de la familia, el más ligero.
- EfficientNetB3 y EfficientNetB7: versiones escaladas con mayor capacidad.
- EfficientNetV2S: la segunda generación de EfficientNet, que mejora la eficiencia del entrenamiento introduciendo capas de entrenamiento progresivo y ajustes en las operaciones (Tan & Le, 2021).

Familia MobileNet (Howard & al., 2017): MobileNet está diseñada para funcionar en dispositivos con recursos limitados, como teléfonos móviles. Su idea clave es la convolución separable por profundidad, que descompone una convolución estándar en dos operaciones más baratas, reduciendo drásticamente el número de parámetros y de operaciones sin sacrificar demasiada precisión. Se evalúan:

5. MobileNetV2: segunda generación, que introduce los bloques residuales invertidos (Sandler & al., 2018).
6. MobileNetV3Large: tercera generación, optimizada además con técnicas de búsqueda de arquitecturas (Howard & al., 2019).

VGG16 (Simonyan & Zisserman, 2015): VGG demostró que una arquitectura muy simple —una pila uniforme de capas convolucionales de 3×3 seguidas de capas de agrupación— podía alcanzar resultados excelentes si era lo suficientemente profunda. Su simplicidad la convierte en un modelo de referencia ideal para comparar arquitecturas más sofisticadas.

InceptionV3 (Szegedy & al., 2016): la idea de Inception es aplicar varios filtros de distinto tamaño (1×1 , 3×3 y 5×5) en paralelo sobre la misma región, de modo que la red pueda captar patrones a diferentes escalas simultáneamente. InceptionV3 refina esta idea con convoluciones factorizadas que reducen el coste computacional.

Xception (Chollet, 2017): Xception lleva la idea de Inception al extremo y se basa exclusivamente en convoluciones separables por profundidad, demostrando que esta operación, combinada de la forma adecuada, puede superar a las arquitecturas clásicas de Inception. Es una arquitectura especialmente eficiente en términos de parámetros.

ConvNeXtTiny (Liu & al., 2022): ConvNeXt es una relectura moderna de las CNN: tomando como base ResNet, incorpora de forma sistemática las ideas que hicieron exitosos a los Transformers (normalizaciones, funciones de activación modernas, menor número de bloques de normalización), demostrando que las CNN "modernizadas" pueden competir con los Transformers en igualdad de condiciones. ConvNeXtTiny es la versión pequeña de esta familia.

La selección de estas dieciséis arquitecturas no es casual: cubre un espectro muy amplio del diseño de redes convolucionales (desde modelos ligeros y eficientes como MobileNetV2 hasta modelos de gran capacidad como ConvNeXt-Tiny), y todas ellas han demostrado un rendimiento destacado en clasificación de imágenes, lo que permite realizar una comparativa rica y representativa sobre la tarea de la detección de neumonía (Zhang & al., 2024).

RED NEURONAL CONVOLUCIONAL (CNN) PARA RADIOGRAFÍAS DE TÓRAX

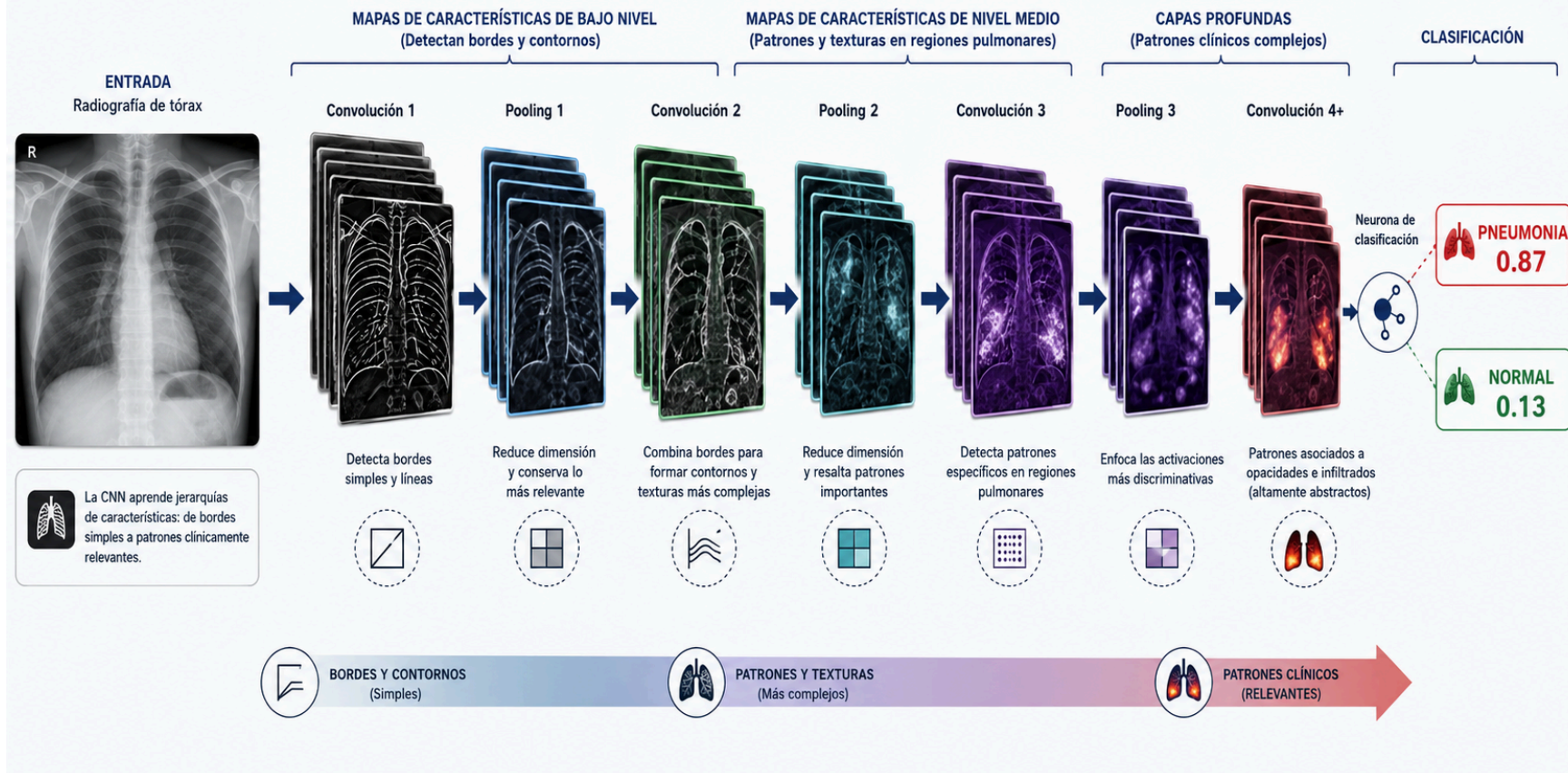


Ilustración x - Representación CNN para Radiografías de Tórax

1.5. *Los Transformers en visión artificial*

1.5.1. El mecanismo de atención

Para entender los modelos más recientes de este proyecto es necesario introducir otra familia de arquitecturas: los Transformers. Su historia comienza en el procesamiento del lenguaje natural. Durante años, las tareas de lenguaje se resolvían con redes recurrentes que procesaban las palabras de una frase una a una, en orden, manteniendo un estado interno. Este enfoque era lento y tenía dificultades para recordar relaciones entre palabras muy separadas dentro del texto.

En 2017, el artículo "Attention is all you need" (Vaswani & al., 2017) propuso una arquitectura radicalmente distinta: el Transformer, basada en un mecanismo denominado atención. La idea esencial de la atención es que, al procesar un elemento de una secuencia (por ejemplo, una palabra), el modelo puede "prestar atención" a todos los demás elementos de la secuencia simultáneamente, ponderando cuánto influye cada uno de ellos. Así, al interpretar la palabra "neumonía" en la frase "la radiografía muestra una neumonía en el lóbulo inferior", el modelo aprende a relacionarla con "radiografía" y "lóbulo inferior" aunque estén lejos en la frase.

Dos conceptos acompañan a la atención: la **autoatención**, que establece relaciones entre todos los elementos de una misma secuencia (cada palabra se relaciona con todas las demás), y las **cabezas de atención**, que son múltiples mecanismos de atención ejecutados en paralelo, cada uno especializado en un tipo distinto de relación. Además, como el Transformer no procesa los elementos en orden, necesita incorporar una **codificación posicional**, una información que le indica en qué posición de la secuencia se encuentra cada elemento (Vaswani & al., 2017).

1.5.2. De las palabras a los píxeles

Los Transformers demostraron ser extraordinarios en el lenguaje, pero las imágenes parecían un territorio hostil para ellos, porque una imagen no es una secuencia de símbolos. La idea que resolvió este problema fue sorprendentemente directa: tratar la imagen como si fuera una secuencia de palabras.

El Vision Transformer (ViT) propuesto por Dosovitskiy y colaboradores en 2021 (Dosovitskiy & al., 2021) divide la imagen en una cuadrícula de pequeños fragmentos cuadrados llamados patches (parches). Cada parche se convierte en un vector numérico mediante una transformación lineal, de forma análoga a como cada palabra se convierte en un vector que la representa. Estos vectores, llamados tokens, se introducen en un Transformer junto con su codificación posicional, de modo que el modelo aplica autoatención entre todos los parches de la imagen. Para producir la clasificación final, el ViT utiliza un token especial denominado token de clasificación, cuyo estado final después de atravesar el modelo se conecta a una capa de clasificación.

La diferencia conceptual con las CNN es profunda y muy relevante para este proyecto. Una CNN procesa la imagen de forma local y progresiva: cada neurona solo "ve" una pequeña región, y la visión global se construye capa a capa. Un Transformer, en cambio, relaciona desde la primera capa todas las regiones de la imagen entre sí, lo que le permite captar dependencias de largo alcance que a una CNN podrían pasarle desapercibidas. En el contexto de la neumonía, esto significa que un Transformer puede relacionar directamente, por ejemplo, la densidad anómala de un lóbulo pulmonar con la expansión del pulmón opuesto, sin necesidad de que esa relación emerja capa tras capa. La contrapartida es que los Transformers son muy "hambrientos de datos": necesitan grandes cantidades de imágenes para aprender correctamente y tienen un coste computacional elevado, especialmente cuando la imagen se divide en muchos parches (Dosovitskiy & al., 2021). Precisamente por eso su aplicación al ámbito médico, donde los conjuntos de datos suelen ser reducidos, exige una evaluación cuidadosa, que es uno de los objetivos de este trabajo.

1.5.3. Las arquitecturas Transformer del proyecto

Este proyecto evalúa tres arquitecturas basadas en atención:

- **DeiT** (Data-efficient Image Transformers) (Touvron & al., 2021): DeiT aborda directamente el problema del hambre de datos de los ViT. Su idea central es el uso de un token de destilación: durante el entrenamiento, un profesor (típicamente una CNN muy potente) "enseña" al Transformer, y el token de destilación es un elemento adicional que el modelo aprende a usar para imitar al profesor. Esto permite que un Transformer se entrene correctamente con muchos menos datos. En este proyecto se emplea la variante *deit-base-distilled*, preentrenada a 224×224 píxeles.
- **Swin Transformer** (Liu & al., 2021): Swin ataca el problema del coste computacional de los Transformers. En lugar de calcular la autoatención sobre todos los parches de la imagen (lo que crece de forma cuadrática con el número de parches), Swin la calcula dentro de ventanas locales, que además se desplazan entre capas consecutivas (ventanas desplazadas). Este diseño jerárquico, inspirado en las CNN, reduce el coste y permite captar información a múltiples escalas. La variante evaluada es *swin-base*, con parches de 4×4 y ventanas de 7×7 , a resolución 224×224 .
- **ViT-384**: es el Vision Transformer estándar, pero configurado para trabajar con una resolución de entrada mayor (384×384 píxeles en lugar de los 224 habituales). Trabajar con más píxeles equivale a dividir la imagen en más parches, lo que proporciona más detalle espacial al modelo (al precio de un mayor coste computacional). La variante utilizada es *vit-base-patch16-384*.

La inclusión de estos tres modelos permite comparar de forma sistemática el paradigma de las CNN con el de los Transformers dentro de un mismo diseño de evaluación, aunque no bajo un protocolo de optimización idéntico. Se mantienen comunes las particiones estratificadas, el número de pliegues, el balanceo del entrenamiento, la aumentación, el conjunto de validación y las métricas; sin embargo, las CNN utilizan una base convolucional congelada y entrenan un nuevo cabezal, mientras que los Transformers ajustan sus pesos preentrenados con una tasa de aprendizaje menor y AdamW. Por ello, los resultados deben interpretarse como una comparación exploratoria de dos familias mediante protocolos adaptados a sus características, no como una atribución causal de cualquier diferencia exclusivamente a la arquitectura (Zhang & al., 2024).

1.6. *La explicabilidad: inteligencia artificial explicable*

1.6.1. El problema de la caja negra

Los modelos de aprendizaje profundo, tanto las CNN como los Transformers, comparten una característica que es, a la vez, su mayor fortaleza y su mayor debilidad: son cajas negras. Internamente están formados por millones de pesos numéricos que se ajustan durante el entrenamiento, y aunque la red sea capaz de clasificar correctamente una radiografía, no es evidente por qué ha llegado a esa conclusión. Un médico no puede aceptar sin más que un sistema le diga "este paciente tiene neumonía": necesita saber si el modelo se está fijando en la zona correcta del pulmón o si, por el contrario, está siendo engañado por algún artefacto de la imagen, una etiqueta o una peculiaridad del conjunto de entrenamiento.

Esta exigencia de transparencia es especialmente crítica en el ámbito clínico, donde las decisiones deben estar fundamentadas y ser auditable. La inteligencia artificial explicable (XAI) es la disciplina que estudia cómo hacer comprensibles las decisiones de los modelos de aprendizaje automático (Holzinger & al., 2022; Linardatos, Papastefanopoulos, & Kotsiantis, 2021; Tjoa & Guan, 2020). No se trata únicamente de generar una imagen bonita con zonas coloreadas, sino de proporcionar al profesional sanitario una herramienta que le permita verificar, de forma visual e intuitiva, qué regiones de la radiografía han sido determinantes para la decisión del modelo. De este modo, si el sistema apunta correctamente a las zonas de opacidad características de la neumonía, el clínico puede confiar en la predicción; si apunta a regiones irrelevantes, la predicción debe ser puesta en duda (Rajaraman & Antani, 2018).

1.6.2. Técnicas de explicabilidad

Este proyecto utiliza tres familias de técnicas de explicabilidad:

Mapas de prominencia o Saliency Maps (Simonyan, Vedaldi, & Zisserman, 2014): la idea es preguntarse directamente al modelo qué píxeles de la imagen han influido más en su decisión. Para ello se calcula la derivada (el gradiente) de la salida de la red respecto a cada píxel de la imagen de entrada. Intuitivamente, el gradiente indica cómo cambiaría la predicción si se modificase un píxel concreto: si un pequeño cambio en un píxel altera mucho la decisión, ese píxel es importante. El resultado es un mapa que resalta las regiones más influyentes.

SmoothGrad (Smilkov & al., 2017): los mapas de saliencia crudos suelen ser muy ruidosos, con pequeñas variaciones que no tienen significado clínico. SmoothGrad mejora esta situación añadiendo ruido aleatorio a la imagen de entrada y promediando los mapas de saliencia obtenidos sobre varias versiones con ruido. El resultado es un mapa más suave y estable, en el que los patrones realmente consistentes del modelo destacan sobre el ruido.

Grad-CAM (Selvaraju & al., 2017): Grad-CAM produce mapas de activación de clase, que resaltan las regiones de la imagen que más contribuyen a la clase predicha. Para ello combina los mapas de características de la última capa convolucional del modelo con los gradientes de la clase predicha, ponderando cada mapa de características por su importancia. El resultado es un mapa de calor grueso y fácil de interpretar, que se superpone a la radiografía original. Grad-CAM es especialmente valioso en medicina porque sus mapas son suaves y corresponden de forma natural a las regiones anatómicas que el clínico puede inspeccionar.

Para las arquitecturas Transformer se emplea una variante complementaria: los mapas de atención, que extraen directamente los pesos de atención de la última capa del modelo. Como se explicó en la sección anterior, los Transformers deciden cuánta importancia prestar a cada parche de la imagen; esos pesos de atención indican, por tanto, qué regiones de la radiografía está "mirando" el modelo (Chefer, Gur, & Wolf, 2021). En este proyecto, los mapas de atención se obtienen promediando las cabezas de atención de la última capa.

1.6.3. La evaluación cuantitativa

Observar mapas de calor a simple vista es necesario, pero no es suficiente para una evaluación científica rigurosa: dos observadores pueden opinar de forma distinta sobre la calidad de un mismo mapa. Por ello, este proyecto incorpora métricas cuantitativas que puntúan de forma objetiva distintas propiedades de las explicaciones generadas. La evaluación de fidelidad mediante borrado e inserción se fundamenta en Petsiuk, Das y Saenko (2018), mientras que la consideración de medidas de concentración y complejidad se encuadra en la literatura general sobre evaluación de métodos XAI (Linardatos, Papastefanopoulos, & Kotsiantis, 2021):

- **Deletion AUC** (área bajo la curva de borrado): mide qué ocurre cuando se borran progresivamente de la imagen las regiones que el mapa considera más importantes. Si al borrarlas la predicción del modelo se degrada rápidamente, el mapa señala realmente las regiones determinantes (cuanto más baja la curva, mejor la explicación).
- **Insertion AUC** (área bajo la curva de inserción): el proceso inverso; se parte de una imagen borrosa y se van añadiendo progresivamente las regiones importantes. Si el modelo acierta pronto al incorporarlas, la explicación es buena (cuanto más alta, mejor).
- **Sparsity** (dispersión): mide si el mapa es compacto y se concentra en pocas regiones, o si por el contrario es una mancha difusa que lo abarca todo. Un buen mapa debe ser directo (Linardatos, Papastefanopoulos, & Kotsiantis, 2021).
- **Entropy** (entropía): evalúa la concentración de la información del mapa desde el punto de vista de la teoría de la información; complementa a la dispersión (Linardatos, Papastefanopoulos, & Kotsiantis, 2021).
- **Stability SSIM**: mide la consistencia de las explicaciones ante pequeñas perturbaciones de la imagen de entrada, utilizando el índice de similitud estructural SSIM (Wang & al., 2004). Un modelo estable produce mapas similares para imágenes casi idénticas.

1.6.4. La calibración

Existe una dimensión adicional de fiabilidad que a menudo se pasa por alto y que este proyecto incorpora como mejora respecto al estudio de referencia: la calibración. Cuando una red dice "neumonía con un 95% de confianza", ese 95% debería significar que, de cada cien casos clasificados con ese nivel de confianza, aproximadamente noventa y cinco son efectivamente neumonía. En la práctica, muchas redes modernas están mal calibradas: son demasiado optimistas, y sus probabilidades declaradas no se corresponden con su precisión real (Guo, Pleiss, Sun, & Weinberger, 2017).

La métrica estándar para medir esta desviación es el Expected Calibration Error (ECE), que cuantifica la diferencia media entre la confianza declarada por el modelo y su precisión real, agrupando las predicciones en intervalos de confianza. Complementariamente se utiliza el Brier Score, una métrica que evalúa la calidad global de las probabilidades predichas. La calibración es un aspecto crítico en entornos clínicos: una alta confianza en una predicción errónea puede tener consecuencias graves, y un sistema que sobrestima su seguridad no es clínicamente fiable. Incluir estas métricas permite evaluar no solo si los modelos aciertan, sino también si saben cuándo aciertan.

En este punto conviene aclarar, por honestidad con el lector, cómo se distribuyen las cuatro técnicas XAI entre las dos vías de uso de la plataforma. En la vía de diagnóstico de la aplicación web, el módulo de explicabilidad implementa las cuatro técnicas visuales descritas (Saliency Maps, SmoothGrad y Grad-CAM para redes convolucionales, y mapas de atención para Transformers), que se generan de forma automática en cada consulta del profesional sanitario. En la vía de laboratorio MLOps, los scripts del pipeline de entrenamiento generan los mapas de explicabilidad cualitativos (Grad-CAM y mapas de saliencia para las CNN, y mapas de saliencia para los Transformers) y calculan las métricas cuantitativas y de calibración descritas en esta sección.

1.7. Las barreras para la adopción clínica de la IA

A pesar del potencial demostrado por la inteligencia artificial en el diagnóstico por imagen, la adopción clínica de estos sistemas se enfrenta a tres barreras fundamentales que este Trabajo Fin de Grado aborda de manera específica. La primera de ellas es la falta de transparencia, que se acaba de describir: los modelos generan predicciones sin proporcionar una justificación comprensible para el facultativo. vitalXAI aborda esta barrera mediante la inteligencia artificial explicable presentada en la sección anterior, incorporando mapas de calor interpretativos y métricas de fiabilidad en cada diagnóstico, de modo que las decisiones del modelo puedan ser verificadas clínicamente.

La segunda barrera es la falta de robustez y generalización. Un modelo puede alcanzar resultados prometedores en las condiciones controladas del laboratorio, pero su rendimiento se degrada de forma significativa al aplicarse sobre conjuntos de datos procedentes de poblaciones diferentes, equipos de adquisición distintos o protocolos de imagen alternativos (Tang & al., 2021). Un sistema que solo funciona bien con las imágenes con las que se entrenó tiene poco valor clínico real. Por ello, la validación externa sobre cohortes independientes —imágenes que el modelo nunca ha visto, procedentes de otro centro y de otra población— constituye un requisito indispensable para cualquier sistema que aspire a un despliegue clínico real (Alsaedi & Alshehri, 2024). En el marco de este proyecto, se emplea el COVID-19 Radiography Database como fuente de imágenes de pacientes adultos para la validación externa, siguiendo la metodología establecida en el estudio de referencia.

Conviene detenerse en este punto por las implicaciones del dataset elegido. El COVID-19 Radiography Database fue desarrollado durante la pandemia de COVID-19 por investigadores de la Universidad de Qatar y la Universidad de Dhaka, precisamente para apoyar la investigación de herramientas de cribado basadas en radiografías de tórax durante la crisis sanitaria (Chowdhury & al., 2020; Rahman & al., 2021). Durante la pandemia, el mundo asistió a la necesidad acuciante de herramientas que ayudaran a clasificar y priorizar pacientes mediante imágenes médicas, y numerosos estudios exploraron el uso de redes neuronales para la detección de COVID-19 en radiografías de tórax (Apostolopoulos & Mpesiana, 2020; Minaee & al., 2020). La existencia de una herramienta robusta de diagnóstico asistido de neumonía podría haber contribuido a aliviar la saturación de los servicios de radiología en contextos de alta demanda. Sin embargo, es esencial ser escrupulosos con lo que este proyecto afirma: todos los entrenamientos y todos los resultados presentados en esta memoria se basan en la detección de neumonía, y no en la detección específica de COVID-19. La elección de este dataset como banco de pruebas de la validación externa responde a que no sabemos con certeza cómo se comportarán los modelos entrenados con radiografías pediátricas al enfrentarse a una población adulta distinta, con equipos y protocolos de adquisición diferentes. Precisamente porque no podemos asumir que el rendimiento se mantenga, lo sometemos a esta prueba de estrés. Sin embargo, un resultado favorable no debe interpretarse por sí solo como evidencia concluyente de robustez: la mezcla de fuentes del dataset puede introducir factores de confusión que permitan distinguir el origen de las imágenes en lugar de la presencia de neumonía (DeGrave, Janizek, & Lee, 2021). Por ello, los resultados sobre esta cohorte se considerarán evidencia complementaria y deberán interpretarse junto con sus limitaciones, no como una demostración directa de potencial utilidad clínica durante una pandemia.

La tercera barrera es la usabilidad. La mayoría de las soluciones de inteligencia artificial desarrolladas en el ámbito académico requieren conocimientos técnicos avanzados para su ejecución: saber programar, instalar entornos, ejecutar scripts o interpretar archivos de resultados. Esto excluye precisamente a los profesionales sanitarios que deberían poder beneficiarse de estas herramientas. Las investigaciones recientes en MLOps (del inglés Machine Learning Operations, las prácticas que automatizan el ciclo de vida completo de los modelos: entrenamiento, validación, despliegue y monitorización) subrayan la necesidad de plataformas que acerquen estas capacidades a usuarios no especializados a través de interfaces intuitivas (Argaw & al., 2024; Kreuzberger, Kühl, & Hirschl, 2023).

VitalXAI ataca esta barrera desde un principio de diseño muy concreto: si los asistentes conversacionales modernos —similares a los sistemas de diálogo basados en grandes modelos de lenguaje que millones de personas utilizan a diario— han conseguido que interactuar con una máquina sea natural e inmediato para cualquiera, ¿por qué no adoptar ese mismo paradigma en una plataforma de diagnóstico médico? La interfaz de vitalXAI se ha concebido siguiendo la apariencia y la dinámica de estos asistentes conversacionales, tipo ChatGPT, porque es algo con lo que cualquier persona está hoy en día familiarizada. El laboratorio de entrenamiento de la plataforma incorpora un asistente conversacional con el que el facultativo puede hablar en lenguaje natural para configurar y lanzar experimentos de entrenamiento, sin escribir una sola línea de código y sin conocer el motor computacional que hay detrás. La complejidad técnica queda completamente oculta tras una conversación: esa es la forma de reducir la curva de aprendizaje y hacer que la tecnología sea realmente utilizable para el personal sanitario.

1.8. Los asistentes conversacionales y los grandes modelos de lenguaje

1.8.1. ¿Qué son los grandes modelos de lenguaje?

Dado que el asistente conversacional es uno de los componentes diferenciales de vitalXAI, es necesario situar esta tecnología en su contexto. Los grandes modelos de lenguaje (del inglés large language models, LLM) son redes neuronales Transformer de enormes dimensiones, entrenadas sobre cantidades ingentes de texto procedente de internet, libros, artículos científicos y otras fuentes. Su tarea de entrenamiento fundamental es sorprendentemente sencilla: predecir la siguiente palabra de un texto. A fuerza de practicar esa tarea sobre millones de documentos, el modelo aprende patrones estadísticos enormemente ricos sobre el lenguaje, el conocimiento del mundo y el razonamiento (Brown & al., 2020; Vaswani & al., 2017).

Pero un modelo que solo predice la siguiente palabra no es todavía un "chatbot útil". Para convertirlo en un asistente conversacional se realiza un paso adicional denominado ajuste por instrucciones y aprendizaje por refuerzo con retroalimentación humana (RLHF): el modelo se entrena, además, para seguir instrucciones de forma útil y para responder en formato de diálogo (Ouyang & al., 2022). El resultado es lo que conocemos como asistentes de inteligencia artificial conversacionales, capaces de mantener conversaciones fluidas, responder preguntas, razonar sobre un problema y, como en el caso que nos ocupa, extraer la información que el usuario introduce en lenguaje natural para realizar una acción técnica concreta.

La aplicación de los grandes modelos de lenguaje al ámbito de la medicina es un campo de investigación muy activo. Se han publicado numerosos trabajos que exploran su uso como apoyo en tareas clínicas: responder preguntas médicas, redactar informes o asistir en el diagnóstico, con resultados prometedores pero también con advertencias importantes sobre la necesidad de validación rigurosa antes de su uso clínico (Thirunavukarasu & al., 2023; Singhal & al., 2023). En este proyecto, sin embargo, el modelo de lenguaje no desempeña funciones de diagnóstico: su papel es el de un asistente de configuración que interpreta las peticiones del usuario sobre qué experimento de entrenamiento quiere lanzar y las traduce a los parámetros técnicos que necesita el sistema.

1.8.2. El panorama de los modelos de lenguaje y la elección de Llama 3

En el momento de desarrollo de este proyecto existen numerosos grandes modelos de lenguaje, con características muy diferentes. Por un lado están los modelos propietarios y cerrados, como GPT-4 de OpenAI, Claude de Anthropic o Gemini de Google, accesibles únicamente a través de sus APIs comerciales. Estos modelos son muy potentes, pero su código no está disponible, el usuario no puede adaptarlos ni ejecutarlos en su propia infraestructura, y su uso está sujeto a las políticas y precios de cada proveedor. Por otro lado están los modelos de acceso abierto, entre los que destacan la familia Llama de Meta y los modelos de Mistral AI, cuyos pesos son publicados y pueden descargarse y utilizarse libremente (Touvron & al., 2023; Jiang & al., 2023).

La elección de Llama 3 (Meta AI, 2024) para el asistente de vitalXAI se justifica por varias razones. En primer lugar, es un modelo de pesos abiertos, lo que se alinea con el espíritu del proyecto: transparencia, reproducibilidad y ausencia de dependencia de un proveedor concreto. En segundo lugar, ofrece un rendimiento competitivo con los mejores modelos propietarios en razonamiento y seguimiento de instrucciones, según las evaluaciones publicadas por el propio equipo de Meta, manteniendo un tamaño razonable (el modelo empleado, Llama 3.3 de 70.000 millones de parámetros, ofrece una relación calidad-coste excelente). En tercer lugar, su licencia permite tanto el uso en investigación como el uso comercial, lo que facilita la eventual transferencia de la tecnología. Y en cuarto lugar, al tratarse de un modelo abierto, es posible adaptarlo o sustituirlo en el futuro sin necesidad de reescribir el sistema.

La ejecución del modelo se delega en la plataforma Groq (Groq, 2024). Groq ofrece inferencia de modelos de lenguaje a través de una API en la nube, pero con una particularidad hardware: utiliza unos chips propios denominados unidades de procesamiento de lenguaje (LPU, del inglés Language Processing Units), diseñados específicamente para ejecutar modelos de Transformer a velocidades extremadamente altas y con una latencia muy reducida, muy por debajo de la que ofrecen las GPUs convencionales. Esta velocidad es clave para la experiencia de usuario: un asistente conversacional que responde con fluidez se siente natural y cercano a lo que el usuario espera de un sistema tipo ChatGPT, mientras que un asistente lento arruinaría la sensación de conversación. Además, Groq ofrece un plan gratuito con cuotas suficientes para un proyecto académico, lo que encaja con las restricciones de coste de un Trabajo Fin de Grado.

El asistente conversacional de vitalXAI se configura con un prompt de sistema cuidadosamente diseñado (una técnica denominada prompt engineering) que define su comportamiento como experto en MLOps médico. El asistente guía al usuario, en lenguaje natural, por los cinco parámetros que definen un experimento de entrenamiento: la ruta del dataset, las arquitecturas a entrenar, el número de épocas, el tamaño de lote y la tasa de aprendizaje. Cuando el usuario ha proporcionado todos los datos, el asistente devuelve un objeto JSON estructurado con la configuración completa, que el sistema interpreta automáticamente para lanzar el pipeline de entrenamiento. El asistente está además internacionalizado en cuatro idiomas (español, inglés, chino e hindi), ampliando su accesibilidad a usuarios de distintas procedencias.

1.9. Metodología experimental

La metodología experimental de este proyecto se estructura como un pipeline completo que amplía y mejora el marco de referencia establecido por Kermany y colaboradores (Kermany & al., 2018). El pipeline se organiza en cuatro fases que se corresponden, de forma natural, con la numeración de los siete scripts de entrenamiento y evaluación que componen el motor computacional del sistema, y que son orquestados automáticamente por el laboratorio MLOps de la plataforma. A lo largo de esta sección se describe cada fase, explicando los conceptos estadísticos y de aprendizaje automático necesarios para comprenderla sin conocimientos previos. Para favorecer la legibilidad, los conceptos se explican la primera vez que aparecen y no se repiten en fases posteriores.

1.9.1. Fase 1: Entrenamiento con validación cruzada

El objetivo de esta primera fase es entrenar cada una de las diecinueve arquitecturas (dieciséis CNN y tres Transformers) y obtener una estimación fiable y comparable de su rendimiento diagnóstico.

Un problema previo: cómo evaluar un modelo de forma fiable. Si entrenásemos un modelo con todas las imágenes y lo evaluásemos con esas mismas imágenes, el resultado sería engañosamente optimista: la red podría haber memorizado las imágenes en lugar de aprender a generalizar. La forma correcta de evaluar es separar las imágenes en dos grupos disjuntos: un conjunto de entrenamiento, con el que el modelo aprende, y un conjunto de validación, con el que se mide su rendimiento sobre datos que no ha visto. Ahora bien, si elegimos de forma arbitraria qué imágenes van a cada grupo, podríamos tener mala suerte y obtener una partición poco representativa. Para evitar este problema se utiliza la validación cruzada de k pliegues (k -fold cross-validation), una técnica que repite la evaluación varias veces con particiones distintas. Concretamente, el proyecto emplea cinco pliegues (5-fold): las imágenes se dividen aleatoriamente en cinco grupos del mismo tamaño; en cada una de las cinco iteraciones, cuatro de los grupos se usan para entrenar y el grupo restante para validar, de modo que cada imagen forma parte del conjunto de validación exactamente una vez. El modelo se entrena, por tanto, cinco veces, y el rendimiento final se resume con la media y la desviación típica de las métricas de las cinco iteraciones. Este diseño proporciona una estimación robusta del rendimiento y reduce la dependencia de una partición concreta.

El problema del desbalanceo. Como se comentó en la sección de datos, el dataset contiene muchas más imágenes de neumonía que de pacientes sanos (4.273 frente a 1.583). Un modelo entrenado con un conjunto desbalanceado puede sesgarse hacia la clase mayoritaria, porque predecir siempre la clase más frecuente reduce su error medio. Existen varias respuestas estándar a este problema, como ponderar la función de pérdida, sobremuestrear la clase minoritaria o utilizar pérdidas diseñadas para datos desbalanceados (He & Garcia, 2009). Este proyecto aplica submuestreo aleatorio (random undersampling): en cada pliegue, del conjunto de entrenamiento se seleccionan aleatoriamente tantas imágenes de la clase mayoritaria como imágenes tiene la clase minoritaria, descartando el excedente para ese pliegue. La decisión ofrece un entrenamiento sencillo y perfectamente balanceado, pero destruye información: sobre el conjunto completo se descartan 2.690 imágenes de neumonía y, dado que cada pliegue utiliza aproximadamente el 80 % de los datos para entrenar, se descartan alrededor de 2.150 imágenes mayoritarias en cada partición de entrenamiento. Además, la selección aleatoria introduce variabilidad adicional en las estimaciones; fijar la semilla y repetir los cinco pliegues la reduce,

pero no elimina el efecto de depender de una muestra parcial. Por tanto, los resultados quedan condicionados a esta decisión y no permiten concluir que el submuestreo sea superior a la ponderación de clases o al sobremuestreo, alternativas que no se han sometido a un contraste experimental independiente. La evaluación sobre el conjunto de validación se realiza siempre sobre la partición completa, sin submuestrear, para que las métricas reflejen el rendimiento sobre datos con su distribución natural.

Transferencia de conocimiento. En las CNN, se cargan los pesos aprendidos sobre ImageNet, se elimina la cabeza de clasificación original y se conserva la base convolucional como extractor de características con todos sus pesos congelados. Sobre ella se añade un cabezal específico para la clasificación binaria, formado por Global Average Pooling, capas densas con activación ReLU, Dropout y una salida sigmoide; únicamente este cabezal se actualiza durante el entrenamiento. Esta decisión reduce el número de parámetros que deben aprenderse con las radiografías y limita el riesgo de sobreajuste, pero también puede impedir que las características preentrenadas se adapten completamente al dominio médico (Yosinski & al., 2014). En los Transformers, en cambio, se parte igualmente de pesos preentrenados para imágenes, pero se ajusta el modelo completo con el optimizador AdamW, regularización por decaimiento de pesos y una tasa de aprendizaje más conservadora para favorecer la convergencia sin destruir las representaciones iniciales. En ambos casos se minimiza una pérdida de entropía cruzada binaria, aunque la implementación de los Transformers la aplica sobre sus logits. Durante el entrenamiento de ambas familias se utiliza la misma aumentación ligera de datos —volteos horizontales aleatorios y pequeños ajustes de brillo y contraste—, que crea variaciones sintéticas y ayuda a reducir el sobreajuste.

Hiperparámetros y criterios de parada. El entrenamiento se configura mediante cuatro hiperparámetros controlables desde la plataforma: el número de épocas (pasadas completas sobre el conjunto de entrenamiento), el tamaño de lote (número de imágenes procesadas por cada actualización de pesos), la tasa de aprendizaje (la magnitud de cada paso de optimización) y la arquitectura a entrenar. Además, el entrenamiento incorpora mecanismos automáticos de control: se guarda en cada pliegue la versión del modelo con menor pérdida de validación (best_foldN), se detiene el entrenamiento de forma anticipada si la pérdida de validación no mejora durante varias épocas (early stopping), y se reduce la tasa de aprendizaje cuando el progreso se estanca (reduce learning rate on plateau). La semilla aleatoria se fija de forma global (42) para que los experimentos sean reproducibles.

Métricas de rendimiento. Para medir la calidad de cada modelo, la implementación calcula cinco métricas estándar y, para una interpretación clínica completa, deben considerarse además la especificidad y el valor predictivo negativo. Para entenderlas es imprescindible presentar primero la matriz de confusión, una tabla que resume los cuatro posibles resultados de un clasificador binario: verdaderos positivos (VP, imágenes con neumonía correctamente clasificadas), verdaderos negativos (VN, sanos correctamente clasificados), falsos positivos (FP, sanos clasificados como enfermos) y falsos negativos (FN, enfermos clasificados como sanos). A partir de esta matriz se definen:

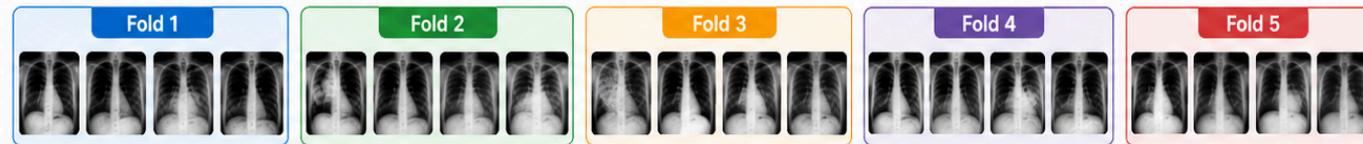
- **Exactitud** (accuracy): proporción de aciertos sobre el total de predicciones. Es intuitiva, pero puede resultar engañosa cuando las clases están desbalanceadas.
- **Precisión** (precision): de todas las imágenes que el modelo clasifica como neumonía, qué proporción son realmente neumonía ($VP / (VP + FP)$). Una precisión baja implica muchas falsas alarmas.

- **Sensibilidad o cobertura (recall):** de todas las imágenes que realmente tienen neumonía, qué proporción detecta el modelo ($VP / (VP + FN)$). En medicina esta es una métrica crítica: el peor error posible es un falso negativo, dejar pasar a un enfermo real. Una sensibilidad baja significa que el modelo pasa por alto pacientes con la enfermedad.
- **F1-score:** la media armónica de precisión y sensibilidad, que resume ambas en una única cifra.
- **Especificidad:** de todas las imágenes sanas, qué proporción identifica correctamente como normales ($VN / (VN + FP)$). Es la medida complementaria a la sensibilidad y permite cuantificar la capacidad de evitar falsos positivos.
- **Valor predictivo negativo (VPN):** de todas las imágenes clasificadas como normales, qué proporción es realmente sana ($VN / (VN + FN)$). Resulta especialmente relevante cuando una predicción negativa se utiliza para descartar enfermedad.
- **Área bajo la curva ROC (AUC):** permite evaluar la discriminación sin fijar un único umbral de decisión. La curva ROC representa, para todos los umbrales posibles, la tasa de verdaderos positivos frente a la tasa de falsos positivos (Fawcett, 2006). El área bajo esa curva (AUC) resume la capacidad del modelo para distinguir enfermos de sanos con un único número entre 0,5 (equivale a lanzar una moneda) y 1 (clasificación perfecta). Un AUC de 0,9 significa que, ante una imagen con neumonía y una sana elegidas al azar, el modelo asigna mayor probabilidad de enfermedad a la correcta en el 90% de los casos. Esta estimación puntual debería acompañarse de un intervalo de confianza, preferiblemente del 95 %, para reflejar la incertidumbre muestral; el pipeline actual almacena el AUC por pliegue y su desviación típica, pero no calcula todavía intervalos de confianza para la AUC, por lo que esta constituye una limitación metodológica que debe declararse.

Cada script de entrenamiento (CNN o Transformer) genera, para cada modelo y cada pliegue, los pesos del mejor modelo, las predicciones sobre el conjunto de validación en formato CSV, y un archivo resumen con las cinco métricas por pliegue junto con su media y desviación típica.

VALIDACIÓN CRUZADA DE CINCO PLIEGUES PARA RADIOGRAFÍAS DE TÓRAX

CONJUNTO DE DATOS
Dividido en 5 pliegues (folds)



El conjunto total se divide en 5 pliegues de tamaño similar



Ilustración x - Representación de Validación Cruzada

1.9.2. Fase 2: Comparación estadística entre modelos

Una vez entrenados todos los modelos de una sesión, la pregunta natural es: ¿qué modelo es el mejor y las diferencias observadas entre ellos son reales o fruto del azar? La comparación estadística no es un detalle metodológico: dos modelos pueden diferir en su AUC medio por unas centésimas, y esa diferencia puede ser puramente aleatoria, consecuencia de la variabilidad inherente al entrenamiento.

El proyecto calcula de forma exploratoria el test de los rangos con signo de Wilcoxon (Wilcoxon, 1945) sobre las cinco AUC obtenidas por cada par de modelos en los cinco pliegues. Sin embargo, este resultado no puede utilizarse como un contraste confirmatorio: con $n = 5$ pares, el p-valor bilateral exacto mínimo alcanzable es 0,0625, incluso cuando todas las diferencias apuntan en la misma dirección. Por construcción, ninguna comparación puede alcanzar el umbral convencional de 0,05. Además, los folds de una validación cruzada no son muestras independientes, porque sus conjuntos de entrenamiento se solapan; por ello, el supuesto de independencia de muchos contrastes pareados no se cumple (Nadeau & Bengio, 2003). La matriz debe interpretarse como un artefacto diagnóstico de las diferencias observadas, no como evidencia de significación estadística.

El script genera además un ranking global de modelos, ordenando las arquitecturas por su AUC medio y mostrando la desviación típica. También construye una matriz de calor con los p-valores exploratorios de las comparaciones por pares. Con diecinueve arquitecturas se realizan 171 comparaciones, y en la validación externa se plantea un número análogo de comparaciones mediante DeLong; por tanto, no es válido presentar p-valores sin corregir como si cada contraste fuese independiente. Si se conserva este análisis exploratorio, debe aplicarse una corrección por comparaciones múltiples, como Holm, y declarar que la dependencia entre folds limita la inferencia. Un análisis confirmatorio requeriría un procedimiento diseñado para comparar clasificadores con remuestreos dependientes, por ejemplo un test corregido de error de generalización (Nadeau & Bengio, 2003), o un rediseño con repeticiones independientes. El ranking y la matriz se integran posteriormente en el informe PDF, pero la captura no debe mostrar celdas interpretadas como significativas con $\alpha = 0,05$: el valor mínimo exacto esperable con cinco pares es 0,0625.

RANKING DE SESIÓN			
POSICIÓN	MODELO	MEDIA AUC	STD DEV
🏆	swin_base	0.9867	±0.0110
2	MobileNetV2	0.9854	±0.0072
3	deit	0.9783	±0.0229
4º	ConvNeXtTiny	0.8038	±0.0448
5º	MobileNetV3Large	0.7389	±0.0648

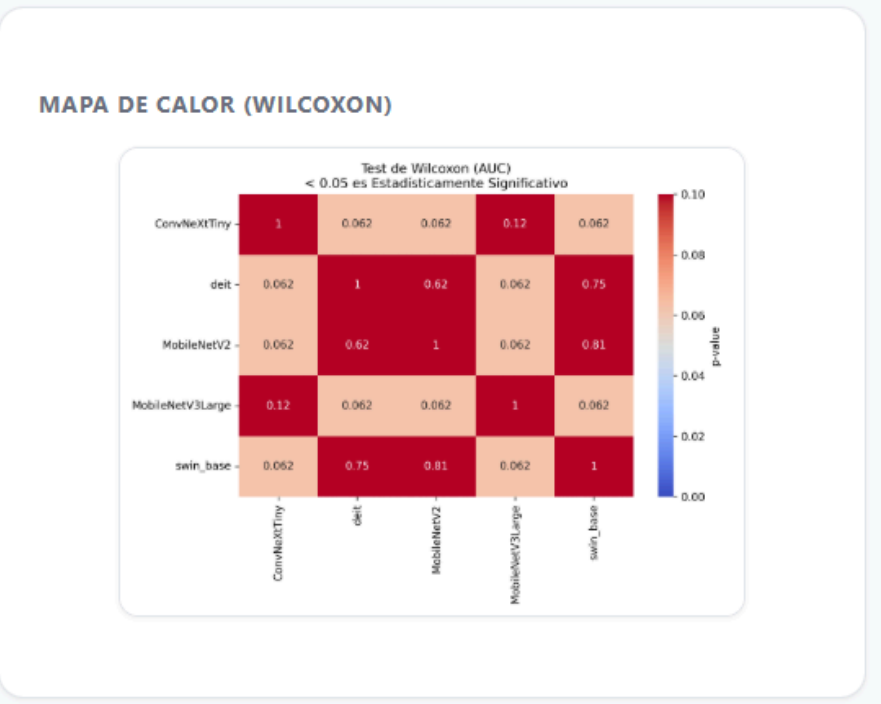


Ilustración x - Ranking global y Matriz de Wilcoxo

1.9.3. Fase 3: Validación externa y test de DeLong

La validación cruzada evalúa a los modelos sobre el mismo dataset pediátrico con el que se entrenaron, aunque en pliegues distintos. Pero el objetivo último de este proyecto es comprobar si los modelos son realmente capaces de generalizar a un entorno diferente. Esta es la pregunta que responde la validación externa.

En esta fase, los modelos entrenados se evalúan —congelados, sin ningún tipo de reaprendizaje— sobre el COVID-19 Radiography Database: quinientas radiografías de pacientes adultos sanos y quinientas de pacientes adultos con neumonía viral, que ningún modelo ha visto jamás. Se vuelven a calcular las cinco métricas de rendimiento sobre este conjunto y se construyen las curvas ROC correspondientes, que representan gráficamente la capacidad de cada modelo para distinguir enfermos de sanos sobre esta población desconocida. Si un modelo mantiene un AUC alto sobre esta cohorte adulta, se obtiene evidencia complementaria de su comportamiento ante un cambio de población y de adquisición, pero no una prueba aislada de robustez debido al posible confusor asociado al origen de las imágenes; si su rendimiento se desploma, habremos identificado una limitación importante de la generalización que debe documentarse con honestidad. La implementación actual utiliza, para cada arquitectura, el archivo `best_fold1`: se trata del mejor punto de control dentro del pliegue 1, seleccionado por su pérdida de validación, y no del mejor modelo entre los cinco pliegues. Esta elección queda fijada antes de evaluar el conjunto externo y evita seleccionar el modelo a partir de su rendimiento externo, pero hace que los resultados representen únicamente ese pliegue y no una agregación de los cinco. Por tanto, la validación externa debe interpretarse como una evaluación complementaria de un checkpoint preespecificado; una estimación más completa requeriría evaluar los cinco checkpoints y agregar sus resultados mediante un procedimiento definido de antemano. La comparación entre arquitecturas se realiza así bajo la misma regla de selección, aunque sus resultados no deben presentarse como el rendimiento externo medio de la validación cruzada (Varma & Simon, 2006).

Para determinar si las diferencias en las curvas ROC observadas en la validación externa son estadísticamente significativas se aplica el test de DeLong (DeLong, DeLong, & Clarke-Pearson, 1988). Este test está específicamente diseñado para comparar las áreas bajo dos curvas ROC obtenidas sobre las mismas muestras, teniendo en cuenta la correlación existente entre ambas predicciones. El script calcula el p-valor de la comparación entre todos los pares de modelos y genera una matriz de calor de significación, análoga a la del test de Wilcoxon, pero referida a la validación externa. De esta manera, el proyecto combina dos aproximaciones estadísticas complementarias: el test de Wilcoxon, que compara las distribuciones de rendimiento a partir de los cinco pliegues de la validación cruzada, y el test de DeLong, que compara las curvas ROC de la validación externa.

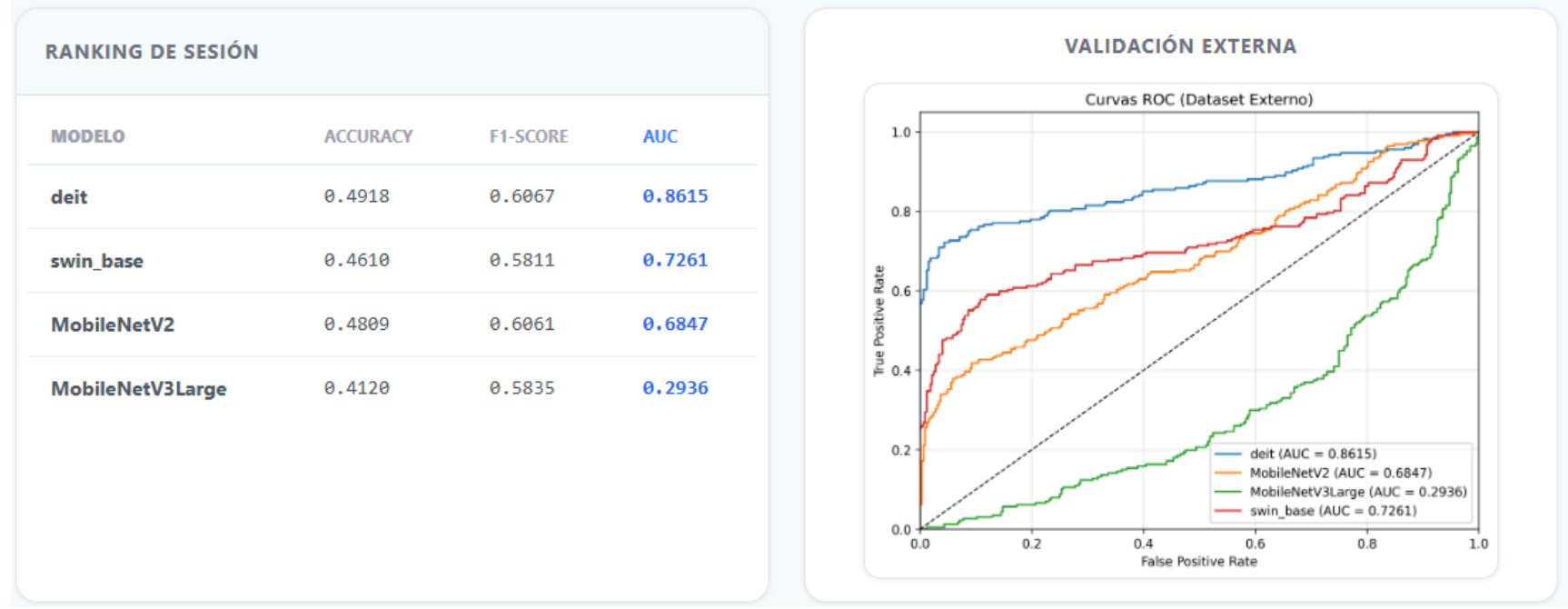


Ilustración x - Ranking de validación externa y curvas ROC

1.9.4. Fase 4: Análisis de explicabilidad

La última fase del pipeline evalúa la calidad de las explicaciones que los modelos son capaces de ofrecer, complementando las métricas de rendimiento predictivo con las métricas de fiabilidad descritas en la sección 1.6.

Análisis cualitativo: sobre un conjunto de imágenes de ejemplo del dataset de entrenamiento, se generan los mapas de explicabilidad visuales para cada modelo: Grad-CAM y mapas de saliencia para las redes convolucionales, y mapas de saliencia para los Transformers. El resultado es una galería de imágenes en las que se superpone el mapa de calor sobre la radiografía original, permitiendo inspeccionar visualmente si el modelo se fija en las regiones pulmonares con opacidad o si, por el contrario, sus explicaciones apuntan a zonas irrelevantes. Este análisis cualitativo es el primer filtro de coherencia clínica.

Análisis cuantitativo: sobre un conjunto de imágenes, se calculan las métricas objetivas de fidelidad de las explicaciones presentadas en la sección 1.6.3: Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM. Además, se calcula la calibración del modelo mediante el Expected Calibration Error (ECE) y el Brier Score, evaluando si la confianza declarada por el modelo se corresponde con su precisión real. El resultado se almacena en archivos comparativos que permiten puntuar numéricamente la calidad de la explicabilidad de cada arquitectura y comparar unas con otras de forma objetiva.

Interpretación de los resultados ilustrativos. Las capturas incluidas en esta fase muestran tanto el funcionamiento de la interfaz como dos comportamientos distintos del pipeline. El resultado de MobileNetV3Large, con exactitud 0,8000, precisión 0,8000, sensibilidad 1,0000 y F1 0,8889 idénticos en los cinco pliegues, junto con desviaciones típicas nulas, es compatible con un clasificador que predice siempre la clase positiva sobre una partición con un 80 % de positivos. Esa combinación no demuestra capacidad diagnóstica y debe considerarse una señal de posible fallo o de comportamiento degenerado que exige revisar las predicciones, las etiquetas, la partición y el preprocesamiento. Del mismo modo, un AUC externo de 0,2936 para MobileNetV3Large está por debajo del azar y puede indicar inversión sistemática de las clases, un problema de codificación de etiquetas, una incompatibilidad de preprocesamiento o un fuerte cambio de dominio; no debe interpretarse como evidencia ordinaria de generalización. La captura de MobileNetV2 complementa este caso al mostrar un comportamiento más coherente en la validación cruzada, pero ambos resultados deben leerse como ejemplos concretos del sistema y no como una selección de resultados representativos de todo el banco de pruebas.

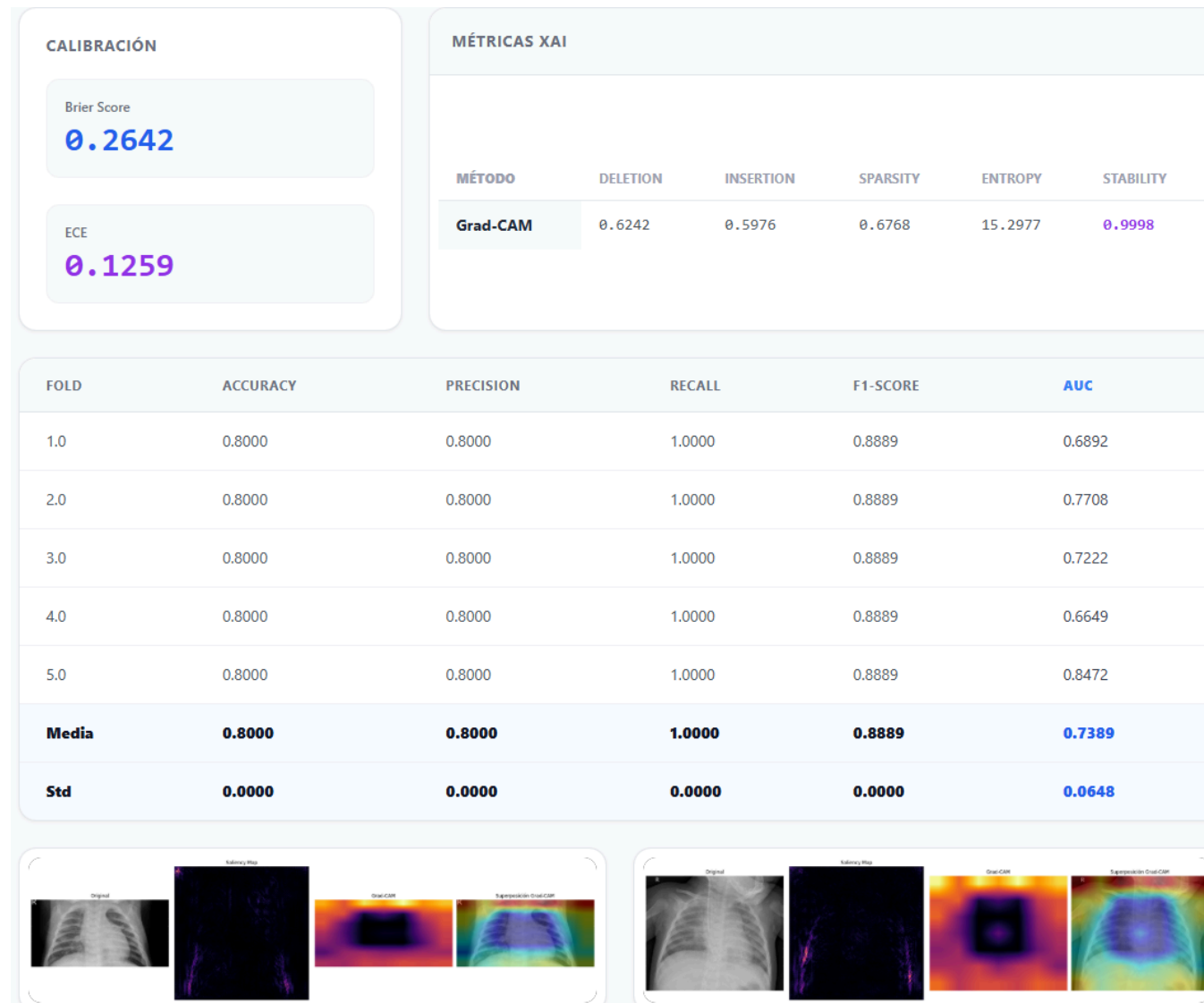


Ilustración x - Análisis cualitativo y mapas de calor MobileNetV3Large

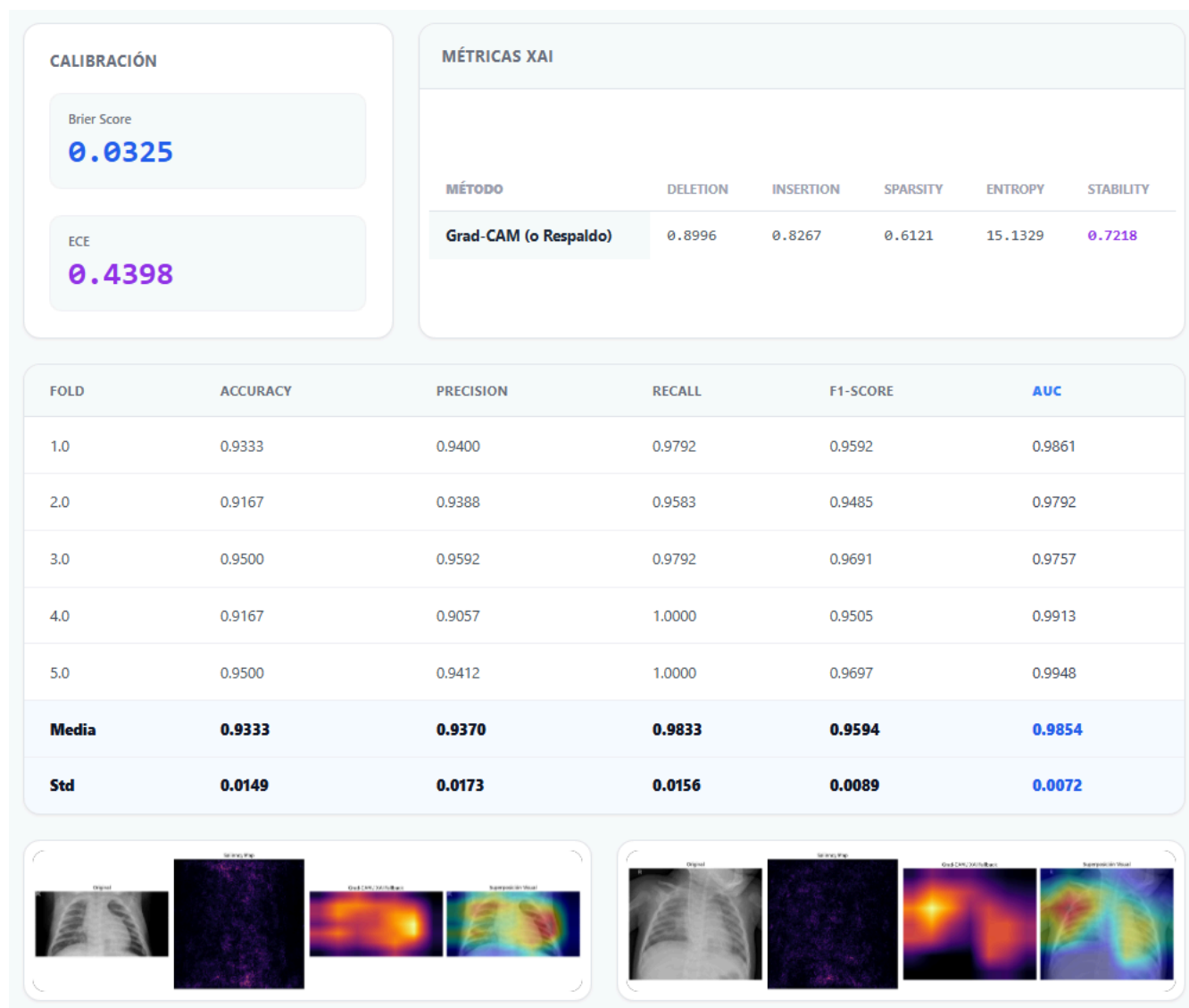


Ilustración x - Análisis cualitativo y mapas de calor MobileNetV2

1.10. Tecnologías y arquitectura del sistema

Para cerrar el capítulo, se resumen las tecnologías sobre las que se construye el sistema, que se describirán en detalle en las partes de análisis y diseño de la memoria. El backend se desarrolla con Python, el lenguaje de referencia en el ecosistema del aprendizaje profundo, y con el framework FastAPI, un framework moderno y de alto rendimiento para construir servicios web REST. La capa de persistencia se apoya en la base de datos relacional MySQL, que almacena los usuarios, el historial de consultas de diagnóstico y los experimentos de entrenamiento. El motor de aprendizaje profundo utiliza TensorFlow y Keras para las arquitecturas convolucionales y la librería HuggingFace Transformers para las arquitecturas Transformer. El asistente conversacional se implementa sobre la API de Groq con el modelo Llama 3 de Meta. El frontend se construye con HTML, CSS y JavaScript vanilla, priorizando la simplicidad y el funcionamiento sin herramientas de compilación, con una interfaz conversacional inspirada en los asistentes modernos tipo ChatGPT. El sistema se organiza en dos grandes bloques: una interfaz clínica para el diagnóstico asistido (carga de radiografías, selección de arquitectura, visualización de mapas de explicabilidad y descarga de informes PDF) y un laboratorio MLOps que automatiza el pipeline experimental descrito en la sección anterior.

2. Metas y Propósitos del Proyecto

El Trabajo Fin de Grado que se presenta surge de la necesidad de contar con una plataforma que reúna, en un mismo sistema, capacidades de diagnóstico automático mediante inteligencia artificial, mecanismos de explicabilidad que permitan a los clínicos comprender el razonamiento de los modelos, y procedimientos de validación rigurosos que garanticen la fiabilidad de los resultados obtenidos. Actualmente, los profesionales sanitarios y los investigadores se encuentran con un panorama fragmentado. Por un lado, repositorios de código y cuadernos de trabajo que implementan modelos prometedores pero que requieren conocimientos avanzados de programación para su ejecución, y que rara vez reportan de forma completa los detalles metodológicos necesarios para reproducir sus resultados (Liu & al., 2019; Varoquaux & Cheplygina, 2019). Por otro lado, plataformas comerciales cerradas que no permiten inspeccionar ni adaptar los algoritmos subyacentes. Este escenario dificulta tanto la investigación traslacional como la adopción clínica de unas tecnologías que han demostrado, en entornos controlados, un potencial extraordinario, alcanzando en algunos dominios de la imagen médica un rendimiento comparable al de los especialistas humanos (Esteva & al., 2017; De Fauw & al., 2018; McKinney & al., 2020). No obstante, la literatura científica también advierte de los riesgos de trasladar estas herramientas a la práctica clínica sin una validación rigurosa, una evaluación estadística adecuada y una documentación transparente de sus limitaciones (Nagendran & al., 2020; Kelly & al., 2019; Aggarwal & al., 2021).

Este capítulo establece los fines que persigue el proyecto. En primer lugar se define el objetivo general, la meta última que orienta todo el trabajo. A continuación se presentan los objetivos específicos, que descomponen el objetivo general en metas concretas y verificables. Estos objetivos se formulan a nivel de resultados y alcance, sin enumerar exhaustivamente todas las decisiones técnicas. La concreción de las arquitecturas, las técnicas y las decisiones de implementación se documenta posteriormente en los capítulos de contextualización, decisiones adoptadas, análisis y diseño, según corresponda.

El objetivo general de este Trabajo Fin de Grado consiste en el desarrollo de vitalXAI, una plataforma web MLOps que integre inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax. La plataforma debe ofrecer a la comunidad clínica e investigadora un entorno unificado, accesible, reproducible y rigurosamente validado que permita ejecutar diagnósticos, comparar arquitecturas de deep learning y evaluar la calidad de las explicaciones generadas por los modelos. El propósito último es poner al alcance de los profesionales sanitarios, sin necesidad de conocimientos técnicos en programación o aprendizaje automático, unas capacidades que tradicionalmente han quedado confinadas a los entornos de investigación, de forma que el potencial de la inteligencia artificial se traduzca en una herramienta realmente utilizable en la práctica clínica.

Del objetivo general se derivan los once objetivos específicos que se describen a continuación. Para facilitar su lectura, se organizan en torno a dos bloques complementarios. El primer bloque, de carácter científico-metodológico, agrupa los objetivos relativos a la fundamentación teórica, al diseño experimental y a la evaluación rigurosa de los modelos. El segundo bloque, de carácter de ingeniería web, agrupa los objetivos relativos a la construcción de la plataforma: su interfaz, su seguridad, su usabilidad y los servicios que ofrece al usuario final.

OE1 — Revisar el estado del arte. Analizar exhaustivamente la literatura científica sobre la aplicación de la inteligencia artificial al diagnóstico por imagen médica, con especial atención a la detección de neumonía mediante radiografías de tórax, a los paradigmas de inteligencia artificial explicable relevantes y a las prácticas de MLOps para la gestión del ciclo de vida de los modelos. Esta revisión, ejecutada en la tarea 0.2 del Sprint 0, tiene como finalidad fundamentar las decisiones de diseño e implementación sobre una base científica sólida y actualizada, tomando como referencia el estudio de Kermany y colaboradores (Kermany & al., 2018) y extendiendo su marco de evaluación con aspectos como la calibración de los modelos, la validación estadística y la reproducibilidad de los experimentos, dimensiones que la literatura reciente señala como imprescindibles para la fiabilidad de los sistemas de inteligencia artificial en medicina (Park & Han, 2018; Varoquaux & Cheplygina, 2019).

OE2 — Diseñar e implementar la arquitectura de persistencia. Crear una infraestructura robusta y segura que soporte el acceso de múltiples usuarios de manera concurrente y la gestión completa del historial de consultas de diagnóstico y de experimentos de entrenamiento. Esta capa debe garantizar el registro estructurado de los parámetros de cada inferencia —imagen, modelo empleado, predicción, nivel de confianza y artefactos generados— así como del histórico de sesiones de entrenamiento con sus configuraciones, métricas y resultados, permitiendo su consulta y comparación posterior.

OE3 — Diseñar e implementar el control de acceso. Desarrollar un mecanismo de autenticación, autorización y seguridad que garantice la integridad y confidencialidad de los datos de cada profesional sanitario, de modo que cada usuario disponga de un espacio de trabajo aislado desde el que gestione sus diagnósticos y resultados de manera exclusiva. Este objetivo incluye el registro de nuevos usuarios, el inicio y el cierre de sesión seguros y la protección frente a las amenazas más habituales de las aplicaciones web (OWASP, 2021), como los ataques de inyección, la falsificación de peticiones entre sitios o los intentos de acceso por fuerza bruta.

OE4 — Implementar el pipeline de entrenamiento. Automatizar el flujo completo de entrenamiento y evaluación de un conjunto amplio y representativo de arquitecturas de deep learning —tanto redes convolucionales como arquitecturas basadas en mecanismos de atención— mediante validación cruzada con particiones balanceadas. El pipeline debe automatizar el preprocesamiento de las imágenes, el entrenamiento con transferencia de conocimiento, el cálculo de las métricas de rendimiento y el almacenamiento de todos los artefactos generados, garantizando la comparabilidad de las arquitecturas mediante un diseño común de particiones, validación y métricas, con protocolos de optimización documentados según cada familia y resultados reproducibles.

OE5 — Desarrollar el módulo de explicabilidad (XAI). Implementar el módulo de inteligencia artificial explicable en sus dos vertientes complementarias. En la vertiente cualitativa, se generarán mapas visuales que permitan inspeccionar las regiones de la radiografía determinantes para cada predicción, facilitando la verificación clínica de las decisiones del modelo. En la vertiente cuantitativa, se calcularán métricas objetivas que puntúen la fidelidad de las explicaciones generadas. Adicionalmente, se incorporará la evaluación de la calibración de las predicciones —es decir, de si la confianza declarada por el modelo se corresponde con su precisión real—, un factor crítico para la fiabilidad clínica del sistema.

OE6 — Ejecutar la validación externa y el análisis estadístico. Implementar el proceso de validación externa sobre una cohorte de imágenes independiente, con el fin de evaluar la capacidad de generalización de los modelos a poblaciones, equipos de adquisición y protocolos de imagen distintos de los del entrenamiento, sin ningún tipo de reaprendizaje. Asimismo, se implementará un análisis estadístico descriptivo y, cuando el diseño muestral lo permita, inferencial, aplicando procedimientos adecuados para datos dependientes y correcciones por comparaciones múltiples tanto a los resultados de la validación cruzada como a las curvas ROC de la validación externa. Las limitaciones de tamaño muestral y dependencia entre folds deberán quedar explícitamente documentadas (Park & Han, 2018; Tang & al., 2021; Nadeau & Bengio, 2003).

OE7 — Garantizar la reproducibilidad y la trazabilidad de los experimentos. Fijar las semillas aleatorias de las librerías implicadas, almacenar la configuración completa de cada experimento y registrar los resultados y artefactos generados, de modo que cualquier resultado empírico pueda verificarse y replicarse bajo las mismas condiciones. Esta dimensión responde directamente a las carencias metodológicas que la literatura ha identificado en numerosos estudios de inteligencia artificial aplicada a la imagen médica (Varoquaux & Cheplygina, 2019).

OE8 — Desarrollar la interfaz clínica de diagnóstico asistido. Diseñar una interfaz web intuitiva, pensada para usuarios sin formación técnica en inteligencia artificial o informática, que permita a los facultativos cargar radiografías de tórax, obtener un diagnóstico con su nivel de confianza, visualizar los mapas de explicabilidad generados por el sistema y descargar un informe en formato PDF, todo ello desde un panel de control claro y sin complejidad técnica. El diseño debe priorizar la reducción de la curva de aprendizaje, siguiendo las pautas recomendadas para la interacción humano-inteligencia artificial (Amershi & al., 2019).

OE9 — Desarrollar el laboratorio MLOps y el asistente conversacional. Construir un laboratorio de entrenamiento integrado en la plataforma web, dotado de un asistente conversacional basado en grandes modelos de lenguaje que guíe al usuario, en lenguaje natural, en la configuración y el lanzamiento de experimentos de entrenamiento, eliminando por completo la necesidad de escribir código. La plataforma debe orquestar automáticamente la ejecución del pipeline experimental descrito en el OE4, el OE5 y el OE6, y poner a disposición del usuario los resultados, las comparativas y los informes generados. La elección del paradigma conversacional responde a la evidencia de que los agentes conversacionales pueden mejorar la accesibilidad de las aplicaciones de salud para usuarios no técnicos (Laranjo & al., 2018; Kocaballi & al., 2019), así como a la familiaridad que el público general tiene con este tipo de asistentes en la actualidad.

OE10 — Diseñar e implementar el procesamiento asíncrono de tareas. Crear un mecanismo de procesamiento asíncrono que permita ejecutar las tareas de larga duración —entrenamientos, análisis de explicabilidad o validación externa— sin bloquear la interfaz web. El sistema debe ofrecer monitorización del estado de cada tarea, consulta de su progreso y, cuando resulte posible, su cancelación. La gestión de las tareas de larga duración y de la complejidad operativa asociada constituye una de las dimensiones clave de la ingeniería de sistemas de aprendizaje automático (Sculley & al., 2015).

OE11 — Incorporar internacionalización. Añadir soporte multilingüe al sistema, de modo que la interfaz web, los informes generados y el asistente conversacional puedan presentarse en múltiples idiomas (español, inglés, chino e hindi). Este objetivo amplía la accesibilidad de la plataforma a entornos sanitarios internacionales.

En conjunto, estos once objetivos específicos conforman una hoja de ruta completa que abarca desde la fundamentación teórica y el diseño de la infraestructura hasta la implementación de los pipelines de entrenamiento, la evaluación de la explicabilidad, la validación estadística rigurosa y la construcción de una plataforma web segura, usable y accesible. Su articulación en dos bloques complementarios (el científico-metodológico y el de ingeniería web) garantiza que el resultado final no sea únicamente un conjunto de modelos validados, sino un sistema realmente utilizable que ponga esas capacidades al servicio de la comunidad clínica. Cada uno de estos objetivos se traduce, a lo largo de la memoria, en requisitos, casos de uso y componentes de diseño, y su cumplimiento se verifica mediante el plan de pruebas y los resultados experimentales presentados en las partes finales del documento. La administración avanzada de la plataforma queda fuera de este conjunto de objetivos y del alcance planificado del proyecto.

3. Estructura organizativa y partes interesadas del proyecto

Este capítulo describe la estructura organizativa del proyecto y las personas e instituciones que participan en él. La organización de un proyecto de las características de este —un desarrollo unipersonal de carácter multidisciplinar— exige una definición clara de quién forma parte del proyecto, qué papel desempeña cada actor y cómo se relacionan entre sí. El capítulo se estructura en tres apartados. El apartado 3.1 presenta el organigrama del proyecto, que representa gráficamente la estructura organizativa y las relaciones entre los interesados. El apartado 3.2 identifica y caracteriza a cada una de las partes interesadas, describiendo su rol general en el proyecto. Por último, el apartado 3.3 detalla las responsabilidades y funciones de cada interesado a lo largo del ciclo de vida y presenta la matriz RACI, que asigna de forma explícita y sin ambigüedades el nivel de implicación de cada parte interesada en las principales fases y entregables

3.1. Estructura organizativa del proyecto

El organigrama del proyecto, que representa la estructura organizativa de todos los interesados involucrados, se presenta en esta sección. El diagrama está compuesto por nueve nodos: Marc Ríos Cadenas, Iván Segura Carmona, Aurelio López Fernández, Luis Carmona Berdugo, el Consultor de Persistencia y Bases de Datos (Vicente de Vides Rodríguez), Rubén Pérez Garrido, el Tribunal de Evaluación del TFG, el Laboratorio Synergia y los facultativos e investigadores clínicos. Estos dos últimos actúan como entidades externas de validación y no están subordinados de manera jerárquica a ningún integrante del proyecto. Esta independencia estructural blindará el proyecto y garantiza una auditoría completamente objetiva de la plataforma.

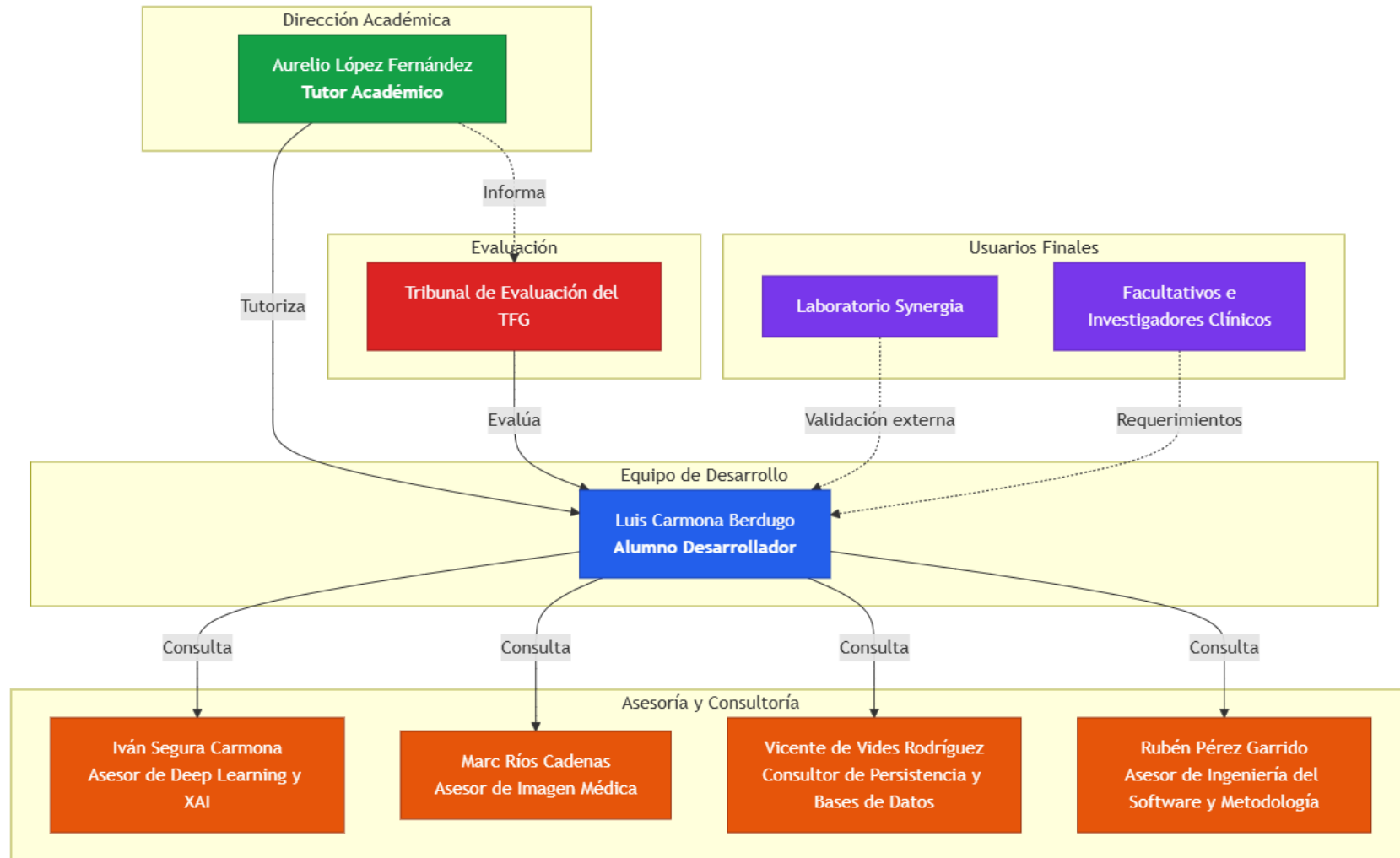


Ilustración x - Organigrama del Proyecto

3.2. Partes interesadas del proyecto

Realizar una identificación correcta de los interesados dentro de la planificación de cualquier proyecto es un paso fundamental porque permite saber quiénes son los afectados por el desarrollo, qué personas contribuyen a él y qué expectativas tienen depositadas en el resultado. En este proyecto, conviven distintos perfiles que van desde el alumno que lleva el peso técnico y documental, hasta investigadores del ámbito clínico que representan el usuario final del sistema. A continuación, se describen en detalle cada uno de los interesados junto con su rol general en el proyecto::

- **Luis Carmona Berdugo (Alumno Desarrollador):** Asume la responsabilidad íntegra del proyecto. Ejecuta y orquesta el desarrollo en su doble dimensión: técnica y documental. Él construye el sistema.
- **Aurelio López Fernández (Tutor Académico):** Dirige el rumbo de la investigación. Proporciona la brújula científica, metodológica y académica que garantiza la solvencia del Trabajo Fin de Grado.
- **Tribunal de Evaluación del TFG (Cliente / Evaluador):** Ejerce como máxima autoridad formal. Este órgano audita el proyecto, califica el trabajo y certifica si el sistema cumple los requisitos académicos exigidos. Su veredicto determina la viabilidad del resultado entregado..
- **Vicente de Vides Rodríguez (Consultor de Persistencia y Bases de Datos, Rol Consultivo):** Aporta su experiencia estructural. Resuelve consultas puntuales sobre el diseño, el modelado y la optimización de la persistencia de datos. (Asignación de especialista pendiente de confirmación).
- **Iván Segura Carmona (Asesor de Deep Learning y XAI):** Domina el núcleo algorítmico. Su consultoría especializada arroja luz sobre el comportamiento de las redes neuronales, los mecanismos de explicabilidad y la correcta interpretación de los resultados del benchmarking.
- **Marc Ríos Cadenas (Asesor de Imagen Médica):** Custodia la veracidad clínica del trabajo. Interviene de forma selectiva para asegurar que el contexto médico y radiológico del proyecto refleje la realidad hospitalaria.
- **Rubén Pérez Garrido (Asesor de Ingeniería del Software y Metodología):** experto en ingeniería del software que ha colaborado en el desarrollo de krill, una herramienta y modelo de trabajo basado en especificaciones y guiado por pruebas (SDD/TDD) que estructura el proceso de desarrollo del proyecto, y que es consultado de manera puntual sobre la metodología de desarrollo y las prácticas de ingeniería aplicadas a la implementación.
- **Laboratorio Synergia (Usuario Externo / Validador):** Representa el primer escalón de adopción. Las líneas de investigación de este equipo convergen con los objetivos de vitalXAI, convirtiéndolos en probadores empíricos de la herramienta.
- **Facultativos e investigadores clínicos (Usuarios Finales):** Constituyen el objetivo último del despliegue. Este colectivo integraría el diagnóstico asistido en su rutina médica e investigadora diaria. Sus necesidades operativas dictan la utilidad real del sistema.

3.3. Responsabilidades, funciones y participación de las partes interesadas

Realizar una definición precisa de las responsabilidades y funciones de cada uno de los interesados es esencial para garantizar que el trabajo se ejecute de manera ordenada, sin solapamientos y con una asignación clara de las principales fases y entregables. Dado que este proyecto tiene un carácter multidisciplinar, esta definición es especialmente relevante ya que las aportaciones de cada uno de los interesados se realizan en fases distintas del ciclo de vida y desde ámbitos de conocimiento muy diferentes entre sí.

Alumno Desarrollador (Luis Carmona Berdugo, LCB): Sobre él recae toda la responsabilidad principal e íntegra de la ejecución del proyecto. Desde un punto de vista científico, es la persona encargada de revisar y sintetizar el estado del arte en inteligencia artificial aplicada al diagnóstico por imagen médica, técnicas XAI y arquitecturas de deep learning, realizando una documentación de todos los hallazgos realizados. Desde el punto de vista técnico, es el encargado de diseñar la arquitectura del sistema, implementar todos los módulos de software, incluyendo los pipelines de entrenamiento para arquitecturas CNN y Transformer, los módulos de generación de explicaciones XAI y los scripts de validación externa y análisis estadístico, y también es el encargado de asegurar que los resultados sean válidos y reproducibles bajo las mismas condiciones. Tomando en cuenta el punto de vista metodológico, diseña y ejecuta el proceso de benchmarking, de manera que se garantice que las comparativas de rendimiento sean representativas en condiciones reales de uso. Desde el punto de vista de la documentación, es el encargado de elaborar toda la documentación del sistema con la calidad y profundidad exigidas para un proyecto de estas características. Por último, es el encargado de mantener la planificación del proyecto, gestionar los riesgos que puedan surgir y coordinar la comunicación con el tutor y los asesores tomando como referencia el prisma de la gestión.

Tutor Académico (Aurelio López Fernández, ALF): Es el responsable de garantizar tanto el rigor académico como el científico del proyecto en todas sus fases. Revisa y proporciona una retroalimentación detallada sobre cada uno de los entregables, señalando errores, inconsistencias y áreas de mejora con el nivel de exigencia propio de un Trabajo Fin de Grado. Orienta al alumno en las decisiones estratégicas más relevantes y actúa como puente entre el perfil técnico del desarrollo y el dominio científico del sistema.

Tribunal de Evaluación del TFG (T. TFG): La principal función de este interesado es la de evaluar el trabajo presentado con base en los criterios académicos establecidos por la universidad, que son el rigor científico, la calidad técnica, la coherencia documental y la capacidad de defensa oral del alumno. Aunque no participa de manera directa en el desarrollo, es el interesado con mayor influencia formal sobre el resultado del proyecto, ya que su valoración determinará la calificación final. Sus expectativas implícitas —claridad expositiva, aplicación de estándares de ingeniería y justificación de decisiones técnicas— se deberán tener en cuenta a lo largo de toda la documentación.

Consultor de Persistencia y Bases de Datos (Vicente de Vides Rodríguez, VVR): Su intervención es puntual y solamente de carácter consultivo. Aporta su experiencia estructural: cuando lo necesita el estudiante, proporciona criterios técnicos acerca de la eficacia de las consultas, la consistencia del modelo de persistencia con los requerimientos del sistema y el diseño, el modelado y la optimización del esquema de datos. Siempre será el estudiante desarrollador quien tome la decisión definitiva sobre cómo se diseñará e implementará la base de datos.

Asesor de Deep Learning y XAI (Iván Segura Carmona, ISC): Participa bajo un esquema puramente consultivo. Ejerce como experto de dominio. Resuelve dudas técnicas complejas acerca del funcionamiento interno de las arquitecturas algorítmicas de deep learning. Guía la correcta aplicación práctica de las técnicas de inteligencia artificial explicable. Aporta un criterio experto fundamental para interpretar correctamente los resultados del benchmarking. Analiza estas métricas desde la perspectiva estricta del rendimiento de modelos algorítmicos. La responsabilidad ejecutiva recae de nuevo sobre el alumno. El diseño arquitectónico, la implementación programada y la validación matemática de todas estas variantes algorítmicas pertenecen exclusivamente a las obligaciones del desarrollador.

Asesor de Imagen Médica (Marc Ríos Cadenas, MRC): Interviene de manera selectiva ofreciendo un asesoramiento consultivo indispensable. Actúa como experto clínico de dominio. Garantiza que el contexto médico y radiológico del proyecto quede representado con absoluta fidelidad. Ofrece contexto sobre cómo los profesionales sanitarios examinan las radiografías de tórax durante el diagnóstico hospitalario de neumonías. Define los rasgos innegociables que debe poseer la herramienta para resultar verdaderamente efectiva en la trinchera clínica. El estudiante recibe este conocimiento y asume la responsabilidad de traducirlo. Aplica este contexto médico a decisiones algorítmicas específicas, impactando directamente en el diseño estructural y en la implementación de la plataforma.

Asesor de Ingeniería del Software y Metodología (Rubén Pérez Garrido, RPG): Su participación es puntual y consultiva. Ha colaborado en el desarrollo de krill, una herramienta y modelo de trabajo basado en especificaciones y guiado por pruebas (SDD/TDD), que estructura el proceso de desarrollo del proyecto en el repositorio. Aporta criterios sobre la aplicación de la metodología, las prácticas de desarrollo guiado por pruebas y la calidad del proceso de ingeniería en las tareas de implementación. La ejecución concreta de dichas tareas y la aplicación de la metodología son responsabilidad del alumno.

Laboratorio Synergia y Facultativos e investigadores clínicos (LS/FIC): Mantienen una posición externa al núcleo de programación. No participan de manera activa en el desarrollo diario del proyecto. Representan, sin embargo, las necesidades reales del usuario final. Sus requerimientos implícitos constituyen el único criterio válido para medir la utilidad empírica del sistema desarrollado. Las líneas de investigación del Laboratorio Synergia convergen de forma natural con los objetivos de esta herramienta. El diseño del ecosistema prioriza constantemente este perfil médico. Garantizamos así que la accesibilidad visual de las interfaces y la interpretabilidad matemática de la inteligencia artificial satisfagan las exigencias de la práctica clínica real.

La Matriz RACI es una herramienta estándar de gestión de proyectos que permite hacer explícito el nivel de implicación de cada uno de los interesados en las principales fases o entregables del proyecto. Su nombre es un acrónimo de los cuatro roles que contempla: Responsable (Responsable), Accountable (Aprobador), Consulted (Consultado) e Informed (Informado). La aplicación de esta matriz es especialmente relevante en proyectos multidisciplinares como el que se está desarrollando, ya que conviven perfiles técnicos, académicos y científicos que tienen diferentes niveles de implicación, y donde una definición imprecisa de los roles podría derivar en solapamientos o en entregables sin un responsable claro.

Cada uno de los cuatro roles tiene un significado preciso dentro del contexto de la gestión de proyectos. El rol R (Responsable) identifica a la persona que realiza físicamente el trabajo, es decir, quien produce el entregable o ejecuta la tarea; en el caso de este proyecto, este rol recaerá siempre sobre el alumno desarrollador. El rol A (Accountable) asigna a la persona que tiene la autoridad final sobre ese entregable y responde por su calidad frente al resto del proyecto; a diferencia del rol anterior, solo puede existir uno por tarea, lo que evita ambigüedades en la toma de decisiones. El rol C (Consulted) corresponde a quienes, sin ejecutar ni decidir, son consultados porque poseen un conocimiento especializado relevante para la tarea; su aportación se produce antes o durante la ejecución y tiene carácter bidireccional; en el caso de este proyecto, serían los asesores y el consultor. Por último, el rol I (Informed) designa a las personas que únicamente son notificadas del avance o resultado de una tarea sin tener intervención en ella.

A continuación, se presenta la Matriz RACI del proyecto, en la que las filas recogen las principales fases y entregables del ciclo de vida del proyecto, y las columnas representan los interesados identificados en el apartado anterior.

Fase / Entregable	LCB	ALF	T. TFG	VVR	ISC	MRC	RPG	LS/FIC
Revisión del estado del arte	R	A	I	-	C	C	-	-
Diseño de la arquitectura del sistema	R	A	I	C	-	-	-	-
Diseño e implementación de la base de datos	R	A	I	C	-	-	-	-
Implementación del pipeline de entrenamiento CNN y Transformer	R	A	I	-	C	-	C	-
Implementación del módulo XAI (cualitativo y cuantitativo)	R	A	I	-	C	C	C	-
Desarrollo de la interfaz web y chatbot conversacional	R	A	I	-	-	-	C	I

Fase / Entregable	LCB	ALF	T. TFG	VVR	ISC	MRC	RPG	LS/FIC
Seguridad y control de acceso	R	A	I	-	-	-	C	-
Pruebas y verificación del sistema	R	A	I	-	C	C	C	-
Laboratorio MLOps	R	A	I	-	C	-	C	-
Procesamiento asíncrono de tareas	R	A	I	-	-	-	C	-
Internacionalización de la plataforma	R	A	I	-	-	-	C	C
Panel de administración	R	A	I	-	-	-	C	-
Gestión de riesgos	R	A	I	-	-	-	C	-
Benchmarking y análisis de resultados	R	A	I	-	C	C	-	-
Validación del contexto clínico	R	A	I	-	-	C	-	-
Validación externa y tests estadísticos	R	A	I	-	C	-	-	-
Documentación del TFG	R	A	I	-	-	-	I	-
Defensa y evaluación final	R	C	A	-	-	-	-	-

Tabla x - Matriz RACI

4. Marco Metodológico de Gestión

Todo proyecto necesita un marco de gestión que ordene su ejecución, y la elección de dicho marco debe ser coherente con la naturaleza del trabajo que se va a desarrollar. Este capítulo describe cómo se gestiona el presente proyecto: los criterios que llevaron a elegir un enfoque ágil, la forma en que se adapta el marco de trabajo Scrum a un proyecto de estas características, las herramientas de seguimiento utilizadas y el alcance de la metodología. El objetivo es que la gestión no sea un mero formalismo, sino un mecanismo que aporte orden, transparencia y capacidad de adaptación al desarrollo.

4.1. *Justificación del enfoque metodológico*

La naturaleza de este proyecto combina desarrollo de software con investigación computacional aplicada al diagnóstico por imagen médica. Esta doble naturaleza introduce un grado de incertidumbre técnica inherente que condiciona la elección de la metodología. Por un lado, el comportamiento de las arquitecturas de deep learning frente a los conjuntos de datos no siempre es predecible: los resultados del benchmarking pueden revelar que es necesario ajustar el diseño, los hiperparámetros o incluso el alcance de lo planificado, y estos ajustes no pueden anticiparse en su totalidad desde el inicio. Por otro lado, el proyecto cuenta con un único desarrollador y con una duración acotada, lo que descarta metodologías de gran peso procedimental, como Métrica V3, cuyo nivel de burocracia resultaría desproporcionado para la escala de trabajo. En la literatura de gestión de proyectos se reconoce que no existe un marco universal válido para todo: la metodología debe elegirse ponderando el tamaño del proyecto, la dinámica de sus requisitos y el entorno en el que se desarrolla (Boehm & Turner, 2004).

A la vista de estas características, se ha adoptado una metodología ágil, concretamente el framework Scrum (Schwaber & Sutherland, 2020), complementado con un tablero Kanban como herramienta de gestión visual del flujo de trabajo. Tres motivos específicos justifican esta elección frente a un enfoque tradicional en cascada. El primero es el componente exploratorio del desarrollo, particularmente en lo relacionado con la comparación de arquitecturas de deep learning y con el ajuste de sus hiperparámetros: la naturaleza iterativa de Scrum permite reorientar los esfuerzos sobre la base de los resultados obtenidos en cada ciclo, en lugar de quedar atados a una planificación rígida. El segundo motivo guarda relación con la estructura iterativa que proporciona Scrum, que posibilita detectar y corregir errores en fases tempranas, reduciendo la probabilidad de que el producto final no se ajuste a los objetivos establecidos. En tercer lugar, tanto la literatura empírica sobre métodos ágiles (Dybå & Dingsøyr, 2008) como la experiencia acumulada de su aplicación (Sutherland, 2014) evidencian que Scrum puede implementarse de manera satisfactoria en contextos de desarrollo individual, adaptando las ceremonias y los roles sin sacrificar las ventajas intrínsecas del marco de trabajo.

4.2. Roles y recursos del proyecto

4.2.1. Roles dentro del marco de trabajo

Scrum define tres roles con responsabilidades bien delimitadas. En este proyecto, al tratarse de un desarrollo unipersonal, la asignación de estos roles se adapta a la realidad del equipo:

- **Product Owner (Aurelio López Fernández):** El tutor académico asume este rol ante la ausencia de un cliente externo. Él valida personalmente las entregas, jerarquiza la importancia de los componentes algorítmicos y define los criterios de aceptación que rigen cada segmento de trabajo.
- **Scrum Master y Developer (Luis Carmona Berdugo):** Al ser un proyecto de desarrollo individual, el alumno concentra ambos roles. Como Scrum Master, es el responsable de la organización del trabajo en sprints, del mantenimiento de los artefactos del proceso y de garantizar que el proyecto avanza de manera coherente con la planificación. Como Developer, asume la totalidad de la responsabilidad técnica de la implementación.
- **Asesores científicos (consultor de persistencia y bases de datos, asesores de deep learning y XAI, de imagen médica y de ingeniería del software):** La terminología de Scrum omite esta figura. Nosotros la asimilamos al concepto de domain experts. Estos especialistas inyectan conocimiento crítico puntualmente durante las encrucijadas del desarrollo, manteniendo una posición externa sin integrarse formalmente en la célula de programación.

4.2.2. Dedicación y recursos materiales

La dedicación de los recursos humanos no es uniforme a lo largo del proyecto, sino que varía en función de la naturaleza de cada tarea y de la participación concreta de cada persona. El alumno asume la mayor parte de la ejecución, pero su porcentaje de dedicación también se ajusta a cada actividad: en las reuniones inicial y final comparte la dedicación al 50 % con el tutor, mientras que en las tareas técnicas suele asumir el 100 %. El tutor, en su papel de Product Owner, participa al 50 % en las tareas 0.1 y 8.3. Los asesores intervienen con porcentajes variables, que en el cronograma oscilan entre el 3 %, el 5 %, el 8 %, el 10 %, el 15 % y el 25 %, según la tarea y el tipo de revisión o consulta requerida. Estas proporciones son las que sustentan el cálculo de horas y costes del apartado 5.3; no existe, por tanto, una dedicación uniforme de los asesores.

En cuanto a los recursos materiales, el proyecto requiere un equipo con capacidad de cómputo GPU para el entrenamiento de los modelos de deep learning, particularmente durante las fases de validación cruzada de las arquitecturas CNN y Transformer. Se emplea una estación de trabajo local equipada con una GPU NVIDIA compatible con CUDA, cuyo coste de uso está incluido en la infraestructura hardware disponible para el desarrollo del TFG, sin necesidad de recurrir a instancias de computación en la nube. Todo el software utilizado es de código abierto y gratuito, por lo que no se incurre en costes de licenciamiento.

4.3. Ciclo de trabajo iterativo

4.3.1. Los sprints y la cadencia del trabajo

El evento central del marco Scrum es el Sprint, entendido en este proyecto como un ciclo iterativo de trabajo con un objetivo y un incremento verificable. La aplicación es una adaptación de Scrum a un TFG unipersonal: los nueve sprints tienen duración desigual, porque la planificación real reserva más tiempo a la fase inicial, a los bloques técnicos de mayor complejidad y a la documentación final. Por tanto, no se afirma que exista una cadencia temporal fija ni que se aplique Scrum de forma ortodoxa; se conservan sus elementos útiles de planificación, revisión, retrospectiva, backlog e incremento, ajustados a las restricciones académicas y al volumen variable de cada bloque. Cada sprint procura producir un incremento potencialmente entregable, integrando diseño, implementación y pruebas cuando la naturaleza de la tarea lo permite. Las pruebas se realizan de forma continua e incremental, incluyendo las pruebas de seguridad en los mismos sprints en que se desarrollan los componentes afectados.

La ejecución de cada tarea de implementación se apoya, además, en el modelo de desarrollo basado en especificaciones y guiado por pruebas (SDD/TDD), diseñado con la colaboración del asesor de ingeniería del software y descrito en el capítulo de organización. Este modelo organiza el trabajo de programación en ciclos en los que primero se define la especificación de la funcionalidad, después se escriben las pruebas que deben satisfacerla y, finalmente, se implementa el código necesario para que esas pruebas pasen y se refactoriza el resultado (Beck, 2000). Esta práctica garantiza que cada funcionalidad quede cubierta por pruebas automatizadas y contribuye a la calidad y a la reproducibilidad del desarrollo.

4.3.2. Ceremonias y reuniones de seguimiento

Cada Sprint se organiza, cuando la fase y la disponibilidad de los participantes lo permiten, alrededor de las actividades adaptadas de planificación, revisión y retrospectiva. El alumno desglosa los elementos del Product Backlog en tareas concretas y utiliza el tablero para realizar el seguimiento. Las revisiones con el tutor se celebran en los hitos previstos y mediante reuniones puntuales, sin asumir dieciocho ceremonias formales con dedicación presupuestada. La retrospectiva se realiza como revisión del proceso al cierre de cada bloque cuando resulta aplicable.

En este proyecto no se realiza ninguna Daily Scrum, ya que no resulta operativo en un contexto de desarrollo individual. Para cubrir la función de estas reuniones diarias se realizan reuniones periódicas de seguimiento con el tutor y, cuando es necesario, con los asesores. En estas reuniones el alumno presenta los avances realizados, expone las dudas técnicas surgidas, recibe retroalimentación sobre los resultados obtenidos y acuerda las tareas a acometer hasta el siguiente encuentro. El registro de estas reuniones queda recogido en el Anexo de Seguimiento del Proyecto, que actúa como bitácora formal del proceso a lo largo del ciclo de vida del desarrollo.

La ejecución del proyecto no ha seguido un calendario artificialmente uniforme, sino la cadencia real que impone un trabajo académico con un único desarrollador. El proyecto se inició a finales de noviembre de 2025. Durante el periodo comprendido entre ese momento y finales de febrero de 2026, la actividad se limitó a un par de reuniones de seguimiento con el tutor y a una pequeña

inicialización del marco tecnológico: la selección de las librerías, la preparación del entorno y el arranque del repositorio. A partir de principios de marzo de 2026 comenzó el desarrollo completo del sistema, que se prolongó hasta principios de junio de 2026, con algunas reuniones puntuales —pocas, pero decisivas— con el tutor y, cuando fue necesario, con los asesores, para revisar los avances y acordar los pasos siguientes. Desde principios de junio de 2026 hasta septiembre de 2026 se desarrolló la documentación completa del proyecto, junto con el benchmarking final, las correcciones y la entrega, de nuevo con algunas reuniones de seguimiento puntuales. Esta distribución temporal, lejos de ser un detalle administrativo, refleja la naturaleza real del trabajo: la fase de implementación concentró la aplicación efectiva de los sprints, mientras que las fases inicial y final fueron más intensivas en documentación, revisión y cierre.

4.3.3. Artefactos del proceso de gestión

El proceso de gestión se apoya en tres artefactos definidos por Scrum. El Product Backlog es la lista ordenada y priorizada de todo el trabajo pendiente: los requisitos funcionales y no funcionales del sistema, las tareas de implementación de los pipelines de entrenamiento para arquitecturas CNN y Transformer, los módulos de generación de explicaciones XAI, los experimentos de benchmarking planificados y los documentos del TFG pendientes de elaboración. Es gestionado por el alumno y revisado de manera conjunta con el tutor en cada Sprint Planning. El Sprint Backlog es el subconjunto de elementos del Product Backlog seleccionados para el Sprint en curso, desglosados en tareas concretas y estimables que guían el trabajo durante el ciclo. El Increment es el conjunto de todos los elementos completados durante el Sprint, que deben cumplir la Definition of Done —el criterio acordado que define cuándo una tarea puede considerarse terminada— antes de ser presentados en el Sprint Review.

4.4. Seguimiento visual del trabajo con Kanban

Además del marco Scrum, se utiliza un tablero Kanban, gestionado a través de GitHub Projects, como instrumento visual para administrar el flujo de trabajo. La elección de GitHub Projects responde a que el repositorio del proyecto está alojado en GitHub, lo que permite asociar de manera directa las tareas del tablero a los commits, pull requests e issues del código, unificando toda la trazabilidad del desarrollo en una única plataforma.

El tablero se organiza en cuatro columnas que representan los estados por los que transita cada tarea: Por Hacer (To Do), para las tareas planificadas y aún no iniciadas; En Progreso (In Progress), para las tareas en desarrollo activo; En Revisión (In Review), para las tareas completadas por el alumno y pendientes de validación por el tutor; y Hecho (Done), para las tareas validadas que cumplen la Definition of Done.

La combinación de Scrum y Kanban, conocida en la literatura como Scrumban, permite aprovechar la estructura iterativa y los ritmos de inspección de Scrum junto con la visibilidad continua del flujo de trabajo que proporciona el tablero (Kniberg & Skarin, 2010; Reddy, 2016). El tablero Kanban aporta, por sí mismo, un mecanismo de gestión del flujo que ayuda a limitar el trabajo en curso y a detectar cuellos de botella (Anderson, 2010), algo especialmente valioso en un proyecto unipersonal en el que la capacidad de autogestión es fundamental.

4.5. Alcance de la metodología

El marco de gestión descrito en este capítulo se aplica exclusivamente al trabajo comprendido en este Plan de Proyecto, es decir, a la concepción, la implementación y la documentación de vitalXAI hasta su entrega. No gobierna, en cambio, las actividades que solo tendrían sentido si la plataforma pasara a explotarse de forma continuada —el mantenimiento del software, el soporte a usuarios, la actualización de los modelos o su despliegue en un centro sanitario—, que quedan fuera del alcance de un trabajo académico con un plazo de entrega fijado. La planificación tiene, por tanto, un horizonte claro: la finalización y la defensa de la memoria. Las posibles líneas de evolución futura de la plataforma se describen en la memoria como trabajo futuro, sin que ello amplíe el alcance de lo planificado ni comprometa la estimación de recursos de este documento. En la práctica, y de acuerdo con el calendario real descrito en el apartado 4.3.2, la metodología se aplicó de forma plena durante la fase de desarrollo activo, mientras que las fases inicial y final, más centradas en la planificación y en la documentación, se apoyaron en las reuniones de seguimiento y en el tablero Kanban más que en la cadencia formal de los sprints.

5. Cronograma y Plan de Trabajo

5.1. *Marco general y objetivos de la planificación*

Este capítulo recoge el programa de trabajo del proyecto: el marco temporal en el que se desarrolla, el desglose de las tareas en sprints y la asignación de recursos y responsabilidades que lo sustentan. El punto de partida es vitalXAI, la plataforma web MLOps que integra inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax, definida en el capítulo de metas y propósitos. Sobre esa base, la planificación debe dar respuesta a los once objetivos específicos del proyecto, organizados en los bloques científico-metodológico y de ingeniería web: desde la revisión del estado del arte hasta la construcción de la interfaz clínica, el laboratorio MLOps, la seguridad y la internacionalización.

El programa de trabajo cubre el ciclo completo de concepción, implementación y documentación del sistema, y tiene un horizonte claro: la entrega y defensa del Trabajo Fin de Grado el 2 de septiembre de 2026. La ejecución se ha orquestado siguiendo la metodología ágil Scrum descrita en el capítulo de gestión, adaptada a la realidad de un proyecto unipersonal: la cadencia formal de los sprints se aplicó de forma plena durante la fase de implementación, mientras que las fases de planificación y de documentación se apoyaron en reuniones periódicas de seguimiento y en el tablero Kanban.

De acuerdo con el calendario real del proyecto, la planificación se articula en tres fases. La fase inicial, de finales de noviembre de 2025 a finales de febrero de 2026, estableció las bases organizativas, técnicas y documentales. La fase de desarrollo, de principios de marzo a principios de junio de 2026, concentró la implementación del sistema en siete sprints. Por último, la fase de documentación, desde principios de junio hasta septiembre de 2026, se dedicó a la elaboración de la memoria, los manuales y el cierre del proyecto.

El esfuerzo total planificado asciende a 474 horas, distribuidas en nueve sprints de duración desigual que reflejan la carga real de cada bloque de trabajo: la infraestructura y el laboratorio MLOps son los bloques más pesados, mientras que la validación externa es el más contenido. Las horas de las tareas representan esfuerzo estimado, mientras que las duraciones del diagrama de Gantt representan ventanas de calendario; ambas magnitudes no se convierten mediante una jornada diaria uniforme. La dedicación efectiva varía según la fase, la naturaleza de las tareas y la participación de los asesores, tal como se detalla en la asignación de recursos del apartado 5.3. Por tanto, el cronograma debe interpretarse como una planificación temporal y de esfuerzo, no como una deducción de horas diarias constantes.

5.2. Desglose de tareas por Sprint

El trabajo se estructura en nueve sprints, cada uno con un propósito definido y un incremento verificable del sistema. Los sprints de desarrollo se conciben como ciclos cerrados de dos semanas de duración, y entre ellos se impone una secuencialidad estricta: ninguno comienza sin que el anterior haya cerrado formalmente su revisión y su retrospectiva. Esta estructura encaja con el calendario real descrito en el capítulo de gestión: el primer sprint corresponde a la fase de planificación, los siete siguientes a la fase de desarrollo y el último a la documentación y el cierre.

El camino crítico del proyecto discurre por las tareas de mayor carga técnica y menor capacidad de paralelización: la implementación del pipeline de entrenamiento de las redes convolucionales (Sprint 4) y su prolongación en el pipeline Transformer (Sprint 5), que deben completarse antes de ejecutar el benchmarking final (Sprint 8). Ninguna de estas tareas puede iniciarse sin que la anterior haya concluido, y su ejecución concentra el mayor riesgo de desviación cronológica de todo el proyecto. En la siguiente figura se representa la estructura de desglose del trabajo (EDT), elaborada manualmente.

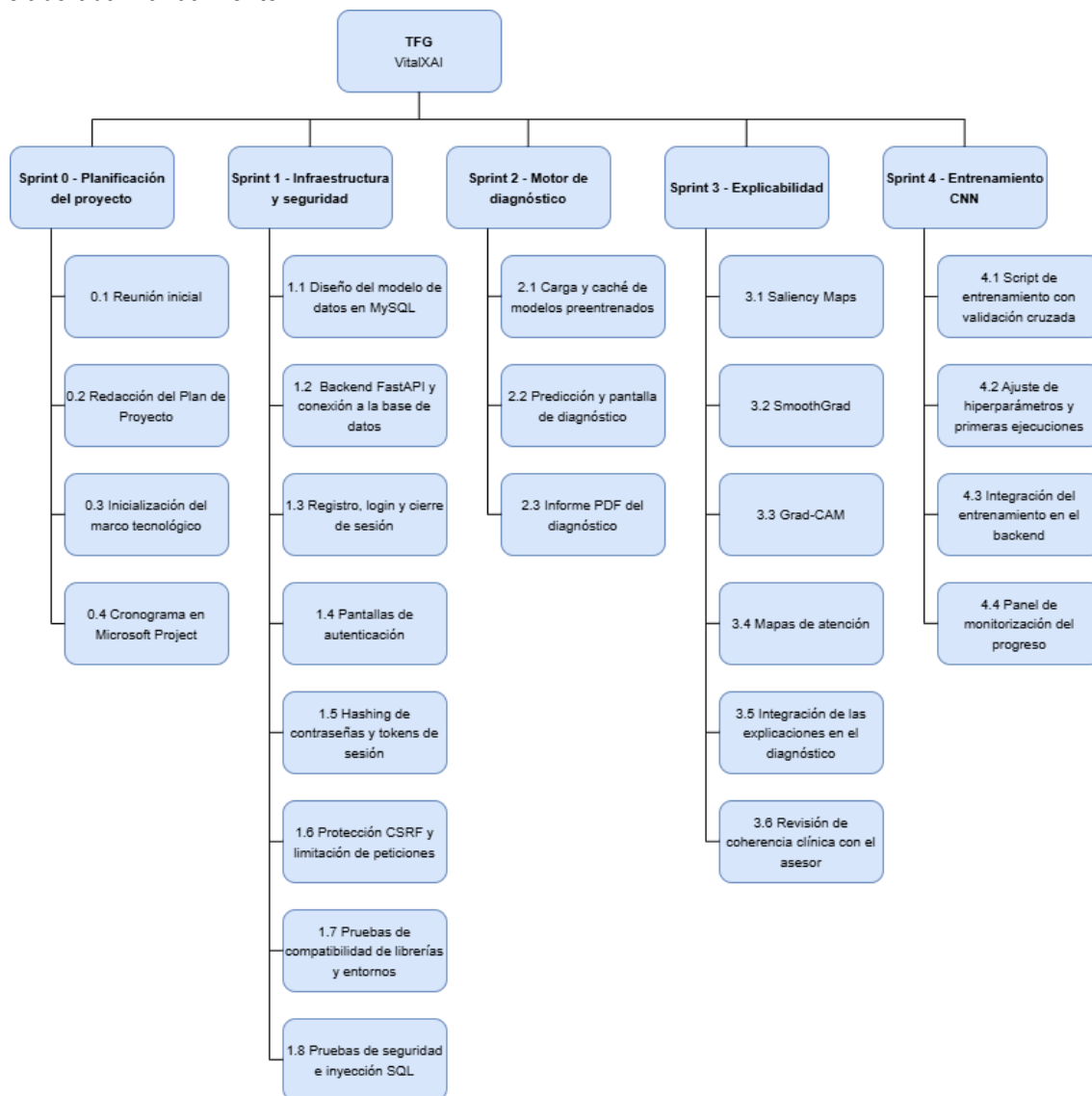


Ilustración x - Diagrama EDT I

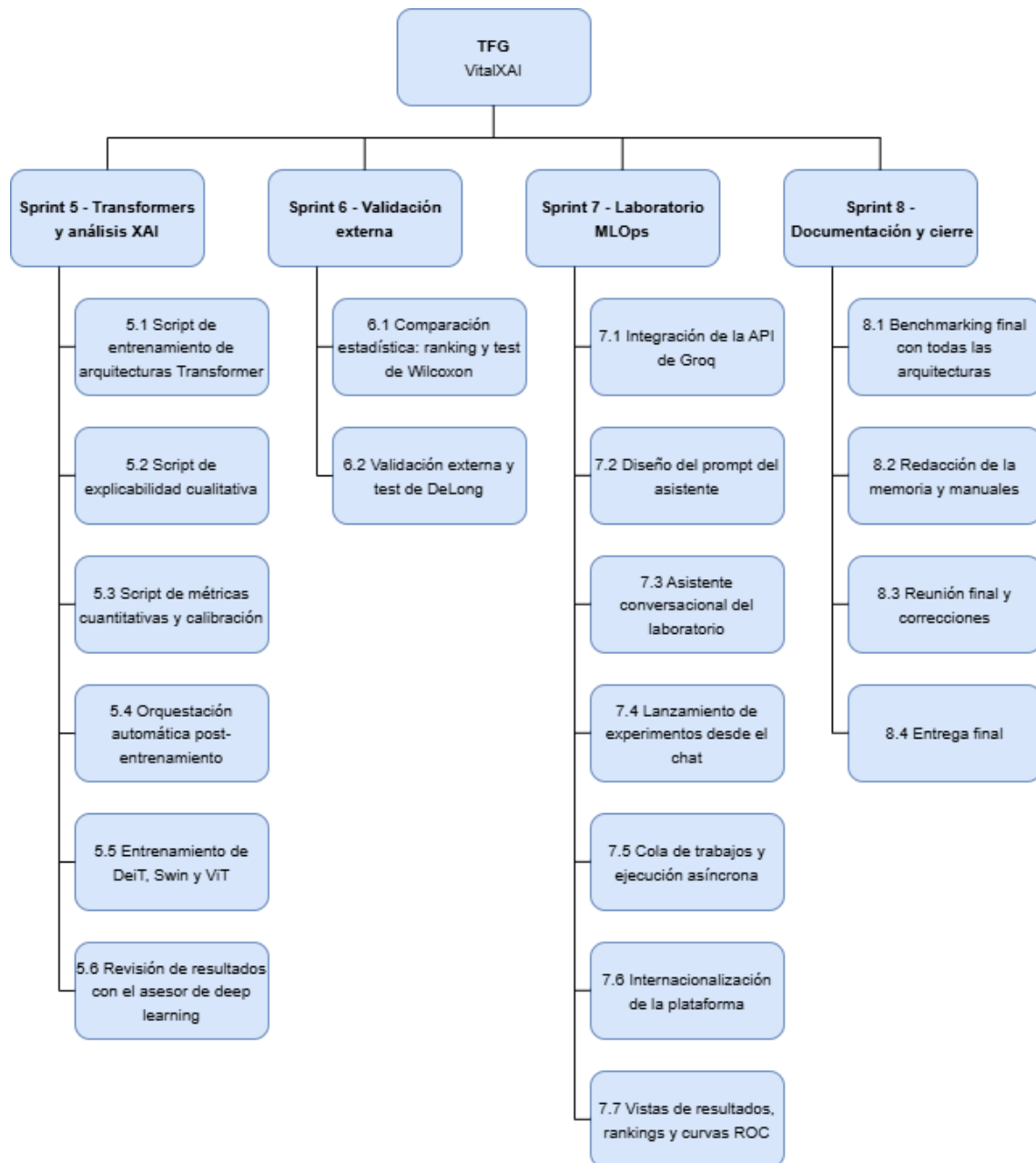


Ilustración x - Diagrama EDT II

5.2.1. Sprint 0 – Planificación del proyecto

Este sprint inicial sienta las bases del proyecto: no hay código, solo la preparación organizativa, técnica y documental. Ocupa la fase de planificación, de finales de noviembre de 2025 a finales de febrero de 2026, con una dedicación ligera y 42 horas de esfuerzo repartidas en cuatro tareas. En este sprint se realiza también la revisión bibliográfica que fundamenta el estado del arte y las decisiones metodológicas posteriores.

Código	Nombre	Responsable(s)	Estimación (h)
0.1	Reunión inicial	Luis Carmona Berdugo, Aurelio López Fernández	3
0.2	Redacción del Plan de Proyecto y revisión bibliográfica	Luis Carmona Berdugo	25
0.3	Inicialización del marco tecnológico	Luis Carmona Berdugo	8
0.4	Cronograma en Microsoft Project	Luis Carmona Berdugo	6

Tabla x - Recursos y estimaciones Sprint 0

Tarea 0.1 – Reunión inicial: encuentro con el tutor para fijar objetivos, alcance, tecnologías (FastAPI, TensorFlow, MySQL y Groq) y metodología.

Tarea 0.2 – Redacción del Plan de Proyecto y revisión bibliográfica: el alumno redacta el Plan de Proyecto, el documento marco que integra la definición de los objetivos, la organización del proyecto, la metodología de gestión, el plan de tareas y la evaluación de riesgos. Esta tarea también fija el marco documental y metodológico sobre el que se desarrolla la revisión del estado del arte. Es la tarea más extensa de este sprint y justifica sus veinticinco horas: sobre ella se sustentan los capítulos de contextualización, organización, metodología y riesgos de esta memoria. La redacción no es un mero trámite administrativo, sino el ejercicio de concreción que da forma al alcance del sistema y a su planificación. El resultado guía la ejecución de todos los sprints posteriores y sirve de contrato frente al tutor y al tribunal.

Tarea 0.3 – Inicialización del marco tecnológico: se fijan las versiones de las librerías, se preparan los entornos virtuales de Anaconda y se crea el repositorio. Esta tarea refleja la pequeña inicialización del marco tecnológico de la fase de planificación y blinda el entorno frente a las incompatibilidades de dependencias (riesgo R02).

Tarea 0.4 – Cronograma en Microsoft Project: se construye el diagrama de Gantt con las tareas, sus dependencias, los recursos y el camino crítico, que servirá de referencia para el seguimiento.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
■ 0. Planificación del Proyecto	70 días	lun 24/11/25	vie 27/02/26		42 horas	
0.1 Reunión Inicial	1 día	lun 24/11/25	lun 24/11/25	Luis Carmona Berdugo[50%];Aurelio López Fernández[50%]	3 horas	
0.2 Redacción del Plan de Proyecto	49 días	mar 25/11/25	vie 30/01/26	Luis Carmona Berdugo	25 horas	3
0.3 Inicialización del marco tecnológico	10 días	lun 02/02/26	vie 13/02/26	Luis Carmona Berdugo	8 horas	4
0.4 Cronograma en Microsoft Project	10 días	lun 16/02/26	vie 27/02/26	Luis Carmona Berdugo	6 horas	5

Ilustración x - Sprint 0 en Project

5.2.2. Sprint 1 – Infraestructura y seguridad

El primer sprint de desarrollo materializa el incremento funcional inaugural del sistema y es, junto con el laboratorio MLOps, el bloque de mayor carga de todo el proyecto, con 70 horas de esfuerzo. El código cobra vida: se construye el backend sobre FastAPI, se despliega la base de datos relacional MySQL que asume la persistencia de la información, se implementa el módulo de autenticación y se sientan las bases de la seguridad transversal. Su amplitud responde a que en este sprint se concentra, además de la construcción de la infraestructura, su blindaje completo: el cifrado de contraseñas, la gestión de sesiones y tokens, la protección frente a falsificación de peticiones, la limitación de acceso, las pruebas de compatibilidad de librerías sobre los entornos de Anaconda y las pruebas de seguridad. Con ocho tareas, es el sprint con más entregables del cronograma. Se desarrolla durante las dos primeras semanas de marzo de 2026 y deja la plataforma lista para recibir el motor de inferencia.

Código	Nombre	Responsable(s)	Estimación (h)
1.1	Diseño del modelo de datos en MySQL	Luis Carmona Berdugo, Vicente de Vides Rodríguez	10
1.2	Backend FastAPI y conexión a la base de datos	Luis Carmona Berdugo	14
1.3	Registro, login y cierre de sesión	Luis Carmona Berdugo	12
1.4	Pantallas de autenticación	Luis Carmona Berdugo	8
1.5	Hashing de contraseñas y tokens de sesión	Luis Carmona Berdugo	6
1.6	Protección CSRF y limitación de peticiones	Luis Carmona Berdugo	6
1.7	Pruebas de compatibilidad de librerías y entornos	Luis Carmona Berdugo	6
1.8	Pruebas de seguridad e inyección SQL	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 1

Tarea 1.1 – Diseño del modelo de datos en MySQL: se diseña el esquema relacional del sistema —usuarios, consultas de diagnóstico, sesiones de entrenamiento, cola de trabajos y tokens de refresco—. El consultor de persistencia y bases de datos aporta su criterio sobre el modelado y la optimización del esquema.

Tarea 1.2 – Backend FastAPI y conexión a la base de datos: se monta el backend con FastAPI y se configura la conexión a MySQL mediante un pool de conexiones, lo que permite atender el acceso concurrente de múltiples usuarios sin degradar el rendimiento. El renderizado de las páginas con Jinja2 y el servidor Uvicorn quedan operativos al cierre de esta tarea. Sobre este esqueleto se construyen todos los endpoints posteriores del sistema.

Tarea 1.3 – Registro, login y cierre de sesión: se implementan los flujos de creación de cuenta, inicio de sesión y cierre de sesión. Las contraseñas se cifran con bcrypt y la sesión se gestiona mediante tokens JWT almacenados en cookies seguras, con tokens de refresco y su rotación en cada uso.

Tarea 1.4 – Pantallas de autenticación: se construyen las interfaces de login y registro con el sistema de estilos de Tailwind. El flujo completo de acceso se verifica de forma integral, desde el formulario hasta la entrada al panel.

Tarea 1.5 – Hashing de contraseñas y tokens de sesión: se integra la capa de cifrado de contraseñas y la emisión y validación de los tokens de sesión y de refresco.

Tarea 1.6 – Protección CSRF y limitación de peticiones: se incorpora el middleware de doble cookie frente a la falsificación de peticiones y la limitación de intentos para frenar los ataques por fuerza bruta.

Tarea 1.7 – Pruebas de compatibilidad de librerías y entornos: se ejecutan pruebas sobre los entornos virtuales de Anaconda para garantizar que las versiones de TensorFlow y Keras conviven sin conflictos antes de construir el motor de inferencia (riesgo R02).

Tarea 1.8 – Pruebas de seguridad e inyección SQL: se validan los mecanismos de autenticación y autorización y se prueban los endpoints frente a inyección SQL y exposición de datos sensibles.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
➤ 1. Infraestructura y Seguridad	12 días	lun 02/03/26	mar 17/03/26		70 horas	
1.1 Diseño del modelo de datos en MySQL	2 días	lun 02/03/26	mar 03/03/26	Luis Carmona Berdugo[85%]; Vicente de Vides Rodríguez[15%]	10 horas	6
1.2 Backend FastAPI y conexión a la BD	2 días	mié 04/03/26	jue 05/03/26	Luis Carmona Berdugo	14 horas	8
1.3 Registro, login y cierre de sesión	2 días	vie 06/03/26	lun 09/03/26	Luis Carmona Berdugo	12 horas	9
1.4 Pantallas de autenticación	2 días	lun 09/03/26	mar 10/03/26	Luis Carmona Berdugo	8 horas	
1.5 Hashing de contraseñas y tokens de sesión	3 días	mar 10/03/26	jue 12/03/26	Luis Carmona Berdugo	6 horas	
1.6 Protección CSRF y limitación de peticiones	2 días	mié 11/03/26	vie 13/03/26	Luis Carmona Berdugo	6 horas	
1.7 Pruebas de compatibilidad de librerías y entorno	2 días	jue 12/03/26	lun 16/03/26	Luis Carmona Berdugo	6 horas	11
1.8 Pruebas de seguridad e inyección SQL	2 días	lun 16/03/26	mar 17/03/26	Luis Carmona Berdugo	8 horas	

Ilustración x - Sprint 1 en Project

5.2.3. Sprint 2 – Motor de diagnóstico

Este sprint construye el núcleo del diagnóstico clínico: la carga de los modelos preentrenados, el endpoint de predicción y la interfaz que permite al facultativo subir una radiografía y ver el resultado. Con 41 horas en tres tareas, cierra la primera versión funcional del flujo de diagnóstico en la segunda mitad de marzo.

Código	Nombre	Responsable(s)	Estimación (h)
2.1	Carga y caché de modelos preentrenados	Luis Carmona Berdugo	15
2.2	Predicción y pantalla de diagnóstico	Luis Carmona Berdugo	18
2.3	Informe PDF del diagnóstico	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 2

Tarea 2.1 – Carga y caché de modelos preentrenados: se desarrolla el motor de carga de los modelos CNN y Transformer desde sus pesos. Un sistema de caché en memoria evita recargar el modelo en cada petición, de modo que el primer diagnóstico de cada arquitectura carga los pesos y los siguientes son instantáneos. Esta optimización es clave para la experiencia del facultativo en la consulta.

Tarea 2.2 – Predicción y pantalla de diagnóstico: se implementa el endpoint de predicción y la pantalla de diagnóstico en la que el facultativo sube la radiografía, selecciona la arquitectura y obtiene el diagnóstico con su nivel de confianza. El procesamiento se encola de forma asíncrona para no bloquear la interfaz durante la inferencia. Es la tarea central de este sprint: sobre ella se apoyan los mapas de explicabilidad del sprint siguiente. La pantalla presenta el resultado de forma clara y sin tecnicismos, pensando en un usuario sin formación técnica.

Tarea 2.3 – Informe PDF del diagnóstico: se implementa la generación del informe PDF que recoge la radiografía, el diagnóstico, el nivel de confianza y el modelo empleado, listo para su descarga y archivo.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
2. Motor de diagnóstico	10 días	mié 18/03/21	mar 31/03/26		41 horas	
2.1 Carga y caché de modelos preentrenados	3 días	mié 18/03/21	vie 20/03/26	Luis Carmona Berdugo	15 horas	15
2.2 Predicción y pantalla de diagnóstico	3 días	lun 23/03/26	mié 25/03/26	Luis Carmona Berdugo	18 horas	17
2.3 Informe PDF del diagnóstico	4 días	jue 26/03/26	mar 31/03/26	Luis Carmona Berdugo	8 horas	18

Ilustración x - Sprint 2 en Project

5.2.4. Sprint 3 – Explicabilidad

Este sprint incorpora la inteligencia artificial explicable al sistema. Se implementan las tres familias de técnicas visuales —Saliency Maps, SmoothGrad y Grad-CAM, además de los mapas de atención para los Transformers— y se integran en el flujo de diagnóstico, de modo que cada predicción pasa a ser una decisión verificable. Con 48 horas repartidas en seis tareas, el sprint se desarrolla a lo largo de las dos primeras semanas de abril. La revisión del asesor de imagen médica cierra el bloque garantizando la coherencia clínica de las explicaciones.

Código	Nombre	Responsable(s)	Estimación (h)
3.1	Saliency Maps	Luis Carmona Berdugo, Iván Segura-Carmona	10
3.2	SmoothGrad	Luis Carmona Berdugo	8
3.3	Grad-CAM	Luis Carmona Berdugo, Iván Segura-Carmona	12
3.4	Mapas de atención	Luis Carmona Berdugo	8
3.5	Integración de las explicaciones en el diagnóstico	Luis Carmona Berdugo	6
3.6	Revisión de coherencia clínica con el asesor	Luis Carmona Berdugo, Marc Ríos Cadenas	4

Tabla x - Recursos y estimaciones Sprint 3

Tarea 3.1 – Saliency Maps: se implementa el mapa de prominencia que calcula el gradiente de la clase predicha respecto a cada píxel, resaltando las regiones más influyentes en la decisión del modelo. La revisión del asesor de deep learning y XAI valida la corrección del cálculo.

Tarea 3.2 – SmoothGrad: se implementa la variante que promedia los mapas de saliencia sobre versiones con ruido gaussiano. El resultado es un mapa más suave y estable, en el que los patrones consistentes del modelo destacan sobre el ruido.

Tarea 3.3 – Grad-CAM: se implementan los mapas de activación de clase a partir de la última capa convolucional, que superponen a la radiografía un mapa de calor grueso y fácil de interpretar. Su suavidad y su correspondencia con las regiones anatómicas lo convierten en la técnica más valiosa para el profesional clínico. Se valida su correcta implementación con el asesor de deep learning y XAI.

Tarea 3.4 – Mapas de atención: se implementa la extracción de los pesos de atención de la última capa de los Transformers, promediando las cabezas de atención para obtener el mapa de relevancia de cada parche.

Tarea 3.5 – Integración de las explicaciones en el diagnóstico: se integra la generación de los mapas en el flujo de predicción, de modo que cada consulta genera automáticamente sus explicaciones.

Tarea 3.6 – Revisión de coherencia clínica con el asesor: el asesor de imagen médica comprueba que los mapas apuntan a las regiones pulmonares relevantes y no a artefactos.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
4 3. Explicabilidad	8 días	mié 01/04/21	vie 10/04/26		48 horas	
3.1 Saliency Maps	2 días	mié 01/04/21	jue 02/04/26	Luis Carmona Berdugo[90%];Iván Segura Carmona[10%]	10 horas	19
3.2 SmoothGrad	1 día	vie 03/04/26	vie 03/04/26	Luis Carmona Berdugo	8 horas	21
3.3 Grad-CAM	2 días	lun 06/04/26	mar 07/04/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	22
3.4 Mapas de atención	1 día	mié 08/04/21	mié 08/04/26	Luis Carmona Berdugo	8 horas	23
3.5 Integración de las explicaciones en el diagnóstico	1 día	jue 09/04/26	jue 09/04/26	Luis Carmona Berdugo	6 horas	24
3.6 Revisión de coherencia clínica con asesor	1 día	vie 10/04/26	vie 10/04/26	Luis Carmona Berdugo[75%];Marc Ríos Cadena[25%]	4 horas	25

Ilustración x - Sprint 3 en Project

5.2.5. Sprint 4 – Entrenamiento CNN

Este sprint implementa el primer pipeline de entrenamiento automatizado. Se desarrolla el script que entrena las arquitecturas convolucionales con validación cruzada de cinco pliegues y balanceo de clases, se integra con el backend para poder lanzar experimentos desde la plataforma y se construye el panel de monitorización del progreso. Es, junto con el sprint siguiente, el corazón del camino crítico del proyecto: sin el pipeline de entrenamiento no existe benchmarking posible. Con 52 horas en cuatro tareas, ocupa la segunda quincena de abril. El asesor de deep learning y XAI revisa la corrección del pipeline y de sus métricas.

Código	Nombre	Responsable(s)	Estimación (h)
4.1	Script de entrenamiento con validación cruzada	Luis Carmona Berdugo, Iván Segura-Carmona	20
4.2	Ajuste de hiperparámetros y primeras ejecuciones	Luis Carmona Berdugo	14
4.3	Integración del entrenamiento en el backend	Luis Carmona Berdugo	8
4.4	Panel de monitorización del progreso	Luis Carmona Berdugo	10

Tabla x - Recursos y estimaciones Sprint 4

Tarea 4.1 – Script de entrenamiento con validación cruzada: se implementa el script que entrena dinámicamente cualquier arquitectura de `tf.keras.applications` con cinco pliegues estratificados y submuestreo aleatorio de la clase mayoritaria. La base preentrenada en ImageNet se mantiene congelada y se sustituye la cabeza de clasificación por una propia, con callbacks de guardado del mejor modelo, parada temprana y reducción de la tasa de aprendizaje. La estimación incluye la resolución de las incompatibilidades entre Keras y `tf_keras`, que consumió más tiempo del previsto. Es la tarea más costosa del sprint y la base de todo el benchmarking posterior.

Tarea 4.2 – Ajuste de hiperparámetros y primeras ejecuciones: se lanzan las primeras ejecuciones sobre arquitecturas ligeras (MobileNetV2, EfficientNetB0) para calibrar el hardware y ajustar el número de épocas, el tamaño de lote y la tasa de aprendizaje. Esta fase acumula experiencia sobre los tiempos reales de entrenamiento y permite estimar con realismo la duración de los sprints siguientes. Es un paso previo imprescindible antes de liberar a las arquitecturas más pesadas.

Tarea 4.3 – Integración del entrenamiento en el backend: se conecta el script de entrenamiento con la API, de modo que un experimento pueda lanzarse desde la plataforma y ejecutarse como un proceso independiente sin bloquear la interfaz.

Tarea 4.4 – Panel de monitorización del progreso: se construye la vista que muestra el estado del entrenamiento en tiempo real, con el progreso por modelo y la consulta del estado de la cola de trabajos. El facultativo investigador puede así seguir la evolución de sus experimentos sin salir de la plataforma.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
4. Entrenamiento CNN	10 días	lun 13/04/26	vie 24/04/26		52 horas	
4.1 Script de entrenamiento con validación cruzada	4 días	lun 13/04/26	jue 16/04/26	Iván Segura Carmona[3%];Luis Carmona Berdugo[97%]	20 horas	26
4.2 Ajuste de hiperparámetros y primeras ejecuciones	2 días	vie 17/04/26	lun 20/04/26	Luis Carmona Berdugo	14 horas	28
4.3 Integración del entrenamiento en el backend	1 día	mar 21/04/26	mar 21/04/26	Luis Carmona Berdugo	8 horas	29
4.4 Panel de monitorización del progreso	3 días	mié 22/04/26	vie 24/04/26	Luis Carmona Berdugo	10 horas	30

Ilustración x - Sprint 4 en Project

5.2.6. Sprint 5 – Transformers y análisis XAI

Este sprint amplía el marco de entrenamiento a las arquitecturas Transformer y desarrolla los módulos de análisis XAI que se ejecutan automáticamente tras cada entrenamiento. Es el segundo bloque del camino crítico del proyecto: el entrenamiento de los Vision Transformers y la evaluación de su explicabilidad condicionan directamente el benchmarking final. Con 62 horas en seis tareas, el sprint se extiende desde finales de abril hasta el 19 de mayo, con el entrenamiento de DeiT, Swin-Base y ViT-384 prolongándose hasta mediados de mayo y la revisión final del asesor realizándose el 18 de mayo. Incluye la implementación de los scripts de análisis cualitativo y cuantitativo, la orquestación automática que garantiza que ningún modelo quede sin su evaluación de explicabilidad y la revisión final del asesor de deep learning y XAI sobre los resultados.

Código	Nombre	Responsable(s)	Estimación (h)
5.1	Script de entrenamiento de arquitecturas Transformer	Luis Carmona Berdugo, Iván Segura-Carmona	16
5.2	Script de explicabilidad cualitativa	Luis Carmona Berdugo	10
5.3	Script de métricas cuantitativas y calibración	Luis Carmona Berdugo, Iván Segura-Carmona	12
5.4	Orquestación automática post-entrenamiento	Luis Carmona Berdugo	6
5.5	Entrenamiento de DeiT, Swin y ViT	Luis Carmona Berdugo	14
5.6	Revisión de resultados con el asesor de deep learning	Luis Carmona Berdugo, Iván Segura-Carmona	4

Tabla x - Recursos y estimaciones Sprint 5

Tarea 5.1 – Script de entrenamiento de arquitecturas Transformer: se implementa el script que entrena DeiT, Swin-Base y ViT-384 con HuggingFace Transformers y validación cruzada de cinco pliegues, reutilizando el esquema de balanceo y de guardado de pesos del pipeline convolucional. La integración con la librería de Transformers obliga a adaptar el manejo de pesos y la tasa de aprendizaje, con ajustes específicos para lograr la convergencia. Es la tarea más exigente del sprint desde el punto de vista técnico. Su correcta ejecución es condición necesaria para el benchmarking final.

Tarea 5.2 – Script de explicabilidad cualitativa: se implementa el script que genera los mapas de explicabilidad visuales sobre imágenes de ejemplo, con Grad-CAM y saliencia para las CNN y saliencia para los Transformers. El resultado es la galería de mapas de cada modelo, que permite la inspección visual de la coherencia de las decisiones.

Tarea 5.3 – Script de métricas cuantitativas y calibración: se implementa el cálculo de las métricas de fidelidad de las explicaciones —Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM— y de la calibración de las predicciones mediante el Expected Calibration Error y el Brier Score. La revisión del asesor de deep learning y XAI valida el cálculo de estas métricas.

Tarea 5.4 – Orquestación automática post-entrenamiento: se automatiza la ejecución de los análisis XAI al finalizar cada entrenamiento, garantizando que ningún modelo quede sin su evaluación de explicabilidad.

Tarea 5.5 – Entrenamiento de DeiT, Swin y ViT: se ejecuta el entrenamiento completo de las tres arquitecturas Transformer sobre el dataset. La duración de esta tarea en el cronograma refleja el coste computacional real de entrenar los Transformers, sensiblemente mayor que el de las CNN. Se aplican los ajustes de tasa de aprendizaje necesarios para lograr la convergencia de los tres modelos.

Tarea 5.6 – Revisión de resultados con el asesor de deep learning: el asesor valida la interpretación de los resultados del entrenamiento y de la explicabilidad de los Transformers.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
➤ 5. Transformers y análisis XAI	17 días	lun 27/04/26	mar 19/05/26		62 horas	
5.1 Script de entrenamiento de arquitecturas Transformer	3 días	lun 27/04/26	mié 29/04/26	Iván Segura Carmona[3%];Luis Carmona Berdugo[97%]	16 horas	31
5.2 Script de explicabilidad cualitativa	3 días	jue 30/04/26	lun 04/05/26	Luis Carmona Berdugo	10 horas	33
5.3 Script de métricas cuantitativas y calibración	3 días	lun 04/05/26	mié 06/05/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	
5.4 Orquestación automática post-entreno	1 día	jue 07/05/26	vie 08/05/26	Luis Carmona Berdugo	6 horas	35
5.5 Entrenamiento de Deit, Swin y ViT	5 días	lun 11/05/26	vie 15/05/26	Luis Carmona Berdugo	14 horas	36
5.6 Revisión de resultados con el asesor de deep learning	1 día	lun 18/05/26	lun 18/05/26	Iván Segura Carmona[25%];Luis Carmona Berdugo[75%]	4 horas	37

Ilustración x - Sprint 5 en Project

5.2.7. Sprint 6 – Validación externa

Este es el sprint más contenido del proyecto, con 26 horas en dos tareas y una duración de cuatro días. Implementa los mecanismos de validación externa y de análisis estadístico que determinan si las diferencias entre modelos son reales o fruto del azar. Se desarrolla en la segunda mitad de mayo.

Código	Nombre	Responsable(s)	Estimación (h)
6.1	Comparación estadística: ranking y test de Wilcoxon	Luis Carmona Berdugo, Iván Segura-Carmona	12
6.2	Validación externa y test de DeLong	Luis Carmona Berdugo	14

Tabla x - Recursos y estimaciones Sprint 6

Tarea 6.1 – Comparación estadística: ranking y test de Wilcoxon: se implementa el script que genera el ranking global de los modelos por su AUC medio y calcula una matriz exploratoria de p-valores por pares. Dado que solo se dispone de cinco folds dependientes, el test de Wilcoxon no se utilizará para afirmar significación confirmatoria; cualquier contraste por pares deberá incorporar una corrección por comparaciones múltiples y sus limitaciones deberán quedar documentadas. Es la tarea más costosa del sprint.

Tarea 6.2 – Validación externa y test de DeLong: se implementa la evaluación de los modelos congelados sobre la cohorte independiente de pacientes adultos y el test de DeLong para comparar las curvas ROC. Esta tarea constituye el examen de generalización del proyecto: sobre sus resultados se sustentan las conclusiones de la memoria. Genera las matrices de significación de la validación externa.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
▸ 6. Validación externa	4 días	mar 19/05/21	vie 22/05/26		26 horas	
6.1 Comparación estadística: ranking y test de Wilcoxon	2 días	mar 19/05/21	mié 20/05/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	38
6.2 Validación externa y test de DeLong	2 días	jue 21/05/26	vie 22/05/26	Luis Carmona Berdugo	14 horas	40

Ilustración x - Sprint 6 en Project

5.2.8. Sprint 7 – Laboratorio MLOps

El laboratorio MLOps es, junto con la infraestructura, el bloque de mayor carga del proyecto, con 66 horas en siete tareas. Integra el asistente conversacional basado en Groq y Llama 3, que permite al usuario configurar y lanzar experimentos en lenguaje natural sin escribir código: desde la integración de la API de Groq y el diseño del prompt hasta el lanzamiento de los experimentos desde la conversación. El sprint incorpora también la cola de trabajos que garantiza la ejecución asíncrona de diagnósticos, entrenamientos y validaciones externas, la internacionalización de la plataforma en cuatro idiomas y las vistas de resultados con el ranking, las matrices de significación y las curvas ROC. Se desarrolla en la última semana de mayo y la primera de junio y convierte el laboratorio en la puerta de entrada de los experimentos para el usuario investigador. Es, junto con el sprint de infraestructura, el más amplio del cronograma.

Código	Nombre	Responsable(s)	Estimación (h)
7.1	Integración de la API de Groq	Luis Carmona Berdugo	8
7.2	Diseño del prompt del asistente	Luis Carmona Berdugo	12
7.3	Asistente conversacional del laboratorio	Luis Carmona Berdugo	14
7.4	Lanzamiento de experimentos desde el chat	Luis Carmona Berdugo	8
7.5	Cola de trabajos y ejecución asíncrona	Luis Carmona Berdugo	8
7.6	Internacionalización de la plataforma	Luis Carmona Berdugo	8
7.7	Vistas de resultados, rankings y curvas ROC	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 7

Tarea 7.1 – Integración de la API de Groq: se configura el cliente de Groq y la conexión con el modelo llama-3.3-70b-versatile que alimenta al asistente conversacional del laboratorio.

Tarea 7.2 – Diseño del prompt del asistente: se diseña el prompt de sistema que define al asistente como experto en MLOps médico. Incluye las reglas de extracción de los cinco parámetros del experimento —ruta del dataset, arquitecturas, épocas, lote y tasa de aprendizaje— y el formato JSON de salida. Es una tarea de diseño más que de código, y por eso su estimación es moderada.

Tarea 7.3 – Asistente conversacional del laboratorio: se implementa el endpoint de chat y la sala de conversación, con el historial de mensajes por sesión. El asistente interpreta la configuración devuelta por el modelo y la traduce al panel del experimento, para que el usuario pueda revisarla y confirmarla. Es la tarea más costosa del sprint.

Tarea 7.4 – Lanzamiento de experimentos desde el chat: se conecta la configuración detectada por el asistente con el lanzamiento del pipeline de entrenamiento, de modo que el experimento se encola y arranca sin escribir código.

Tarea 7.5 – Cola de trabajos y ejecución asíncrona: se implementa la cola de trabajos que gestiona los diagnósticos, los entrenamientos y las validaciones externas, con monitorización de estado y cancelación de las tareas pendientes.

Tarea 7.6 – Internacionalización de la plataforma: se incorpora el soporte multilingüe de la interfaz, los informes y el asistente en los cuatro idiomas de la plataforma, mediante atributos de traducción y diccionarios en JavaScript.

Tarea 7.7 – Vistas de resultados, rankings y curvas ROC: se construyen las vistas del laboratorio que muestran el ranking de modelos, las matrices de significación y las curvas ROC de la validación externa.

Nombre de tarea	Duración	Comienzo	Fin	Nombres de los recursos	Trabajo	Predecesoras
7. Laboratorio MLOps	10 días	lun 25/05/26	vie 05/06/26		66 horas	
7.1 Integración de la API de Groq	1 día	lun 25/05/26	lun 25/05/26	Luis Carmona Berdugo	8 horas	41
7.2 Diseño del prompt del asistente	2 días	mar 26/05/26	mié 27/05/26	Luis Carmona Berdugo	12 horas	43
7.3 Asistente conversacional del laboratorio	2 días	jue 28/05/26	vie 29/05/26	Luis Carmona Berdugo	14 horas	44
7.4 Lanzamiento de experimentos desde el chat	1 día	lun 01/06/26	lun 01/06/26	Luis Carmona Berdugo	8 horas	45
7.5 Cola de trabajos y ejecución asíncrona	1 día	mar 02/06/26	mar 02/06/26	Luis Carmona Berdugo	8 horas	46
7.6 Internacionalización de la plataforma	1 día	mié 03/06/26	mié 03/06/26	Luis Carmona Berdugo	8 horas	47
7.7 Vistas de resultados, rankings y curvas ROC	1 día	jue 04/06/26	jue 04/06/26	Luis Carmona Berdugo	8 horas	48

Ilustración x - Sprint 7 en Project

5.2.9. Sprint 8 – Documentación y cierre

El sprint final abandona el código para concentrarse en el benchmarking definitivo, la redacción de la memoria y de los manuales, las correcciones finales y la entrega. Con 67 horas en cuatro tareas, se desarrolla desde el 5 de junio hasta el 2 de septiembre de 2026, con una dedicación repartida entre la ejecución experimental, la documentación y el cierre. La redacción de la memoria es la tarea más extensa del sprint, mientras que las correcciones finales y la entrega se han separado en tareas específicas. El sprint culmina con la entrega del 2 de septiembre de 2026.

Código	Nombre	Responsable(s)	Estimación (h)
8.1	Benchmarking final con todas las arquitecturas	Luis Carmona Berdugo, Iván Segura-Carmona	18
8.2	Redacción de la memoria y manuales	Luis Carmona Berdugo	30
8.3	Reunión final y correcciones	Luis Carmona Berdugo, Aurelio López Fernández	16
8.4	Entrega final	Luis Carmona Berdugo	3

Tabla x - Recursos y estimaciones Sprint 8

Tarea 8.1 – Benchmarking final con todas las arquitecturas: se ejecuta la evaluación completa del banco de pruebas, consolidando las métricas de rendimiento, calibración y explicabilidad de todas las arquitecturas. El asesor de deep learning y XAI y el asesor de imagen médica revisan la coherencia de los resultados y de los mapas de explicabilidad. Es la tarea técnica más densa del sprint, ya que cierra el proceso experimental del proyecto. Sus conclusiones alimentan directamente los capítulos de resultados y conclusiones de la memoria.

Tarea 8.2 – Redacción de la memoria y manuales: se redacta la memoria completa del Trabajo Fin de Grado, integrando el plan de proyecto, el análisis, el diseño, la implementación, las pruebas y las conclusiones, junto con el manual de usuario orientado a facultativos sin formación técnica. La redacción se realiza de forma incremental a lo largo de la fase de documentación, incorporando la revisión y las correcciones del tutor. Cada capítulo se elabora y se revisa por separado antes de su integración en el documento final. Es la tarea de mayor carga individual del proyecto, y su extensión justifica con holgura las treinta horas estimadas. El manual de usuario se redacta pensando en el profesional sanitario, con un lenguaje no técnico y capturas de los flujos principales.

Tarea 8.3 – Reunión final y cierre: reunión con el tutor y periodo de corrección programados del 28 de agosto al 1 de septiembre. Esta tarea permite revisar el conjunto del trabajo, aplicar las correcciones detectadas, preparar la defensa y dejar la versión final lista antes de la entrega.

Tarea 8.4 – Entrega final: depósito de la memoria y de los entregables del proyecto el 2 de septiembre de 2026. Esta tarea constituye el cierre administrativo posterior a la revisión y corrección final.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
➤ 8. Documentación y cierre	64 días	vie 05/06/26	mié 02/09/26		67 horas	
8.1 Benchmarking final con todas las arquitecturas	10 días	vie 05/06/26	jue 18/06/26	Iván Segura Carmona[5%];Luis Carmona Berdugo[90%];Marc Ríos Cadena[5%]	18 horas	49
8.2 Redacción de la memoria y manuales	50 días	vie 19/06/26	jue 27/08/26	Luis Carmona Berdugo	30 horas	51
8.3 Reunión final y correcciones	3 días	vie 28/08/26	mar 01/09/26	Aurelio López Fernández[50%];Luis Carmona Berdugo[50%]	16 horas	52
8.4 Entrega final	1 día	mié 02/09/26	mié 02/09/26	Luis Carmona Berdugo	3 horas	53

Ilustración x - Sprint 8 en Project

5.3. Recursos y costes del proyecto

Este apartado detalla la asignación de recursos del proyecto, abarcando todo lo necesario para ejecutar las tareas descritas anteriormente. Se establecen dos categorías fundamentales: los recursos de trabajo, centrados en el capital humano, y los recursos materiales, que engloban la infraestructura y el equipamiento tecnológico.

Microsoft Project centraliza la administración del proyecto. Los recursos ocasionales, como el tutor y los asesores, reciben un horario fijo; esto impide que el software reasigne automáticamente su esfuerzo ante variaciones en la duración de las tareas. Todas las asignaciones se han auditado manualmente para que la carga de los recursos laborables sea coherente con la dedicación real de cada fase del proyecto.

Recursos de trabajo:

- **Luis Carmona Berdugo:** asume la carga técnica integral del proyecto. Acumula 455,14 horas distribuidas a lo largo de todos los sprints. Con una tarifa estándar de 20 €/h, su esfuerzo se cuantifica en 9.102,80 €.
- **Aurelio López Fernández (tutor):** interviene estratégicamente en los hitos de apertura (tarea 0.1) y de clausura y correcciones (tarea 8.3). Dedicar 9,5 horas de supervisión a 50 €/h, sumando un total de 475,00 €.
- **Vicente de Vides Rodríguez (consultor de persistencia y bases de datos):** aporta su experiencia estructural en el diseño y la optimización de la persistencia de datos. Participa en la tarea 1.1 con 1,5 horas a 50 €/h, con un total de 75,00 €.
- **Iván Segura Carmona (asesor Deep Learning y XAI):** supervisa el núcleo algorítmico. Valida las implementaciones de explicabilidad, los pipelines CNN y Transformer, la comparación estadística y el benchmarking definitivo. Acumula 5,96 horas a 50 €/h, con un total de 298,00 €.
- **Marc Ríos Cadenas (asesor Imagen Médica):** aporta el rigor clínico. Participa en la revisión de la coherencia de los mapas de explicabilidad y en la validación del benchmarking. Dedicar 1,9 horas a 50 €/h, sumando 95,00 €.

El coste total de los recursos de trabajo asciende, por tanto, a aproximadamente 10.045,80 €.

Nombre del recurso ▼	Trabajo ▼	Costo ▼	Comienzo ▼	Fin ▼
▷ Luis Carmona Berdugo	455,14 horas	9.102,80 €	lun 24/11/25	mié 02/09/26
▷ Aurelio López Fernández	9,5 horas	475,00 €	lun 24/11/25	mar 01/09/26
▷ Vicente de Vides Rodríguez	1,5 horas	75,00 €	lun 02/03/26	mar 03/03/26
▷ Iván Segura Carmona	5,96 horas	298,00 €	mié 01/04/26	jue 18/06/26
▷ Marc Ríos Cadena	1,9 horas	95,00 €	vie 10/04/26	jue 18/06/26

Ilustración x - Vista "Uso de Recursos" en Project

Recursos Materiales:

- **Equipo de desarrollo:** se utiliza un equipo local con GPU NVIDIA compatible con CUDA, disponible para el desarrollo. El plan no especifica el modelo de GPU ni su memoria VRAM, por lo que la viabilidad de las arquitecturas con mayor resolución o capacidad no puede justificarse únicamente desde este apartado. Al tratarse de infraestructura ya disponible, no se imputa una amortización adicional al TFG.
- **Licencias de software:** coste nulo. La plataforma descansa enteramente sobre tecnologías de código abierto (Python, TensorFlow, FastAPI, MySQL), eliminando cualquier gasto de licenciamiento.
- **Servicios cloud:** no se presupuesta ningún servicio cloud en el plan base. La nube aparece únicamente como contingencia ante un fallo del equipo local, pero no existe una reserva económica asignada; si fuera necesario activarla, habría que aprobar un coste adicional o reducir el alcance del benchmarking y documentar la decisión.
- **Electricidad y API de Groq:** no se dispone de una medición o factura imputable exclusivamente al proyecto, por lo que ambos conceptos quedan fuera del presupuesto cuantificado. El consumo energético se reconoce como impacto operativo y el uso de Groq queda sujeto al plan y a las cuotas vigentes del servicio.

El coste económico documentado del proyecto asciende a 10.045,80 €, correspondiente a los recursos humanos presupuestados. Esta cifra no debe interpretarse como un coste completo de explotación: excluye la amortización del equipo ya disponible, la electricidad, una eventual infraestructura cloud y cualquier consumo facturado de la API de Groq.

5.4. *Reparto de responsabilidades*

Este apartado recoge la distribución de responsabilidades sobre las tareas definidas en el apartado 5.2. Para cada tarea se identifica qué persona asume la titularidad, qué interesados intervienen de forma puntual y en qué proporción. El reparto porcentual de cada responsable sobre cada tarea —recogido en las tablas del apartado 5.2— constituye la base sobre la que se calculan las horas y los costes del apartado 5.3, y se apoya en los roles definidos en la matriz RACI del capítulo de organización. La lógica del reparto es sencilla: el alumno desarrollador concentra la práctica totalidad de la carga de trabajo, mientras que el resto de los interesados participa de forma puntual y consultiva en los hitos donde su conocimiento especializado resulta indispensable.

La responsabilidad absoluta sobre la ejecución del proyecto recae sobre **Luis Carmona Berdugo**. El alumno asume en solitario todo el peso técnico y documental a lo largo de los nueve sprints, acumulando 455,14 horas de trabajo, lo que representa prácticamente la totalidad del esfuerzo planificado. Su figura absorbe íntegramente los roles de analista, diseñador, desarrollador backend y frontend, ingeniero de deep learning, tester y documentador: analiza los requisitos, diseña la arquitectura, implementa tanto el backend como la interfaz, construye los pipelines de entrenamiento y los módulos XAI, ejecuta el benchmarking y redacta toda la documentación del proyecto. Al tratarse de un Trabajo Fin de Grado, el desarrollo carece de un reparto transversal de tareas entre distintos perfiles profesionales, y es precisamente esta concentración la que justifica que el alumno asuma prácticamente el cien por cien de las horas en la mayoría de las tareas. En las tareas en las que intervienen los asesores, su participación sigue siendo mayoritaria, con porcentajes que oscilan entre el 50% de las reuniones y el 97% de las tareas de entrenamiento.

No obstante, la profunda naturaleza multidisciplinar del sistema exige el apoyo puntual y consultivo de varios interesados. Estas figuras aportan su conocimiento especializado y auditan el proceso únicamente en aquellos hitos donde su validación resulta indispensable para garantizar el rigor del proyecto. A continuación se describe de forma pormenorizada la participación exacta de cada uno de ellos.

Aurelio López Fernández asume la figura central de tutor académico y científico del proyecto. Su participación remunerada se concentra en dos hitos concretos del cronograma: la tarea 0.1, correspondiente a la reunión inicial, y la tarea 8.3, correspondiente a la reunión final y al periodo de correcciones. En ambas comparte la dedicación al cincuenta por ciento con el alumno, lo que supone un total de 9,5 horas de supervisión y un coste de 475 €. El seguimiento del resto del ciclo se realiza mediante comunicación y revisiones puntuales dentro de la dedicación académica del proyecto, pero no se computa como dieciocho reuniones formales ni como horas adicionales en el presupuesto.

Iván Segura Carmona actúa como asesor experto en aprendizaje profundo e inteligencia artificial explicable. Su participación, de naturaleza estrictamente consultiva, se distribuye a lo largo de los sprints de implementación técnica, donde valida el trabajo del alumno en los puntos más sensibles del proceso experimental. En el Sprint 3 revisa la implementación de las técnicas XAI (tareas 3.1 y 3.3, con una dedicación del 10% y del 8% respectivamente); en el Sprint 4 inspecciona el pipeline de entrenamiento convolucional (tarea 4.1, 3%); en el Sprint 5 valida el script de entrenamiento de los Transformers y el cálculo de las métricas cuantitativas (tareas 5.1 y 5.3, con un 3% y un 8%) y revisa los resultados junto al alumno (tarea 5.6, 25%); en el Sprint 6 revisa la comparación estadística (tarea 6.1, 8%); y en el Sprint 8 certifica los resultados del benchmarking final (tarea 8.1, 5%). Su dedicación total asciende a 5,96 horas, con un coste de 298 €.

Marc Ríos Cadenas interviene como asesor especialista en imagen médica. Su participación mantiene un perfil consultivo y se concentra en dos hitos que exigen criterio clínico: la revisión de la coherencia de los mapas de explicabilidad en la tarea 3.6, donde comprueba que los mapas apuntan a las regiones pulmonares relevantes y no a artefactos, y la validación de los resultados del benchmarking final en la tarea 8.1, donde certifica la coherencia diagnóstica de las explicaciones frente al criterio médico humano. Su dedicación, del 25% en la tarea 3.6 y del 5% en la tarea 8.1, asciende a 1,9 horas y a un coste de 95 €.

Vicente de Vides Rodríguez actúa como consultor de persistencia y bases de datos. Su intervención es puntual y estrictamente consultiva, concentrándose en la tarea 1.1, correspondiente al diseño del modelo de datos en MySQL. En ella aporta su experiencia estructural y verifica que el modelo de persistencia de la información resulta coherente con las exigencias de almacenamiento del sistema, sin asumir en ningún caso la implementación, que corresponde al alumno. Su dedicación del 15% sobre dicha tarea supone 1,5 horas y un coste de 75 €.

El resto de las partes interesadas identificadas en el capítulo de organización, como el **asesor de ingeniería del software y metodología**, participan únicamente a través de la matriz RACI, sin asumir tareas concretas ni horas asignadas en el cronograma del proyecto. En conjunto, el reparto de responsabilidades refleja una realidad propia de un trabajo académico unipersonal: una carga técnica casi íntegramente asumida por el alumno, complementada por intervenciones puntuales y de alto valor de los asesores, cuyos costes se consolidan en el apartado 5.3.

6. Identificación y Mitigación de Riesgos

Una actividad ineludible dentro de la planificación de cualquier proyecto radica en la gestión exhaustiva de los riesgos. Esta exigencia se multiplica críticamente cuando el sistema incorpora tecnologías de altísima complejidad técnica, como ocurre en este caso con el aprendizaje profundo, la inteligencia artificial explicable y la integración simultánea de múltiples arquitecturas algorítmicas. Se define formalmente como riesgo a cualquier evento latente que, de materializarse, impactaría negativamente sobre el alcance, la planificación temporal, la calidad técnica o el coste global del proyecto. El objetivo de este apartado no persigue la erradicación total de los riesgos, un escenario operativamente imposible en un desarrollo de esta envergadura. La meta consiste en identificar anticipadamente todas las amenazas, evaluando objetivamente su probabilidad y su impacto, para estructurar un plan de actuación sólido que reduzca las posibilidades de fallo y garantice una respuesta protocolizada y ordenada si terminan sucediendo.

Para cada una de las amenazas identificadas, la estrategia de defensa establece dos protocolos diferenciados: el plan de mitigación y el plan de contingencia. El plan de mitigación engloba las acciones estrictamente preventivas que se deben ejecutar antes de que el evento adverso ocurra, buscando asfixiar su probabilidad de aparición o reducir drásticamente su impacto. Por su parte, el plan de contingencia dicta las acciones reactivas y de contención que se despliegan una vez que el riesgo ya se ha materializado de manera inevitable, con el objetivo estricto de minimizar su efecto destructivo sobre el avance del proyecto.

6.1. *Escala de valoración de riesgos*

Antes de exponer el catálogo de amenazas identificadas, resulta imperativo establecer la métrica de evaluación. El sistema define una escala estricta para cuantificar la probabilidad, la severidad y la prioridad de cada riesgo, eliminando así cualquier margen de ambigüedad en su interpretación empírica.

La probabilidad mide la viabilidad de que la amenaza logre materializarse durante el ciclo de vida del proyecto. Se divide en tres umbrales:

- **Baja:** El escenario desencadenante resulta estadísticamente anómalo o se encuentra bajo un control técnico absoluto.
- **Media:** Existe una posibilidad empírica y real de que el evento adverso quiebre las defensas bajo condiciones operativas específicas.
- **Alta:** El vector de fallo representa una vulnerabilidad común, recurrente e inherente a los desarrollos de esta naturaleza tecnológica.

La severidad cuantifica el impacto destructivo sobre el sistema si el riesgo logra ejecutar su amenaza. Se estructuran cuatro niveles de daño:

- **Baja:** Desviación superficial. El ecosistema absorbe el impacto de manera orgánica sin exigir alteraciones significativas.
- **Media:** Alteración notable. El impacto penaliza el tiempo, el coste o la calidad del código, exigiendo maniobras tácticas de corrección para garantizar la continuidad operativa.
- **Alta:** Daño crítico. El evento adverso compromete de forma grave uno o varios pilares fundamentales del proyecto, forzando un replanteamiento estructural o temporal.
- **Muy Alta:** Colapso. La materialización del riesgo amenaza de muerte la viabilidad integral del proyecto o destruye por completo la validez matemática y clínica de los resultados.

Finalmente, la prioridad fusiona ambos vectores (probabilidad y severidad). Esta intersección determina el nivel de alerta y el rigor del blindaje que exige cada amenaza, cristalizando en la matriz de evaluación que se expone en la siguiente tabla:

Probabilidad \ Severidad	Baja	Media	Alta	Muy Alta
Baja	Baja	Baja	Media	Alta
Media	Baja	Media	Alta	Alta
Alta	Media	Alta	Alta	Muy Alta

Tabla x - Matriz de Prioridades

6.2. Catálogo de riesgos identificados

A continuación, se presentan los riesgos identificados para este proyecto, clasificados por su origen, evaluados según su probabilidad de materialización y la severidad de su impacto, y priorizados aplicando la matriz del apartado anterior. Se han detectado un total de nueve riesgos, que abarcan desde la gestión y planificación del proyecto hasta aspectos puramente tecnológicos, incluyendo la incompatibilidad de librerías, la disponibilidad de los conjuntos de datos médicos y el rendimiento de los modelos de aprendizaje profundo.

Id	Origen	Descripción	Probabilidad	Severidad	Prioridad
R01	Gestión	Planificación incorrecta de la duración, terminando el proyecto después de la fecha prevista.	Media	Media	Media
R02	Tecnológico	Incompatibilidades entre versiones de librerías de deep learning (TensorFlow, Keras, Transformers, CUDA Toolkit).	Alta	Alta	Alta
R03	Tecnológico	Dificultad en la integración de los módulos de entrenamiento Python con el backend FastAPI.	Media	Alta	Alta
R04	Gestión	Subestimación del tiempo de entrenamiento de los modelos de deep learning, especialmente en los Sprints de validación cruzada.	Alta	Alta	Alta
R05	Tecnológico	Dataset médico no disponible, inadecuado o con licencia incompatible para las pruebas de validación.	Media	Media	Media
R06	Tecnológico	Bloqueo o degradación de la interfaz web durante la ejecución prolongada de un entrenamiento.	Baja	Media	Baja
R07	Externo	Pérdida de código fuente por fallo del sistema de archivos o del equipo de desarrollo.	Baja	Muy Alta	Alta
R08	Infraestructura	Sobrecalentamiento o agotamiento de recursos de la GPU durante entrenamientos prolongados, provocando interrupciones.	Media	Alta	Alta
R09	Tecnológico	Falta de convergencia o rendimiento insuficiente de los modelos entrenados, invalidando los resultados del benchmarking.	Media	Alta	Alta

Tabla x - Riesgos Identificados

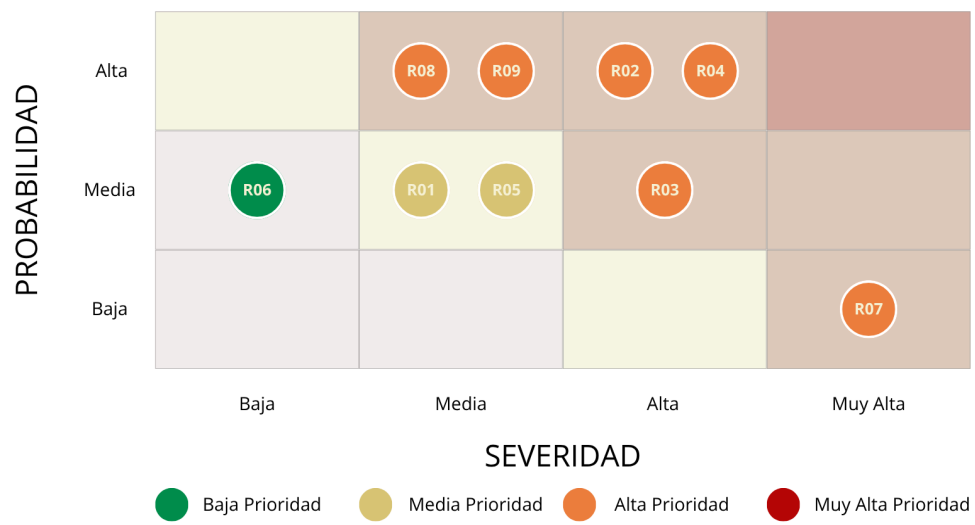


Ilustración x - Mapa de Prioridad

6.3. Plan de mitigación (medidas preventivas)

El plan de mitigación orquesta la barrera defensiva del proyecto. Reúne las medidas preventivas exigidas antes de que la amenaza logre materializarse, con el objetivo innegociable de asfixiar su probabilidad de ocurrencia o amortiguar drásticamente su impacto estructural.

Frente al riesgo de una planificación incorrecta (R01), el sistema impone hitos de control estrictos. El cierre de cada Sprint exige una revisión con el tutor para auditar el progreso real frente al teórico. Si el algoritmo de gestión detecta una desviación acumulada superior al quince por ciento, el cronograma sufre una reestructuración táctica inmediata antes de autorizar el avance hacia la siguiente iteración, garantizando un ajuste realista de la carga pendiente.

Para neutralizar las incompatibilidades entre librerías de aprendizaje profundo (R02), el código ancla las versiones exactas de todas las dependencias desde el minuto cero. El despliegue exige entornos virtuales de Anaconda herméticamente aislados para evitar la contaminación del sistema anfitrión. Adicionalmente, el Sprint 1 ejecuta pruebas de estrés de compatibilidad antes de permitir la construcción del motor de inferencia, erradicando los conflictos desde la base.

Ante la dificultad de integrar los módulos de Python con el backend FastAPI (R03), la arquitectura del Sprint 1 impone una separación quirúrgica entre la capa web y la computación científica. Los algoritmos se invocan como procesos autónomos. El Sprint 4 arranca con una prueba de concepto implacable, validando la fluidez de las comunicaciones entre la API y los scripts de entrenamiento antes de autorizar el desarrollo del resto de los endpoints.

Para combatir la subestimación del tiempo de entrenamiento (R04), el calendario impone una escalada táctica. El motor computa primero las arquitecturas más ligeras (MobileNetV2, EfficientNetB0) para calibrar el hardware, antes de liberar a los colosos pesados (ConvNeXt-Tiny, Transformer). El final del Sprint 4 marca un punto de control ineludible para recalcular las proyecciones temporales futuras basándose exclusivamente en el rendimiento empírico del silicio.

Respecto a la potencial indisponibilidad del conjunto de datos médicos (R05), el protocolo exige la extracción y descarga local de las radiografías durante el Sprint 1. El sistema audita el formato, la integridad estructural y los permisos de licencia. Mantener el volumen de datos anclado en el disco duro local blindo al proyecto frente a caídas de red o alteraciones imprevistas en los repositorios externos durante todo el ciclo de desarrollo.

Para impedir el bloqueo de la interfaz durante los masivos ciclos de cálculo (R06), el diseño estructural asume un paradigma estrictamente asíncrono. FastAPI delega el procesamiento pesado a tareas en segundo plano. El sistema inyecta mecanismos de sondeo continuo y despliega telemetría de progreso en tiempo real sobre el panel de control, desterrando cualquier riesgo de congelación gráfica desde la fase de diseño.

Frente a la amenaza catastrófica de pérdida de código fuente (R07), el protocolo despliega un repositorio privado en GitHub desde el primer día. La normativa impone la ejecución de un commit descriptivo al clausurar cada jornada de trabajo, sin excepciones. Esta red de seguridad se refuerza mediante el volcado sistemático de una copia de seguridad semanal en un hardware de almacenamiento externo.

Para evitar el colapso térmico o el estrangulamiento de la GPU (R08), el motor de ejecución inyecta periodos de latencia programados entre las distintas arquitecturas para disipar el calor del hardware. Herramientas de telemetría auditan la temperatura del procesador gráfico en tiempo real. Las arquitecturas más exigentes quedan relegadas a ventanas temporales de ejecución bajo supervisión humana estricta y activa.

Finalmente, para mitigar el fracaso en la convergencia matemática de los modelos (R09), los sprints de entrenamiento despliegan un sistema de alerta temprana: el Sprint 4 para las arquitecturas CNN y el Sprint 5 para los Transformers. El código monitoriza las curvas de pérdida desde las primeras épocas y utiliza parada temprana y reducción de la tasa de aprendizaje cuando detecta estancamiento. Como línea de base, los algoritmos arrancan con configuraciones de hiperparámetros documentadas y revisadas durante las primeras ejecuciones, que se ajustan cuando los resultados experimentales evidencian problemas de convergencia.

6.4. Plan de contingencia (medidas correctivas)

El plan de contingencia actúa como la última línea de defensa. Despliega maniobras reactivas y de contención una vez que la amenaza ha logrado fracturar las medidas preventivas, con el objetivo innegociable de minimizar el daño estructural y garantizar la supervivencia del proyecto.

R01 – Desviación temporal consolidada: Si el cronograma colapsa, el sistema aplica una poda táctica. Se ejecuta una revisión inmediata de la ruta crítica, amputando el alcance de los elementos no esenciales (variantes accesorias de Transformer o florituras gráficas de la interfaz). El objetivo es salvaguardar la entrega de un núcleo plenamente funcional. Esta reducción del alcance se documentará y justificará explícitamente en la memoria.

R02 – Conflicto estructural de librerías: En el caso de que se produzca un conflicto de incompatibilidad entre versiones de librerías (R02), el protocolo de actuación se estructura en tres niveles. En primer lugar, se aísla la dependencia conflictiva y se consulta la documentación oficial de las librerías implicadas para identificar la combinación de versiones compatible, estableciendo un plazo máximo de cuarenta y ocho horas para localizar la solución. En segundo lugar, la resolución del conflicto se apoya en el uso de entornos virtuales de Anaconda: se crea un entorno conda aislado en el que se instalan las versiones exactas de las dependencias, de modo que la combinación compatible se despliega y valida en ese entorno sin contaminar el resto de instalaciones de la máquina anfitriona, garantizando además la reproducibilidad de los experimentos. En tercer lugar, el despliegue del sistema se apoya en la estrategia de exposición del servicio: vitalXAI se publica a través de un túnel de Cloudflare que expone el servicio alojado en el servidor local, de modo que el entorno de ejecución (versiones de librerías, configuración del toolkit de CUDA y entorno Python) queda confinado y validado en la máquina de despliegue antes de que el servicio se publique a través del túnel. Este diseño garantiza que la versión del sistema que llega a los usuarios se ejecute siempre sobre un entorno controlado y coherente, permitiendo corregir y verificar cualquier incompatibilidad de versiones en la máquina anfitriona con total trazabilidad antes de exponer el servicio.

R03 – Fractura en la integración del backend: Si el acoplamiento orgánico fracasa, el diseño adopta una arquitectura de trinchera. Los scripts de entrenamiento se encapsulan herméticamente como procesos independientes, invocados desde FastAPI mediante el módulo de subprocessos (subprocess). Esta cirugía informática desacopla por completo la carga computacional de la petición web, permitiendo validar la comunicación capa por capa.

R04 – Estrangulamiento del tiempo computacional: Si el silicio no cumple las proyecciones de velocidad, el alcance del benchmarking sufre un recorte quirúrgico. La evaluación masiva se limitará a un subconjunto representativo (por ejemplo, dos arquitecturas CNN y un Transformer), salvaguardando la demostración de viabilidad del concepto. Las arquitecturas restantes quedarán relegadas a trabajo futuro, dejando constancia de esta limitación técnica en la memoria.

R05 – Incompatibilidad sobrevenida del dataset: Ante la corrupción o inutilidad de los datos anclados en las pruebas de validación, la contingencia no puede consistir en un cambio automático de conjunto de datos. Sustituir una cohorte médica por otra altera el protocolo clínico y de validación del estudio: un conjunto de datos distinto introduce variaciones en la población, el equipo de adquisición y los criterios de etiquetado, por lo que cualquier resultado obtenido sin volver a ejecutar el protocolo completo carecería de validez científica. Por ello, la actuación consiste en identificar un conjunto de datos alternativo que cumpla los requisitos del estudio —disponibilidad, anonimización, licencia compatible y coherencia con el problema clínico— y, en caso de adoptarse, reejecutar de forma íntegra el pipeline de entrenamiento, la validación cruzada, la validación externa y los análisis de explicabilidad sobre la nueva cohorte, con la trazabilidad y la reproducibilidad exigidas en la metodología. Cualquier cambio de dataset y sus implicaciones sobre la interpretación de los resultados se documentarán de forma explícita y rigurosa en la memoria, de modo que el estudio no presente resultados que no provengan de un protocolo íntegramente validado.

R06 – Congelación de la interfaz gráfica: Si la sobrecarga algorítmica logra bloquear el frontend durante las pruebas de integración, la implementación de colas asíncronas asume máxima e inmediata prioridad. Si el reloj del Sprint impide una solución estructural definitiva, el código inyectará un parche preventivo: un panel de advertencia informando al clínico sobre el tiempo estimado de espera y habilitando la consulta del estado en diferido.

R07 – Destrucción física del código fuente: Ante un fallo catastrófico del hardware del equipo, el protocolo de recuperación extrae de urgencia el último estado seguro desde el repositorio privado en GitHub hacia una máquina de soporte. Tras restablecer el entorno de trabajo, se cuantifica el volumen del código evaporado y se reestructura internamente el Sprint en curso para absorber el impacto sin alterar la fecha de entrega.

R08 – Colapso térmico de la GPU: El sistema ordena una interrupción instantánea del entrenamiento para purgar el calor del procesador gráfico y salvar su integridad. Los ciclos computacionales masivos se fragmentarán en bloques más pequeños con pausas de refrigeración obligatorias. Si la asfixia del hardware persiste, el algoritmo ejecutará un recorte drástico sobre el tamaño de los lotes (batch size) o el límite de épocas.

R09 – Fracaso en la convergencia matemática: El algoritmo entra en cuarentena investigativa. El desarrollador audita de forma sistemática las tasas de aprendizaje, las capas de preprocesamiento y la hermeticidad de las fronteras de validación. Si el modelo se niega a aprender tras múltiples ajustes, será reemplazado por una arquitectura de solvencia probada en la literatura. Este colapso predictivo se documentará como un hallazgo científico de alto valor sobre la aplicabilidad de dicho modelo en el dominio médico.

7. Planes de apoyo a la gestión del proyecto

Este capítulo reúne los planes de apoyo que complementan el marco de gestión descrito en el capítulo 4. Más allá de la metodología y de la organización del trabajo, un proyecto necesita mecanismos concretos que ordenen tres dimensiones transversales: cómo se comunica el equipo, cómo se garantiza la calidad del producto y del proceso, y cómo se gestiona el código fuente y sus entregables. El capítulo se estructura en tres apartados que desarrollan cada una de estas dimensiones: el plan de comunicación, el plan de calidad y el plan de gestión de la configuración.

7.1. Plan de Comunicación

La comunicación del proyecto se ancla en la cadencia adaptada del marco Scrum y en las particularidades de un desarrollo unipersonal. La planificación y la revisión de cada iteración se realizan mediante el tablero y reuniones de seguimiento programadas según la fase y la disponibilidad del tutor, no mediante dieciocho ceremonias formales con dedicación presupuestada. En la práctica, esta cadencia se materializa en un par de reuniones durante la planificación inicial, reuniones más frecuentes durante el desarrollo activo y reuniones puntuales durante la fase de documentación. Las horas imputadas al tutor se concentran en las tareas 0.1 y 8.3; el resto de las comunicaciones y revisiones puntuales no se contabiliza como una dedicación adicional en el apartado de costes.

Al margen de estos hitos formales, la sincronización operativa fluye de forma asíncrona mediante el tablero Kanban, que proyecta en tiempo real el estado del desarrollo, y el correo electrónico para la gestión logística. Las consultas técnicas con los asesores exigen reuniones quirúrgicas, convocadas exclusivamente cuando la complejidad del avance lo demande, destacando las fases críticas de integración de módulos XAI y la construcción de los pipelines algorítmicos.

7.2. Plan de Calidad

El aseguramiento de la calidad del sistema se blindo mediante cuatro líneas de actuación innegociables.

A nivel de testing, el código se somete a pruebas unitarias sobre los módulos de procesamiento y predicción; pruebas de integración sobre el flujo diagnóstico completo (ingesta de imagen, inferencia, síntesis XAI y exportación PDF); pruebas de seguridad sobre las barreras de autenticación y acceso; y pruebas de rendimiento para auditar que la carga computacional de los entrenamientos respeta los márgenes temporales exigidos. El desarrollo se apoya además en el modelo de desarrollo basado en especificaciones y guiado por pruebas (SDD/TDD) descrito en el capítulo 4, de modo que cada funcionalidad queda cubierta por pruebas automatizadas antes de considerarse completada.

Respecto a los estándares estructurales, todo el ecosistema Python acata implacablemente la normativa PEP 8, auditada mediante herramientas de análisis estático. Los scripts de entrenamiento y el backend imponen una separación arquitectónica estricta, blindando sus funciones con docstrings que documentan su propósito operativo, parámetros de entrada y tipología de retorno.

En el vector de la automatización, el repositorio despliega un flujo de integración continua (CI) impulsado por GitHub Actions. Este motor intercepta cada Pull Request para ejecutar de manera automática los tests con pytest, auditar el estilo con ruff y comprobar que la cobertura del código no desciende por debajo del umbral mínimo establecido, bloqueando inexorablemente cualquier código defectuoso antes de que logre fusionarse con la rama principal.

Finalmente, para mejorar la reproducibilidad de los experimentos frente a la variabilidad debida a la inicialización aleatoria, el código fija semillas en las librerías críticas (TensorFlow, NumPy y Python) y almacena las configuraciones de cada experimento en archivos JSON vinculados a la sesión de entrenamiento. Estas medidas facilitan la repetición y la trazabilidad de los resultados, pero no garantizan por sí solas un determinismo bit a bit: las operaciones concurrentes de la GPU, las versiones del software y otros factores del entorno pueden introducir pequeñas diferencias entre ejecuciones. Por ello, los resultados deben reproducirse bajo las mismas condiciones documentadas y compararse atendiendo a la variabilidad observada.

7.3. *Plan de Gestión de la Configuración*

El código fuente del proyecto se confina en un repositorio privado alojado en GitHub. La arquitectura de control de versiones adopta el estándar GitHub Flow. Este paradigma protege una rama principal (main) que custodia exclusivamente código estable y desplegable. Cada nueva funcionalidad exige la creación de una rama efímera con nomenclatura descriptiva (por ejemplo, feature/xai-quantitative-metrics o feature/training-pipeline-cnn). La inyección de código hacia main se ejecuta obligatoriamente mediante Pull Requests, incluso operando en solitario, forzando así el disparo automático de las validaciones de GitHub Actions previas a la integración. Como red de seguridad adicional, se realiza un commit descriptivo al final de cada jornada de trabajo y se mantiene una copia de seguridad semanal del repositorio en un almacenamiento externo, en línea con lo establecido en el plan de riesgos del proyecto.

El repositorio integra un archivo .gitignore configurado quirúrgicamente para vetar el rastreo de elementos ajenos al código fuente: los conjuntos de datos radiológicos, los pesos sinápticos de los modelos entrenados (archivos .keras, .h5), los artefactos dinámicos generados (radiografías subidas, PDFs, mapas de calor) y los residuos del entorno de desarrollo (cachés de Python, entornos virtuales, configuraciones locales). Esta barrera mantiene el repositorio ágil y hermético, bloqueando la exposición de datos sensibles y garantizando que solo se almacene lo necesario para replicar el ecosistema.

El versionado del software acata la norma Semantic Versioning (SemVer), estructurando las entregas bajo la sintaxis MAJOR.MINOR.PATCH. El cierre de cada Sprint cristaliza en la generación de etiquetas de Git (por ejemplo, v1.3.0), permitiendo identificar de forma unívoca y auditable el estado exacto del código en cada hito temporal de la planificación.

8. Cuestiones abiertas y decisiones adoptadas

A fecha de elaboración del presente documento, los debates técnicos han quedado definitivamente cerrados. Tras la evaluación empírica de las alternativas, se exponen a continuación las decisiones consolidadas que fijan el rumbo innegociable del proyecto:

Dataset de entrenamiento: Este documento concluye el tema. El sistema utilizará el conjunto de datos público de radiografías de tórax pediátricas publicado por Kermay y colaboradores, procedente del Guangzhou Women and Children's Medical Center y disponible en Kaggle. Este dataset contiene 5.856 radiografías en formato JPEG, clasificadas en dos categorías: NORMAL (1.583 imágenes) y PNEUMONIA (4.273 imágenes), estas últimas subdivididas en neumonía bacteriana y viral. Se ha seleccionado este conjunto por ser el mismo utilizado en el estudio de referencia (Kermay & al., 2018), lo que permite contextualizar los resultados del proyecto respecto a un trabajo publicado que emplea los mismos datos, sin asumir que ambos estudios utilizan el mismo protocolo experimental o comparan las mismas arquitecturas. El desbalanceo natural entre clases se abordará mediante la técnica de submuestreo aleatorio en cada pliegue de la validación cruzada, garantizando particiones balanceadas. Durante la fase de desarrollo y depuración del código se han empleado subconjuntos reducidos de este dataset para agilizar las pruebas, pero la metodología del proyecto está concebida para trabajar con el conjunto original completo.

Dataset de validación externa: Este documento concluye el tema. Se empleará el conjunto de datos COVID-19 Radiography Database, desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka, que contiene radiografías de tórax de pacientes adultos. Para la validación externa se extraerán 500 imágenes de pacientes sanos y 500 imágenes de pacientes con neumonía viral, garantizando un balance perfecto entre clases. Este dataset ha sido seleccionado porque representa una población diferente (adultos frente a niños), con equipos de adquisición y protocolos de imagen distintos, lo que constituye un escenario exigente para evaluar la capacidad de generalización de los modelos. Al igual que con el dataset de entrenamiento, durante las pruebas de desarrollo se han utilizado subconjuntos reducidos del conjunto externo.

Arquitecturas de deep learning: Este documento concluye el tema. El banco de pruebas estará compuesto por dieciséis arquitecturas convolucionales (ResNet50, ResNet101, ResNet152, ResNet50V2, EfficientNetB0, EfficientNetB3, EfficientNetB7, EfficientNetV2S, DenseNet121, DenseNet201, MobileNetV2, MobileNetV3Large, VGG16, InceptionV3, Xception y ConvNeXtTiny) y tres arquitecturas Transformer (DeiT, Swin-Base y ViT-384). Esta selección cubre un espectro representativo de enfoques arquitectónicos, desde modelos ligeros como MobileNetV2 hasta modelos de alta capacidad como ConvNeXt-Tiny y los Vision Transformers, y permite comparar de forma sistemática el paradigma de las redes convolucionales con el de los mecanismos de atención.

Técnicas XAI: Este documento concluye el tema. Se implementarán cuatro técnicas de inteligencia artificial explicable: Saliency Maps, SmoothGrad y Grad-CAM para arquitecturas convolucionales, y mapas de atención para arquitecturas Transformer. Para la evaluación cuantitativa de las explicaciones se calcularán las métricas Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM. Adicionalmente, se incorporará la métrica Expected Calibration Error (ECE) para evaluar la calibración de las predicciones. Las cuatro técnicas visuales están disponibles en el módulo de explicabilidad de la vía de diagnóstico de la aplicación web. En la vía de laboratorio MLOps, los scripts generan los mapas cualitativos previstos para el análisis experimental —Grad-CAM y mapas de saliencia para las CNN, y mapas de saliencia para los Transformers— y calculan las métricas cuantitativas y de calibración.

Tecnologías del sistema: Este documento concluye el tema. La pila tecnológica queda definida en detalle en este apartado, estructurándola por capas, con el fin de justificar cada elección sobre la base de las tecnologías realmente empleadas en el desarrollo.

En cuanto al **backend**, se ha optado por Python 3.11 como lenguaje principal, dado su ecosistema maduro en el ámbito del aprendizaje profundo y su amplia adopción en la comunidad de investigación médica. Sobre él se asienta el framework FastAPI, un marco ASGI moderno y de alto rendimiento que aprovecha las anotaciones de tipos de Python para generar de forma automática la validación de los datos de entrada y la serialización de las respuestas en JSON mediante Pydantic (versión 2). El servidor se ejecuta bajo Uvicorn, el servidor ASGI de referencia, que atiende las peticiones de forma asíncrona y eficiente. El renderizado de las páginas web se realiza en el servidor con Jinja2, el motor de plantillas estándar de Python, que permite componer las páginas HTML de forma dinámica incorporando los datos del sistema. La capa de persistencia se apoya en la base de datos relacional MySQL (MariaDB en el entorno de desarrollo mediante XAMPP), accedida a través del conector nativo mysql-connector-python, que gestiona un pool de conexiones para soportar el acceso concurrente de múltiples usuarios. El esquema relacional almacena los usuarios, las consultas de diagnóstico, las sesiones de entrenamiento, la cola de trabajos y los tokens de refresco de sesión.

En cuanto a la **seguridad**, el sistema incorpora una capa de protección transversal. Las contraseñas se almacenan aplicando el algoritmo de hash bcrypt, que incluye un factor de coste configurable. La autenticación se gestiona mediante tokens JWT (generados con la librería python-jose, algoritmo HS256) almacenados en cookies httpOnly, complementados con tokens de refresco opacos persistidos en la base de datos y sometidos a rotación en cada uso, con detección de su reutilización para revocar la sesión. Las peticiones que modifican estado están protegidas por un middleware de CSRF de doble cookie. Se aplica además limitación de peticiones con la librería slowapi (un límite general y un límite específico y más estricto sobre el inicio de sesión) y un middleware que inyecta cabeceras de seguridad (Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, entre otras).

En cuanto al **frontend**, la interfaz se construye con HTML5, CSS y JavaScript vanilla, descartándose de forma deliberada los frameworks de componentes como React, Vue o Angular. La decisión no ignora las ventajas de estos frameworks, sino que responde a la adecuación al proyecto: la aplicación no requiere un estado de interfaz altamente reactivo ni componentes complejos, y el renderizado en el servidor con Jinja2 permite entregar una interfaz accesible y mantenible sin introducir una cadena de herramientas de compilación (npm, webpack o similares) que añadiría complejidad operativa. Las páginas de la aplicación —login, registro, panel de diagnóstico, laboratorio de entrenamiento y panel de administración— se componen en el servidor a partir de plantillas Jinja2 que inyectan dinámicamente los datos del sistema y los textos de la interfaz. El estilo visual se apoya en Tailwind CSS, cargado mediante CDN y configurado con soporte de modo oscuro mediante la estrategia de clase (`darkMode: 'class'`), lo que permite alternar entre el tema claro y el oscuro de forma dinámica, y en la librería de iconos Font Awesome (versión 6.4), también cargada por CDN. El uso directo de las utilidades de Tailwind en las plantillas, junto con un pequeño conjunto de estilos propios, elimina la necesidad de escribir hojas de estilo extensas y mantiene la coherencia visual entre las distintas pantallas, incluyendo las tablas de resultados, los gráficos y los visores de imágenes.

La **interactividad** se implementa en JavaScript nativo (ES6), organizado en módulos por ámbito funcional: `dashboard.js` para el panel de diagnóstico, `training.js` para el laboratorio de entrenamiento, `admin.js` para la administración e `i18n.js`, compartido, para la internacionalización. La comunicación con la API se realiza mediante `fetch`, enviando los formularios de forma asíncrona con `FormData` para no recargar la página; un interceptor global de peticiones añade automáticamente la cabecera de protección CSRF (X-CSRF-Token) y gestiona la renovación automática de la sesión cuando el token de acceso expira, reintentando la petición original. El estado de los procesos en segundo plano se actualiza mediante polling periódico del panel de la cola de trabajos, y las imágenes de resultados pueden inspeccionarse a tamaño completo mediante un visor modal. La internacionalización se apoya en atributos `data-i18n` distribuidos en las plantillas y en un diccionario de traducciones en JavaScript que cubre cuatro idiomas (español, inglés, chino e hindú), de modo que al cambiar el idioma la interfaz se re-renderiza al instante, incluyendo los informes y el asistente conversacional.

En cuanto al **motor de inteligencia artificial**, el proyecto utiliza TensorFlow (versión 2.18) y Keras (versión 3) para la construcción, el entrenamiento y la inferencia de las arquitecturas convolucionales. Las arquitecturas se cargan dinámicamente desde `tf.keras.applications` con pesos preentrenados en ImageNet, aplicando transferencia de conocimiento: la base del modelo se mantiene congelada y se sustituye el cabezal de clasificación por uno propio adaptado al problema binario de la neumonía. La librería `tf_keras` se incorpora para garantizar la compatibilidad con los pesos guardados en formatos heredados de Keras. Los modelos Transformer (DeiT, Swin-Base y ViT-384) se integran mediante HuggingFace Transformers, empleando la clase `TFAutoModelForImageClassification` para su entrenamiento y carga en el backend. El preprocesamiento de las imágenes se realiza con OpenCV y con las utilidades de datos de TensorFlow.

En cuanto al **análisis de datos y a la generación de informes**, el sistema utiliza pandas y NumPy para la gestión y el procesamiento de los datos, scikit-learn para las métricas de rendimiento, la validación cruzada estratificada y las curvas ROC, y SciPy para el test estadístico de Wilcoxon; el test de DeLong para la comparación de curvas ROC se implementa de forma propia en el pipeline de validación externa. Las representaciones gráficas —curvas ROC, matrices de calor de significación, rankings y mapas de explicabilidad— se generan con matplotlib y seaborn. Los informes descargables en formato PDF se generan con la librería fpdf2, tanto para los diagnósticos individuales como para los informes completos de las sesiones de entrenamiento.

En cuanto a la **inteligencia conversacional**, la plataforma se conecta a la API de Groq, delegando la inferencia lingüística al modelo Llama 3 de Meta (concretamente la variante llama-3.3-70b-versatile). El asistente conversacional se configura mediante técnicas de prompt engineering, definiendo un prompt de sistema que fuerza la devolución de una salida JSON estructurada con la configuración de los experimentos, y se encuentra internacionalizado en los cuatro idiomas de la plataforma. La elección de Groq responde a la velocidad de inferencia de sus unidades de procesamiento de lenguaje (LPU) y a su disponibilidad gratuita, adecuada para un proyecto académico.

Por último, en cuanto al **despliegue**, el sistema se publica exponiendo el servidor local a través de un túnel de Cloudflare, de modo que el entorno de ejecución queda confinado y validado en la máquina anfitriona antes de que el servicio se ponga a disposición de los usuarios a través del túnel, garantizando la coherencia del entorno de librerías en ejecución.

9. Consideraciones complementarias del proyecto

Las consideraciones éticas y de privacidad de los datos médicos es uno de los aspectos que se debe tener en cuenta, dado que el proyecto utiliza radiografías de tórax de origen humano para el entrenamiento y evaluación de los modelos de inteligencia artificial. Se ha adoptado una política estricta de uso de datos que garantiza el cumplimiento de la normativa vigente.

Todos los datasets que se han utilizado para el entrenamiento y la validación externa proceden únicamente de repositorios públicos y anonimizados. El dataset de Kermany y colaboradores, utilizado para el entrenamiento, fue originalmente obtenido del Guangzhou Women and Children's Medical Center y posteriormente publicado en Kaggle con licencia open data, con todas las imágenes desprovistas de cualquier información de identificación personal. El dataset COVID-19 Radiography Database, utilizado para la validación externa, ha sido igualmente anonimizado por sus autores antes de su publicación. Este enfoque garantiza el cumplimiento del Reglamento General de Protección de Datos (RGPD) y evita cualquier implicación ética derivada del manejo de historiales clínicos o datos sensibles de pacientes.

El sistema desarrollado no almacena ni procesa datos de pacientes individuales identificables. Las radiografías que los facultativos suben a la plataforma para su diagnóstico se almacenan de manera temporal en el servidor únicamente para su procesamiento, y los archivos generados (mapas XAI, informes PDF) quedan asociados a la cuenta del profesional que realizó la consulta, sin exponer información del paciente en la interfaz. Es responsabilidad del personal clínico garantizar que las imágenes que introducen en el sistema han sido previamente anonimizadas y que cumplen con la normativa de protección de datos de su centro sanitario.

Es importante señalar que vitalXAI se concibe como una herramienta de ayuda al diagnóstico, no como un sustituto del criterio clínico. Las predicciones generadas por los modelos de inteligencia artificial deben ser siempre evaluadas por un facultativo cualificado antes de tomar cualquier decisión clínica. El sistema proporciona mapas de explicabilidad para facilitar esta evaluación, pero la responsabilidad última del diagnóstico recae en el profesional sanitario. Esta limitación se indica de manera explícita en la interfaz de la plataforma y en los informes PDF generados.

Por último, desde el punto de vista medioambiental, el entrenamiento de modelos de deep learning conlleva un consumo energético que debe ser considerado. En este proyecto, los entrenamientos se han realizado preferentemente en un equipo local con GPU, limitando el número de épocas y modelos a los estrictamente necesarios para obtener conclusiones significativas. La preferencia por la computación local responde a criterios de coste y de disponibilidad de la infraestructura. No se ha imputado al presupuesto una medición específica de electricidad ni existe una reserva económica para cloud; si el hardware local dejara de estar disponible, el uso de infraestructura en la nube requeriría una decisión adicional de coste o de alcance y quedaría documentado explícitamente. En cualquier caso, la contención en el número de experimentos y en su coste computacional contribuye a reducir la huella de carbono asociada al desarrollo del sistema.

ANEXO II – PLANTILLA DOCUMENTO ANALISIS

10. Planteamiento del análisis del sistema

Para que un sistema se diseñe e implemente con garantías no basta con una idea general: es necesario formalizar el análisis, es decir, convertir la visión del proyecto en un conjunto de afirmaciones verificables sobre qué debe hacer el sistema, bajo qué restricciones y para qué usuarios. Esta parte de la memoria se ocupa precisamente de eso, y este capítulo abre el análisis presentando su propósito y sus dimensiones.

10.1. *Propósito del análisis*

El documento de análisis fija de manera explícita y sin ambigüedades las bases sobre las que se construirá el sistema. Su utilidad es doble. Por un lado, evita que durante el diseño y la implementación se asuman capacidades que no se planificaron o se omitan requisitos que sí se acordaron. Por otro lado, documenta las decisiones de alcance y las restricciones del entorno, de modo que cualquier lector —el tutor, el tribunal o un futuro mantenedor— pueda entender por qué el sistema tiene la forma que tiene y cuáles de sus límites son deliberados y cuáles responden a restricciones técnicas.

En este proyecto, la formalización del análisis es especialmente relevante porque el sistema aúna dos naturalezas: una aplicación web de uso clínico y un laboratorio de investigación de aprendizaje automático. La primera impone requisitos de claridad, seguridad y facilidad de uso; la segunda impone requisitos de flexibilidad, reproducibilidad y capacidad de ejecutar experimentos completos. Formalizar el análisis permite reconciliar ambas perspectivas desde el inicio y evitar que una de las dos naturalezas acabe imponiéndose sobre la otra sin que esa decisión quede documentada.

10.2. *Dimensiones del análisis*

El análisis del sistema comprende, en primer lugar, la delimitación del contexto en el que se inscribe la plataforma. Esta delimitación se apoya en cuatro dimensiones: el alcance funcional del sistema, que fija qué capacidades se compromete a entregar y cuáles quedan deliberadamente fuera de su ámbito de responsabilidad; el entorno tecnológico, que describe el ecosistema sobre el que se construye la plataforma y las restricciones que impone al diseño; la normativa y los estándares que el sistema debe observar; y la caracterización de los perfiles de usuario que interactuarán con él, atendiendo a sus conocimientos, sus necesidades y su nivel de acceso. Estas cuatro dimensiones, junto con los objetivos del sistema que las enmarcan, se desarrollan íntegramente en el capítulo 2.

Sobre esa base, el análisis especifica qué debe hacer el sistema y cómo se organiza internamente para lograrlo, y es esta segunda parte la que constituye el grueso del análisis. Se estructura en los capítulos 3 a 7:

- **Especificación de requisitos y casos de uso (capítulo 3).** Se definen los requisitos funcionales y no funcionales del sistema y los casos de uso que describen la interacción de cada perfil de usuario con la plataforma.
- **Identificación de subsistemas (capítulo 4).** Las capacidades del sistema se agrupan en subsistemas de análisis, lo que permite organizar el resto del análisis y, posteriormente, el diseño.
- **Modelo de dominio (capítulo 5).** Se identifican las entidades que el sistema debe gestionar y se describe su comportamiento dinámico mediante secuencias de interacción.
- **Verificación de la consistencia (capítulo 6).** Se comprueba la coherencia entre los objetivos, los requisitos, los casos de uso y los subsistemas.
- **Plan de pruebas (capítulo 7).** Se define la estrategia de pruebas que verificará el cumplimiento de lo especificado.

11. Definición del ámbito del sistema

El contexto y el estado del arte en el que se inscribe vitalXAI se presentaron en el capítulo 1, donde se describieron el problema de la neumonía, la radiografía de tórax como imagen digital, la evolución del aprendizaje profundo aplicado al diagnóstico por imagen, los conjuntos de datos y los modelos de referencia, y las plataformas MLOps existentes. Este capítulo no repite ese análisis: asume lo allí expuesto y se centra en delimitar el ámbito del sistema, es decir, qué capacidades incluye vitalXAI como producto, bajo qué entorno tecnológico, con qué normativa y para qué usuarios.

vitalXAI pretende ocupar el espacio que los sistemas existentes no cubren de forma conjunta: una plataforma web que reúna el diagnóstico asistido con inteligencia artificial explicable, un laboratorio de entrenamiento reproducible accesible sin escribir código y una validación estadística rigurosa de los resultados. El sistema combina la potencia de los modelos de investigación, la capacidad de gestión de las plataformas MLOps y la sencillez de uso de los productos comerciales, con la transparencia que estos últimos no ofrecen.

La elección de una plataforma web es deliberada. Para el usuario final, la plataforma elimina cualquier requisito de instalación: el acceso se realiza desde un navegador, sin dependencia de un sistema operativo concreto ni de un equipo determinado, lo que descarta los desarrollos móviles o de escritorio, que exigirían instalar y mantener una aplicación en cada terminal. Conviene precisar, no obstante, que esa ausencia de requisitos técnicos se refiere al cliente y no al sistema en su conjunto. La operación de vitalXAI impone exigencias propias de infraestructura: el entrenamiento requiere una estación de trabajo con GPU NVIDIA compatible con CUDA, la publicación del servicio depende de un túnel de Cloudflare y ciertas operaciones del laboratorio, como la selección de la carpeta del dataset (RF-019), se realizan sobre el sistema de ficheros del servidor. Estos requisitos, que se detallan en la sección 11.1, recaen sobre el entorno que despliega y mantiene la plataforma, mientras que el profesional que la utiliza solo necesita un navegador.

Esta decisión condiciona de forma notable el alcance del sistema y la arquitectura que debe adoptarse, porque toda la complejidad computacional —los modelos, el entrenamiento, el almacenamiento de los datos— queda confinada en el servidor. La plataforma se estructura, así, en dos caras complementarias: la que ve el usuario, pensada para ser lo más simple posible, y la que no ve, donde residen los motores de cálculo.

Desde el punto de vista funcional, el sistema se organiza en torno a dos núcleos principales y a varias capacidades de apoyo, y conviene anticiparlos aquí porque constituyen el esqueleto sobre el que se definen los objetivos y los requisitos. El primer núcleo es la interfaz clínica de diagnóstico: permite al facultativo cargar una radiografía, seleccionar la arquitectura con la que desea realizar la consulta, obtener un diagnóstico con su nivel de confianza, visualizar los mapas de explicabilidad que justifican la predicción y descargar un informe en PDF, manteniendo además un historial de consultas aislado por usuario que el profesional puede consultar, renombrar o depurar cuando lo necesite. El segundo núcleo es el laboratorio MLOps: permite al investigador configurar y lanzar experimentos de entrenamiento mediante un asistente conversacional, monitorizar su progreso en tiempo real y consultar los resultados comparativos y estadísticos, incluyendo el ranking de modelos, las matrices de significación y los análisis de explicabilidad y calibración.

Junto a estos dos núcleos, el sistema incorpora una serie de capacidades de apoyo que lo hacen operativo en condiciones reales de uso. La ejecución asíncrona de las tareas largas garantiza que la interfaz no se bloquee durante los entrenamientos, que pueden prolongarse durante horas. El soporte multilingüe amplía la accesibilidad de la plataforma a usuarios de distintas procedencias. El panel de administración permite a un usuario con rol de administrador gestionar las cuentas y supervisar la actividad del sistema. Y la capa de autenticación y seguridad protege los datos de cada usuario de forma transversal. Para formalizar este alcance, en primer lugar se definen los objetivos del sistema; a continuación se caracterizan el entorno tecnológico, la normativa aplicable y los perfiles de usuario que interactuarán con la plataforma.

11.1. Objetivos del sistema

Los objetivos del sistema constituyen la declaración formal de lo que vitalXAI debe conseguir como producto. Se obtienen a partir de las reuniones mantenidas con el tutor y de los objetivos generales del proyecto, y sirven de anclaje para los requisitos que se definen en los capítulos siguientes del análisis. Cada objetivo se describe mediante una ficha en la que se recogen su identificador, su nombre, la descripción de lo que debe lograr el sistema, su importancia dentro del conjunto y el estado de la decisión. Conviene distinguir estos objetivos de los objetivos del plan de proyecto presentados en la primera parte de la memoria: aquellos describen lo que el alumno debe conseguir durante el desarrollo; estos describen lo que el sistema debe ofrecer como producto final, con independencia de cómo se implemente. Para que el lector pueda comprobar que el producto especificado cubre los compromisos adquiridos en el plan de proyecto, la tabla 15, presentada al final de esta sección, establece la correspondencia entre cada objetivo del sistema y los objetivos específicos del plan que lo sustentan.

OBJ-001 – Diagnóstico asistido con inteligencia artificial explicable

El diagnóstico asistido es el núcleo de la plataforma y el punto de partida de todo el sistema. Un profesional sanitario, sin conocimientos de inteligencia artificial, debe poder cargar una radiografía de tórax y obtener, en un tiempo razonable, un diagnóstico con su nivel de confianza y una explicación visual de los motivos que sustentan la decisión del modelo. La confianza es importante porque permite al facultativo calibrar la fiabilidad de la predicción, y la explicación visual es importante porque le permite comprobar si el modelo se está fijando en las regiones pulmonares correctas. Esta ficha fija ese compromiso.

Campo	Contenido
ID	OBJ-001
Nombre	Diagnóstico asistido con inteligencia artificial explicable
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe permitir al profesional sanitario cargar una radiografía de tórax, seleccionar una arquitectura de deep learning y obtener un diagnóstico con su nivel de confianza, acompañado de los mapas de explicabilidad que justifican la predicción.
Importancia	Alta
Estado	Aprobado
Comentarios	Este objetivo recoge el núcleo clínico del sistema. El mecanismo concreto de generación de las explicaciones corresponde al ámbito del diseño.

Tabla x - OBJ-001: Diagnóstico asistido con inteligencia artificial explicable

OBJ-002 – Gestión del historial de consultas

Un facultativo realiza numerosas consultas a lo largo del tiempo, y cada una de ellas debe quedar registrada para poder ser consultada, revisada o corregida posteriormente. Sin un historial organizado, el profesional perdería la capacidad de comparar la evolución de un mismo caso o de recuperar un diagnóstico anterior. Este objetivo garantiza que el sistema conserve el historial completo de diagnósticos de cada usuario de forma aislada y gestionable.

Campo	Contenido
ID	OBJ-002
Nombre	Gestión del historial de consultas
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe conservar el historial de consultas de diagnóstico de cada usuario, permitiendo consultarlas, renombrarlas y eliminarlas, de modo que cada profesional gestione únicamente sus propios registros.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo complementa al OBJ-001 y refuerza el aislamiento de datos entre usuarios.

Tabla x - OBJ-002: Gestión del historial de consultas

OBJ-003 – Generación de informes descargables

Los resultados de un diagnóstico no deben quedarse en la pantalla: el profesional necesita un documento que pueda archivar, imprimir o incorporar a su flujo de trabajo habitual. Este objetivo cubre la generación de informes en formato PDF. El informe consolidado de una sesión de entrenamiento está formalizado en el requisito RF-030, que fija su contenido (configuración del experimento, ranking de modelos, comparativas estadísticas, validación externa y métricas de explicabilidad y calibración). El informe individual de un diagnóstico se entrega junto con el resultado de la consulta: la capacidad está implementada y verificada en la prueba PU-017, aunque la especificación de requisitos no le dedica un requisito funcional propio, una laguna que deberá resolverse en una revisión posterior del catálogo de requisitos.

Campo	Contenido
ID	OBJ-003
Nombre	Generación de informes descargables
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe generar informes en formato PDF. El informe consolidado de cada sesión de entrenamiento se especifica en el requisito RF-030. El informe individual de cada diagnóstico se genera junto con el resultado de la consulta y se verifica en la prueba PU-017.
Importancia	Media
Estado	Aprobado
Comentarios	El informe de sesión está formalizado en el requisito RF-030. El informe individual del diagnóstico está implementado y probado (PU-017), pero no dispone de un requisito funcional propio en la especificación actual, laguna que debe corregirse en una revisión posterior.

Tabla x - OBJ-003: Generación de informes descargables

OBJ-004 – Laboratorio de entrenamiento MLOps

El segundo núcleo del sistema es el laboratorio de entrenamiento. Su propósito es que un investigador pueda configurar y lanzar experimentos de entrenamiento de modelos sin escribir código, apoyándose en un asistente conversacional que interpreta sus indicaciones y las traduce a la configuración técnica del pipeline. El usuario debe poder consultar los resultados de forma organizada y conocer el estado de sus trabajos. Conviene distinguir aquí dos nociones relacionadas pero distintas: el estado de la cola de trabajos, cubierto por el requisito RF-036 (pendiente, en ejecución, completado o fallido), y el progreso del entrenamiento por épocas, que el laboratorio calcula y muestra mientras la ejecución avanza. El cálculo de ese progreso está implementado y verificado en la prueba PU-023, aunque la especificación de requisitos no le dedica un requisito funcional propio, laguna que deberá resolverse en una revisión posterior del catálogo de requisitos.

Campo	Contenido
ID	OBJ-004
Nombre	Laboratorio de entrenamiento MLOps
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe permitir al usuario configurar y lanzar experimentos de entrenamiento de modelos mediante un asistente conversacional, consultar los resultados generados por el pipeline de forma organizada y conocer el estado de sus trabajos.
Importancia	Alta
Estado	Aprobado
Comentarios	Este objetivo engloba el ciclo completo del experimento desde la perspectiva del usuario: configurar, lanzar, consultar el estado y recuperar los resultados. El estado de la cola está cubierto por el requisito RF-036; el progreso por épocas está implementado y verificado en la prueba PU-023, pero carece de un requisito funcional propio en la especificación actual.

Tabla x - OBJ-004: Laboratorio de entrenamiento MLOps

OBJ-005 – Evaluación rigurosa de los modelos

De nada sirve entrenar modelos si no se puede determinar de forma objetiva cuál es mejor y si las diferencias observadas entre ellos son reales o fruto del azar. Este objetivo garantiza que el sistema automatice la evaluación estadística de los modelos —comparación de rendimiento, validación externa sobre una cohorte independiente y contrastes de significación— y que presente los resultados de forma comprensible para el investigador, de manera que las conclusiones puedan defenderse con rigor científico.

Campo	Contenido
ID	OBJ-005
Nombre	Evaluación rigurosa de los modelos
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe automatizar la evaluación estadística de los modelos entrenados: comparación de rendimiento, validación externa sobre una cohorte independiente y contrastes de significación, presentando los resultados de forma comprensible para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	La concreción de las pruebas estadísticas se especifica en los requisitos derivados de este objetivo.

Tabla x - OBJ-005: Evaluación rigurosa de los modelos

OBJ-006 – Reproducibilidad de los experimentos

La reproducibilidad es una exigencia de la investigación científica y, en particular, del aprendizaje automático, donde una inicialización aleatoria distinta o un orden de datos diferente pueden alterar los resultados. Este objetivo garantiza que cada experimento quede asociado a su configuración completa y a las semillas aleatorias utilizadas, de modo que cualquier resultado pueda verificarse y replicarse bajo las mismas condiciones.

Campo	Contenido
ID	OBJ-006
Nombre	Reproducibilidad de los experimentos
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe favorecer la reproducibilidad de los experimentos de entrenamiento, fijando las semillas aleatorias y almacenando la configuración completa de cada sesión junto con sus resultados, de modo que los resultados puedan verificarse y replicarse bajo las mismas condiciones.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo responde a las carencias de reproducibilidad identificadas en la literatura sobre aprendizaje automático.

Tabla x - OBJ-006: Reproducibilidad de los experimentos

OBJ-007 – Acceso seguro y personalizado al sistema

El sistema trata información de carácter clínico, aunque anonimizada, y cada usuario debe disponer de un espacio de trabajo aislado. Este objetivo cubre el registro de nuevos usuarios, la autenticación, el cierre de sesión y la garantía de que cada profesional accede únicamente a sus propios datos. El mecanismo de autorización que aquí se define distingue los perfiles de acceso, pero las operaciones de gobierno reservadas al administrador se recogen como capacidad propia en el OBJ-011, de modo que ambos objetivos no se solapan: el acceso seguro determina quién puede entrar y sobre qué datos, y la administración cubre las operaciones sobre el conjunto de la plataforma.

Campo	Contenido
ID	OBJ-007
Nombre	Acceso seguro y personalizado al sistema
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe disponer de un módulo de gestión de usuarios que permita el registro, la autenticación y el cierre de sesión de forma segura, garantizando que cada usuario opere únicamente sobre sus propios datos y consultas. El rol de administrador y las operaciones de gobierno se recogen en el OBJ-011.
Importancia	Alta
Estado	Aprobado
Comentarios	La autenticación es un requisito no negociable dado el carácter clínico de los datos gestionados. El mecanismo concreto de autenticación corresponde al diseño.

Tabla x - OBJ-007: Acceso seguro y personalizado al sistema

OBJ-008 – Persistencia y trazabilidad de la información

El sistema debe almacenar de forma persistente las consultas de diagnóstico y las sesiones de entrenamiento, junto con su configuración y sus resultados, de modo que el usuario pueda consultar su historial en cualquier momento. La persistencia no es una funcionalidad visible para el usuario final, sino la base sobre la que se sustentan el historial clínico, la comparación de experimentos y la trazabilidad de los resultados.

Campo	Contenido
ID	OBJ-008
Nombre	Persistencia y trazabilidad de la información
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe almacenar de forma persistente las consultas de diagnóstico y las sesiones de entrenamiento, junto con su configuración y sus resultados, de modo que el usuario pueda consultar su historial en cualquier momento.
Importancia	Media
Estado	Aprobado
Comentarios	La persistencia es un requisito transversal: sustenta al resto de los objetivos aunque no sea una funcionalidad visible para el usuario final.

Tabla x - OBJ-008: Persistencia y trazabilidad de la información

OBJ-009 – Usabilidad e internacionalización de la interfaz

El sistema está pensado para usuarios sin formación técnica, por lo que la interfaz debe ser intuitiva y no exigir en ningún momento interactuar con código ni con entornos de programación. Además, el sistema debe estar disponible en varios idiomas, de modo que el usuario pueda cambiar de idioma de forma dinámica, ampliando la accesibilidad de la plataforma a entornos multilingües.

Campo	Contenido
ID	OBJ-009
Nombre	Usabilidad e internacionalización de la interfaz
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe ofrecer una interfaz web intuitiva para usuarios con perfil no técnico, que permita realizar todas las operaciones sin interactuar con código ni con entornos de programación, y debe estar disponible en varios idiomas, permitiendo cambiar de idioma de forma dinámica.
Importancia	Media
Estado	Aprobado
Comentarios	La usabilidad determina la adopción del sistema por parte de los usuarios finales; no afecta a la corrección funcional de los modelos.

Tabla x - OBJ-009: Usabilidad e internacionalización de la interfaz

OBJ-010 – Ejecución asíncrona de tareas de larga duración

El entrenamiento de un modelo de deep learning puede prolongarse durante horas, y durante ese tiempo la plataforma debe seguir respondiendo al usuario. Este objetivo garantiza que las tareas largas —entrenamientos, análisis de explicabilidad y validación externa— se ejecuten de forma asíncrona a través de una cola de trabajos, de modo que la interfaz permanezca operativa y el usuario pueda consultar el estado de cada tarea e incluso cancelar las tareas pendientes.

Campo	Contenido
ID	OBJ-010
Nombre	Ejecución asíncrona de tareas de larga duración
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe ejecutar las tareas de larga duración —entrenamientos, análisis de explicabilidad y validación externa— de forma asíncrona, de modo que la interfaz permanezca operativa durante su ejecución y el usuario pueda consultar el estado de cada tarea.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo garantiza la disponibilidad de la plataforma durante los procesos computacionales más costosos.

Tabla x - OBJ-010: Ejecución asíncrona de tareas de larga duración

OBJ-011 – Administración de la plataforma

El gobierno de una plataforma con múltiples usuarios requiere de un perfil con privilegios de administración que pueda gestionar las cuentas y supervisar la actividad registrada. Este objetivo cubre la existencia de un panel de administración y de un rol diferenciado dentro del mecanismo de autorización. La frontera con el OBJ-007 queda así definida: el acceso seguro garantiza quién puede entrar y sobre qué datos, mientras que la administración cubre las operaciones de gobierno sobre el conjunto de la plataforma, reservadas al administrador. Esta frontera se refleja en la trazabilidad de los requisitos: el requisito transversal de roles y control de acceso (RF-006) se asocia a ambos objetivos, mientras que la auditoría de la actividad administrativa (RNF-006) se asocia únicamente al OBJ-011.

Campo	Contenido
ID	OBJ-011
Nombre	Administración de la plataforma
Versión	01
Autores	Luis Carmona Berdugo
Fuente	Reunión de inicio con el tutor
Descripción	El sistema debe disponer de un panel de administración que permita gestionar las cuentas de usuario y supervisar las consultas y sesiones de entrenamiento registradas en la plataforma, con acceso restringido al rol de administrador.
Importancia	Media
Estado	Aprobado
Comentarios	Este objetivo define la administración como las operaciones de gobierno reservadas al administrador, complementando al OBJ-007, que cubre el acceso seguro y el aislamiento de datos. La auditoría de la actividad administrativa se especifica en el requisito no funcional RNF-006.

Tabla x - OBJ-011: Administración de la plataforma

La siguiente tabla establece la correspondencia entre los once objetivos del sistema y los objetivos específicos del plan de proyecto presentados en el capítulo 2 del Plan de Proyecto (OE1 a OE11). No se trata de una relación uno a uno: varios objetivos del sistema se apoyan en un mismo objetivo del plan, y algunos objetivos del plan son de carácter científico-metodológico y sustentan a más de un objetivo del producto. El objetivo específico OE1 (revisar el estado del arte) no figura en la correspondencia porque pertenece a la fase de fundamentación del proyecto y no se materializa como una capacidad del producto.

Objetivo del sistema	Objetivo(s) específico(s) del plan	Relación
OBJ-001 Diagnóstico asistido con IA explicable	OE5, OE8	La explicabilidad (OE5) y la interfaz clínica (OE8) sustentan el diagnóstico explicable.
OBJ-002 Gestión del historial de consultas	OE2	La arquitectura de persistencia (OE2) soporta el historial.
OBJ-003 Generación de informes descargables	OE8, OE9	El informe individual se entrega con el diagnóstico (OE8); el informe de sesión se consolida en el laboratorio (OE9).
OBJ-004 Laboratorio de entrenamiento MLOps	OE9	El laboratorio y el asistente conversacional (OE9) materializan el laboratorio.
OBJ-005 Evaluación rigurosa de los modelos	OE4, OE6	El pipeline (OE4) produce las métricas; la validación externa y el análisis estadístico (OE6) las evalúan.
OBJ-006 Reproducibilidad de los experimentos	OE7	La reproducibilidad y la trazabilidad (OE7) se materializan en este objetivo.
OBJ-007 Acceso seguro y personalizado al sistema	OE3	El control de acceso (OE3) sustenta este objetivo.
OBJ-008 Persistencia y trazabilidad de la información	OE2	La arquitectura de persistencia (OE2) sustenta este objetivo.
OBJ-009 Usabilidad e internacionalización de la interfaz	OE8, OE11	La usabilidad se garantiza en la interfaz (OE8); la internacionalización (OE11).

Objetivo del sistema	Objetivo(s) específico(s) del plan	Relación
OBJ-010 Ejecución asíncrona de tareas de larga duración	OE10	El procesamiento asíncrono (OE10) se materializa en este objetivo.
OBJ-011 Administración de la plataforma	—	Sin objetivo específico directo: la administración amplía el alcance planificado y se apoya, para el rol, en el control de acceso (OE3).

11.2. Contexto tecnológico y restricciones de entorno

El sistema no se construye sobre un lienzo en blanco, sino que se incorpora a un ecosistema tecnológico preexistente que establece condiciones para su desarrollo. Comprender este contexto es fundamental durante el análisis, porque delimita el marco en el que el sistema debe operar. Este apartado distingue con claridad dos tipos de elementos: las restricciones que el entorno impone de forma ineludible y que condicionan cualquier diseño, y las decisiones tecnológicas que corresponden al proyecto y que, por tanto, se justifican en la parte de diseño de la memoria.

La principal imposición del entorno es el lenguaje de programación. El ecosistema de aprendizaje automático en el que se apoya el proyecto —las librerías de deep learning, los formatos de los modelos y las herramientas de evaluación— está desarrollado íntegramente en Python, por lo que el entorno de ejecución del sistema debe ser compatible con Python y con las librerías de ese ecosistema. Esta restricción es ineludible: optar por otro lenguaje supondría renunciar a la práctica totalidad del ecosistema científico, de las arquitecturas preentrenadas y de los procedimientos experimentales sobre los que se sustenta el proyecto.

La segunda imposición del entorno es la necesidad de computación GPU para el entrenamiento. Una red neuronal está formada por millones de parámetros que se ajustan durante el entrenamiento, y en cada paso del proceso hay que ejecutar un número ingente de operaciones matemáticas sobre los píxeles de las imágenes. Un procesador convencional (CPU) ejecuta estas operaciones de forma secuencial, lo que convertiría el entrenamiento de los modelos en un proceso inviable en los plazos de un Trabajo Fin de Grado. Las unidades de procesamiento gráfico (GPU) ejecutan miles de operaciones en paralelo, lo que reduce drásticamente los tiempos de entrenamiento. El proyecto se desarrolla sobre un equipo portátil equipado con una GPU NVIDIA RTX 4060 Laptop con 8 GB de VRAM, capacidad suficiente para entrenar, con tamaños de lote reducidos y precisión mixta, las arquitecturas más exigentes del banco de pruebas (ViT-384 y EfficientNetB7). Sin esa capacidad, la fase de validación cruzada de los modelos —en la que cada arquitectura se entrena varias veces— resultaría inviable.

El equipo forma parte de la infraestructura ya disponible para el desarrollo del proyecto: el plan de proyecto lo refleja así y no imputa su amortización al coste del TFG. La alternativa de contratar instancias de computación en la nube fue descartada por dos motivos. El primero es económico: alojar el entrenamiento en la nube supone un coste por hora de uso que, multiplicado por las decenas de entrenamientos que exige el proyecto, se convierte en una partida significativa. El segundo es práctico: trabajar en local permite disponer de los pesos de los modelos y de los datos de forma inmediata, sin depender de la conectividad ni de las políticas de almacenamiento de un proveedor externo. Esta restricción tiene una consecuencia directa sobre la planificación: los tiempos de entrenamiento deben estimarse con prudencia y los experimentos más costosos deben programarse con antelación.

Sobre estas imposiciones, el sistema opera en un marco tecnológico que conviene fijar a efectos de análisis. El proyecto se apoya en un servicio web que atiende las peticiones del navegador, un motor de aprendizaje automático para las arquitecturas convolucionales y de atención, una base de datos relacional para la persistencia y un asistente conversacional integrado con un gran modelo de lenguaje. La elección concreta de cada uno de estos componentes —el framework de servicios web, el gestor de base de datos, las librerías y sus versiones— es una decisión de diseño y, como tal, se selecciona y justifica en la parte de diseño de la memoria. El análisis únicamente necesita conocer el contexto tecnológico de trabajo, que se resume en la siguiente tabla, sin entrar en decisiones de implementación como el algoritmo de cifrado de contraseñas, la librería de gestión de tokens o el limitador de peticiones, que pertenecen al diseño.

Componente	Tecnología
Lenguaje	Python
Servicio web y presentación	FastAPI y Uvicorn, con renderizado de plantillas Jinja2
Persistencia	MySQL
Aprendizaje automático	TensorFlow y Keras (arquitecturas convolucionales)
Modelos de atención	Ecosistema Hugging Face Transformers
Visión por computador	OpenCV
Análisis de datos	pandas, NumPy, scikit-learn, SciPy
Representación gráfica	matplotlib, seaborn
Informes PDF	fpdf2
Asistente conversacional	API de Groq (modelo Llama 3)

Tabla x - Dependencias tecnológicas del sistema

La tabla anterior recoge el marco tecnológico de trabajo, no la especificación de implementación. Las versiones exactas de las dependencias, las librerías de seguridad (cifrado de contraseñas, gestión de tokens, limitación de peticiones) y las herramientas de calidad y pruebas se seleccionan y justifican en la parte de diseño, donde se describe el entorno de construcción del sistema.

En cuanto al despliegue, la forma de publicar el servicio es también una decisión del proyecto, no una imposición del entorno. El sistema se ejecuta sobre la máquina local de desarrollo y debe ser accesible desde cualquier navegador, incluidos los de equipos ajenos. Existen varias alternativas para lograr ese acceso: abrir puertos en el router del entorno local, alojar el servicio en una máquina virtual con dirección pública o desplegarlo en un contenedor en la nube. Para este proyecto se opta por un túnel seguro: un canal cifrado que conecta el servidor local con un dominio público proporcionado por un proveedor, de modo que quien accede a ese dominio llega, a través del canal cifrado, al servidor local. Se utiliza un túnel de Cloudflare porque evita exponer el equipo a los riesgos de abrir puertos en el router y no requiere alquilar infraestructura adicional; las alternativas (apertura de puertos, VPS o contenedor en la nube) se descartan por

razones de seguridad, coste y simplicidad operativa. La justificación detallada de esta elección se desarrolla en la parte de diseño, junto con el resto de decisiones de despliegue.

En cuanto a la reproducibilidad, la fijación de versiones y el aislamiento del entorno son condiciones necesarias, pero no suficientes, para reproducir un experimento: garantizan que dos ejecuciones parten del mismo entorno de software, lo que elimina una fuente importante de variabilidad. No obstante, como se señala en el objetivo de reproducibilidad (OBJ-006), la ejecución de los modelos no pretende un determinismo bit a bit entre ejecuciones, sino que cualquier resultado pueda verificarse y replicarse bajo las mismas condiciones. La gestión de las dependencias se orienta, por tanto, a que el entorno sea coherente y reconstruible, no a eliminar toda variabilidad numérica del proceso de entrenamiento.

En definitiva, la distinción entre restricciones y decisiones permite evaluar el sistema sabiendo qué elementos venían condicionados de antemano y cuáles eran libres. La elección de Python y la necesidad de computación GPU son imposiciones del entorno, ineludibles para el proyecto. En cambio, el framework de servicios web, la base de datos, las librerías de seguridad, las versiones concretas de las dependencias y el mecanismo de despliegue son decisiones de diseño que se justifican en la parte de diseño de la memoria y que no deben atribuirse al entorno.

11.3. Normativa y estándares de referencia

El desarrollo del sistema está sujeto a una serie de normas y estándares que se derivan de su propia naturaleza y del ámbito en el que se inscribe. Estas normas condicionan desde la estructura del proyecto hasta el tratamiento de los datos personales, el ámbito sanitario, la seguridad de las comunicaciones y la calidad del código. Conviene examinarlas una a una porque cada una afecta a un aspecto distinto del sistema.

En primer lugar, la cuestión más sensible es la de la protección de datos. El sistema gestiona cuentas de usuarios registrados y procesa imágenes de origen médico. Los datos personales son, por definición, cualquier información relativa a una persona física identificada o identificable, y en el ámbito sanitario esa sensibilidad se multiplica: una imagen de un paciente, su historial de consultas o los resultados asociados a su cuenta son información que debe tratarse con las máximas garantías. En el ámbito de la Unión Europea se aplica el Reglamento General de Protección de Datos (RGPD, Reglamento UE 2016/679), que regula el tratamiento de los datos personales y establece principios como la minimización de los datos —no recoger más información de la necesaria—, la limitación de la finalidad —no usar los datos para fines distintos de los que motivaron su recogida— y la garantía del anonimato cuando se manejan datos de naturaleza biomédica (Parlamento Europeo y Consejo de la Unión Europea, 2016). A nivel nacional, el RGPD se complementa con la Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales, que adapta el reglamento europeo al ordenamiento jurídico español (España, 2018). Conviene señalar que el acceso del administrador a los datos de otros usuarios en el ejercicio de sus funciones de supervisión constituye igualmente un tratamiento de datos: queda acotado al rol y a la finalidad de gobierno de la plataforma, se recoge como excepción del aislamiento de datos en el requisito RF-005 y se regula en el módulo de administración (RF-033 a RF-035).

Conviene ser preciso sobre la anonimización. El proyecto asume que las imágenes que llegan al sistema están anonimizadas y que los conjuntos de datos utilizados son públicos y anónimos, pero esa asunción no equivale a una garantía implementada: si un facultativo sube una radiografía con identificadores visibles en la imagen o con metadatos DICOM que revelen la identidad del paciente, el sistema los almacena junto con la imagen. En el estado actual del proyecto no existe un mecanismo que despoje automáticamente esos metadatos ni una validación que compruebe la ausencia de identificadores antes de aceptar una carga. Esta circunstancia debe documentarse como una limitación: la anonimización previa recae hoy en el responsable de la captura, y una evolución del sistema debería incorporar la limpieza de metadatos y la detección de identificadores como requisitos explícitos.

El tratamiento por parte de terceros merece también atención. El asistente conversacional del laboratorio envía los mensajes del usuario a la API de Groq, un proveedor externo, para obtener las respuestas que se traducen en la configuración del experimento. Aunque el contenido de esas conversaciones se refiere a la configuración de entrenamientos y no a datos clínicos de pacientes, constituye un envío de información del usuario a un tercero y debe hacerse constar como tal. El proyecto lo limita en la medida de lo posible: la conversación se orienta a parámetros técnicos del experimento, no se ofrecen campos para introducir datos personales y el tratamiento se ampara en la necesidad de prestar el servicio al propio usuario. No obstante, esta sección debe dejar constancia de la transferencia para que el responsable del sistema pueda valorar, en cada despliegue, si debe formalizarse un encargo de tratamiento o una cláusula de protección de datos con el proveedor.

En segundo lugar, el sistema se inscribe en el ámbito de las tecnologías sanitarias y de la inteligencia artificial, donde dos reglamentos europeos son especialmente relevantes. El Reglamento (UE) 2017/745 sobre productos sanitarios (MDR) establece los requisitos que debe cumplir un producto destinado a un fin médico para su comercialización en la Unión Europea; un sistema de ayuda al diagnóstico, según su finalidad declarada, podría quedar comprendido en su ámbito, aunque el proyecto se desarrolla en un contexto académico y de investigación, sin intención de comercialización. Por otra parte, el Reglamento (UE) 2024/1689 por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de IA) clasifica los sistemas de IA según su riesgo y establece obligaciones específicas para los de alto riesgo; un sistema de IA destinado a la asistencia al diagnóstico médico es, con alta probabilidad, un sistema de alto riesgo bajo esa norma, con independencia de que el proyecto actual se limite a un entorno de demostración. La memoria reconoce ambas normas como marco de referencia del ámbito y asume que una eventual evolución hacia un producto real requeriría un análisis de conformidad mucho más profundo que el presentado en este trabajo.

En tercer lugar, el sistema mantiene una comunicación constante entre el servidor y el navegador del cliente, a través de la cual viajan credenciales de acceso y datos de las consultas. Estas comunicaciones deben protegerse mediante el protocolo HTTPS, que se fundamenta en el estándar TLS (IETF, 2018) y cifra el contenido de la comunicación, de modo que ni las credenciales ni los datos transmitidos puedan ser leídos por un tercero durante el tránsito por la red. Conviene precisar dónde termina ese cifrado cuando el servicio se publica mediante un túnel, como se describe en el apartado 11.1. En el modelo habitual, el certificado TLS termina en el servidor que aloja la aplicación; con un túnel, el cifrado TLS termina en el borde del proveedor del túnel, y el tramo entre ese borde y el servidor local viaja por el canal cifrado que el propio túnel establece. Esto significa que el proveedor del túnel actúa como extremo de confianza de la conexión HTTPS y, por tanto, debe tratarse como un tercero que puede inspeccionar el tráfico que no está cifrado de extremo a extremo. Esta circunstancia no invalida la protección de las comunicaciones, pero obliga a tener presente que la confianza no termina en el servidor de la aplicación, sino en el operador del túnel.

En cuarto lugar, dado que se trata de una aplicación web accesible desde Internet, la seguridad de la aplicación se fundamenta en el catálogo de riesgos de aplicaciones web de OWASP. OWASP es una organización internacional sin ánimo de lucro dedicada a la seguridad del software, y su publicación más conocida, el Top 10, reúne los diez riesgos de seguridad más críticos que afectan a las aplicaciones web —inyección de código, exposición de datos sensibles, falsificación de peticiones entre sitios, entre otros— (OWASP, 2021). Este catálogo guía las medidas de protección implementadas en el sistema, que incluyen el cifrado de contraseñas, la gestión segura de sesiones, la protección frente a la falsificación de peticiones entre sitios, la limitación de peticiones y el establecimiento de cabeceras de seguridad.

En quinto lugar, el código del sistema se desarrolla íntegramente en Python y debe seguir la guía de estilo PEP 8. Una guía de estilo es un conjunto de convenciones sobre cómo escribir el código —nombres de variables, sangrado, longitud de las líneas, organización de las importaciones— con el fin de que el código sea coherente, legible y fácil de mantener. PEP 8 es un Python Enhancement Proposal, un documento oficial de la comunidad Python: fue redactado originalmente por Guido van Rossum, Barry Warsaw y Nick Coghlan y es mantenido y publicado por la Python Software Foundation, donde se actualiza (van Rossum, Warsaw, & Coghlan, 2001; Python Software Foundation, 2024). En un proyecto de estas dimensiones, donde numerosos módulos interactúan entre sí, la legibilidad del código es un requisito de calidad imprescindible.

Por último, la documentación del proyecto se rige por la normativa de Trabajo Fin de Grado de la Escuela Politécnica Superior de la Universidad Pablo de Olavide, que establece la estructura, el formato y los requisitos formales que deben cumplir los documentos entregables asociados a este TFG (Universidad Pablo de Olavide, 2014). La Tabla 17 resume los estándares y normativas aplicables al sistema.

Estándar / Norma	Ámbito de aplicación
RGPD (Reglamento UE 2016/679)	Protección de datos y tratamiento de información biomédica.
LOPDGDD (Ley Orgánica 3/2018)	Adaptación del RGPD al ordenamiento jurídico español.
MDR (Reglamento UE 2017/745)	Requisitos de los productos sanitarios destinados a un fin médico.
Reglamento de IA (Reglamento UE 2024/1689)	Clasificación y obligaciones de los sistemas de inteligencia artificial.
HTTPS / TLS	Protocolo seguro de comunicación entre cliente y servidor.
OWASP Top 10	Seguridad de las aplicaciones web.
PEP 8	Guía de estilo y nomenclatura del código Python.
Normativa TFG EPS-UPO	Estructura y formato de la documentación del proyecto.

Tabla x - Estándares y normativas aplicables al sistema

11.4. *Perfiles de usuario del sistema*

Identificar correctamente a los usuarios que van a utilizar el sistema es una tarea fundamental dentro del análisis, porque determina desde qué perspectivas se diseña el sistema y qué funcionalidades debe ofrecer a cada tipo de actor. En este apartado conviene distinguir dos planos complementarios que con frecuencia se confunden. El primero es el análisis de los perfiles de usuario, que caracteriza a los colectivos que usarán la plataforma por sus necesidades y por su actividad. El segundo es el modelo de autorización, que define los roles de acceso mediante los que se controla qué puede hacer cada cuenta. Ambos planos se describen a continuación: los perfiles de usuario, en primer lugar, y los roles de acceso, a continuación.

El plan de proyecto identifica dos colectivos de usuarios finales a los que se dirige vitalXAI. El primero lo forman los **facultativos e investigadores clínicos**, el objetivo último del despliegue: profesionales que incorporarían el diagnóstico asistido a su rutina médica e investigadora diaria. Sus necesidades operativas dictan la utilidad real del sistema: obtener un diagnóstico claro y rápido sobre la radiografía, comprender los motivos que lo sustentan mediante los mapas de explicabilidad, poder descargar un informe y disponer de un laboratorio para configurar, comparar y evaluar modelos sin necesidad de escribir código. El segundo colectivo es el **Laboratorio Synergia**, identificado en el plan como usuario externo y primer escalón de adopción: sus líneas de investigación convergen con los objetivos de vitalXAI, por lo que actúa como probador empírico de la herramienta, y sus observaciones condicionan el refinamiento de la interfaz y de los flujos de trabajo. Ninguno de estos colectivos dispone de formación técnica en programación o en aprendizaje automático, de modo que la facilidad de uso y la claridad de la interfaz son requisitos transversales de su adopción, tal y como recoge el requisito RNF-013.

Una decisión de diseño relevante es que la plataforma no restringe el acceso en función del perfil profesional: cualquier usuario con una cuenta puede utilizar tanto la interfaz de diagnóstico como el laboratorio de entrenamiento. Esta decisión simplifica el modelo de acceso, pero tiene una consecuencia de primer orden que debe analizarse. Significa que cualquier facultativo o investigador autenticado puede lanzar entrenamientos que consumen la GPU del servidor durante horas y compiten por la cola de trabajos, por lo que un uso intensivo del laboratorio puede afectar a la experiencia del resto de los usuarios. La garantía de que un entrenamiento no deje la plataforma inaccesible para el resto se apoya en la ejecución asíncrona y en la cola persistente de trabajos (OBJ-010), y queda recogida en el requisito RNF-017, que exige que el servicio permanezca disponible y operativo durante las tareas de larga duración. No obstante, conviene reconocer el límite de esta solución: la interfaz permanece operativa porque las peticiones se encolan, pero los entrenamientos compiten por los recursos computacionales de la estación de trabajo descrita en el apartado 11.1, y una saturación prolongada de la GPU puede degradar el tiempo de respuesta de los diagnósticos que se procesan simultáneamente. El análisis asume este compromiso y lo señala como un punto de evolución: una solución futura debería valorar mecanismos de priorización de trabajos o de limitación de entrenamientos concurrentes por cuenta.

Sobre los perfiles anteriores se define el modelo de autorización, que materializa el control de acceso mediante tres roles. El **usuario visitante** representa a todas las personas que acceden a la aplicación sin haberse autenticado; su acceso se limita a las funcionalidades públicas del sistema: la creación de una nueva cuenta, el inicio de sesión y el cambio de idioma de la interfaz. El **usuario autenticado** es el perfil central del sistema: agrupa a todos los profesionales con cuenta registrada, con independencia de que su actividad sea fundamentalmente clínica o investigadora, y cubre la totalidad de la funcionalidad no administrativa —diagnóstico, historial,

laboratorio MLOps y resultados—. El **usuario administrador** es el responsable del gobierno de la plataforma: gestiona las cuentas y supervisa la actividad registrada, y constituye el único caso en el que el acceso está restringido por rol. Es importante señalar que el administrador es, ante todo, un usuario autenticado: el rol añade capacidades de gobierno, pero no introduce un perfil técnico distinto, de modo que se mantiene la premisa, recogida en RNF-013, de que la plataforma no exige conocimientos técnicos a ninguno de sus usuarios.

La siguiente tabla resume los roles de acceso identificados, con su descripción, el perfil técnico exigido y su nivel de acceso.

Tipo de usuario	Descripción	Perfil técnico	Nivel de acceso
Usuario visitante	Persona que accede a la plataforma sin autenticarse.	No se exigen conocimientos técnicos.	Registro, inicio de sesión y cambio de idioma.
Usuario autenticado	Profesional con cuenta (clínica o investigadora) que utiliza el sistema.	No se exigen conocimientos técnicos.	Acceso completo a la funcionalidad no administrativa: diagnóstico, historial, laboratorio MLOps y resultados.
Usuario administrador	Usuario autenticado con capacidades de gobierno sobre la plataforma.	No se exigen conocimientos técnicos.	Gestión de usuarios y supervisión de la actividad.

Tabla x - Roles de acceso del sistema

Con la definición de los objetivos del sistema, el contexto tecnológico, la normativa aplicable y los perfiles de usuario, queda delimitado el ámbito del sistema. Sobre este marco, los capítulos siguientes del análisis descomponen el sistema en subsistemas, traducen las necesidades de los usuarios en requisitos y casos de uso, y definen el plan de pruebas que verificará el cumplimiento de lo especificado.

12. Especificación de requisitos del sistema

Los requisitos constituyen el contrato entre lo que el análisis afirma que el sistema debe hacer y la solución que se diseña e implementa: son la traducción formal de las necesidades del proyecto en condiciones concretas y verificables que el sistema debe satisfacer (Wieggers & Beatty, 2013). Un requisito bien definido debe ser inequívoco, verificable y trazable: inequívoco, porque dos personas distintas no pueden interpretarlo de forma diferente; verificable, porque debe existir una forma objetiva de comprobar que el sistema lo cumple; y trazable, porque debe poder relacionarse con la necesidad o el objetivo que lo origina. Sobre estas tres propiedades descansa la utilidad de los requisitos como punto de unión entre las necesidades del cliente y el diseño de la solución técnica: sin requisitos precisos, el análisis se queda en una declaración de intenciones y la implementación carece de criterio para decidir qué construir y cómo validarlo (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Este capítulo recoge las especificaciones completas de los requisitos de vitalXAI, obtenidas a partir de los objetivos específicos definidos en el capítulo de metas y propósitos y de los casos de uso identificados en el análisis del sistema, y organizadas de forma que la trazabilidad entre la necesidad, el caso de uso y el requisito quede garantizada en todo momento.

12.1. *Obtención y organización de los requisitos*

Los requisitos del sistema se han obtenido de tres fuentes complementarias. La primera son los objetivos específicos del capítulo 2, que descomponen el objetivo general en metas concretas y verificables y actúan como punto de anclaje de cada requisito. La segunda son los casos de uso recogidos en el análisis, que describen las interacciones de los actores con el sistema y permiten identificar las capacidades concretas que deben implementarse. La tercera son las reuniones mantenidas con el tutor y con los asesores, que aportan el criterio académico y de dominio necesario para matizar el alcance de cada capacidad y para validar la coherencia de los requisitos con la realidad clínica y técnica del proyecto.

A partir de estas fuentes, los requisitos se han clasificado en dos grandes grupos. Los requisitos funcionales describen qué debe hacer el sistema: cada uno especifica una capacidad concreta, definida desde el punto de vista del usuario y con independencia de cómo se implemente. Los requisitos no funcionales describen cómo debe comportarse el sistema en cuanto a calidad, seguridad, rendimiento y cumplimiento normativo. Cada requisito se identifica de forma inequívoca mediante un código —RF-XXX para los funcionales y RNF-XXX para los no funcionales—, se describe de forma precisa y se asocia a los objetivos y a los casos de uso de los que se deriva, de modo que la trazabilidad entre el análisis y la implementación quede garantizada en todo momento.

12.1.1. Requisitos funcionales del sistema

Los requisitos funcionales describen las capacidades y funcionalidades que el sistema debe ofrecer al usuario para cumplir con los objetivos definidos en el capítulo de metas y propósitos. Cada requisito funcional especifica una acción concreta, definida desde el punto de vista del usuario y con total independencia de cómo se implemente posteriormente; en consecuencia, los requisitos se formulan sin referirse a tecnologías o mecanismos de construcción concretos, que quedan reservados para la parte de diseño de la memoria.

Los requisitos funcionales se organizan en módulos que se corresponden con las áreas funcionales identificadas en el análisis: el módulo de autenticación y cuenta, el módulo de diagnóstico clínico, el módulo de historial de consultas, el módulo del laboratorio MLOps, el módulo de administración y el módulo transversal. Cada módulo agrupa los requisitos relacionados con un ámbito funcional homogéneo y se corresponde, de forma natural, con los módulos de casos de uso definidos en el análisis. En esta sección se detalla el primero de ellos, el módulo de autenticación y cuenta, que constituye la puerta de acceso al resto de la funcionalidad del sistema.

12.1.1.1. Módulo de autenticación y cuenta

Este módulo agrupa los requisitos relacionados con el control de acceso al sistema. Su propósito es garantizar que únicamente los usuarios registrados y autenticados puedan acceder a las funcionalidades privadas de la plataforma, y que cada usuario pueda operar únicamente con sus propios datos. La autenticación es un requisito transversal: ninguna de las funcionalidades de diagnóstico, historial, laboratorio o administración puede utilizarse sin completar previamente este proceso. Los requisitos de este módulo se corresponden con los casos de uso CU-001 a CU-004 y dan respuesta al objetivo específico de diseñar e implementar el control de acceso del sistema.

RF-001 — Registro de usuario

La plataforma está concebida para ser utilizada por múltiples profesionales sanitarios e investigadores, y cada uno de ellos debe disponer de un espacio de trabajo propio y aislado. Para que ese modelo de uso sea posible, el sistema debe ofrecer un proceso de registro que permita a cualquier persona crear una cuenta. El registro es el punto de entrada de todos los usuarios del sistema: sin una cuenta, no existe forma de acceder a las funcionalidades privadas de la plataforma. Es especialmente relevante que las contraseñas se almacenen de forma segura, de modo que, ante un acceso no autorizado a la base de datos, las credenciales de los usuarios no queden expuestas. El requisito RF-001 recoge este comportamiento.

Campo	Contenido
ID	RF-001
Nombre	Registro de usuario
Objetivos relacionados	OE3
Descripción	La plataforma debe facilitar la creación de cuentas para perfiles no autenticados, capturando nombre, apellidos, correo y credenciales. Es imperativo validar la integridad de la información y la inexistencia previa del usuario, asegurando que las contraseñas se resguarden mediante mecanismos de persistencia segura que eviten su almacenamiento en texto claro.
Importancia	Alta
Estado	Aprobado
Comentarios	Aunque la técnica de cifrado se delega a la fase de diseño, este requisito impone la prohibición de almacenar credenciales legibles en el soporte de datos.

Tabla x - RF-001 — Registro de usuario

RF-002 — Inicio de sesión

Una vez que el usuario dispone de una cuenta, el siguiente paso es poder acceder al sistema. El inicio de sesión es la acción mediante la cual el sistema verifica la identidad del usuario a partir de sus credenciales y, si estas son correctas, le otorga acceso a las áreas privadas de la plataforma. Se trata del punto de entrada a toda la funcionalidad privada: ninguna operación de diagnóstico, historial o laboratorio puede realizarse sin haber superado este proceso. El requisito RF-002 especifica este comportamiento, dejando al diseño la elección del mecanismo concreto de gestión de la sesión.

Campo	Contenido
ID	RF-002
Nombre	Inicio de sesión
Objetivos relacionados	OE3
Descripción	La plataforma debe implementar un mecanismo de autenticación que verifique la identidad del usuario a través de sus credenciales registradas. Tras una validación exitosa, el sistema otorgará acceso exclusivo a las dimensiones privadas y funcionalidades restringidas de la aplicación.
Importancia	Alta
Estado	Aprobado
Comentarios	La elección técnica sobre la gestión de la sesión se delega al ámbito del diseño arquitectónico. Asimismo, las salvaguardas frente a accesos malintencionados o reiterados se contemplan de forma transversal en los requisitos de seguridad.

Tabla x - RF-002 — Inicio de sesión

RF-003 — Cierre de sesión

El flujo de acceso a la plataforma no se considera íntegro sin la capacidad de finalizar la estancia de forma controlada. Esta funcionalidad permite que el usuario revoque su acceso de manera fiable, impidiendo que el uso de terminales compartidos o dispositivos en espacios públicos comprometa la privacidad de la información clínica. Se trata de una operación fundamental para salvaguardar la integridad del sistema en contextos de concurrencia, asegurando que ninguna interacción con las dimensiones privadas sea posible sin una nueva validación de credenciales. El requisito RF-003 formaliza esta capacidad.

Campo	Contenido
ID	RF-003
Nombre	Cierre de sesión
Objetivos relacionados	OE3
Descripción	El sistema debe proporcionar un mecanismo de finalización de sesión que garantice la revocación del acceso del usuario. Tras la ejecución de esta acción, la plataforma debe denegar cualquier interacción con las áreas restringidas hasta que se valide una nueva identidad.
Importancia	Alta
Estado	Aprobado
Comentarios	Este requisito es vital para evitar el acceso no autorizado en equipos de uso común. Los detalles sobre la invalidación técnica de la sesión se concretan en la fase de diseño.

Tabla x - RF-003 — Cierre de sesión

RF-004 — Cambio de idioma de la interfaz

La plataforma está dirigida a un público sanitario e investigador que puede proceder de entornos lingüísticos muy diversos, y uno de sus objetivos es ampliar la accesibilidad de la herramienta. Para ello, la interfaz debe poder presentarse en varios idiomas y el usuario debe poder cambiar de idioma en cualquier momento, sin necesidad de reconfigurar nada. Este cambio debe aplicarse de forma dinámica en toda la interfaz y, cuando proceda, también en los informes generados y en el asistente conversacional, de modo que la experiencia de uso resulte coherente en el idioma elegido. El requisito RF-004 recoge este comportamiento y está disponible tanto para visitantes como para usuarios autenticados.

Campo	Contenido
ID	RF-004
Nombre	Cambio de idioma de la interfaz
Objetivos relacionados	OE11
Descripción	El sistema debe permitir que cualquier perfil de usuario alterne de forma dinámica el idioma de navegación entre las opciones de español, inglés, chino e hindi. Dicha transición debe reflejarse de manera integral en los elementos visuales, el motor de asistencia por diálogo y la exportación de documentos.
Importancia	Media
Estado	Aprobado
Comentarios	Esta capacidad materializa el compromiso con la internacionalización de la plataforma y es accesible transversalmente para todos los roles de usuario.

Tabla x - RF-004 — Cambio de idioma de la interfaz

RF-005 — Aislamiento de datos entre usuarios

El sistema gestiona información de carácter clínico, aunque anonimizada, y cada usuario desarrolla su actividad sobre sus propios datos: sus consultas de diagnóstico, sus sesiones de entrenamiento y sus resultados. Garantizar que ningún usuario autenticado pueda acceder a los datos de otro en el uso ordinario de la plataforma es un requisito de seguridad y de privacidad de primer orden, y de especial relevancia en un ámbito donde los datos tratados pueden ser sensibles. Este aislamiento no debe ser una propiedad coyuntural de una u otra funcionalidad, sino un principio transversal que se garantice en todas las operaciones de acceso a datos: tanto en la consulta del historial como en la gestión de las sesiones de entrenamiento o en la descarga de informes, el sistema debe verificar siempre que el recurso solicitado pertenece al usuario que lo solicita. De este modo, el aislamiento se convierte en una salvaguarda aplicada de forma sistemática en toda la plataforma. El acceso del administrador a los datos de otros usuarios, limitado a sus funciones de supervisión y definido en el módulo de administración (RF-033 a RF-035), constituye la única excepción a este principio. El requisito RF-005 recoge el aislamiento entre usuarios.

Campo	Contenido
ID	RF-005
Nombre	Aislamiento de datos entre usuarios
Objetivos relacionados	OE3
Descripción	El sistema debe garantizar que, en el uso ordinario de la plataforma, cada usuario autenticado solo puede acceder a sus propias consultas de diagnóstico, sesiones de entrenamiento y resultados. El acceso del administrador a los datos de otros usuarios, limitado a sus funciones de supervisión y regulado por el módulo de administración (RF-033 a RF-035), constituye la única excepción a este aislamiento.
Importancia	Alta
Estado	Aprobado
Comentarios	Este requisito es transversal a todos los módulos del sistema y debe estar garantizado en todas las operaciones que impliquen el acceso a datos por parte de los usuarios autenticados. La excepción administrativa queda acotada al rol de administrador y se justifica por la función de supervisión de la plataforma.

Tabla x - RF-005 — Aislamiento de datos entre usuarios

RF-006 — Roles y control de acceso

La plataforma distingue dos formas de participación: la del usuario que utiliza las funcionalidades de diagnóstico y de laboratorio, y la del administrador que gobierna el sistema. La experiencia de acceso es la misma para todos los usuarios autenticados —cualquier usuario con cuenta puede utilizar tanto la interfaz de diagnóstico como el laboratorio, sin restricción por perfil profesional—, pero las funcionalidades de administración deben quedar reservadas para el rol de administrador, que gestiona las cuentas y supervisa la actividad registrada. Esta distinción entre el rol de usuario y el rol de administrador debe materializarse en el mecanismo de autorización del sistema. El requisito RF-006 recoge este comportamiento.

Campo	Contenido
ID	RF-006
Nombre	Roles y control de acceso
Objetivos relacionados	OE3, OE12
Descripción	La aplicación debe articular un esquema de autorización con dos perfiles diferenciados: el usuario autenticado, con capacidad operativa sobre los núcleos clínico e investigador, y el usuario administrador, facultado con atribuciones de gestión que permanecen restringidas para el resto de actores.
Importancia	Media
Estado	Aprobado
Comentarios	La implementación del rol de administración garantiza el gobierno centralizado de la plataforma. El desacoplamiento jerárquico se materializa a través de la capa de lógica de negocio y los permisos de acceso.

Tabla x - RF-006 — Roles y control de acceso

12.1.1.2. Módulo de diagnóstico clínico

Este módulo agrupa los requisitos de la interfaz clínica, el primer núcleo funcional del sistema. A través de ella, el usuario autenticado puede realizar un diagnóstico asistido de neumonía a partir de una radiografía de tórax: cargar la imagen, seleccionar la arquitectura con la que desea realizar la consulta, obtener un diagnóstico con su nivel de confianza y visualizar los mapas de explicabilidad que lo justifican. El diagnóstico se procesa de forma asíncrona, de modo que la interfaz permanece operativa mientras el sistema analiza la imagen. Los requisitos de este módulo se corresponden con los casos de uso CU-005 a CU-010 y se asocian al objetivo de diagnóstico asistido con inteligencia artificial explicable (OBJ-001).

RF-007 — Acceso al panel de diagnóstico

Toda la funcionalidad de diagnóstico se organiza en torno a un panel que el usuario autenticado utiliza como punto de partida de sus consultas. Este panel reúne, en una única vista, la carga de la radiografía, la selección del modelo y el acceso al historial, de modo que el profesional dispone de un entorno de trabajo claro y ordenado, sin necesidad de navegar entre secciones. El requisito RF-007 recoge la necesidad de que este panel exista y sea accesible para todo usuario autenticado.

Campo	Contenido
ID	RF-007
Nombre	Acceso al panel de diagnóstico
Objetivos relacionados	OBJ-001
Descripción	La plataforma debe proporcionar un panel operativo para el usuario autenticado que centralice la carga de imágenes radiológicas, la elección del modelo y la supervisión del historial de sus consultas clínicas.
Importancia	Alta
Estado	Aprobado
Comentarios	Dicha interfaz se sitúa como el eje fundamental para el despliegue de las capacidades clínicas dentro de la solución tecnológica.

Tabla x - RF-007 — Acceso al panel de diagnóstico

RF-008 — Subida de una radiografía de tórax

El primer paso de cualquier diagnóstico es incorporar la imagen al sistema. El facultativo debe poder seleccionar una radiografía de tórax desde su equipo, y el sistema debe validarla antes de aceptarla: el formato debe ser un tipo de imagen admitido (JPEG o PNG) y el tamaño no debe superar los 10 MB. Esta validación evita que un archivo incorrecto o excesivamente grande llegue al motor de inferencia, donde no podría procesarse y degradaría el flujo de trabajo del profesional. La imagen se conserva en el servidor asociada a la consulta, de modo que pueda recuperarse desde el historial del usuario. El requisito RF-008 recoge este comportamiento.

Campo	Contenido
ID	RF-008
Nombre	Subida de una radiografía de tórax
Objetivos relacionados	OBJ-001
Descripción	El sistema debe permitir al usuario autenticado subir una radiografía de tórax desde su equipo, validando que el formato sea JPEG o PNG y que su tamaño no supere los 10 MB, y conservarla asociada a la consulta para que pueda recuperarse desde el historial.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación del formato y del tamaño se realiza antes de encolar la consulta. El límite de 10 MB es un valor único del catálogo y debe mantenerse coherente con el caso de uso CU-006.

Tabla x - RF-008 — Subida de una radiografía de tórax

RF-009 — Selección de la arquitectura para el diagnóstico

El sistema dispone de varias arquitecturas de deep learning entrenadas, y el resultado del diagnóstico depende del modelo empleado. El usuario debe poder elegir, entre las arquitecturas disponibles, el modelo con el que desea realizar la consulta, de modo que pueda comparar el comportamiento de distintos modelos sobre una misma imagen. El requisito RF-009 recoge esta capacidad de selección.

Campo	Contenido
ID	RF-009
Nombre	Selección de la arquitectura para el diagnóstico
Objetivos relacionados	OBJ-001
Descripción	El sistema debe permitir al usuario autenticado seleccionar, entre las arquitecturas de deep learning disponibles, el modelo con el que desea realizar el diagnóstico.
Importancia	Media
Estado	Aprobado
Comentarios	La lista de arquitecturas disponibles se muestra en el panel de diagnóstico.

Tabla x - RF-009 — Selección de la arquitectura para el diagnóstico

RF-010 — Solicitud de un diagnóstico

Con la imagen cargada y el modelo seleccionado, el usuario solicita el diagnóstico. Esta petición es el punto de entrada del flujo clínico: el sistema debe aceptar la solicitud, encolarla y procesarla en segundo plano, de modo que la interfaz permanezca operativa mientras el trabajo se ejecuta. Una vez finalizado el procesamiento, la consulta queda registrada en el historial del usuario con su resultado. La presentación de la predicción y de su confianza se recoge en el requisito RF-011, la generación de los mapas de explicabilidad en el RF-012 y la ejecución asíncrona de tareas de larga duración es una capacidad transversal del sistema (OBJ-010). El requisito RF-010 recoge exclusivamente la solicitud y el encolado del diagnóstico.

Campo	Contenido
ID	RF-010
Nombre	Solicitud de un diagnóstico
Objetivos relacionados	OBJ-001, OBJ-010
Descripción	El sistema debe permitir al usuario autenticado solicitar un diagnóstico sobre la imagen cargada y con la arquitectura seleccionada, encolando la consulta para su procesamiento en segundo plano y registrándola en el historial cuando finalice.
Importancia	Alta
Estado	Aprobado
Comentarios	La generación de los mapas de explicabilidad se recoge en el requisito RF-012 y la ejecución asíncrona es una capacidad transversal (OBJ-010), de modo que cada capacidad es verificable de forma independiente.

Tabla x - RF-010 — Solicitud de un diagnóstico

RF-011 — Visualización del resultado del diagnóstico

Cuando la consulta finaliza, el sistema debe presentar al usuario el resultado del diagnóstico: la predicción (PNEUMONIA o NORMAL), el nivel de confianza asociado y el modelo empleado. Esta información se muestra de forma clara y sin tecnicismos, de modo que el profesional pueda interpretarla de inmediato y decidir si confía en ella. La presentación del nivel de confianza es especialmente relevante, porque permite al facultativo calibrar la fiabilidad de la predicción antes de incorporarla a su valoración clínica. La consulta queda registrada en el historial con su resultado. El requisito RF-011 recoge esta presentación del resultado.

Campo	Contenido
ID	RF-011
Nombre	Visualización del resultado del diagnóstico
Objetivos relacionados	OBJ-001
Descripción	El sistema debe mostrar al usuario el resultado de cada consulta de diagnóstico: la predicción, el nivel de confianza y el modelo empleado, de forma clara y comprensible.
Importancia	Alta
Estado	Aprobado
Comentarios	La consulta queda registrada en el historial con su resultado.

Tabla x - RF-011 — Visualización del resultado del diagnóstico

RF-012 — Visualización de los mapas de explicabilidad

Un diagnóstico asistido no es útil si el profesional no puede comprobar los motivos de la decisión del modelo. Por ello, cada consulta debe acompañarse de los mapas de explicabilidad que justifican la predicción: Saliency Maps, que resaltan los píxeles más influyentes; SmoothGrad, que suaviza el ruido de los mapas de saliencia; y Grad-CAM, que produce mapas de activación de clase gruesos y fácilmente interpretables, todos ellos superpuestos sobre la radiografía original. Para las arquitecturas Transformer se emplean mapas de atención, que muestran qué regiones de la imagen está ponderando el modelo. Estos mapas permiten al facultativo verificar que el modelo se fija en las regiones pulmonares relevantes y no en artefactos de la imagen, lo que constituye el fundamento de la confianza clínica en el sistema. El requisito RF-012 recoge esta visualización.

Campo	Contenido
ID	RF-012
Nombre	Visualización de los mapas de explicabilidad
Objetivos relacionados	OBJ-001
Descripción	El sistema debe mostrar, junto al resultado de cada consulta, los mapas de explicabilidad que justifican la predicción, superpuestos sobre la radiografía original: Saliency Maps, SmoothGrad y Grad-CAM para las arquitecturas convolucionales, y mapas de atención para las arquitecturas Transformer.
Importancia	Alta
Estado	Aprobado
Comentarios	La generación de las explicaciones se apoya en las técnicas XAI descritas en el capítulo 1, y su evaluación cuantitativa se recoge en el laboratorio (RF-022).

Tabla x - RF-012 — Visualización de los mapas de explicabilidad

12.1.1.3. Módulo de historial de consultas

Este módulo agrupa los requisitos relacionados con la gestión del historial de consultas de diagnóstico. Un facultativo realiza numerosas consultas a lo largo del tiempo, y cada una de ellas debe quedar registrada para poder ser consultada, revisada o corregida posteriormente. El sistema conserva el historial de cada usuario de forma aislada, de modo que cada profesional gestione únicamente sus propios registros. Los requisitos de este módulo se corresponden con los casos de uso CU-011 a CU-014 y se asocian al objetivo de gestión del historial de consultas (OBJ-002).

RF-013 — Consultar el historial de consultas

El profesional necesita recuperar en cualquier momento sus consultas anteriores para revisar un diagnóstico, comparar la evolución de un caso o reutilizar una imagen. El sistema debe mostrar el listado de sus consultas, con los datos esenciales de cada una —fecha, modelo empleado, resultado y confianza—, de modo que pueda localizarlas con rapidez. El requisito RF-013 recoge este comportamiento.

Campo	Contenido
ID	RF-013
Nombre	Consultar el historial de consultas
Objetivos relacionados	OBJ-002, OBJ-008
Descripción	El sistema debe permitir al usuario autenticado consultar el listado de sus consultas de diagnóstico, mostrando para cada una la fecha, el modelo empleado, el resultado y el nivel de confianza.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las consultas del propio usuario, en cumplimiento del aislamiento de datos entre usuarios.

Tabla x - RF-013 — Consultar el historial de consultas

RF-014 — Visualización del detalle de una consulta del historial

Además del listado, el profesional debe poder abrir el detalle completo de una consulta concreta para recuperar la imagen original, el resultado, la confianza y los mapas de explicabilidad asociados. Este detalle permite revisar un diagnóstico en profundidad o utilizarlo como referencia para una nueva consulta, por lo que constituye una operación habitual en el uso clínico de la plataforma. El requisito RF-014 recoge esta visualización del detalle.

Campo	Contenido
ID	RF-014
Nombre	Visualización del detalle de una consulta del historial
Objetivos relacionados	OBJ-002, OBJ-008
Descripción	El sistema debe permitir al usuario autenticado visualizar el detalle de una consulta de su historial, incluyendo la imagen original, el resultado, el nivel de confianza y los mapas de explicabilidad.
Importancia	Media
Estado	Aprobado
Comentarios	El acceso al detalle está restringido a las consultas del propio usuario.

Tabla x - RF-014 — Visualización del detalle de una consulta del historial

RF-015 — Renombrar una consulta del historial

El profesional puede querer dar a sus consultas un nombre más descriptivo para identificarlas mejor, por ejemplo cuando un mismo paciente tiene varias placas. El sistema debe permitir modificar el nombre de una consulta propia. El requisito RF-015 recoge esta operación.

Campo	Contenido
ID	RF-015
Nombre	Renombrar una consulta del historial
Objetivos relacionados	OBJ-002
Descripción	El sistema debe permitir al usuario autenticado modificar el nombre de una de sus consultas del historial.
Importancia	Baja
Estado	Aprobado
Comentarios	El nuevo nombre no puede estar vacío y solo aplica a consultas del propio usuario.

Tabla x - RF-015 — Renombrar una consulta del historial

RF-016 — Eliminar una consulta del historial

El profesional debe poder depurar su historial, eliminando las consultas que ya no necesita. La eliminación es definitiva: el registro deja de aparecer en el listado y no puede recuperarse posteriormente, y tampoco queda accesible para el administrador en el ejercicio de su supervisión. Esta decisión se corresponde con el derecho de supresión previsto en el RGPD, que exige precisamente resolver qué ocurre con los datos cuando el interesado solicita su borrado. El requisito RF-016 recoge esta operación.

Campo	Contenido
ID	RF-016
Nombre	Eliminar una consulta del historial
Objetivos relacionados	OBJ-002
Descripción	El sistema debe permitir al usuario autenticado eliminar una de sus consultas del historial de forma definitiva, de modo que el registro y sus artefactos asociados dejen de estar disponibles tanto para el propio usuario como para el administrador.
Importancia	Baja
Estado	Aprobado
Comentarios	El sistema debe permitir al usuario autenticado eliminar una de sus consultas del historial de forma definitiva, de modo que el registro y sus artefactos asociados dejen de estar disponibles tanto para el propio usuario como para el administrador.

Tabla x - RF-016 — Eliminar una consulta del historial

12.1.1.4. Módulo de laboratorio MLOps

Este módulo agrupa los requisitos del laboratorio de entrenamiento MLOps, el segundo núcleo funcional del sistema. A través de él, el usuario investigador puede configurar y lanzar experimentos de entrenamiento mediante un asistente conversacional, monitorizar su progreso y consultar los resultados comparativos y estadísticos. El laboratorio orquesta automáticamente la secuencia de entrenamiento, análisis de explicabilidad, comparación estadística y validación externa, de modo que el usuario no necesita escribir código en ningún momento. Los requisitos de este módulo se corresponden con los casos de uso CU-015 a CU-030 y se asocian a los objetivos de laboratorio de entrenamiento MLOps (OBJ-004), evaluación rigurosa de los modelos (OBJ-005) y ejecución asíncrona de tareas (OBJ-010).

RF-017 — Acceso al laboratorio de entrenamiento

El laboratorio de entrenamiento es el espacio desde el que el usuario lanza y consulta sus experimentos. Al igual que el panel de diagnóstico para la actividad clínica, el laboratorio debe ser accesible para todo usuario autenticado y presentar, en una única vista, el asistente conversacional, las sesiones de entrenamiento y el acceso a los resultados. Esta accesibilidad es condición necesaria para que el resto de los requisitos de este módulo tengan sentido. El requisito RF-017 recoge la necesidad de que el laboratorio exista y sea accesible.

Campo	Contenido
ID	RF-017
Nombre	Acceso al laboratorio de entrenamiento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario autenticado acceder al laboratorio de entrenamiento, desde el que pueda conversar con el asistente, lanzar experimentos y consultar sus sesiones.
Importancia	Alta
Estado	Aprobado
Comentarios	El laboratorio es accesible para todo usuario autenticado, con independencia de su perfil profesional.

Tabla x - RF-017 — Acceso al laboratorio de entrenamiento

RF-018 — Conversación con el asistente para configurar un experimento

El laboratorio se maneja mediante un asistente conversacional que permite al usuario definir un experimento en lenguaje natural, sin escribir código. El usuario debe poder comunicarse con el asistente indicando qué arquitecturas quiere entrenar y con qué hiperparámetros, y el asistente debe interpretar esas indicaciones y traducirlas a la configuración técnica del experimento. Para ello, el asistente debe conocer los parámetros que definen el experimento —las arquitecturas a entrenar, el número de épocas, el tamaño de lote y la tasa de aprendizaje— y, cuando disponga de todos ellos, devolver la configuración estructurada para que el usuario pueda revisarla y confirmarla. Si falta algún parámetro, el asistente debe solicitarlo antes de completar la configuración. La ruta del dataset no forma parte de la conversación: se selecciona de forma validada mediante el requisito RF-019 y se incorpora a la configuración del experimento. La conversación debe desarrollarse en el idioma seleccionado por el usuario. El requisito RF-018 recoge este comportamiento.

Campo	Contenido
ID	RF-018
Nombre	Conversación con el asistente para configurar un experimento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario configurar un experimento de entrenamiento mediante un asistente conversacional en lenguaje natural, que interprete los parámetros del experimento (arquitecturas, épocas, tamaño de lote y tasa de aprendizaje) y devuelva la configuración estructurada para su revisión, incorporando la ruta del dataset seleccionada conforme al RF-019.
Importancia	Alta
Estado	Aprobado
Comentarios	El asistente debe completar los parámetros faltantes preguntando al usuario antes de devolver la configuración. La ruta del dataset se obtiene mediante RF-019.

Tabla x - RF-018 — Conversación con el asistente para configurar un experimento

RF-019 — Selección de la carpeta del dataset

Antes de lanzar un experimento, el sistema debe disponer de la ruta del dataset sobre el que se entrenarán los modelos. La selección de esa ruta no puede exponer al usuario el sistema de ficheros completo del servidor: un usuario autenticado cualquiera no debe poder navegar por rutas arbitrarias de la máquina (RNF-013) ni alcanzar directorios ajenos a los datasets permitidos (RF-005). Por ello, la selección se confina a un directorio raíz de datasets predefinido en el entorno: el sistema debe limitar las opciones a ese directorio y validar que la ruta resultante permanece dentro de él, rechazando cualquier ruta que lo abandone. La misma operación cubre la selección del dataset externo empleado en la validación externa. El requisito RF-019 recoge esta selección confinada.

Campo	Contenido
ID	RF-019
Nombre	Selección de la carpeta del dataset
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario seleccionar el dataset de entrenamiento y el dataset externo de validación dentro de un directorio raíz de datasets permitido, validando que la ruta resultante permanece dentro de ese directorio y sin exponer el resto del sistema de ficheros del servidor.
Importancia	Media
Estado	Aprobado
Comentarios	La selección se confina al directorio de datasets permitido para impedir el acceso a rutas arbitrarias del servidor y respetar el aislamiento entre usuarios (RF-005).

Tabla x - RF-019 — Selección de la carpeta del dataset

RF-020 — Lanzamiento de un experimento de entrenamiento

Con la configuración definida, el usuario lanza el experimento. El sistema debe crear una sesión de entrenamiento y encolar la ejecución del entrenamiento de las arquitecturas solicitadas en segundo plano, de modo que la interfaz permanezca operativa y el usuario pueda monitorizar el progreso y continuar utilizando la plataforma. La metodología experimental concreta del entrenamiento —particiones de validación cruzada, métricas y evaluación estadística— corresponde al diseño del pipeline y sus resultados se consultan mediante los requisitos del laboratorio (RF-022 a RF-025); no forma parte de este requisito. El requisito RF-020 recoge únicamente el lanzamiento del experimento y su encolado asíncrono.

Campo	Contenido
ID	RF-020
Nombre	Lanzamiento de un experimento de entrenamiento
Objetivos relacionados	OBJ-004, OBJ-010, OBJ-006
Descripción	El sistema debe permitir al usuario lanzar un experimento de entrenamiento con la configuración definida, creando la sesión y encolando la ejecución de forma asíncrona, de modo que la interfaz permanezca operativa durante el entrenamiento.
Importancia	Alta
Estado	Aprobado
Comentarios	El análisis de explicabilidad y la comparación estadística se consultan mediante los requisitos RF-023 a RF-025; su ejecución automática tras el entrenamiento es una decisión del diseño del pipeline. La creación de la sesión con su configuración completa sustenta el objetivo de reproducibilidad (OBJ-006).

Tabla x - RF-020 — Lanzamiento de un experimento de entrenamiento

RF-021 — Consulta de las sesiones de entrenamiento

El usuario puede lanzar varios experimentos a lo largo del tiempo, y cada uno queda registrado como una sesión de entrenamiento. El sistema debe mostrar el listado de sus sesiones, con su estado y sus modelos, de modo que el usuario pueda localizarlas y consultar sus resultados. El requisito RF-021 recoge este listado.

Campo	Contenido
ID	RF-021
Nombre	Consulta de las sesiones de entrenamiento
Objetivos relacionados	OBJ-004, OBJ-008
Descripción	El sistema debe permitir al usuario autenticado consultar el listado de sus sesiones de entrenamiento con su estado y sus modelos.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las sesiones del propio usuario.

Tabla x - RF-021 — Consulta de las sesiones de entrenamiento

RF-022 — Consulta de los resultados de un modelo de la sesión

Cada sesión produce, para cada modelo entrenado, un conjunto de resultados cuantitativos que permiten evaluar su rendimiento y su fiabilidad. El sistema debe mostrar, para un modelo concreto, las métricas de la validación cruzada (exactitud, precisión, sensibilidad, F1 y AUC), las métricas cuantitativas de explicabilidad (Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM) y las métricas de calibración (Brier Score y Expected Calibration Error), con su media y su desviación sobre los pliegues. La desagregación por pliegue es un formato de presentación que corresponde al diseño y debe ser coherente con el modelo de resultados persistido de la sesión. El requisito RF-022 recoge esta consulta.

Campo	Contenido
ID	RF-022
Nombre	Consulta de los resultados de un modelo de la sesión
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar al usuario, para un modelo de la sesión, las métricas de la validación cruzada (exactitud, precisión, sensibilidad, F1 y AUC), las métricas cuantitativas de explicabilidad y las métricas de calibración, con su media y su desviación sobre los pliegues.
Importancia	Alta
Estado	Aprobado
Comentarios	La presentación de los resultados debe ser coherente con el modelo de resultados persistido de la sesión; la desagregación por pliegue es una decisión del diseño de presentación.

Tabla x - RF-022 — Consulta de los resultados de un modelo de la sesión

RF-023 — Visualización de los mapas de explicabilidad de un modelo

Además de las métricas numéricas, el laboratorio debe permitir la inspección visual de las explicaciones de cada modelo. El sistema debe mostrar la galería de mapas de calor generados por el análisis XAI cualitativo sobre imágenes de ejemplo, lo que permite comprobar si el modelo se fija en las regiones pulmonares relevantes. El requisito RF-023 recoge esta visualización.

Campo	Contenido
ID	RF-023
Nombre	Visualización de los mapas de explicabilidad de un modelo
Objetivos relacionados	OBJ-005
Descripción	El sistema debe permitir al usuario visualizar la galería de mapas de explicabilidad generados por el análisis XAI cualitativo de un modelo de la sesión.
Importancia	Media
Estado	Aprobado
Comentarios	Los mapas se generan sobre imágenes de ejemplo del dataset de entrenamiento.

Tabla x - RF-023 — Visualización de los mapas de explicabilidad de un modelo

RF-024 — Consulta del ranking de modelos de la sesión

Una vez completada la comparación estadística, el sistema debe presentar el ranking global de los modelos de la sesión, ordenados por su AUC medio de la validación cruzada. Este ranking permite identificar de un vistazo las arquitecturas con mejor rendimiento dentro de la sesión. El requisito RF-024 recoge esta consulta.

Campo	Contenido
ID	RF-024
Nombre	Consulta del ranking de modelos de la sesión
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar el ranking global de los modelos de la sesión, ordenados por su AUC medio de la validación cruzada, junto con su desviación típica.
Importancia	Media
Estado	Aprobado
Comentarios	El ranking se regenera al ejecutar la comparación estadística.

Tabla x - RF-024 — Consulta del ranking de modelos de la sesión

RF-025 — Consulta de la comparativa estadística de la sesión

Determinar si las diferencias entre modelos son reales o fruto del azar es una exigencia metodológica del proyecto. El sistema debe mostrar la matriz de significación estadística que compara los modelos de la sesión, con los p-valores del test de Wilcoxon sobre el AUC de los pliegues y, cuando la validación externa se ha ejecutado, la matriz del test de DeLong sobre las curvas ROC. Estas matrices permiten al investigador identificar de un vistazo qué diferencias son estadísticamente significativas y cuáles son atribuibles al azar. El requisito RF-025 recoge esta consulta.

Campo	Contenido
ID	RF-025
Nombre	Consulta de la comparativa estadística de la sesión
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar la matriz de significación estadística de la sesión, con los p-valores del test de Wilcoxon y, cuando proceda, los del test de DeLong sobre las curvas ROC de la validación externa.
Importancia	Alta
Estado	Aprobado
Comentarios	La interpretación de la significación debe tener en cuenta la potencia limitada de la prueba con el número de pliegues disponible, cuestión que se analiza en la parte experimental de la memoria; este requisito no fija un umbral de significación.

Tabla x - RF-025 — Consulta de la comparativa estadística de la sesión

RF-026 — Solicitud del recálculo de la comparativa estadística

La comparativa estadística de la sesión puede quedar incompleta en dos circunstancias: cuando un entrenamiento se interrumpe o falla a mitad de su ejecución, o cuando se incorporan nuevos resultados a una sesión ya finalizada. En esas circunstancias, el usuario debe poder solicitar la regeneración de la comparativa estadística y del ranking. El sistema debe recalcular el ranking y el test de Wilcoxon en segundo plano y notificar al usuario cuando el proceso finalice. El requisito RF-026 recoge esta operación de recuperación.

Campo	Contenido
ID	RF-026
Nombre	Solicitud del recálculo de la comparativa estadística
Objetivos relacionados	OBJ-005
Descripción	El sistema debe permitir al usuario solicitar el recálculo de la comparativa estadística de la sesión, regenerando el ranking y el test de Wilcoxon de forma asíncrona.
Importancia	Media
Estado	Aprobado
Comentarios	El usuario es notificado cuando el recálculo finaliza.

Tabla x - RF-026 — Solicitud del recálculo de la comparativa estadística

RF-027 — Ejecución del análisis de explicabilidad de un modelo

El pipeline automático genera el análisis de explicabilidad al completar el entrenamiento de cada modelo, pero ese análisis puede quedar sin ejecutarse si el entrenamiento se interrumpe o si se incorpora a la sesión un modelo que no llegó a analizarse. En esas circunstancias, el usuario debe poder ejecutar manualmente el análisis XAI cualitativo y cuantitativo del modelo, regenerando sus mapas y sus métricas. El requisito RF-027 recoge esta operación de recuperación.

Campo	Contenido
ID	RF-027
Nombre	Ejecución del análisis de explicabilidad de un modelo
Objetivos relacionados	OBJ-005
Descripción	El sistema debe permitir al usuario ejecutar manualmente el análisis de explicabilidad de un modelo de la sesión, regenerando sus mapas y sus métricas cuantitativas.
Importancia	Media
Estado	Aprobado
Comentarios	La ejecución manual complementa el análisis automático del pipeline.

Tabla x - RF-027 — Ejecución del análisis de explicabilidad de un modelo

RF-028 — Solicitud de la validación externa de la sesión

La validación externa constituye la prueba de generalización de los modelos. El usuario debe poder solicitar la evaluación de los modelos entrenados sobre el dataset externo de pacientes adultos, con los modelos congelados y sin ningún tipo de reaprendizaje. El sistema debe encolar la tarea, evaluar cada modelo sobre la cohorte externa calculando las cinco métricas y las curvas ROC, y aplicar el test de DeLong para comparar las curvas entre modelos. Dado su coste computacional, la validación externa se ejecuta de forma asíncrona. El requisito RF-028 recoge este comportamiento.

Campo	Contenido
ID	RF-028
Nombre	Solicitud de la validación externa de la sesión
Objetivos relacionados	OBJ-005, OBJ-010
Descripción	El sistema debe permitir al usuario solicitar la validación externa de la sesión, evaluando los modelos congelados sobre el dataset externo y aplicando el test de DeLong, de forma asíncrona.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación externa se encola y se procesa en segundo plano.

Tabla x - RF-028 — Solicitud de la validación externa de la sesión

RF-029 — Consulta de los resultados de la validación externa

Cuando la validación externa finaliza, el sistema debe presentar sus resultados: las métricas de cada modelo sobre la cohorte externa, las curvas ROC y la matriz de significación del test de DeLong. Estos datos determinan qué arquitecturas generalizan mejor a poblaciones y condiciones de adquisición distintas. El requisito RF-029 recoge esta consulta.

Campo	Contenido
ID	RF-029
Nombre	Consulta de los resultados de la validación externa
Objetivos relacionados	OBJ-005
Descripción	El sistema debe mostrar los resultados de la validación externa de la sesión: métricas sobre la cohorte externa, curvas ROC y matriz de significación del test de DeLong.
Importancia	Media
Estado	Aprobado
Comentarios	Los resultados solo están disponibles tras completar la validación externa.

Tabla x - RF-029 — Consulta de los resultados de la validación externa

RF-030 — Generación del informe PDF de la sesión

El laboratorio debe consolidar los resultados de una sesión en un informe descargable en PDF, que recoge la configuración del experimento, el ranking de modelos, la matriz de Wilcoxon, los resultados de la validación externa con su matriz de DeLong y las métricas de explicabilidad y calibración por modelo. Este informe permite archivar y compartir los resultados de la sesión. El requisito RF-030 recoge la generación del informe.

Campo	Contenido
ID	RF-030
Nombre	Generación del informe PDF de la sesión
Objetivos relacionados	OBJ-004, OBJ-003
Descripción	El sistema debe generar un informe descargable en PDF que consolide la configuración, las métricas, las comparativas estadísticas y las gráficas de la sesión de entrenamiento.
Importancia	Media
Estado	Aprobado
Comentarios	El informe integra ranking, matrices de significación, curvas ROC y métricas XAI.

Tabla x - RF-030 — Generación del informe PDF de la sesión

RF-031 — Renombrar una sesión de entrenamiento

El usuario debe poder dar a sus sesiones de entrenamiento un nombre más descriptivo para identificarlas mejor. El sistema debe permitir modificar el nombre de una sesión propia. El requisito RF-031 recoge esta operación.

Campo	Contenido
ID	RF-031
Nombre	Renombrar una sesión de entrenamiento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario autenticado modificar el nombre de una de sus sesiones de entrenamiento.
Importancia	Baja
Estado	Aprobado
Comentarios	El nuevo nombre no puede estar vacío y solo aplica a sesiones del propio usuario.

Tabla x - RF-031 — Renombrar una sesión de entrenamiento

RF-032 — Eliminar una sesión de entrenamiento

El usuario debe poder eliminar las sesiones de entrenamiento que ya no necesita. La eliminación retira la sesión y sus resultados del laboratorio del usuario. El requisito RF-032 recoge esta operación.

Campo	Contenido
ID	RF-032
Nombre	Eliminar una sesión de entrenamiento
Objetivos relacionados	OBJ-004
Descripción	El sistema debe permitir al usuario autenticado eliminar una de sus sesiones de entrenamiento y sus resultados asociados.
Importancia	Baja
Estado	Aprobado
Comentarios	Solo se pueden eliminar sesiones del propio usuario.

Tabla x - RF-032 — Eliminar una sesión de entrenamiento

12.1.1.5. Módulo de administración

Este módulo agrupa los requisitos del panel de administración, destinados al administrador de la plataforma. Su propósito es permitir el gobierno del sistema: la gestión y supervisión de las cuentas de usuario y de la actividad registrada. Los requisitos de este módulo se corresponden con los casos de uso CU-031 a CU-033 y se asocian al objetivo de administración de la plataforma (OBJ-011). La gestión de cuentas de usuario (RF-040) completa este módulo. Todas las funcionalidades de este módulo están restringidas al rol de administrador, que constituye el único perfil con acceso a estas operaciones.

RF-033 — Consulta del listado de usuarios

El administrador necesita una visión global de quién utiliza la plataforma. El sistema debe mostrar el listado de los usuarios registrados, de modo que el administrador pueda identificar cuentas y comprobar el estado de la plataforma. Esta consulta constituye la base de la supervisión del sistema y del resto de las operaciones de administración. El requisito RF-033 recoge este listado.

Campo	Contenido
ID	RF-033
Nombre	Consulta del listado de usuarios
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador consultar el listado de usuarios registrados en la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador. El acceso a los datos de otros usuarios queda acotado a la función de supervisión definida como excepción en RF-005 y se registra conforme al requisito de auditoría RNF-006.

Tabla x - RF-033 — Consulta del listado de usuarios

RF-034 — Consulta de las consultas de un usuario

La supervisión de la actividad registrada exige poder examinar la actividad de un usuario concreto. El sistema debe permitir al administrador seleccionar un usuario y consultar su historial de consultas de diagnóstico. Esta capacidad permite al administrador comprobar el uso que se hace de la plataforma y detectar posibles incidencias. El requisito RF-034 recoge esta consulta.

Campo	Contenido
ID	RF-034
Nombre	Consulta de las consultas de un usuario
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador consultar el historial de consultas de diagnóstico de un usuario concreto de la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador. El acceso al historial de otro usuario queda acotado a la función de supervisión definida como excepción en RF-005, se limita a las consultas no eliminadas (RF-016) y se registra conforme al requisito de auditoría RNF-006.

Tabla x - RF-034 — Consulta de las consultas de un usuario

RF-035 — Visualización del detalle de una consulta de un usuario

Para completar la supervisión, el administrador debe poder abrir el detalle completo de una consulta de un usuario, incluyendo la imagen, el resultado, la confianza y los metadatos asociados. Esta operación permite auditar un caso concreto ante cualquier incidencia. El requisito RF-035 recoge esta visualización.

Campo	Contenido
ID	RF-035
Nombre	Visualización del detalle de una consulta de un usuario
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador visualizar el detalle de una consulta de diagnóstico de un usuario de la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador. La visualización de la imagen de una consulta de otro usuario queda acotada a la función de supervisión definida como excepción en RF-005, se limita a las consultas no eliminadas (RF-016) y se registra conforme al requisito de auditoría RNF-006.

Tabla x - RF-035 — Visualización del detalle de una consulta de un usuario

12.1.1.6. Módulo transversal

Este módulo agrupa los requisitos que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma: la consulta y cancelación de los trabajos de la cola, que cubren los diagnósticos, los entrenamientos y las validaciones externas, y la personalización del tema visual de la interfaz. Los requisitos de este módulo se corresponden con los casos de uso CU-034 a CU-036 y se asocian al objetivo de ejecución asíncrona de tareas (OBJ-010) y, en el caso del tema visual, al objetivo de usabilidad e internacionalización de la interfaz (OBJ-009).

RF-036 — Consulta del estado de la cola de trabajos

El sistema ejecuta de forma asíncrona los diagnósticos, los entrenamientos y las validaciones externas, y el usuario necesita conocer en cada momento el estado de sus trabajos. El sistema debe mostrar un panel de la cola de trabajos que refleje el estado de cada tarea —pendiente, en ejecución, completada o fallida— y que se actualice automáticamente al menos cada cinco segundos, sin exigir que el usuario recargue la página. Esta visibilidad es la que permite al usuario saber cuándo estará disponible un resultado o si un trabajo ha fallado. El requisito RF-036 recoge este panel.

Campo	Contenido
ID	RF-036
Nombre	Consulta del estado de la cola de trabajos
Objetivos relacionados	OBJ-010
Descripción	El sistema debe mostrar al usuario el estado de los trabajos de la cola (diagnósticos, entrenamientos y validaciones externas), actualizado automáticamente al menos cada cinco segundos.
Importancia	Media
Estado	Aprobado
Comentarios	El panel cubre todos los tipos de trabajo y muestra su estado.

Tabla x - RF-036 — Consulta del estado de la cola de trabajos

RF-037 — Cancelación de un trabajo de la cola

En determinadas circunstancias, el usuario puede necesitar detener un trabajo que ha encolado por error o que ya no le interesa. El sistema debe permitir cancelar un trabajo pendiente, de modo que no llegue a ejecutarse, y, cuando resulte técnicamente posible, interrumpir un trabajo en ejecución. Un entrenamiento lanzado por error no debe quedar monopolizando la GPU durante horas sin posibilidad de detenerlo, especialmente en una plataforma multiusuario con un único equipo de cómputo. La cancelación de un trabajo en ejecución puede dejar resultados parciales, y el sistema debe indicar que el trabajo quedó cancelado y no completado. El requisito RF-037 recoge esta operación.

Campo	Contenido
ID	RF-037
Nombre	Cancelación de un trabajo de la cola
Objetivos relacionados	OBJ-010
Descripción	El sistema debe permitir al usuario cancelar un trabajo pendiente de la cola y, cuando resulte técnicamente posible, interrumpir un trabajo en ejecución.
Importancia	Media
Estado	Aprobado
Comentarios	La cancelación de un trabajo pendiente evita su ejecución; la de un trabajo en ejecución puede dejar resultados parciales, que quedan marcados como cancelados y no como completados.

Tabla x - RF-037 — Cancelación de un trabajo pendiente de la cola

RF-038 — Cambio del tema visual de la interfaz

El usuario debe poder personalizar la apariencia de la interfaz eligiendo entre el tema claro y el tema oscuro, según su preferencia visual. El cambio debe aplicarse de forma inmediata en toda la interfaz. El requisito RF-038 recoge esta personalización.

Campo	Contenido
ID	RF-038
Nombre	Cambio del tema visual de la interfaz
Objetivos relacionados	OBJ-009
Descripción	El sistema debe permitir al usuario autenticado alternar entre el tema claro y el tema oscuro de la interfaz.
Importancia	Baja
Estado	Aprobado
Comentarios	La preferencia se aplica de forma inmediata en toda la interfaz.

Tabla x - RF-038 — Cambio del tema visual de la interfaz

RF-039 — Generación del informe PDF del diagnóstico

Cada consulta de diagnóstico debe poder descargarse como un informe en PDF que el profesional pueda archivar, imprimir o incorporar a su flujo de trabajo habitual. El informe debe recoger la identificación de la consulta, la imagen original, la predicción, el nivel de confianza, el modelo empleado y los mapas de explicabilidad. Este requisito cubre el informe individual de la consulta; el informe consolidado de la sesión de entrenamiento se recoge en el requisito RF-030. El requisito RF-039 pertenece al módulo de diagnóstico clínico.

Campo	Contenido
ID	RF-039
Nombre	Generación del informe PDF del diagnóstico
Objetivos relacionados	OBJ-001, OBJ-003
Descripción	El sistema debe generar un informe descargable en PDF para cada consulta de diagnóstico, que recoja la imagen original, la predicción, el nivel de confianza, el modelo empleado y los mapas de explicabilidad.
Importancia	Media
Estado	Aprobado
Comentarios	Este requisito completa la cobertura del objetivo de generación de informes (OBJ-003), junto con el informe de sesión (RF-030). Su verificación se apoya en la prueba PU-017.

Tabla x - RF-039 — Generación del informe PDF del diagnóstico

RF-040 — Gestión de cuentas de usuario por el administrador

La administración de la plataforma no se limita a la consulta: el objetivo OBJ-011 exige gestionar las cuentas de usuario. El administrador debe poder desactivar una cuenta, cambiar el rol de un usuario y eliminar una cuenta, de modo que el gobierno del sistema no quede reducido a la lectura de datos. La gestión de cuentas se ejecuta de forma controlada: las operaciones quedan registradas conforme a la auditoría administrativa (RNF-006) y respetan el aislamiento de datos entre usuarios (RF-005). El requisito RF-040 pertenece al módulo de administración.

Campo	Contenido
ID	RF-040
Nombre	Gestión de cuentas de usuario por el administrador
Objetivos relacionados	OBJ-011
Descripción	El sistema debe permitir al administrador gestionar las cuentas de usuario de la plataforma: desactivar una cuenta, cambiar el rol de un usuario y eliminar una cuenta, con registro de la operación conforme al requisito RNF-030.
Importancia	Media
Estado	Aprobado
Comentarios	Este requisito completa la cobertura del objetivo de administración de la plataforma (OBJ-011), que hasta ahora solo contaba con operaciones de consulta.

Tabla x - RF-040 — Gestión de cuentas de usuario por el administrador

RF-041 — Limitación de entrenamientos simultáneos y de la cola de trabajos

El entrenamiento de modelos compite por los recursos computacionales de la estación de trabajo, que cuenta con una única GPU. Para que un uso intensivo del laboratorio por parte de un usuario no degrade el servicio del resto, y para que un lanzamiento accidental no sature la plataforma, el sistema debe limitar el número de entrenamientos que se ejecutan simultáneamente y el número de trabajos de entrenamiento que pueden permanecer encolados. Los entrenamientos que excedan el límite de ejecución concurrente deben esperar en cola y ejecutarse cuando haya recursos disponibles. El requisito RF-041 pertenece al módulo de laboratorio MLOps.

Campo	Contenido
ID	RF-041
Nombre	Limitación de entrenamientos simultáneos y de la cola
Objetivos relacionados	OBJ-004, OBJ-010
Descripción	El sistema debe limitar el número de entrenamientos que se ejecutan simultáneamente en la estación de trabajo y el número de trabajos de entrenamiento encolados, de modo que la competición por la GPU no degrade el servicio para el resto de los usuarios y un lanzamiento masivo no sature la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Esta limitación es global a la plataforma y complementa el requisito de disponibilidad del servicio (RNF-017). Los valores concretos de los límites se configuran en el entorno.

Tabla x - RF-041 — Limitación de entrenamientos simultáneos y de la cola de trabajos

12.1.1.7. Matriz de trazabilidad entre requisitos funcionales y objetivos del sistema

La matriz de trazabilidad que se presenta a continuación relaciona cada requisito funcional con el objetivo u objetivos del sistema a los que da respuesta. Su finalidad es permitir una verificación en ambas direcciones: que todos los objetivos del sistema quedan cubiertos por al menos un requisito funcional, y que todo requisito funcional está justificado por al menos un objetivo del sistema. Esta verificación garantiza que el catálogo de requisitos es completo, que no existe ningún objetivo huérfano sin capacidades asociadas y que ninguna capacidad implementada carece de fundamento. Las columnas representan los objetivos del sistema definidos en el capítulo de ámbito del sistema (OBJ-001 a OBJ-011) y las filas los requisitos funcionales; una X en la intersección indica que el requisito contribuye a la consecución del objetivo correspondiente. Conviene señalar el criterio aplicado a dos requisitos transversales: el aislamiento de datos (RF-005) se asocia al objetivo de acceso seguro (OBJ-007), porque protege a los usuarios entre sí en su uso ordinario; el requisito de roles y control de acceso (RF-006) se asocia además al objetivo de administración (OBJ-011), porque define el rol de administrador sobre el que se sustenta la gestión de la plataforma.

[illegible]

RF	Denominación	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RF-011	Visualización del resultado del diagnóstico	X										
RF-012	Visualización de los mapas de explicabilidad	X										
RF-013	Consultar el historial de consultas		X						X			
RF-014	Visualización del detalle de una consulta del historial		X						X			
RF-015	Renombrar una consulta del historial		X									
RF-016	Eliminar una consulta del historial		X									
RF-017	Acceso al laboratorio de entrenamiento				X							
RF-018	Conversación con el asistente para configurar un experimento				X							
RF-019	Selección de la carpeta del dataset				X							
RF-020	Lanzamiento de un experimento de entrenamiento				X		X				X	
RF-021	Consulta de las sesiones de entrenamiento				X				X			
RF-022	Consulta de los resultados de un modelo de la sesión					X						
RF-023	Visualización de los mapas de explicabilidad de un modelo					X						

[illegible]

RF	Denominación	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RF-036	Consulta del estado de la cola de trabajos										X	
RF-037	Cancelación de un trabajo de la cola										X	
RF-038	Cambio del tema visual de la interfaz									X		
RF-039	Generación del informe PDF del diagnóstico	X		X								
RF-040	Gestión de cuentas de usuario por el administrador											X
RF-041	Limitación de entrenamientos simultáneos y de la cola				X						X	

Tabla x - Matriz de trazabilidad entre requisitos funcionales y objetivos del sistema

La matriz confirma que todos los objetivos del sistema quedan cubiertos por al menos un requisito funcional y que cada requisito está justificado por uno o varios objetivos: los objetivos de diagnóstico asistido con inteligencia artificial explicable (OBJ-001), gestión del historial (OBJ-002), laboratorio de entrenamiento (OBJ-004) y evaluación rigurosa de los modelos (OBJ-005) concentran la mayor parte de los requisitos. El objetivo de generación de informes (OBJ-003) se cubre con el informe de sesión (RF-030) y con el informe individual del diagnóstico (RF-039); el objetivo de administración de la plataforma (OBJ-011) se cubre con las operaciones de consulta (RF-033 a RF-035) y con la gestión de cuentas de usuario (RF-040); y el objetivo de ejecución asíncrona de tareas (OBJ-010) se cubre con los requisitos de la cola de trabajos (RF-036, RF-037 y RF-041). La verificación de la cobertura entre requisitos y casos de uso se aborda, de forma análoga, en el apartado correspondiente del modelado de casos de uso.

12.1.2. Requisitos no funcionales del sistema

Los requisitos no funcionales describen las condiciones de calidad en las que el sistema debe satisfacer su comportamiento, con independencia de las funcionalidades concretas que ofrece. A diferencia de los requisitos funcionales, que responden a la pregunta de qué debe hacer el sistema, los requisitos no funcionales responden a la pregunta de cómo debe comportarse: qué nivel de seguridad debe garantizar, cómo debe responder en términos de rendimiento, qué experiencia de uso debe ofrecer o cómo debe salvaguardar los datos. Cada requisito no funcional se formula con un umbral o una condición comprobable, de modo que pueda verificarse de forma objetiva; cuando el umbral depende de una configuración, se indica el valor aplicado en este proyecto. Estos requisitos condicionan de forma transversal todo el diseño e influyen de manera decisiva en la aceptación del sistema por parte de sus usuarios finales, especialmente en un ámbito clínico donde la seguridad, la fiabilidad y la privacidad no son negociables.

Los requisitos no funcionales se han obtenido de los objetivos del sistema, de las restricciones del entorno tecnológico descritas en el análisis y de las buenas prácticas recogidas en la literatura y en los planes de apoyo del proyecto. Se organizan en siete grupos: seguridad de la plataforma y del acceso, confidencialidad y protección de los datos, rendimiento y capacidad de respuesta, sencillez de uso y accesibilidad, robustez y disponibilidad del servicio, persistencia y salvaguarda de los datos, y reproducibilidad del sistema. Al final de la sección se incluye, por un lado, una subsección dedicada a los compromisos del proceso de desarrollo y a los entregables del proyecto, que no constituyen requisitos del sistema y se presentan por separado para no mezclar ambas naturalezas, y por otro, la matriz de trazabilidad que permite verificar, de forma análoga a la de los requisitos funcionales, que todos los objetivos del sistema con implicaciones de calidad quedan cubiertos por al menos un requisito no funcional.

12.1.2.1. Seguridad de la plataforma y del acceso

Este grupo agrupa los requisitos no funcionales relacionados con la protección de la plataforma y de las credenciales de sus usuarios. La seguridad es un requisito transversal de primer orden: un fallo en el tratamiento de las contraseñas, en la gestión de las sesiones o en la protección frente a los ataques web más comunes podría comprometer la confidencialidad de los datos clínicos gestionados por el sistema. Los requisitos de este grupo se asocian a los objetivos de acceso seguro y personalizado al sistema (OBJ-007) y de administración de la plataforma (OBJ-011). El marco de referencia de este grupo es el catálogo de riesgos de aplicaciones web de OWASP (OWASP, 2021), descrito en el análisis, por lo que los requisitos cubren, al menos, los riesgos más críticos: inyección, exposición de datos sensibles, pérdida de control de acceso, configuraciones inseguras, gestión vulnerable de componentes, autenticación fallida y falsificación de peticiones.

RNF-001 — Cifrado de contraseñas

La contraseña es la credencial principal con la que el usuario accede al sistema, y su almacenamiento debe protegerla frente a un acceso no autorizado a la base de datos. El sistema debe almacenar la contraseña de forma que nunca pueda recuperarse en claro: debe aplicarse una función de hash no reversible con un factor de coste configurable que dificulte los ataques de fuerza bruta. Esta garantía condiciona directamente la confianza que los usuarios pueden depositar en la plataforma. El requisito RNF-001 recoge esta exigencia.

Campo	Contenido
ID	RNF-001
Nombre	Cifrado de contraseñas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe almacenar las contraseñas de los usuarios mediante una función de hash no reversible con un factor de coste configurable, de modo que ninguna contraseña quede en claro y que la verificación se realice siempre sobre el valor derivado.
Importancia	Alta
Estado	Aprobado
Comentarios	Esta exigencia complementa al requisito funcional de registro de usuario. La elección concreta del algoritmo es una decisión de diseño.

Tabla x - RNF-001 — Cifrado de contraseñas

RNF-002 — Gestión segura de la sesión

Una vez autenticado el usuario, la sesión debe gestionarse de forma que sus credenciales no queden expuestas en cada petición. El sistema debe basar la sesión en credenciales que el navegador no pueda leer desde scripts y que se asocien a un periodo de validez limitado, y debe permitir renovar el acceso sin que el usuario tenga que volver a autenticarse mediante una credencial de refresco de vida limitada y revocable. Esta gestión evita que las credenciales viajen o se almacenen de forma vulnerable. El requisito RNF-002 recoge esta exigencia.

Campo	Contenido
ID	RNF-002
Nombre	Gestión segura de la sesión
Objetivos relacionados	OBJ-007
Descripción	El sistema debe gestionar las sesiones mediante credenciales que el navegador no pueda leer desde scripts, con validez limitada, y debe permitir renovar el acceso sin una nueva autenticación mediante una credencial de refresco de vida limitada y revocable.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de cookies y de rotación de la credencial de refresco son decisiones de diseño.

Tabla x - RNF-002 — Gestión segura de la sesión

RNF-003 — Protección frente a la falsificación de peticiones entre sitios

Las aplicaciones web que autentican a sus usuarios mediante cookies son susceptibles de ataques de falsificación de peticiones entre sitios (CSRF), en los que una página maliciosa induce al navegador del usuario a enviar peticiones no deseadas al sistema con sus credenciales. El sistema debe protegerse frente a este vector exigiendo, para toda petición que modifique el estado del sistema, la presencia de un token de protección que no pueda ser forjado por un tercero. El requisito RNF-003 recoge esta exigencia.

Campo	Contenido
ID	RNF-003
Nombre	Protección frente a la falsificación de peticiones entre sitios
Objetivos relacionados	OBJ-007
Descripción	El sistema debe proteger las peticiones que modifican el estado frente a ataques de falsificación de peticiones entre sitios, exigiendo un token de protección válido en cada petición.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de generación y verificación del token es una decisión de diseño.

Tabla x - RNF-003 — Protección frente a la falsificación de peticiones entre sitios

RNF-004 — Limitación de las peticiones de acceso

El proceso de inicio de sesión es el principal objetivo de los ataques de fuerza bruta, en los que un atacante prueba contraseñas de forma masiva hasta acertar. El sistema debe limitar el número de intentos de acceso fallidos desde una misma dirección y, al superar el umbral, bloquear nuevas peticiones durante un intervalo de tiempo definido. En este proyecto el umbral es de cinco peticiones fallidas por minuto desde una misma dirección, con un bloqueo de diez minutos. El requisito RNF-004 recoge esta exigencia.

Campo	Contenido
ID	RNF-004
Nombre	Limitación de las peticiones de acceso
Objetivos relacionados	OBJ-007
Descripción	El sistema debe limitar a cinco peticiones fallidas por minuto los intentos de acceso desde una misma dirección y bloquear nuevas peticiones durante diez minutos cuando se supere el umbral.
Importancia	Media
Estado	Aprobado
Comentarios	Se aplica un límite estricto sobre el inicio de sesión y un límite general sobre el resto de las peticiones. Los valores pueden ajustarse mediante configuración, pero los indicados son los aplicados en este proyecto.

Tabla x - RNF-004 — Limitación de las peticiones de acceso

RNF-005 — Cabeceras de seguridad en las respuestas

Además de los mecanismos de autenticación, el sistema debe reforzar la seguridad del navegador mediante cabeceras HTTP de protección. Estas cabeceras restringen las políticas de contenido que el navegador puede cargar, fuerzan el uso de conexiones seguras y bloquean la inclusión de la aplicación en marcos de terceros. Aunque el usuario no las percibe directamente, reducen la superficie de ataque de la plataforma. El requisito RNF-005 recoge esta exigencia.

Campo	Contenido
ID	RNF-005
Nombre	Cabeceras de seguridad en las respuestas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe incluir en sus respuestas cabeceras HTTP de seguridad que restrinjan las políticas de contenido del navegador, fuercen el uso de conexiones seguras y bloqueen la inclusión en marcos de terceros.
Importancia	Media
Estado	Aprobado
Comentarios	El conjunto concreto de cabeceras y sus políticas se define en la configuración del middleware de seguridad, como decisión de diseño.

Tabla x - RNF-005 — Cabeceras de seguridad en las respuestas

RNF-006 — Auditoría de la actividad administrativa

Las operaciones de administración afectan al conjunto de la plataforma —la gestión de los usuarios, la supervisión de las consultas y la configuración general del sistema—, por lo que deben poder supervisarse de forma fiable. El sistema debe registrar la actividad administrativa de modo que quede constancia de qué operación se realizó, quién la ejecutó y en qué momento. Este registro, de acceso restringido al propio administrador, constituye un control de gobierno que permite detectar usos indebidos y auditar el funcionamiento de la plataforma. El requisito RNF-006 recoge esta exigencia.

Campo	Contenido
ID	RNF-006
Nombre	Auditoría de la actividad administrativa
Objetivos relacionados	OBJ-007, OBJ-011
Descripción	El sistema debe registrar la actividad de administración, de modo que quede constancia de las operaciones realizadas, del usuario que las ejecutó y del momento en que se produjeron, con acceso restringido al administrador.
Importancia	Media
Estado	Aprobado
Comentarios	El registro constituye un control de gobierno y auditoría de la plataforma. Se asocia tanto a la seguridad del acceso como al gobierno de la plataforma.

Tabla x - RNF-006 — Auditoría de la actividad administrativa

RNF-007 — Validación y saneamiento de las entradas

La inyección de código, en particular la inyección SQL, es el riesgo de seguridad más crítico de las aplicaciones web según el catálogo de OWASP. El sistema debe tratar toda entrada procedente del usuario como no confiable y procesarla mediante consultas parametrizadas que separen la instrucción de sus valores, sin concatenar nunca el contenido recibido en una sentencia SQL. Esta garantía debe aplicarse en todos los puntos en los que el sistema accede a la base de datos. El requisito RNF-007 recoge esta exigencia.

Campo	Contenido
ID	RNF-007
Nombre	Validación y saneamiento de las entradas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe tratar las entradas del usuario como no confiables y procesarlas mediante consultas parametrizadas que eviten la inyección de código, en particular la inyección SQL, en todos los accesos a la base de datos.
Importancia	Alta
Estado	Aprobado
Comentarios	La verificación de este requisito se apoya en las pruebas de seguridad del plan de pruebas y en el análisis estático del código.

Tabla x - RNF-007 — Validación y saneamiento de las entradas

RNF-008 — Subida de ficheros segura

El sistema acepta la subida de imágenes de diagnóstico, por lo que debe validar el contenido real del fichero y no solo su extensión o su tipo declarado. El sistema debe comprobar que el contenido corresponde a una imagen admitida (JPEG o PNG) y rechazar cualquier fichero cuyo contenido no sea el esperado, con independencia de su nombre o de su tipo MIME declarado. El requisito RNF-008 recoge esta exigencia.

Campo	Contenido
ID	RNF-008
Nombre	Subida de ficheros segura
Objetivos relacionados	OBJ-007
Descripción	El sistema debe validar el contenido real de los ficheros de imagen subidos, y no solo su extensión o su tipo MIME declarado, rechazando cualquier fichero cuyo contenido no corresponda a una imagen admitida.
Importancia	Alta
Estado	Aprobado
Comentarios	Complementa al requisito funcional RF-008, que fija el formato y el tamaño máximo de las imágenes.

Tabla x - RNF-008 — Subida de ficheros segura

RNF-009 — Registro de eventos de seguridad

La detección y el análisis de incidentes requieren un registro de los eventos relevantes de seguridad. El sistema debe registrar los acontecimientos relacionados con la seguridad —intentos de acceso fallidos, bloqueos por límite de peticiones, operaciones de administración y errores de autenticación—, con la fecha, la dirección de origen y, cuando proceda, el usuario implicado. Este registro debe conservarse de forma que pueda consultarse posteriormente para auditar una incidencia. El requisito RNF-009 recoge esta exigencia.

Campo	Contenido
ID	RNF-009
Nombre	Registro de eventos de seguridad
Objetivos relacionados	OBJ-007, OBJ-011
Descripción	El sistema debe registrar los eventos de seguridad relevantes —accesos fallidos, bloqueos por límite de peticiones y operaciones de administración— con su fecha, su origen y, cuando proceda, el usuario implicado.
Importancia	Media
Estado	Aprobado
Comentarios	El registro de eventos de seguridad complementa al requisito de auditoría de la actividad administrativa (RNF-006).

Tabla x - RNF-009 — Registro de eventos de seguridad

RNF-010 — Política de contraseñas

La fortaleza de las credenciales depende en parte de las contraseñas elegidas por los usuarios. El sistema debe exigir una política mínima de contraseñas en el registro y en el cambio de contraseña: una longitud mínima de ocho caracteres y la comprobación de que no sea idéntica a datos personales básicos del usuario, como el nombre de usuario o el correo electrónico. El requisito RNF-010 recoge esta exigencia.

Campo	Contenido
ID	RNF-010
Nombre	Política de contraseñas
Objetivos relacionados	OBJ-007
Descripción	El sistema debe exigir una contraseña de al menos ocho caracteres en el registro y en el cambio de contraseña, y rechazar las contraseñas idénticas a datos personales básicos del usuario.
Importancia	Media
Estado	Aprobado
Comentarios	La política complementa al requisito de cifrado de contraseñas (RNF-001).

Tabla x - RNF-010 — Política de contraseñas

RNF-011 — Gestión de secretos

El sistema depende de credenciales externas que no deben quedar incrustadas en el código ni en los ficheros versionados, como la clave de la API del asistente conversacional, la clave de firma de las sesiones o las credenciales de acceso a la base de datos. El sistema debe obtener estos secretos de la configuración del entorno y no exponerlos en las respuestas, en los registros ni en la interfaz. El requisito RNF-011 recoge esta exigencia.

Campo	Contenido
ID	RNF-011
Nombre	Gestión de secretos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe obtener las credenciales externas (clave de la API del asistente, clave de firma y credenciales de la base de datos) de la configuración del entorno, sin incrustarlas en el código ni exponerlas en respuestas, registros o interfaz.
Importancia	Alta
Estado	Aprobado
Comentarios	Esta exigencia cubre la clave de la API de Groq y el resto de secretos del entorno.

Tabla x - RNF-011 — Gestión de secretos

12.1.2.2. Confidencialidad y protección de los datos

Este grupo agrupa los requisitos no funcionales relacionados con la privacidad y la protección de los datos que el sistema trata. El sistema maneja información de origen médico, aunque anonimizada, y su tratamiento debe ajustarse a la normativa vigente en materia de protección de datos personales. Los requisitos de este grupo se asocian al objetivo de acceso seguro y personalizado al sistema (OBJ-007) y se apoyan en el análisis de la normativa presentado en el capítulo de ámbito del sistema.

RNF-012 — Cumplimiento del Reglamento General de Protección de Datos

El sistema gestiona cuentas de usuarios registrados y procesa imágenes de origen médico, por lo que su tratamiento de datos debe ajustarse a la normativa europea y nacional vigente: el Reglamento General de Protección de Datos (RGPD) y su adaptación española, la Ley Orgánica 3/2018. Esta normativa establece principios como la minimización de los datos, la limitación de la finalidad y la garantía de privacidad de los interesados, y condiciona el diseño del registro de usuarios, el almacenamiento de sus datos y el tratamiento de las imágenes subidas a la plataforma. El sistema debe observar estos principios en el conjunto de su funcionamiento, no como un requisito puntual, sino como una condición transversal. El requisito RNF-012 recoge esta exigencia.

Campo	Contenido
ID	RNF-012
Nombre	Cumplimiento del Reglamento General de Protección de Datos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe cumplir la normativa vigente en materia de protección de datos personales, en particular el Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica 3/2018, en el tratamiento de los datos de los usuarios y de las imágenes gestionadas.
Importancia	Alta
Estado	Aprobado
Comentarios	El cumplimiento condiciona el diseño del registro, el almacenamiento y el tratamiento de los datos.

Tabla x - RNF-012 — Cumplimiento del Reglamento General de Protección de Datos

RNF-013 — Anonimización de los conjuntos de datos

Los conjuntos de datos utilizados para el entrenamiento y la validación de los modelos proceden de repositorios públicos y están anonimizados, es decir, desprovistos de cualquier información que permita identificar a los pacientes. El sistema debe operar sobre los conjuntos de datos de entrenamiento y de validación externa empleados en el laboratorio, que son públicos y anónimos. Conviene precisar el alcance de esta garantía: el sistema no puede verificar, por sí mismo, que una imagen subida por un facultativo en el flujo de diagnóstico esté anonimizada, como reconoce el requisito RNF-014; esa verificación recae en el responsable de la captura. Por ello, este requisito se limita a los conjuntos de datos gestionados por el propio sistema. El requisito RNF-013 recoge esta exigencia.

Campo	Contenido
ID	RNF-013
Nombre	Anonimización de los conjuntos de datos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe operar exclusivamente sobre conjuntos de datos de entrenamiento y de validación externa que sean públicos y estén anonimizados. Esta garantía se limita a los conjuntos gestionados por el sistema y no puede extenderse a las imágenes subidas por los usuarios en el diagnóstico.
Importancia	Alta
Estado	Aprobado
Comentarios	La verificación de la anonimización de las imágenes subidas por los facultativos no es realizable por el sistema y recae en el responsable de la captura, como señala el requisito RNF-014.

Tabla x - RNF-013 — Anonimización de los conjuntos de datos

RNF-014 — Exclusión de datos personales identificables

El sistema no debe almacenar ni procesar datos de pacientes individuales identificables. Las radiografías que los facultativos suben para su diagnóstico se conservan asociadas a la consulta del profesional que realizó el diagnóstico, de modo que pueden recuperarse desde el historial (RF-008), sin exponer información del paciente en la interfaz. Es responsabilidad del personal clínico garantizar la anonimización previa de las imágenes que introduce en el sistema, dado que el sistema no dispone de un mecanismo que la verifique. El requisito RNF-014 recoge esta exigencia.

Campo	Contenido
ID	RNF-014
Nombre	Exclusión de datos personales identificables
Objetivos relacionados	OBJ-007
Descripción	El sistema no debe almacenar ni procesar datos de pacientes individuales identificables, y debe asociar los artefactos generados a la cuenta del profesional que realizó la consulta, conservando las imágenes mientras exista la consulta asociada.
Importancia	Alta
Estado	Aprobado
Comentarios	La anonimización previa de las imágenes es responsabilidad del personal clínico, dado que el sistema no puede verificarla. La conservación de las imágenes está vinculada a la consulta y a su recuperación desde el historial, conforme al requisito RF-008.

Tabla x - RNF-014 — Exclusión de datos personales identificables

RNF-015 — Confidencialidad de las comunicaciones

Las comunicaciones entre el navegador y el servidor transportan credenciales de acceso y datos de las consultas, por lo que deben protegerse frente a su interceptación. El sistema debe servir sus páginas mediante un canal de comunicación cifrado, de modo que ni las credenciales ni los datos transmitidos puedan ser leídos por un tercero durante el tránsito por la red. Esta protección es una condición de seguridad transversal que afecta a todas las comunicaciones de la plataforma. El requisito RNF-015 recoge esta exigencia.

Campo	Contenido
ID	RNF-015
Nombre	Confidencialidad de las comunicaciones
Objetivos relacionados	OBJ-007
Descripción	El sistema debe proteger las comunicaciones entre el cliente y el servidor mediante un canal de comunicación cifrado, de modo que las credenciales y los datos no viajen en claro.
Importancia	Alta
Estado	Aprobado
Comentarios	El mecanismo concreto de cifrado de las comunicaciones es una decisión de diseño.

Tabla x - RNF-015 — Confidencialidad de las comunicaciones

RNF-016 — Retención y borrado de los datos

Los datos almacenados, en particular las imágenes de origen clínico y sus artefactos, no deben conservarse indefinidamente. El sistema debe disponer de una política de retención que fije el plazo de conservación de las consultas y de sus artefactos, y debe soportar el borrado definitivo de una consulta cuando el usuario lo solicita, conforme al derecho de supresión del RGPD (RF-016). El requisito RNF-016 recoge esta exigencia.

Campo	Contenido
ID	RNF-016
Nombre	Retención y borrado de los datos
Objetivos relacionados	OBJ-007
Descripción	El sistema debe aplicar una política de retención de los datos y de sus artefactos, y debe permitir el borrado definitivo de una consulta y de sus ficheros asociados cuando el usuario lo solicita, en coherencia con el requisito RF-016 y con el derecho de supresión.
Importancia	Alta
Estado	Aprobado
Comentarios	El borrado debe abarcar tanto los registros como los ficheros de imagen, explicación e informe asociados a la consulta.

Tabla x - RNF-016 — Retención y borrado de los datos

RNF-017 — Cifrado de los datos en reposo

El sistema almacena imágenes de origen clínico y credenciales de usuarios, por lo que los datos persistidos deben estar protegidos ante un acceso no autorizado al almacenamiento. El sistema debe conservar la información sensible cifrada en reposo, de modo que una exposición de los ficheros o de la base de datos no revele directamente su contenido. El requisito RNF-017 recoge esta exigencia.

Campo	Contenido
ID	RNF-017
Nombre	Cifrado de los datos en reposo
Objetivos relacionados	OBJ-007
Descripción	El sistema debe conservar cifrados en reposo los datos sensibles que persiste, incluidas las imágenes de diagnóstico y las credenciales, de modo que una exposición del almacenamiento no revele su contenido.
Importancia	Media
Estado	Aprobado
Comentarios	Las contraseñas se protegen mediante el requisito RNF-001; este requisito cubre el resto de datos sensibles persistidos.

Tabla x - RNF-017 — Cifrado de los datos en reposo

RNF-018 — Tratamiento de datos por terceros

El sistema envía al proveedor externo del asistente conversacional los mensajes que el usuario introduce en el laboratorio para configurar un experimento. Aunque el contenido de esas conversaciones se refiere a parámetros técnicos del experimento y no a datos clínicos de pacientes, constituye una transferencia de información del usuario a un tercero. El sistema debe limitar esa transferencia al contenido necesario para la configuración del experimento, no ofrecer campos para introducir datos personales y dejar constancia del tratamiento, de modo que el responsable pueda formalizar, en cada despliegue, las condiciones de protección de datos con el proveedor. El requisito RNF-018 recoge esta exigencia.

Campo	Contenido
ID	RNF-018
Nombre	Tratamiento de datos por terceros
Objetivos relacionados	OBJ-007
Descripción	El sistema debe limitar la transferencia al proveedor externo del asistente conversacional al contenido necesario para la configuración del experimento, sin ofrecer campos para datos personales, y dejar constancia de ese tratamiento para que pueda formalizarse con el proveedor.
Importancia	Media
Estado	Aprobado
Comentarios	El tratamiento por el tercero debe valorarse en cada despliegue, conforme al análisis de la normativa del capítulo 11.

Tabla x - RNF-018 — Tratamiento de datos por terceros

12.1.2.3. Rendimiento y capacidad de respuesta

Este grupo agrupa los requisitos no funcionales relacionados con el rendimiento del sistema y con su capacidad para responder de forma ágil a los usuarios. El rendimiento condiciona directamente la experiencia de uso: un diagnóstico que tarda demasiado, una interfaz que se bloquea durante un entrenamiento o una plataforma que se degrada ante varios usuarios simultáneos comprometerían la utilidad de la herramienta en un entorno clínico real. Los requisitos de este grupo se asocian a los objetivos de diagnóstico asistido con inteligencia artificial explicable (OBJ-001) y de ejecución asíncrona de tareas de larga duración (OBJ-010).

RNF-019 — Tiempo de respuesta de la inferencia

El diagnóstico asistido se realiza en el momento de la consulta, por lo que la inferencia debe completarse en un tiempo acotado. En este proyecto se fija que la inferencia de un diagnóstico con una arquitectura ya cargada debe completarse en menos de quince segundos, y que el envío de la petición debe devolver el identificador del trabajo encolado en menos de dos segundos. La primera consulta con una arquitectura puede requerir un tiempo adicional de carga de los pesos del modelo, que no se computa en el umbral anterior. El requisito RNF-019 recoge esta exigencia.

Campo	Contenido
ID	RNF-019
Nombre	Tiempo de respuesta de la inferencia
Objetivos relacionados	OBJ-001
Descripción	El sistema debe completar la inferencia de un diagnóstico con una arquitectura ya cargada en menos de quince segundos y devolver el identificador del trabajo encolado en menos de dos segundos.
Importancia	Alta
Estado	Aprobado
Comentarios	La primera carga de una arquitectura puede exceder el umbral y no se computa en él. La reutilización de los modelos cargados en memoria es una solución de diseño, no el requisito en sí.

Tabla x - RNF-019 — Tiempo de respuesta de la inferencia

RNF-020 — Ejecución sin bloqueo de la interfaz

Las tareas de larga duración —entrenamientos, análisis de explicabilidad y validación externa— pueden prolongarse durante horas, y durante ese tiempo la plataforma debe seguir respondiendo al usuario. El sistema debe ejecutar estas tareas de forma asíncrona, de modo que la interfaz permanezca operativa y el usuario pueda continuar utilizando la plataforma, consultar el estado de sus trabajos o lanzar nuevas operaciones. Sin esta condición, un único entrenamiento bloquearía el acceso a toda la plataforma durante horas. El requisito RNF-020 recoge esta exigencia.

Campo	Contenido
ID	RNF-020
Nombre	Ejecución sin bloqueo de la interfaz
Objetivos relacionados	OBJ-010
Descripción	El sistema debe ejecutar las tareas de larga duración de forma asíncrona, de modo que la interfaz permanezca operativa durante su ejecución.
Importancia	Alta
Estado	Aprobado
Comentarios	La cola de trabajos garantiza que la interfaz no se bloquee durante los procesos computacionales.

Tabla x - RNF-020 — Ejecución sin bloqueo de la interfaz

RNF-021 — Capacidad de acceso concurrente

La plataforma está concebida para ser utilizada por varios profesionales de forma simultánea. En este proyecto se fija que el sistema debe soportar al menos diez usuarios autenticados concurrentes realizando operaciones de consulta y de gestión del historial sin que el tiempo de respuesta de esas operaciones se degrade por encima del doble del tiempo con carga baja, y debe preservar el aislamiento de datos entre usuarios incluso bajo esa carga. El requisito RNF-021 recoge esta exigencia.

Campo	Contenido
ID	RNF-021
Nombre	Capacidad de acceso concurrente
Objetivos relacionados	OBJ-010
Descripción	El sistema debe soportar al menos diez usuarios autenticados concurrentes en operaciones de consulta y gestión del historial sin degradar el tiempo de respuesta por encima del doble del tiempo con carga baja, preservando el aislamiento de datos entre usuarios.
Importancia	Media
Estado	Aprobado
Comentarios	La gestión de las conexiones a la base de datos es una solución de diseño al servicio de este requisito.

Tabla x - RNF-021 — Capacidad de acceso concurrente

RNF-022 — Comportamiento del sistema con la GPU ocupada

El entrenamiento de modelos compite por la GPU de la estación de trabajo, que es un recurso único. El sistema debe garantizar que, cuando la GPU está ocupada por uno o varios entrenamientos, las peticiones de diagnóstico continúan encolándose y procesándose cuando el recurso queda libre, sin pérdida de trabajos y sin que un entrenamiento monopolice el servicio indefinidamente. La duración máxima de un entrenamiento no se fija como umbral del sistema, sino que se determina por la configuración del experimento y por el límite de trabajos del requisito RF-041. El requisito RNF-022 recoge esta exigencia.

Campo	Contenido
ID	RNF-022
Nombre	Comportamiento del sistema con la GPU ocupada
Objetivos relacionados	OBJ-010
Descripción	El sistema debe encolar y procesar las peticiones de diagnóstico y de entrenamiento cuando la GPU está ocupada, sin pérdida de trabajos y de modo que un entrenamiento no monopolice el recurso indefinidamente.
Importancia	Media
Estado	Aprobado
Comentarios	Complementa al requisito funcional RF-041, que limita los entrenamientos simultáneos y encolados.

Tabla x - RNF-022 — Comportamiento del sistema con la GPU ocupada

12.1.2.4. Sencillez de uso y accesibilidad

Este grupo agrupa los requisitos no funcionales relacionados con la facilidad de uso de la plataforma y con su accesibilidad para los distintos perfiles de usuarios. vitalXAI está concebido para ser utilizado por profesionales sanitarios sin formación técnica en inteligencia artificial o informática, por lo que la interfaz debe resultar intuitiva y no exigir en ningún momento conocimientos de programación. Los requisitos de este grupo se asocian al objetivo de usabilidad e internacionalización de la interfaz (OBJ-009). La usabilidad y la accesibilidad no se declaran como intenciones: se fijan criterios comprobables y un método de validación con usuarios.

RNF-023 — Interfaz intuitiva para usuarios sin formación técnica

El usuario principal de la plataforma es el profesional sanitario, que no dispone de formación en inteligencia artificial ni en informática. La interfaz debe emplear un lenguaje no técnico, presentar la información de forma clara y guiar al usuario en cada flujo sin exigirle conocer el funcionamiento interno del sistema. La comprobación de este requisito se realiza mediante una evaluación con usuarios representativos del perfil sanitario, que deben completar las tareas principales del sistema —registro, diagnóstico, consulta del historial y lanzamiento de un experimento— con una tasa de éxito del cien por cien en el flujo principal y sin asistencia del evaluador. El requisito RNF-023 recoge esta exigencia.

Campo	Contenido
ID	RNF-023
Nombre	Interfaz intuitiva para usuarios sin formación técnica
Objetivos relacionados	OBJ-009
Descripción	El sistema debe permitir a usuarios representativos del perfil sanitario completar las tareas principales —registro, diagnóstico, consulta del historial y lanzamiento de un experimento— con una tasa de éxito del cien por cien en el flujo principal y sin asistencia del evaluador.
Importancia	Alta
Estado	Aprobado
Comentarios	El método de validación se describe en el plan de pruebas; la medición se realiza con tareas cronometradas y usuarios representativos.

Tabla x - RNF-023 — Interfaz intuitiva para usuarios sin formación técnica

RNF-024 — Soporte plurilingüe de la plataforma

La plataforma está dirigida a un público sanitario e investigador que puede proceder de entornos lingüísticos diversos, por lo que debe poder presentarse en varios idiomas. El sistema debe ofrecer la interfaz, los informes generados y el asistente conversacional en los cuatro idiomas de la plataforma —español, inglés, chino e hindú—, de modo que el usuario pueda cambiar de idioma de forma dinámica sin reconfigurar nada. El requisito RNF-024 recoge esta exigencia.

Campo	Contenido
ID	RNF-024
Nombre	Soporte plurilingüe de la plataforma
Objetivos relacionados	OBJ-009
Descripción	El sistema debe ofrecer la interfaz, los informes y el asistente conversacional en los cuatro idiomas de la plataforma, permitiendo cambiar de idioma de forma dinámica.
Importancia	Media
Estado	Aprobado
Comentarios	El cambio de idioma se aplica en toda la plataforma y en el asistente conversacional.

Tabla x - RNF-024 — Soporte plurilingüe de la plataforma

RNF-025 — Legibilidad y presentación adaptada a la interfaz web

La interfaz debe presentar la información de forma legible y adaptarse a distintos tamaños de pantalla, de modo que el profesional pueda utilizarla desde su equipo habitual sin pérdida de información. El diseño debe garantizar una jerarquía visual clara en las tablas de resultados y una disposición ordenada de los elementos en los paneles de diagnóstico y de laboratorio. La comprobación se realiza verificando que las vistas principales se adaptan a las resoluciones de pantalla de uso habitual sin pérdida de contenido y sin barras de desplazamiento horizontales no deseadas. El requisito RNF-025 recoge esta exigencia.

Campo	Contenido
ID	RNF-025
Nombre	Legibilidad y presentación adaptada a la interfaz web
Objetivos relacionados	OBJ-009
Descripción	El sistema debe presentar la información de forma legible y ordenada, con jerarquía visual clara, y adaptarse a las resoluciones de pantalla de uso habitual sin pérdida de contenido ni barras de desplazamiento horizontales no deseadas.
Importancia	Media
Estado	Aprobado
Comentarios	La verificación se realiza sobre las vistas principales del sistema en las resoluciones de referencia.

Tabla x - RNF-025 — Legibilidad y presentación adaptada a la interfaz web

RNF-026 — Accesibilidad de la interfaz

El sistema debe ser utilizable por personas con discapacidad, conforme a las pautas de accesibilidad para contenido web (WCAG) en su nivel de conformidad AA. La interfaz debe ser navegable por teclado, los elementos interactivos deben tener etiquetas descriptivas que puedan interpretar los lectores de pantalla, y los elementos textuales deben mantener un contraste suficiente con su fondo. La comprobación se realiza mediante una revisión de las pautas WCAG sobre las vistas principales del sistema. El requisito RNF-026 recoge esta exigencia.

Campo	Contenido
ID	RNF-026
Nombre	Accesibilidad de la interfaz
Objetivos relacionados	OBJ-009
Descripción	El sistema debe cumplir las pautas de accesibilidad WCAG en su nivel AA: navegación por teclado, etiquetas descriptivas interpretables por lectores de pantalla y contraste suficiente entre texto y fondo.
Importancia	Media
Estado	Aprobado
Comentarios	La verificación se realiza mediante revisión de las pautas sobre las vistas principales.

Tabla x - RNF-026 — Accesibilidad de la interfaz

12.1.2.5. Robustez y disponibilidad del servicio

Este grupo agrupa los requisitos no funcionales relacionados con la robustez del sistema y con su disponibilidad continua durante el uso. En un entorno clínico, la plataforma debe permanecer operativa mientras se ejecutan las tareas de larga duración, y los errores aislados de un trabajo no deben comprometer el servicio en su conjunto. Los requisitos de este grupo se asocian al objetivo de ejecución asíncrona de tareas de larga duración (OBJ-010).

RNF-027 — Disponibilidad del servicio durante las tareas de larga duración

El sistema debe permanecer disponible y operativo mientras se ejecutan los entrenamientos y las validaciones externas, que pueden prolongarse durante horas. Un entrenamiento no debe dejar la plataforma inaccesible para el resto de los usuarios: durante su ejecución, el profesional debe poder continuar utilizando la interfaz, realizar diagnósticos o consultar el estado de sus trabajos. El requisito RNF-027 recoge esta exigencia.

Campo	Contenido
ID	RNF-027
Nombre	Disponibilidad del servicio durante las tareas de larga duración
Objetivos relacionados	OBJ-010
Descripción	El sistema debe permanecer disponible y operativo mientras se ejecutan las tareas de larga duración, de modo que un entrenamiento no deje la plataforma inaccesible para el resto de los usuarios.
Importancia	Alta
Estado	Aprobado
Comentarios	La ejecución asíncrona garantiza la disponibilidad continua del servicio.

Tabla x - RNF-027 — Disponibilidad del servicio durante las tareas de larga duración

RNF-028 — Gestión de los errores de los trabajos de la cola

Las tareas que se ejecutan en segundo plano pueden fallar por motivos diversos —una incompatibilidad, un dataset inaccesible o un error en los scripts—. El sistema debe gestionar estos fallos de forma ordenada: registrar el error asociado al trabajo, marcarlo como fallido y notificar al usuario, de modo que la plataforma no se bloquee y el usuario pueda conocer la causa del fallo y decidir cómo proceder. Un error aislado en un trabajo no debe interrumpir el resto de los trabajos de la cola. El requisito RNF-028 recoge esta exigencia.

Campo	Contenido
ID	RNF-028
Nombre	Gestión de los errores de los trabajos de la cola
Objetivos relacionados	OBJ-010
Descripción	El sistema debe gestionar los errores de los trabajos de la cola registrando el fallo, marcando el trabajo como fallido y notificando al usuario, sin interrumpir el resto de los trabajos.
Importancia	Alta
Estado	Aprobado
Comentarios	Un error aislado no debe comprometer la disponibilidad del servicio.

Tabla x - RNF-028 — Gestión de los errores de los trabajos de la cola

RNF-029 — Recuperación de los trabajos tras un reinicio del servidor

Un reinicio del servidor no debe provocar la pérdida de los trabajos que estaban en ejecución. El sistema debe recuperar la cola de trabajos al arrancar, de modo que las tareas que quedaron en ejecución vuelvan a quedar pendientes y puedan procesarse de nuevo. Dado que un trabajo interrumpido a mitad de su ejecución pudo escribir resultados parciales, la recuperación debe ser idempotente: el sistema debe comprobar, antes de reprocesar un trabajo, si ya existen artefactos del trabajo anterior y, en tal caso, sustituirlos de forma consistente para evitar duplicados o artefactos corruptos. El requisito RNF-029 recoge esta exigencia.

Campo	Contenido
ID	RNF-029
Nombre	Recuperación de los trabajos tras un reinicio del servidor
Objetivos relacionados	OBJ-010
Descripción	El sistema debe recuperar la cola de trabajos al arrancar, devolviendo a pendiente los trabajos que quedaron en ejecución, y debe reprocesarlos de forma idempotente, sustituyendo de forma consistente los artefactos parciales del intento anterior para evitar duplicados o resultados corruptos.
Importancia	Media
Estado	Aprobado
Comentarios	La recuperación evita la pérdida de trabajos ante un reinicio; la idempotencia evita que el reprocesamiento genere duplicados o artefactos incompletos.

Tabla x - RNF-029 — Recuperación de los trabajos tras un reinicio del servidor

RNF-030 — Aislamiento de los fallos entre trabajos

Cada trabajo de la cola debe ejecutarse de forma aislada, de modo que un fallo en uno de ellos no afecte al resto. Esta separación es especialmente relevante en el laboratorio de entrenamiento, donde los experimentos se procesan de forma independiente: un error en el entrenamiento de un modelo no debe impedir la ejecución de los demás trabajos encolados. El requisito RNF-030 recoge esta exigencia.

Campo	Contenido
ID	RNF-030
Nombre	Aislamiento de los fallos entre trabajos
Objetivos relacionados	OBJ-010
Descripción	El sistema debe ejecutar los trabajos de la cola de forma aislada, de modo que un fallo en uno de ellos no afecte al resto de los trabajos.
Importancia	Media
Estado	Aprobado
Comentarios	El aislamiento garantiza la robustez del procesamiento asíncrono.

Tabla x - RNF-030 — Aislamiento de los fallos entre trabajos

12.1.2.6. Persistencia y salvaguarda de los datos

Este grupo agrupa los requisitos no funcionales relacionados con la integridad y la salvaguarda de la información persistida. La capa de persistencia almacena las cuentas de usuario, las consultas de diagnóstico con sus artefactos y las configuraciones de los experimentos, y esa información debe conservarse de forma íntegra y recuperable. Los requisitos de este grupo se asocian al objetivo de persistencia y trazabilidad de la información (OBJ-008).

RNF-031 — Integridad y durabilidad de la persistencia

La capa de persistencia es la encargada de almacenar la información que la plataforma genera: las cuentas de usuario, las consultas de diagnóstico con sus artefactos y las configuraciones de los experimentos. Esta información debe conservarse de forma íntegra y duradera: los registros deben mantenerse coherentes entre sí y no corromperse con el paso del tiempo ni ante la ejecución concurrente de operaciones. El sistema debe garantizar la coherencia de las operaciones de escritura y la conservación de los datos a lo largo del ciclo de vida de la plataforma. El requisito RNF-031 recoge esta exigencia.

Campo	Contenido
ID	RNF-031
Nombre	Integridad y durabilidad de la persistencia
Objetivos relacionados	OBJ-008
Descripción	El sistema debe conservar la información almacenada de forma íntegra y duradera, garantizando la coherencia de los registros ante el paso del tiempo y la ejecución concurrente de operaciones.
Importancia	Media
Estado	Aprobado
Comentarios	Aplica a las cuentas de usuario, las consultas de diagnóstico y las configuraciones de los experimentos.

Tabla x - RNF-031 — Integridad y durabilidad de la persistencia

RNF-032 — Copia de seguridad y recuperación de los datos

Los datos almacenados por la plataforma deben poder recuperarse ante un incidente que los ponga en riesgo, como un fallo del hardware, un error de configuración o una corrupción accidental de la base de datos. El sistema debe disponer de un mecanismo de copia de seguridad de la información almacenada y de un procedimiento de recuperación que permita restaurarla. En este proyecto, la copia de seguridad se realiza con una periodicidad diaria e incluye la base de datos y los ficheros de artefactos necesarios para reconstruir una consulta completa (imágenes, mapas de explicación e informes). El objetivo de recuperación se fija en un máximo de veinticuatro horas tras el incidente. El requisito RNF-032 recoge esta exigencia.

Campo	Contenido
ID	RNF-032
Nombre	Copia de seguridad y recuperación de los datos
Objetivos relacionados	OBJ-008
Descripción	El sistema debe disponer de una copia de seguridad diaria que incluya la base de datos y los ficheros de artefactos necesarios para reconstruir una consulta completa, y de un procedimiento de recuperación que permita restaurar los datos en un máximo de veinticuatro horas.
Importancia	Media
Estado	Aprobado
Comentarios	El alcance de la copia incluye los ficheros de imagen, explicación e informe, no solo la base de datos, porque su pérdida impide reconstruir una consulta completa.

Tabla x - RNF-032 — Copia de seguridad y recuperación de los datos

12.1.2.7. Reproducibilidad y sostenibilidad del desarrollo

Este grupo agrupa los requisitos no funcionales relacionados con la reproducibilidad de los experimentos del laboratorio. La reproducibilidad es una exigencia científica del proyecto: cualquier resultado empírico debe poder verificarse y replicarse bajo las mismas condiciones, lo que constituye un requisito imprescindible en el ámbito de la investigación en aprendizaje automático. Los requisitos de este grupo se asocian al objetivo de reproducibilidad de los experimentos (OBJ-006).

RNF-033 — Reproducibilidad de los experimentos

Los experimentos de entrenamiento pueden verse afectados por la inicialización aleatoria de los pesos, el orden de procesamiento de los datos o la concurrencia del hardware, de modo que dos ejecuciones del mismo experimento pueden producir resultados ligeramente distintos. Para que los resultados sean verificables y replicables, el sistema debe fijar las semillas aleatorias de las librerías de aprendizaje automático y de la biblioteca estándar de Python, y almacenar la configuración completa de cada experimento en un archivo asociado a la sesión de entrenamiento. Con esta información, cualquier resultado empírico puede reproducirse bajo las mismas condiciones y puede trazarse su origen. El requisito RNF-033 recoge esta exigencia.

Campo	Contenido
ID	RNF-033
Nombre	Reproducibilidad de los experimentos
Objetivos relacionados	OBJ-006
Descripción	El sistema debe fijar las semillas aleatorias de las librerías de aprendizaje automático y de la biblioteca estándar de Python, y almacenar la configuración completa de cada sesión de entrenamiento, de modo que cualquier resultado pueda reproducirse bajo las mismas condiciones.
Importancia	Alta
Estado	Aprobado
Comentarios	La configuración almacenada permite replicar y trazar cualquier resultado empírico. No se exige determinismo bit a bit entre ejecuciones.

Tabla x - RNF-033 — Reproducibilidad de los experimentos

RNF-034 — Reproducibilidad del entorno de ejecución

Además de la semilla aleatoria, la reproducibilidad depende del entorno de ejecución: una versión distinta de una librería puede alterar los resultados o romper la compatibilidad entre componentes. El sistema debe fijar las versiones exactas de las dependencias y aislar el entorno de ejecución, de modo que el conjunto de librerías pueda reconstruirse de forma reproducible. Esta fijación de versiones se refiere a un entorno con las características de la estación de trabajo descrita en el análisis, incluida la GPU y el toolkit CUDA correspondiente; no se pretende la reproducibilidad en máquinas de características distintas. El requisito RNF-034 recoge esta exigencia.

Campo	Contenido
ID	RNF-034
Nombre	Reproducibilidad del entorno de ejecución
Objetivos relacionados	OBJ-006
Descripción	El sistema debe fijar las versiones exactas de las dependencias y aislar el entorno de ejecución, de modo que el conjunto de librerías pueda reconstruirse de forma reproducible en un entorno con las características de la estación de trabajo, incluida su GPU y su toolkit CUDA.
Importancia	Alta
Estado	Aprobado
Comentarios	La reproducibilidad se garantiza para el entorno concreto de la estación de trabajo, no para máquinas de características distintas.

Tabla x - RNF-034 — Reproducibilidad del entorno de ejecución

12.1.2.8. Compromisos del proceso de desarrollo y entregables del proyecto

RNF-035 — Cumplimiento de estándares de codificación

El código del sistema debe cumplir las convenciones de estilo de Python (PEP 8), verificadas mediante herramientas de análisis estático, y sus funciones deben documentarse con docstrings que describan su propósito, sus parámetros y su valor de retorno. Este compromiso garantiza la legibilidad y la coherencia del código y facilita su revisión.

Campo	Contenido
ID	RNF-035
Nombre	Cumplimiento de estándares de codificación
Objetivos relacionados	—
Descripción	El código del proyecto debe cumplir las convenciones de estilo PEP 8, verificadas con herramientas de análisis estático, y documentar las funciones mediante docstrings.
Importancia	Media
Estado	Aprobado
Comentarios	Compromiso del proceso de desarrollo, no requisito del sistema.

Tabla x - RNF-035 — Cumplimiento de estándares de codificación

RNF-036 — Separación de responsabilidades y modularidad

El código del proyecto debe mantener una separación clara de responsabilidades entre sus componentes: los scripts de entrenamiento y evaluación deben permanecer desacoplados de la capa web, de modo que puedan ejecutarse como procesos independientes y evolucionar por separado. Este compromiso reduce el acoplamiento entre el motor científico y la plataforma.

Campo	Contenido
ID	RNF-036
Nombre	Separación de responsabilidades y modularidad
Objetivos relacionados	—
Descripción	El código del proyecto debe mantener separados los módulos de computación científica de la capa web, de modo que puedan ejecutarse de forma independiente y mantenerse de forma aislada.
Importancia	Media
Estado	Aprobado
Comentarios	Compromiso del proceso de desarrollo, no requisito del sistema.

Tabla x - RNF-036 — Separación de responsabilidades y modularidad

RNF-037 — Documentación de la memoria del Trabajo Fin de Grado (entregable)

El proyecto debe documentarse mediante la memoria del Trabajo Fin de Grado, que recoge de forma estructurada el plan de proyecto, el análisis, el diseño, la implementación, las pruebas y las conclusiones del trabajo. La memoria debe cumplir la estructura y el formato exigidos por la normativa universitaria. Este documento es el entregable central del proyecto.

Campo	Contenido
ID	RNF-037
Nombre	Documentación de la memoria del Trabajo Fin de Grado
Objetivos relacionados	—
Descripción	El proyecto debe documentarse mediante la memoria del Trabajo Fin de Grado, que cumpla la estructura y el formato exigidos por la normativa universitaria.
Importancia	Alta
Estado	Aprobado
Comentarios	Entregable del proyecto, no requisito del sistema.

Tabla x - RNF-037 — Documentación de la memoria del Trabajo Fin de Grado (entregable)

RNF-038 — Manual de usuario (entregable)

El proyecto debe acompañarse de un manual de usuario orientado a los profesionales sanitarios que no disponen de formación técnica. El manual debe explicar, con un lenguaje no técnico y con capturas de los flujos principales, cómo registrarse, realizar un diagnóstico, interpretar los mapas de explicabilidad y utilizar el laboratorio de entrenamiento.

Campo	Contenido
ID	RNF-038
Nombre	Manual de usuario
Objetivos relacionados	—
Descripción	El proyecto debe incluir un manual de usuario en lenguaje no técnico, dirigido a los profesionales sanitarios, que describa paso a paso los flujos principales de la plataforma.
Importancia	Media
Estado	Aprobado
Comentarios	Entregable del proyecto, no requisito del sistema.

Tabla x - RNF-038 — Manual de usuario (entregable)

Conviene señalar que el manual de instalación y configuración, que en una primera versión del catálogo se recogía como requisito, no aparece en la planificación del proyecto: el cronograma contempla la memoria y el manual de usuario como entregables finales, pero no asigna horas a un manual de instalación. Por ello, no se incluye como entregable en esta sección; si se desea incorporar en el futuro, debe planificarse previamente.

12.1.2.9. Matriz de trazabilidad entre requisitos no funcionales y objetivos del sistema

La matriz de trazabilidad que se presenta a continuación relaciona cada requisito no funcional con el objetivo u objetivos del sistema a los que da respuesta, siguiendo el mismo criterio que la matriz de los requisitos funcionales. Esta matriz permite verificar que los objetivos con implicaciones transversales de calidad quedan cubiertos por los requisitos no funcionales correspondientes. Los objetivos de carácter puramente funcional —como la gestión del historial de consultas (OBJ-002) o la generación de informes descargables (OBJ-003)— se materializan mediante requisitos funcionales y no presentan asignaciones en esta matriz. No ocurre lo mismo con los objetivos que sí tienen implicaciones de calidad: el laboratorio de entrenamiento (OBJ-004) y la evaluación rigurosa de los modelos (OBJ-005) se asocian, respectivamente, al requisito de comportamiento del sistema con la GPU ocupada y a los requisitos de reproducibilidad de los experimentos, que condicionan la calidad de sus resultados. Los compromisos del proceso de desarrollo y los entregables (RNF-035 a RNF-038) no se incluyen en la matriz por no ser requisitos del sistema. Las columnas representan los objetivos del sistema y las filas los requisitos no funcionales; una X en la intersección indica que el requisito contribuye a la consecución del objetivo correspondiente.

RNF	Nombre	OBJ-001	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RNF-001	Cifrado de contraseñas					X				
RNF-002	Gestión segura de la sesión					X				
RNF-003	Protección frente a la falsificación de peticiones entre sitios					X				
RNF-004	Limitación de las peticiones de acceso					X				
RNF-005	Cabeceras de seguridad en las respuestas					X				
RNF-006	Auditoría de la actividad administrativa					X				X
RNF-007	Validación y saneamiento de las entradas					X				
RNF-008	Subida de ficheros segura					X				
RNF-009	Registro de eventos de seguridad					X				X
RNF-010	Política de contraseñas					X				

[illegible]

RNF	Nombre	OBJ-001	OBJ-004	OBJ-005	OBJ-006	OBJ-007	OBJ-008	OBJ-009	OBJ-010	OBJ-011
RNF-026	Accesibilidad de la interfaz							X		
RNF-027	Disponibilidad del servicio durante las tareas de larga duración								X	
RNF-028	Gestión de los errores de los trabajos de la cola								X	
RNF-029	Recuperación de los trabajos tras un reinicio del servidor								X	
RNF-030	Aislamiento de los fallos entre trabajos								X	
RNF-031	Integridad y durabilidad de la persistencia						X			
RNF-032	Copia de seguridad y recuperación de los datos						X			
RNF-033	Reproducibilidad de los experimentos			X	X					
RNF-034	Reproducibilidad del entorno de ejecución			X	X					

Tabla x - Matriz de trazabilidad entre requisitos no funcionales y objetivos del sistema

La matriz confirma la cobertura de los objetivos con implicaciones de calidad transversal: la seguridad y la confidencialidad se concentran en el objetivo de acceso seguro y personalizado (OBJ-007); la usabilidad, la accesibilidad y el multilingüismo en el de usabilidad e internacionalización (OBJ-009); la disponibilidad y la robustez en el de ejecución asíncrona de tareas (OBJ-010); la integridad y la salvaguarda de los datos en el de persistencia y trazabilidad (OBJ-008); y la reproducibilidad en el de reproducibilidad de los experimentos (OBJ-006), que además se asocia a la evaluación rigurosa de los modelos (OBJ-005), porque la calidad de los resultados estadísticos depende de esas condiciones. El objetivo de diagnóstico asistido (OBJ-001) se complementa con el requisito de rendimiento de la inferencia, y el laboratorio de entrenamiento (OBJ-004) con el comportamiento del sistema cuando la GPU está ocupada. Esta verificación, junto con la matriz de los requisitos funcionales, garantiza que todos los objetivos del sistema quedan cubiertos por los niveles funcional y no funcional de la especificación.

12.2. Especificación de los casos de uso

Un caso de uso describe una interacción concreta entre un actor y el sistema, orientada a obtener un resultado que aporta valor a dicho actor (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Para reconocer qué es y qué no es un caso de uso basta con aplicar una sencilla regla práctica: si la acción la inicia el actor y produce un efecto observable para él, es un caso de uso; si se trata de un mecanismo interno que el actor no percibe ni desencadena, no lo es. Así, «el facultativo sube una radiografía y recibe un diagnóstico explicado» es un caso de uso, mientras que «la base de datos persiste la consulta» no lo es, porque constituye un detalle de implementación ajeno a la experiencia del usuario (Cockburn, 2001). La especificación de los casos de uso ocupa un lugar central en esta memoria porque tiende el puente entre lo que el análisis afirma que el sistema debe hacer y la forma concreta en que cada usuario lo realiza: mientras que los requisitos funcionales definen las capacidades del sistema de forma técnica y verificable, los casos de uso las describen desde la perspectiva de quien las emplea, relatando el flujo de acciones y las respuestas del sistema paso a paso (Wiegers & Beatty, 2013). Sobre esta especificación se apoyan, además, tres actividades posteriores del proyecto: el diseño de los subsistemas, la definición de las pruebas de sistema y la validación final de que la plataforma entrega lo prometido. Este apartado modela los treinta y seis casos de uso de vitalXAI, agrupados en cinco módulos funcionales que se corresponden con los ámbitos de la plataforma identificados en el análisis: la gestión del acceso y de la cuenta, la interfaz de diagnóstico asistido, el laboratorio de experimentación MLOps, la supervisión y administración de la plataforma y las capacidades transversales. Para cada módulo se presenta, en primer lugar, el diagrama de casos de uso y, a continuación, la especificación detallada de cada caso con sus flujos normal y alternativo y las condiciones previas y posteriores que lo delimitan.

12.2.1. Diagramas de casos de uso por módulo

El diagrama de casos de uso proporciona una visión de conjunto de las interacciones que los actores pueden mantener con la plataforma, representando de forma gráfica a los actores, a los casos de uso y a las relaciones de asociación que los vinculan. En la notación empleada, cada actor se representa mediante una figura acompañada de su nombre, cada caso de uso mediante una elipse con su denominación y su código identificador, y cada asociación mediante una línea continua que une al actor con el caso de uso que inicia. Para facilitar la lectura y la comprensión del conjunto, el diagrama general se ha dividido en cinco diagramas independientes, uno por cada módulo funcional, de modo que cada figura recoge exclusivamente los casos de uso de su ámbito y los actores que participan en ellos. Esta división responde a un criterio de legibilidad: el sistema completo reúne treinta y seis casos de uso, y su representación conjunta en un único diagrama dificultaría la interpretación sin aportar información adicional. Respecto a la notación de las relaciones entre casos de uso, conviene recordar el significado de los dos estereotipos empleados en los diagramas. Una relación de extensión, representada con el estereotipo «extend», indica un comportamiento opcional que amplía el flujo de un caso de uso base: el caso de uso que extiende solo se activa cuando el actor decide utilizarlo, y no forma parte del flujo principal. Una relación de inclusión, representada con el estereotipo «include», indica un paso obligatorio que el caso de uso base delega en otro para poder completarse: el caso de uso incluido se ejecuta siempre como parte del flujo del que depende (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). En los diagramas de este capítulo las flechas de extensión se dibujan en la dirección de la navegación, es decir, desde el caso de uso base hacia la extensión, de modo que reflejan el orden en que el actor recorre la interfaz; la flecha de inclusión, por el contrario, se dibuja desde el caso de uso que incluye hacia el caso incluido, conforme a la notación estándar. En vitalXAI la mayoría de los casos de uso representan interacciones directas e independientes del actor con el sistema, pero existen relaciones de extensión allí donde la navegación de la interfaz conduce de una consulta general a un detalle o a una acción opcional sobre ese detalle —por ejemplo, abrir el detalle de una consulta del historial o solicitar el recálculo de una comparativa estadística—, y una relación de inclusión en el laboratorio, donde el lanzamiento de un experimento no puede completarse sin la configuración previa definida con el asistente. Estas relaciones se representan en los diagramas mediante flechas discontinuas etiquetadas con el estereotipo correspondiente. Finalmente, cabe aclarar el modelo de actores empleado en las figuras: el Administrador no constituye un perfil aislado, sino un usuario autenticado que conserva todas las capacidades del resto de los perfiles y añade las operaciones de gobierno del sistema (CU-031 a CU-033). Por ello, en los diagramas de los módulos no administrativos el actor se denomina «Usuario autenticado» y representa a todos los perfiles con cuenta, incluido el administrador. A continuación se presentan los diagramas de los cinco módulos, precedidos cada uno de una breve descripción de su propósito y de los casos de uso que agrupa.

12.2.1.1. Gestión del acceso y de la cuenta

Este módulo agrupa los casos de uso relacionados con el control de acceso a la plataforma, el punto de partida de cualquier uso del sistema. Su propósito es doble. Por un lado, garantiza que únicamente las personas registradas y autenticadas puedan acceder a las funcionalidades privadas de la plataforma, de modo que la información clínica y los resultados de los experimentos queden reservados a quienes disponen de una cuenta. Por otro lado, asegura que cada usuario opere exclusivamente con sus propios datos, en cumplimiento del aislamiento de información entre cuentas que exige la plataforma y que constituye uno de los principios de diseño del sistema. La autenticación se configura así como una condición transversal: ninguna de las capacidades de diagnóstico, laboratorio o administración puede utilizarse sin haberla completado, y cualquier intento de acceder a ellas sin una sesión activa debe ser rechazado por el sistema.

El módulo comprende cuatro casos de uso, todos ellos de naturaleza directa. La creación de la cuenta (CU-001) permite al visitante registrarse aportando sus datos personales y una contraseña, que el sistema almacena siempre cifrada; el acceso con credenciales (CU-002) verifica la identidad del usuario y abre la sesión, habilitando el resto de los módulos; el cierre seguro de la sesión (CU-003) revoca el acceso a las áreas privadas cuando el usuario termina su trabajo; y el cambio del idioma de la interfaz (CU-004) permite adaptar el idioma de la plataforma —entre los disponibles, que incluyen el español, el inglés, el chino y el hindú— tanto desde el área pública como desde el área privada, sin perder el estado de navegación.

Conviene aclarar la naturaleza del actor Administrador en este módulo: se trata de un usuario autenticado con las mismas capacidades que el resto de los perfiles, cuyas operaciones específicas de gobierno se recogen en el módulo de supervisión y administración. Por ello, en este diagrama el actor «Usuario autenticado» representa a todos los perfiles con cuenta, incluido el administrador. Al no existir comportamientos opcionales ni pasos obligatorios compartidos, los cuatro casos de uso se representan como interacciones directas e independientes del actor con el sistema, sin relaciones de inclusión o extensión entre sí.

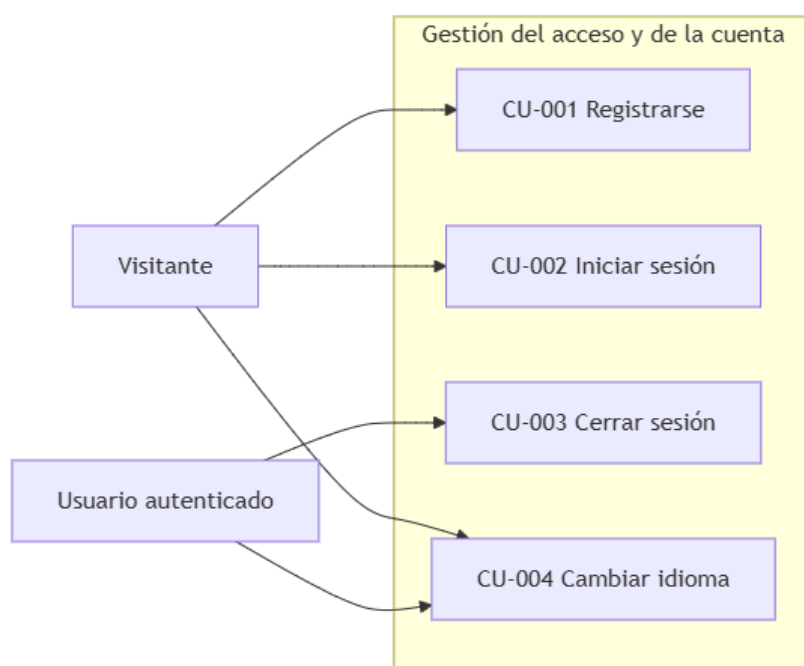


Ilustración x - Casos de uso del módulo de gestión del acceso y de la cuenta

12.2.1.2. Interfaz de diagnóstico asistido

Este módulo agrupa los casos de uso de la interfaz clínica, el primer núcleo funcional del sistema. A través de ella, el usuario autenticado realiza un diagnóstico asistido de neumonía a partir de una radiografía de tórax: carga la imagen, elige la arquitectura con la que desea realizar la consulta, solicita el diagnóstico y obtiene el resultado junto con su nivel de confianza y los mapas de explicabilidad que lo justifican. El diagnóstico se procesa de forma asíncrona mediante la cola de trabajos: en cuanto el usuario envía la petición, el sistema encola el análisis y la interfaz permanece operativa, de modo que el facultativo puede seguir trabajando mientras el modelo procesa la imagen, genera la predicción y produce la explicación visual.

El módulo cubre también la gestión del historial de consultas, que permite al facultativo recuperar en cualquier momento sus diagnósticos anteriores, revisar un resultado, comparar la evolución de un caso o reutilizar una imagen ya analizada. Cada consulta queda registrada de forma aislada en el historial del usuario que la realizó, y solo ese usuario puede consultarla, renombrarla o eliminarla. En total, el módulo reúne once casos de uso: el acceso al panel (CU-005), que presenta en una única vista la carga de la imagen, la selección del modelo y el acceso al historial; la subida de la radiografía (CU-006); la selección de la arquitectura para el diagnóstico (CU-007); la solicitud del diagnóstico (CU-008); la visualización del resultado (CU-009) y de los mapas de explicabilidad (CU-010); la consulta del listado del historial (CU-011); el detalle de una consulta (CU-012); su renombrado (CU-013), su eliminación (CU-014) y la generación del informe PDF de la consulta (CU-037).

En la parte del historial, las relaciones de extensión reflejan el orden en que el actor recorre la interfaz. Desde el listado de consultas (CU-011) se accede, de forma opcional, al detalle de una de ellas (CU-012), donde el usuario recupera la imagen original, el resultado, la confianza y los mapas de explicabilidad asociados; y desde ese detalle pueden ejecutarse igualmente, de forma opcional, el renombrado (CU-013) o la eliminación (CU-014) de la consulta. El resto de los casos de uso del módulo, centrados en la solicitud y visualización del diagnóstico, se representan como interacciones directas e independientes del actor con el sistema.

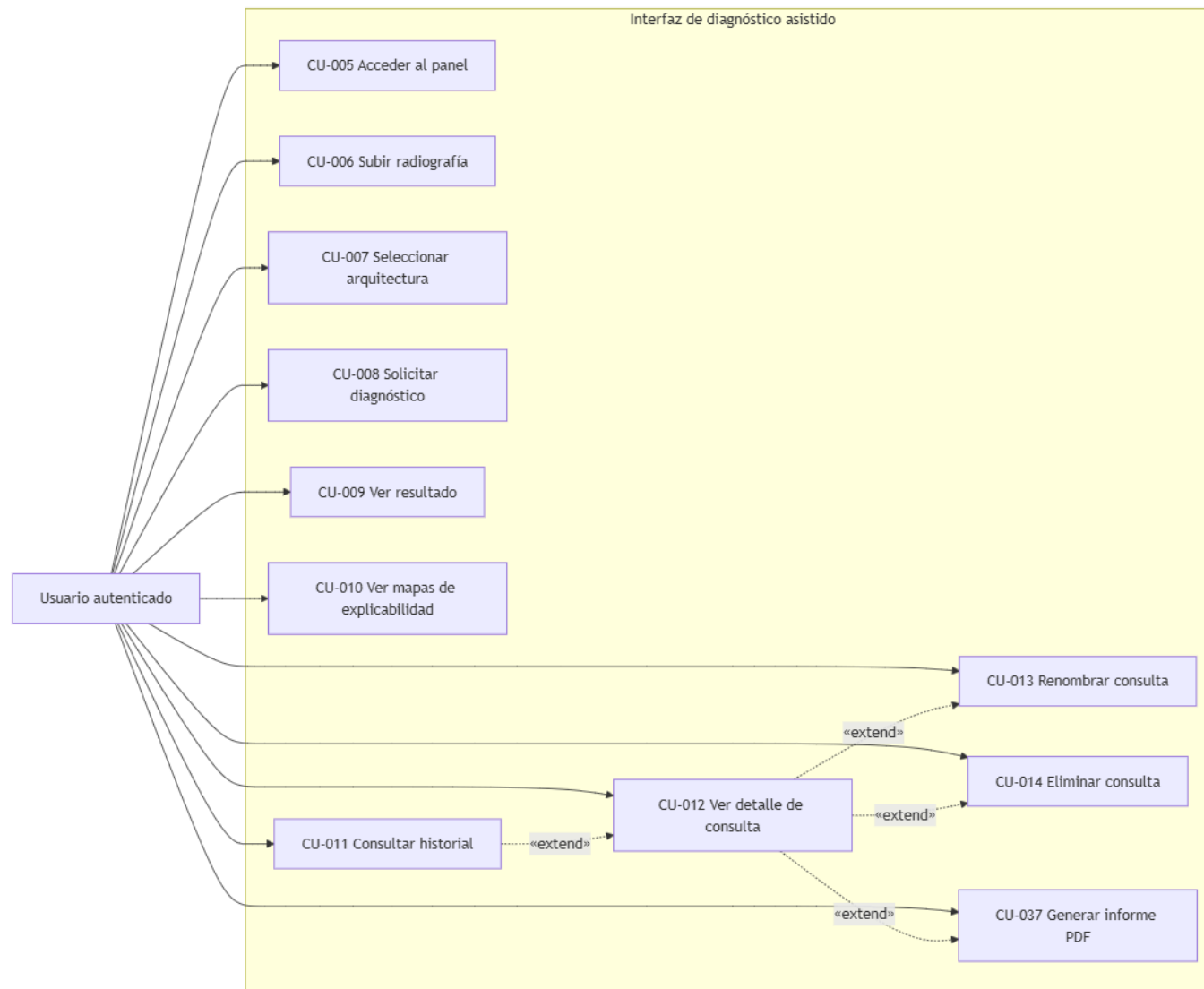


Ilustración x - Casos de uso del módulo de interfaz de diagnóstico asistido

12.2.1.3. Laboratorio de experimentación MLOps

Este módulo agrupa los casos de uso del laboratorio de entrenamiento, el segundo núcleo funcional del sistema. A través de él, el usuario configura y lanza experimentos de entrenamiento mediante un asistente conversacional, monitoriza la ejecución del pipeline y consulta los resultados comparativos y estadísticos de sus sesiones. El laboratorio está concebido para que el investigador no necesite escribir código en ningún momento: el asistente, basado en un modelo de lenguaje de última generación ejecutado a través de una API externa, interpreta las indicaciones del usuario en lenguaje natural, extrae los parámetros del experimento y rellena la configuración correspondiente.

El laboratorio orquesta automáticamente la secuencia de entrenamiento, análisis de explicabilidad, comparación estadística y validación externa. Una vez lanzado el experimento, el pipeline entrena cada arquitectura seleccionada mediante validación cruzada, genera los mapas de calor de explicabilidad y las métricas cuantitativas asociadas, compara estadísticamente los modelos mediante el ranking y el test de Wilcoxon y, cuando el usuario lo solicita, evalúa los modelos congelados sobre un conjunto de datos externo aplicando el test de DeLong. Este módulo es el más extenso de la plataforma y reúne diecisiete casos de uso: el acceso al laboratorio (CU-015), la conversación con el asistente (CU-016), la selección de la carpeta del dataset (CU-017), el lanzamiento del experimento (CU-018), la consulta de las sesiones (CU-019), los resultados de un modelo (CU-020), los mapas de calor de explicabilidad (CU-021), el ranking de modelos (CU-022), la comparativa estadística (CU-023), el recálculo de la comparativa (CU-024), la ejecución del análisis de explicabilidad (CU-025), la solicitud de la validación externa (CU-026) y la consulta de sus resultados (CU-027), la generación del informe PDF de la sesión (CU-028), el renombrado (CU-029) y eliminación (CU-030) de sesiones, y la comprobación de la limitación de entrenamientos simultáneos y encolados (CU-039).

El lanzamiento del experimento (CU-018) incluye de forma obligatoria la conversación con el asistente (CU-016), porque la configuración del experimento —arquitecturas, número de épocas, tamaño de lote y tasa de aprendizaje— debe quedar completa antes de iniciar el entrenamiento. Por esa razón la flecha de inclusión parte del caso que lanza el experimento y apunta hacia el caso incluido, conforme a la notación estándar de los diagramas de casos de uso.

Las relaciones de extensión, por su parte, siguen la navegación del usuario por el laboratorio. Desde la consulta de las sesiones (CU-019) se accede a los resultados de un modelo concreto (CU-020), al ranking de la sesión (CU-022), a la comparativa estadística (CU-023) o a la generación del informe consolidado (CU-028); desde los resultados de un modelo se alcanzan sus mapas de calor de explicabilidad (CU-021); y desde la comparativa estadística puede solicitarse su recálculo (CU-024) cuando el usuario desea actualizar los resultados. El resto de las operaciones del módulo se representan como interacciones directas e independientes del actor con el sistema.

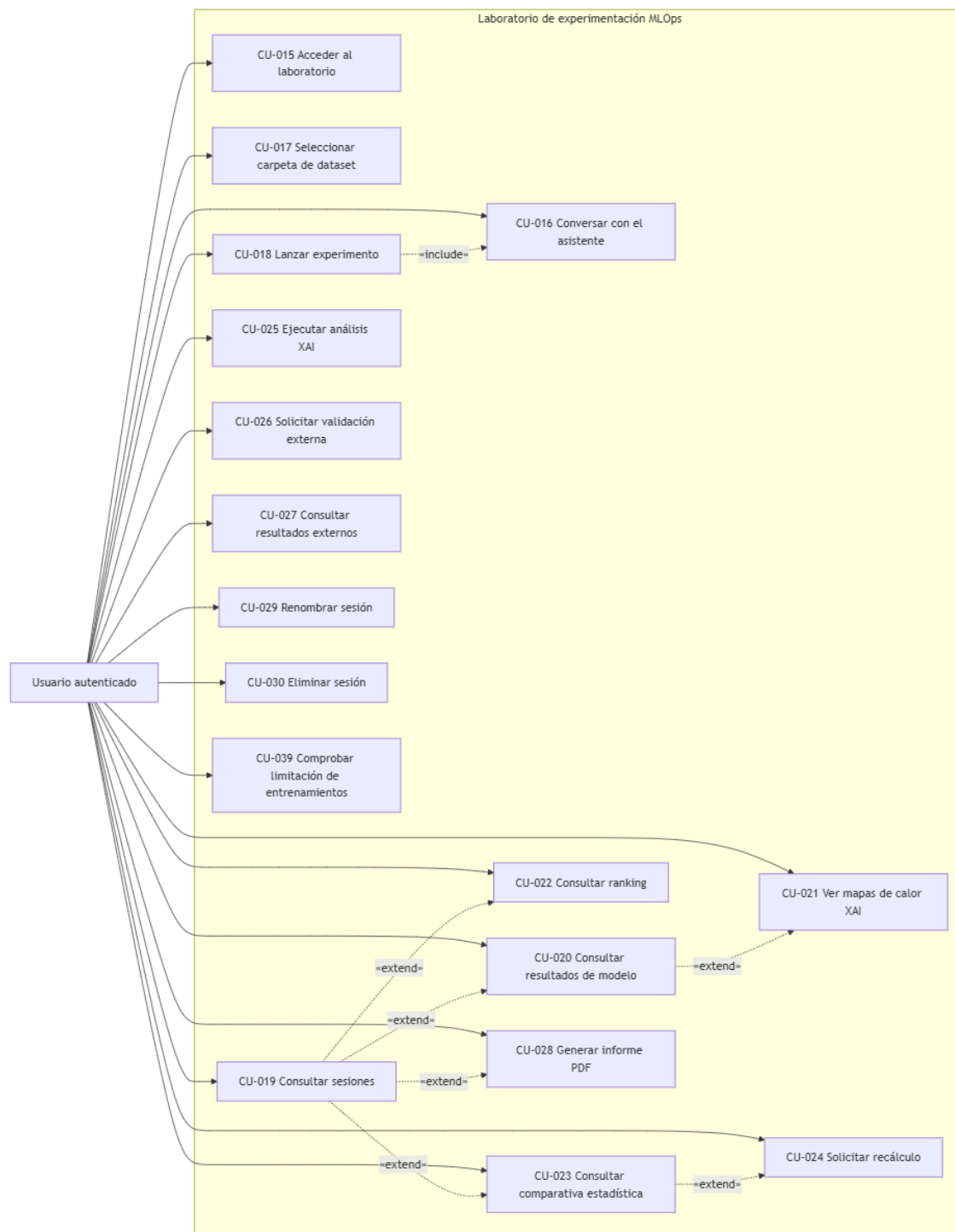


Ilustración x - Casos de uso del módulo de laboratorio de experimentación MLOps

12.2.1.4. Supervisión y administración de la plataforma

Este módulo agrupa los casos de uso del panel de administración, destinados al gobierno de la plataforma: la gestión y supervisión de las cuentas de usuario y de la actividad registrada en el sistema. Su existencia responde a la necesidad de que alguien asuma la responsabilidad sobre el conjunto de la plataforma: conocer quién la utiliza, supervisar la actividad que en ella se genera y disponer de los medios para auditar cualquier incidencia. Todas las operaciones de este módulo están restringidas al rol de administrador, que constituye el único perfil con acceso a ellas.

Es importante insistir en la naturaleza de este actor. El Administrador es, ante todo, un usuario autenticado que conserva todas las capacidades descritas en los demás módulos —diagnóstico, historial, laboratorio y funcionalidades transversales— y que añade, además, las operaciones de gobierno que aquí se recogen. En la figura se representa como Administrador para distinguir las operaciones exclusivas de este rol; el resto de sus capacidades coinciden con las del actor «Usuario autenticado» de los diagramas anteriores. El módulo comprende cuatro casos de uso: la consulta del listado de usuarios (CU-031), que ofrece una visión global de las cuentas registradas; la consulta de las consultas de diagnóstico de un usuario concreto (CU-032), que permite revisar la actividad clínica de una cuenta; la visualización del detalle de una de esas consultas (CU-033), con la imagen, el resultado, la confianza y los metadatos asociados; y la gestión de las cuentas de usuario (CU-038), que permite desactivar una cuenta, cambiar un rol o eliminar una cuenta.

Estos casos de uso permiten al administrador identificar las cuentas activas de la plataforma, supervisar la actividad que en ella se registra, auditar un caso concreto ante cualquier incidencia y ejercer el gobierno de las cuentas. Las relaciones de extensión reflejan la navegación desde el listado general hasta el caso concreto: desde el listado de usuarios (CU-031) se accede a las consultas de un usuario (CU-032), y desde ellas al detalle completo de una consulta (CU-033), de modo que el administrador pueda profundizar progresivamente en la información que necesita. La gestión de cuentas (CU-038) se representa como una operación independiente del administrador sobre el conjunto de cuentas.

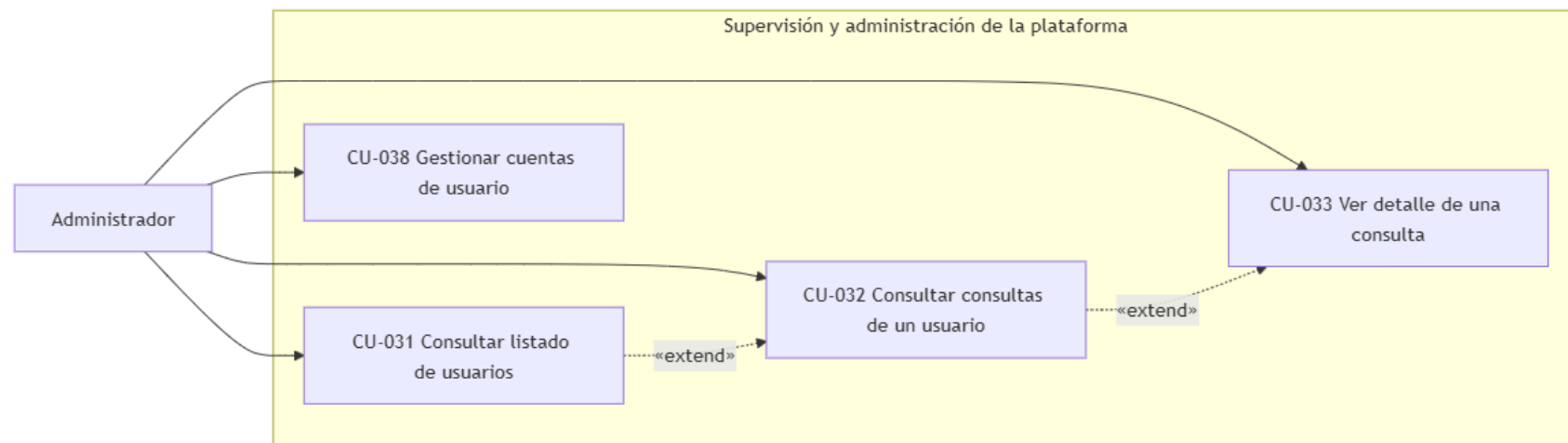


Ilustración x - Casos de uso del módulo de supervisión y administración de la plataforma

12.2.1.5. Capacidades transversales de la plataforma

Este módulo agrupa los casos de uso que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma. Su carácter transversal se aprecia en que están disponibles para todo usuario autenticado, con independencia del núcleo funcional en el que trabaje: un profesional puede consultar el estado de sus diagnósticos mientras un investigador supervisa sus entrenamientos, y ambos pueden cancelar un trabajo que haya quedado pendiente en la cola. El cambio del tema visual, por su parte, afecta a la totalidad de la interfaz y responde a una preferencia personal del usuario.

El módulo comprende tres casos de uso. La consulta del estado de la cola de trabajos (CU-034) muestra en tiempo real el estado de cada tarea pendiente, en ejecución, completada o fallida, cubriendo los tres tipos de trabajo que la plataforma ejecuta en segundo plano: los diagnósticos, los entrenamientos y las validaciones externas; el panel se actualiza de forma periódica y permite al usuario conocer en qué momento estará disponible un resultado o si un trabajo ha fallado. La cancelación de un trabajo de la cola (CU-035) impide que un trabajo pendiente llegue a ejecutarse y, cuando resulta técnicamente posible, interrumpe un trabajo en ejecución, de modo que un entrenamiento lanzado por error no monopolice la GPU sin posibilidad de detenerse. La alternancia entre el tema claro y el tema oscuro de la interfaz (CU-036) permite al usuario elegir la presentación visual que mejor se adapte a sus condiciones de trabajo.

Ninguno de los tres casos de uso presenta relaciones de inclusión o extensión con otros casos de uso de la plataforma: la consulta de la cola, la cancelación de trabajos y el cambio de tema se ejecutan de forma autónoma, sin delegar pasos obligatorios en otros casos ni ampliar ningún flujo base.

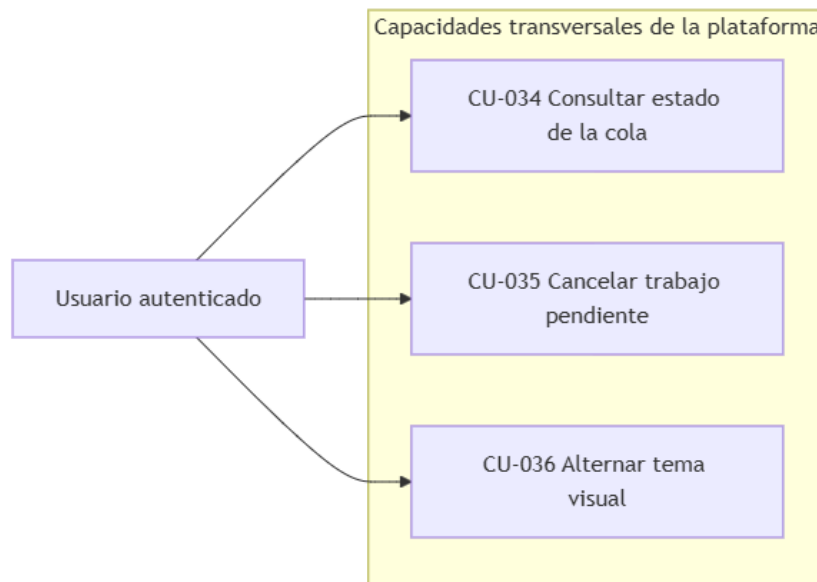


Ilustración x - Casos de uso del módulo de capacidades transversales de la plataforma

12.2.2. Actores del sistema y su ámbito de interacción

Un actor representa todo aquello que interactúa con el sistema desde el exterior, ya sea una persona o un sistema externo que produce o consume información de la plataforma (Jacobson, Booch, & Rumbaugh, 1999; Cockburn, 2001). Conviene distinguir con claridad entre los actores del sistema y los interesados del proyecto: un interesado es cualquier persona u organización que participa en el desarrollo o se ve afectada por su resultado —el tutor, los asesores, la universidad o los profesionales que aportaron su criterio durante el análisis—, mientras que un actor es únicamente quien mantiene una interacción directa con el sistema en tiempo de ejecución. Esta distinción evita confundir la lista de participantes en el proyecto con los roles que el sistema debe reconocer a la hora de autorizar y atender sus operaciones.

En vitalXAI se han identificado tres actores, todos ellos perfiles humanos que interactúan con la plataforma a través de la interfaz web. Los servicios internos que ejecutan los procesos —la cola de trabajos, el motor de inteligencia artificial y el asistente conversacional— forman parte de la arquitectura interna del sistema y no se modelan como actores, porque nunca inician una interacción con el usuario: responden a las solicitudes que este realiza a través de la interfaz, y su papel se describe en la parte de diseño de la memoria. Los perfiles humanos se derivan de la definición de usuarios del análisis: el visitante, que accede sin autenticarse; el usuario autenticado, que reúne a todos los profesionales con cuenta registrada, con independencia de que su actividad sea clínica o investigadora; y el administrador, responsable del gobierno de la plataforma. La Tabla 21 resume las características de los tres actores.

Actor	Tipo	Descripción	Alcance de acceso
Visitante	Primario	Persona que interactúa con la aplicación sin haber realizado el proceso de autenticación.	Funcionalidades de carácter público: registro, validación de credenciales y ajuste de idioma.
Usuario autenticado	Primario	Profesional registrado con acceso unificado a las dimensiones clínicas y de investigación del sistema.	Diagnóstico clínico, gestión de historial, laboratorio de entrenamiento MLOps y servicios transversales.
Administrador	Primario	Perfil facultado para el gobierno global y la supervisión de la actividad en la plataforma.	Capacidades operativas completas, gestión de cuentas y auditoría de consultas (CU-031 a CU-033).

Tabla x - Actores del sistema

El visitante es toda persona que accede a la plataforma sin haber iniciado sesión. Su interacción se limita a las funcionalidades públicas del sistema: la creación de una cuenta, el inicio de sesión y el cambio del idioma de la interfaz. Este perfil existe para que la plataforma permanezca abierta a nuevos registros sin exponer las funcionalidades de diagnóstico o de entrenamiento a quien no está autenticado. Tras completar el registro o el inicio de sesión, el visitante deja de serlo y pasa a disponer de una cuenta autenticada, con lo que se incorpora al conjunto de usuarios que pueden operar sobre las áreas privadas.

El usuario autenticado es el actor central del sistema y agrupa a todos los profesionales con cuenta registrada. Como se señaló en el análisis, la plataforma no restringe el acceso en función del perfil profesional: cualquier usuario con una cuenta puede utilizar tanto la interfaz de diagnóstico como el laboratorio de entrenamiento, de modo que la distinción entre el uso clínico y el uso investigador no se traduce en perfiles de acceso distintos, sino en la manera en que cada profesional emplea las mismas funcionalidades. El facultativo sin formación técnica encuentra en el diagnóstico asistido una herramienta clara y sin tecnicismos, mientras que el investigador dispone en el laboratorio de las capacidades de experimentación y de análisis estadístico que necesita. Este actor protagoniza la práctica totalidad de los casos de uso de la plataforma.

El administrador es un usuario autenticado que, además de conservar todas las capacidades del resto de los perfiles, dispone de las operaciones de gobierno de la plataforma recogidas en el módulo de supervisión y administración: la consulta del listado de usuarios, la consulta de las consultas de diagnóstico de un usuario concreto y la visualización del detalle de una de esas consultas. Su función es conocer quién utiliza la plataforma, supervisar la actividad que en ella se genera y auditar cualquier incidencia. La existencia de este rol se materializa mediante el mecanismo de roles y control de acceso del sistema, que restringe estas operaciones al perfil correspondiente.

El conjunto de actores descrito cubre la totalidad de las interacciones que la plataforma mantiene con el exterior: el visitante entra, el usuario autenticado trabaja y el administrador gobierna. Esta correspondencia garantiza que cada actor dispone de los casos de uso que necesita y que ningún rol accede a funcionalidades ajenas a su alcance, en cumplimiento del aislamiento de datos entre usuarios y de la restricción de las operaciones administrativas al rol correspondiente. Los diagramas presentados en el apartado anterior reflejan precisamente esta asignación de casos de uso a actores, de modo que la lectura conjunta de ambos apartados permite verificar de un vistazo quién puede hacer cada cosa y qué le aporta cada operación.

12.2.3. Actores del sistema y su ámbito de interacción

Los diagramas presentados en el apartado 12.2.1 ofrecen una visión de conjunto de los casos de uso y de los actores que los inician, pero no bastan para especificar el comportamiento del sistema: muestran qué interacciones existen y quién las inicia, pero no dicen cómo se desarrollan. Para cubrir esa carencia, cada caso de uso se describe aquí de forma detallada, relatando el flujo de acciones que el actor realiza y las respuestas que el sistema le ofrece en cada paso, de modo que la especificación resultante sea completa, inequívoca y verificable (Cockburn, 2001). Esta especificación constituye el contrato de comportamiento de la plataforma: fija qué condiciones deben darse para que la interacción tenga lugar, qué secuencia de pasos la compone, qué caminos alternativos pueden producirse cuando algo no sucede como se espera y qué estado queda en el sistema al terminar. Es, por tanto, el documento de referencia al que acudir tanto para diseñar los subsistemas como para decidir si una implementación cumple o no lo acordado.

A diferencia de los requisitos funcionales, que declaran qué debe hacer el sistema de forma técnica y verificable, los casos de uso describen esa misma capacidad desde la perspectiva del usuario, sin entrar en los detalles de implementación (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004). Un requisito funcional afirma, por ejemplo, que el sistema debe permitir iniciar sesión y otorgar acceso a las áreas privadas; el caso de uso correspondiente describe el recorrido completo que realiza la persona para lograrlo: cómo llega al formulario, qué introduce, qué verifica el sistema y qué ocurre tanto cuando todo es correcto como cuando no lo es. Esta perspectiva centrada en el usuario es precisamente lo que permite validar que el sistema responde a la manera en que los profesionales trabajan en la práctica y no solo a cómo se ha especificado sobre el papel: al leer un caso de uso, cualquier persona implicada en el proyecto —el tutor, un facultativo o el propio tribunal— puede seguir la secuencia y comprobar si refleja el comportamiento esperado. La redacción se mantiene, no obstante, al nivel de detalle suficiente para que la secuencia sea reproducible por quien vaya a diseñar o probar la funcionalidad, sin descender a mecanismos internos que el actor no percibe.

Cada caso de uso se identifica de forma inequívoca mediante el código CU-XXX y se especifica mediante una ficha que recoge los siguientes campos. El identificador y el nombre del caso de uso permiten referenciarlo de forma estable en todo el resto de la memoria. La fuente expresa el requisito o requisitos funcionales de los que deriva, de modo que quede constancia de qué capacidad del sistema materializa la interacción. Los actores indican qué perfiles participan en ella, y la descripción resume en una frase el propósito de la interacción. Las precondiciones enumeran las condiciones que deben cumplirse antes de iniciar el flujo; el flujo normal enumera, paso a paso y en orden, la secuencia de acciones y respuestas del camino de éxito; los flujos alternativos recogen los caminos que se desvían de ese recorrido, como los errores de validación, los datos ya existentes o las situaciones excepcionales, etiquetados con el número del paso del flujo normal en el que se producen; las postcondiciones describen el estado en que queda el sistema al finalizar, tanto tras el éxito como tras los caminos alternativos que sí dejan efectos; y, finalmente, la importancia y el estado del caso de uso completan la ficha. La trazabilidad con los requisitos funcionales queda así garantizada, porque cada caso de uso mantiene la misma estructura de módulos y deriva directamente de los requisitos del módulo correspondiente (Wieggers & Beatty, 2013).

Conviene precisar dos convenciones empleadas en la redacción de los flujos. En primer lugar, cada paso se numera de forma correlativa y se describe desde la perspectiva del actor: los pasos que inicia el actor se expresan en voz activa («el usuario sube la radiografía»), mientras que las respuestas del sistema se expresan como acciones de este («el sistema encola el análisis»), de modo que el lector distingue en todo momento quién actúa y quién responde. En segundo lugar, los flujos alternativos se referencian con el número del paso del flujo normal en el que se desvían, seguido de una letra que distingue cada alternativa: por ejemplo, la alternativa 4a se produce durante el paso 4 del flujo normal, y la 4b es una segunda desviación posible en ese mismo paso. Esta notación, tomada de la práctica habitual en la especificación de casos de uso, permite localizar con precisión dónde y en qué condiciones se produce cada desviación sin romper la lectura del flujo principal (Cockburn, 2001).

La relación entre los casos de uso y los requisitos funcionales no es, sin embargo, biunívoca: mientras que la mayoría de los requisitos funcionales se materializan en un caso de uso concreto, algunos requisitos de carácter transversal —como el aislamiento de datos entre usuarios o los roles y el control de acceso— no dan lugar a una interacción específica del actor, sino que condicionan la forma en que se ejecutan el resto de las interacciones. Estos requisitos se reflejan en las precondiciones y en los flujos alternativos de los casos de uso a los que afectan —por ejemplo, la comprobación de que un recurso pertenece al usuario que lo solicita—, y su cumplimiento global se verifica mediante los requisitos no funcionales correspondientes. De esta manera, la especificación de casos de uso no duplica la de requisitos, sino que la complementa: los requisitos declaran qué debe ofrecerse y bajo qué condiciones de calidad, y los casos de uso concretan cómo se materializa cada capacidad en la interacción diaria del usuario con la plataforma.

A continuación se especifican los treinta y seis casos de uso de vitalXAI, organizados en los cinco módulos funcionales presentados en el apartado de diagramas. El módulo de gestión del acceso y de la cuenta (CU-001 a CU-004) reúne las interacciones necesarias para crear una cuenta, entrar, salir y adaptar el idioma; la interfaz de diagnóstico asistido (CU-005 a CU-014) cubre el ciclo completo de una consulta clínica, desde la carga de la radiografía hasta la gestión del historial; el laboratorio de experimentación MLOps (CU-015 a CU-030) agrupa la configuración, el lanzamiento y el análisis de los resultados de los experimentos de entrenamiento; la supervisión y administración de la plataforma (CU-031 a CU-033) reúne las operaciones reservadas al administrador; y las capacidades transversales (CU-034 a CU-036) recogen la consulta de la cola de trabajos, la cancelación de trabajos pendientes y el cambio del tema visual. Los flujos se describen desde el punto de vista del actor, indicando en cada paso la acción que este realiza y la respuesta del sistema, de modo que la especificación pueda servir de base para el diseño de los subsistemas, para la elaboración de las pruebas de sistema y para la verificación final de que la plataforma entrega las capacidades comprometidas.

12.2.3.1. Módulo de gestión del acceso y de la cuenta

Este módulo agrupa los casos de uso que permiten a los usuarios acceder a la plataforma y gestionar su identidad en ella, y constituye el punto de entrada de todo el sistema: ninguna funcionalidad privada puede utilizarse sin haber completado previamente el proceso de autenticación que aquí se define. El módulo comprende cuatro casos de uso: la creación de la cuenta (CU-001), el inicio de sesión (CU-002), el cierre de sesión (CU-003) y el cambio del idioma de la interfaz (CU-004). Los tres primeros se corresponden con los requisitos funcionales de registro, acceso y cierre de sesión (RF-001 a RF-003), mientras que el cambio de idioma deriva del requisito de internacionalización (RF-004); los restantes requisitos del módulo —el aislamiento de datos entre usuarios (RF-005) y los roles y el control de acceso (RF-006)— no se materializan en casos de uso propios, porque constituyen condiciones transversales que se aplican a todas las interacciones de la plataforma y no interacciones concretas del actor.

Como se señaló al presentar los actores, el actor Administrador es un usuario autenticado con las mismas capacidades que el resto de los perfiles, por lo que en este módulo el actor «Usuario autenticado» lo incluye. Al tratarse de interacciones directas e independientes, ninguno de los casos de uso del módulo presenta relaciones de inclusión o extensión con los demás: el registro, el acceso, el cierre de sesión y el cambio de idioma se ejecutan de forma autónoma. A continuación se especifica cada uno de ellos.

CU-001 — Registrarse

Campo	Contenido
ID	CU-001
Nombre	Registrarse
Fuente	RF-001
Actores	Visitante
Descripción	El visitante crea una cuenta en la plataforma proporcionando nombre de usuario, nombre, apellidos, correo electrónico y contraseña. El sistema valida los datos, comprueba que la identidad no esté ya registrada y crea el registro almacenando la contraseña cifrada, de modo que nunca quede en texto plano.
Precondiciones	El visitante no está autenticado. El nombre de usuario y el correo electrónico no están registrados previamente.
Flujo normal	1. El visitante accede al formulario de registro desde la página de inicio de sesión. 2. El sistema muestra el formulario con los campos nombre de usuario, nombre, apellidos, correo electrónico y contraseña. 3. El visitante introduce sus datos y confirma el registro. 4. El sistema valida el formato de los datos y la fortaleza de la contraseña. 5. El sistema comprueba que el nombre de usuario y el correo electrónico no existen previamente. 6. El sistema cifra la contraseña mediante un algoritmo de hash robusto e irreversible, como bcrypt, y crea el registro del usuario. 7. El sistema muestra una confirmación y redirige al visitante a la página de inicio de sesión.
Flujo alternativo	4a. Si algún dato es inválido o la contraseña no cumple los requisitos de fortaleza, el sistema muestra un mensaje de error indicando el campo afectado y no crea el registro. 5a. Si el nombre de usuario o el correo electrónico ya están registrados, el sistema muestra un mensaje de error y no crea el registro.
Postcondiciones	El usuario queda registrado en la plataforma y puede iniciar sesión con sus credenciales.
Importancia	Alta
Estado	Aprobado
Comentarios	La contraseña se almacena siempre cifrada; los detalles del mecanismo concreto de cifrado son una decisión de diseño. El tratamiento de los datos personales del registro se ajusta al RGPD.

Tabla x - CU-001 — Registrarse

CU-002 — Iniciar sesión

Campo	Contenido
ID	CU-002
Nombre	Iniciar sesión
Fuente	RF-002
Actores	Visitante
Descripción	El visitante introduce sus credenciales y el sistema, tras verificarlas contra las almacenadas, inicia una sesión segura y le otorga acceso a las áreas privadas de la plataforma.
Precondiciones	El visitante dispone de una cuenta registrada. El visitante no tiene una sesión activa.
Flujo normal	1. El visitante accede al formulario de inicio de sesión. 2. El sistema muestra el formulario con los campos nombre de usuario y contraseña. 3. El visitante introduce sus credenciales y confirma. 4. El sistema verifica que la contraseña coincide con el hash almacenado para ese nombre de usuario. 5. El sistema genera el token de acceso y el token de refresco, y los establece en cookies seguras. 6. El sistema redirige al usuario a su panel.
Flujo alternativo	4a. Si las credenciales son incorrectas, el sistema muestra un mensaje de error genérico que no indica si el fallo está en el nombre de usuario o en la contraseña. 4b. Si se supera el límite de intentos permitidos, el sistema bloquea temporalmente las peticiones desde esa dirección y lo informa al usuario.
Postcondiciones	La sesión queda iniciada y el usuario accede a las áreas privadas de la plataforma.
Importancia	Alta
Estado	Aprobado
Comentarios	Los tokens se gestionan mediante cookies seguras. El límite de intentos y los mensajes genéricos previenen los ataques de fuerza bruta y de enumeración de usuarios, en línea con los requisitos no funcionales de seguridad.

Tabla x - CU-002 — Iniciar sesión

CU-003 — Cerrar sesión

Campo	Contenido
ID	CU-003
Nombre	Cerrar sesión
Fuente	RF-003
Actores	Usuario autenticado
Descripción	El usuario selecciona una imagen de su equipo para ser analizada. El sistema valida el tipo de archivo (JPEG o PNG) y su tamaño (máximo 10 MB) y la conserva en el servidor asociada a la consulta, de modo que pueda recuperarse desde el historial.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario selecciona el archivo de imagen desde su equipo. 2. El sistema valida el formato y el tamaño del archivo. 3. El sistema conserva la imagen asociada a la consulta, de modo que pueda recuperarse desde el historial.
Flujo alternativo	N/A
Postcondiciones	La sesión queda revocada y el acceso a las áreas privadas queda bloqueado hasta un nuevo inicio de sesión.
Importancia	Alta
Estado	Aprobado
Comentarios	El actor Administrador, por ser también un usuario autenticado, participa en este caso de uso con el mismo comportamiento.

Tabla x - CU-003 — Cerrar sesión

CU-004 — Cambiar el idioma de la interfaz

Campo	Contenido
ID	CU-004
Nombre	Cambiar el idioma de la interfaz
Fuente	RF-004
Actores	Visitante, Usuario autenticado
Descripción	El actor selecciona el idioma de la interfaz entre los disponibles (español, inglés, chino e hindú). El sistema guarda la preferencia y aplica las traducciones en la interfaz, en los informes generados y en el asistente conversacional.
Precondiciones	El actor accede a la plataforma, esté o no autenticado.
Flujo normal	1. El actor selecciona el idioma deseado en el selector de idioma de la interfaz. 2. El sistema guarda la preferencia. 3. El sistema aplica las traducciones correspondientes en toda la interfaz sin recargar la página.
Flujo alternativo	N/A
Postcondiciones	La interfaz se muestra en el idioma seleccionado y la preferencia queda guardada.
Importancia	Media
Estado	Aprobado
Comentarios	El actor Administrador, por ser también un usuario autenticado, queda incluido en el actor «Usuario autenticado» de este caso de uso.

Tabla x - CU-004 — Cambiar el idioma de la interfaz

12.2.3.2. Módulo de interfaz de diagnóstico asistido

Este módulo agrupa los casos de uso de la interfaz clínica, el primer núcleo funcional del sistema y el eje de la actividad de diagnóstico del profesional. A través de ella, el usuario autenticado realiza un diagnóstico asistido de neumonía a partir de una radiografía de tórax: carga la imagen, elige la arquitectura con la que desea realizar la consulta, solicita el diagnóstico y obtiene el resultado junto con su nivel de confianza y los mapas de explicabilidad que lo justifican. El diagnóstico se procesa de forma asíncrona mediante la cola de trabajos: en cuanto el usuario envía la petición, el sistema encola el análisis y la interfaz permanece operativa, de modo que el facultativo puede seguir trabajando mientras el modelo procesa la imagen, genera la predicción y produce la explicación visual. El módulo comprende los casos de uso CU-005 a CU-014, que se corresponden con los requisitos funcionales de la interfaz clínica (RF-007 a RF-012) y de la gestión del historial de consultas (RF-013 a RF-016). El informe PDF individual del diagnóstico se recoge en el caso de uso CU-037, asociado al requisito RF-039.

El módulo se organiza en dos bloques. El primero, centrado en la consulta de diagnóstico, reúne el acceso al panel (CU-005), la subida de la radiografía (CU-006), la selección de la arquitectura (CU-007), la solicitud del diagnóstico (CU-008), la visualización del resultado (CU-009) y la visualización de los mapas de explicabilidad (CU-010). El segundo, centrado en la gestión del historial, reúne la consulta del listado (CU-011), el detalle de una consulta (CU-012), su renombrado (CU-013) y su eliminación (CU-014). En este segundo bloque, las relaciones de extensión reflejan el orden en que el actor recorre la interfaz: desde el listado (CU-011) se accede al detalle (CU-012), y desde el detalle pueden ejecutarse, de forma opcional, el renombrado (CU-013) o la eliminación (CU-014). El aislamiento de datos entre usuarios (RF-005) se materializa en este módulo como una condición aplicada a todas las operaciones sobre consultas: cada consulta solo puede ser consultada, renombrada o eliminada por el usuario que la realizó.

CU-005 — Acceder al panel de diagnóstico

Campo	Contenido
ID	CU-005
Nombre	Acceder al panel de diagnóstico
Fuente	RF-007
Actores	Usuario autenticado
Descripción	El usuario autenticado accede al panel de diagnóstico, desde el que puede cargar una radiografía, seleccionar un modelo y consultar su historial de consultas.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al panel de diagnóstico desde la navegación principal. 2. El sistema valida la sesión activa del usuario. 3. El sistema carga el panel y lo muestra.
Flujo alternativo	2a. Si el usuario no tiene una sesión activa, el sistema deniega el acceso y redirige al usuario a la página de inicio de sesión.
Postcondiciones	El panel de diagnóstico queda visible y operativo para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	El panel constituye el punto de entrada a la funcionalidad clínica de la plataforma.

Tabla x - CU-005 — Acceder al panel de diagnóstico

CU-006 — Subir una radiografía de tórax

Campo	Contenido
ID	CU-006
Nombre	Subir una radiografía de tórax
Fuente	RF-008
Actores	Usuario autenticado
Descripción	El usuario selecciona una imagen de su equipo para ser analizada. El sistema valida el tipo de archivo (JPEG o PNG) y su tamaño (máximo 10 MB) y la almacena de forma temporal en el servidor, asociándola a la consulta en curso.
Precondiciones	El usuario se encuentra en el panel de diagnóstico.
Flujo normal	1. El usuario selecciona el archivo de imagen desde su equipo. 2. El sistema valida el formato y el tamaño del archivo. 3. El sistema almacena la imagen de forma temporal y la asocia a la consulta en curso.
Flujo alternativo	2a. Si el formato o el tamaño no son válidos, el sistema muestra un mensaje de error y rechaza la imagen, que no queda asociada a ninguna consulta.
Postcondiciones	La imagen queda disponible para el diagnóstico y asociada a la consulta en curso.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación se realiza antes de encolar la consulta, en línea con el requisito RF-008.

Tabla x - CU-006 — Subir una radiografía de tórax

CU-007 — Seleccionar la arquitectura para el diagnóstico

Campo	Contenido
ID	CU-007
Nombre	Selección de arquitectura de inferencia
Fuente	RF-009
Actores	Usuario autenticado
Descripción	El usuario elige, entre las arquitecturas de deep learning disponibles en la interfaz, el modelo con el que desea realizar el diagnóstico.
Precondiciones	El usuario se encuentra en el panel de diagnóstico.
Flujo normal	1. El usuario despliega el selector de modelos del panel. 2. El sistema muestra la lista de arquitecturas disponibles. 3. El usuario selecciona la arquitectura deseada.
Flujo alternativo	N/A
Postcondiciones	La arquitectura queda seleccionada para la consulta en curso.
Importancia	Media
Estado	Aprobado
Comentarios	La lista de arquitecturas disponibles se muestra en el panel de diagnóstico.

Tabla x - CU-007 — Seleccionar la arquitectura para el diagnóstico

CU-008 — Solicitar un diagnóstico

Campo	Contenido
ID	CU-008
Nombre	Solicitar un diagnóstico
Fuente	RF-010
Actores	Usuario autenticado
Descripción	El usuario envía la petición de diagnóstico con la imagen y el modelo seleccionados. El sistema valida la petición, encola el trabajo de diagnóstico y lo procesa en segundo plano, sin bloquear la interfaz.
Precondiciones	Hay una imagen subida y un modelo seleccionado en la consulta en curso.
Flujo normal	1. El usuario envía la petición de diagnóstico. 2. El sistema valida la imagen y el modelo seleccionado. 3. El sistema encola el trabajo de diagnóstico. 4. El worker procesa el trabajo y genera la predicción, los mapas de explicabilidad y el informe de la consulta. 5. El sistema notifica al usuario la finalización del diagnóstico.
Flujo alternativo	2a. Si la petición no es válida, el sistema muestra un mensaje de error y no encola el trabajo. 4a. Si el procesamiento falla, el trabajo pasa a estado fallido, se registra el error y se informa al usuario.
Postcondiciones	La consulta queda registrada en el historial con su resultado, su confianza, sus mapas de explicabilidad y su informe.
Importancia	Alta
Estado	Aprobado
Comentarios	La presentación del resultado (CU-009), de los mapas (CU-010) y del informe (CU-037) son capacidades verificables por separado; este caso de uso cubre exclusivamente la solicitud y el encolado del diagnóstico.

Tabla x - CU-008 — Solicitar un diagnóstico

CU-009 — Visualizar el resultado del diagnóstico

Campo	Contenido
ID	CU-009
Nombre	Visualizar el resultado del diagnóstico
Fuente	RF-011
Actores	Usuario autenticado
Descripción	Cuando el trabajo de diagnóstico finaliza, el sistema muestra el resultado de la consulta: la predicción (PNEUMONIA o NORMAL), el nivel de confianza asociado y el modelo utilizado.
Precondiciones	La consulta ha sido procesada por el worker y ha pasado a estado completado.
Flujo normal	1. El usuario espera a que la consulta pase a estado completado. 2. El sistema muestra el resultado con su nivel de confianza y el modelo empleado.
Flujo alternativo	1a. Si la consulta ha pasado a estado fallido, el sistema muestra el estado fallido e informa del error, sin presentar ningún resultado.
Postcondiciones	El resultado queda visible para el usuario y registrado en su historial.
Importancia	Alta
Estado	Aprobado
Comentarios	La consulta queda registrada en el historial con su resultado, en línea con el requisito RF-011.

Tabla x - CU-009 — Visualizar el resultado del diagnóstico

CU-010 — Visualizar los mapas de explicabilidad

Campo	Contenido
ID	CU-010
Nombre	Visualización de los mapas de explicabilidad
Fuente	RF-012
Actores	Usuario autenticado
Descripción	El usuario visualiza los mapas de calor generados por el sistema (Saliency Maps, SmoothGrad y Grad-CAM para arquitecturas convolucionales, o mapas de atención para arquitecturas Transformer), superpuestos sobre la radiografía original.
Precondiciones	La consulta ha sido procesada y los mapas de explicabilidad han sido generados.
Flujo normal	1. El usuario selecciona la consulta completada. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema muestra el mosaico con la radiografía original y los mapas de explicabilidad.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no muestra la información.
Postcondiciones	Los mapas de explicabilidad quedan visibles para su inspección.
Importancia	Alta
Estado	Aprobado
Comentarios	La generación de las explicaciones se apoya en las técnicas XAI descritas en el capítulo 1, y su evaluación cuantitativa se recoge en el laboratorio (CU-020).

Tabla x - CU-010 — Visualizar los mapas de explicabilidad

CU-011 — Consultar el historial de consultas

Campo	Contenido
ID	CU-011
Nombre	Consulta del registro de diagnósticos
Fuente	RF-013
Actores	Usuario autenticado
Descripción	El usuario consulta el listado de sus consultas de diagnóstico, en el que se muestran la fecha, el modelo empleado, el resultado y la confianza de cada una.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede a su historial desde el panel de diagnóstico. 2. El sistema recupera y muestra únicamente las consultas del propio usuario.
Flujo alternativo	2a. Si el usuario no tiene consultas registradas, el sistema muestra un mensaje informativo y un listado vacío.
Postcondiciones	El listado del historial queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las consultas del propio usuario, en cumplimiento del aislamiento de datos entre usuarios.

Tabla x - CU-011 — Consultar el historial de consultas

CU-012 — Ver el detalle de una consulta del historial

Campo	Contenido
ID	CU-012
Nombre	Ver el detalle de una consulta del historial
Fuente	RF-014
Actores	Usuario autenticado
Descripción	El usuario abre el detalle de una consulta de su historial, que incluye la radiografía original, el resultado, la confianza, los mapas de explicabilidad y los metadatos asociados
Precondiciones	La consulta pertenece al usuario.
Flujo normal	1. El usuario selecciona una consulta del listado del historial. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema muestra el detalle completo de la consulta.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no muestra la información.
Postcondiciones	El detalle de la consulta queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-011: el detalle se alcanza desde el listado del historial.

Tabla x - CU-012 — Ver el detalle de una consulta del historial

CU-013 — Renombrar una consulta del historial

Campo	Contenido
ID	CU-013
Nombre	Renombrar una consulta del historial
Fuente	RF-015
Actores	Usuario autenticado
Descripción	El usuario modifica el nombre de una de sus consultas del historial para identificarla mejor.
Precondiciones	La consulta pertenece al usuario.
Flujo normal	1. El usuario indica el nuevo nombre de la consulta desde el detalle de la misma. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema valida que el nuevo nombre no está vacío. 4. El sistema actualiza el nombre de la consulta.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no modifica nada. 3a. Si el nuevo nombre está vacío, el sistema muestra un mensaje de error y mantiene el nombre anterior.
Postcondiciones	La consulta queda renombrada en el historial del usuario.
Importancia	Baja
Estado	Aprobado
Comentarios	El renombrado no afecta a la imagen, al resultado ni a los mapas de la consulta. Este caso de uso extiende CU-012.

Tabla x - CU-013 — Renombrar una consulta del historial

CU-014 — Eliminar una consulta del historial

Campo	Contenido
ID	CU-014
Nombre	Eliminar una consulta del historial
Fuente	RF-016
Actores	Usuario autenticado
Descripción	El usuario elimina una consulta de su historial de forma definitiva, de modo que el registro y sus artefactos asociados dejan de estar disponibles tanto para el propio usuario como para el administrador.
Precondiciones	La consulta pertenece al usuario.
Flujo normal	1. El usuario solicita la eliminación de la consulta desde el detalle de la misma. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema solicita confirmación de la eliminación. 4. El usuario confirma la eliminación. 5. El sistema elimina el registro de la consulta y sus artefactos asociados.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no elimina nada. 4a. Si el usuario cancela la operación, la consulta se conserva sin cambios.
Postcondiciones	La consulta y sus artefactos quedan eliminados de forma definitiva del sistema.
Importancia	Baja
Estado	Aprobado
Comentarios	La eliminación se realiza únicamente sobre consultas del propio usuario y es definitiva, en coherencia con el derecho de supresión del RGPD. Este caso de uso extiende CU-012.

Tabla x - CU-014 — Eliminar una consulta del historial

12.2.3.3. Módulo de laboratorio de experimentación MLOps

Este módulo agrupa los casos de uso del laboratorio de entrenamiento, el segundo núcleo funcional del sistema y el espacio en el que se desarrolla la actividad investigadora. A través de él, el usuario configura y lanza experimentos de entrenamiento mediante un asistente conversacional, monitoriza la ejecución del pipeline y consulta los resultados comparativos y estadísticos de sus sesiones. El laboratorio orquesta automáticamente la secuencia de entrenamiento, análisis de explicabilidad, comparación estadística y validación externa, de modo que el investigador no necesita escribir código en ningún momento: el asistente interpreta sus indicaciones, el pipeline ejecuta los procesos en segundo plano y la interfaz presenta los resultados de forma organizada. El módulo comprende los casos de uso CU-015 a CU-030, que se corresponden con los requisitos funcionales del laboratorio (RF-017 a RF-032) y se asocian a los objetivos de laboratorio de entrenamiento MLOps (OBJ-004), evaluación rigurosa de los modelos (OBJ-005) y ejecución asíncrona de tareas (OBJ-010). La limitación de los entrenamientos simultáneos y encolados se recoge en el caso de uso CU-039, asociado al requisito RF-041.

El lanzamiento del experimento (CU-018) incluye de forma obligatoria la conversación con el asistente (CU-016), porque la configuración del experimento debe quedar completa antes de iniciar el entrenamiento. Las relaciones de extensión siguen la navegación del usuario por el laboratorio: desde la consulta de las sesiones (CU-019) se accede a los resultados de un modelo (CU-020), al ranking (CU-022), a la comparativa estadística (CU-023) o al informe PDF (CU-028); desde los resultados de un modelo se alcanzan sus mapas de explicabilidad (CU-021); y desde la comparativa estadística puede solicitarse su recálculo (CU-024). El aislamiento de datos entre usuarios (RF-005) se aplica también en este módulo: cada sesión solo puede ser consultada, renombrada o eliminada por el usuario que la lanzó.

CU-015 — Acceso al laboratorio de entrenamiento

Campo	Contenido
ID	CU-015
Nombre	Acceso al laboratorio de entrenamiento
Fuente	RF-017
Actores	Usuario autenticado
Descripción	El usuario autenticado accede al laboratorio de entrenamiento, desde el que puede conversar con el asistente, lanzar experimentos y consultar sus sesiones.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al laboratorio desde la navegación principal. 2. El sistema valida la sesión activa del usuario. 3. El sistema carga el entorno del laboratorio y lo muestra.
Flujo alternativo	2a. Si el usuario no tiene una sesión activa, el sistema deniega el acceso y redirige al usuario a la página de inicio de sesión.
Postcondiciones	El laboratorio queda visible y operativo para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	El laboratorio es accesible para todo usuario autenticado, con independencia de su perfil profesional.

Tabla x - CU-015 — Acceso al laboratorio de entrenamiento

CU-016 — Conversar con el asistente para configurar un experimento

Campo	Contenido
ID	CU-016
Nombre	Conversar con el asistente para configurar un experimento
Fuente	RF-018
Actores	Usuario autenticado
Descripción	El usuario conversa en lenguaje natural con el asistente conversacional para definir los parámetros del experimento: arquitecturas a entrenar, número de épocas, tamaño de lote y tasa de aprendizaje. El asistente interpreta las indicaciones y, cuando dispone de todos los parámetros, devuelve la configuración estructurada. La ruta del dataset se incorpora mediante el caso de uso CU-017.
Precondiciones	El usuario se encuentra en el laboratorio.
Flujo normal	1. El usuario envía un mensaje al asistente indicando los parámetros del experimento. 2. El sistema envía la petición al modelo de lenguaje con el prompt de sistema definido. 3. El modelo extrae los parámetros mencionados en el mensaje. 4. Si todos los parámetros están definidos, el asistente devuelve la configuración estructurada. 5. El sistema rellena el panel de configuración con los valores obtenidos.
Flujo alternativo	4a. Si faltan parámetros, el asistente pregunta por ellos y la conversación continúa hasta completarlos. 2a. Si el servicio del modelo de lenguaje no está disponible, el sistema informa del error y la configuración no se completa.
Postcondiciones	La configuración del experimento queda disponible para su revisión y lanzamiento, con la ruta del dataset incorporada.
Importancia	Alta
Estado	Aprobado
Comentarios	La conversación se desarrolla en el idioma seleccionado por el usuario. La ruta del dataset se obtiene mediante CU-017, no mediante la conversación.

Tabla x - CU-016 — Conversar con el asistente para configurar un experimento

CU-017 — Seleccionar la carpeta del dataset

Campo	Contenido
ID	CU-017
Nombre	Seleccionar la carpeta del dataset
Fuente	RF-019
Actores	Usuario autenticado
Descripción	El usuario selecciona el dataset de entrenamiento y el dataset externo de validación dentro del directorio raíz de datasets permitido. El sistema valida que la ruta permanece dentro de ese directorio y no expone el resto del sistema de ficheros del servidor.
Precondiciones	El usuario se encuentra en el laboratorio y está configurando un experimento.
Flujo normal	1. El usuario solicita seleccionar el dataset. 2. El sistema muestra las opciones disponibles dentro del directorio raíz de datasets permitido. 3. El usuario confirma la ruta. 4. El sistema valida que la ruta permanece dentro del directorio permitido.
Flujo alternativo	2a. Si la ruta indicada no es accesible, el sistema muestra un mensaje de error y permite seleccionar otra.
Postcondiciones	La ruta del dataset queda disponible para el experimento.
Importancia	Media
Estado	Aprobado
Comentarios	La selección se confina al directorio de datasets permitido para impedir el acceso a rutas arbitrarias del servidor y respetar el aislamiento entre usuarios (RF-005).

Tabla x - CU-017 — Seleccionar la carpeta del dataset

CU-018 — Lanzar un experimento de entrenamiento

Campo	Contenido
ID	CU-018
Nombre	Lanzar un experimento de entrenamiento
Fuente	RF-020
Actores	Usuario autenticado
Descripción	El usuario lanza el experimento con la configuración definida. El sistema crea una sesión de entrenamiento y encola la ejecución de forma asíncrona, de modo que la interfaz permanece operativa durante el entrenamiento.
Precondiciones	La configuración del experimento está completa, incluida la ruta del dataset (CU-016 y CU-017).
Flujo normal	1. El usuario envía la configuración del experimento. 2. El sistema crea la sesión de entrenamiento y encola el trabajo. 3. El worker ejecuta el entrenamiento de las arquitecturas solicitadas. 4. El sistema actualiza el estado de la sesión al finalizar.
Flujo alternativo	3a. Si algún script del pipeline falla, la sesión queda registrada con el error correspondiente y el usuario es informado.
Postcondiciones	La sesión de entrenamiento queda creada con su estado.
Importancia	Alta
Estado	Aprobado
Comentarios	Este caso de uso incluye CU-016 y CU-017: la configuración y el dataset deben estar definidos antes de lanzar el experimento. El análisis de explicabilidad y la comparación estadística se consultan mediante CU-020 a CU-023.

Tabla x - CU-018 — Lanzar un experimento de entrenamiento

CU-019 — Consultar las sesiones de entrenamiento

Campo	Contenido
ID	CU-019
Nombre	Consultar las sesiones de entrenamiento
Fuente	RF-021
Actores	Usuario autenticado
Descripción	El usuario consulta el listado de sus sesiones de entrenamiento con su estado y sus modelos.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al listado de sesiones del laboratorio. 2. El sistema recupera y muestra únicamente las sesiones del propio usuario, con su estado y sus modelos.
Flujo alternativo	2a. Si el usuario no tiene sesiones de entrenamiento, el sistema muestra un mensaje informativo y un listado vacío.
Postcondiciones	El listado de sesiones queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Solo se muestran las sesiones del propio usuario, en cumplimiento del aislamiento de datos entre usuarios.

Tabla x - CU-019 — Consultar las sesiones de entrenamiento

CU-020 — Consultar los resultados de un modelo de la sesión

Campo	Contenido
ID	CU-020
Nombre	Consultar los resultados de un modelo de la sesión
Fuente	RF-022
Actores	Usuario autenticado
Descripción	El usuario consulta los resultados cuantitativos de un modelo concreto de la sesión: las métricas de la validación cruzada (exactitud, precisión, sensibilidad, F1 y AUC), las métricas XAI cuantitativas y las métricas de calibración, con su media y su desviación sobre los pliegues.
Precondiciones	La sesión dispone de resultados para el modelo seleccionado.
Flujo normal	1. El usuario selecciona un modelo de la sesión. 2. El sistema recupera y muestra las métricas del modelo con su media y desviación sobre los pliegues.
Flujo alternativo	2a. Si el modelo no dispone de resultados, el sistema muestra un mensaje informativo.
Postcondiciones	Los resultados del modelo quedan visibles para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-019. La desagregación por pliegue es una decisión del diseño de presentación, coherente con el modelo de resultados persistido de la sesión.

Tabla x - CU-020 — Consultar los resultados de un modelo de la sesión

CU-021 — Visualizar los mapas de calor de explicabilidad de un modelo

Campo	Contenido
ID	CU-021
Nombre	Visualizar los mapas de calor de explicabilidad de un modelo
Fuente	RF-023
Actores	Usuario autenticado
Descripción	El usuario visualiza la galería de mapas de calor de explicabilidad del modelo, generados por el análisis XAI cualitativo sobre imágenes de ejemplo.
Precondiciones	El modelo dispone de mapas de explicabilidad generados.
Flujo normal	1. El usuario selecciona el modelo de la sesión. 2. El sistema muestra la galería de imágenes XAI del modelo.
Flujo alternativo	2a. Si el modelo no dispone de mapas de explicabilidad, el sistema muestra un mensaje informativo.
Postcondiciones	Los mapas de calor quedan visibles para su inspección.
Media	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-020: la galería se alcanza desde los resultados del modelo.

Tabla x - CU-021 — Visualizar los mapas de calor de explicabilidad de un modelo

CU-022 — Consultar el ranking de modelos de la sesión

Campo	Contenido
ID	CU-022
Nombre	Consultar el ranking de modelos de la sesión
Fuente	RF-024
Actores	Usuario autenticado
Descripción	El usuario consulta el ranking global de los modelos de la sesión, ordenado por su AUC medio de la validación cruzada, junto con su desviación típica.
Precondiciones	La comparación estadística de la sesión se ha generado.
Flujo normal	1. El usuario solicita el ranking de la sesión. 2. El sistema recupera y muestra el ranking de los modelos ordenados por su AUC medio.
Flujo alternativo	2a. Si la comparación estadística no se ha generado, el sistema muestra un mensaje informativo.
Postcondiciones	El ranking de modelos queda visible para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-019. El ranking se regenera al ejecutar la comparación estadística.

Tabla x - CU-022 — Consultar el ranking de modelos de la sesión

CU-023 — Consultar la comparativa estadística de la sesión

Campo	Contenido
ID	CU-023
Nombre	Consultar la comparativa estadística de la sesión
Fuente	RF-025
Actores	Usuario autenticado
Descripción	El usuario consulta la matriz de significación estadística que compara los modelos de la sesión: los p-valores del test de Wilcoxon y, cuando la validación externa se ha ejecutado, los del test de DeLong sobre las curvas ROC.
Precondiciones	La comparación estadística de la sesión se ha generado.
Flujo normal	1. El usuario accede a la vista de comparativa de la sesión. 2. El sistema muestra la matriz de significación correspondiente.
Flujo alternativo	2a. Si la comparación estadística no se ha generado, el sistema muestra un mensaje informativo.
Postcondiciones	La comparativa estadística queda visible para el usuario.
Importancia	Alta
Estado	Aprobado
Comentarios	La interpretación de la significación debe considerar la potencia limitada de la prueba con el número de pliegues disponible; este caso de uso no fija un umbral. Extiende CU-019.

Tabla x - CU-023 — Consultar la comparativa estadística de la sesión

CU-024 — Solicitar el recálculo de la comparativa estadística

Campo	Contenido
ID	CU-024
Nombre	Solicitar el recálculo de la comparativa estadística
Fuente	RF-026
Actores	Usuario autenticado
Descripción	El usuario solicita el recálculo de la comparativa estadística de la sesión, regenerando el ranking y el test de Wilcoxon de forma asíncrona.
Precondiciones	La sesión dispone de resultados de sus modelos.
Flujo normal	1. El usuario solicita el recálculo de la comparativa. 2. El sistema lanza el proceso de recálculo en segundo plano. 3. El sistema actualiza la comparativa y notifica al usuario cuando el recálculo finaliza.
Flujo alternativo	2a. Si el proceso de recálculo falla, el sistema registra el error e informa al usuario.
Postcondiciones	La comparativa estadística de la sesión queda actualizada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-023: el recálculo se solicita desde la comparativa estadística.

Tabla x - CU-024 — Solicitar el recálculo de la comparativa estadística

CU-025 — Ejecutar el análisis de explicabilidad de un modelo

Campo	Contenido
ID	CU-025
Nombre	Ejecutar el análisis de explicabilidad de un modelo
Fuente	RF-027
Actores	Usuario autenticado
Descripción	El usuario solicita la ejecución manual del análisis de explicabilidad (cualitativo y cuantitativo) de un modelo de la sesión, regenerando sus mapas y sus métricas.
Precondiciones	El modelo pertenece a una sesión del usuario.
Flujo normal	1. El usuario solicita generar el análisis XAI del modelo. 2. El sistema ejecuta los scripts de explicabilidad en segundo plano. 3. El sistema notifica al usuario cuando el análisis finaliza.
Flujo alternativo	2a. Si el análisis falla, el sistema registra el error e informa al usuario.
Postcondiciones	Los mapas y las métricas XAI del modelo quedan regenerados.
Importancia	Media
Estado	Aprobado
Comentarios	La ejecución manual complementa el análisis automático del pipeline.

Tabla x - CU-025 — Ejecutar el análisis de explicabilidad de un modelo

CU-026 — Solicitar la validación externa de la sesión

Campo	Contenido
ID	CU-026
Nombre	Solicitar la validación externa de la sesión
Fuente	RF-028
Actores	Usuario autenticado
Descripción	El usuario solicita la validación externa de la sesión sobre el dataset externo de pacientes adultos. El sistema encola el trabajo, que evalúa los modelos congelados y aplica el test de DeLong para comparar sus curvas ROC, de forma asíncrona.
Precondiciones	La sesión dispone de modelos entrenados y de un dataset externo disponible.
Flujo normal	1. El usuario solicita la validación externa de la sesión. 2. El sistema encola el trabajo de validación externa. 3. El worker evalúa los modelos congelados y aplica el test de DeLong. 4. El sistema notifica al usuario cuando la validación finaliza.
Flujo alternativo	3a. Si la validación externa falla, el sistema registra el error e informa al usuario.
Postcondiciones	Los resultados de la validación externa quedan disponibles para su consulta.
Importancia	Alta
Estado	Aprobado
Comentarios	La validación externa se encola y se procesa en segundo plano, en línea con el objetivo de ejecución asíncrona (OBJ-010).

Tabla x - CU-026 — Solicitar la validación externa de la sesión

CU-027 — Consultar los resultados de la validación externa

Campo	Contenido
ID	CU-027
Nombre	Consultar los resultados de la validación externa
Fuente	RF-029
Actores	Usuario autenticado
Descripción	El usuario consulta los resultados de la validación externa de la sesión: las métricas sobre la cohorte externa, las curvas ROC de los modelos y la matriz de significación del test de DeLong.
Precondiciones	La validación externa de la sesión se ha ejecutado.
Flujo normal	1. El usuario accede a los resultados externos de la sesión. 2. El sistema muestra las métricas, las curvas ROC y la matriz de DeLong.
Flujo alternativo	2a. Si la validación externa no se ha ejecutado, el sistema muestra un mensaje informativo.
Postcondiciones	Los resultados externos quedan visibles para el usuario.
Importancia	Media
Estado	Aprobado
Comentarios	Los resultados solo están disponibles tras completar la validación externa.

Tabla x - CU-027 — Consultar los resultados de la validación externa

CU-028 — Generar el informe PDF de la sesión

Campo	Contenido
ID	CU-028
Nombre	Generar el informe PDF de la sesión
Fuente	RF-030
Actores	Usuario autenticado
Descripción	El usuario genera y descarga el informe PDF consolidado de la sesión, que recoge la configuración, el ranking, la matriz de Wilcoxon, los resultados de la validación externa con su matriz de DeLong y las métricas de explicabilidad y calibración por modelo.
Precondiciones	La sesión pertenece al usuario y dispone de resultados.
Flujo normal	1. El usuario solicita el informe de la sesión. 2. El sistema genera el documento PDF con los resultados consolidados. 3. El sistema lo descarga al equipo del usuario.
Flujo alternativo	2a. Si la generación del informe falla, el sistema informa del error y no descarga el documento.
Postcondiciones	El usuario dispone del informe PDF de la sesión.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-019: el informe se genera desde la sesión.

Tabla x - CU-028 — Generar el informe PDF de la sesión

CU-029 — Renombrar una sesión de entrenamiento

Campo	Contenido
ID	CU-029
Nombre	Renombrar una sesión de entrenamiento
Fuente	RF-031
Actores	Usuario autenticado
Descripción	El usuario modifica el nombre de una de sus sesiones de entrenamiento para identificarla mejor.
Precondiciones	La sesión pertenece al usuario.
Flujo normal	1. El usuario indica el nuevo nombre de la sesión. 2. El sistema comprueba que la sesión pertenece al usuario. 3. El sistema valida que el nuevo nombre no está vacío. 4. El sistema actualiza el nombre de la sesión.
Flujo alternativo	2a. Si la sesión no pertenece al usuario, el sistema deniega el acceso y no modifica nada. 3a. Si el nuevo nombre está vacío, el sistema muestra un mensaje de error y mantiene el nombre anterior.
Postcondiciones	La sesión queda renombrada en el laboratorio del usuario.
Baja	Baja
Estado	Aprobado
Comentarios	El renombrado no afecta a los modelos ni a los resultados de la sesión.

Tabla x - CU-029 — Renombrar una sesión de entrenamiento

CU-030 — Eliminar una sesión de entrenamiento

Campo	Contenido
ID	CU-030
Nombre	Eliminar una sesión de entrenamiento
Fuente	RF-032
Actores	Usuario autenticado
Descripción	El usuario elimina una de sus sesiones de entrenamiento y sus resultados asociados.
Precondiciones	La sesión pertenece al usuario.
Flujo normal	1. El usuario solicita la eliminación de la sesión. 2. El sistema comprueba que la sesión pertenece al usuario. 3. El sistema solicita confirmación de la eliminación. 4. El usuario confirma la eliminación. 5. El sistema elimina la sesión y sus artefactos asociados.
Flujo alternativo	2a. Si la sesión no pertenece al usuario, el sistema deniega el acceso y no elimina nada. 4a. Si el usuario cancela la operación, la sesión se conserva sin cambios.
Postcondiciones	La sesión desaparece del laboratorio del usuario.
Baja	Baja
Estado	Aprobado
Comentarios	La eliminación se realiza únicamente sobre sesiones del propio usuario.

Tabla x - CU-030 — Eliminar una sesión de entrenamiento

12.2.3.4. Módulo de supervisión y administración de la plataforma

Este módulo agrupa los casos de uso del panel de administración, destinados al gobierno de la plataforma: la gestión y supervisión de las cuentas de usuario y de la actividad registrada en el sistema. Su existencia responde a la necesidad de que alguien asuma la responsabilidad sobre el conjunto de la plataforma: conocer quién la utiliza, supervisar la actividad que en ella se genera y disponer de los medios para auditar cualquier incidencia. Todas las operaciones de este módulo están restringidas al rol de administrador, que constituye el único perfil con acceso a ellas, de modo que el mecanismo de roles y control de acceso (RF-006) garantiza que ningún otro usuario puede invocar estos casos de uso. El módulo comprende los casos de uso CU-031 a CU-033, que se corresponden con los requisitos funcionales de administración (RF-033 a RF-035) y se asocian al objetivo de administración de la plataforma (OBJ-011). La gestión de cuentas de usuario se recoge en el caso de uso CU-038, asociado al requisito RF-040.

El Administrador es, ante todo, un usuario autenticado que conserva todas las capacidades descritas en los demás módulos y que añade las operaciones de gobierno que aquí se recogen. Los tres casos de uso se encadenan mediante relaciones de extensión que reflejan la navegación del administrador desde el listado general hasta el caso concreto: desde el listado de usuarios (CU-031) se accede a las consultas de un usuario (CU-032), y desde ellas al detalle completo de una consulta (CU-033). Esta progresión permite al administrador profundizar gradualmente en la información, partiendo de una visión global de las cuentas y llegando, si lo necesita, al detalle de una consulta individual con su imagen, su resultado y sus metadatos.

CU-031 — Consultar el listado de usuarios

Campo	Contenido
ID	CU-031
Nombre	Consultar el listado de usuarios
Fuente	RF-033
Actores	Administrador
Descripción	El administrador consulta el listado de usuarios registrados en la plataforma, lo que le permite identificar cuentas y comprobar el estado del sistema.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador accede al panel de administración. 2. El sistema verifica el rol de administración del actor. 3. El sistema recupera y muestra el listado de usuarios registrados.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no muestra el listado.
Postcondiciones	El listado de usuarios queda visible para el administrador.
Importancia	Media
Estado	Aprobado
Comentarios	Esta funcionalidad está restringida al rol de administrador.

Tabla x - CU-031 — Consultar el listado de usuarios

CU-032 — Consultar las consultas de un usuario

Campo	Contenido
ID	CU-032
Nombre	Consultar las consultas de un usuario
Fuente	RF-034
Actores	Administrador
Descripción	El administrador consulta el historial de consultas de diagnóstico no eliminadas de un usuario concreto de la plataforma, dentro de su función de supervisión.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador selecciona un usuario del listado. 2. El sistema verifica el rol de administración del actor. 3. El sistema recupera y muestra las consultas de diagnóstico no eliminadas de ese usuario. 4. El sistema registra la operación de auditoría.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no muestra las consultas.
Postcondiciones	El listado de consultas del usuario queda visible para el administrador y la operación queda registrada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-031. El acceso del administrador a los datos de otro usuario es la excepción al aislamiento definida en RF-005, queda acotado a la función de supervisión, se limita a las consultas no eliminadas (RF-016) y se registra conforme a RNF-006.

Tabla x - CU-032 — Consultar las consultas de un usuario

CU-033 — Ver el detalle de una consulta de un usuario

Campo	Contenido
ID	CU-033
Nombre	Ver el detalle de una consulta de un usuario
Fuente	RF-035
Actores	Administrador
Descripción	El administrador consulta el detalle completo de una consulta de diagnóstico no eliminada de un usuario, incluidos la imagen, el resultado, la confianza y los metadatos asociados.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador selecciona una consulta de las consultas de un usuario. 2. El sistema verifica el rol de administración del actor. 3. El sistema recupera y muestra el detalle completo de la consulta. 4. El sistema registra la operación de auditoría.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no muestra el detalle.
Postcondiciones	El detalle de la consulta queda visible para el administrador y la operación queda registrada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-032. El acceso del administrador a la imagen de una consulta de otro usuario es la excepción al aislamiento definida en RF-005, queda acotado a la función de supervisión, se limita a las consultas no eliminadas (RF-016) y se registra conforme a RNF-006.

Tabla x - CU-033 — Ver el detalle de una consulta de un usuario

12.2.3.5. Módulo de capacidades transversales de la plataforma

Este módulo agrupa los casos de uso que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma. Incluye la consulta y la cancelación de los trabajos de la cola —que cubren los diagnósticos, los entrenamientos y las validaciones externas que el sistema ejecuta de forma asíncrona— y la personalización del tema visual de la interfaz. Su carácter transversal se aprecia en que estos casos de uso están disponibles para todo usuario autenticado, con independencia del núcleo funcional en el que trabaje: un profesional puede consultar el estado de sus diagnósticos mientras un investigador supervisa sus entrenamientos, y ambos pueden cancelar un trabajo que haya quedado pendiente en la cola. El módulo comprende los casos de uso CU-034 a CU-036, que se corresponden con los requisitos funcionales transversales (RF-036 a RF-038) y se asocian al objetivo de ejecución asíncrona de tareas (OBJ-010) y, en el caso del tema visual, al objetivo de usabilidad e internacionalización de la interfaz (OBJ-009).

Ninguno de los tres casos de uso presenta relaciones de inclusión o extensión con otros casos de uso de la plataforma: la consulta de la cola, la cancelación de trabajos y el cambio de tema se ejecutan de forma autónoma, sin delegar pasos obligatorios en otros casos ni ampliar ningún flujo base. La cancelación de un trabajo (CU-035), no obstante, se describe de forma coherente con la consulta de la cola (CU-034), porque es desde el panel de la cola donde el usuario identifica el trabajo que desea cancelar, aunque ambas operaciones se representan como casos de uso independientes.

CU-034 — Consultar el estado de la cola de trabajos

Campo	Contenido
ID	CU-034
Nombre	Consultar el estado de la cola de trabajos
Fuente	RF-036
Actores	Usuario autenticado
Descripción	El usuario consulta el panel de la cola de trabajos, que muestra el estado de los trabajos pendientes, en ejecución, completados o fallidos, y que se actualiza automáticamente al menos cada cinco segundos. El panel cubre los diagnósticos, los entrenamientos y las validaciones externas.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario accede al panel de la cola de trabajos. 2. El sistema muestra el estado de los trabajos del usuario. 3. El sistema actualiza el estado de los trabajos automáticamente al menos cada cinco segundos.
Flujo alternativo	2a. Si el usuario no tiene trabajos en la cola, el sistema muestra un panel con el estado vacío.
Postcondiciones	El usuario conoce el estado de sus trabajos en la cola.
Importancia	Media
Estado	Aprobado
Comentarios	El panel cubre todos los tipos de trabajo y muestra su estado.

Tabla x - CU-034 — Consultar el estado de la cola de trabajos

CU-035 — Cancelar un trabajo de la cola

Campo	Contenido
ID	CU-035
Nombre	Cancelar un trabajo de la cola
Fuente	RF-037
Actores	Usuario autenticado
Descripción	El usuario cancela un trabajo de la cola (un diagnóstico, un entrenamiento o una validación externa): si está pendiente, no llega a ejecutarse; si está en ejecución, el sistema intenta interrumpirlo cuando resulta técnicamente posible.
Precondiciones	El usuario tiene un trabajo en la cola.
Flujo normal	1. El usuario solicita la cancelación de un trabajo de la cola. 2. Si el trabajo está pendiente, el sistema lo cancela y evita su ejecución. 3. Si el trabajo está en ejecución y resulta técnicamente posible, el sistema lo interrumpe y lo marca como cancelado.
Flujo alternativo	2a. Si el trabajo está en ejecución y no puede interrumpirse de forma segura, el sistema informa de esa limitación y no modifica el trabajo. 2b. Si el trabajo ya ha finalizado, el sistema informa de que no puede cancelarse.
Postcondiciones	El trabajo queda cancelado (estuviera pendiente o en ejecución) o no se modifica si la interrupción no es posible.
Importancia	Media
Estado	Aprobado
Comentarios	La cancelación de un trabajo en ejecución puede dejar resultados parciales, que quedan marcados como cancelados y no como completados.

Tabla x - CU-035 — Cancelar un trabajo pendiente de la cola

CU-036 — Alternar el tema visual de la interfaz

Campo	Contenido
ID	CU-036
Nombre	Alternar el tema visual de la interfaz
Fuente	RF-038
Actores	Usuario autenticado
Descripción	El usuario alterna entre el tema claro y el tema oscuro de la interfaz, según su preferencia visual.
Precondiciones	El usuario tiene una sesión iniciada.
Flujo normal	1. El usuario activa el cambio de tema en la interfaz. 2. El sistema aplica el tema seleccionado en toda la interfaz de forma inmediata.
Flujo alternativo	N/A
Postcondiciones	La interfaz se muestra con el tema seleccionado.
Importancia	Baja
Estado	Aprobado
Comentarios	La preferencia se aplica de forma inmediata en toda la interfaz.

Tabla x - CU-036 — Alternar el tema visual de la interfaz

CU-037 — Generar el informe PDF del diagnóstico

Campo	Contenido
ID	CU-037
Nombre	Generar el informe PDF del diagnóstico
Fuente	RF-039
Actores	Usuario autenticado
Descripción	El usuario genera y descarga el informe PDF de una consulta de diagnóstico, que recoge la imagen original, la predicción, el nivel de confianza, el modelo empleado y los mapas de explicabilidad.
Precondiciones	La consulta pertenece al usuario y dispone de resultado.
Flujo normal	1. El usuario solicita el informe de la consulta desde su detalle. 2. El sistema comprueba que la consulta pertenece al usuario. 3. El sistema genera el documento PDF. 4. El sistema lo descarga al equipo del usuario.
Flujo alternativo	2a. Si la consulta no pertenece al usuario, el sistema deniega el acceso y no genera el informe. 3a. Si la generación del informe falla, el sistema informa del error y no descarga el documento.
Postcondiciones	La interfaz se muestra con el tema seleccionado.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso extiende CU-012: el informe se genera desde el detalle de la consulta. Cubre el informe individual del diagnóstico; el informe consolidado de la sesión se recoge en el CU-028.

Tabla x - CU-037 — Generar el informe PDF del diagnóstico

CU-038 — Gestionar las cuentas de usuario

Campo	Contenido
ID	CU-038
Nombre	Gestionar las cuentas de usuario
Fuente	RF-040
Actores	Administrador
Descripción	El administrador gestiona las cuentas de usuario de la plataforma: desactiva una cuenta, cambia el rol de un usuario o elimina una cuenta, con registro de la operación.
Precondiciones	El administrador tiene una sesión iniciada con rol de administración.
Flujo normal	1. El administrador selecciona la cuenta sobre la que desea actuar. 2. El sistema verifica el rol de administración del actor. 3. El administrador ejecuta la operación (desactivar, cambiar rol o eliminar). 4. El sistema aplica la operación y registra la auditoría.
Flujo alternativo	2a. Si el actor no tiene rol de administración, el sistema deniega el acceso y no modifica nada.
Postcondiciones	La cuenta queda gestionada y la operación registrada.
Importancia	Media
Estado	Aprobado
Comentarios	Este caso de uso completa la cobertura del objetivo de administración de la plataforma (OBJ-011), junto con las operaciones de consulta (CU-031 a CU-033).

Tabla x - CU-038 — Gestionar las cuentas de usuario

CU-039 — Comprobar la limitación de entrenamientos

Campo	Contenido
ID	CU-039
Nombre	Comprobar la limitación de entrenamientos
Fuente	RF-041
Actores	Usuario autenticado
Descripción	Cuando se supera el límite de entrenamientos simultáneos o encolados, el sistema mantiene el trabajo en espera y el usuario comprueba esa situación desde el panel de la cola.
Precondiciones	El usuario ha lanzado o va a lanzar un experimento de entrenamiento.
Flujo normal	1. El usuario lanza un experimento. 2. El sistema comprueba el límite de entrenamientos simultáneos y encolados. 3. Si se supera el límite, el sistema encola el trabajo en espera. 4. El usuario consulta el estado del trabajo en el panel de la cola.
Flujo alternativo	3a. Si no se supera el límite, el entrenamiento se ejecuta de forma inmediata.
Postcondiciones	El trabajo queda encolado o en ejecución conforme al límite de recursos.
Importancia	Media
Estado	Aprobado
Comentarios	La limitación es global a la plataforma y complementa la disponibilidad del servicio (RNF-027). Este caso de uso se relaciona con la consulta de la cola (CU-034).

Tabla x - CU-039 — Comprobar la limitación de entrenamientos

12.2.3.6. Matriz de trazabilidad requisitos – casos de uso

La matriz de trazabilidad que se presenta a continuación relaciona cada requisito funcional con los casos de uso en los que se materializa, de modo que pueda verificarse en ambas direcciones que todos los requisitos quedan cubiertos por al menos un caso de uso y que todos los casos de uso están respaldados por al menos un requisito. Esta verificación garantiza la coherencia entre los dos niveles de especificación del capítulo: los requisitos funcionales declaran qué debe hacer el sistema y los casos de uso concretan cómo se realiza cada capacidad, por lo que no debe existir ningún requisito sin caso de uso que lo materialice ni ningún caso de uso sin requisito que lo justifique.

Como se señaló al especificar los casos de uso, la mayoría de los requisitos funcionales se materializan en un caso de uso concreto del mismo módulo, lo que se refleja en las matrices de forma directa: cada caso de uso marca con una X la celda correspondiente a su requisito fuente. No obstante, dos requisitos de carácter transversal no dan lugar a un caso de uso propio: el aislamiento de datos entre usuarios (RF-005) y los roles y el control de acceso (RF-006). Estos requisitos condicionan la ejecución del resto de las interacciones y se reflejan en las matrices marcando las celdas de los casos de uso a los que afectan —las operaciones sobre consultas de diagnóstico, sesiones de entrenamiento y datos de administración—, sin constituir por ello interacciones independientes. Para facilitar la lectura, la matriz se divide en cinco tablas, una por módulo funcional, de modo que cada tabla relaciona los requisitos del módulo con los casos de uso que los materializan.

Matriz de trazabilidad del módulo de gestión del acceso y de la cuenta

En este módulo, cada requisito se materializa en un único caso de uso: el registro (RF-001) se concreta en la creación de la cuenta (CU-001), el acceso (RF-002) en el inicio de sesión (CU-002), el cierre de sesión (RF-003) en la finalización de la sesión (CU-003) y el cambio de idioma (RF-004) en la adaptación de la interfaz (CU-004). Los requisitos transversales de aislamiento y de roles afectan a la totalidad de las interacciones de la plataforma, por lo que también se reflejan en este módulo.

RF	Denominación	CU-001	CU-002	CU-003	CU-004
RF-001	Registro de usuario	X			
RF-002	Inicio de sesión		X		
RF-003	Cierre de sesión			X	
RF-004	Cambio de idioma de la interfaz				X
RF-005	Aislamiento de datos entre usuarios	X		X	
RF-006	Roles y control de acceso		X	X	

Tabla x - Matriz de trazabilidad RF-CU del módulo de gestión del acceso y de la cuenta

Matriz de trazabilidad del módulo de interfaz de diagnóstico asistido

En este módulo, cada caso de uso del bloque de consulta de diagnóstico se corresponde con su requisito fuente: el acceso al panel (CU-005) con RF-007, la subida de la radiografía (CU-006) con RF-008, la selección de la arquitectura (CU-007) con RF-009, la solicitud del diagnóstico (CU-008) con RF-010, la visualización del resultado (CU-009) con RF-011 y la visualización de los mapas de explicabilidad (CU-010) con RF-012. Los casos de uso del bloque de historial se corresponden con los requisitos de gestión del historial: la consulta del listado (CU-011) con RF-013, el detalle (CU-012) con RF-014, el renombrado (CU-013) con RF-015 y la eliminación (CU-014) con RF-016. El aislamiento de datos (RF-005) se aplica a todas las operaciones sobre las consultas del usuario.

RF	Denominación	CU-005	CU-006	CU-007	CU-008	CU-009	CU-010	CU-011	CU-012	CU-013	CU-014	CU-037
RF-007	Acceso al panel de diagnóstico	X										
RF-008	Subida de una radiografía de tórax		X									
RF-009	Selección de la arquitectura para el diagnóstico			X								
RF-010	Solicitud de un diagnóstico				X							
RF-011	Visualización del resultado del diagnóstico					X						
RF-012	Visualización de los mapas de explicabilidad						X					
RF-013	Consultar el historial de consultas							X				
RF-014	Visualización del detalle de una consulta del historial								X			
RF-015	Renombrar una consulta del historial									X		
RF-016	Eliminar una consulta del historial										X	
RF-039	Generación del informe PDF del diagnóstico											X
RF-005	Aislamiento de datos entre usuarios					X	X	X	X	X	X	X

Tabla x - Matriz de trazabilidad RF-CU del módulo de interfaz de diagnóstico asistido

Matriz de trazabilidad del módulo de laboratorio de experimentación MLOps

En este módulo, los dieciséis casos de uso se corresponden de forma biunívoca con sus requisitos fuente, desde el acceso al laboratorio (CU-015 con RF-017) hasta la eliminación de la sesión (CU-030 con RF-032). El aislamiento de datos (RF-005) se aplica a las operaciones sobre las sesiones de entrenamiento del usuario.

RF	Denominación	CU-015	CU-016	CU-017	CU-018	CU-019	CU-020	CU-021	CU-022
RF-017	Acceso al laboratorio de entrenamiento	X							
RF-018	Conversación con el asistente para configurar un experimento		X						
RF-019	Selección de la carpeta del dataset			X					
RF-020	Lanzamiento de un experimento de entrenamiento				X				
RF-021	Consulta de las sesiones de entrenamiento					X			
RF-022	Consulta de los resultados de un modelo de la sesión						X		
RF-023	Visualización de los mapas de explicabilidad de un modelo							X	
RF-024	Consulta del ranking de modelos de la sesión								X

Tabla x - Matriz de trazabilidad RF-CU del módulo de laboratorio de experimentación MLOps (I)

RF	Denominación	CU-023	CU-024	CU-025	CU-026	CU-027	CU-028	CU-029	CU-030	CU-039
RF-025	Consulta de la comparativa estadística de la sesión	X								
RF-026	Solicitud del recálculo de la comparativa estadística		X							
RF-027	Ejecución del análisis de explicabilidad de un modelo			X						
RF-028	Solicitud de la validación externa de la sesión				X					
RF-029	Consulta de los resultados de la validación externa					X				
RF-030	Generación del informe PDF de la sesión						X			
RF-031	Renombrar una sesión de entrenamiento							X		
RF-032	Eliminar una sesión de entrenamiento								X	
RF-041	Limitación de entrenamientos simultáneos y de la cola									X
RF-005	Aislamiento de datos entre usuarios	X	X	X	X	X	X	X	X	

Tabla x - Matriz de trazabilidad RF-CU del módulo de laboratorio de experimentación MLOps (II)

Matriz de trazabilidad del módulo de supervisión y administración de la plataforma

En este módulo, los tres casos de uso se corresponden con sus requisitos fuente y se refleja además la incidencia del requisito de roles y control de acceso (RF-006), que restringe estas operaciones al administrador. El acceso del administrador a los datos de los usuarios supervisados es la excepción al aislamiento de datos (RF-005) definida en este módulo: no contradice el requisito porque queda limitada al rol y a la función de supervisión, y porque el administrador solo puede examinar los datos a los que su rol le da acceso.

RF	Denominación	CU-031	CU-032	CU-033	CU-038
RF-033	Consulta del listado de usuarios	X			
RF-034	Consulta de las consultas de un usuario		X		
RF-035	Visualización del detalle de una consulta de un usuario			X	
RF-040	Gestión de cuentas de usuario por el administrador				X
RF-006	Roles y control de acceso	X	X	X	X

Tabla x - Matriz de trazabilidad RF-CU del módulo de supervisión y administración de la plataforma

Matriz de trazabilidad del módulo de capacidades transversales de la plataforma

En este módulo, los tres casos de uso se corresponden con sus requisitos fuente: la consulta de la cola (CU-034) con RF-036, la cancelación de trabajos (CU-035) con RF-037 y el cambio del tema visual (CU-036) con RF-038.

RF	Denominación	CU-034	CU-035	CU-036
RF-036	Consulta del estado de la cola de trabajos	X		
RF-037	Cancelación de un trabajo de la cola		X	
RF-038	Cambio del tema visual de la interfaz			X

Tabla x - Matriz de trazabilidad RF-CU del módulo de capacidades transversales de la plataforma

El conjunto de las cinco tablas permite verificar la cobertura completa entre los requisitos funcionales y los casos de uso: cada requisito funcional queda materializado en al menos un caso de uso y cada caso de uso está respaldado por el requisito que lo origina. Los únicos requisitos que no poseen un caso de uso propio son los de carácter transversal —el aislamiento de datos (RF-005) y los roles y el control de acceso (RF-006)—, cuya cobertura se refleja en las matrices mediante las celdas marcadas sobre los casos de uso a los que condicionan. Los requisitos añadidos para completar la cobertura de objetivos (RF-039 informe del diagnóstico, RF-040 gestión de cuentas y RF-041 limitación de entrenamientos) se materializan en los casos de uso CU-037, CU-038 y CU-039, incorporados a las matrices de sus módulos. De este modo, la trazabilidad entre ambos niveles de especificación queda garantizada en todo el capítulo, de forma análoga a la matriz de trazabilidad entre requisitos y objetivos del sistema presentada en la sección 12.1.

13. Análisis de subsistemas

Una vez que el análisis ha fijado el ámbito del sistema (capítulo 11) y ha especificado en detalle sus requisitos y sus casos de uso (capítulo 12), el siguiente paso consiste en estructurar la plataforma en unidades de análisis que permitan abordar el diseño con orden. La descomposición del sistema en subsistemas de análisis persigue identificar las áreas funcionales que componen la plataforma y establecer de forma clara las responsabilidades de cada una antes de entrar en el diseño detallado. Cada subsistema agrupa un conjunto cohesionado de casos de uso y de requisitos funcionales que comparten una misma naturaleza, de modo que los límites entre subsistemas minimicen el acoplamiento entre áreas y maximicen la cohesión interna de cada una, dos propiedades que facilitan tanto el análisis posterior como la evolución del sistema (Larman, 2004).

Conviene precisar qué son y qué no son estos subsistemas. Los subsistemas identificados en este capítulo no son componentes de implementación ni deciden ninguna tecnología concreta: son unidades lógicas de análisis que servirán de base para la arquitectura del sistema y para el análisis de clases. La elección de las tecnologías que materializarán cada subsistema es una decisión que compete a la fase de diseño, que se aborda en capítulos posteriores de esta memoria. La descomposición que aquí se presenta se apoya directamente en los módulos funcionales declarados en la especificación de requisitos del capítulo 12, de forma que exista una correspondencia clara entre los subsistemas de análisis y las áreas funcionales ya definidas (Jacobson, Booch y Rumbaugh, 1999).

Cada subsistema se describe mediante una ficha normalizada en la que se recogen su identificador, su nombre, su descripción, los casos de uso y los requisitos funcionales que agrupa y los requisitos no funcionales que le resultan de aplicación. Esta forma de presentación permite verificar de un vistazo qué responsabilidades asume cada subsistema y qué exigencias del capítulo 12 sostienen su comportamiento. El sistema se descompone en los seis subsistemas de análisis siguientes: el subsistema de acceso y gestión de cuentas (SS-001), el subsistema de diagnóstico asistido (SS-002), el subsistema de gestión del historial de consultas (SS-003), el subsistema de laboratorio de experimentación MLOps (SS-004), el subsistema de supervisión y administración (SS-005) y el subsistema de capacidades transversales (SS-006). A continuación se describen uno a uno.

13.1. SS-001 — Subsistema de acceso y gestión de cuentas

Este subsistema es responsable de controlar el acceso a la plataforma y de gestionar la identidad de los usuarios registrados. Constituye la puerta de entrada al sistema, pues ninguna funcionalidad privada resulta accesible sin haber completado previamente el proceso de autenticación. Además del registro de nuevas cuentas y del inicio y cierre de sesión, este subsistema permite al usuario adaptar la interfaz a su idioma, una capacidad transversal que se gestiona desde la cuenta. Desde el punto de vista técnico, este subsistema garantiza que las contraseñas se almacenen de forma segura mediante técnicas de cifrado irreversible, que las sesiones se gestionen con las salvaguardas necesarias y que el acceso a las áreas privadas quede restringido a usuarios autenticados. Asimismo, establece la base lógica de identidad sobre la que se sustenta el aislamiento de datos entre usuarios (RF-005) y el control de roles (RF-006), dos requisitos de carácter transversal que condicionan el comportamiento del resto de los subsistemas.

Campo	Contenido
ID	SS-001
Nombre	Subsistema de acceso y gestión de cuentas
Descripción	Este subsistema garantiza el registro de nuevos usuarios, el inicio y el cierre de sesión y la adaptación del idioma de la interfaz. Asegura que las contraseñas se almacenen mediante técnicas de cifrado irreversible, que la sesión se gestione de forma segura y que las áreas privadas queden restringidas a usuarios autenticados. Establece, además, la base lógica de identidad necesaria para el aislamiento transversal de datos entre usuarios y para el control de roles.
Casos de uso relacionados	CU-001, CU-002, CU-003, CU-004
Requisitos funcionales relacionados	RF-001, RF-002, RF-003, RF-004, RF-005, RF-006
Requisitos no funcionales relacionados	RNF-001, RNF-002, RNF-003, RNF-004, RNF-005, RNF-007, RNF-010, RNF-011, RNF-024

Tabla x - SS-001: Subsistema de Acceso y Gestión de Cuentas

13.2. SS-002 — Subsistema de diagnóstico asistido

Este subsistema agrupa la funcionalidad clínica de la plataforma: el flujo completo que permite al profesional sanitario obtener un diagnóstico asistido sobre una radiografía de tórax. Comprende el acceso al panel de diagnóstico, la subida de la imagen radiológica, la selección de la arquitectura de red neuronal con la que se desea realizar la inferencia, la solicitud del diagnóstico, la visualización del resultado con su nivel de confianza y la visualización de los mapas de explicabilidad que justifican la decisión del modelo. Por manejar datos sanitarios, este subsistema queda sometido a los requisitos de confidencialidad y protección de datos del Reglamento General de Protección de Datos, a la anonimización de los conjuntos de datos y a la exclusión de cualquier dato personal identificable. Asimismo, la solicitud del diagnóstico se ejecuta de forma asíncrona, de modo que la interfaz no se bloquea durante la inferencia y el tiempo de respuesta se mantiene dentro de los límites exigidos.

Campo	Contenido
ID	SS-002
Nombre	Subsistema de Diagnóstico Asistido
Descripción	Este subsistema cubre el flujo clínico de la plataforma: acceso al panel de diagnóstico, subida de una radiografía de tórax, selección de la arquitectura del modelo, solicitud asíncrona del diagnóstico, visualización del resultado con su confianza, visualización de los mapas de explicabilidad y generación del informe PDF de la consulta. Su comportamiento queda sujeto a los requisitos de confidencialidad y protección de datos, así como a los de rendimiento de la inferencia y de ejecución sin bloqueo de la interfaz.
Casos de uso relacionados	CU-005, CU-006, CU-007, CU-008, CU-009, CU-010, CU-037
Requisitos funcionales relacionados	RF-007, RF-008, RF-009, RF-010, RF-011, RF-012, RF-039
Requisitos no funcionales relacionados	RNF-008, RNF-012, RNF-013, RNF-014, RNF-015, RNF-019, RNF-020

Tabla x - SS-002: Subsistema de Diagnóstico Asistido

13.3. SS-003 — Subsistema de gestión del historial de consultas

Este subsistema es el encargado de conservar y gestionar el historial de consultas de diagnóstico de cada usuario autenticado. Permite consultar el listado de las consultas realizadas, visualizar el detalle completo de una consulta anterior —incluidos la imagen, el resultado, la confianza y los metadatos asociados—, renombrar una consulta para organizarla según la preferencia del usuario y eliminar aquellas consultas que ya no se deseen conservar. Al operar sobre registros de diagnóstico con datos sanitarios, este subsistema comparte con el de diagnóstico asistido los requisitos de confidencialidad y protección de datos, y añade los requisitos de integridad y durabilidad de la persistencia, que garantizan que el historial se conserva de forma fiable a lo largo del tiempo y que cada consulta queda correctamente asociada a su propietario, en línea con el aislamiento de datos entre usuarios (RF-005).

Campo	Contenido
ID	SS-003
Nombre	Subsistema de gestión del historial de consultas
Descripción	Este subsistema gestiona el historial de consultas de diagnóstico del usuario autenticado: la consulta del listado de consultas, la visualización del detalle de una consulta anterior, el renombrado y la eliminación de consultas. Preserva la confidencialidad de los datos sanitarios y la integridad y durabilidad de la persistencia del historial.
Casos de uso relacionados	CU-011, CU-012, CU-013, CU-014
Requisitos funcionales relacionados	RF-013, RF-014, RF-015, RF-016
Requisitos no funcionales relacionados	RNF-012, RNF-013, RNF-014, RNF-016, RNF-031

Tabla x - SS-003: Subsistema de Gestión del Historial de Consultas

13.4. SS-004 — Subsistema de laboratorio de experimentación MLOps

Este subsistema constituye el segundo núcleo funcional de la plataforma y agrupa la experimentación MLOps: el asistente conversacional que configura los experimentos, la selección de la carpeta del dataset, el lanzamiento asíncrono de los entrenamientos, la gestión de las sesiones de experimentación y la consulta de sus resultados. Dentro de una sesión, este subsistema cubre la consulta de los resultados de cada modelo, la visualización de sus mapas de explicabilidad, el ranking de modelos, la comparativa estadística con su recálculo, la validación externa sobre datos independientes y la generación del informe PDF. Su naturaleza asíncrona lo vincula directamente al mecanismo de cola de trabajos, de modo que los entrenamientos de larga duración se ejecutan sin bloquear la interfaz y sus resultados quedan disponibles cuando finalizan. Por tratarse del área donde se entrenan y comparan los modelos, este subsistema incorpora los requisitos de reproducibilidad de los experimentos y del entorno de ejecución, junto con los de concurrencia, robustez y disponibilidad del servicio durante las tareas de larga duración.

Campo	Contenido
ID	SS-004
Nombre	Subsistema de laboratorio de experimentación MLOps
Descripción	Este subsistema cubre la experimentación MLOps de la plataforma: la configuración de experimentos mediante el asistente conversacional, la selección del dataset, el lanzamiento asíncrono de los entrenamientos con la limitación de entrenamientos simultáneos, la gestión de las sesiones y la consulta de resultados, mapas de explicabilidad, ranking, comparativa estadística, validación externa e informes PDF. Depende de la ejecución asíncrona de tareas y aplica los requisitos de reproducibilidad, concurrencia y robustez.
Casos de uso relacionados	CU-015, CU-016, CU-017, CU-018, CU-019, CU-020, CU-021, CU-022, CU-023, CU-024, CU-025, CU-026, CU-027, CU-028, CU-029, CU-030, CU-039
Requisitos funcionales relacionados	RF-017, RF-018, RF-019, RF-020, RF-021, RF-022, RF-023, RF-024, RF-025, RF-026, RF-027, RF-028, RF-029, RF-030, RF-031, RF-032, RF-041
Requisitos no funcionales relacionados	RNF-012, RNF-020, RNF-021, RNF-022, RNF-027, RNF-028, RNF-029, RNF-030, RNF-033, RNF-034

Tabla x - SS-004: Subsistema de Laboratorio de Experimentación MLOps

13.5. SS-005 — Subsistema de supervisión y administración de la plataforma

Este subsistema agrupa las operaciones de gobierno de la plataforma, restringidas al rol de administrador. Permite consultar el listado de usuarios registrados, examinar el historial de consultas de diagnóstico de un usuario concreto y visualizar el detalle completo de una consulta, incluidos la imagen, el resultado, la confianza y los metadatos asociados. Su propósito es proporcionar al administrador una visión global del uso que se hace de la plataforma y los medios para auditar cualquier incidencia ante la que deba intervenir. Por supervisar datos personales y de diagnóstico de terceros, este subsistema queda sometido a los requisitos de protección de datos y al requisito de auditoría de la actividad administrativa, que garantiza la trazabilidad de las acciones de supervisión. El control de roles (RF-006) delimita estrictamente quién puede invocar estas operaciones, de modo que solo el administrador accede a ellas.

Campo	Contenido
ID	SS-005
Nombre	Subsistema de supervisión y administración de la plataforma
Descripción	Este subsistema agrupa las operaciones de administración restringidas al rol de administrador: la consulta del listado de usuarios, la consulta de las consultas de diagnóstico de un usuario, la visualización del detalle de una consulta y la gestión de las cuentas de usuario (desactivar, cambiar rol o eliminar). Aplica los requisitos de protección de datos, de auditoría de la actividad administrativa y de registro de eventos de seguridad.
Casos de uso relacionados	CU-031, CU-032, CU-033, CU-038
Requisitos funcionales relacionados	RF-033, RF-034, RF-035, RF-040
Requisitos no funcionales relacionados	RNF-006, RNF-009, RNF-012

Tabla x - SS-005: Subsistema de Supervisión y Administración de la Plataforma

13.6. SS-006 — Subsistema de capacidades transversales de la plataforma

Este subsistema reúne las capacidades que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma. Por un lado, gestiona la cola de trabajos asíncronos, que cubre los diagnósticos, los entrenamientos y las validaciones externas: permite consultar el estado de cada trabajo —pendiente, en ejecución, completado o fallido— y cancelar un trabajo de la cola, esté pendiente o, cuando resulte técnicamente posible, en ejecución. Al ser el mecanismo compartido de ejecución asíncrona, este subsistema integra los requisitos de robustez y disponibilidad que garantizan que las tareas de larga duración no degradan el servicio, que los errores de un trabajo se gestionan de forma aislada y que los trabajos se recuperan tras un reinicio del servidor. Por otro lado, este subsistema cubre las capacidades de interfaz de carácter transversal, como la personalización del tema visual, y aplica los requisitos de usabilidad, multilingüismo y accesibilidad de la interfaz.

Campo	Contenido
ID	SS-006
Nombre	Subsistema de capacidades transversales de la plataforma
Descripción	Este subsistema agrupa las capacidades transversales de la plataforma: la consulta del estado de la cola de trabajos asíncronos, la cancelación de los trabajos de la cola y la personalización del tema visual de la interfaz. Integra los requisitos de robustez y disponibilidad del servicio durante las tareas de larga duración, así como los de usabilidad, multilingüismo y accesibilidad de la interfaz.
Casos de uso relacionados	CU-034, CU-035, CU-036
Requisitos funcionales relacionados	RF-036, RF-037, RF-038
Requisitos no funcionales relacionados	RNF-023, RNF-024, RNF-025, RNF-026, RNF-027, RNF-028, RNF-029, RNF-030

Tabla x - SS-006: Subsistema de Capacidades Transversales de la Plataforma

13.7. *Síntesis de la descomposición en subsistemas*

La descomposición presentada en este capítulo recoge, en los seis subsistemas descritos, la totalidad de las capacidades de la plataforma declaradas en el capítulo 12: los treinta y nueve casos de uso y los cuarenta y un requisitos funcionales quedan distribuidos entre los subsistemas, de modo que cada caso de uso pertenece exactamente a un subsistema y cada requisito funcional queda asignado al subsistema que lo materializa. Los dos requisitos de carácter transversal —el aislamiento de datos entre usuarios (RF-005) y el control de roles (RF-006)— se declaran formalmente en el subsistema de acceso y gestión de cuentas, que establece la identidad y la sesión sobre las que ambos se aplican, pero condicionan el comportamiento de todos los demás subsistemas, como se ha señalado en las fichas.

Conviene destacar las dos dependencias principales que se desprenden de esta descomposición. En primer lugar, los subsistemas de diagnóstico asistido (SS-002), de laboratorio de experimentación (SS-004) y de supervisión y administración (SS-005) presuponen que el usuario se ha autenticado a través del subsistema de acceso (SS-001) y que, por tanto, existe una identidad sobre la que aplicar el aislamiento de datos y el control de roles. En segundo lugar, los subsistemas SS-002 y SS-004 delegan la ejecución de sus tareas de larga duración en el mecanismo de cola de trabajos del subsistema de capacidades transversales (SS-006), que actúa como servicio común de ejecución asíncrona.

Estos subsistemas de análisis constituyen la unidad de trabajo sobre la que se apoyarán las etapas siguientes de la memoria: el análisis de clases, que identificará las entidades del dominio y sus responsabilidades, y el diseño del sistema, que tomará las decisiones tecnológicas que materializarán cada subsistema. La correspondencia entre los subsistemas de análisis, los módulos funcionales de requisitos y los módulos de casos de uso garantiza que ninguna capacidad declarada en la especificación de requisitos quede sin representación en el análisis, de forma coherente con el enfoque dirigido por requisitos que guía este proyecto (Larman, 2004).

14. Modelo de dominio y entidades del sistema

El modelado conceptual del dominio es la etapa del análisis del sistema en la que se detectan y representan las entidades del problema, sus atributos, sus responsabilidades y los vínculos que se establecen entre ellas. El propósito de esta sección no es definir la implementación específica del sistema, sino construir un modelo conceptual del dominio que sirva como base para las decisiones de diseño que se adoptarán en capítulos posteriores. El modelo de clases constituye, junto con los casos de uso del capítulo 12 y los subsistemas de análisis del capítulo 13, uno de los tres pilares sobre los que se asienta el diseño de vitalXAI: los casos de uso describen el comportamiento observable del sistema, los subsistemas agrupan sus responsabilidades y el modelo de clases identifica las entidades sobre las que ese comportamiento se apoya (Larman, 2004).

El modelado del dominio se organiza en dos secciones. La primera, la estructura estática de las entidades, presenta el modelo de negocio mediante un diagrama de clases que recoge las entidades del sistema, sus atributos y sus relaciones. La segunda sección contiene los diagramas de secuencia del sistema, que describen el comportamiento dinámico de la plataforma ante los estímulos del actor principal para cada caso de uso. Mientras que el diagrama de clases responde a la pregunta de qué entidades gestiona el sistema y cómo se relacionan entre sí, los diagramas de secuencia responden a la pregunta de cómo colaboran esas entidades cuando el usuario lleva a cabo una acción concreta. Ambas visiones, la estática y la dinámica, son complementarias y necesarias para comprender por completo el comportamiento esperado de la plataforma antes de abordar su diseño.

14.1. Estructura estática de las entidades del sistema

El diagrama de clases del sistema recoge las ocho clases de negocio que se han identificado durante el análisis del dominio de vitalXAI: Usuario, ConsultaDiagnostico, MapaXAI, SesionExperimentacion, ModeloEntrenado, ResultadoModelo, ValidacionExterna y TrabajoCola. Estas clases representan las entidades centrales que la plataforma debe persistir y gestionar a lo largo de su ciclo de vida, y sus relaciones reflejan la estructura conceptual del problema que la plataforma resuelve: la asistencia al diagnóstico de neumonía a partir de radiografías de tórax y la experimentación MLOps para entrenar y evaluar los modelos que sustentan ese diagnóstico. La elección de estas ocho clases responde a los módulos funcionales y a los casos de uso del capítulo 12, de modo que cada entidad tiene una correspondencia directa con las capacidades declaradas en la especificación de requisitos.

A nivel metodológico se han adoptado dos decisiones de modelado que conviene señalar. En primer lugar, los parámetros que definen un experimento se reparten entre dos entidades. Las arquitecturas a entrenar y los hiperparámetros —número de épocas, tamaño de lote y tasa de aprendizaje—, que el asistente conversacional del laboratorio interpreta a partir de la conversación con el usuario (CU-016), se consolidan como atributos de la clase ModeloEntrenado, descartando la necesidad de una clase separada para la configuración del experimento: la configuración producida por el asistente se materializa directamente en la entidad que representa al modelo entrenado. La ruta del dataset, en cambio, es un atributo de la clase SesionExperimentacion, porque cada sesión agrupa los modelos entrenados sobre un mismo dataset, y se selecciona de forma validada mediante el caso de uso CU-017, no mediante la conversación con el asistente. En segundo lugar, la clase Usuario constituye la entidad raíz del sistema: todas las demás entidades pertenecen a un usuario concreto y no son accesibles desde otras cuentas, de manera que se garantice el aislamiento de datos declarado en el requisito RF-005. Un usuario puede realizar múltiples consultas de diagnóstico y poseer múltiples sesiones de experimentación, y ninguna de estas entidades puede existir sin estar asociada a su propietario.

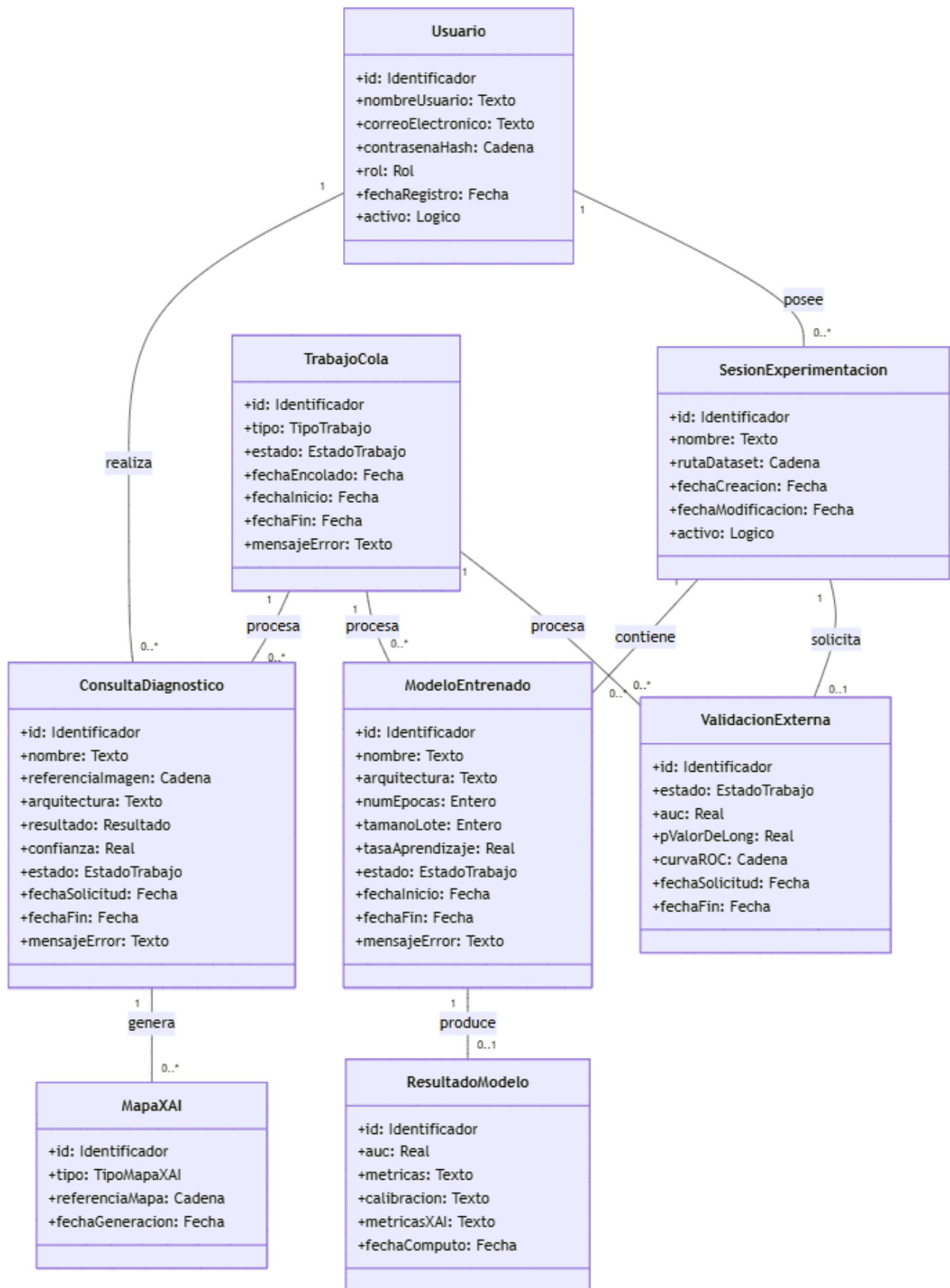


Ilustración x - Diagrama de clases del sistema

La clase Usuario almacena la información de identidad de cada cuenta: el nombre de usuario, el correo electrónico, el hash de la contraseña (nunca la contraseña en claro, conforme al requisito RNF-001), el rol, la fecha de registro y el estado activo de la cuenta. Un usuario puede realizar múltiples consultas de diagnóstico, representadas por la clase ConsultaDiagnostico, que agrupa los datos de una consulta concreta: la referencia a la imagen radiológica, la referencia al informe PDF de la consulta, la arquitectura seleccionada para el diagnóstico, el resultado (PNEUMONIA o NORMAL), la confianza asociada, el estado del trabajo y las fechas de solicitud y finalización. Cada consulta genera, a su vez, los mapas de explicabilidad que la justifican, representados por la clase MapaXAI, que distingue el tipo de mapa —mapas de saliencia, SmoothGrad, Grad-CAM para arquitecturas convolucionales y mapas de atención para arquitecturas Transformer— mediante el atributo tipo.

Por otra parte, un usuario puede poseer múltiples sesiones de experimentación, representadas por la clase SesionExperimentacion, que actúa como contenedor organizativo de los experimentos del laboratorio: cada sesión agrupa el conjunto de modelos entrenados sobre un mismo dataset. La clase ModeloEntrenado representa cada uno de los modelos entrenados dentro de una sesión, con la arquitectura y los hiperparámetros utilizados —épocas, tamaño de lote y tasa de aprendizaje—, su estado y sus fechas de ejecución. Cuando un modelo completa su entrenamiento, produce un resultado, representado por la clase ResultadoModelo, que almacena las métricas de la validación cruzada, la calibración y las métricas de explicabilidad. La clase ValidacionExterna representa la validación de los modelos de una sesión sobre una cohorte externa, con su AUC y el p-valor del test de DeLong. Finalmente, la clase TrabajoCola representa los trabajos asíncronos de la cola —diagnósticos, entrenamientos y validaciones externas—, que se procesan sin bloquear la interfaz conforme al requisito RF-036.

El modelo de relaciones garantiza el aislamiento de datos declarado en el requisito RF-005: todas las entidades de negocio parten de la clase Usuario, de modo que cada consulta de diagnóstico, cada sesión de experimentación y, a través de ellas, cada modelo y cada resultado, quedan asociados a un propietario concreto. Las relaciones de composición entre SesionExperimentacion y ModeloEntrenado, y entre ModeloEntrenado y ResultadoModelo, reflejan el ciclo de vida de los experimentos: un modelo no puede existir fuera de su sesión y un resultado no puede existir sin el modelo que lo produce. La relación entre TrabajoCola y las entidades procesadas —ConsultaDiagnostico, ModeloEntrenado y ValidacionExterna— representa la ejecución asíncrona de los tres tipos de trabajo, que comparten el mismo mecanismo de cola descrito en los casos de uso CU-034 y CU-035.

El diagrama de clases presentado en la figura 8 constituye el modelo estático de dominio de vitalXAI y servirá como base para el análisis dinámico del sistema: los diagramas de secuencia de la sección 14.2 muestran cómo colaboran estas entidades cuando el usuario lleva a cabo cada uno de los casos de uso especificados en el capítulo 12. De este modo, el modelado del dominio cierra el ciclo de la etapa de análisis iniciada en el capítulo 10, proporcionando una visión completa y coherente de las entidades que la plataforma debe gestionar, de sus relaciones y de su comportamiento.

14.2. Comportamiento dinámico del sistema y DSS

La visión estática del dominio descrita en la sección 14.1 es necesaria pero no suficiente para comprender el sistema: las entidades se relacionan entre sí, pero la forma en que colaboran cuando el usuario interactúa con la plataforma no queda recogida en el diagrama de clases. Las secuencias de interacción del sistema cubren precisamente esta carencia, representando de forma visual y ordenada cómo se comporta la plataforma ante cada una de las acciones que el actor principal lleva a cabo sobre ella. Cada secuencia muestra, paso a paso y en orden cronológico, qué hace el usuario, qué solicita al sistema y qué respuesta recibe, de modo que el comportamiento esperado de la plataforma ante cada caso de uso queda descrito de forma inequívoca y sin necesidad de anticipar decisiones técnicas de implementación (Larman, 2004).

En cada secuencia de interacción intervienen dos participantes: el actor que inicia la acción, representado en el margen izquierdo del diagrama, y el sistema en su conjunto, representado en el margen derecho. Los mensajes que el actor envía al sistema corresponden a las acciones que realiza sobre la interfaz gráfica, como rellenar un formulario o pulsar un botón, mientras que el sistema procesa la información recibida, realiza las operaciones necesarias y devuelve una respuesta al actor, que puede ser una confirmación, un listado de datos, un fichero descargable o un mensaje de error. Las operaciones que el sistema ejecuta de forma interna, como la validación de datos o el acceso a la información persistida, se representan igualmente en el diagrama mediante mensajes autorecursivos, lo que ofrece una imagen más completa de lo que ocurre en cada paso sin entrar en los detalles de implementación.

Las secuencias se presentan organizadas por subsistemas, siguiendo la estructura definida en el capítulo 13, de forma que resulte sencillo localizar el comportamiento de la plataforma para cada funcionalidad. Para mantener la claridad de la presentación, cada diagrama recoge únicamente el flujo habitual de la acción, es decir, el camino que sigue la interacción cuando todo funciona de forma correcta. Los comportamientos alternativos, como los mensajes de error o la cancelación de una operación, se describen en la especificación detallada de cada caso de uso del capítulo 12 (sección 12.2.3), donde se documentan los flujos normal y alternativo de cada caso.

14.2.1. SS-001 – Subsistema de Acceso y Gestión de Cuentas

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de acceso y gestión de cuentas, descrito en el capítulo 13. Todos ellos tienen como actor al visitante o al usuario autenticado según corresponda, e interactúan con el sistema para gestionar la identidad y el acceso a la plataforma: el registro de una nueva cuenta (CU-001), el inicio de sesión (CU-002), el cierre de sesión (CU-003) y el cambio del idioma de la interfaz (CU-004). Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-001: Registrarse

El registro es la puerta de entrada de la plataforma y la primera interacción que un visitante mantiene con ella. El visitante accede al formulario de registro desde la página de inicio de sesión, introduce sus datos —nombre de usuario, nombre, apellidos, correo electrónico y contraseña— y los envía al sistema. Este valida el formato de los datos recibidos y la fortaleza de la contraseña, comprueba que no exista ninguna cuenta previa con el mismo nombre de usuario o correo electrónico y, si todo es correcto, cifra la contraseña mediante un algoritmo de hash robusto e irreversible y crea el registro del nuevo usuario. Finalmente, el sistema confirma el alta y redirige al visitante a la página de inicio de sesión.

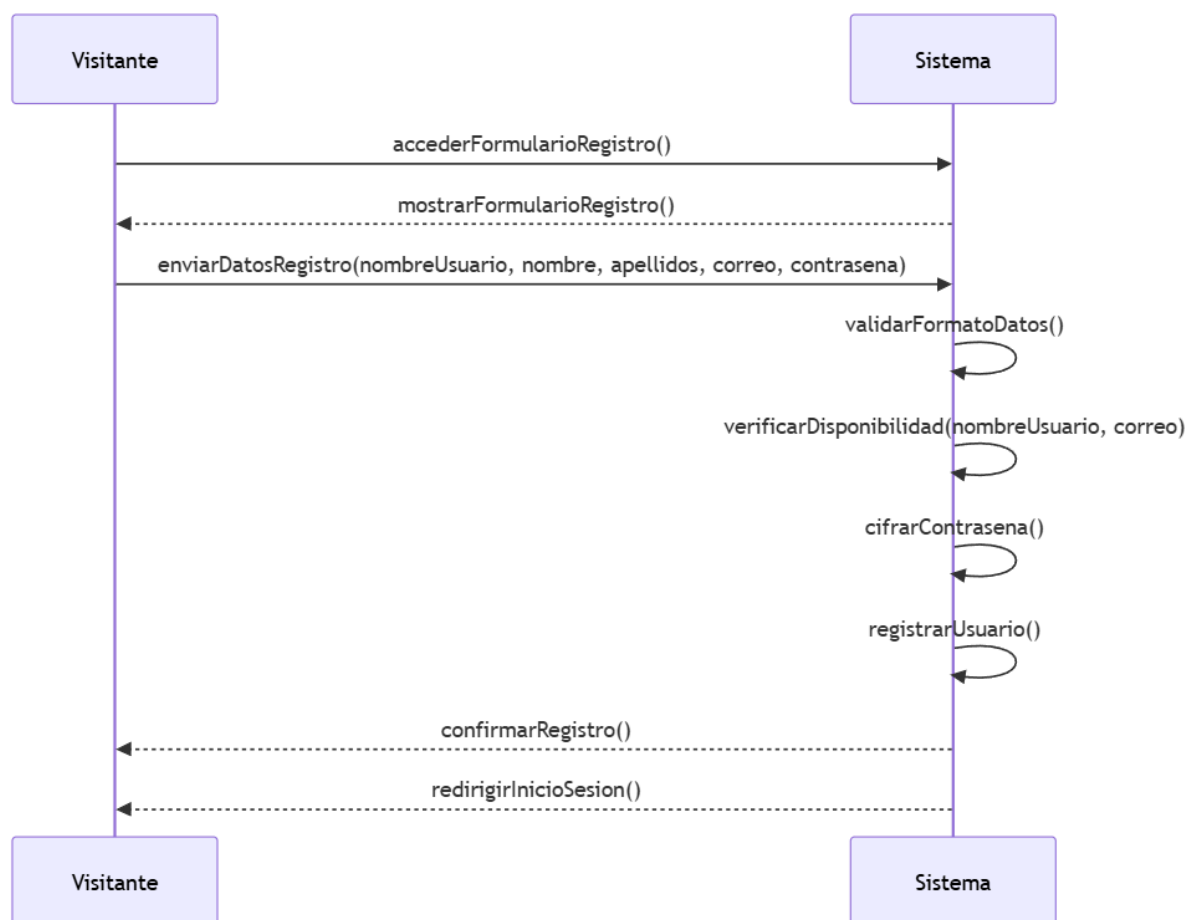


Ilustración x - Secuencia de interacción del registro de una cuenta (CU-001)

Secuencia CU-002: Iniciar sesión

El inicio de sesión es el punto de acceso a toda la funcionalidad privada de la plataforma. El visitante accede al formulario de inicio de sesión, introduce sus credenciales —nombre de usuario y contraseña— y las envía al sistema. Este verifica que la contraseña introducida coincida con el hash almacenado para ese nombre de usuario y, si la verificación es correcta, genera el token de acceso y el token de refresco, los establece en cookies seguras y redirige al usuario a su panel, concediéndole acceso a las áreas privadas.

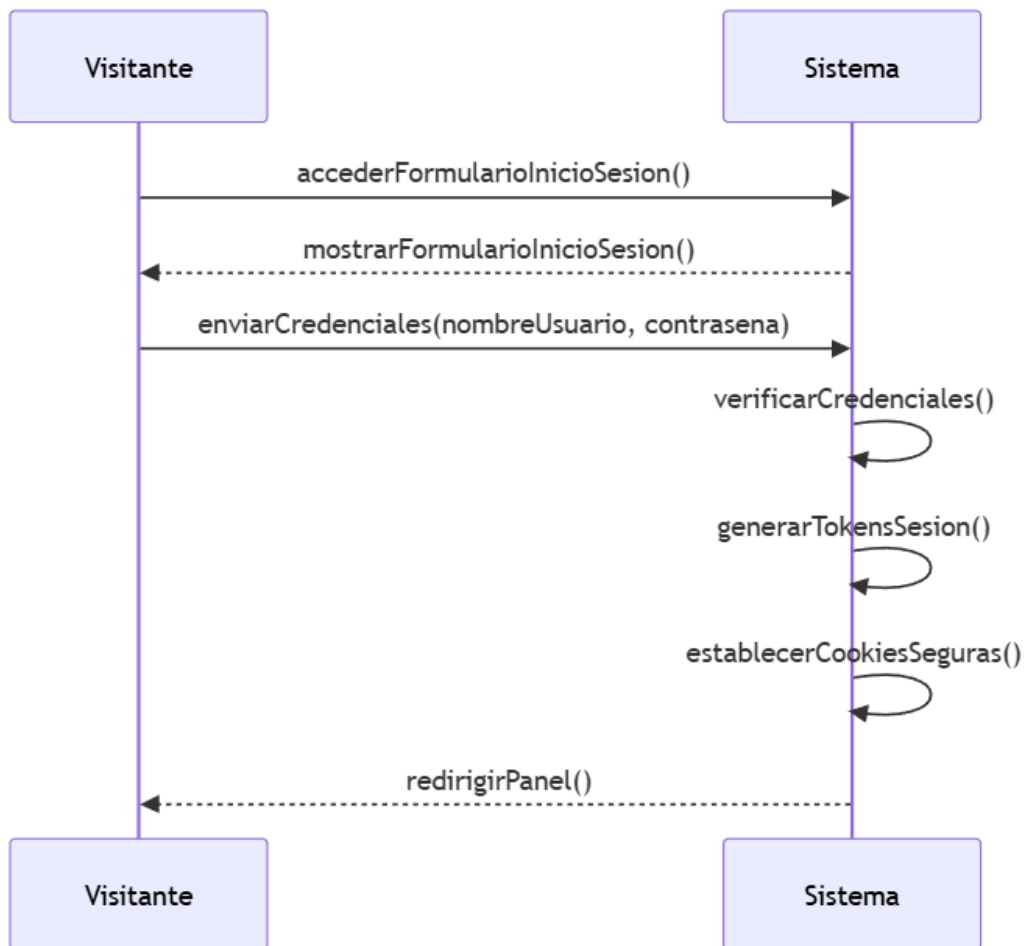


Ilustración x - Secuencia de interacción del inicio de sesión (CU-002)

Secuencia CU-003: Cerrar sesión

El cierre de sesión completa el ciclo de acceso y es especialmente relevante en entornos de uso compartido. El usuario autenticado selecciona la opción de cerrar sesión y el sistema revoca el token de refresco asociado a su sesión, elimina las cookies de sesión del navegador y redirige al usuario a la página de inicio de sesión. A partir de ese momento, cualquier intento de acceder a las áreas privadas desde el mismo equipo es rechazado hasta que se vuelva a autenticar.

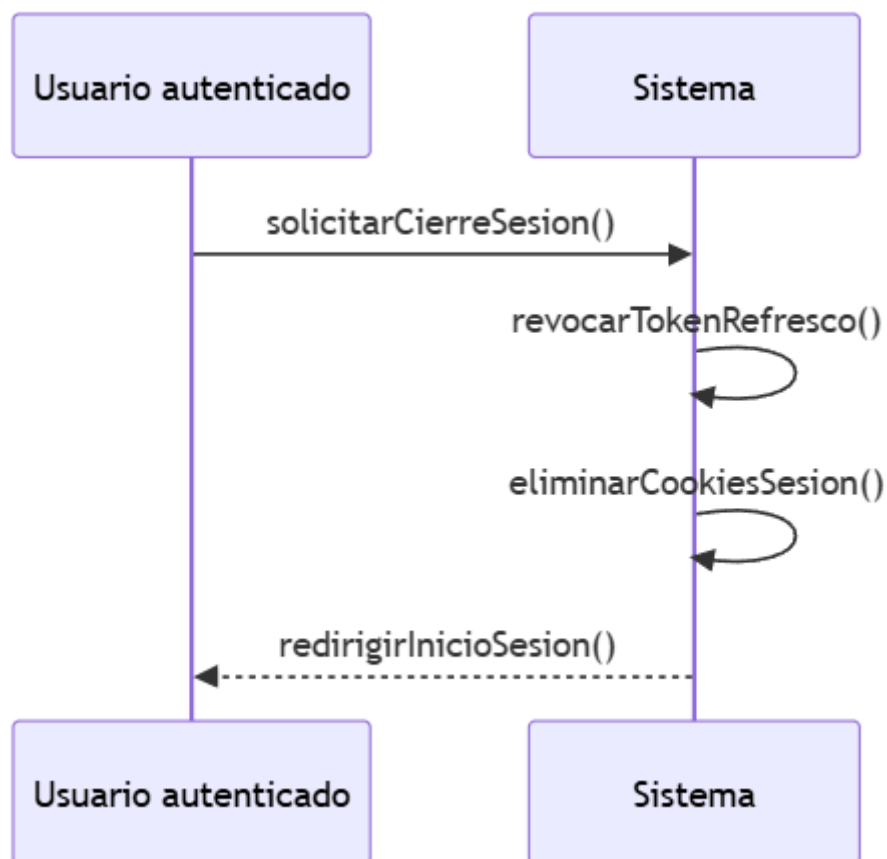


Ilustración x - Secuencia de interacción del cierre de sesión (CU-003)

Secuencia CU-004: Cambiar el idioma de la interfaz

El cambio de idioma es una interacción transversal disponible tanto para el visitante en el área pública como para el usuario autenticado en el área privada. El actor selecciona el idioma deseado en el selector de idioma de la interfaz y el sistema guarda la preferencia seleccionada, de modo que persista durante la sesión, y aplica las traducciones correspondientes en toda la interfaz sin necesidad de recargar la página, sin interrumpir el estado de navegación y, cuando procede, también en los informes generados y en el asistente conversacional.

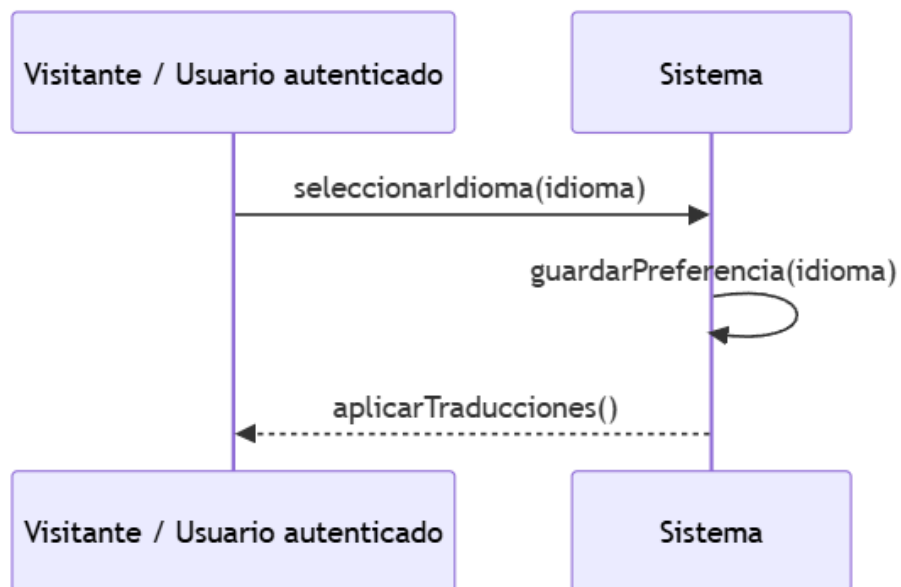


Ilustración x - Secuencia de interacción del cambio de idioma (CU-004)

14.2.2. SS-002 – Subsistema de Diagnóstico Asistido

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de diagnóstico asistido, descrito en el capítulo 13. Este subsistema agrupa el flujo clínico de la plataforma: el acceso al panel de diagnóstico (CU-005), la subida de una radiografía de tórax (CU-006), la selección de la arquitectura del modelo (CU-007), la solicitud del diagnóstico (CU-008), la visualización del resultado (CU-009) y la visualización de los mapas de explicabilidad (CU-010). Todos ellos tienen como actor al usuario autenticado, que opera sobre su propia consulta en curso. Las tres primeras secuencias preparan la consulta sobre la que se ejecutará el diagnóstico, mientras que las tres últimas completan el flujo clínico desde la solicitud hasta la presentación de los resultados. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-005: Acceder al panel de diagnóstico

El panel de diagnóstico es el punto de partida de toda la actividad clínica del usuario y el acceso a él constituye el primer contacto del usuario autenticado con la funcionalidad clínica de la plataforma. El usuario accede al panel desde la navegación principal y el sistema valida que su sesión se encuentra activa. Si la sesión es válida, el sistema carga el panel de diagnóstico y lo muestra, quedando disponibles la carga de la radiografía, el selector de modelos y el acceso al historial.

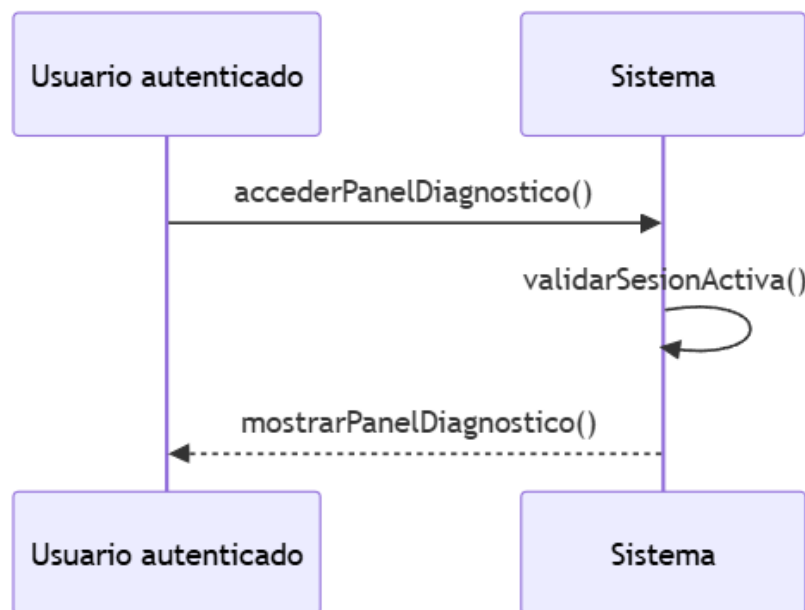


Ilustración x - Secuencia de interacción del acceso al panel de diagnóstico (CU-005)

Secuencia CU-006: Subir una radiografía de tórax

El primer paso de cualquier diagnóstico es incorporar la imagen al sistema. El usuario selecciona el archivo de la radiografía desde su equipo y el sistema valida que el formato sea un tipo de imagen admitido (JPEG o PNG) y que el tamaño no supere el límite máximo fijado. Una vez validada, la imagen se almacena de forma temporal en el servidor y queda asociada a la consulta en curso, de modo que el usuario puede continuar con la selección del modelo.

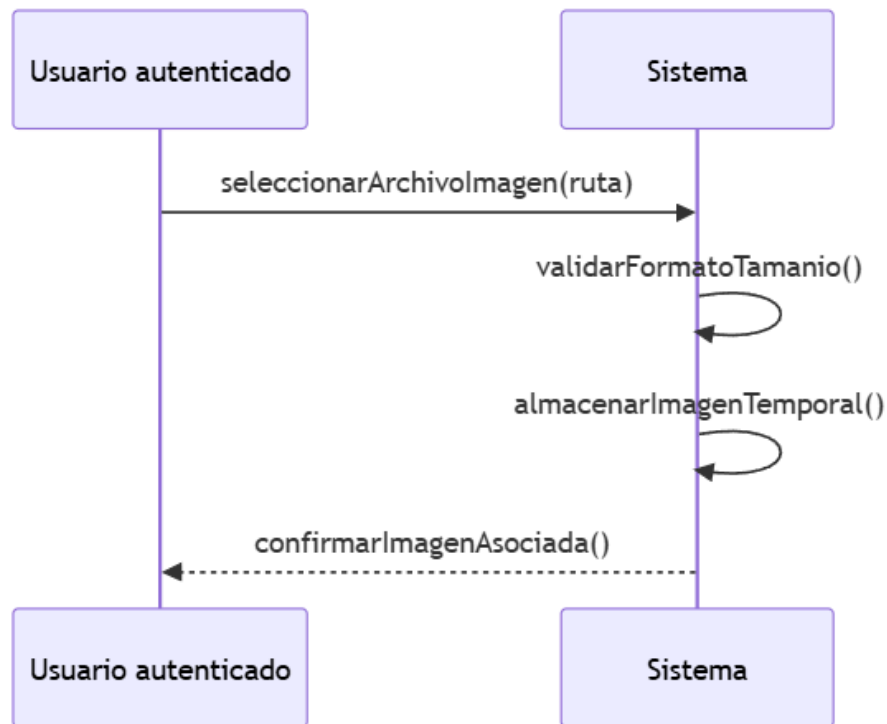


Ilustración x - Secuencia de interacción de la subida de una radiografía (CU-006)

Secuencia CU-007: Seleccionar la arquitectura para el diagnóstico

El resultado del diagnóstico depende del modelo empleado, por lo que el usuario debe poder elegir la arquitectura con la que desea realizar la consulta. El usuario despliega el selector de modelos del panel y el sistema muestra la lista de arquitecturas de deep learning disponibles. El usuario selecciona la arquitectura deseada y esta queda asociada a la consulta en curso.

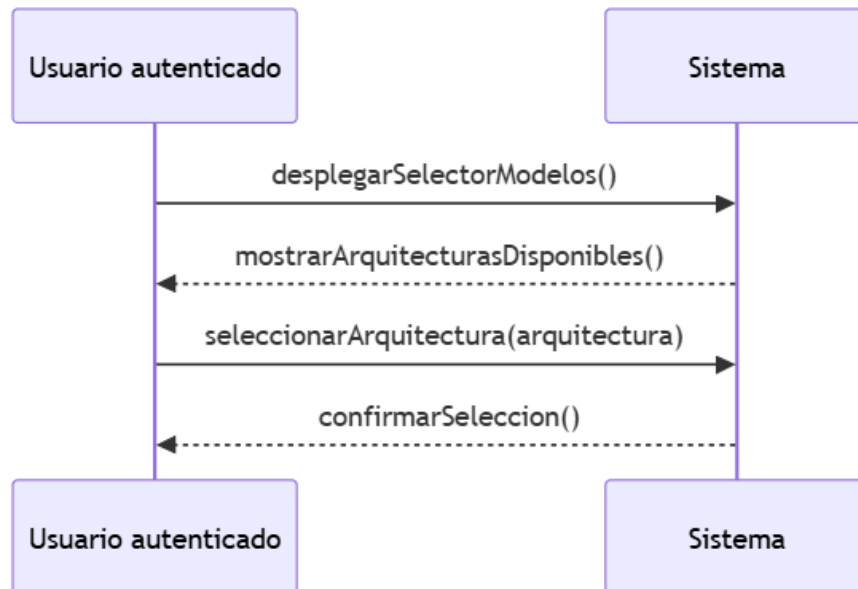


Ilustración x - Secuencia de interacción de la selección de la arquitectura (CU-007)

Secuencia CU-008: Solicitar un diagnóstico

La solicitud del diagnóstico es el punto de entrada del flujo clínico. El usuario envía la petición con la imagen cargada y el modelo seleccionado, y el sistema valida que la petición sea correcta. Para evitar bloquear la interfaz durante el procesamiento, el sistema encola el trabajo de diagnóstico y lo procesa en segundo plano: el worker realiza la predicción, genera los mapas de explicabilidad, genera el informe de la consulta y guarda la consulta. Cuando el trabajo finaliza, el sistema notifica al usuario la disponibilidad del resultado. La presentación del resultado, de los mapas y del informe se describe en las secuencias CU-009, CU-010 y CU-037.

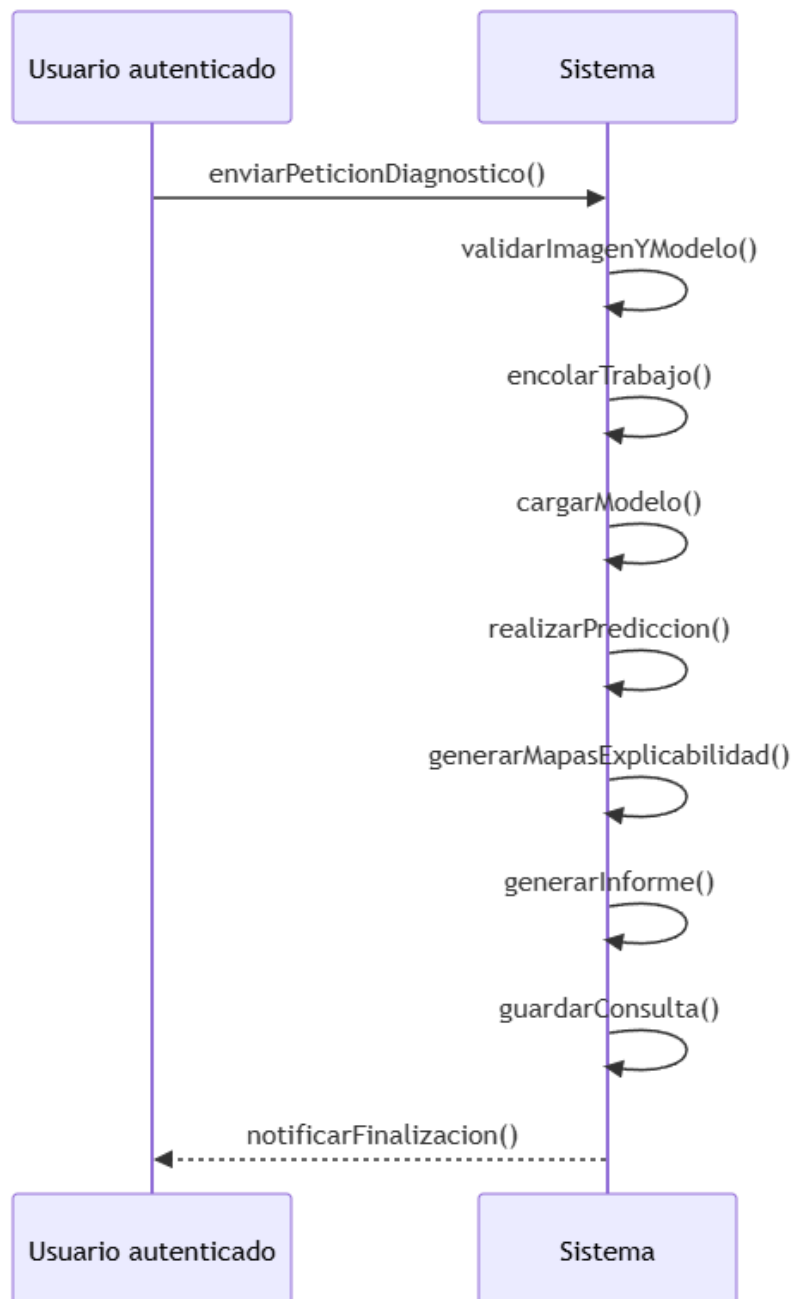


Ilustración x - Secuencia de interacción de la solicitud de un diagnóstico (CU-008)

Secuencia CU-009: Visualizar el resultado del diagnóstico

Cuando el trabajo de diagnóstico finaliza, el usuario consulta el estado de su consulta y el sistema comprueba que ha pasado a estado completado. Si es así, el sistema muestra el resultado de la consulta: la predicción (PNEUMONIA o NORMAL), el nivel de confianza asociado y el modelo empleado, presentados de forma clara y sin tecnicismos. El resultado queda registrado en el historial, de modo que pueda recuperarse en cualquier momento.

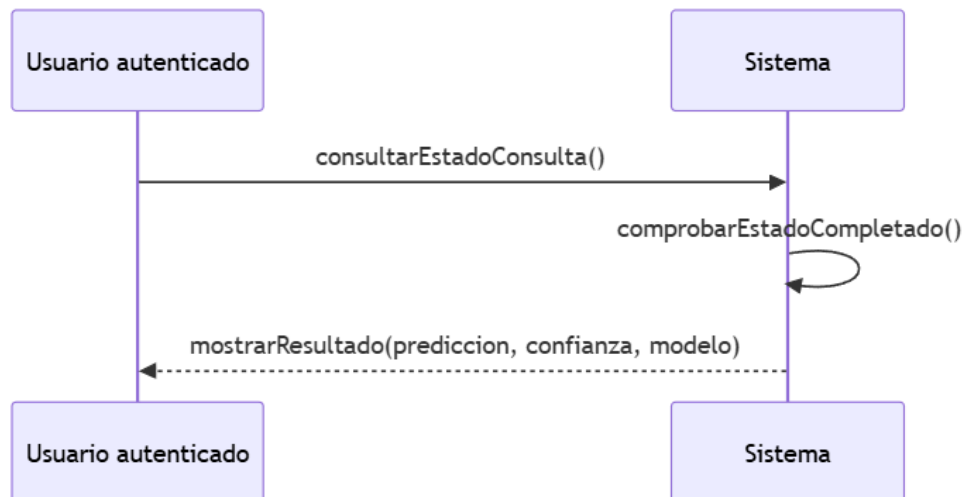


Ilustración x - Secuencia de interacción de la visualización del resultado (CU-009)

Secuencia CU-010: Visualizar los mapas de explicabilidad

Un diagnóstico asistido no es útil si el profesional no puede comprobar los motivos de la decisión del modelo. Cuando la consulta está completada, el usuario la selecciona para inspeccionar sus explicaciones y el sistema comprueba que la consulta pertenece al usuario, garantizando el aislamiento de datos. El sistema muestra entonces el mosaico con la radiografía original y los mapas de explicabilidad generados: Saliency Maps, SmoothGrad y Grad-CAM para las arquitecturas convolucionales, o mapas de atención para las arquitecturas Transformer.

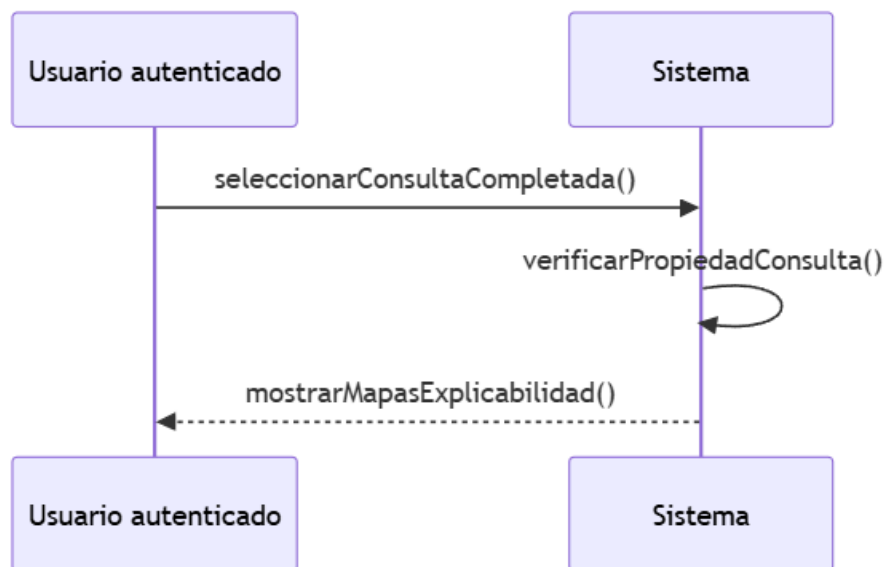


Ilustración x - Secuencia de interacción de la visualización de los mapas de explicabilidad (CU-010)

Secuencia CU-037: Generar el informe PDF del diagnóstico

Un diagnóstico asistido no es útil si el profesional no puede comprobar los motivos de la decisión del modelo. Cuando la consulta está completada, el usuario la selecciona para inspeccionar sus explicaciones y el sistema comprueba que la consulta pertenece al usuario, garantizando el aislamiento de datos. El sistema muestra entonces el mosaico con la radiografía original y los mapas de explicabilidad generados: Saliency Maps, SmoothGrad y Grad-CAM para las arquitecturas convolucionales, o mapas de atención para las arquitecturas Transformer.

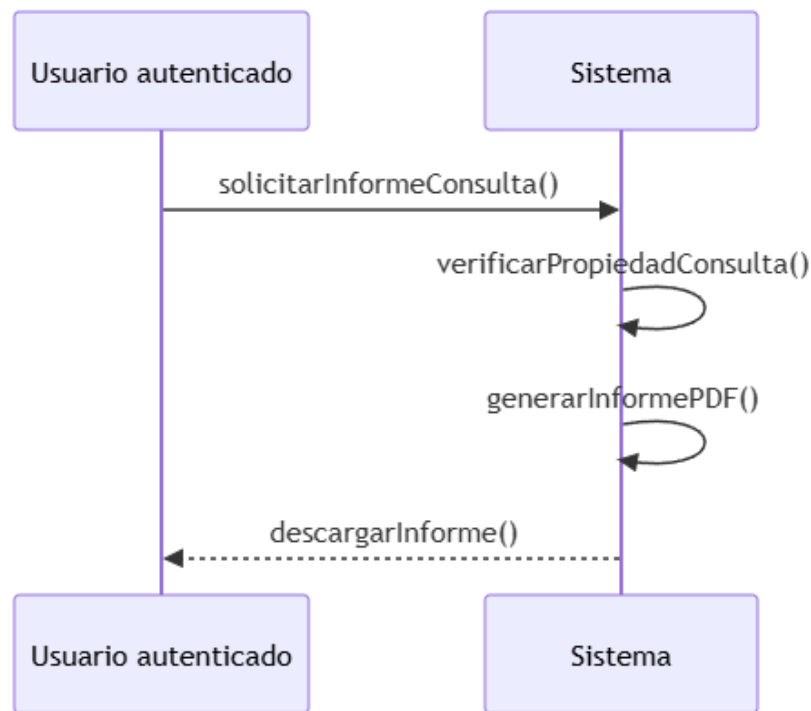


Ilustración x - Secuencia de interacción de la generación del informe PDF del diagnóstico (CU-037)

14.2.3. SS-003 – Subsistema de Gestión del Historial

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de gestión del historial, descrito en el capítulo 13. Este subsistema agrupa la recuperación y la gestión de las consultas de diagnóstico ya realizadas: la consulta del listado del historial (CU-011), la visualización del detalle de una consulta (CU-012), el renombrado de una consulta (CU-013) y su eliminación (CU-014). Todos ellos tienen como actor al usuario autenticado y operan exclusivamente sobre las consultas del propio usuario, en cumplimiento del aislamiento de datos entre usuarios: antes de mostrar o modificar cualquier información, el sistema verifica que la consulta pertenece al usuario que la solicita. La secuencia de CU-011 constituye el punto de entrada del subsistema, desde el que se alcanzan opcionalmente el detalle (CU-012) y, desde este, el renombrado (CU-013) y la eliminación (CU-014). Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-011: Consultar el historial de consultas

El profesional recupera sus consultas anteriores para revisar un diagnóstico, comparar la evolución de un caso o reutilizar una imagen. El usuario accede a su historial desde el panel de diagnóstico y el sistema recupera únicamente las consultas del propio usuario, mostrando el listado con los datos esenciales de cada una: fecha, modelo empleado, resultado y confianza.

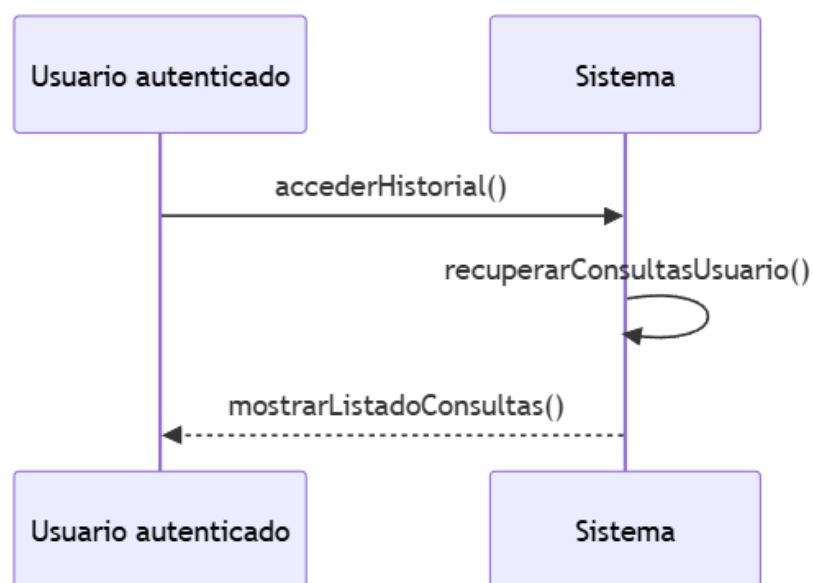


Ilustración x - Secuencia de interacción de la consulta del historial (CU-011)

Secuencia CU-012: Ver el detalle de una consulta del historial

El usuario abre el detalle completo de una consulta concreta para recuperar la imagen original, el resultado, la confianza y los mapas de explicabilidad asociados. El usuario selecciona una consulta del listado del historial y el sistema comprueba que la consulta pertenece al usuario. Si la verificación es correcta, el sistema muestra el detalle completo de la consulta, incluyendo los metadatos asociados. La figura muestra esta secuencia.

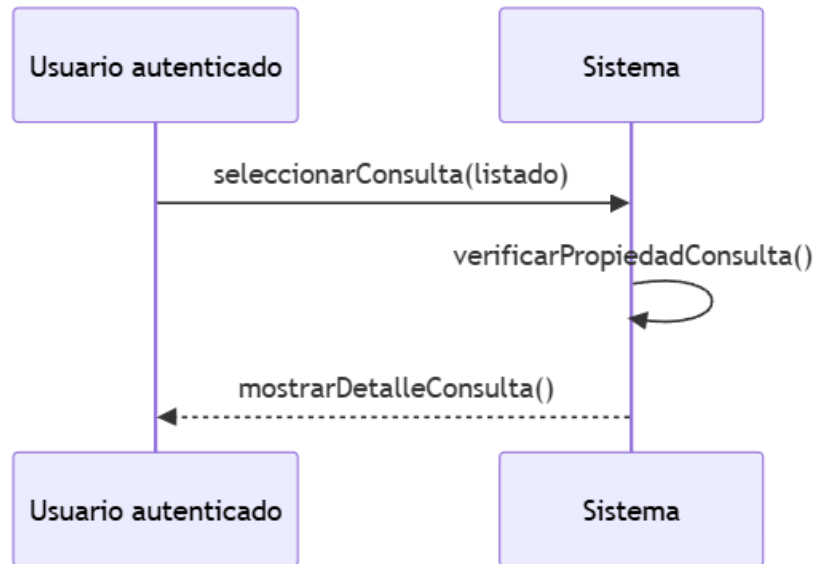


Ilustración x - Secuencia de interacción de la visualización del detalle (CU-012)

Secuencia CU-013: Renombrar una consulta del historial

El usuario modifica el nombre de una de sus consultas para identificarla mejor, por ejemplo cuando un mismo paciente tiene varias placas. El usuario indica el nuevo nombre desde el detalle de la consulta, el sistema comprueba que la consulta pertenece al usuario y valida que el nuevo nombre no esté vacío. Si la validación es correcta, el sistema actualiza el nombre de la consulta, sin alterar la imagen, el resultado ni los mapas de explicabilidad.

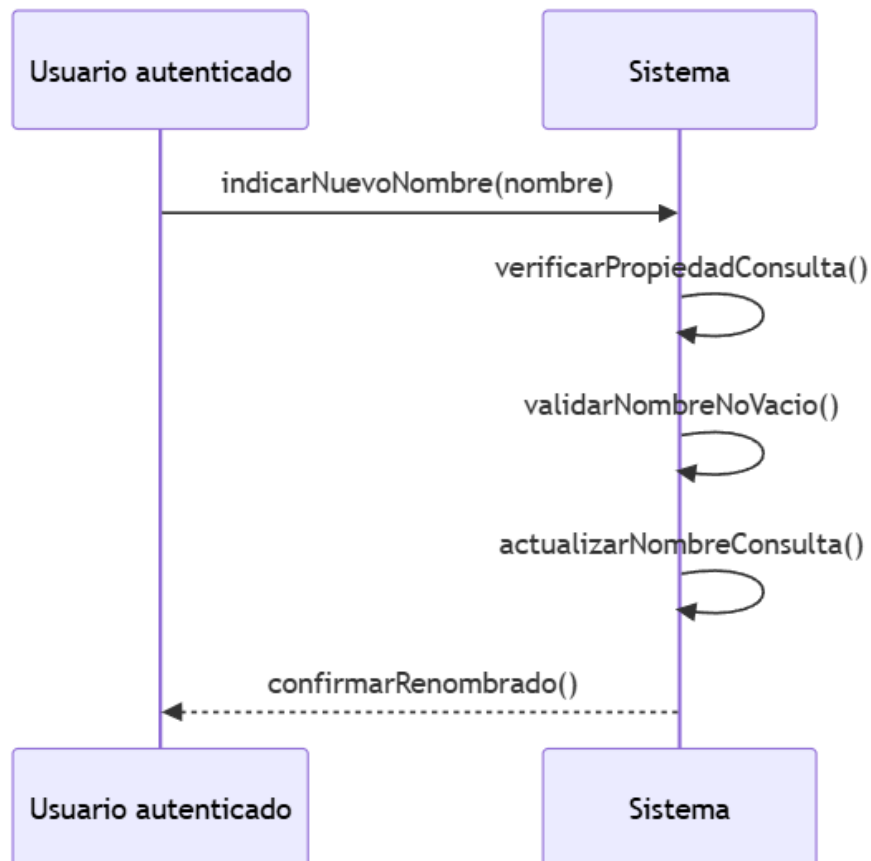


Ilustración x - Secuencia de interacción del renombrado de una consulta (CU-013)

Secuencia CU-014: Eliminar una consulta del historial

El usuario depura su historial eliminando las consultas que ya no necesita. Al tratarse de una operación que retira información clínica, el sistema comprueba primero que la consulta pertenece al usuario y solicita confirmación antes de ejecutarla. El usuario confirma la eliminación y el sistema retira el registro de la consulta, que deja de aparecer en su listado activo.

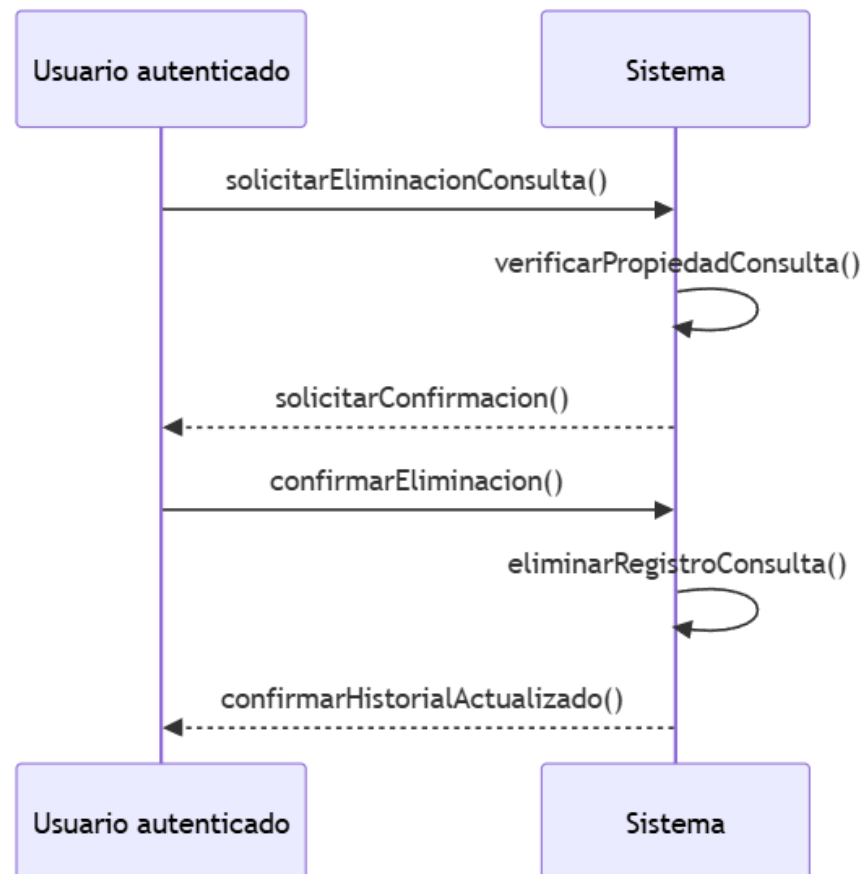


Ilustración x - Secuencia de interacción de la eliminación de una consulta (CU-014)

14.2.4. SS-004 – Subsistema de Laboratorio MLOps

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de laboratorio MLOps, descrito en el capítulo 13. Este subsistema agrupa la actividad investigadora de la plataforma: el acceso al laboratorio (CU-015), la conversación con el asistente para configurar un experimento (CU-016), la selección de la carpeta del dataset (CU-017), el lanzamiento del experimento (CU-018), la consulta de las sesiones (CU-019), la consulta de los resultados de un modelo (CU-020), la visualización de sus mapas de explicabilidad (CU-021), la consulta del ranking (CU-022), la comparativa estadística (CU-023) y su recálculo (CU-024), el análisis de explicabilidad (CU-025), la validación externa (CU-026) y su consulta (CU-027), la generación del informe PDF (CU-028), el renombrado y la eliminación de sesiones (CU-029 y CU-030) y la comprobación de la limitación de entrenamientos simultáneos y encolados (CU-039). Todos ellos tienen como actor al usuario autenticado, que opera exclusivamente sobre sus propias sesiones en cumplimiento del aislamiento de datos entre usuarios. Las secuencias se presentan siguiendo el orden natural de la actividad del laboratorio, desde la configuración del experimento hasta la consulta y gestión de los resultados. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-015: Acceder al laboratorio de entrenamiento

El laboratorio de entrenamiento es el espacio en el que se desarrolla la actividad investigadora de la plataforma. El usuario accede al laboratorio desde la navegación principal y el sistema valida que su sesión se encuentra activa. Si la sesión es válida, el sistema carga el entorno del laboratorio y lo muestra, quedando disponibles el asistente conversacional, el listado de sesiones y el resto de las funcionalidades del módulo.

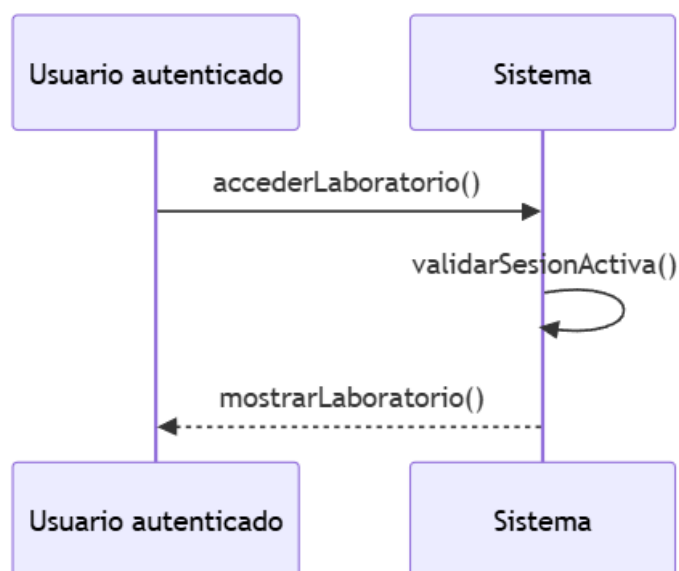


Ilustración x - Secuencia de interacción del acceso al laboratorio (CU-015)

Secuencia CU-016: Conversar con el asistente para configurar un experimento

El asistente conversacional es la interfaz principal de configuración del experimento y su interacción es un diálogo bidireccional: el usuario no entrega la configuración completa de una sola vez, sino que mantiene un intercambio con el asistente hasta que todos los parámetros quedan definidos. El usuario envía un mensaje en lenguaje natural indicando los parámetros que desea y el sistema prepara el prompt de sistema definido y envía la petición al modelo de lenguaje. El modelo extrae los parámetros mencionados en el mensaje y, si falta alguno de los parámetros que definen el experimento —arquitecturas, número de épocas, tamaño de lote y tasa de aprendizaje—, el asistente pregunta por el parámetro faltante y la conversación continúa hasta completarlo. La ruta del dataset no forma parte de la conversación: se selecciona de forma validada mediante el caso de uso CU-017 y se incorpora a la configuración. Cuando el asistente dispone de todos los parámetros, devuelve la configuración estructurada, el sistema rellena el panel de configuración con los valores obtenidos y el usuario la revisa y confirma.

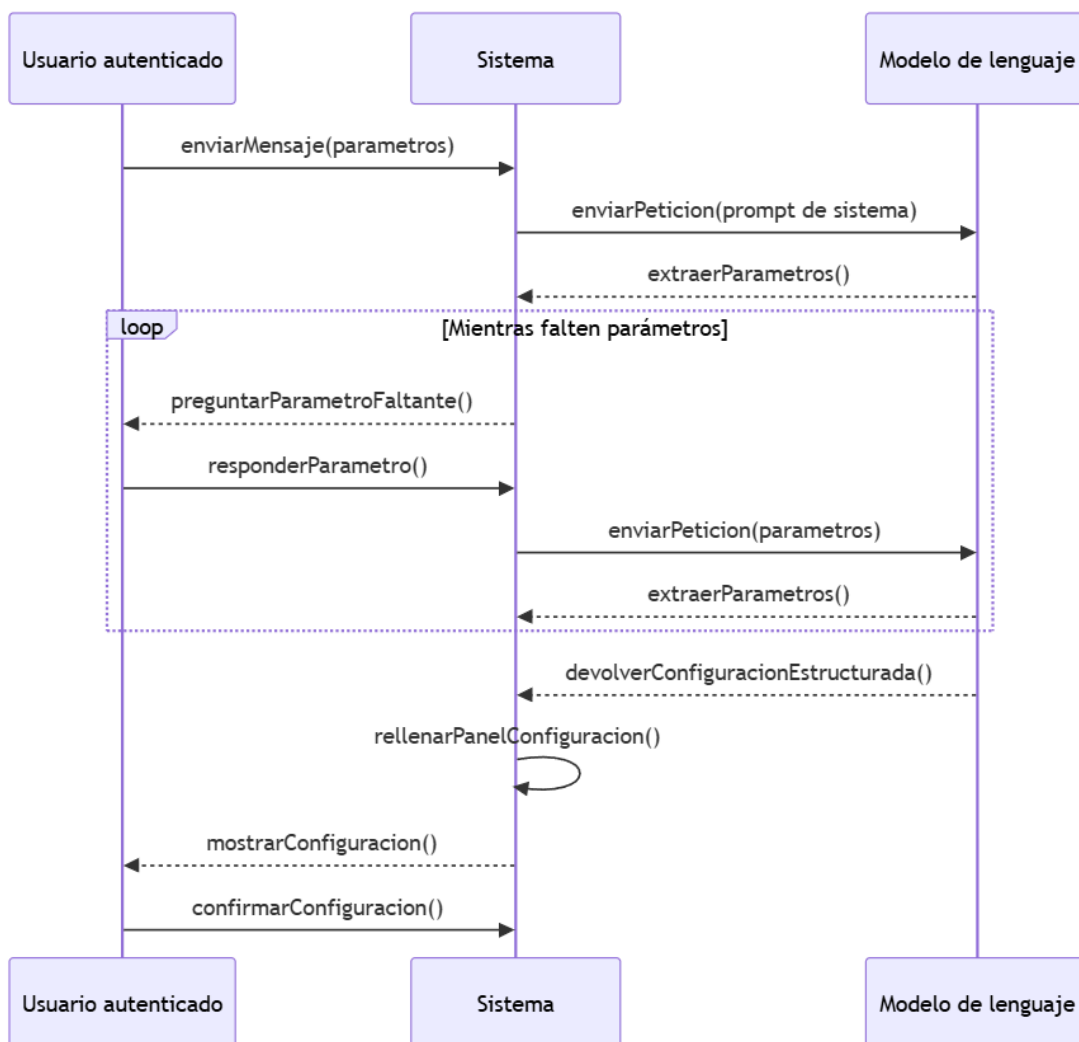


Ilustración x - Secuencia de interacción de la conversación con el asistente (CU-016)

Secuencia CU-017: Seleccionar la carpeta del dataset

Antes de lanzar el experimento, el usuario debe indicar la carpeta del conjunto de datos sobre el que se entrenará el modelo. El usuario solicita explorar la carpeta y el sistema devuelve la ruta, preconfigurada en el entorno o seleccionada por el propio usuario. El usuario confirma la ruta y esta queda asociada a la configuración del experimento.

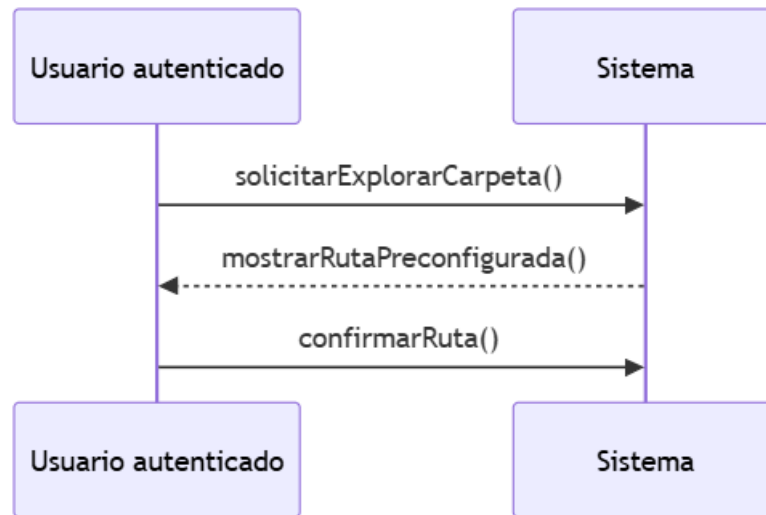


Ilustración x - Secuencia de interacción de la selección de la carpeta del dataset (CU-017)

Secuencia CU-018: Lanzar un experimento de entrenamiento

El lanzamiento del experimento inicia el proceso de entrenamiento. El usuario envía la configuración del experimento, ya completada mediante la conversación con el asistente, y el sistema crea la sesión de entrenamiento y encola el trabajo de ejecución. El worker procesa el trabajo en segundo plano: ejecuta el entrenamiento con validación cruzada, el análisis de explicabilidad y la comparación estadística. Al finalizar, el sistema actualiza el estado de la sesión y notifica al usuario.

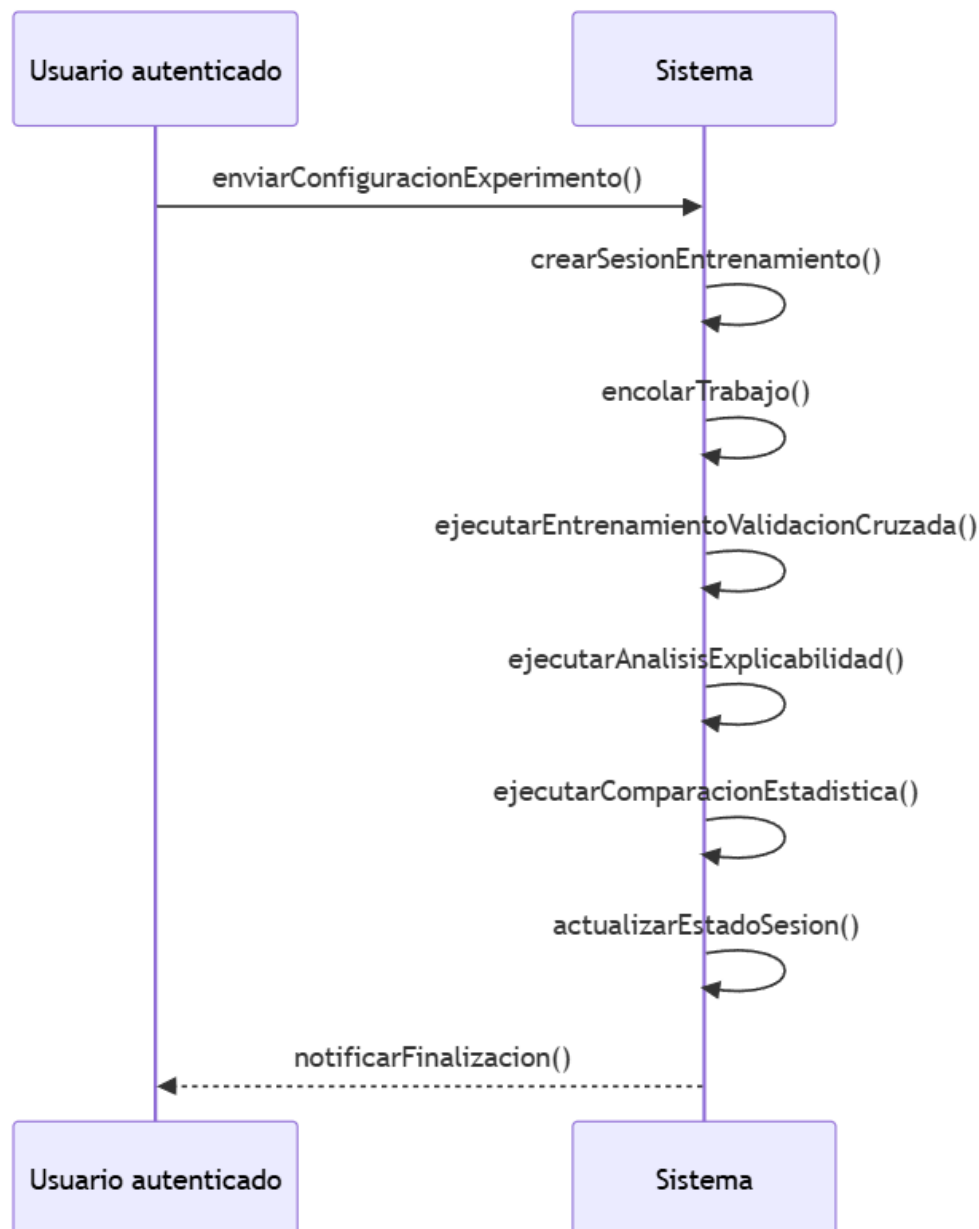


Ilustración x - Secuencia de interacción del lanzamiento de un experimento (CU-018)

Secuencia CU-019: Consultar las sesiones de entrenamiento

El investigador necesita recuperar sus sesiones de entrenamiento para revisar su estado y sus resultados. El usuario accede al listado de sesiones del laboratorio y el sistema recupera únicamente las sesiones del propio usuario, mostrando para cada una su estado y los modelos generados.

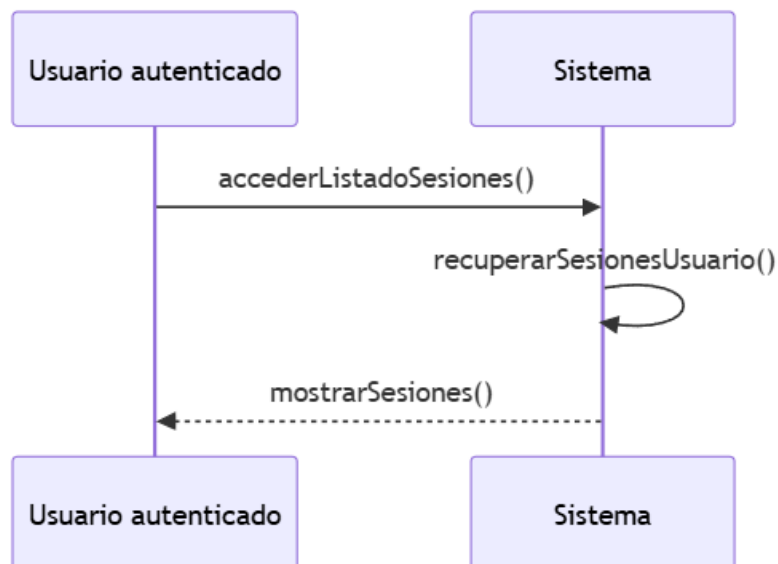


Ilustración x - Secuencia de interacción de la consulta de las sesiones (CU-019)

Secuencia CU-020: Consultar los resultados de un modelo de la sesión

El usuario selecciona un modelo de la sesión para inspeccionar sus resultados. El sistema recupera y muestra las métricas del modelo, con su media y desviación sobre los pliegues de la validación cruzada, junto con los artefactos asociados. Esta información permite al investigador evaluar el rendimiento del modelo antes de decidir sobre su uso.

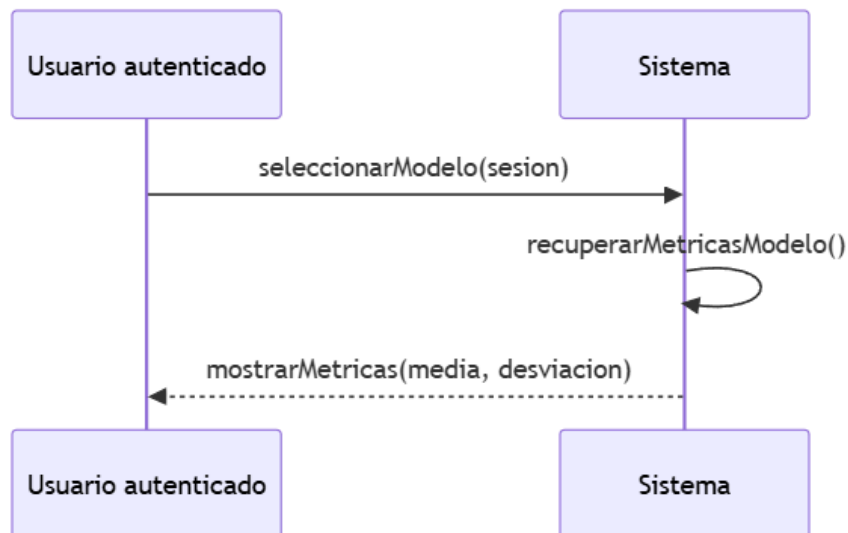


Ilustración x - Secuencia de interacción de la consulta de los resultados de un modelo (CU-020)

Secuencia CU-021: Visualizar los mapas de calor de explicabilidad de un modelo

La explicabilidad de los modelos es uno de los pilares de la plataforma. El usuario selecciona un modelo de la sesión y el sistema recupera las imágenes XAI generadas durante el análisis de explicabilidad, mostrándolas en una galería. De este modo, el investigador puede inspeccionar visualmente qué regiones de las imágenes pondera cada modelo antes de tomar decisiones.

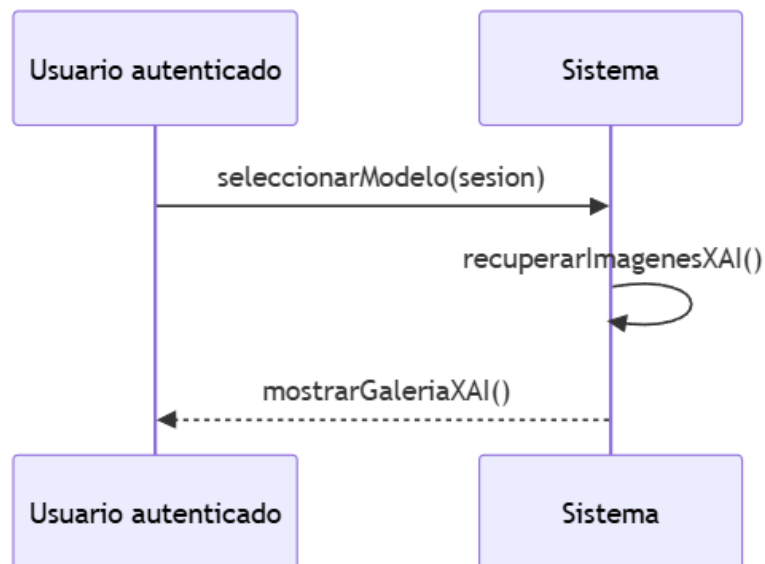


Ilustración x - Secuencia de interacción de la visualización de los mapas de explicabilidad (CU-021)

Secuencia CU-022: Consultar el ranking de modelos de la sesión

Para comparar rápidamente los modelos generados, el usuario puede consultar el ranking de la sesión. El usuario solicita el ranking y el sistema recupera y muestra los modelos ordenados por su AUC medio, de modo que el investigador identifica de un vistazo cuál de ellos presenta el mejor rendimiento discriminativo.

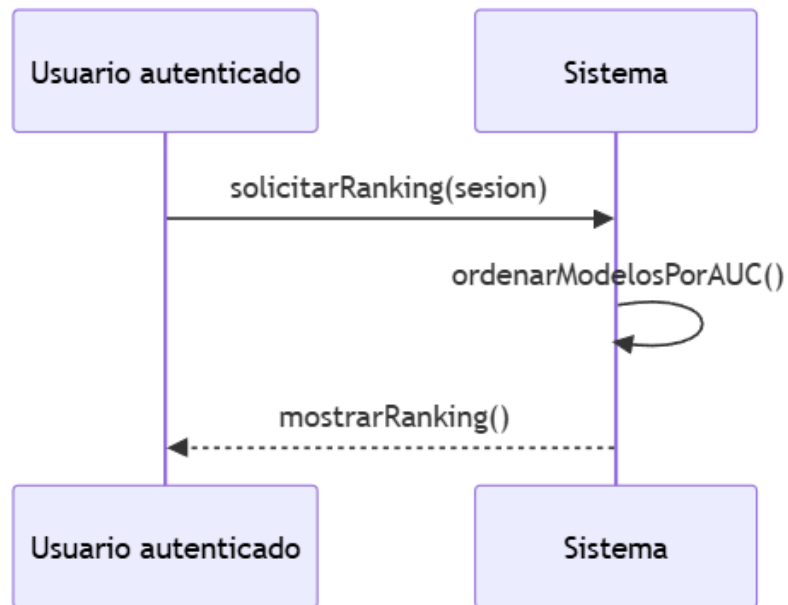


Ilustración x - Secuencia de interacción de la consulta del ranking (CU-022)

Secuencia CU-023: Consultar la comparativa estadística de la sesión

La comparativa estadística complementa el ranking con la significación de las diferencias entre modelos. El usuario accede a la vista de comparativa de la sesión y el sistema muestra la matriz de significación correspondiente, que indica si las diferencias de rendimiento entre cada par de modelos son estadísticamente significativas.

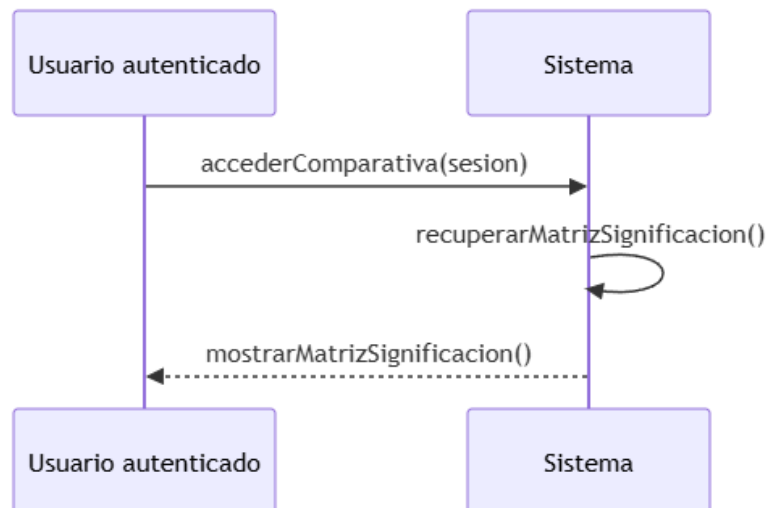


Ilustración x - Secuencia de interacción de la consulta de la comparativa estadística (CU-023)

Secuencia CU-024: Solicitar el recálculo de la comparativa estadística

Si el investigador modifica las condiciones del análisis, puede solicitar que la comparativa estadística se recalcule. El usuario solicita el recálculo y el sistema lanza el proceso en segundo plano, de modo que la interfaz permanece operativa. Cuando el recálculo finaliza, el sistema actualiza la comparativa y notifica al usuario la disponibilidad de los nuevos resultados.

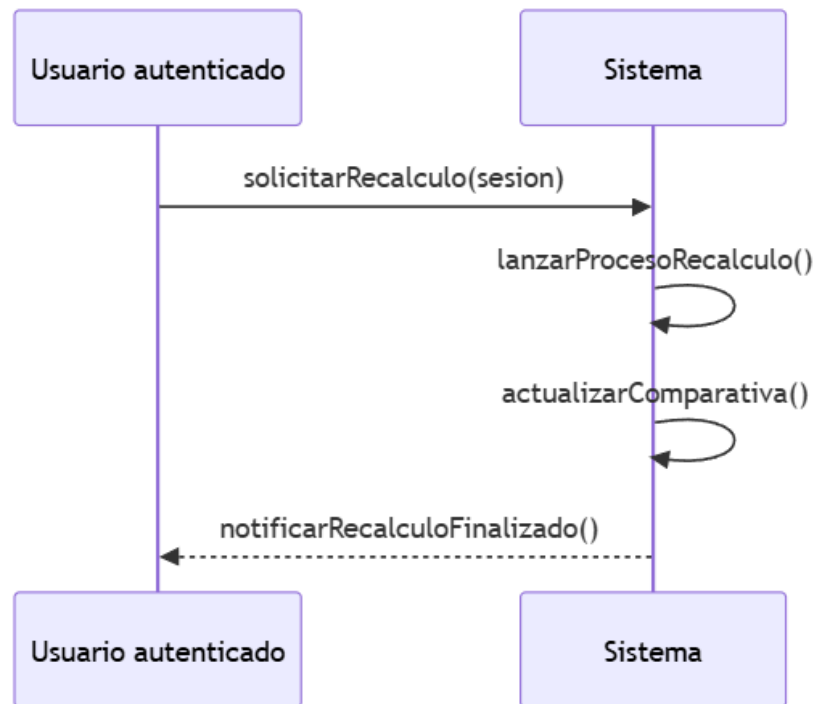


Ilustración x - Secuencia de interacción del recálculo de la comparativa (CU-024)

Secuencia CU-025: Ejecutar el análisis de explicabilidad de un modelo

El análisis de explicabilidad puede ejecutarse de forma independiente sobre un modelo concreto de la sesión. El usuario solicita generar el análisis XAI del modelo y el sistema ejecuta los scripts de explicabilidad en segundo plano. Cuando el análisis finaliza, el sistema notifica al usuario, que puede consultar las imágenes XAI generadas.

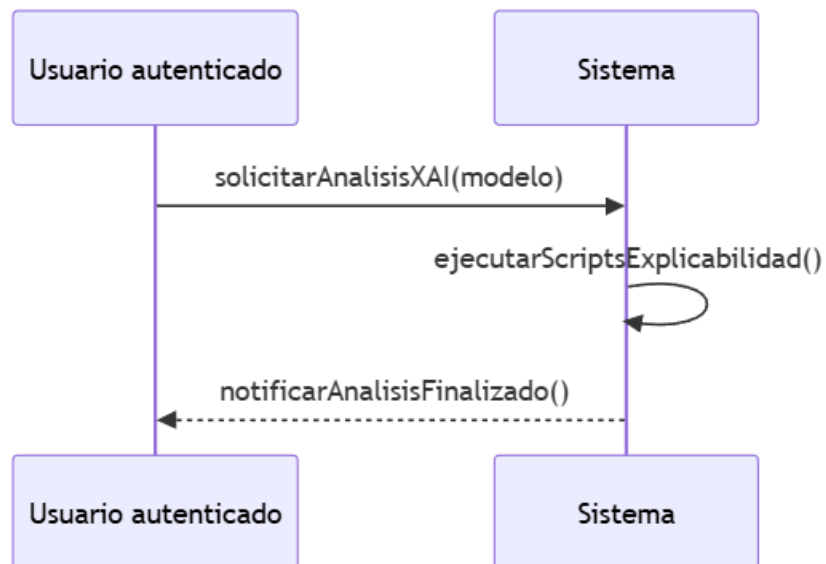


Ilustración x - Secuencia de interacción de la ejecución del análisis de explicabilidad (CU-025)

Secuencia CU-026: Solicitar la validación externa de la sesión

La validación externa aporta evidencia adicional sobre el rendimiento de los modelos con datos independientes. El usuario solicita la validación externa de la sesión y el sistema encola el trabajo de validación. El worker evalúa los modelos congelados sobre el conjunto externo y aplica el test de DeLong para comparar sus curvas ROC. Cuando la validación finaliza, el sistema notifica al usuario.

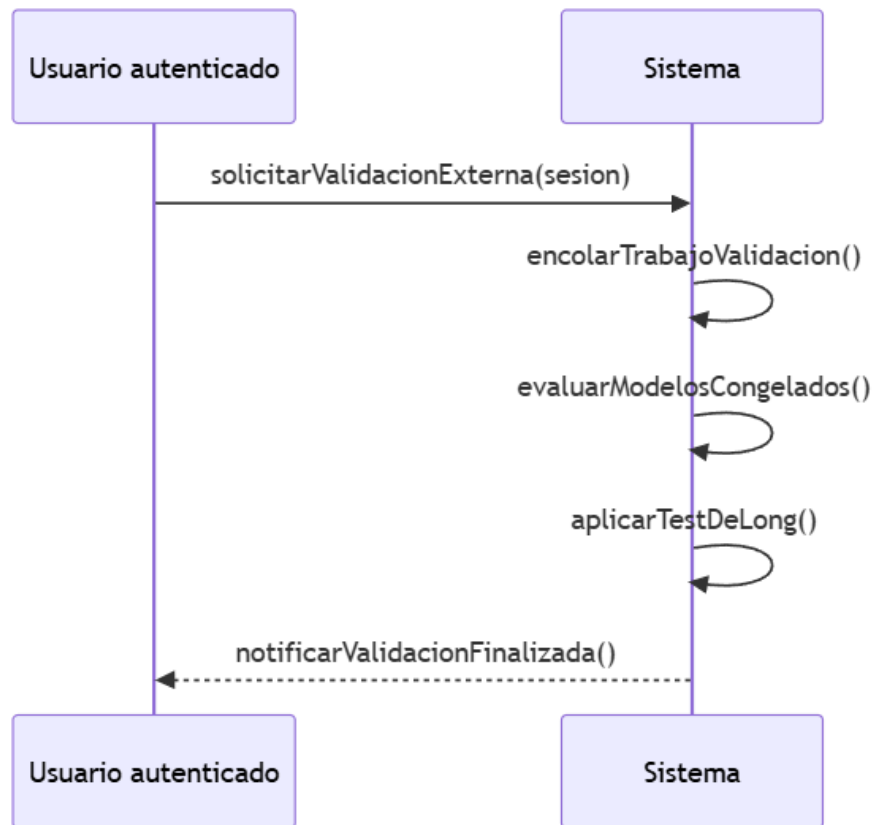


Ilustración x - Secuencia de interacción de la solicitud de la validación externa (CU-026)

Secuencia CU-027: Consultar los resultados de la validación externa

Una vez finalizada la validación externa, el usuario consulta sus resultados. El usuario accede a los resultados externos de la sesión y el sistema muestra las métricas de los modelos sobre el conjunto externo, las curvas ROC y la matriz de DeLong resultante del test estadístico.

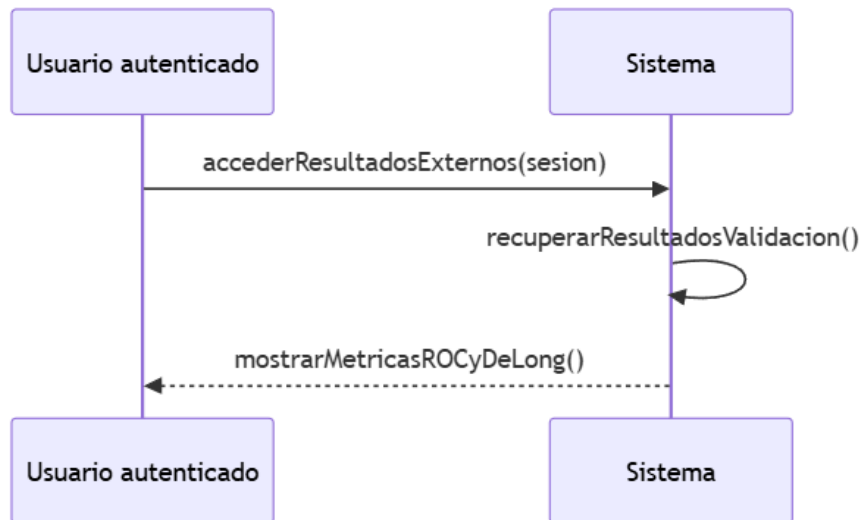


Ilustración x - Secuencia de interacción de la consulta de la validación externa (CU-027)

Secuencia CU-028: Generar el informe PDF de la sesión

El informe PDF consolida los resultados de la sesión para su difusión o conservación. El usuario solicita el informe de la sesión y el sistema genera el documento PDF con los resultados consolidados, incluyendo las métricas, los mapas de explicabilidad y la comparativa. Finalmente, el sistema inicia la descarga del documento al equipo del usuario.

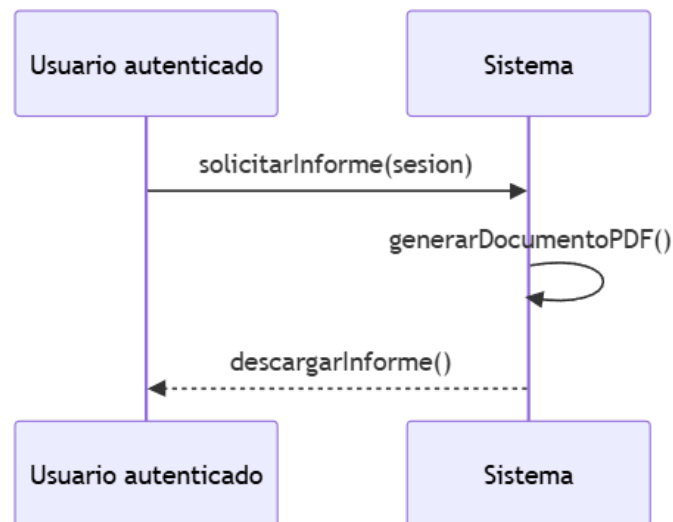


Ilustración x - Secuencia de interacción de la generación del informe PDF (CU-028)

Secuencia CU-029: Renombrar una sesión de entrenamiento

El investigador puede dar a sus sesiones un nombre más descriptivo para identificarlas mejor. El usuario indica el nuevo nombre de la sesión y el sistema comprueba que la sesión pertenece al usuario y valida que el nuevo nombre no esté vacío. Si la validación es correcta, el sistema actualiza el nombre de la sesión, sin alterar los modelos ni los artefactos asociados.

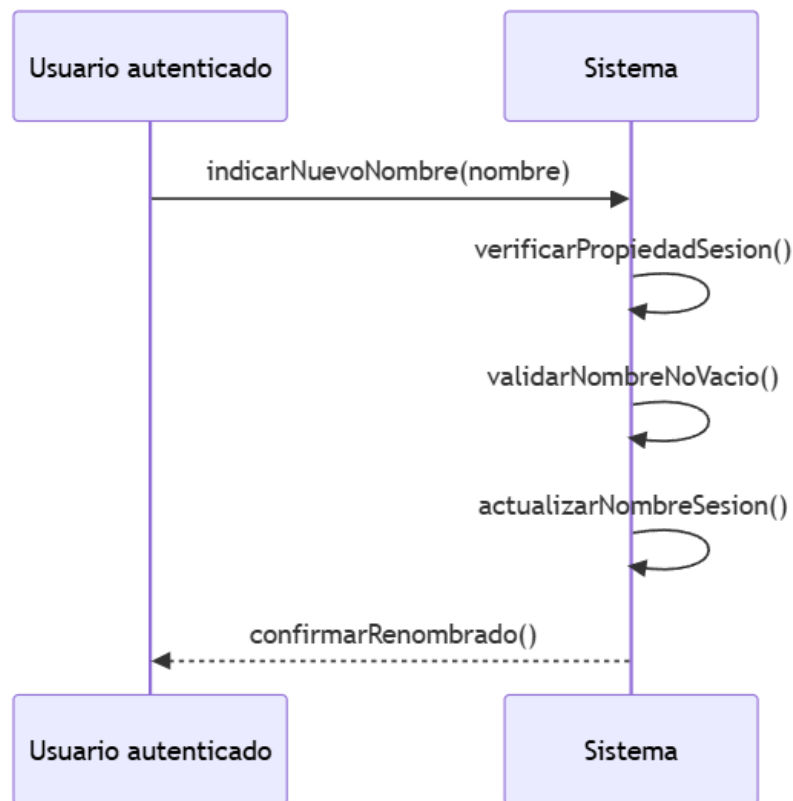


Ilustración x - Secuencia de interacción del renombrado de una sesión (CU-029)

Secuencia CU-030: Eliminar una sesión de entrenamiento

El investigador puede eliminar las sesiones que ya no necesita. Al tratarse de una operación que retira artefactos de entrenamiento, el sistema comprueba primero que la sesión pertenece al usuario y solicita confirmación antes de ejecutarla. El usuario confirma la eliminación y el sistema retira la sesión junto con sus artefactos asociados.

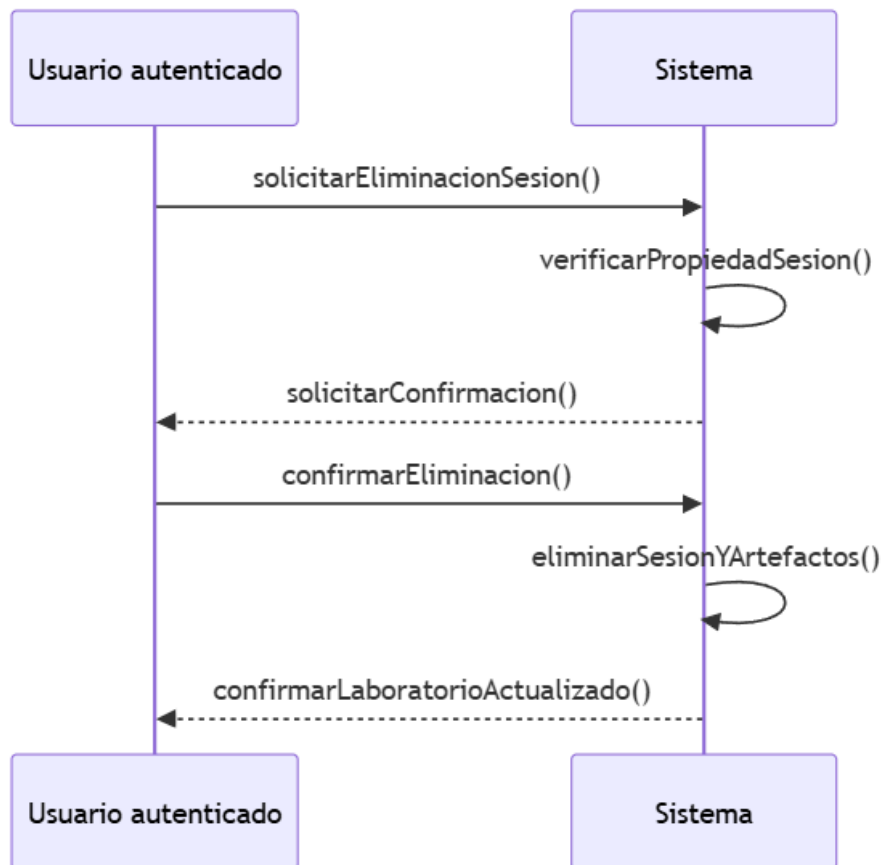


Ilustración x - Secuencia de interacción de la eliminación de una sesión (CU-030)

Secuencia CU-039: Comprobar la limitación de entrenamientos

La limitación de entrenamientos es una capacidad del sistema que evita que la competición por la GPU degrade el servicio para el resto de los usuarios. Cuando el usuario lanza un experimento y se supera el límite de entrenamientos simultáneos o encolados, el sistema mantiene el trabajo en espera en la cola y el usuario puede comprobarlo desde el panel de la cola de trabajos.

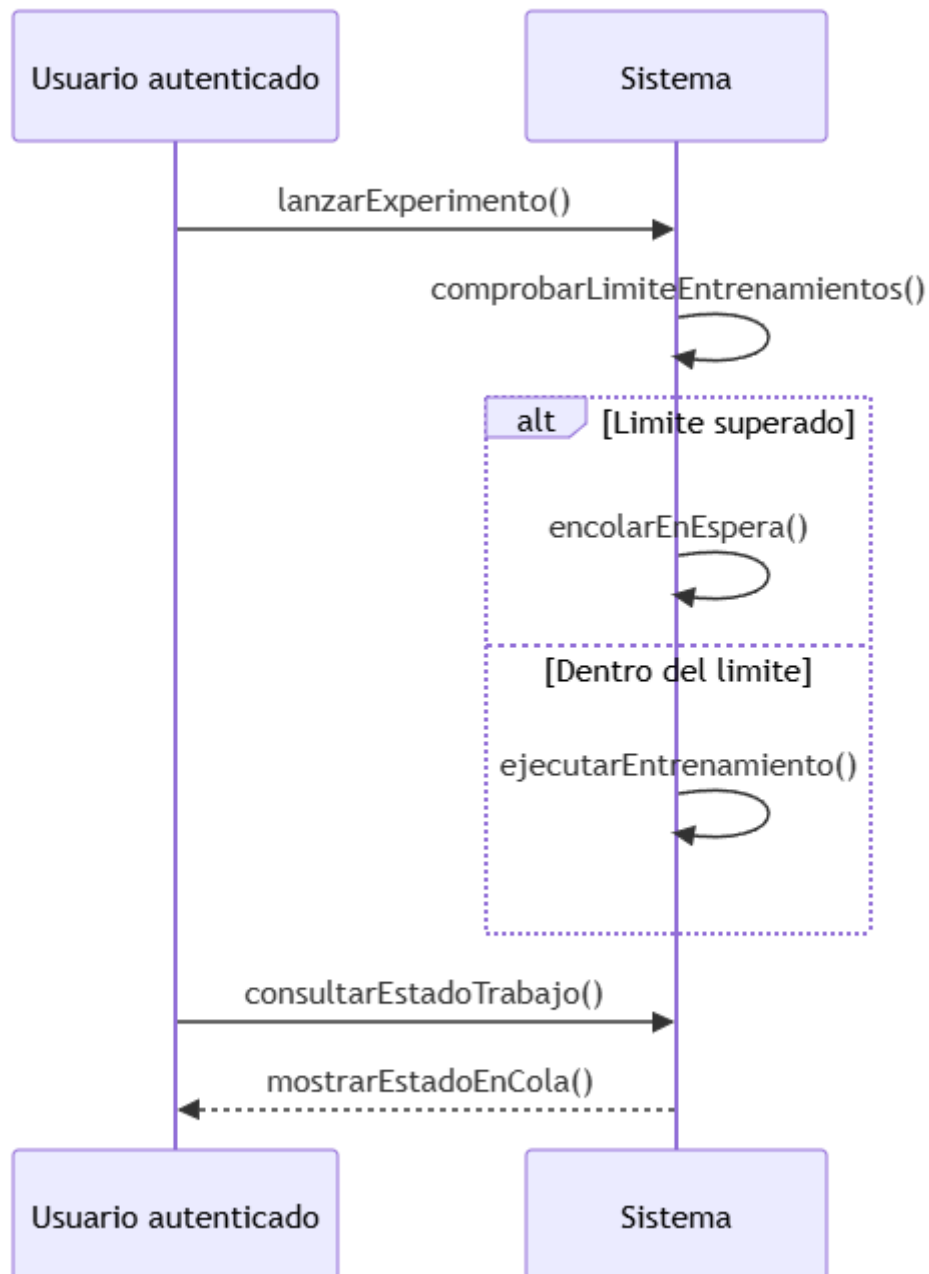


Ilustración x - Secuencia de interacción de la limitación de entrenamientos (CU-039)

14.2.5. SS-005 – Subsistema de Supervisión y Administración

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de supervisión y administración, descrito en el capítulo 13. Este subsistema agrupa las operaciones de supervisión de la plataforma: la consulta del listado de usuarios (CU-031), la consulta de las consultas de diagnóstico de un usuario concreto (CU-032) y la visualización del detalle de una de esas consultas (CU-033). Todos ellos tienen como actor al administrador y están restringidos a su rol: en cada operación el sistema verifica el rol del actor antes de mostrar cualquier información. Las secuencias siguen la navegación descendente de la supervisión, desde la visión global del listado hasta el detalle de una consulta concreta, permitiendo auditar un caso ante cualquier incidencia. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-031: Consultar el listado de usuarios

El listado de usuarios constituye la base de la supervisión de la plataforma. El administrador accede al panel de administración y el sistema verifica que el actor dispone del rol de administración. Si la verificación es correcta, el sistema recupera y muestra el listado de usuarios registrados, permitiendo al administrador identificar cuentas y comprobar el estado del sistema.

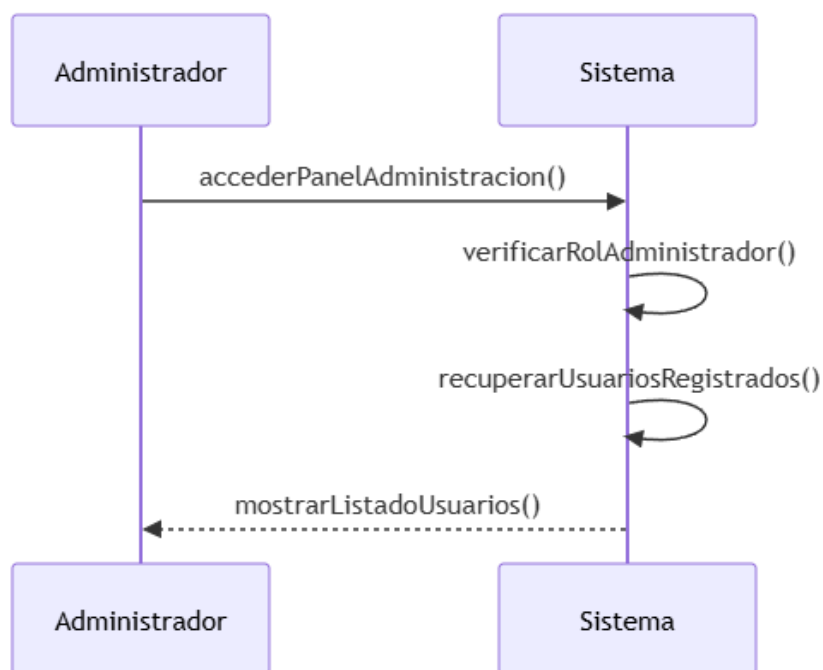


Ilustración x - Secuencia de interacción de la consulta del listado de usuarios (CU-031)

Secuencia CU-032: Consultar las consultas de un usuario

La supervisión de la actividad registrada exige poder examinar la actividad de un usuario concreto. El administrador selecciona un usuario del listado y el sistema verifica de nuevo el rol de administración del actor. Si la verificación es correcta, el sistema recupera y muestra las consultas de diagnóstico de ese usuario, permitiendo comprobar el uso que se hace de la plataforma y detectar posibles incidencias.

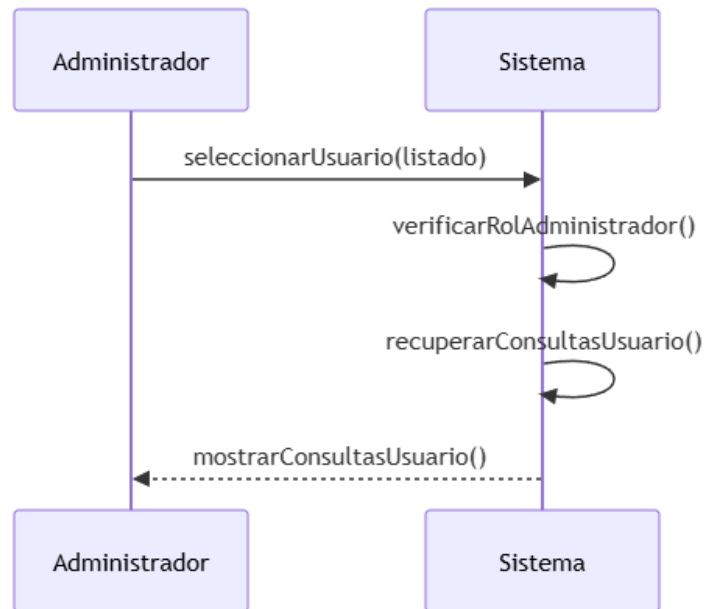


Ilustración x - Secuencia de interacción de la consulta de las consultas de un usuario (CU-032)

Secuencia CU-033: Ver el detalle de una consulta de un usuario

Para completar la supervisión, el administrador puede abrir el detalle completo de una consulta de un usuario, auditable ante cualquier incidencia. El administrador selecciona una consulta del listado de consultas del usuario y el sistema verifica el rol de administración del actor. Si la verificación es correcta, el sistema recupera y muestra el detalle completo de la consulta, incluidos la imagen, el resultado, la confianza y los metadatos asociados.

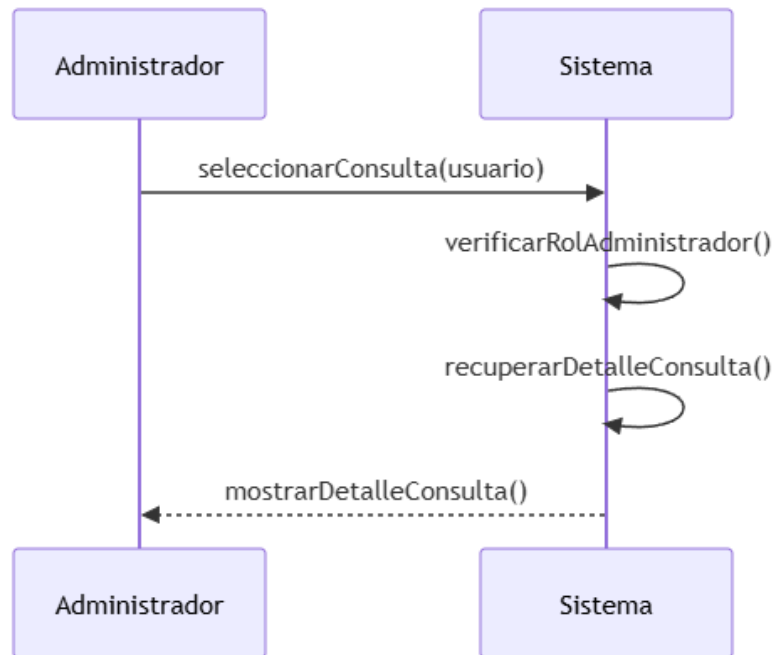


Ilustración x - Secuencia de interacción de la visualización del detalle de una consulta (CU-033)

Secuencia CU-038: Gestionar las cuentas de usuario

La administración de la plataforma no se limita a la consulta: el objetivo OBJ-011 exige gestionar las cuentas de usuario. El administrador selecciona una cuenta y el sistema verifica el rol de administración del actor. A continuación, el administrador ejecuta la operación deseada —desactivar la cuenta, cambiar el rol o eliminar la cuenta—, el sistema la aplica y registra la operación de auditoría.

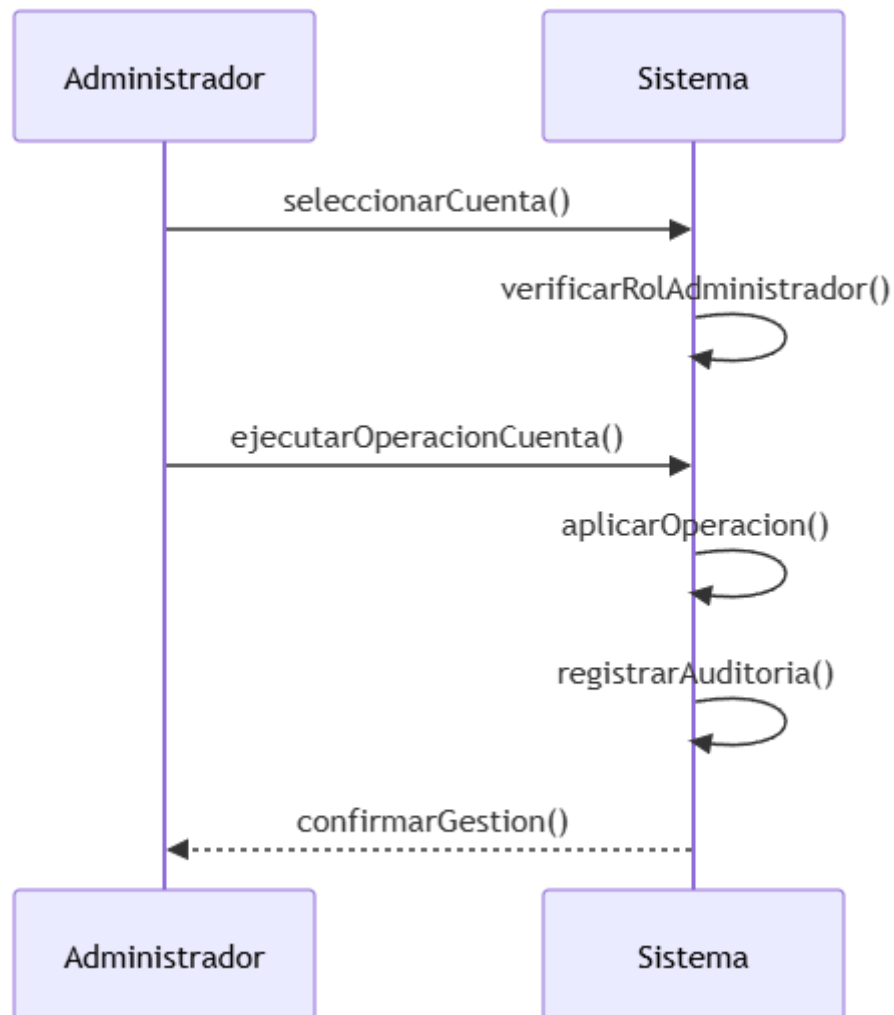


Ilustración x - Secuencia de interacción de la gestión de las cuentas de usuario (CU-038)

14.2.6. SS-006 – Subsistema de Capacidades Transversales

Este subapartado recoge las secuencias de interacción correspondientes a los casos de uso del subsistema de capacidades transversales, descrito en el capítulo 13. Este subsistema agrupa las funcionalidades que no pertenecen a un ámbito funcional concreto, sino que afectan a toda la plataforma y están disponibles para todo usuario autenticado: la consulta del estado de la cola de trabajos (CU-034), la cancelación de un trabajo pendiente (CU-035) y el cambio del tema visual de la interfaz (CU-036). Las dos primeras están vinculadas a la ejecución asíncrona de los diagnósticos, los entrenamientos y las validaciones externas, mientras que la tercera responde a la personalización de la interfaz. A diferencia del resto de subsistemas, ninguna de estas secuencias verifica la propiedad de un recurso concreto del usuario ni un rol específico, puesto que sus operaciones son de carácter general sobre la propia sesión. Se presentan a continuación, precedidos cada uno de una breve descripción de la interacción que modelan.

Secuencia CU-034: Consultar el estado de la cola de trabajos

El sistema ejecuta de forma asíncrona los diagnósticos, los entrenamientos y las validaciones externas, y el usuario necesita conocer en cada momento el estado de sus trabajos. El usuario accede al panel de la cola de trabajos y el sistema muestra el estado de los trabajos del usuario: pendientes, en ejecución, completados o fallidos. El sistema actualiza el estado de forma periódica, de modo que el usuario sabe cuándo estará disponible un resultado o cuándo debe repetir un trabajo que ha fallado.

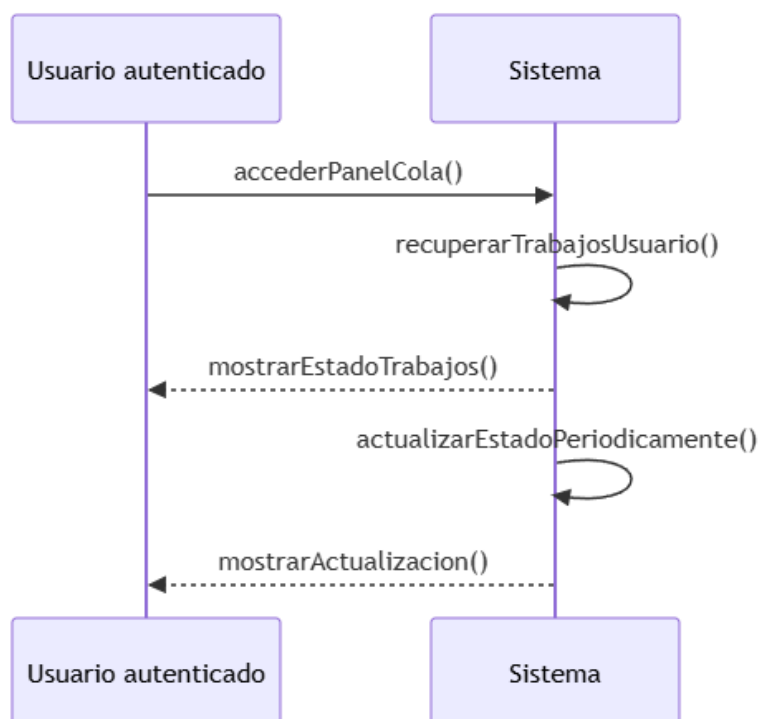


Ilustración x - Secuencia de interacción de la consulta del estado de la cola (CU-034)

Secuencia CU-035: Cancelar un trabajo de la cola

El usuario puede necesitar detener un trabajo que ha encolado por error o que ya no le interesa. El usuario solicita la cancelación de un trabajo de la cola y el sistema comprueba que el trabajo se encuentra en estado pendiente. Si es así, el sistema cancela el trabajo y lo elimina de la cola, de modo que no llega a ejecutarse. Si el trabajo ya está en ejecución o ha finalizado, el sistema informa de que no es cancelable y no lo modifica.

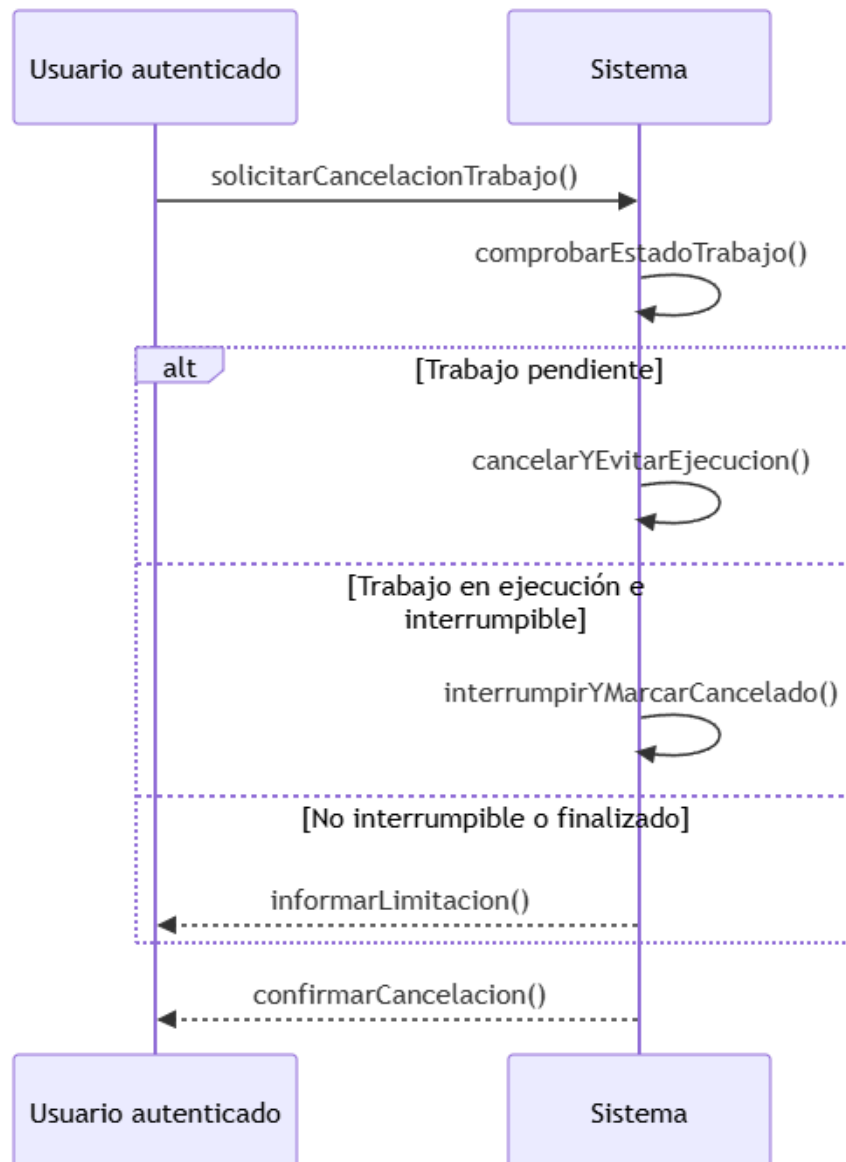


Ilustración x - Secuencia de interacción de la cancelación de un trabajo de la cola (CU-035)

Secuencia CU-036: Alternar el tema visual de la interfaz

El usuario personaliza la apariencia de la interfaz eligiendo entre el tema claro y el tema oscuro, según su preferencia visual y sus condiciones de trabajo. El usuario activa el cambio de tema en la interfaz y el sistema aplica el tema seleccionado en toda la interfaz de forma inmediata, sin interrumpir la navegación ni el estado del trabajo que el usuario esté realizando.

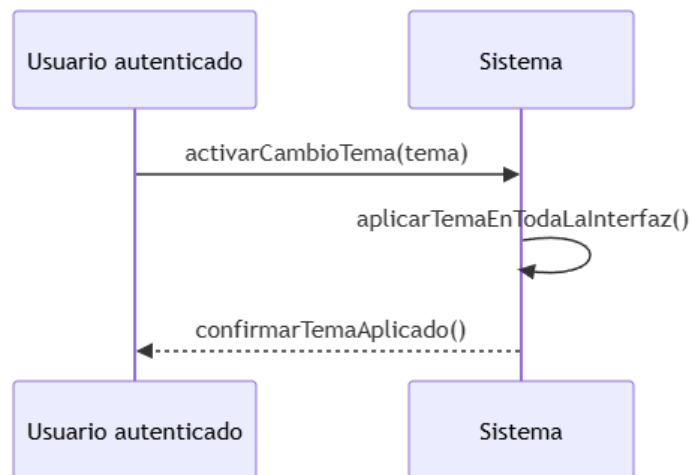


Ilustración x - Secuencia de interacción del cambio del tema visual (CU-036)

15. Verificación de la consistencia del análisis

El análisis del sistema de vitalXAI ha producido, a lo largo de los capítulos 10 a 14, un conjunto de artefactos complementarios que describen la plataforma desde perspectivas distintas: los objetivos del sistema (capítulo 11) declaran los fines que se persiguen, los requisitos funcionales y no funcionales (capítulo 12) concretan las capacidades y las condiciones de calidad que deben cumplirse, los casos de uso (capítulo 12) describen el comportamiento observable del sistema, los subsistemas de análisis (capítulo 13) agrupan esas capacidades en unidades cohesivas y el modelo de dominio (capítulo 14) identifica las entidades sobre las que todo ese comportamiento se apoya. La utilidad de este conjunto de artefactos depende de que exista una correspondencia coherente entre ellos: si un objetivo no tiene requisitos que lo materialicen, si un requisito no se refleja en ningún caso de uso, o si un caso de uso no tiene entidades del dominio sobre las que operar, el análisis adolecería de lagunas que se propagarían al diseño y, finalmente, al producto construido.

Este capítulo cierra la etapa de análisis realizando una verificación cruzada de la consistencia entre los artefactos producidos. El objetivo no es describir de nuevo el contenido de los capítulos anteriores, sino comprobar, de forma sistemática y verificable, que el modelo conceptual del sistema es completo, coherente y trazable en todas sus dimensiones. Para ello, el capítulo se organiza en tres secciones. La primera examina la coherencia entre los objetivos del sistema y los requisitos que los materializan. La segunda comprueba la cobertura de los requisitos por parte de los casos de uso y la correspondencia entre estos y los subsistemas de análisis. La tercera recoge las decisiones estratégicas de análisis que, derivadas de la verificación anterior, consolidan la estabilidad y la calidad metodológica del modelo y condicionarán las decisiones de diseño de los capítulos siguientes (Larman, 2004; Jacobson, Booch & Rumbaugh, 1999).

15.1. *Coherencia entre los objetivos del sistema y los requisitos*

La primera dimensión de la verificación de consistencia relaciona los objetivos del sistema, declarados en el capítulo 11, con los requisitos funcionales y no funcionales que los materializan. Los once objetivos del sistema (OBJ-001 a OBJ-011) cubren las dimensiones funcionales y de calidad de la plataforma: el diagnóstico asistido con inteligencia artificial explicable (OBJ-001), la gestión del historial de consultas (OBJ-002), la generación de informes descargables (OBJ-003), el laboratorio de entrenamiento MLOps (OBJ-004), la evaluación rigurosa de los modelos (OBJ-005), la reproducibilidad de los experimentos (OBJ-006), el acceso seguro y personalizado al sistema (OBJ-007), la persistencia y trazabilidad de la información (OBJ-008), la usabilidad e internacionalización de la interfaz (OBJ-009), la ejecución asíncrona de tareas de larga duración (OBJ-010) y la administración de la plataforma (OBJ-011).

15.1.1. Cobertura de los objetivos por los requisitos funcionales

La matriz de trazabilidad entre requisitos funcionales y objetivos del sistema, presentada en la sección 12.1.1.7 del capítulo 12, permite comprobar la cobertura de los objetivos por los cuarenta y un requisitos funcionales (RF-001 a RF-041). El examen de la matriz confirma que todos los objetivos del sistema quedan cubiertos por al menos un requisito funcional y que todos los requisitos funcionales están justificados por el objetivo al que responden, de modo que no existe ningún objetivo huérfano sin capacidades asociadas ni ninguna capacidad carente de fundamento.

Objetivo	Requisitos funcionales que lo materializan
OBJ-001 Diagnóstico asistido con IA explicable	RF-007, RF-008, RF-009, RF-010, RF-011, RF-012, RF-039
OBJ-002 Gestión del historial de consultas	RF-013, RF-014, RF-015, RF-016
OBJ-003 Generación de informes descargables	RF-030, RF-039
OBJ-004 Laboratorio de entrenamiento MLOps	RF-017, RF-018, RF-019, RF-020, RF-021, RF-030, RF-031, RF-032, RF-041
OBJ-005 Evaluación rigurosa de los modelos	RF-022, RF-023, RF-024, RF-025, RF-026, RF-027, RF-028, RF-029
OBJ-006 Reproducibilidad de los experimentos	RF-020
OBJ-007 Acceso seguro y personalizado al sistema	RF-001, RF-002, RF-003, RF-005, RF-006
OBJ-008 Persistencia y trazabilidad de la información	RF-013, RF-021
OBJ-009 Usabilidad e internacionalización de la interfaz	RF-004, RF-038
OBJ-010 Ejecución asíncrona de tareas de larga duración	RF-010, RF-020, RF-028, RF-036, RF-037, RF-041
OBJ-011 Administración de la plataforma	RF-006, RF-033, RF-034, RF-035, RF-040

Tabla x - Cobertura de los objetivos del sistema por los requisitos funcionales

La tabla resume la asignación de los requisitos funcionales a los objetivos del sistema que ya se detallaba en la matriz de la sección 12.1.1.7. De su lectura se desprende que la distribución es equilibrada y refleja la naturaleza de cada objetivo: los objetivos de carácter funcional, como el diagnóstico asistido (OBJ-001) o la evaluación rigurosa de los modelos (OBJ-005), concentran un número elevado de requisitos, mientras que los objetivos transversales se apoyan en un número menor de requisitos de aplicación general, como el aislamiento de datos entre usuarios (RF-005) en el caso del acceso seguro (OBJ-007) o los requisitos de la cola de trabajos (RF-036 y RF-037) en el de la ejecución asíncrona (OBJ-010). Esta distribución es coherente con el diseño del sistema: las capacidades nucleares requieren una especificación funcional más detallada, mientras que las propiedades transversales se declaran mediante requisitos de aplicación amplia.

15.1.2. Cobertura de los objetivos por los requisitos no funcionales

La segunda matriz de la sección 12.1.2.9 del capítulo 12 relaciona los requisitos no funcionales (RNF-001 a RNF-034) con los objetivos del sistema a los que aportan condiciones de calidad. El examen de esta matriz confirma que todos los objetivos con implicaciones transversales de calidad quedan cubiertos por los requisitos no funcionales correspondientes. Los objetivos de carácter puramente funcional —la gestión del historial (OBJ-002) y la generación de informes descargables (OBJ-003)— no presentan asignaciones en esta matriz, pues su consecución se materializa mediante requisitos funcionales; en cambio, el laboratorio de entrenamiento (OBJ-004) y la evaluación rigurosa de los modelos (OBJ-005) sí cuentan con requisitos de calidad asociados, porque la fiabilidad del laboratorio y la corrección de sus resultados dependen de condiciones no funcionales, y no solo de capacidades funcionales.

Objetivo	Requisitos no funcionales que aportan condiciones de calidad
OBJ-001 Diagnóstico asistido con IA explicable	RNF-019
OBJ-004 Laboratorio de entrenamiento MLOps	RNF-022
OBJ-005 Evaluación rigurosa de los modelos	RNF-033, RNF-034
OBJ-006 Reproducibilidad de los experimentos	RNF-033, RNF-034
OBJ-007 Acceso seguro y personalizado al sistema	RNF-001 a RNF-018
OBJ-008 Persistencia y trazabilidad de la información	RNF-031, RNF-032
OBJ-009 Usabilidad e internacionalización de la interfaz	RNF-023, RNF-024, RNF-025, RNF-026
OBJ-010 Ejecución asíncrona de tareas de larga duración	RNF-020, RNF-021, RNF-022, RNF-027, RNF-028, RNF-029, RNF-030
OBJ-011 Administración de la plataforma	RNF-006, RNF-009

Tabla x - Cobertura de los objetivos del sistema por los requisitos no funcionales

La tabla muestra la asignación de los requisitos no funcionales a los objetivos con implicaciones de calidad. La concentración más notable se produce en el objetivo de acceso seguro y personalizado al sistema (OBJ-007), que agrupa los requisitos de seguridad y confidencialidad (RNF-001 a RNF-018), lo que refleja la relevancia que la protección de los datos clínicos tiene en el ámbito sanitario de la plataforma. Le sigue el objetivo de ejecución asíncrona (OBJ-010), con los requisitos de robustez y rendimiento (RNF-020 a RNF-022 y RNF-027 a RNF-030), y el de reproducibilidad (OBJ-006), con los requisitos de reproducibilidad del sistema (RNF-033 y RNF-034). Esta distribución evidencia que las dimensiones de calidad del sistema se concentran, como cabía esperar, en los objetivos con mayor sensibilidad clínica y computacional.

15.2. Cobertura de los requisitos por los casos de uso y los subsistemas

La segunda dimensión de la verificación desciende un nivel en la cadena de trazabilidad: comprueba que los requisitos funcionales se materializan en casos de uso y que estos, a su vez, se agrupan en los subsistemas de análisis definidos en el capítulo 13. Esta verificación garantiza que ninguna capacidad declarada en la especificación de requisitos quede sin representación en el comportamiento observable del sistema ni en la estructura de análisis sobre la que se apoyará el diseño.

15.2.1. Correspondencia entre requisitos funcionales y casos de uso

La matriz de trazabilidad requisitos-casos de uso de la sección 12.2.4 del capítulo 12, desglosada en cinco tablas, una por módulo funcional, relaciona los cuarenta y un requisitos funcionales con los treinta y nueve casos de uso del sistema. El examen de las matrices confirma la cobertura en ambas direcciones: todos los requisitos funcionales quedan materializados en al menos un caso de uso y todos los casos de uso están justificados por el requisito funcional que los origina. Se cumple además la correspondencia biunívoca declarada en la especificación: la mayoría de los requisitos se materializan en un único caso de uso del mismo módulo, y solo los dos requisitos de carácter transversal —el aislamiento de datos entre usuarios (RF-005) y los roles y el control de acceso (RF-006)— no dan lugar a un caso de uso propio, condicionando el comportamiento del resto de las interacciones en lugar de constituir interacciones independientes.

Módulo funcional	Requisitos	Casos de uso
Gestión del acceso y de la cuenta	RF-001 a RF-006	CU-001 a CU-004
Interfaz de diagnóstico asistido	RF-007 a RF-016, RF-039	CU-005 a CU-014, CU-037
Laboratorio de experimentación MLOps	RF-017 a RF-032, RF-041	CU-015 a CU-030, CU-039
Supervisión y administración de la plataforma	RF-033 a RF-035, RF-040	CU-031 a CU-033, CU-038
Capacidades transversales de la plataforma	RF-036 a RF-038	CU-034 a CU-036

Tabla x - Correspondencia entre los módulos funcionales, los requisitos y los casos de uso

La tabla 47 resume la correspondencia entre módulos funcionales, requisitos y casos de uso que ya detallaban las cinco matrices de la sección 12.2.4. La distribución de los casos de uso entre los módulos es proporcional a la complejidad funcional de cada uno: el módulo de laboratorio MLOps, el más extenso de la plataforma, concentra diecisiete casos de uso (CU-015 a CU-030 y CU-039) que materializan los diecisiete requisitos del laboratorio (RF-017 a RF-032 y RF-041), el módulo de interfaz de diagnóstico reúne once casos de uso (CU-005 a CU-014 y CU-037), el de administración cuatro (CU-031 a CU-033 y CU-038), y los módulos de acceso y de capacidades transversales se limitan a cuatro y tres casos de uso respectivamente. Esta proporcionalidad es un indicador de la consistencia de la especificación: los ámbitos funcionales que concentran más capacidades son también los que reciben un mayor nivel de detalle en su comportamiento.

15.2.2. Correspondencia entre casos de uso y subsistemas de análisis

La descomposición en subsistemas del capítulo 13 agrupa los casos de uso en seis subsistemas de análisis, de modo que cada caso de uso pertenece exactamente a un subsistema y cada requisito funcional queda asignado al subsistema que lo materializa. La correspondencia entre los subsistemas de análisis y los módulos funcionales de requisitos es directa, como se comprobó al construir las fichas de subsistema del capítulo 13: el subsistema de acceso y gestión de cuentas (SS-001) recoge los casos de uso de autenticación (CU-001 a CU-004), el de diagnóstico asistido (SS-002) los del flujo clínico y del informe de la consulta (CU-005 a CU-010 y CU-037), el de gestión del historial (SS-003) los de recuperación y gestión de consultas (CU-011 a CU-014), el de laboratorio MLOps (SS-004) los de experimentación y de limitación de entrenamientos (CU-015 a CU-030 y CU-039), el de supervisión y administración (SS-005) los de administración y gestión de cuentas (CU-031 a CU-033 y CU-038) y el de capacidades transversales (SS-006) los de ejecución asíncrona y personalización (CU-034 a CU-036).

La verificación de esta dimensión revela además dos dependencias estructurales que condicionan el diseño y que ya se señalaron en la síntesis del capítulo 13. En primer lugar, los subsistemas SS-002, SS-003, SS-004 y SS-005 presuponen la existencia de una identidad autenticada establecida por el subsistema de acceso (SS-001), sobre la que se aplican el aislamiento de datos entre usuarios (RF-005) y el control de roles (RF-006). En segundo lugar, los subsistemas SS-002 y SS-004 delegan la ejecución de sus tareas de larga duración —los diagnósticos y los entrenamientos— en el mecanismo de cola de trabajos del subsistema de capacidades transversales (SS-006), que actúa como servicio común de ejecución asíncrona. La existencia de estas dependencias, lejos de ser una inconsistencia, refleja la cohesión del modelo: los subsistemas mantienen una alta cohesión interna y un bajo acoplamiento externo, y las únicas interacciones entre ellos son las estrictamente necesarias para el flujo funcional completo.

15.2.3. Cobertura de los requisitos no funcionales por los subsistemas

La cobertura de los requisitos no funcionales presenta una naturaleza distinta a la de los funcionales, puesto que estos requisitos no se materializan en casos de uso ni se asignan a un único subsistema, sino que condicionan de forma transversal el comportamiento de la plataforma. Aun así, es posible verificar que cada grupo de requisitos no funcionales tiene un subsistema que vela de forma principal por su cumplimiento, lo que garantiza que ninguna condición de calidad queda sin responsable en la estructura de análisis.

Grupo de requisitos no funcionales	Requisitos	Subsistema responsable principal
Seguridad de la plataforma y del acceso	RNF-001 a RNF-011	SS-001 Acceso y gestión de cuentas
Confidencialidad y protección de los datos	RNF-012 a RNF-018	SS-001 Acceso y gestión de cuentas
Rendimiento y capacidad de respuesta	RNF-019 a RNF-022	SS-002 Diagnóstico asistido
Sencillez de uso y accesibilidad	RNF-023 a RNF-026	SS-006 Capacidades transversales
Robustez y disponibilidad del servicio	RNF-027 a RNF-030	SS-006 Capacidades transversales
Persistencia y salvaguarda de los datos	RNF-031, RNF-032	SS-001 Acceso y gestión de cuentas
Reproducibilidad del sistema	RNF-033, RNF-034	SS-004 Laboratorio MLOps
Documentación y entregables	RNF-037, RNF-038	Transversal a toda la plataforma

Tabla x - Asignación de los grupos de requisitos no funcionales a los subsistemas

La tabla recoge la asignación de los grupos de requisitos no funcionales al subsistema que vela de forma principal por su cumplimiento. La asignación se ha realizado atendiendo al ámbito funcional en el que cada grupo produce su efecto: la seguridad y la confidencialidad se asocian al subsistema que establece la identidad y la sesión (SS-001), el rendimiento de la inferencia al subsistema que ejecuta los diagnósticos (SS-002), la sencillez de uso, la robustez y la disponibilidad al subsistema que provee los servicios transversales y la cola de trabajos (SS-006), y la reproducibilidad al subsistema que orquesta la experimentación (SS-004). El grupo de documentación y entregables se declara transversal, puesto que la documentación y el manual de usuario afectan al conjunto de la plataforma y no a un subsistema concreto.

15.3. Decisiones estratégicas del análisis

La verificación de consistencia realizada en las secciones anteriores confirma la integridad del modelo de análisis de vitalXAI: todos los objetivos tienen requisitos que los materializan, todos los requisitos funcionales se reflejan en casos de uso, todos los casos de uso se agrupan en subsistemas y todos los grupos de requisitos no funcionales tienen un subsistema responsable. Superada esta verificación, conviene consolidar las decisiones estratégicas que el análisis ha adoptado y que, por su alcance transversal, condicionarán las decisiones de diseño de los capítulos siguientes. Estas decisiones son el resultado de un proceso dirigido por requisitos y objetivos, en línea con el enfoque metodológico descrito en el capítulo 10, y su justificación se apoya en los artefactos producidos a lo largo de la etapa de análisis (Jacobson, Booch & Rumbaugh, 1999; Larman, 2004).

Arquitectura modular por subsistemas. El dominio se ha dividido en seis subsistemas de análisis con alta cohesión y bajo acoplamiento, de modo que la plataforma pueda extenderse en el futuro. Esta separación asegura que la evolución de un ámbito funcional —por ejemplo, la incorporación de nuevas arquitecturas de deep learning en el laboratorio de experimentación— no implique modificar la lógica de presentación, la gestión de usuarios o la supervisión administrativa. La correspondencia biunívoca entre los subsistemas y los módulos funcionales de requisitos garantiza, además, que el cambio de un subsistema afecte únicamente a los requisitos y casos de uso de su ámbito.

Aislamiento de datos y control de acceso como requisitos transversales. El aislamiento de datos entre usuarios (RF-005) y los roles y el control de acceso (RF-006) se han declarado como requisitos transversales que condicionan el comportamiento de todos los subsistemas, materializándose en el subsistema de acceso (SS-001) que establece la identidad sobre la que ambos se aplican. Esta decisión, coherente con la entidad raíz Usuario del modelo de dominio del capítulo 14, garantiza que cada usuario solo accede a sus propias consultas, sesiones y datos, y que las operaciones de administración quedan restringidas al rol correspondiente.

Ejecución asíncrona de las tareas de larga duración. Los diagnósticos, los entrenamientos y las validaciones externas se ejecutan de forma asíncrona mediante un mecanismo de cola de trabajos, desacoplado de la interfaz. Esta decisión, declarada en el objetivo OBJ-010 y en los requisitos de robustez y capacidad de respuesta (RNF-020 a RNF-022 y RNF-027 a RNF-030), preserva la capacidad de respuesta de la aplicación web: el procesamiento de una radiografía o el entrenamiento de un modelo no bloquea el hilo principal de la navegación, y el usuario continúa interactuando con la plataforma mientras el trabajo se procesa en segundo plano, siendo notificado cuando el resultado queda disponible.

Explicabilidad integrada en el flujo clínico y en el laboratorio. La explicabilidad no se ha concebido como un módulo aislado, sino como una capacidad integrada tanto en el diagnóstico asistido —cada consulta se acompaña de sus mapas de explicabilidad (Saliency Maps, SmoothGrad y Grad-CAM, o mapas de atención para arquitecturas Transformer)— como en el laboratorio de experimentación —cada modelo entrenado se somete a un análisis de explicabilidad—. Esta decisión materializa el objetivo OBJ-001 y constituye el fundamento de la confianza clínica en el sistema: el profesional no solo recibe la predicción, sino los motivos que la justifican, de modo que pueda verificar que el modelo se fija en las regiones pulmonares relevantes.

Modelo de dominio centrado en las entidades de negocio. El modelo de dominio del capítulo 14 identifica ocho clases de negocio que responden directamente a los módulos funcionales y a los casos de uso del capítulo 12, de modo que cada entidad tiene una correspondencia clara con las capacidades declaradas en la especificación. Las decisiones de modelado adoptadas —la consolidación de los parámetros del experimento interpretados por el asistente en la clase `ModeloEntrenado`, con la ruta del dataset en la sesión de experimentación, y el establecimiento de `Usuario` como entidad raíz— simplifican el modelo sin perder capacidad expresiva y garantizan la coherencia entre el comportamiento dinámico descrito en las secuencias de interacción y la estructura estática sobre la que se apoya.

Persistencia y reproducibilidad de la experimentación. El laboratorio de entrenamiento registra de forma persistente las sesiones, los modelos, los resultados y la configuración de los experimentos, de modo que el investigador pueda recuperar, comparar y reproducir sus experimentos. Esta decisión materializa los objetivos de reproducibilidad (OBJ-006) y de persistencia y trazabilidad (OBJ-008), y se apoya en los requisitos no funcionales de reproducibilidad y de integridad de la persistencia (RNF-031, RNF-033 y RNF-034), que condicionarán las decisiones de diseño de la capa de datos y del pipeline de entrenamiento.

Con estas decisiones, el análisis de vitalXAI queda cerrado de forma consistente y trazable en todas sus dimensiones. Los artefactos producidos en los capítulos sobre objetivos, requisitos, casos de uso, subsistemas y modelo de dominio, constituyen la base sobre la que se adoptarán las decisiones de diseño del sistema en los capítulos siguientes, de modo que la transición del análisis al diseño se realiza sobre un modelo verificado y sin lagunas de cobertura.

16. Pruebas

La verificación de la plataforma es una actividad transversal que acompaña a todo el ciclo de desarrollo y que permite comprobar, de forma sistemática y repetible, que el sistema construido se comporta conforme a lo especificado en el análisis. El plan de pruebas que se presenta en este capítulo recoge las verificaciones previstas para vitalXAI, de modo que se garantice el cumplimiento de los requisitos funcionales y no funcionales definidos en el capítulo 12: la corrección del comportamiento de cada componente, la interacción correcta entre los subsistemas, la protección de la plataforma frente a los vectores de ataque web más habituales, el rendimiento de las operaciones críticas y la integridad de los flujos completos de uso. Cada prueba se identifica mediante un código, describe el comportamiento que verifica, indica los subsistemas implicados y establece el criterio de éxito que debe satisfacerse para considerarse superada.

El desarrollo de vitalXAI ha seguido una metodología dirigida por pruebas (TDD, Test-Driven Development), de modo que la comprobación del comportamiento no es una actividad posterior a la implementación, sino un mecanismo de trabajo integrado en ella: cada capacidad funcional se ha desarrollado escribiendo primero la prueba que verifica su comportamiento esperado y, después, la implementación mínima que la hace superar. Esta metodología ha dado lugar a una batería de pruebas exhaustiva —más de ciento ochenta pruebas automatizadas distribuidas entre pruebas unitarias y de integración— que constituye la base sobre la que se asienta el plan de pruebas presentado a continuación. La automatización de la verificación, junto con su integración en el flujo de integración continua, garantiza que cualquier cambio posterior en el código pueda detectar regresiones de forma inmediata (Myers, Sandler & Badgett, 2011).

Las pruebas se organizan en cinco categorías, atendiendo al nivel de verificación que proporcionan y a la naturaleza de lo que comprueban: verificación de componentes individuales, verificación de los flujos entre subsistemas, pruebas de protección y control de acceso, pruebas de rendimiento y capacidad de respuesta y verificación integral de los flujos completos de uso. Las dos primeras categorías se corresponden con las pruebas unitarias y de integración ya implementadas en el proyecto; las tres restantes comprenden, junto con las verificaciones de seguridad ya automatizadas, el conjunto de pruebas de rendimiento, protección y sistema que el plan define para completar la estrategia de calidad de la plataforma. Las referencias a los ficheros de pruebas se indican en cada categoría, de modo que la correspondencia entre el plan y la implementación real de las pruebas quede documentada de forma inequívoca.

16.1. Verificación de componentes individuales

Las pruebas de esta categoría verifican el comportamiento correcto de los componentes del sistema de manera aislada, sin dependencia de la red, de la base de datos ni de servicios externos, que se sustituyen por dobles de prueba. Se centran en la lógica de negocio de mayor criticidad funcional y se organizan atendiendo a los subsistemas de análisis del capítulo 13, de modo que cada componente se verifica en el ámbito funcional en el que se utiliza. El conjunto comprende aproximadamente ciento ochenta y seis pruebas unitarias distribuidas en veinte ficheros bajo el directorio de pruebas unitarias del proyecto, y cubren la totalidad de los servicios y enrutadores del backend: la gestión de cuentas y sesiones, la validación de entradas, el motor de inferencia, la generación de explicaciones, la gestión del historial, el laboratorio de entrenamiento, la cola de trabajos, la internacionalización y la capa de acceso a datos.

16.1.1. Componentes del subsistema de acceso y gestión de cuentas

Las pruebas de este grupo verifican la autenticación y la gestión de la cuenta, así como la validación de las entradas del formulario de registro. Se comprueba que el registro crea la cuenta con la contraseña cifrada mediante un algoritmo de hash irreversible, que el inicio de sesión con credenciales correctas genera una sesión válida y que las credenciales incorrectas son rechazadas sin generar sesión. También se verifican los tokens de acceso y de refresco: su creación, su verificación, la invalidación de los tokens revocados y el mecanismo de rotación, que detecta el robo de una sesión y revoca todos los tokens asociados. La tabla 38 recoge una muestra representativa de estas pruebas.

ID	Descripción	Componente	Criterio de éxito
PU-001	Verificar que el registro de usuario crea la cuenta con la contraseña cifrada.	auth_service, auth_router	El hash almacenado difiere de la contraseña original y es verificable mediante la función de verificación.
PU-002	Verificar que el inicio de sesión con credenciales correctas genera una sesión válida.	auth_router	La sesión generada es válida y contiene el identificador del usuario.
PU-003	Verificar que el inicio de sesión con credenciales incorrectas devuelve un error.	auth_router	El sistema rechaza el acceso sin generar sesión.
PU-004	Verificar que un token de acceso caducado es rechazado.	auth_service	La verificación del token caducado devuelve sin identificar al usuario.
PU-005	Verificar que la rotación del token de refresco invalida el anterior.	auth_service	El token antiguo deja de ser válido tras la rotación y el nuevo permite identificar al usuario.

ID	Descripción	Componente	Criterio de éxito
PU-006	Verificar que la detección de robo de sesión revoca todos los tokens del usuario.	auth_service	Tras el uso de un token revocado, todos los tokens asociados quedan invalidados.
PU-007	Verificar que se rechaza un correo con formato no válido en el registro.	input_validation	El registro se rechaza indicando el campo erróneo y no se crea la cuenta.
PU-008	Verificar que se rechaza una contraseña de menos de ocho caracteres.	input_validation	El registro se rechaza indicando el campo de contraseña.
PU-009	Verificar que se rechaza un usuario duplicado.	auth_router	El registro devuelve un error de usuario existente y no crea la cuenta.

Tabla x - Verificación de componentes del subsistema de acceso y gestión de cuentas

Los ficheros de pruebas que materializan estas verificaciones son `test_auth_service.py`, `test_auth_router.py` e `test_input_validation.py`, junto con `test_database.py`, que verifica la configuración del pool de conexiones, la lectura de credenciales desde el entorno y la creación de las tablas, y `test_rate_limiting.py`, que comprueba que el proceso de inicio de sesión está limitado a cinco peticiones por minuto.

16.1.2. Componentes del subsistema de diagnóstico asistido

Las pruebas de este grupo verifican el motor de inferencia y la generación de las explicaciones, que constituyen el núcleo clínico de la plataforma. Se comprueba que los modelos convolucionales se cargan correctamente desde sus pesos entrenados, que los modelos basados en atención se cargan desde su arquitectura de Hugging Face, que las predicciones producen las clases esperadas (PNEUMONIA o NORMAL), que los tamaños de entrada se ajustan a la arquitectura seleccionada y que el nivel de confianza se acota al intervalo válido. También se verifican los componentes de generación de los mapas de explicabilidad y de los informes PDF. La tabla 39 recoge una muestra representativa de estas pruebas.

ID	Descripción	Componente	Criterio de éxito
PU-010	Verificar que un modelo convolucional se carga desde sus pesos entrenados.	ml_engine	El modelo se carga y queda disponible para la inferencia.
PU-011	Verificar que un modelo Transformer se carga desde su arquitectura preentrenada.	ml_engine	El modelo se carga con sus pesos y queda disponible para la inferencia.
PU-012	Verificar que la predicción sobre una imagen con neumonía produce la clase PNEUMONIA.	ml_engine	El modelo clasifica la imagen en la clase PNEUMONIA.
PU-013	Verificar que la predicción sobre una imagen sana produce la clase NORMAL.	ml_engine	El modelo clasifica la imagen en la clase NORMAL.
PU-014	Verificar que la confianza de la predicción se acota al intervalo 0-100.	ml_engine	La confianza devuelta nunca sale del intervalo válido.
PU-015	Verificar que se genera el mapa de explicabilidad para una arquitectura convolucional.	xai_generator	El mapa devuelto es una matriz normalizada de las dimensiones esperadas.
PU-016	Verificar que se genera el mapa de atención para una arquitectura Transformer.	xai_generator	El mapa de atención se genera y normaliza correctamente.
PU-017	Verificar que el informe PDF de una consulta se genera correctamente.	pdf_generator	La prueba devuelve la ruta del documento PDF generado.

Tabla 39 - Verificación de componentes del subsistema de diagnóstico asistido

Los ficheros de pruebas que materializan estas verificaciones son `test_ml_engine.py`, `test_inference_router.py`, `test_xai_generator.py` y `test_pdf_generator.py`. Las pruebas del motor de inferencia emplean dobles de los modelos de TensorFlow para no depender de la carga real de los pesos durante la ejecución de la batería, en línea con la estrategia de aislamiento de las pruebas unitarias.

16.1.3. Componentes del subsistema de laboratorio MLOps

Las pruebas de este grupo constituyen el bloque más extenso de la verificación unitaria, dada la complejidad funcional del laboratorio. Verifican el asistente conversacional (la creación y reutilización de las sesiones de conversación, el manejo de los errores del servicio externo de Groq y la lectura de la clave de la API desde el entorno), el lanzamiento del entrenamiento (la construcción del dataset con sus etiquetas, la selección de la arquitectura, el cálculo del progreso y la actualización del estado en la base de datos), la gestión de las sesiones (consulta, renombrado, eliminación y verificación de propiedad) y la generación del informe PDF de la sesión. La tabla 40 recoge una muestra representativa de estas pruebas.

ID	Descripción	Componente	Criterio de éxito
PU-018	Verificar que una nueva conversación con el asistente crea la sesión de chat.	trainer_router	La conversación se inicia y la sesión queda registrada.
PU-019	Verificar que una conversación existente reutiliza su histórico.	trainer_router	Los mensajes anteriores se conservan y se añade el nuevo mensaje.
PU-020	Verificar el manejo del error cuando el servicio de Groq no responde.	trainer_router	El sistema devuelve un mensaje de error y no interrumpe el resto del flujo.
PU-021	Verificar que el dataset de entrenamiento se construye con sus etiquetas.	trainer_engine	El dataframe contiene las imágenes y las etiquetas correspondientes.
PU-022	Verificar que la selección de la arquitectura produce el modelo esperado.	trainer_engine	La arquitectura solicitada devuelve el objeto de modelo correspondiente.
PU-023	Verificar que el cálculo del progreso del entrenamiento es correcto.	trainer_engine	El progreso calculado refleja la época actual respecto del total.
PU-024	Verificar que un dataset ausente marca la sesión como fallida.	trainer_engine	La sesión pasa a estado fallido sin iniciar el entrenamiento.
PU-025	Verificar que solo el propietario puede consultar una sesión.	trainer_router, mlops_engine	El acceso a una sesión ajena devuelve un error de autorización.
PU-026	Verificar que solo el propietario puede eliminar una sesión.	trainer_router	La eliminación de una sesión ajena devuelve un error de autorización.
PU-027	Verificar que el informe PDF de una sesión se genera correctamente.	trainer_router	La petición de informe de una sesión válida devuelve el documento PDF.

Tabla x - Verificación de componentes del subsistema de laboratorio MLOps

Los ficheros de pruebas que materializan estas verificaciones son `test_trainer_router.py`, `test_trainer_engine.py` y `test_mlops_engine.py`. Este último incluye también las verificaciones de la lógica de selección de la carpeta del dataset y del aislamiento de la propiedad de las sesiones, que se comprueba en la práctica totalidad de las operaciones del laboratorio.

16.1.4. Componentes de los subsistemas de historial, administración y transversales

El grupo de historial verifica la consulta del listado de consultas, la visualización del detalle, el renombrado y la eliminación, comprobando en todos los casos que el sistema verifica la propiedad de la consulta y rechaza el acceso a consultas ajenas. El grupo de administración verifica el control de acceso basado en el rol de administrador para el listado de usuarios, la consulta de las consultas de un usuario y su detalle. Finalmente, el grupo transversal verifica la cola de trabajos (la consulta del estado, la cancelación de un trabajo pendiente y la gestión de la cancelación de un trabajo en ejecución según su interrumpibilidad), el worker de la cola (la reclamación de trabajos, la transición de estados y el manejo de cargas) y la internacionalización (los cuatro idiomas soportados, la selección del idioma desde la cookie y la recuperación de textos).

ID	Descripción	Componente	Criterio de éxito
PU-028	Verificar que la consulta del historial devuelve solo las consultas del usuario.	history_router	El listado contiene únicamente las consultas propias.
PU-029	Verificar que el acceso al detalle de una consulta ajena es denegado.	history_router	El sistema devuelve un error de autorización sin mostrar la información.
PU-030	Verificar que el renombrado de una consulta ajena es denegado.	history_router	La operación devuelve un error de autorización y no modifica la consulta.
PU-031	Verificar que el listado de usuarios requiere el rol de administrador.	admin_router	Un usuario sin rol de administración recibe un error 403.
PU-032	Verificar que la consulta de las consultas de un usuario requiere el rol de administrador.	admin_router	Un usuario sin rol de administración recibe un error 403.
PU-033	Verificar que se cancela un trabajo pendiente de la cola.	queue_router	El trabajo pendiente se cancela y deja de estar en la cola.
PU-034	Verificar que la cancelación de un trabajo en ejecución se gestiona según su interrumpibilidad.	queue_worker	El sistema interrumpe el trabajo cuando resulta técnicamente posible y lo marca como cancelado; si no es interrumpible, informa de la limitación y no lo modifica.
PU-035	Verificar que el worker rellama y procesa un trabajo de la cola.	queue_worker	El trabajo pasa a estado en ejecución y, al finalizar, ha completado con su resultado.

ID	Descripción	Componente	Criterio de éxito
PU-036	Verificar la selección del idioma desde la cookie del navegador.	lang	La cookie selecciona el idioma correspondiente (es, en, zh o hi).
PU-037	Validar la recuperación de literales localizados con mecanismo de respaldo al castellano.	lang	Los textos se devuelven en la lengua solicitada o, en su defecto, en el idioma base.

Tabla x - Verificación de componentes de los subsistemas de historial, administración y transversales

Los ficheros de pruebas que materializan estas verificaciones son `test_history_router.py`, `test_admin_router.py`, `test_queue_router.py`, `test_queue_worker.py` y `test_lang.py`. Con esta cobertura, la totalidad de los subsistemas de análisis del capítulo 13 dispone de verificaciones unitarias sobre sus componentes de mayor criticidad.

16.2. Verificación de los flujos entre subsistemas

Las pruebas de esta categoría verifican el comportamiento del sistema cuando varios de sus subsistemas interactúan entre sí para completar un flujo funcional, superando el nivel de aislamiento de las pruebas unitarias. A diferencia de estas, que verifican cada componente por separado, las pruebas de integración ejercitan la colaboración entre las capas del sistema: la API, la capa de negocio y la base de datos, que se sustituye por una instancia en memoria durante la ejecución de las pruebas. El conjunto comprende las pruebas de integración implementadas bajo el directorio de pruebas de integración del proyecto, que cubren los flujos críticos de autenticación y de acceso, y se completan en el plan con las verificaciones de los flujos clínicos y de laboratorio previstas.

Las pruebas de integración implementadas verifican el flujo completo de acceso a la plataforma: el registro de un nuevo usuario, el inicio de sesión con las credenciales registradas, el acceso al panel y la verificación de que un usuario no autenticado es redirigido al punto de entrada. También se verifica que un usuario recién registrado no dispone de consultas en su historial. Estas pruebas confirman, de extremo a extremo entre la interfaz de la API y la base de datos, el comportamiento especificado en los casos de uso CU-001, CU-002 y CU-005. La tabla 42 recoge estas pruebas y las ampliaciones previstas.

ID	Descripción	Componentes implicados	Criterio de éxito
PI-001	Verificar el flujo completo de registro, inicio de sesión y acceso al panel.	SS-001, persistencia	El usuario se registra, inicia sesión con sus credenciales y accede al panel de diagnóstico.
PI-002	Verificar que un usuario no autenticado es redirigido al punto de entrada.	SS-001	El acceso a un recurso privado sin sesión redirige al inicio de sesión.
PI-003	Verificar que un usuario recién registrado no tiene consultas en el historial.	SS-001, SS-003	El historial del nuevo usuario se muestra vacío.
PI-004	Verificar el flujo completo de un diagnóstico: subida, solicitud, encolado y resultado.	SS-002, SS-006, persistencia	La radiografía se sube, el diagnóstico se encola y el resultado queda registrado y accesible.
PI-005	Verificar el aislamiento de datos entre dos usuarios.	SS-001, SS-003, SS-004	Un usuario no puede acceder a las consultas ni a las sesiones del otro, obteniendo un error de autorización.

ID	Descripción	Componentes implicados	Criterio de éxito
PI-006	Verificar que el administrador puede supervisar las consultas de un usuario.	SS-001, SS-005	El administrador consulta el detalle de una consulta de otro usuario sin restricciones de propiedad.
PI-007	Verificar el flujo de lanzamiento de un experimento desde el asistente hasta la cola.	SS-004, SS-006, persistencia	La conversación completa los parámetros y el experimento se encola y ejecuta en segundo plano.

Tabla x - Verificación de los flujos entre subsistemas

Las pruebas implementadas se encuentran en el fichero `test_auth_flow.py` del directorio de pruebas de integración. Las ampliaciones previstas (PI-004 a PI-007) cubren los flujos de extremo a extremo de mayor valor funcional de la plataforma: el diagnóstico asistido, el aislamiento de datos, la supervisión administrativa y el laboratorio de entrenamiento. Estas ampliaciones se ejecutan con la base de datos sustituida por una instancia en memoria y con los servicios externos —el modelo de Groq y los modelos de inferencia— simulados, de modo que la verificación se centra en la colaboración entre los subsistemas sin depender del entorno de producción.

16.3. Pruebas de protección y control de acceso

Las pruebas de esta categoría verifican que la plataforma aplica correctamente los mecanismos de protección especificados en los requisitos no funcionales de seguridad del capítulo 12 (RNF-001 a RNF-011). Se centran en los vectores de ataque más habituales sobre aplicaciones web autenticadas y en las cabeceras de seguridad que el sistema debe emitir en sus respuestas. Aunque el proyecto distingue la verificación de estos mecanismos como un nivel propio dentro del plan, las pruebas que los materializan están implementadas y se ejecutan como parte de la batería de pruebas del proyecto. La tabla 43 recoge una muestra representativa de estas pruebas.

ID	Descripción	Componente	Criterio de éxito
PS-001	Verificar que una petición POST sin token CSRF es rechazada.	csrf_middleware	El sistema devuelve un error 403 y no procesa la petición.
PS-002	Verificar que una petición POST con token CSRF válido se procesa.	csrf_middleware	La petición se procesa correctamente.
PS-003	Verificar que una petición POST con token CSRF erróneo es rechazada.	csrf_middleware	El sistema devuelve un error 403 y rechaza la petición.
PS-004	Verificar que las peticiones GET están exentas de la protección CSRF.	csrf_middleware	Las peticiones GET se procesan sin exigir token CSRF.
PS-005	Verificar que las respuestas incluyen la cabecera Content-Security-Policy.	security_headers	Todas las respuestas incluyen la cabecera con la política configurada.
PS-006	Verificar que las respuestas incluyen la cabecera Strict-Transport-Security.	security_headers	La cabecera HSTS se incluye en todas las respuestas.
PS-007	Verificar que las respuestas incluyen la cabecera X-Content-Type-Options.	security_headers	La cabecera se incluye para evitar la interpretación indebida de los tipos MIME.
PS-008	Verificar que el inicio de sesión está limitado en frecuencia.	rate_limiting	La sexta petición de acceso en el mismo minuto es rechazada.
PS-009	Verificar que el acceso con un token de refresco revocado es rechazado.	auth_service	El token revocado no permite renovar la sesión.

Tabla x - Pruebas de protección y control de acceso

Los ficheros de pruebas que materializan estas verificaciones son `test_csrf_middleware.py`, `test_security_headers.py`, `test_rate_limiting.py` y las pruebas de gestión de tokens de `test_auth_service.py`. El plan prevé además una prueba de auditoría de la persistencia que inspeccione directamente las tablas de la base de datos para confirmar que el 100% de las contraseñas se almacena mediante hashes irreversibles y que ninguna entrada contiene texto plano, en línea con el requisito RNF-001 y con el registro de la actividad administrativa exigido por RNF-006.

16.4. Pruebas de rendimiento y capacidad de respuesta

Las pruebas de esta categoría verifican que la plataforma satisface los requisitos no funcionales de rendimiento y capacidad de respuesta definidos en el capítulo 12: el tiempo de respuesta de la inferencia (RNF-019), la ejecución sin bloqueo de la interfaz durante las tareas de larga duración (RNF-020) y la capacidad de acceso concurrente (RNF-021). A diferencia de las categorías anteriores, cuya implementación forma parte de la batería de pruebas automatizadas, estas verificaciones se definen en el plan como pruebas de rendimiento a ejecutar sobre el entorno de despliegue, de modo que sus criterios de éxito se miden sobre el sistema real y no sobre dobles de prueba. La tabla 44 recoge estas pruebas.

ID	Descripción	Componente	Criterio de éxito
PR-001	Verificar que la primera inferencia sobre una radiografía se completa en un tiempo razonable.	SS-002	La primera consulta carga el modelo y produce el resultado en un tiempo que no hace incómodo el uso de la herramienta.
PR-002	Verificar que las inferencias posteriores son casi instantáneas.	SS-002	Las consultas siguientes sobre el mismo modelo se completan de forma sensiblemente más rápida que la primera, al reutilizar los pesos ya cargados.
PR-003	Verificar que la interfaz permanece operativa durante una tarea de larga duración.	SS-004, SS-006	Durante un entrenamiento o un análisis de explicabilidad, la plataforma continúa respondiendo a las peticiones del usuario sin bloquearse.
PR-004	Verificar que el sistema soporta la carga de varios usuarios concurrentes.	Todos	Con un número razonable de usuarios concurrentes, el tiempo de respuesta no se degrada de forma significativa.
PR-005	Verificar que el pool de conexiones gestiona el acceso concurrente a la base de datos.	Persistencia	Las peticiones concurrentes se atienden sin errores de conexión ni contención excesiva.

Tabla x - Pruebas de rendimiento y capacidad de respuesta

Las pruebas PR-001 y PR-002 se asocian directamente al requisito RNF-019, que exige que el sistema gestione la carga de los modelos de forma eficiente; la prueba PR-003 al requisito RNF-020, que exige que las tareas de larga duración se ejecuten de forma asíncrona sin bloquear la interfaz; y las pruebas PR-004 y PR-005 al requisito RNF-021, que exige la gestión del acceso concurrente mediante el pool de conexiones. Los umbrales concretos de tiempo para las pruebas PR-001 a PR-004 se fijan tras una primera campaña de medición sobre el entorno de despliegue, de modo que los criterios reflejen la infraestructura real de la plataforma.

16.5. Verificación integral de los flujos completos de uso

Las pruebas de esta categoría completan el plan de pruebas con la verificación integral de los flujos de uso de la plataforma, ejercitando la interacción del usuario con la interfaz a través de todos los subsistemas implicados, de manera análoga al uso real del sistema. Se definen en el plan como pruebas de sistema, dado que requieren el entorno de ejecución completo —la interfaz, la API, la capa de negocio, la base de datos y la cola de trabajos—, y se ejecutarán de forma manual o semiautomatizada sobre el despliegue de la plataforma. La tabla 45 recoge estas pruebas.

ID	Descripción de la prueba	Criterio de éxito
PE-001	Verificar el flujo clínico completo: subida de una radiografía, selección del modelo, solicitud del diagnóstico, visualización del resultado, de los mapas de explicabilidad y generación del informe PDF.	El flujo completo se realiza sin errores y el profesional obtiene el diagnóstico, su confianza, los mapas de explicabilidad y el informe de la consulta.
PE-002	Verificar la gestión del historial: consulta del listado, detalle de una consulta, renombrado y eliminación.	Todas las operaciones del historial se ejecutan correctamente y las consultas se recuperan tras cada operación.
PE-003	Verificar el flujo del laboratorio: conversación con el asistente, lanzamiento del experimento, consulta de las sesiones y de los resultados, y generación del informe PDF.	El experimento se configura y lanza mediante el asistente, se ejecuta en segundo plano y sus resultados e informe se obtienen sin errores.
PE-004	Verificar la ejecución asíncrona: consulta del estado de la cola y cancelación de un trabajo de la cola (pendiente o en ejecución cuando resulta interrumpible).	El usuario conoce el estado de sus trabajos y puede cancelar un trabajo pendiente o, cuando es posible, interrumpir uno en ejecución.
PE-005	Verificar la supervisión administrativa: listado de usuarios, consulta de las consultas de un usuario y su detalle.	El administrador supervisa la actividad de un usuario concreto desde el panel de administración.
PE-006	Verificar las capacidades transversales: cambio de idioma y del tema visual de la interfaz.	El idioma y el tema se aplican en toda la interfaz sin interrumpir la navegación.

Tabla x - Verificación integral de los flujos completos de uso

El conjunto de las cinco categorías de pruebas presentadas en este capítulo constituye la estrategia de calidad de vitalXAI. Las pruebas unitarias y de integración, ya implementadas y automatizadas, proporcionan una verificación continua y repetible de la lógica de negocio y de los flujos críticos; las pruebas de protección verifican los mecanismos de seguridad especificados en el análisis; y las pruebas de rendimiento y de sistema completan la verificación sobre el entorno real de despliegue. Esta estrategia garantiza que el sistema construido satisface los requisitos funcionales y no funcionales del capítulo 12 y que las evoluciones futuras de la plataforma podrán incorporarse con un riesgo de regresión controlado y medible.

ANEXO III – PLANTILLA DOCUMENTO DE DISEÑO

17. Diseño y arquitectura de vitalXAI

El presente capítulo inicia la parte de diseño de vitalXAI y transforma las decisiones funcionales establecidas durante el análisis en una organización técnica concreta. Mientras que los capítulos anteriores describen qué debe hacer la plataforma, qué actores intervienen y qué requisitos debe satisfacer, el diseño determina cómo se estructuran los componentes que permiten ofrecer esas capacidades. Esta transición se realiza manteniendo la trazabilidad con los objetivos, requisitos, casos de uso y subsistemas definidos en los capítulos 11 a 14.

El diseño se apoya en una aplicación web de servidor desarrollada con Python y FastAPI. La solución proporciona al usuario una interfaz accesible desde el navegador, expone los servicios funcionales mediante rutas HTTP, persiste la información operativa en MySQL y delega los trabajos de mayor duración en un worker interno de ejecución asíncrona. Sobre esta estructura se integran el motor de inferencia de imágenes, los módulos de explicabilidad, el laboratorio de experimentación MLOps, el asistente conversacional conectado con Groq y los generadores de informes PDF.

La organización técnica no reproduce de forma literal la distribución de los subsistemas de análisis. Estos subsistemas continúan siendo la referencia funcional para garantizar la trazabilidad, pero en el diseño se agrupan los elementos según sus responsabilidades de ejecución: presentación, entrada HTTP, lógica de aplicación y aprendizaje automático, persistencia y procesamiento asíncrono. Esta separación permite que la interfaz no tenga que conocer los detalles del motor de inferencia o de la base de datos, y que las tareas computacionalmente costosas puedan continuar sin mantener bloqueada la petición que las originó (Larman, 2004).

La arquitectura descrita en este capítulo corresponde a la implementación actual del sistema. Por este motivo, se documentan FastAPI, las plantillas Jinja2, los scripts JavaScript, MySQL, el pool de conexiones, la cola persistida en la base de datos y el worker basado en ``asyncio``, sin sustituirlos por tecnologías previstas en versiones anteriores de la documentación o por componentes que no forman parte del código ejecutable. La correspondencia entre esta solución y los subsistemas de análisis se desarrolla en los apartados posteriores del documento de diseño.

17.1. Organización arquitectónica de la solución

La solución adopta una arquitectura cliente-servidor organizada en capas lógicas y servicios internos especializados. El cliente es el navegador del usuario, que recibe las páginas HTML renderizadas mediante Jinja2 y ejecuta los recursos JavaScript y CSS de la interfaz. El servidor concentra la lógica de negocio, la validación de las peticiones, la gestión de la identidad, el acceso a los datos y la coordinación de los trabajos de inteligencia artificial. La base de datos MySQL proporciona la persistencia estructurada, mientras que la cola de trabajos y el worker interno desacoplan las operaciones de larga duración del ciclo de petición HTTP.

Esta decisión responde a dos características esenciales de vitalXAI. La primera es que la complejidad computacional debe permanecer en el servidor: el navegador envía una imagen, una configuración o una solicitud de consulta, pero no carga los modelos de aprendizaje profundo ni ejecuta los entrenamientos. La segunda es que algunas operaciones no pueden resolverse de manera fiable dentro del tiempo de una petición ordinaria. La inferencia, la generación de mapas de explicabilidad, el entrenamiento y la validación externa pueden requerir un tiempo considerable; por ello, se registran como trabajos en `job_queue` y se procesan posteriormente mediante el worker de la aplicación.

17.1.1. Capa de presentación y acceso desde el navegador

La capa de presentación está formada por las plantillas HTML del directorio `templates`, los recursos estáticos del directorio `static` y el navegador del usuario. La aplicación no utiliza una SPA independiente: las páginas se generan en el servidor mediante Jinja2 y se sirven como respuestas HTML desde las rutas correspondientes. Entre las vistas principales se encuentran las páginas de inicio de sesión y registro, el panel de diagnóstico, el laboratorio de entrenamiento y la interfaz de administración.

Los scripts JavaScript asociados a las páginas complementan este renderizado con interacciones asíncronas. Por medio de `fetch`, el navegador puede enviar formularios, cargar imágenes, consultar estados de trabajos, recuperar resultados y actualizar partes de la interfaz sin tener que reconstruir toda la página en cada operación. Los recursos `dashboard.js`, `training.js`, `admin.js` e `i18n.js` concentran la lógica de interacción propia de las vistas, mientras que los estilos y las imágenes se sirven como recursos estáticos.

La capa de presentación no constituye una frontera de seguridad suficiente por sí misma. Su función es mostrar la información y recoger las acciones del usuario, pero la autenticación, la autorización, la validación definitiva de los datos y el aislamiento entre usuarios se vuelven a comprobar en el servidor. Esta decisión evita que una modificación de las peticiones desde el navegador permita acceder a consultas, sesiones de entrenamiento o funciones administrativas que no correspondan al usuario autenticado.

17.1.2. Capa HTTP y coordinación de los subsistemas funcionales

FastAPI constituye la entrada principal al servidor. La instancia se crea en `main.py`, donde se registra el ciclo de vida de la aplicación, se montan los recursos estáticos y se incorporan los routers funcionales. La organización por routers mantiene separadas las responsabilidades de acceso y evita concentrar todos los endpoints en un único módulo. La aplicación incorpora los routers de autenticación, diagnóstico, historial, cola de trabajos, laboratorio MLOps y administración.

Cada router recibe la petición HTTP, valida sus parámetros, obtiene la identidad del usuario cuando es necesario y delega la operación en el componente especializado. Así, `auth.py` coordina el registro, el inicio y el cierre de sesión; `inference.py` inicia las consultas de diagnóstico; `history.py` gestiona el historial; `trainer.py` coordina el laboratorio MLOps; `queue.py` expone el estado y la cancelación de trabajos; y `admin.py` restringe las operaciones de supervisión al rol de administrador. Esta distribución mantiene la relación entre las rutas de la API y los casos de uso sin convertir cada router en el responsable de la persistencia o del cálculo científico.

La instancia de FastAPI también concentra los elementos transversales que deben aplicarse a todas o a gran parte de las peticiones. `CSRFMiddleware` comprueba el token de protección en las operaciones que modifican el estado, `SecurityHeadersMiddleware` añade las cabeceras de seguridad configuradas y el limitador basado en `slowapi` controla la frecuencia de las operaciones sensibles, especialmente el acceso. Estos mecanismos se registran en el punto de creación de la aplicación para que no dependan de que cada router los aplique manualmente.

17.1.3. Servicios de aplicación y motores de inteligencia artificial

La lógica que no pertenece directamente al protocolo HTTP se organiza en el directorio `services`. Esta capa contiene servicios de aplicación y motores especializados que son invocados por los routers o por el worker de la cola. La separación es especialmente importante en `vitalXAI` porque las operaciones de una consulta clínica no se limitan a producir una etiqueta: también pueden generar un mapa de explicabilidad, crear un informe PDF y persistir el resultado asociado al usuario.

El servicio de autenticación encapsula la creación y comprobación de credenciales, el hash de contraseñas mediante `bcrypt`, la generación de tokens y la gestión de su renovación y revocación. `ml_engine.py` prepara la imagen, carga las arquitecturas disponibles y ejecuta la inferencia mediante TensorFlow/Keras o mediante modelos de Transformers. Las arquitecturas basadas en atención se obtienen a través de la configuración de Hugging Face utilizada por el proyecto. `xai_generator.py` produce los mapas de explicabilidad correspondientes a la arquitectura seleccionada, y `pdf_generator.py` transforma el resultado de una consulta en un informe descargable.

El laboratorio de experimentación se divide en servicios que reflejan sus distintas responsabilidades. `chatbot_service.py` encapsula la comunicación con la API de Groq y permite que el asistente interprete los parámetros del experimento; `trainer_engine.py` prepara los datos y ejecuta el entrenamiento; `mlops_engine.py` gestiona las sesiones, los modelos y los resultados; y `pdf_generator_mlops.py` genera los informes de las sesiones de experimentación. Esta división permite que el router del laboratorio coordine el flujo sin contener la implementación completa del entrenamiento o de la comunicación con el servicio externo.

Los servicios externos se tratan como fronteras de integración. La API de Groq se utiliza únicamente desde el servicio conversacional y requiere la variable de entorno `GROQ_API_KEY`. Los modelos de Transformers se resuelven desde Hugging Face cuando corresponde y se cargan en el motor de aprendizaje automático. Las respuestas o fallos de estos servicios no se exponen directamente al usuario como excepciones internas: la capa de aplicación los convierte en estados o mensajes que la interfaz puede mostrar y que el worker puede registrar como trabajos fallidos.

17.1.4. Persistencia relacional y ejecución asíncrona

La persistencia se implementa mediante MySQL y el conector oficial `mysql-connector-python`. El módulo `database.py` proporciona un `MySQLConnectionPool` reutilizable, obtiene los parámetros de conexión desde variables de entorno y expone la función que entrega conexiones a los routers y servicios que necesitan consultar o modificar datos. La inicialización de la aplicación verifica la existencia de las tablas principales: `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`.

Las relaciones de propiedad se mantienen en la base de datos mediante las claves ajenas que vinculan los trabajos y las consultas con los usuarios. De este modo, la identidad obtenida durante la autenticación puede utilizarse para filtrar el historial, las sesiones de entrenamiento y los trabajos de cada cuenta. El diseño también permite persistir el resultado de una operación larga después de que la petición HTTP inicial haya finalizado: el cliente conserva el identificador del trabajo y consulta posteriormente su estado y su resultado.

La ejecución asíncrona no depende de un broker externo. `queue.py` ofrece las operaciones de consulta y cancelación, mientras que `queue_worker.py` implementa el consumidor interno. Durante el ciclo de vida de FastAPI, `start_worker` restablece los trabajos que quedaron en ejecución tras una interrupción y crea una tarea `asyncio` que inspecciona periódicamente la tabla `job_queue`. El worker selecciona el siguiente trabajo pendiente, intenta reclamarlo de forma condicional y ejecuta el diagnóstico, el entrenamiento o la validación externa según el tipo registrado.

El procesamiento pesado se ejecuta mediante `run_in_executor`, evitando que el cálculo bloquee el bucle de eventos de la aplicación. Una vez terminada la operación, el worker almacena el resultado y marca el trabajo como completado; si se produce una excepción, registra el error y lo marca como fallido. Esta máquina de estados —`queued`, `running`, `completed` y `failed`— permite que la interfaz informe del progreso sin mantener abierta la petición que creó el trabajo. La cancelación se limita a los trabajos que aún están pendientes, lo que evita interrumpir de forma inconsistente una operación que ya ha comenzado.

17.1.5. Vista global de la arquitectura

La siguiente ilustración resume las relaciones entre los elementos principales de la solución. El navegador consume las páginas y recursos de presentación y se comunica con FastAPI mediante peticiones HTTP. FastAPI dirige cada operación hacia el router y el servicio correspondiente, utiliza MySQL para la información persistente y registra en `job_queue` las operaciones que deben ejecutarse fuera del ciclo inmediato de la petición. El worker recupera esos trabajos y utiliza los motores de inferencia, explicabilidad, entrenamiento y generación de informes antes de persistir sus resultados.

La arquitectura resultante conserva una separación clara entre la interacción del usuario, la coordinación HTTP, la lógica especializada, la persistencia y la ejecución de trabajos. La aplicación sigue siendo desplegable como una única solución, pero sus responsabilidades internas están suficientemente delimitadas para que una modificación en la interfaz no obligue a reescribir el motor de inferencia, que una nueva arquitectura de modelo se incorpore dentro de los servicios de aprendizaje automático y que los cambios en la presentación no alteren el mecanismo de cola. Esta organización materializa las dependencias identificadas en el análisis: los subsistemas de diagnóstico y laboratorio dependen de la identidad y la persistencia, y ambos utilizan el mecanismo transversal de trabajos asíncronos para ejecutar las operaciones de larga duración.

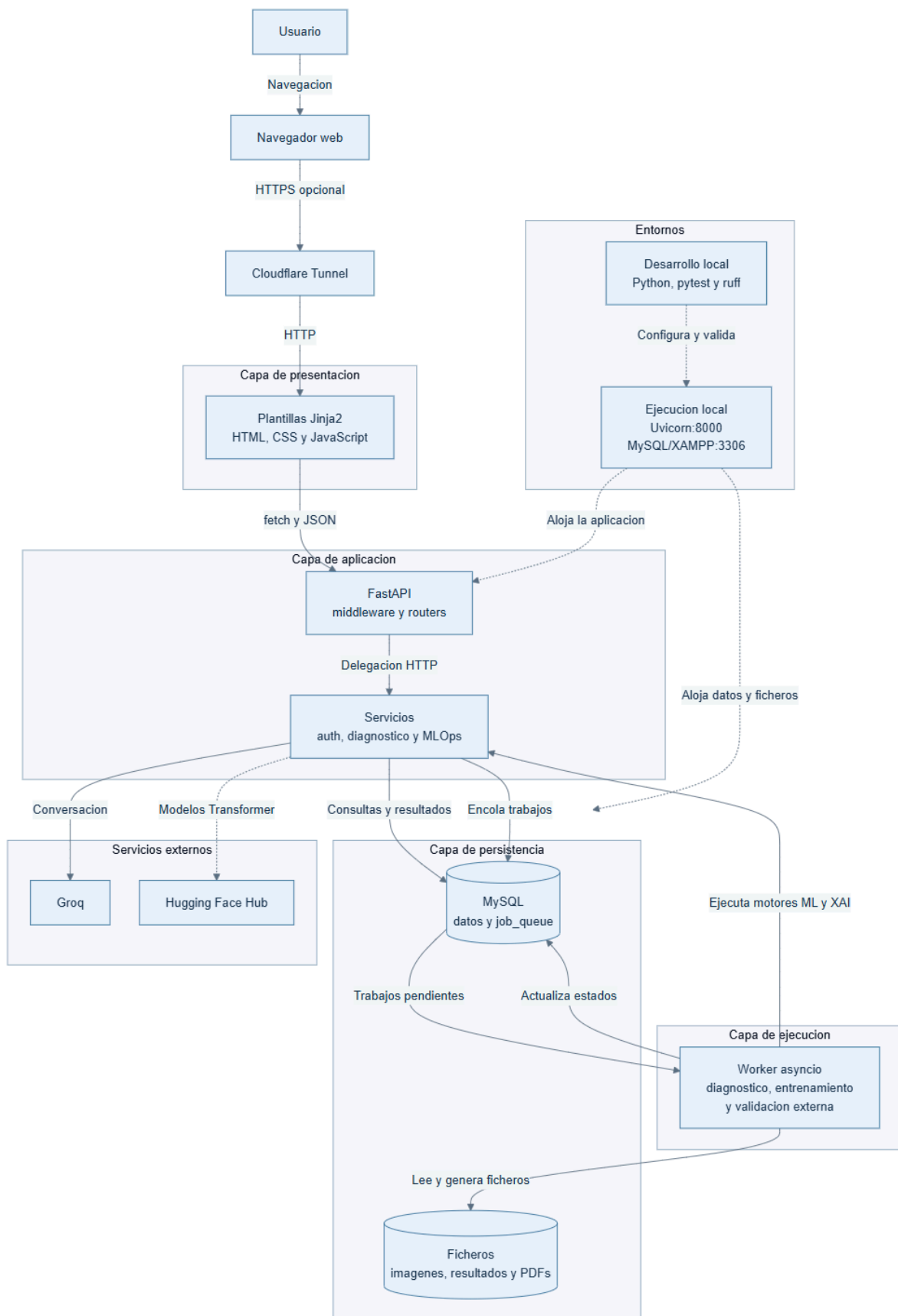


Ilustración x - Organización arquitectónica de vitalXAI

17.2. Condiciones de calidad, normativa y restricciones del diseño

La arquitectura definida en el apartado anterior no constituye una elección aislada de tecnologías. Cada una de sus decisiones responde a los requisitos no funcionales, a las normas aplicables al dominio sanitario y a las limitaciones del entorno en el que se construye y ejecuta vitalXAI. En consecuencia, este apartado relaciona las condiciones de calidad del capítulo 12 con las decisiones técnicas que las materializan, distingue los estándares que deben observarse durante el desarrollo y delimita las restricciones que condicionan la solución.

17.2.1. Requisitos no funcionales y respuesta arquitectónica

Los requisitos no funcionales se han agrupado en el análisis en siete dimensiones: seguridad y acceso, confidencialidad de los datos, rendimiento, sencillez de uso, robustez, reproducibilidad y documentación. En la fase de diseño no todos ellos se traducen en una clase o un módulo independiente. Algunos se materializan mediante middleware, configuración o separación de responsabilidades; otros corresponden a procedimientos del entorno de ejecución y a los documentos que acompañan al software. La siguiente tabla resume la relación entre cada grupo y la respuesta adoptada en la implementación actual.

Grupo de requisitos	Requisitos relacionados	Decisión de diseño y mecanismo aplicado
Seguridad del acceso	RNF-001 a RNF-011	Hash de contraseñas con bcrypt, tokens de acceso y refresco, cookies HttpOnly con SameSite=Lax, middleware CSRF, limitación de acceso mediante slowapi y cabeceras de seguridad. Las operaciones administrativas se protegen mediante el rol del usuario.
Confidencialidad de los datos	RNF-012 a RNF-018	Separación de los datos por user_id, comprobación de propiedad en las rutas privadas, uso de datasets anonimizados y exposición del servicio mediante HTTPS cuando se accede desde el exterior.
Rendimiento y respuesta	RNF-019 a RNF-022	Reutilización de modelos cargados, pool de conexiones MySQL, ejecución de trabajos mediante job_queue y worker asyncio, y delegación del cálculo pesado en run_in_executor.
Usabilidad y accesibilidad	RNF-023 a RNF-026	Interfaz HTML/Jinja2 con JavaScript vanilla, asistencia conversacional para configurar experimentos, servicio de traducciones, cambio de tema visual y diseño adaptable de las vistas.
Robustez y disponibilidad	RNF-027 a RNF-032	Estados persistentes de los trabajos, registro de errores, recuperación de trabajos que quedaron en ejecución al reiniciar el servidor y claves ajenas en MySQL. Las copias de seguridad dependen del procedimiento operativo del entorno MySQL.

Grupo de requisitos	Requisitos relacionados	Decisión de diseño y mecanismo aplicado
Reproducibilidad y sostenibilidad	RNF-033 a RNF-034	Versiones fijadas en requirements.txt, entorno virtual de Python, separación entre routers, servicios y persistencia, y ejecución de comprobaciones automatizadas mediante pytest y ruff.
Documentación y entregables	RNF-037 a RNF-038	Memoria estructurada por fases, guía de despliegue y documentación de configuración, instalación y uso de la plataforma.

Tabla x - Relación entre los requisitos no funcionales y las decisiones de diseño

17.2.1.1. Seguridad, identidad y control del acceso

La identidad del usuario constituye la primera condición de seguridad de la plataforma. El servicio de autenticación no almacena contraseñas originales, sino valores generados mediante bcrypt, y la tabla users conserva únicamente el campo password_hash. La sesión se articula mediante un token de acceso y un token de refresco. Ambos se entregan mediante cookies protegidas frente al acceso de JavaScript mediante el atributo HttpOnly, mientras que el token de refresco limita además su ruta de envío al endpoint de renovación. La rotación y revocación de los tokens se apoya en la tabla refresh_tokens, que conserva su hash, su fecha de expiración y su estado de revocación.

El diseño incorpora una protección adicional frente a peticiones CSRF. El middleware genera una cookie csrf_token accesible desde el código de la interfaz y exige que las peticiones que modifican el estado incluyan el mismo valor en la cabecera X-CSRF-Token. Las operaciones de lectura permanecen fuera de esta comprobación y la ruta de inicio de sesión se trata como excepción específica para permitir el acceso inicial. La comparación se realiza en el servidor antes de entregar la petición al router correspondiente.

La limitación de frecuencia se aplica especialmente al inicio de sesión, que se encuentra limitado a cinco peticiones por minuto mediante slowapi. Esta medida reduce la viabilidad de ataques automatizados de fuerza bruta sin tener que incorporar la lógica de limitación dentro del router de autenticación. A este mecanismo se suman las cabeceras emitidas por SecurityHeadersMiddleware, entre ellas Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security y Referrer-Policy.

El control de roles se aplica en el router de administración. El usuario administrador puede consultar el listado de usuarios y supervisar las consultas de una cuenta, mientras que un usuario ordinario no puede invocar esas operaciones. El aislamiento de datos se comprueba de forma independiente en cada operación que recupera o modifica una consulta, una sesión de entrenamiento o un trabajo. Esta comprobación no se delega en la interfaz, ya que el navegador puede ser manipulado por el usuario que realiza la petición.

Debe distinguirse entre el control de acceso administrativo y la auditoría formal de la actividad administrativa. La implementación actual restringe las rutas de administración mediante el rol, pero no introduce una tabla independiente de trazas con el detalle histórico de cada operación. Por ello, RNF-006 se mantiene como condición de calidad y de gobierno del sistema, mientras que el mecanismo de autorización sí forma parte de la arquitectura implementada. Esta precisión evita presentar como existente un componente de auditoría que no aparece en el código actual.

17.2.1.2. Confidencialidad y tratamiento de la información

La aplicación trabaja con cuentas de usuario, radiografías y resultados de diagnóstico. El diseño relacional vincula las consultas, los trabajos y los entrenamientos con el identificador del usuario propietario, y las rutas utilizan esa relación para limitar los resultados. Esta regla se aplica tanto al historial clínico como a las sesiones del laboratorio MLOps y a los trabajos almacenados en `job_queue`.

El cumplimiento de los requisitos de privacidad no se resuelve únicamente con una configuración de software. Los datasets de entrenamiento y validación deben estar anonimizados antes de incorporarse al proyecto, y el personal que sube una imagen debe garantizar que no contiene información identificable del paciente. La plataforma almacena rutas de ficheros y resultados asociados al usuario profesional, pero no incorpora un módulo de identificación del paciente ni debe utilizarse para introducir datos personales que no sean necesarios para la consulta.

La confidencialidad durante el tránsito depende del entorno de exposición. En el uso local, Uvicorn atiende la aplicación en el puerto 8000; cuando se publica el sistema mediante el túnel configurado para la demostración, el acceso exterior se realiza mediante HTTPS. Las cookies de sesión incorporan actualmente `HttpOnly` y `SameSite=Lax`; la protección del canal debe mantenerse como condición obligatoria del entorno de despliegue para evitar que las credenciales y los datos viajen sin cifrar.

17.2.1.3. Rendimiento, concurrencia y ejecución sin bloqueo

El requisito de rendimiento más relevante no consiste únicamente en reducir el tiempo de una función concreta, sino en impedir que una operación de aprendizaje automático monopolice el ciclo de petición. Por este motivo, el diagnóstico, el entrenamiento y la validación externa se registran primero como trabajos en la tabla `job_queue`. La petición HTTP puede devolver el identificador y el estado inicial del trabajo, mientras que el worker continúa el procesamiento de forma independiente.

El worker se inicia durante el ciclo de vida de FastAPI y consulta periódicamente los trabajos pendientes. Para evitar que el cálculo de TensorFlow, la generación de mapas XAI o la ejecución del entrenamiento bloqueen el bucle de eventos, el trabajo se envía a `run_in_executor`. El resultado o el mensaje de error se almacena después en MySQL. Así, el usuario puede consultar la cola mientras el procesamiento continúa y la plataforma puede atender otras peticiones.

La concurrencia de acceso a la persistencia se aborda mediante `MySQLConnectionPool`. El tamaño del pool se configura con `DB_POOL_SIZE`, mientras que los restantes datos de conexión se obtienen mediante variables de entorno. Esta decisión evita crear una conexión nueva de forma indiscriminada para cada operación y permite controlar el número de conexiones simultáneas en función del entorno disponible. La concurrencia no elimina la necesidad de validar la propiedad de cada registro: el rendimiento y el aislamiento de datos son condiciones simultáneas.

17.2.1.4. Usabilidad, internacionalización y presentación

La elección de plantillas Jinja2 y JavaScript vanilla reduce la complejidad del cliente y evita introducir una cadena de compilación independiente. Las páginas se entregan directamente desde el servidor y los scripts añaden únicamente las interacciones necesarias para cargar imágenes, consultar estados, mostrar resultados y actualizar el laboratorio. La arquitectura facilita que la interfaz pueda funcionar desde un navegador sin instalar software adicional.

El servicio lang.py centraliza la recuperación de textos traducidos y permite seleccionar el idioma mediante la cookie correspondiente. Esta capacidad se utiliza desde las vistas de autenticación, el panel de diagnóstico y el laboratorio. El cambio de tema visual se realiza en el cliente y se conserva como preferencia de la interfaz. La presentación de los resultados se organiza alrededor de las necesidades del usuario: diagnóstico y confianza, mapas de explicabilidad, histórico de consultas, progreso de trabajos y resultados del laboratorio.

17.2.1.5. Robustez, persistencia y recuperación

La tabla job_queue actúa como soporte persistente de la ejecución asíncrona. A diferencia de una cola exclusivamente residente en memoria, permite conservar el estado de un trabajo y recuperarlo después de reiniciar la aplicación. Al iniciar el worker, _reset_running_jobs() devuelve al estado queued los trabajos que quedaron en running, de modo que no permanezcan bloqueados indefinidamente. Cada ejecución puede terminar en completed o failed, y en el segundo caso se conserva un mensaje de error limitado en longitud.

Las claves ajenas de las tablas consultations, training_jobs, job_queue y refresh_tokens apuntan a users. Esta relación refuerza la integridad del modelo y permite eliminar los datos dependientes de un usuario cuando la operación está configurada con ON DELETE CASCADE. Las escrituras se confirman explícitamente mediante commit y las conexiones se cierran al finalizar cada operación.

La copia de seguridad periódica y la restauración de MySQL pertenecen al entorno operativo, no a un servicio interno de la aplicación. El diseño identifica esta dependencia para que el despliegue no confunda la persistencia local con una estrategia de recuperación completa. La guía de despliegue debe conservar, por tanto, las instrucciones necesarias para iniciar MySQL y verificar la disponibilidad de la base de datos antes de ejecutar la plataforma.

17.2.2. Estándares y normas aplicables

El sistema se desarrolla bajo normas legales y técnicas que afectan a distintos niveles de la solución. En materia de privacidad se aplica el Reglamento General de Protección de Datos, Reglamento (UE) 2016/679, junto con la Ley Orgánica 3/2018 de protección de datos personales y garantía de los derechos digitales. Estas normas condicionan la minimización de la información recogida, la anonimización de las imágenes, la asociación de los datos a una cuenta y la forma en que se documenta el tratamiento de la información sanitaria (Parlamento Europeo y Consejo de la Unión Europea, 2016; España, 2018).

Para la protección de las comunicaciones se adopta HTTPS apoyado en TLS. El túnel utilizado para exponer la aplicación debe proporcionar el canal cifrado entre el navegador y el entorno local, y las cabeceras de seguridad del servidor complementan esa protección. La existencia de HTTPS no sustituye a la autenticación, a la autorización ni a la protección CSRF; son mecanismos que operan en capas distintas y deben mantenerse conjuntamente (IETF, 2018).

El código Python sigue las convenciones de PEP 8. La aplicación utiliza ruff como comprobación automatizada de estilo y mantiene un fichero requirements.txt con versiones fijadas de sus dependencias principales. La documentación de la memoria y los manuales se elaboran conforme a la normativa del Trabajo Fin de Grado de la Escuela Politécnica Superior de la Universidad Pablo de Olavide, que constituye el marco formal de los entregables del proyecto (van Rossum, Warsaw, & Coghlan, 2001; Universidad Pablo de Olavide, 2014).

La siguiente tabla resume el ámbito de aplicación de las normas anteriores.

Estándar / Norma	Ámbito de aplicación
RGPD (Reglamento UE 2016/679)	Protección de datos y tratamiento de información biomédica.
LOPDGDD (Ley Orgánica 3/2018)	Adaptación del RGPD al ordenamiento jurídico español.
HTTPS / TLS	Protocolo seguro de comunicación entre cliente y servidor.
OWASP Top 10	Seguridad de las aplicaciones web.
PEP 8	Estándar de estilo y nomenclatura del código Python.

Tabla x - Normas y estándares aplicables al diseño

17.2.3. Restricciones técnicas y límites del entorno

La primera restricción técnica es el ecosistema de aprendizaje automático. La aplicación se ejecuta sobre Python 3.11 y depende de TensorFlow, Keras, OpenCV, scikit-learn y Transformers. Esta combinación condiciona la versión del intérprete, el sistema operativo y la disponibilidad de recursos de memoria. Los modelos convolucionales y Transformer no se consideran componentes ligeros del servidor: sus pesos deben cargarse en memoria y los entrenamientos pueden requerir una GPU compatible con el entorno de TensorFlow.

La segunda restricción es la capacidad de cómputo local. El entrenamiento y la validación de las arquitecturas demandan más recursos que una consulta web ordinaria, por lo que el entorno de desarrollo debe disponer de hardware suficiente, especialmente durante la ejecución del laboratorio MLOps. La interfaz y los routers no deben asumir que el resultado está disponible de forma inmediata; esta condición es precisamente la que justifica el diseño basado en trabajos persistidos y worker interno.

La tercera restricción es la disponibilidad de MySQL. La aplicación depende de una instancia accesible mediante los parámetros DB_HOST, DB_USER, DB_PASSWORD, DB_NAME y DB_POOL_SIZE. En el entorno de demostración, MySQL se ejecuta mediante XAMPP. Si el servicio no está iniciado o las credenciales no son válidas, la inicialización de la base de datos no puede completarse, aunque el ciclo de vida de FastAPI informa del problema y continúa hasta iniciar el worker.

La cuarta restricción procede de los servicios externos. El asistente conversacional requiere GROQ_API_KEY y conectividad con la API de Groq. La carga de los modelos Transformer puede requerir acceso a Hugging Face, salvo que los modelos ya estén disponibles localmente. Estas dependencias introducen límites de disponibilidad y latencia que el sistema debe tratar como fallos de integración, no como errores de lógica interna. Por este motivo, las pruebas unitarias sustituyen los servicios externos por mocks y la ejecución de producción debe conservar las variables de entorno fuera del código fuente.

Finalmente, la exposición remota mediante el túnel no convierte el entorno local en una infraestructura distribuida. La aplicación, el worker, los ficheros y la base de datos continúan alojados en la máquina de ejecución. El túnel únicamente proporciona el canal de acceso desde el navegador externo. Esta decisión simplifica la demostración y mantiene bajo control las versiones del entorno, pero limita la escalabilidad, la tolerancia a fallos hardware y la disponibilidad de la plataforma si la máquina anfitriona se apaga o pierde conectividad.

En conjunto, estas restricciones explican por qué el diseño prioriza una aplicación servidor sencilla, una persistencia relacional accesible, un mecanismo de trabajos basado en MySQL y una separación clara entre la petición web y la computación científica. No se introducen colas, bases de datos o servicios de despliegue adicionales que no sean necesarios para la implementación actual. La arquitectura conserva así una relación directa entre los requisitos del análisis, las condiciones de calidad, el entorno disponible y los componentes que realmente ejecuta vitalXAI.

17.3. Subsistemas de Diseño

Los subsistemas de diseño son la traducción técnica de los seis subsistemas de análisis definidos en el capítulo 13. La correspondencia se mantiene de forma directa: cada subsistema SS conserva sus responsabilidades funcionales y recibe un identificador SD que agrupa los componentes de implementación que las realizan. Esta decisión permite recorrer la trazabilidad desde los requisitos y casos de uso hasta los routers, servicios, mecanismos de persistencia y procesos que intervienen en la ejecución.

La correspondencia no implica que cada subsistema de diseño sea una aplicación independiente. Todos forman parte de la misma aplicación FastAPI y comparten la base de datos, la autenticación, la capa de presentación y el worker de trabajos. La separación se realiza por responsabilidades y límites funcionales, no mediante despliegues separados. De esta manera, el código mantiene una organización modular sin introducir una complejidad distribuida que no exige el entorno actual.

Subsistema de diseño	Subsistema de análisis	Responsabilidad principal	Componentes de implementación
SD-001	SS-001	Gestionar la identidad, las sesiones y el acceso a la plataforma.	routers/auth.py, services/auth_service.py, users, refresh_tokens y middleware de seguridad.
SD-002	SS-002	Recibir una radiografía, solicitar el diagnóstico y producir sus artefactos.	routers/inference.py, services/ml_engine.py, services/xai_generator.py, services/pdf_generator.py y job_queue.
SD-003	SS-003	Consultar y administrar el historial de diagnósticos del usuario.	routers/history.py, tabla consultations, ficheros de resultados y dashboard.js.
SD-004	SS-004	Configurar, ejecutar y consultar experimentos MLOps.	routers/trainer.py, chatbot_service.py, trainer_engine.py, mlops_engine.py y pdf_generator_mlops.py.
SD-005	SS-005	Supervisar usuarios, consultas y sesiones desde el ámbito administrativo.	routers/admin.py, consultas MySQL y funciones de consulta de mlops_engine.py.
SD-006	SS-006	Coordinar trabajos asíncronos y capacidades transversales de la interfaz.	routers/queue.py, services/queue_worker.py, services/lang.py, middleware global y recursos estáticos.

Tabla x - Correspondencia entre subsistemas de análisis y subsistemas de diseño

17.3.1. SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa el subsistema de análisis SS-001 y cubre el registro, el inicio y el cierre de sesión, la renovación de credenciales y la identificación del usuario en las áreas privadas. Su responsabilidad no se limita a validar el formulario de acceso: establece la identidad que utilizarán los demás subsistemas para aplicar el aislamiento de datos y las comprobaciones de rol.

El punto de entrada del subsistema es `routers/auth.py`. El router sirve las páginas de inicio de sesión y registro y procesa los formularios recibidos desde la interfaz. Antes de crear una cuenta valida el formato del nombre de usuario, la longitud mínima de la contraseña y la presencia de los datos personales básicos. A continuación, consulta MySQL para comprobar que el usuario no existe, genera el hash de la contraseña y almacena la cuenta en la tabla `users`. La contraseña original no se transmite a ningún componente de persistencia después de completar el proceso de hash.

La lógica criptográfica y la gestión del ciclo de vida de los tokens se concentra en `services/auth_service.py`. El token de acceso se firma con JWT y contiene el identificador del usuario y su fecha de expiración. El token de refresco se genera como un valor aleatorio, pero en la base de datos solo se conserva su hash SHA-256 junto con el usuario, la fecha de expiración y el indicador de revocación. Esta diferencia permite renovar la sesión sin guardar una credencial reutilizable en texto plano.

El cierre de sesión revoca el token de refresco y elimina las cookies de la respuesta. La rotación genera un token nuevo y revoca el anterior; si se detecta el uso de un token revocado fuera del periodo de gracia configurado, el servicio invalida todos los tokens del usuario. El mecanismo está diseñado para tolerar peticiones concurrentes de renovación durante un intervalo corto sin interpretar automáticamente cada repetición como un robo de sesión.

El subsistema se apoya también en los mecanismos transversales registrados en `main.py`. `CSRFMiddleware` protege las peticiones que modifican el estado, `SecurityHeadersMiddleware` refuerza las respuestas dirigidas al navegador y `slowapi` limita el endpoint de inicio de sesión. La sesión se almacena en cookies `HttpOnly` y con alcance `SameSite=Lax`, mientras que el resto de routers utiliza `get_user_id_from_token()` para obtener la identidad antes de realizar operaciones privadas. SD-001 es, por tanto, un subsistema transversal: SD-002, SD-003, SD-004, SD-005 y SD-006 dependen de la identidad que establece.

Desde el punto de vista de la trazabilidad, SD-001 realiza principalmente los casos de uso CU-001 a CU-004. CU-001 se materializa en la combinación de validación de los campos, consulta de unicidad, generación del hash e inserción en `users`; CU-002 añade la verificación de la contraseña y la creación de las dos cookies de sesión; CU-003 revoca el token de refresco y elimina las cookies; y CU-004 utiliza el servicio de idioma para adaptar la presentación. Los casos de uso de otros subsistemas no necesitan conocer cómo se firma un token: solo reciben la identidad resuelta o una respuesta de acceso no autorizado.

La configuración sensible permanece fuera del código. `JWT_SECRET_KEY`, `JWT_ACCESS_EXPIRE_MINUTES`, `JWT_REFRESH_EXPIRE_DAYS` y `REFRESH_ROTATION_GRACE_SECONDS` se obtienen del entorno, al igual que los parámetros de MySQL. Esta decisión evita que la clave criptográfica o la duración de las credenciales queden ligadas al repositorio. El servicio emite además una advertencia cuando se utiliza la clave de desarrollo por defecto, lo que convierte una configuración insegura en una condición visible durante el arranque.

El subsistema establece contratos sencillos con sus consumidores. Un componente que necesita identificar al usuario entrega el token de acceso a `get_user_id_from_token()` y recibe un identificador entero o `None`; no interpreta directamente el payload JWT ni duplica la lógica de expiración. Los routers, a su vez, traducen la ausencia de identidad en una respuesta HTTP 401. La comprobación del rol se realiza únicamente en las operaciones administrativas, por lo que la existencia de una sesión válida no equivale a disponer de privilegios elevados.

La principal decisión de SD-001 es conservar el token de acceso en la cookie y el token de refresco en una cookie con ruta restringida, en lugar de enviar las credenciales en el cuerpo de cada petición. Esto simplifica la interacción de las plantillas y los scripts del navegador y reduce la exposición accidental de los tokens en el almacenamiento de la interfaz. La protección efectiva depende además de que el entorno de publicación utilice HTTPS y de que la clave secreta de producción esté configurada correctamente.

17.3.2. SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 corresponde a SS-002 y concentra el flujo clínico de la plataforma. Su función es recibir una imagen válida, asociarla al usuario autenticado, registrar una solicitud de diagnóstico, ejecutar la inferencia con la arquitectura seleccionada y conservar los artefactos que permiten consultar y justificar el resultado.

`routers/inference.py` actúa como fachada HTTP del subsistema. La ruta `/predict` comprueba la existencia de una sesión válida y valida el tipo MIME de la imagen. Solo se aceptan imágenes JPEG y PNG, y se aplica un límite de 10 MB antes de escribir el fichero en `static/uploads`. Esta validación se realiza antes de encolar el trabajo, de modo que una petición inválida no ocupa una posición en la cola ni genera un registro de procesamiento incompleto.

Una vez almacenada temporalmente la imagen, el router crea un registro en `job_queue` con el tipo `diagnosis`, el identificador del usuario y un payload JSON que incluye la arquitectura, la ruta de la imagen y el idioma. La respuesta HTTP informa de que el trabajo ha quedado encolado, devuelve su identificador y calcula su posición. El router no carga el modelo ni ejecuta la predicción, ya que esas operaciones pertenecen al worker y a la capa de servicios.

El worker dirige el trabajo de diagnóstico hacia `services/ml_engine.py`, que prepara la imagen, carga o reutiliza el modelo y produce la etiqueta y la confianza de la predicción. La implementación combina arquitecturas convolucionales gestionadas mediante TensorFlow/Keras y arquitecturas basadas en Transformers. Después de la inferencia, `services/xai_generator.py` genera el mapa de explicabilidad adecuado para el modelo utilizado. Finalmente, `services/pdf_generator.py` genera el informe descargable con la imagen original, el resultado, la confianza y el artefacto XAI.

El resultado se registra en la tabla `consultations`, donde quedan asociados el usuario, el modelo utilizado, las rutas de los ficheros, la etiqueta de predicción, la confianza y el informe PDF. Esta persistencia permite que el historial y la interfaz administrativa consulten el resultado sin repetir la inferencia. Si cualquier fase falla, el worker marca el trabajo como `failed` y conserva el mensaje de error en `job_queue`, evitando que la petición original quede abierta indefinidamente.

El flujo interno de SD-002 se divide en cinco pasos. Primero, el router autentica al usuario y valida el fichero recibido. Segundo, conserva la imagen original en el área de cargas y crea el payload serializable del trabajo. Tercero, el worker reclama el registro y ejecuta la inferencia. Cuarto, genera los artefactos derivados, que son el mapa XAI y el informe PDF. Quinto, persiste la consulta y actualiza el estado de la cola. La división evita que una ruta HTTP tenga que mantener referencias abiertas a un fichero o a un modelo durante toda la operación.

El payload de un trabajo de diagnóstico contiene únicamente la información necesaria para repetir el procesamiento: `model_name`, `image_path` y `lang`, además del identificador del usuario almacenado en la propia fila de la cola. No se serializa el objeto del modelo ni la imagen completa dentro de MySQL. Los datos voluminosos se mantienen en el sistema de ficheros y la base de datos conserva sus rutas, de manera que la cola sigue siendo ligera y puede consultarse con rapidez.

La separación entre `ml_engine`, `xai_generator` y `pdf_generator` también delimita los fallos. Una arquitectura no reconocida debe fallar antes de producir un resultado ambiguo; una imagen no válida no debe alcanzar el motor; y un error al construir el informe no debe transformarse en una consulta marcada como completada. El worker agrupa el flujo completo dentro del procesamiento del trabajo y utiliza la transición a `failed` como contrato común para que la interfaz pueda mostrar que la operación terminó con error.

La reutilización de modelos cargados responde al RNF-019. El coste de la primera consulta puede incluir la lectura de los pesos y la construcción del modelo, mientras que las consultas posteriores pueden aprovechar el modelo disponible en memoria. Esta optimización pertenece al motor de aprendizaje automático y no al router, por lo que la capa HTTP permanece independiente de si la arquitectura seleccionada es convolucional o Transformer.

SD-002 también actúa como frontera de validación de ficheros. El tipo MIME y el tamaño se comprueban antes de escribir en disco, pero la validez científica de la imagen y la interpretación clínica del resultado siguen siendo responsabilidades distintas. El sistema ofrece una clasificación asistida, una confianza y explicaciones visuales; no sustituye la valoración del profesional sanitario. Esta separación de responsabilidades es importante para no convertir el diseño técnico del subsistema en una afirmación clínica que la implementación no puede garantizar.

17.3.3. SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa SS-003 y proporciona la recuperación y gestión de las consultas ya realizadas. Su entidad persistente principal es `consultations`, que almacena las rutas de los artefactos, la arquitectura utilizada, la predicción, la confianza, el nombre mostrado al usuario y la fecha de la operación.

`routers/history.py` expone la consulta del listado y las operaciones de actualización y eliminación. La consulta del historial filtra siempre por el `user_id` obtenido de la cookie de acceso y ordena los resultados por fecha descendente. Antes de modificar o eliminar una consulta, `_check_consultation_ownership()` comprueba que el registro existe y que pertenece al usuario solicitante. La misma función contempla la excepción controlada del administrador, que puede supervisar consultas desde SD-005.

El subsistema no vuelve a ejecutar el modelo para mostrar una consulta anterior. Recupera desde MySQL los metadatos y devuelve las rutas de la imagen original, el mapa XAI y el informe PDF para que la interfaz pueda presentarlos o descargarlos. Esta decisión reduce el coste de acceso al historial y conserva la separación entre una consulta nueva y la visualización de un resultado ya calculado.

La interfaz del historial se integra en el panel de diagnóstico y utiliza JavaScript para mostrar los resultados, cambiar el nombre visible de una consulta y solicitar su eliminación. El router devuelve códigos diferenciados para la falta de autenticación, la consulta inexistente y la ausencia de permisos. Esta distinción permite que la capa de presentación informe al usuario de la situación sin exponer detalles internos de la base de datos.

La relación entre SD-003 y SD-002 es de productor-consumidor de resultados. SD-002 crea el registro de `consultations` cuando finaliza un diagnóstico; SD-003 no modifica la predicción ni sus artefactos, sino que proporciona operaciones de consulta y organización. El nombre mostrado al usuario se actualiza sobre `patient_name`, que funciona como etiqueta de organización de la consulta y no como identificación clínica del paciente. Esta decisión mantiene separados los metadatos de presentación de los datos técnicos de la inferencia.

El listado devuelve los campos necesarios para construir las tarjetas o filas del historial: identificador, fecha, modelo, rutas de imágenes, etiqueta, confianza, nombre y PDF. Las fechas se convierten a una representación textual antes de formar la respuesta JSON, de modo que el navegador no necesita conocer el tipo específico de fecha de MySQL. Los artefactos se sirven mediante las rutas estáticas configuradas en la aplicación y no mediante una nueva ruta de cálculo.

La eliminación se implementa actualmente como una eliminación física de la fila de consulta. Por esa razón, el diseño no debe describir SD-003 como un subsistema de borrado lógico ni afirmar que conserva automáticamente una auditoría de cada eliminación. La integridad de la operación depende de que se compruebe la propiedad antes de ejecutar el `DELETE`, y de que la transacción se confirme solo después de que la actualización haya sido aceptada por MySQL. La decisión se documenta aquí porque afecta a la interpretación del historial y a futuras ampliaciones del modelo de persistencia.

El administrador constituye una excepción controlada al aislamiento ordinario. `_check_consultation_ownership()` permite continuar cuando el usuario tiene rol admin, pero el acceso administrativo se comprueba en el servidor y no se infiere de un parámetro enviado por el navegador. En todos los demás casos, una consulta de otro usuario produce una respuesta 403. Así, SD-003 mantiene una política única de propiedad y permite que SD-005 la utilice sin duplicar una segunda consulta de autorización.

17.3.4. SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 corresponde a SS-004 y constituye el bloque de mayor complejidad funcional. Gestiona la configuración conversacional del experimento, la selección del dataset, el entrenamiento de modelos, la ejecución de los análisis de explicabilidad, la comparación estadística, la validación externa, la consulta de resultados y la generación de informes.

`routers/trainer.py` se ha diseñado como una fachada ligera. Sus rutas comprueban la autenticación, validan que la sesión de experimentación pertenece al usuario y delegan las operaciones especializadas en `mlops_engine.py` o en los servicios correspondientes. Entre sus operaciones se encuentran la conversación con el asistente, la exploración de carpetas, el inicio de entrenamientos, la consulta de logs, la recuperación de sesiones, la consulta de resultados, el ranking, el recálculo de la comparativa, la validación externa y la generación de informes.

La configuración conversacional se encapsula en `services/chatbot_service.py`, que se comunica con Groq y devuelve una configuración estructurada cuando se han proporcionado los parámetros necesarios. El router no incorpora la lógica del modelo de lenguaje: únicamente valida la sesión y entrega el mensaje al servicio. Esta separación permite que la comunicación con Groq se trate como una frontera externa y que los errores del proveedor no se confundan con errores de persistencia o de entrenamiento.

`services/mlops_engine.py` organiza las sesiones del laboratorio y resuelve la lectura y escritura de los resultados en `training_results`. `trainer_engine.py` contiene la lógica de preparación del dataset, construcción de modelos, entrenamiento, cálculo del progreso y actualización del estado. Los resultados generados por el pipeline incluyen las métricas de rendimiento, los artefactos de explicabilidad, los datos de comparación y la información necesaria para la validación externa. Los informes finales se generan mediante `pdf_generator_mlops.py`.

El inicio de un entrenamiento crea la sesión, comprueba que la ruta del dataset existe y registra en `job_queue` un trabajo de tipo training con los modelos e hiperparámetros solicitados. La validación externa sigue el mismo mecanismo con un trabajo de tipo `external_validation`. El worker ejecuta ambos tipos fuera del ciclo de petición y actualiza el estado de la cola al finalizar. Esta decisión permite que el laboratorio conserve una interfaz interactiva aunque el entrenamiento tarde mucho más que una petición HTTP normal.

La propiedad de las sesiones es una regla central del subsistema. Antes de mostrar resultados, eliminar o renombrar una sesión, el router invoca la comprobación de propiedad de `mlops_engine`; el administrador puede acceder a estas operaciones de supervisión cuando la ruta lo permite. Así, la configuración, los resultados y los artefactos de una experimentación permanecen asociados a la cuenta que la creó.

La configuración de una sesión contiene los parámetros que necesita el pipeline: ruta del dataset, modelos seleccionados, número de épocas, tamaño de lote y tasa de aprendizaje. El asistente conversacional permite completar esa información mediante mensajes, pero la ejecución no comienza hasta que el router dispone de una configuración completa y válida. La conversación y el entrenamiento son responsabilidades distintas: Groq ayuda a interpretar la intención del usuario, mientras que mlops_engine y trainer_engine validan y ejecutan la configuración.

El lanzamiento de un entrenamiento sigue una secuencia concreta. La ruta comprueba la autenticación, verifica que existe la ruta del dataset, crea la sesión de entrenamiento y registra en job_queue un payload con la lista de modelos y los hiperparámetros. El identificador de la sesión se conserva tanto en la fila del trabajo como en el directorio de resultados. Esta doble referencia permite consultar el estado desde MySQL y recuperar los artefactos experimentales desde el sistema de ficheros.

La información de una sesión no se reduce a un único valor de rendimiento. El pipeline genera resultados por arquitectura y por pliegue, métricas agregadas, artefactos de explicabilidad, configuraciones y datos de comparación estadística. get_model_results_data() y get_session_ranking_data() transforman esos ficheros en estructuras que el frontend puede mostrar. Esta decisión evita trasladar al navegador la responsabilidad de recorrer todos los artefactos o de repetir los cálculos del ranking.

La comparación estadística tiene un flujo diferenciado del entrenamiento. La ruta de comparación comprueba la propiedad y programa run_statistical_comparison mediante BackgroundTasks, mientras que el entrenamiento y la validación externa se insertan explícitamente en job_queue para ser procesados por queue_worker.py. Esta diferencia es una característica de la implementación actual: ambos mecanismos permiten devolver pronto la respuesta, pero no deben documentarse como si fueran el mismo canal de ejecución. El estado del recálculo se consulta mediante una ruta específica de la sesión.

La validación externa también mantiene la separación entre entrenamiento y evaluación. La ruta verifica que el dataset externo existe y que el usuario tiene acceso a la sesión; después encola un trabajo con el identificador de la sesión y la ruta del dataset. El worker ejecuta la validación sobre los modelos disponibles sin modificar la configuración original del entrenamiento. Los resultados se almacenan como artefactos de la sesión y quedan disponibles para su consulta posterior.

El informe de MLOps se genera en una fase posterior, cuando la sesión dispone de los datos necesarios. pdf_generator_mlops.py recibe la información preparada por el motor y produce un documento descargable, evitando que el router conozca el formato interno del informe. De este modo, el laboratorio puede cambiar la estructura visual del PDF sin cambiar el contrato de las rutas que consultan los resultados.

Desde la perspectiva de los errores, SD-004 debe diferenciar una configuración inválida, un dataset inexistente, una sesión ajena y un fallo del entrenamiento. Las dos primeras situaciones se rechazan antes de crear o encolar el trabajo; la tercera devuelve 403; y la cuarta se registra durante el procesamiento y cambia el estado del trabajo. Esta clasificación permite que la interfaz muestre una corrección de datos, una falta de permisos o un fallo computacional sin confundirlos.

17.3.5. SD-005: Supervisión y administración

El subsistema SD-005 deriva de SS-005 y reúne las operaciones que solo puede realizar un usuario con rol de administrador. Su objetivo es proporcionar una visión supervisada del uso de la plataforma sin mezclar las operaciones administrativas con las rutas que utiliza un usuario ordinario.

`routers/admin.py` centraliza la comprobación de permisos mediante `_require_admin()`. La función obtiene primero la identidad desde el token de acceso y consulta el rol en la tabla `users`. El router diferencia entre una petición no autenticada, una petición autenticada sin permisos y una petición administrativa válida, devolviendo códigos HTTP diferentes en cada caso.

La consulta del listado administrativo combina la información de `users` con el número de consultas de `consultations` y obtiene también el número de sesiones de laboratorio a partir de la información disponible en `training_results`. Las rutas específicas permiten consultar las consultas de un usuario y visualizar el detalle de una consulta concreta. En la segunda operación se incluyen también las sesiones de entrenamiento recuperadas por `mlops_engine`.

SD-005 no duplica la lógica de propiedad del historial ni la del laboratorio. Su responsabilidad es establecer la autorización administrativa y coordinar las consultas globales; la lectura de los datos sigue utilizando las mismas tablas y servicios que el resto de la aplicación. Esta decisión reduce la duplicación y mantiene una frontera clara entre el control de permisos y la representación de la información.

El flujo administrativo comienza siempre por `_require_admin()`. La función identifica al solicitante, consulta su rol y devuelve un resultado ternario en la práctica: no hay identidad, existe identidad sin privilegios o existe identidad con rol de administrador. Cada endpoint interpreta ese resultado antes de abrir la consulta global correspondiente. Esta secuencia evita que una ruta administrativa ejecute una consulta de usuarios o de consultas antes de haber comprobado el permiso.

La vista global combina dos fuentes de información. Los usuarios y el número de diagnósticos se obtienen mediante una consulta agregada sobre `users` y `consultations`; el número de sesiones del laboratorio se calcula inspeccionando las configuraciones disponibles en `training_results`. Esta decisión refleja la persistencia híbrida actual de la plataforma: la información relacional se consulta en MySQL, mientras que parte de los artefactos de las sesiones se conserva en el sistema de ficheros.

Las rutas de supervisión mantienen la granularidad necesaria para la interfaz administrativa. Una ruta devuelve el listado de usuarios, otra recupera las consultas y sesiones asociadas a un usuario concreto y una tercera muestra el detalle de una consulta. El administrador puede acceder a la información de otros usuarios por su rol, pero el sistema no convierte esas rutas en operaciones de modificación general: su responsabilidad actual es de consulta y supervisión.

La protección del subsistema se completa con las medidas de SD-001 y SD-006. El router administrativo no implementa una segunda autenticación ni una segunda gestión de tokens; reutiliza la identidad de la sesión y consulta el rol en la base de datos. Tampoco escribe una traza de auditoría independiente en cada operación, por lo que el requisito de auditoría administrativa debe considerarse una condición pendiente del modelo operativo y no una tabla o servicio que ya exista.

17.3.6. SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa SS-006 y reúne las capacidades que sirven de apoyo a varios flujos funcionales. Su núcleo es la cola persistente de trabajos, utilizada por los diagnósticos, los entrenamientos y las validaciones externas. También incluye la consulta de idioma, la personalización de la interfaz y los mecanismos transversales que se aplican desde la configuración principal de FastAPI.

`routers/queue.py` expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente. La ruta de estado filtra por el usuario autenticado, devuelve los trabajos recientes y calcula la posición de los que permanecen en `queued`. También interpreta el payload según el tipo de trabajo para mostrar el nombre del modelo o el identificador de la sesión sin enviar al navegador el contenido interno completo del payload.

La cancelación se realiza mediante una actualización condicional: solo puede cambiar a `cancelled` un trabajo que pertenezca al usuario y que continúe en estado `queued`. Si el worker ya lo ha reclamado, la operación no lo interrumpe de forma abrupta. Esta restricción evita inconsistencias entre el estado persistido y el cálculo que ya se está ejecutando.

`services/queue_worker.py` es el consumidor de la cola. Al iniciar la aplicación restablece los trabajos que quedaron en estado `running`, selecciona el siguiente trabajo pendiente y lo reclama mediante una actualización que exige que siga en estado `queued`. Después ejecuta el flujo correspondiente en el executor y marca el resultado como completado o fallido. El worker comparte la persistencia con los routers, pero no comparte con ellos el ciclo de petición: esta es la frontera que permite mantener disponible la interfaz durante las tareas de larga duración.

La parte de internacionalización se concentra en `services/lang.py`, que obtiene el idioma seleccionado y devuelve los textos de la interfaz y de los mensajes de respuesta. Los recursos JavaScript aplican el idioma y el tema visual en el navegador. Estas capacidades no necesitan un servicio externo ni una nueva base de datos, por lo que permanecen como mecanismos transversales de presentación y aplicación.

La tabla de responsabilidades concluye la descomposición del diseño. SD-001 establece la identidad; SD-002 produce el diagnóstico; SD-003 conserva y recupera sus consultas; SD-004 gestiona la experimentación; SD-005 supervisa la plataforma; y SD-006 proporciona la ejecución asíncrona y las capacidades comunes. La dependencia entre ellos es deliberada y coincide con la verificación de consistencia del capítulo 15: los subsistemas funcionales necesitan la identidad de SD-001, los trabajos de SD-002 y SD-004 pasan por SD-006, y SD-003 y SD-005 acceden a la persistencia respetando reglas diferentes de propiedad y administración.

La cola aplica además una política de ordenación que prioriza los trabajos de diagnóstico frente a los de entrenamiento. En `_next_job()`, los trabajos de tipo `training` reciben una prioridad posterior, mientras que los diagnósticos y las validaciones externas pueden ser seleccionados antes cuando comparten el estado `queued`. Esta decisión intenta proteger la respuesta del flujo clínico sin eliminar la posibilidad de que los entrenamientos se ejecuten. La posición que observa el usuario se calcula desde el router de cola y se adapta al tipo de trabajo.

El mecanismo de reclamación evita que dos iteraciones del worker procesen el mismo registro. Primero se selecciona una fila pendiente y después `_claim_job()` ejecuta una actualización condicionada a que el estado siga siendo `queued`. Solo si la actualización afecta a una fila se considera que el worker ha adquirido el trabajo. Aunque la configuración actual inicia un worker interno, esta comprobación introduce una garantía de consistencia útil si el proceso evoluciona hacia una ejecución con más de un consumidor.

La máquina de estados de SD-006 es compartida por los subsistemas que generan trabajos. `queued` indica que la petición ha sido aceptada y espera procesamiento; `running` indica que el worker la ha reclamado; `completed` conserva un resultado; `failed` conserva el error; y `cancelled` identifica una cancelación solicitada por el usuario antes del inicio. No todos los estados se pueden alcanzar desde todas las rutas: la cancelación solo opera sobre `queued`, y un trabajo `running` no se interrumpe mediante una actualización administrativa.

El reinicio de la aplicación se trata como un evento de recuperación. Durante el lifespan de FastAPI se inicializa la base de datos y se llama a `start_worker()`. Este método restablece los trabajos `running` y crea la tarea de `worker_loop()`. La recuperación no pretende reconstruir el estado intermedio de un entrenamiento parcialmente ejecutado; vuelve a poner el trabajo en condición de ser procesado, por lo que los motores deben tolerar que una tarea se reinicie desde la información disponible en su payload y en sus artefactos.

El subsistema también resuelve aspectos que no deben mezclarse con la cola. `lang.py` proporciona textos traducidos para los routers y la interfaz, mientras que los scripts JavaScript gestionan el tema visual y la presentación de los estados. El middleware CSRF y el middleware de cabeceras se registran globalmente en `main.py`, aunque funcionalmente apoyan a todos los subsistemas. Se incluyen en la visión transversal de SD-006 por su alcance común, pero sus responsabilidades de seguridad no sustituyen las comprobaciones de propiedad específicas de SD-003 y SD-004.

El límite principal de SD-006 es que la cola está implementada sobre MySQL y el worker se ejecuta dentro del mismo proceso de la aplicación. Esta solución es suficiente para el entorno actual y evita introducir Redis, Celery o un sistema distribuido que no forma parte del código. A cambio, el diseño depende de la disponibilidad del proceso y de la base de datos local, y no proporciona por sí mismo escalado horizontal de workers ni una garantía independiente de disponibilidad. Esta limitación queda recogida para que las futuras decisiones de soporte no confundan la ejecución asíncrona actual con una arquitectura distribuida.

La descomposición de SD-006 permite finalmente que los demás subsistemas compartan un contrato común para las operaciones largas. SD-002 entrega un trabajo de diagnóstico; SD-004 entrega trabajos de entrenamiento y validación; SD-006 los procesa, actualiza sus estados y devuelve la información a los routers de consulta. El usuario observa siempre una misma idea de funcionamiento —petición aceptada, trabajo pendiente, procesamiento y resultado— aunque los motores internos sean diferentes. Esta uniformidad es la principal aportación arquitectónica del subsistema transversal.

17.3.7. Relaciones de integración entre los subsistemas de diseño

La separación anterior permite identificar qué subsistemas inician una operación, cuáles la ejecutan y cuáles presentan sus resultados. SD-001 constituye el punto de entrada común porque proporciona la identidad y el rol. SD-002 y SD-004 son los productores de los trabajos de mayor coste; SD-006 se encarga de procesarlos; SD-003 y SD-005 consumen la información persistida desde perspectivas diferentes. Esta organización evita que el flujo observable de un caso de uso dependa de una llamada directa desde la interfaz a cada motor interno.

En el flujo de diagnóstico, el navegador envía la imagen y la arquitectura a SD-002. El router obtiene el identificador del usuario desde SD-001, valida la carga y registra un trabajo en SD-006. El worker ejecuta el motor de SD-002, produce sus ficheros y escribe la consulta final en MySQL. Cuando el usuario abre el historial, SD-003 recupera la fila de consultations y entrega las rutas de los artefactos ya generados. Si el usuario dispone del rol adecuado, SD-005 puede recuperar una consulta desde la perspectiva administrativa, pero no altera el modo en que SD-002 calculó el resultado.

El flujo del laboratorio es más amplio. El usuario comienza una conversación que SD-004 dirige hacia Groq; después, la configuración devuelta se valida antes de crear una sesión. El lanzamiento registra los parámetros en los artefactos del laboratorio y en el payload de SD-006. El worker invoca a los motores de entrenamiento y evaluación, y los resultados se almacenan en el directorio de la sesión. Las rutas posteriores de SD-004 consultan esos artefactos, calculan o recuperan el ranking, solicitan comparativas y generan el informe. SD-005 puede contabilizar y consultar las sesiones asociadas a un usuario, pero la gestión detallada de los resultados sigue perteneciendo a SD-004.

Las dependencias de persistencia son deliberadamente asimétricas. SD-001 escribe usuarios y tokens; SD-002 escribe trabajos y consultas; SD-003 modifica únicamente los datos organizativos de las consultas; SD-004 combina MySQL para la cola y el sistema de ficheros para los resultados de entrenamiento; SD-005 realiza consultas globales; y SD-006 actualiza los estados operativos de los trabajos. Ningún subsistema debe acceder a los datos de otro sin aplicar la regla de propiedad o la autorización administrativa que corresponda.

La aplicación mantiene también una separación entre datos estructurados y artefactos. Las cuentas, las consultas, los tokens y los estados de la cola se guardan en MySQL porque necesitan filtros, relaciones y actualizaciones transaccionales. Las imágenes, los mapas XAI, los informes PDF, los logs y las configuraciones de entrenamiento se guardan en el sistema de ficheros porque son archivos o estructuras generadas por los motores. Las tablas conservan referencias a los artefactos, pero no incorporan el contenido binario completo en cada registro.

La gestión de errores sigue el límite del subsistema que los origina. Un error de autenticación se resuelve en SD-001 con una respuesta de acceso no autorizado; un fichero que no cumple las restricciones se rechaza en SD-002 antes de generar un trabajo; una consulta ajena se bloquea en SD-003; una configuración de entrenamiento inválida se rechaza en SD-004; un usuario sin rol recibe 403 en SD-005; y un fallo durante una tarea se registra mediante SD-006 como trabajo fallido. Esta distribución evita que los routers tengan que conocer detalles internos de todos los componentes para construir mensajes de error.

La extensibilidad del diseño se apoya en los puntos donde la variación es real. Añadir una nueva arquitectura de inferencia afecta principalmente a `ml_engine` y a las funciones XAI compatibles; añadir un nuevo tipo de tarea requiere ampliar la selección de `_execute_job()` y los datos que presenta `queue.py`; incorporar una nueva operación administrativa se restringe a SD-005; y ampliar los idiomas se concentra en los recursos gestionados por `lang.py` y los scripts de interfaz. Estas extensiones no exigen modificar el mecanismo general de autenticación ni reconstruir la persistencia completa.

El criterio de diseño adoptado es, por tanto, la cohesión funcional con dependencias explícitas. Los componentes de un subsistema colaboran porque comparten una responsabilidad y no únicamente porque estén escritos en Python. Los routers conocen los contratos HTTP; los servicios conocen la lógica de aplicación; el worker conoce la ejecución prolongada; MySQL conoce el estado estructurado; y el navegador conoce la representación y la interacción. Esta distribución hace posible comparar cada subsistema con los casos de uso del capítulo 12 y comprobar, durante las pruebas, si el fallo pertenece a la entrada, a la coordinación, al cálculo o a la persistencia.

Flujo de integración	Subsistema iniciador	Subsistema coordinador	Resultado persistido o presentado
Registro e inicio de sesión	SD-001	SD-001 y middleware global	Cookies de sesión y registro en users y refresh_tokens.
Diagnóstico asistido	SD-002	SD-006 mediante job_queue	Predicción, confianza, mapas XAI, PDF y fila en consultations.
Consulta del historial	SD-003	SD-001 para identidad	Listado de consultas y acceso a sus artefactos.
Entrenamiento MLOps	SD-004	SD-006 mediante job_queue	Sesión, modelos, métricas, artefactos y logs.
Validación externa	SD-004	SD-006 mediante job_queue	Resultados de validación asociados a la sesión.
Supervisión administrativa	SD-005	SD-001 para autenticación y rol	Listado de usuarios, consultas y sesiones supervisadas.
Consulta y cancelación de trabajos	SD-006	SD-001 para identidad	Estado, posición, error o cancelación del trabajo.

Tabla x - Relaciones de integración entre subsistemas de diseño

17.4. Operación segura y controles de protección

La operación de vitalXAI debe conservar las garantías de confidencialidad, integridad y disponibilidad definidas durante el análisis. Estas garantías no dependen de una única función, sino de la combinación de controles distribuidos entre la sesión, los routers, los servicios, la base de datos, el sistema de ficheros y el entorno de publicación. El propósito de este apartado es describir cómo debe funcionar la plataforma de forma segura cuando se inicia, recibe peticiones, procesa trabajos y se detiene, manteniendo la correspondencia con los requisitos RNF-001 a RNF-034 y con los casos de uso que implican autenticación, datos sanitarios o ejecución asíncrona.

La operación segura se organiza en seis ámbitos: control de identidad y permisos, protección de datos y ficheros, comunicaciones y secretos, seguridad de las tareas asíncronas, persistencia y recuperación y condiciones del despliegue. La siguiente tabla resume los controles principales y su punto de aplicación.

Ámbito operativo	Controles principales	Componentes implicados
Identidad y sesión	Hash de contraseñas, JWT con expiración, refresh tokens, rotación, revocación y cookies protegidas.	auth.py, auth_service.py y tabla refresh_tokens en MySQL.
Autorización	Comprobación de autenticación, propiedad de los recursos y rol de administrador.	Routers de history, trainer, queue y administración.
Protección web	CSRF, cabeceras de seguridad y limitación de peticiones.	CSRFMiddleware, SecurityHeadersMiddleware y slowapi.
Datos y ficheros	Datasets anonimizados, validación de imágenes, asociación por usuario y control de rutas.	MySQL, inference.py, consultations y directorios de resultados.
Trabajos asíncronos	Estados persistentes, reclamación condicional, aislamiento de errores y recuperación al reiniciar.	Tabla job_queue y ejecución en queue_worker.py.
Operación y recuperación	Variables de entorno, MySQL/XAMPP, HTTPS mediante el túnel y copias operativas de la base de datos.	main.py, .env, Unicorn, XAMPP y Cloudflare Tunnel.

Tabla x - Controles de operación segura de vitalXAI

17.4.1. Control de identidad, sesión y autorización

El acceso a las funciones privadas comienza con la identificación del usuario a partir de la cookie `access_token`. El token contiene el identificador de la cuenta y una fecha de expiración, y se firma mediante HS256 con la clave configurada en `JWT_SECRET_KEY`. El servidor no confía en un identificador recibido como un campo ordinario de la petición: cada router obtiene el usuario desde la credencial de sesión y utiliza ese valor para realizar sus consultas.

Durante el registro, `auth.py` valida los campos recibidos y comprueba la unicidad del nombre de usuario antes de insertar la cuenta. La contraseña se transforma mediante `bcrypt` y solo el resultado se almacena en `password_hash`. Durante el inicio de sesión se recupera el hash de la cuenta y se compara con la contraseña recibida; si la comparación falla, no se crean las cookies de sesión. Este comportamiento evita que una petición con credenciales inválidas genere una identidad parcial o un token utilizable.

La sesión utiliza dos credenciales con funciones distintas. El token de acceso se emplea para identificar las peticiones ordinarias y tiene una duración configurable. El token de refresco se utiliza exclusivamente en `/api/token/refresh`, se registra mediante un hash en `refresh_tokens` y permite crear un nuevo acceso sin repetir el formulario de inicio de sesión. La rotación revoca el token anterior y genera otro valor aleatorio. Si se detecta el uso de una credencial revocada fuera del intervalo de tolerancia, el servicio revoca el conjunto de tokens asociado al usuario.

El cierre de sesión debe revocar el refresh token y eliminar las cookies desde la respuesta. La revocación se realiza en la base de datos, por lo que no depende de que el navegador elimine correctamente sus datos locales. El token de acceso puede seguir presente hasta su expiración natural, pero las operaciones de renovación quedan bloqueadas por el estado persistido del token de refresco. Esta diferencia debe considerarse al operar el sistema: cerrar la sesión evita la renovación, pero la protección del canal y la duración razonable del token de acceso siguen siendo necesarias.

La autorización se aplica después de la autenticación. En el historial, las consultas deben pertenecer al usuario salvo cuando la operación se realiza desde el ámbito administrativo. En el laboratorio, las sesiones y sus resultados se comprueban mediante `_verify_session_ownership()` antes de ser consultados, renombrados o eliminados. En la cola, tanto la consulta del estado como la cancelación filtran por `user_id`. En administración, `_require_admin()` consulta el rol en `users` y bloquea al usuario autenticado que no tenga el valor `admin`.

Este diseño evita dos errores habituales. El primero es confundir que un usuario esté autenticado con que pueda acceder a cualquier registro; la sesión solo proporciona identidad, mientras que la propiedad decide el alcance. El segundo es dejar la autorización en el cliente; aunque la interfaz oculte el panel administrativo a un usuario ordinario, el servidor vuelve a comprobar el rol cuando recibe la petición.

La protección CSRF se aplica a las operaciones que cambian el estado. El middleware genera una cookie de token y exige que el navegador copie el mismo valor en la cabecera `X-CSRF-Token`. Las peticiones de lectura no necesitan esta comprobación y el inicio de sesión se excluye para permitir que un visitante obtenga su primera sesión. Esta excepción no convierte el resto de las rutas públicas en operaciones sin protección: cada petición que modifica datos debe incluir el token correspondiente.

17.4.2. Protección de los datos y de los artefactos

La plataforma debe operar exclusivamente con imágenes y datasets anonimizados. La arquitectura no incorpora un componente que identifique o elimine automáticamente los datos personales que puedan aparecer dentro de una radiografía; por tanto, la anonimización previa de los ficheros es una condición del procedimiento de uso. La interfaz no debe utilizarse para introducir nombres, identificadores sanitarios u otros metadatos que no sean necesarios para el diagnóstico asistido.

La ruta de diagnóstico aplica controles básicos antes de escribir una imagen: restringe el tipo MIME a JPEG y PNG y limita el tamaño a 10 MB. Después genera un nombre basado en la fecha y conserva el fichero en static/uploads. El trabajo encolado contiene la ruta de la imagen y la arquitectura solicitada, mientras que la fila final de consultations conserva las rutas de los artefactos asociados. Las rutas no deben construirse a partir de concatenaciones posteriores de datos no validados, y el entorno debe restringir los permisos de escritura de los directorios de cargas y resultados al proceso que ejecuta la aplicación.

La asociación por usuario es el control principal de confidencialidad lógica. consultations, training_jobs, job_queue y refresh_tokens contienen un user_id que permite aplicar filtros explícitos. Las consultas de historial y de cola usan ese filtro directamente; las operaciones del laboratorio validan la propiedad de la sesión; y las operaciones administrativas se separan mediante el rol. Esta estrategia no sustituye a los permisos del sistema operativo sobre los directorios, pero evita que un usuario autenticado acceda a los metadatos de otra cuenta a través de las rutas funcionales.

Los ficheros generados por el diagnóstico incluyen la imagen original, el mapa de explicabilidad y el informe PDF. El informe puede contener el resultado, la confianza, el modelo utilizado y la imagen, por lo que debe considerarse parte de la información protegida aunque se sirva como un documento descargable. El sistema conserva las rutas y la interfaz muestra los artefactos dentro del flujo autorizado de la consulta. La configuración del servidor debe impedir que el directorio de resultados se convierta en un repositorio de ficheros accesible sin control cuando la aplicación se publique fuera del entorno local.

El acceso a MySQL se realiza mediante consultas parametrizadas y conexiones procedentes del pool. Las operaciones de lectura y escritura cierran la conexión al terminar, y las actualizaciones importantes ejecutan commit() de forma explícita. Las claves ajenas vinculan los registros dependientes con users, lo que evita mantener consultas huérfanas cuando se elimina una cuenta con las reglas configuradas en el esquema.

La protección lógica de los datos debe complementarse con una política de retención. La implementación actual permite eliminar una consulta de forma física desde el historial y no incorpora un subsistema de archivado o borrado lógico para las consultas. Por tanto, el operador debe conocer que la eliminación no equivale a una copia de seguridad ni a una auditoría histórica. Las copias de la base de datos y de los directorios de resultados deben gestionarse por separado si se necesita recuperar una operación eliminada.

17.4.3. Comunicaciones, configuración y secretos

En el entorno local, Uvicorn atiende la aplicación en el puerto 8000 y MySQL se ejecuta normalmente mediante XAMPP en el puerto 3306. Para permitir el acceso desde un navegador externo se utiliza el túnel configurado para la demostración. El túnel actúa como punto de publicación y debe proporcionar HTTPS hacia el exterior; el backend no debe exponerse directamente abriendo puertos de la base de datos a Internet.

La comunicación entre el navegador y las rutas de FastAPI transporta credenciales, imágenes y resultados. Por ello, el acceso remoto debe realizarse siempre mediante HTTPS basado en TLS. Las cabeceras añadidas por SecurityHeadersMiddleware complementan el cifrado del canal: Content-Security-Policy restringe los orígenes de recursos, X-Frame-Options evita la inclusión en marcos, X-Content-Type-Options bloquea la interpretación MIME indebida, Strict-Transport-Security comunica la preferencia por HTTPS y Referrer-Policy limita la información que se comparte entre páginas.

La aplicación utiliza variables de entorno para los valores que no deben formar parte del código fuente. Entre ellas se encuentran la clave JWT, las credenciales de MySQL, el tamaño del pool, las duraciones de los tokens y GROQ_API_KEY. El fichero .env, cuando se utiliza en desarrollo, debe quedar fuera del control de versiones y sus permisos deben limitarse al usuario que ejecuta la aplicación. La clave por defecto de desarrollo que auth_service.py detecta mediante una advertencia no es una configuración válida para producción.

La integración con Groq exige controlar tanto la disponibilidad como el secreto de acceso. chatbot_service.py es el único componente que necesita la clave de la API y traduce la respuesta del servicio externo al flujo conversacional del laboratorio. Los routers no deben incluir la clave en las respuestas JSON ni en los logs. Si Groq no está disponible, la conversación debe informar del error sin revelar la configuración del entorno ni interrumpir la disponibilidad del resto de la plataforma.

La integración con Hugging Face presenta una restricción diferente: puede requerir conectividad durante la descarga inicial de los modelos Transformer y memoria suficiente para mantenerlos cargados. Una vez disponibles, el motor puede reutilizar los modelos según su configuración. La operación debe distinguir entre un fallo de red durante la carga y una predicción inválida, registrando el trabajo como fallido cuando no pueda completarse y evitando devolver al usuario una salida parcial.

La configuración de desarrollo se debe validar antes de publicar el servicio. El operador debe comprobar que Python y las dependencias coinciden con requirements.txt, que MySQL está iniciado, que las variables de entorno están definidas, que existe la estructura de directorios necesaria y que el acceso al túnel funciona mediante HTTPS. Esta verificación se complementa con la ejecución de las pruebas automatizadas y evita que un fallo de configuración se interprete como un error del modelo o de la lógica de negocio.

17.4.4. Seguridad de la ejecución asíncrona

Los trabajos largos se consideran recursos operativos que requieren control de ciclo de vida. La petición que los crea no ejecuta el cálculo directamente: primero valida la identidad, registra el tipo de trabajo, asocia el payload al usuario y devuelve su identificador. El worker recupera la información desde MySQL y no debe aceptar órdenes procedentes directamente del navegador para ejecutar una función arbitraria. El tipo `job_type` determina si se llama al flujo de diagnóstico, entrenamiento o validación externa.

El payload contiene rutas y parámetros que deben haber sido validados por el router que creó el trabajo. El worker vuelve a interpretar el JSON antes de utilizarlo y falla de forma controlada si faltan claves obligatorias. Esta doble frontera es necesaria porque la fila podría conservar datos antiguos o haber sido alterada por un fallo de operación. El worker nunca debe tratar el contenido de un payload como código ejecutable ni utilizarlo para seleccionar un módulo fuera de los tipos explícitamente soportados.

La reclamación condicional del trabajo protege frente a duplicados. `_claim_job()` solo cambia el estado si sigue siendo `queued`; por tanto, un trabajo cancelado o reclamado por otro consumidor no se ejecuta de nuevo desde esa iteración. Después, `_finish_job()` guarda el resultado serializado y `_fail_job()` limita el mensaje de error a 500 caracteres. Este límite reduce el riesgo de llenar la base de datos con trazas descontroladas, aunque los diagnósticos detallados deben consultarse en los logs del entorno cuando sean necesarios.

El aislamiento de errores se basa en el tratamiento individual de cada trabajo. Una excepción durante la inferencia, la generación XAI o el entrenamiento se captura en el bloque de procesamiento y cambia la fila a `failed`. El bucle principal continúa con el siguiente trabajo y, si se produce un error externo al procesamiento, espera antes de volver a consultar la cola. Así, un dataset defectuoso o una arquitectura incompatible no detiene por completo el servicio.

La recuperación tras un reinicio es deliberadamente conservadora. `start_worker()` restablece los trabajos que quedaron en `running` a `queued`, porque no existe un registro externo que garantice en qué punto exacto del cálculo se encontraba el proceso. Esta estrategia puede provocar que una tarea vuelva a comenzar, pero evita dejarla invisible para el usuario. El diseño debe asumir que las operaciones de entrenamiento y validación pueden repetirse después de una interrupción y que los artefactos parciales no deben considerarse resultados válidos hasta que el trabajo se marque como completado.

La cancelación está restringida al estado `queued`. No se intenta finalizar abruptamente un hilo o un cálculo que ya está en `run_in_executor`, porque esa interrupción podría dejar ficheros incompletos, sesiones en un estado ambiguo o conexiones abiertas. El usuario recibe una respuesta de recurso no encontrado cuando el trabajo no existe, no le pertenece o ya ha comenzado. Este comportamiento prioriza la integridad del procesamiento frente a una cancelación inmediata sin garantías.

17.4.5. Persistencia, arranque y recuperación operativa

El arranque de la aplicación se concentra en la función `lifespan` de `main.py`. Primero se carga la configuración del entorno, después se ejecuta `init_db()` para comprobar o crear las tablas principales y, finalmente, se inicia el worker. Si MySQL no está disponible, la aplicación informa de la incidencia con un mensaje destinado al operador. Esta información no debe considerarse una sustitución de la recuperación: la plataforma necesita que el servicio de base de datos esté operativo para autenticar usuarios, consultar trabajos y persistir resultados.

El pool de conexiones se crea de forma diferida cuando se solicita la primera conexión. Esta decisión evita abrir conexiones innecesarias al importar el módulo y permite que los parámetros se obtengan del entorno ya cargado. El tamaño por defecto es cinco, pero puede modificarse con `DB_POOL_SIZE` según la capacidad real del servidor. El operador debe evitar configurar un pool superior al número de conexiones que MySQL puede atender con estabilidad.

Las tablas principales cumplen funciones diferentes. `users` y `refresh_tokens` sostienen la identidad; `consultations` conserva el historial clínico; `training_jobs` conserva información de entrenamientos; y `job_queue` permite coordinar los trabajos del diagnóstico, el entrenamiento y la validación externa. La persistencia de los artefactos se completa con `static/uploads`, `static/results` y `training_results`. Un procedimiento de copia completo debe incluir tanto el contenido relacional como los directorios de ficheros, ya que restaurar solo MySQL dejaría rutas sin sus documentos asociados.

La recuperación operativa debe contemplar tres fallos distintos. Ante un fallo de MySQL, no se pueden resolver las operaciones que dependen de la cuenta o de la cola; ante un fallo del sistema de ficheros, los registros pueden conservar rutas cuyos documentos no existen; y ante un fallo del worker, los trabajos running pueden volver a quedar encolados durante el siguiente arranque. La guía de despliegue debe permitir identificar cuál de estos componentes ha fallado antes de reiniciar indiscriminadamente toda la aplicación.

La integridad de los resultados requiere una comprobación adicional durante la operación. Una consulta marcada como `completed` debe disponer de sus artefactos esperados y de la fila de consulta correspondiente. Un trabajo `failed` debe conservar un mensaje suficiente para distinguir un error del modelo, del dataset, de la API externa o de la persistencia. El sistema actual registra estos estados, pero no incorpora un servicio separado de observabilidad; por ello, el operador debe revisar la salida del proceso y los logs disponibles cuando se investigue una incidencia.

17.4.6. Límites actuales y responsabilidades del operador

La solución actual está diseñada para un entorno local o de demostración controlada. La aplicación, MySQL, los ficheros y el worker pueden residir en la misma máquina, y el túnel proporciona el acceso remoto sin convertir el sistema en una plataforma distribuida. Esta decisión reduce la complejidad, pero implica que la disponibilidad depende del equipo anfitrión, de XAMPP, de la conectividad y de la cuota de Groq cuando se utiliza el asistente.

El operador debe iniciar MySQL antes de ejecutar la aplicación, verificar las variables de entorno, evitar el uso de la clave JWT de desarrollo, conservar las dependencias del entorno virtual y comprobar que el directorio de resultados tiene espacio suficiente. También debe controlar los datasets que se cargan, confirmar que están anonimizados y no exponer directamente los puertos 3306 ni otros servicios internos a Internet. La URL pública del túnel debe compartirse solo con los usuarios autorizados durante la demostración.

La implementación actual no proporciona alta disponibilidad, balanceo de workers, almacenamiento de secretos dedicado, cifrado de ficheros en reposo ni un sistema formal de auditoría administrativa. Estas ausencias no se ocultan detrás de nombres arquitectónicos: son límites operativos que deben tenerse en cuenta al valorar la plataforma. Los controles implementados —hashes, tokens, CSRF, cabeceras, rate limiting, filtros de propiedad, estados de cola y recuperación de trabajos— aportan una base de seguridad, pero no equivalen por sí solos a una certificación clínica ni a un entorno de producción de alta disponibilidad.

En consecuencia, el uso de vitalXAI debe mantenerse dentro del alcance declarado en el proyecto. La plataforma ofrece apoyo al diagnóstico y experimentación de modelos, pero la decisión clínica corresponde al profesional. Los datos de pacientes deben anonimizarse antes de la carga, los resultados deben interpretarse junto con los mapas de explicabilidad y los informes no deben considerarse una autorización para automatizar decisiones médicas sin supervisión. La operación segura incluye tanto las medidas técnicas como estas reglas de uso.

La combinación de controles de software y responsabilidades operativas completa el diseño de seguridad. SD-001 protege la identidad; SD-002 limita y procesa las entradas de diagnóstico; SD-003 y SD-004 restringen el acceso a los datos propios; SD-005 separa la supervisión administrativa; SD-006 controla los trabajos y sus estados; y el entorno mantiene MySQL, los ficheros, las claves y el canal HTTPS bajo condiciones conocidas. Esta distribución permite verificar cada requisito de operación en el punto donde realmente se aplica y evita atribuir a una única capa garantías que dependen del sistema completo.

18. Arquitectura de soporte y mecanismos transversales

La arquitectura de soporte reúne los elementos que permiten que los subsistemas funcionales operen de forma coordinada, segura y mantenible. A diferencia de los subsistemas de diseño del capítulo 18, estos componentes no representan una capacidad clínica o investigadora concreta para el usuario. Su responsabilidad es proporcionar persistencia, almacenamiento, configuración, protección, validación y ejecución común a las funcionalidades de autenticación, diagnóstico, historial, laboratorio y administración.

La separación entre funcionalidad y soporte no significa que ambos ámbitos estén desconectados. SD-002 depende del almacenamiento y de la cola; SD-004 necesita el sistema de ficheros, MySQL y la configuración de los servicios externos; SD-001 utiliza el middleware de seguridad; y todos los routers dependen del ciclo de vida de FastAPI. La arquitectura de soporte permite que estas dependencias se apliquen de forma uniforme, evitando que cada módulo implemente su propia conexión, su propio control de errores o su propia interpretación del entorno.

En la implementación actual se distinguen dos dimensiones. La primera está formada por los subsistemas de soporte, que agrupan servicios o recursos con una identidad técnica propia: persistencia relacional, almacenamiento de ficheros, entorno de ejecución e integración externa y calidad operativa. La segunda está formada por los mecanismos de diseño transversales, que son reglas aplicadas a varios componentes: gestión de configuración, validación de entradas, autenticación y autorización, manejo de errores, ejecución asíncrona, modularidad y pruebas automatizadas.

18.1. Subsistemas de Soporte

Los subsistemas de soporte descritos en esta sección no sustituyen a SD-006 ni duplican sus responsabilidades. SD-006 pertenece a la lógica de capacidades transversales que el usuario puede utilizar, como consultar o cancelar un trabajo. En cambio, los subsistemas de soporte proporcionan la infraestructura que hace posible esa capacidad: la conexión a MySQL, el sistema de ficheros, el entorno de ejecución y los servicios que se integran con la plataforma.

Subsistema de soporte	Responsabilidad	Componentes actuales
SSOP-001: Persistencia relacional	Crear conexiones, conservar entidades y asegurar la integridad básica de las relaciones.	database.py, MySQLConnectionPool, MySQL/XAMPP y tablas del esquema.
SSOP-002 Almacenamiento de ficheros y recursos	Conservar imágenes, mapas XAI, informes, resultados MLOps, plantillas y recursos estáticos.	static/, templates/, training_results/ y montajes de main.py.
SSOP-003 Entorno de ejecución e integración	Proporcionar el proceso web, las dependencias Python y el acceso a APIs y repositorios externos.	static/, templates/, training_results/ y montajes de main.py.
SSOP-004 Calidad y verificación	Comprobar el comportamiento de los componentes y detectar regresiones durante el desarrollo.	pytest, fixtures de tests/conftest.py, mocks, pytest-cov, ruff y CI.

Tabla x - Subsistemas de soporte de vitalXAI

18.1.1. SSOP-001: Persistencia relacional y conexiones de base de datos

El subsistema SSOP-001 encapsula el acceso de la aplicación a MySQL. Su componente principal es `database.py`, que mantiene una referencia al pool de conexiones y expone `get_db_connection()` para que los routers y servicios puedan solicitar una conexión sin conocer cómo se crea. La configuración se obtiene de `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` y `DB_POOL_SIZE`, con valores de desarrollo definidos cuando el entorno no proporciona una variable concreta.

El pool se crea de forma diferida. La primera operación que necesita una conexión invoca `_get_pool()`, que construye un `MySQLConnectionPool` con el tamaño configurado. Las operaciones posteriores reutilizan el mismo objeto y solicitan conexiones disponibles mediante `get_connection()`. Este comportamiento reduce el coste de crear conexiones repetidas y proporciona un punto único desde el que controlar el acceso concurrente a la base de datos.

La inicialización del esquema se realiza mediante `init_db()`, llamada durante el lifespan de FastAPI. El procedimiento comprueba la existencia de las tablas `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`, y crea las que no existan. Esta estrategia es adecuada para el entorno local y de demostración actual, donde la base de datos se inicializa desde la aplicación. No equivale a un sistema de migraciones versionadas: una evolución posterior del esquema deberá gestionarse con un procedimiento controlado para no depender únicamente de sentencias `CREATE TABLE IF NOT EXISTS`.

La persistencia relacional cumple dos funciones diferentes. Por un lado, conserva información de negocio, como las cuentas y consultas. Por otro, sirve de soporte operacional para la cola de trabajos y los tokens de refresco. En ambos casos se utilizan consultas parametrizadas, se confirma la escritura con `commit()` y se cierra la conexión tras terminar. Las claves ajenas de las tablas de trabajos, consultas y tokens apuntan a `users`, lo que mantiene la relación de propiedad a nivel de esquema.

Los componentes funcionales no comparten una conexión global abierta. Cada router o servicio solicita una conexión para una operación concreta, obtiene un cursor, ejecuta la consulta y cierra la conexión. Esta decisión limita el alcance de un error de conexión y evita que una petición retenga indefinidamente un recurso del pool. El coste de esta estrategia se controla mediante `DB_POOL_SIZE`, que debe ajustarse a la capacidad real de MySQL y al número de peticiones que se espera atender.

El subsistema también define el comportamiento ante indisponibilidad. Si MySQL no puede conectarse durante el arranque, `main.py` muestra un aviso operativo indicando que debe iniciarse XAMPP o revisarse la configuración. La aplicación puede continuar hasta iniciar el worker, pero las operaciones que requieren identidad, persistencia o cola no podrán completarse correctamente. Esta respuesta informa del problema sin ocultar que MySQL es una dependencia obligatoria.

18.1.2. SSOP-002: Almacenamiento de ficheros y recursos de aplicación

El subsistema SSOP-002 gestiona los artefactos que no son adecuados para almacenarse directamente como columnas relacionales. La aplicación utiliza `static/uploads` para las imágenes originales de diagnóstico, `static/results` para los mapas XAI y `training_results` para las configuraciones, logs y resultados del laboratorio. Los informes PDF se generan en las rutas que devuelven `pdf_generator.py` y `pdf_generator_mlops.py`. MySQL conserva las referencias a los ficheros cuando forman parte de una consulta o de un resultado.

La separación entre base de datos y sistema de ficheros evita insertar imágenes y documentos binarios grandes dentro de las tablas principales. La base de datos mantiene los metadatos que necesita el historial —modelo, etiqueta, confianza, usuario y rutas— y el sistema de ficheros conserva el contenido. Cuando SD-003 recupera una consulta, no recalcula ni copia el documento: utiliza las referencias persistidas para que la interfaz solicite el artefacto correspondiente.

Las plantillas Jinja2 y los recursos JavaScript pertenecen también a este subsistema de soporte, aunque se consumen desde la capa de presentación. `main.py` monta `static` como directorio de recursos y crea el directorio de resultados de entrenamiento si no existe. Las plantillas se resuelven mediante Jinja2Templates desde `templates`. Esta configuración permite que la aplicación sirva la interfaz sin un servidor frontend independiente ni un proceso de compilación adicional.

El almacenamiento se organiza según el origen del artefacto. Las cargas de diagnóstico reciben un nombre generado con una marca temporal y el nombre original; los resultados XAI se generan en un subdirectorio específico; y las sesiones de entrenamiento utilizan su identificador para agrupar configuración, logs y resultados. Esta organización facilita que el worker encuentre los artefactos a partir del payload y que el administrador pueda contabilizar sesiones inspeccionando sus configuraciones.

El control de acceso a los ficheros debe entenderse junto al control de acceso de los registros. La aplicación comprueba la propiedad de la consulta o de la sesión antes de devolver los metadatos que apuntan a los artefactos. Además, el entorno operativo debe limitar los permisos de escritura de los directorios al proceso de la aplicación y evitar la exposición directa de los directorios de datos cuando no sea necesaria. La implementación actual ofrece validación de tipo MIME y tamaño para las imágenes, pero no incorpora un antivirus ni un servicio independiente de análisis de contenido.

La recuperación de este subsistema requiere copiar tanto los directorios de ficheros como las tablas que contienen sus rutas. Una copia de MySQL sin `static/uploads`, `static/results` o `training_results` restauraría referencias incompletas; una copia de los ficheros sin la base de datos perdería la asociación con usuarios, consultas y sesiones. Esta dependencia debe reflejarse en el procedimiento operativo de copia y restauración.

18.1.3. SSOP-003: Entorno de ejecución e integraciones externas

SSOP-003 agrupa los elementos necesarios para iniciar la aplicación y comunicarse con los servicios que no forman parte del proceso local. El servidor se ejecuta con Uvicorn sobre Python 3.11 y utiliza FastAPI como framework web. El fichero requirements.txt fija las versiones de las dependencias principales, entre ellas TensorFlow, Keras, Transformers, mysql-connector-python, Groq, fpdf2, bcrypt, python-jose, slowapi, pytest y ruff.

El entorno virtual aísla estas dependencias de otras instalaciones de Python de la máquina. Antes de iniciar el sistema se debe comprobar que el intérprete activo corresponde al entorno del proyecto y que las versiones instaladas coinciden con el fichero de requisitos. Esta condición es especialmente importante para TensorFlow, Keras y Transformers, cuyas incompatibilidades pueden impedir la importación del motor de inferencia aunque el código de los routers sea correcto.

La aplicación se integra con Groq mediante chatbot_service.py. El servicio externo recibe los mensajes de configuración y devuelve una respuesta que el laboratorio procesa. El subsistema de soporte no expone una ruta genérica para llamar a Groq: la integración se encapsula en un único componente y utiliza GROQ_API_KEY desde el entorno. Esta frontera reduce la propagación de credenciales y permite sustituir o simular el proveedor durante las pruebas.

La carga de modelos Transformer constituye la otra integración externa relevante. ml_engine.py utiliza la configuración de Hugging Face para resolver las arquitecturas y pesos correspondientes. El primer uso puede requerir conectividad y espacio local para descargar o almacenar los modelos; los usos posteriores pueden beneficiarse de la caché disponible. La operación debe registrar como fallo de integración la imposibilidad de cargar un modelo, sin atribuir ese problema a la validación de la imagen o a la base de datos.

El entorno de publicación utiliza opcionalmente un túnel de Cloudflare para que un navegador externo acceda al servidor local. El túnel no aloja los datos ni ejecuta los modelos; solo proporciona el canal de entrada hasta Uvicorn. MySQL permanece en la máquina de ejecución y sus puertos no deben publicarse. Esta configuración conserva bajo control las versiones y los ficheros del proyecto, aunque no proporciona por sí misma alta disponibilidad ni escalado horizontal.

18.1.4. SSOP-003: Entorno de ejecución e integraciones externas

El subsistema SSOP-004 agrupa las herramientas que permiten comprobar que los cambios no rompen el comportamiento existente. Las pruebas unitarias se encuentran en tests/unit, las de integración en tests/integration y las fixtures compartidas en tests/conftest.py. Los servicios externos se simulan mediante mocks y la base de datos se sustituye en las pruebas de integración por el entorno definido para ese nivel.

La estrategia de pruebas refleja la arquitectura. Los routers se prueban comprobando respuestas, autenticación, propiedad y errores; los servicios de aprendizaje automático se prueban con modelos simulados; el worker se prueba con trabajos controlados; y los middlewares se prueban de forma aislada para confirmar CSRF, cabeceras y limitación. La batería existente incluye pruebas de autenticación, validación de entradas, base de datos, inferencia, XAI, MLOps, cola, seguridad e internacionalización, además del flujo de autenticación integrado.

pytest-cov permite medir la cobertura de los módulos principales de services, routers y database.py. ruff verifica el estilo del código y el workflow de integración continua ejecuta las comprobaciones automatizadas. La calidad no se concentra en una única prueba de aceptación: se distribuye entre verificaciones rápidas durante el desarrollo y comprobaciones más amplias antes de integrar cambios.

Las fixtures deben mantenerse deterministas y sin datos reales de pacientes. Los modelos, clientes externos y conexiones de producción se sustituyen por dobles de prueba en los niveles que requieren aislamiento. La prueba de integración debe comprobar la colaboración entre capas, pero no convertir el entorno de producción ni la API de Groq en dependencias necesarias para cada ejecución local. Esta política mejora la repetibilidad y evita que una interrupción externa produzca falsos fallos del código.

18.2. Mecanismos de diseño transversales

Los mecanismos de diseño son reglas que se aplican a varios subsistemas independientemente de la funcionalidad concreta que implementen. Se distinguen de los subsistemas de soporte porque no son necesariamente un servicio con una identidad propia: son decisiones repetidas sobre cómo validar, autenticar, persistir, ejecutar y comunicar errores. En vitalXAI se identifican seis mecanismos principales.

18.2.1. Configuración externa y separación de secretos

La configuración que cambia entre máquinas o que contiene información sensible se obtiene desde variables de entorno. Este mecanismo se aplica a autenticación, MySQL, Groq, duración de tokens y tamaño del pool. El código conserva valores por defecto únicamente para permitir el desarrollo local y emite advertencias cuando una configuración insegura puede afectar a la operación.

La separación de secretos evita que una clave JWT, una contraseña de MySQL o una clave de Groq quede incrustada en un router o en un servicio. Los componentes reciben la configuración en el momento de importación o de ejecución y utilizan nombres de variables comunes. El fichero de ejemplo o la guía de despliegue puede documentar los nombres requeridos, pero no debe contener valores válidos.

Este mecanismo también mejora la portabilidad. El mismo código puede ejecutarse con una base de datos local, una URL pública del túnel diferente o una duración de tokens ajustada al entorno sin modificar los routers. La configuración de desarrollo, pruebas y demostración debe mantenerse separada para evitar que una prueba utilice accidentalmente datos o credenciales del entorno operativo.

18.2.2. Validación en las fronteras de entrada

Toda información procedente del navegador se considera no confiable hasta ser validada. FastAPI valida tipos y parámetros declarados por las rutas; los routers añaden comprobaciones de negocio; y los servicios vuelven a comprobar las condiciones necesarias antes de utilizar los datos en operaciones sensibles.

En el registro se comprueba el correo, la longitud de la contraseña y los campos obligatorios. En el diagnóstico se validan MIME y tamaño de la imagen. En el laboratorio se comprueba la existencia del dataset y la propiedad de la sesión. En la cola se valida la identidad y la pertenencia del trabajo. La validación del cliente mediante JavaScript puede mejorar la experiencia, pero no es la fuente de verdad: la petición debe poder rechazarse correctamente aunque se construya con un cliente distinto del frontend oficial.

Las consultas SQL utilizan parámetros separados de las instrucciones, y los nombres de archivo se generan dentro del servidor antes de copiar el contenido recibido. Los datos estructurados del payload de la cola se serializan como JSON y se interpretan según un tipo de trabajo controlado. La combinación de estas decisiones reduce el riesgo de que una entrada del usuario se convierta directamente en una consulta, una ruta o una orden ejecutable.

18.2.3. Autenticación, autorización y propiedad de recursos

El mecanismo de identidad se reutiliza en todos los routers privados mediante `get_user_id_from_token()`. Esto evita que cada subsistema implemente una forma distinta de interpretar la sesión. Después de obtener el usuario, cada operación aplica el alcance correspondiente: propiedad del registro, rol administrativo o acceso general a una capacidad transversal.

La autorización se realiza cerca del recurso que se protege. El historial comprueba la propiedad de la consulta; el laboratorio comprueba la sesión; la cola comprueba el trabajo; y administración comprueba el rol. Esta decisión impide que un identificador enviado por el navegador sea suficiente para acceder a un objeto ajeno. Las funciones compartidas, como `_check_consultation_ownership()` y `_require_admin()`, concentran reglas repetidas sin convertirlas en un bypass general de seguridad.

CSRF, cabeceras y rate limiting se aplican mediante middleware o decoradores para que no dependan de que cada router recuerde incluirlos. No obstante, estos mecanismos son complementarios: un token CSRF válido no concede permisos sobre otro usuario y una cabecera de seguridad no sustituye a la comprobación de rol.

18.2.4. Persistencia de estado y transacciones simples

La aplicación utiliza MySQL como fuente de verdad para el estado de cuentas, consultas, tokens y trabajos. Las operaciones de escritura siguen una secuencia corta: obtener conexión, preparar una consulta parametrizada, ejecutar, confirmar y cerrar. Esta política reduce el tiempo durante el que se mantiene un recurso del pool y facilita localizar la operación que produjo un error.

Los estados de los trabajos se persisten en lugar de mantenerse solo en una variable del proceso. Esto permite que el router de cola y el worker observen la misma información y que un reinicio pueda recuperar trabajos que quedaron en ejecución. La actualización condicional de `_claim_job()` actúa como mecanismo sencillo de exclusión para evitar que dos consumidores reclamen la misma tarea.

La persistencia de MySQL y la de ficheros deben coordinarse de manera explícita. Una fila de consulta puede existir antes de que todos sus artefactos hayan terminado de generarse, pero solo debe presentarse como resultado completado cuando el worker haya concluido el flujo. Los fallos de escritura de un informe o de un mapa XAI deben dejar constancia en el estado del trabajo y no ocultarse con una fila incompleta.

18.2.5. Ejecución asíncrona y aislamiento de trabajos

La ejecución asíncrona se implementa mediante una cola persistida y un worker interno. Los routers registran el trabajo y devuelven la respuesta; `queue_worker.py` lo reclama y envía el cálculo pesado a `run_in_executor`. Este patrón libera el ciclo de petición y permite que la interfaz consulte el progreso sin esperar a que termine el entrenamiento o la inferencia.

El worker limita sus operaciones a los tipos explícitos `diagnosis`, `training` y `external_validation`. Cada tipo tiene una función de procesamiento que interpreta su `payload`, invoca al servicio adecuado y devuelve un resultado serializable. El bucle exterior captura los errores de un trabajo, los registra como `failed` y continúa con el siguiente. La recuperación de trabajos `running` al arrancar evita que una interrupción deje tareas invisibles.

La ejecución asíncrona no significa que todas las tareas tengan el mismo nivel de prioridad o el mismo mecanismo. Los entrenamientos y validaciones utilizan `job_queue`; el recálculo estadístico del laboratorio usa `BackgroundTasks`; y las peticiones de consulta permanecen síncronas porque solo recuperan información. Documentar esta diferencia evita que una futura modificación asuma garantías de persistencia para una tarea que actualmente se ejecuta como trabajo en segundo plano del proceso web.

18.2.6. Errores, estados y comunicación con el usuario

Los errores se traducen en respuestas que la interfaz puede interpretar. Las rutas devuelven 401 cuando falta autenticación, 403 cuando el usuario no tiene autorización, 404 cuando el recurso no existe y 400 cuando los datos recibidos no cumplen una condición de negocio. Las excepciones inesperadas se convierten en respuestas 500 y, cuando pertenecen a un trabajo asíncrono, se conservan en el estado `failed` de la cola.

La capa de presentación recibe mensajes localizados mediante `lang.py`, por lo que los routers no deberían mezclar en cada respuesta textos técnicos sin una razón concreta. Al mismo tiempo, el cliente no debe recibir claves secretas, trazas completas ni credenciales. El detalle necesario para diagnosticar se conserva en la salida del servidor o en el estado de error del trabajo, mientras que la interfaz obtiene un mensaje comprensible y seguro.

Los estados `queued`, `running`, `completed`, `failed` y `cancelled` forman un contrato de comunicación entre el worker, los routers y el navegador. El usuario puede saber si la petición fue aceptada, si se está ejecutando, si terminó correctamente o si debe revisar un error. La uniformidad de este contrato reduce la necesidad de que cada vista invente una representación diferente para los trabajos.

18.2.7. Modularidad, pruebas y evolución controlada

La modularidad se aplica separando routers, servicios, worker, persistencia y recursos de presentación. Los routers coordinan HTTP; los servicios encapsulan lógica especializada; database.py proporciona conexiones; queue_worker.py ejecuta trabajos; y las plantillas y scripts representan la interacción. La separación no es absoluta —los routers actuales acceden directamente a `get_db_connection()` en varias operaciones—, pero las responsabilidades principales siguen identificables y permiten probar cada grupo de forma aislada.

Las pruebas sirven como contrato evolutivo de esos límites. Los cambios en autenticación deben conservar las pruebas de tokens y cookies; los cambios en diagnóstico deben conservar la validación de imágenes y la integración con la cola; los cambios en MLOps deben conservar la propiedad de sesiones y los estados; y los cambios en middleware deben conservar las pruebas CSRF, cabeceras y rate limiting. Esta relación entre mecanismo de diseño y prueba reduce el riesgo de que una refactorización altere una garantía transversal sin que el cambio sea detectado.

18.2.8. Ciclo de vida y sustitución controlada de componentes

Los componentes de soporte deben poder iniciarse, utilizarse y detenerse sin dejar recursos abiertos ni estados imposibles de interpretar. Durante el arranque, FastAPI carga la configuración, comprueba la base de datos e inicia el worker. Durante la operación, los routers solicitan conexiones y ficheros únicamente cuando los necesitan. Durante una parada o reinicio, los trabajos que estaban ejecutándose deben quedar identificados para que el siguiente arranque pueda devolverlos a la cola. Esta secuencia vincula el ciclo de vida del proceso web con la persistencia de `job_queue`.

La separación de soporte también facilita sustituir un componente si el entorno lo exige. MySQL podría cambiarse por otro motor compatible si se mantiene el contrato de `get_db_connection()` y se adaptan las consultas y el esquema; Groq podría reemplazarse si el servicio conversacional conserva la entrada de mensajes y la salida estructurada; y el directorio local de artefactos podría trasladarse a un almacenamiento especializado si se mantiene la relación entre rutas, usuarios y resultados. Estas posibilidades no forman parte de la implementación actual, pero muestran que la dependencia se concentra en puntos identificables y no se dispersa por toda la interfaz.

La sustitución no debe realizarse sin revisar los mecanismos transversales. Cambiar MySQL afecta al pool, a las transacciones, a las claves ajenas y a la recuperación de la cola; cambiar el almacenamiento afecta a las rutas de imágenes, informes y resultados; y cambiar el proveedor externo afecta a las credenciales, los tiempos de espera y el tratamiento de errores. Por ello, cualquier cambio de infraestructura debe acompañarse de pruebas de integración y de una comprobación de que los estados y permisos siguen siendo equivalentes.

El entorno de construcción conserva este principio mediante dependencias fijadas y pruebas reproducibles. Una nueva versión de TensorFlow, Keras, Transformers o mysql-connector-python no debe incorporarse únicamente porque resuelva un problema local: debe instalarse en un entorno aislado, ejecutarse la batería de pruebas y comprobarse que los modelos, los ficheros y las consultas mantienen el comportamiento esperado. La configuración de producción y la de pruebas deben permanecer separadas para que una migración o una prueba de recuperación no altere los datos de uso.

La arquitectura de soporte concluye así la estructura técnica de vitalXAI. SSOP-001 sostiene las relaciones y estados persistentes; SSOP-002 conserva los artefactos y recursos; SSOP-003 proporciona el entorno y las integraciones; SSOP-004 verifica el comportamiento. Sobre ellos se aplican la configuración externa, la validación, la seguridad, la persistencia de estados, la asincronía, la comunicación de errores y la modularidad. Ninguno de estos elementos sustituye a los subsistemas funcionales, pero todos son necesarios para que puedan ejecutarse de manera coherente con los requisitos y restricciones del proyecto.

La responsabilidad queda distribuida de forma verificable: el desarrollador mantiene el código y las pruebas; el operador prepara el entorno, las credenciales y MySQL; y el sistema conserva los estados y errores necesarios para diagnosticar una ejecución. Esta distribución evita que una incidencia de infraestructura se oculte como un fallo funcional y facilita localizar el punto de actuación adecuado.

Esta arquitectura deja definidos los puntos que deben comprobarse cuando se implanta o modifica la plataforma: disponibilidad de MySQL y de los directorios, validez de las variables de entorno, compatibilidad de las dependencias, funcionamiento del worker, conservación de los estados de la cola y ejecución de las pruebas automatizadas. La operación de vitalXAI depende de que estos elementos funcionen conjuntamente; por ello, el soporte no es un añadido posterior, sino la base que permite que la lógica clínica y experimental se ejecute de forma repetible y controlada.

El resultado es una infraestructura proporcionada al alcance académico y técnico del proyecto.

19. Persistencia relacional y esquema de datos de vitalXAI

Este capítulo describe cómo vitalXAI conserva la información que produce y sobre la que opera a lo largo de su funcionamiento. Hasta aquí, el análisis del capítulo 14 identificó las entidades del dominio y el diseño de los capítulos 18 y 19 fijó la arquitectura y los componentes de soporte; queda por concretar la representación física de los datos. Esta concreción incluye las tablas que existen, las columnas que las componen, los tipos de datos que admiten, las claves que las relacionan y las restricciones que las gobiernan. El objetivo es que el esquema resultante quede definido con la precisión necesaria para poder reproducirlo, verificarlo y hacerlo evolucionar sin ambigüedades durante el resto del proyecto.

El motor sobre el que se sustenta esta persistencia es MySQL, un sistema de gestión de bases de datos relacional de código abierto, accedido desde la aplicación mediante el conector oficial `mysql-connector-python` y un pool de conexiones `MySQLConnectionPool`. La definición del esquema se realiza con sentencias `CREATE TABLE IF NOT EXISTS` que la función `init_db()` ejecuta durante el arranque de la aplicación; no se emplea un ORM ni un sistema de migraciones versionadas. Esta forma de trabajar simplifica la puesta en marcha en el entorno local y de demostración, pero tiene una consecuencia que debe tenerse presente: cualquier cambio posterior sobre las tablas, como añadir una columna o un índice, no se aplicará automáticamente y deberá planificarse como una operación controlada sobre la base de datos.

Conviene destacar, además, que el almacenamiento de vitalXAI no se limita a las tablas de MySQL. El sistema conserva en el sistema de ficheros los artefactos de mayor tamaño o de estructura variable —las imágenes radiológicas, los mapas de explicabilidad, los informes en PDF y los resultados del laboratorio de entrenamiento— y las tablas guardan únicamente las rutas o referencias que permiten localizarlos. Esta separación entre datos relacionales y artefactos por fichero es una decisión de diseño deliberada, coherente con el subsistema de soporte SSOP-002 del capítulo 19, y condiciona de forma directa el procedimiento de copia y recuperación de la información.

El presente capítulo se limita al esquema de persistencia: el apartado 20.1 presenta los criterios de diseño que lo rigen, los diagramas que lo representan y la descripción detallada de cada una de las cinco tablas que lo componen. El acceso a los datos —la configuración del pool, el ciclo de vida de las operaciones y la inicialización del esquema— se documenta en el capítulo 19 dentro del subsistema de soporte SSOP-001 y, por tanto, no se repite aquí.

19.1. *Diseño del esquema de persistencia*

Las entidades que el análisis identificó en el capítulo 14 no se traducen todas a una tabla relacional. La naturaleza híbrida del almacenamiento hace que unas entidades se materialicen directamente como tablas, que otras queden representadas mediante columnas de una tabla y que otras, en particular las del laboratorio de experimentación, se conserven íntegramente en el sistema de ficheros. El resultado es un esquema compuesto por cinco tablas, todas ellas organizadas en torno a la cuenta de usuario: `users`, `consultations`, `job_queue`, `refresh_tokens` y `training_jobs`.

Antes de describir cada tabla en detalle, conviene fijar los criterios que se aplican de forma uniforme a todo el esquema. Estos criterios responden a las necesidades del proyecto y explican por qué el esquema tiene la forma que tiene.

- **Identificadores enteros autoincrementales.** Cada tabla define su clave primaria como una columna `id` de tipo `INT` con la propiedad `AUTO_INCREMENT`, de modo que MySQL asigna un valor secuencial al insertar una nueva fila. Se evita así la generación externa de identificadores, que no aporta ventajas en un sistema de alcance local y que añadiría complejidad innecesaria a la inserción.
- **La cuenta de usuario como eje de propiedad.** Las cuatro tablas dependientes de `users` incorporan una clave ajena `user_id` que apunta a `users.id`. Esta clave traslada al esquema el aislamiento de datos entre cuentas declarado en el requisito RF-005: ninguna consulta, trabajo, token o entrenamiento puede existir sin una cuenta propietaria. La base de datos garantiza la existencia de la cuenta referenciada, y la aplicación complementa esa garantía con las comprobaciones de propiedad y de rol que aplica en sus rutas.
- **Tipos de datos ajustados al contenido.** Las cadenas acotadas se declaran con `VARCHAR` y una longitud máxima, los valores numéricos con `INT` o `FLOAT` según su naturaleza, las marcas temporales con `DATETIME` y los contenidos variables con `JSON` o `TEXT`. Esta elección mantiene las tablas compactas y permite a MySQL indexar y comparar los datos con eficiencia. La precisión de los valores de confianza y de progreso se conserva con `FLOAT`, suficiente para las probabilidades y porcentajes que maneja la aplicación.
- **Borrado en cascada frente a borrado por flujo de negocio.** La relación de `consultations` con `users` no especifica una acción de borrado, mientras que `job_queue`, `refresh_tokens` y `training_jobs` declaran `ON DELETE CASCADE`. La diferencia responde a la forma en que cada información desaparece del sistema: la eliminación de una consulta la gestiona el propio flujo de negocio del historial, que borra el registro y sus artefactos de forma definitiva conforme al derecho de supresión del RGPD (RF-016), mientras que los trabajos, los tokens y los entrenamientos dependientes deben desaparecer automáticamente cuando se elimina su cuenta, para no dejar datos huérfanos.
- **Persistencia relacional y artefactos por fichero.** El esquema no pretende guardar en columnas los datos binarios ni la información variable del laboratorio. Las imágenes, los mapas, los informes y los resultados de entrenamiento se escriben en directorios del sistema de ficheros, y las tablas conservan las rutas que los referencian. Esta decisión mantiene las tablas ligeras y evita que el crecimiento de los artefactos degrade el rendimiento de las consultas.

- **Creación y evolución del esquema sin migraciones versionadas.** Las tablas se crean mediante `CREATE TABLE IF NOT EXISTS` durante el arranque. El código Python actúa, por tanto, como fuente efectiva de definición del esquema. Esta estrategia permite levantar una instalación nueva, pero no modifica tablas existentes; la evolución del esquema deberá gestionarse como un cambio de base de datos planificado y probado, sin atribuir a las sentencias de creación una capacidad de migración que no tienen.

En cuanto a los índices, el esquema no declara índices adicionales explícitos: MySQL/InnoDB crea automáticamente los índices necesarios para las claves primarias, las restricciones de unicidad y las claves ajenas. Estos índices son suficientes para el volumen de datos del proyecto. Posibles optimizaciones futuras, como índices sobre las columnas de fecha del historial o sobre el estado de los trabajos, quedan fuera de la implementación actual.

La correspondencia entre el modelo conceptual y el esquema físico merece una precisión adicional. De las ocho entidades del capítulo 14, Usuario se materializa en `users`, ConsultaDiagnostico en `consultations` y TrabajoCola en `job_queue`; los mapas de explicabilidad (MapaXAI) quedan representados por la columna `xai_image_path` de `consultations`, que apunta al fichero generado. Las entidades del laboratorio —SesionExperimentacion, ModeloEntrenado, ResultadoModelo y ValidacionExterna— no tienen tabla propia: se conservan en el directorio `training_results`, organizadas por un identificador de sesión con formato `RUN_AAAAMMDD_HHMMSS`. Esta decisión se justifica por la estructura variable de los artefactos experimentales, cuya normalización en tablas exigiría un esquema mucho más complejo, desproporcionado para el alcance del proyecto. Ambos capítulos deben leerse de forma complementaria: el 14 describe las entidades conceptuales y este describe su materialización física.

El diagrama entidad-relación de la ilustración representa el esquema en notación Chen. En esta notación, las entidades se dibujan como rectángulos, las relaciones como rombos y las cardinalidades se indican sobre las aristas (Chen, 1976; Elmasri & Navathe, 2016). La entidad `USERS` actúa como raíz y participa en cuatro relaciones de tipo uno a muchos con el resto de las entidades. El diagrama se completa a continuación con un boceto en Mermaid que puede adaptarse y refinarse.

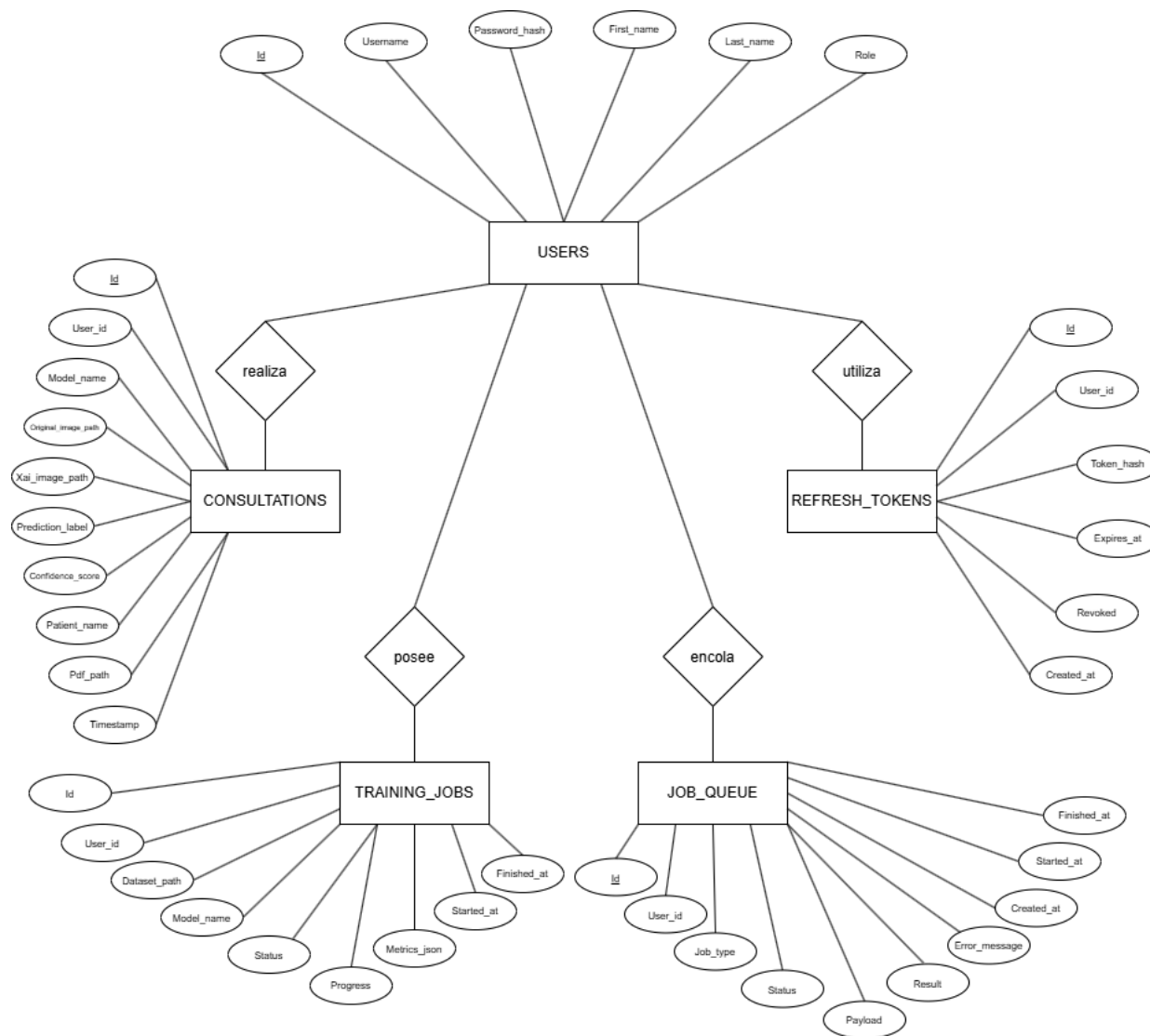


Ilustración x - Diagrama entidad-relación del esquema de persistencia en notación Che

El diagrama relacional de la siguiente ilustración representa el esquema desde la perspectiva de las tablas y sus claves. A diferencia del anterior, que se centra en las entidades y sus cardinalidades, este diagrama destaca las columnas que actúan como clave primaria y como clave ajena, y la forma en que las tablas se conectan entre sí y con el sistema de ficheros. Se incluye también como boceto en Mermaid, listo para completar.

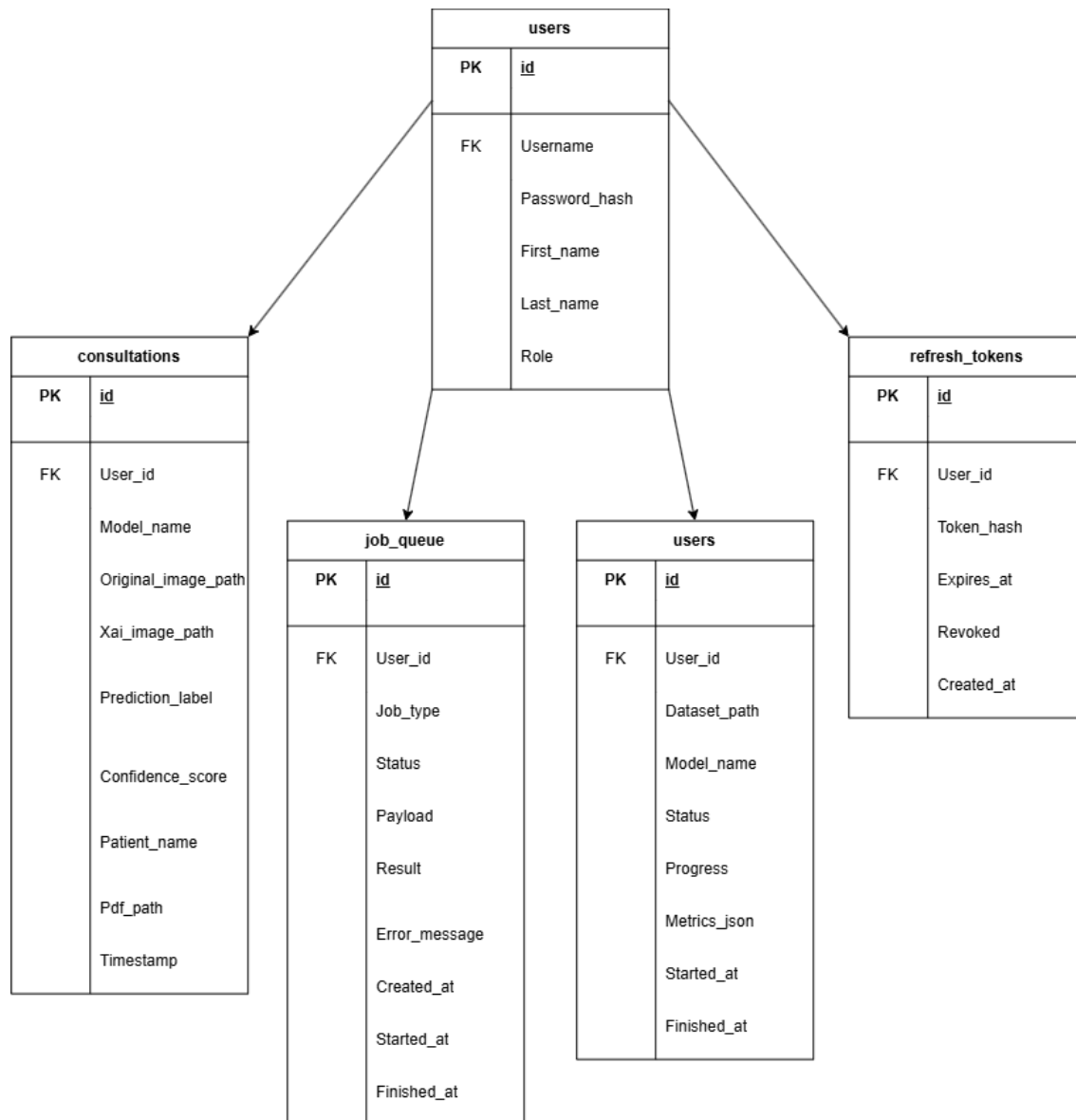


Ilustración x - Diagrama relacional de tablas y claves de vitalXAI

A continuación se describen, mediante una ficha por tabla, la estructura y las restricciones de cada una de las cinco tablas del esquema. Cada ficha indica el nombre, la descripción, los atributos con su tipo y su obligatoriedad, la clave primaria, las claves ajenas, las claves únicas y las restricciones que le aplican.

Nombre	users		
Descripción	Almacena los datos de identidad de las cuentas registradas. Es la entidad central del modelo y el origen de las relaciones de propiedad del resto de tablas.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno de la cuenta. Clave primaria autoincremental.
Username	VARCHAR(255)	S	Identificador de acceso, validado como correo electrónico.
Password_hash	VARCHAR(255)	S	Hash de la contraseña generado con bcrypt. Nunca se almacena la contraseña en claro.
First_name	VARCHAR(255)	S	Nombre mostrado del usuario.
Last_name	VARCHAR(255)	S	Apellidos mostrados del usuario.
Role	VARCHAR(255)	S	Perfil de autorización de la cuenta (ordinario o admin).
Clave primaria			
Nombre	Columnas		Secuencia
users_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino		Columnas
N/A	N/A		N/A
Claves únicas			
Nombre	Columnas		
users_username_key	Username		
Restricciones			
Nombre	Columnas		Restricción
Username	Username		Username
Username	Role		La aplicación restringe el rol a los valores ordinario y admin; el esquema no define un CHECK.

Tabla x - TB-01: users

La unicidad de username se aplica a nivel de base de datos y constituye la última barrera frente a cuentas duplicadas, incluso cuando dos peticiones de registro llegan de forma concurrente; antes de insertar, el router de autenticación valida el formato y comprueba la disponibilidad del identificador. La columna password_hash conserva el resultado de aplicar la función de hash bcrypt, de modo que la contraseña original nunca queda almacenada, en coherencia con el requisito RNF-001. La columna role es interpretada por la aplicación al proteger las rutas administrativas mediante la comprobación `_require_admin()` descrita en el capítulo 18; el esquema no introduce una tabla separada de roles, decisión suficiente para el alcance actual.

Nombre	consultations		
Descripción	Almacena el historial de consultas de diagnóstico de los usuarios. Cada fila representa una consulta completada y las rutas de sus artefactos asociados.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno de la consulta. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria de la consulta. Clave ajena a users.
Model_name	VARCHAR(100)	N	Arquitectura utilizada en el diagnóstico.
Original_image_path	VARCHAR(500)	N	Ruta de la imagen radiológica original.
Xai_image_path	VARCHAR(500)	N	Ruta del mapa de explicabilidad generado.
Prediction_label	VARCHAR(50)	N	Clasificación devuelta (PNEUMONIA o NORMAL).
Confidence_score	FLOAT	N	Confianza asociada a la predicción.
Patient_name	VARCHAR(255)	N	Nombre visible de organización de la consulta.
Pdf_path	VARCHAR(500)	N	Ruta del informe PDF generado.
Timestamp	DATETIME	S	Fecha y hora del registro, con valor por defecto CURRENT_TIMESTAMP.
Clave primaria			
Nombre	Columnas	Secuencia	
consultations_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
consultations_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
consultations_user_id_not_null	User_id	Valor no nulo: toda consulta debe asociarse a una cuenta válida.	
consultations_prediction_label_check	Prediction_label	La aplicación restringe la etiqueta a PNEUMONIA o NORMAL; el esquema no define un CHECK.	

Tabla x - TB-02: consultations

Esta tabla se alimenta desde el worker de la cola cuando un diagnóstico termina: una vez obtenida la predicción y generados los artefactos de explicabilidad y el informe, el worker registra la consulta con sus metadatos y las rutas de los ficheros. Las columnas de rutas apuntan a los ficheros conservados en el sistema de ficheros, no a su contenido, de modo que reconstruir una consulta completa exige disponer tanto de la fila como de los ficheros referenciados; una copia de seguridad de MySQL sin los directorios correspondientes restauraría referencias incompletas. patient_name actúa como una etiqueta visible que el usuario puede renombrar para organizar su historial, y debe tratarse como un dato de presentación, no como una autorización para guardar identificadores personales del paciente. El estado operativo del diagnóstico no reside en esta tabla: se conserva en job_queue, y la presencia de una fila aquí indica que el diagnóstico alcanzó la fase de persistencia del resultado.

Nombre	job_queue		
Descripción	Almacena los trabajos asíncronos de la plataforma —diagnósticos, entrenamientos y validaciones externas— con su tipo, su estado, sus parámetros y su resultado.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del trabajo. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria del trabajo. Clave ajena a users.
Job_type	VARCHAR(20)	S	Tipo de trabajo: diagnosis, training o external_validation.
Status	VARCHAR(20)	S	Estado del ciclo de vida, con valor por defecto queued.
Payload	JSON	N	Parámetros necesarios para ejecutar la tarea.
Result	JSON	N	Resultado serializado del trabajo.
Error_message	TEXT	N	Motivo de un fallo, si lo hubo.
Created_at	DATETIME	S	Fecha de encolado, con valor por defecto CURRENT_TIMESTAMP.
Started_at	DATETIME	N	Fecha de inicio de la ejecución.
Finished_at	DATETIME	N	Fecha de finalización o cancelación.
Clave primaria			
Nombre	Columnas		Secuencia
job_queue_pkey	Id		Auto incremental de MySQL (AUTO_INCREMENT)
Claves ajenas			
Nombre	Destino		Columnas
job_queue_user_id_fkey	users		User_id
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
job_queue_user_id_not_null	User_id	Valor no nulo: todo trabajo debe asociarse a una cuenta válida.	
job_queue_job_type_check	Job_type	La aplicación restringe el tipo a diagnosis, training o external_validation; el esquema no define un CHECK.	
job_queue_status_check	Status	La aplicación restringe el estado a queued, running, completed, failed o cancelled; el esquema no define un CHECK.	

Tabla x - TB-03: job_queue

Esta tabla es el soporte persistente de la ejecución asíncrona. Permite que el worker reclame y ejecute los trabajos, que el usuario consulte su progreso o los cancele, y que un reinicio del servidor recupere el estado de las tareas que quedaron en marcha. Las columnas de fecha reconstruyen la evolución temporal de cada trabajo. Las columnas payload y result, de tipo JSON, permiten guardar parámetros heterogéneos sin una columna por valor posible; a cambio, su contenido debe validarse en la aplicación, ya que MySQL no conoce su estructura interna. La tabla se asocia al usuario mediante user_id, pero no mantiene claves ajenas hacia consultations ni hacia las sesiones de entrenamiento: la relación entre un trabajo y su resultado es lógica y se expresa mediante el job_type y el contenido del payload.

Nombre	refresh_tokens		
Descripción	Almacena las credenciales de refresco de las sesiones, con su hash, su expiración y su estado de revocación.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del registro. Clave primaria autoincremental.
User_id	INT	S	Cuenta asociada al token. Clave ajena a users.
Token_hash	VARCHAR(255)	S	Hash del valor aleatorio del token.
Expires_at	DATETIME	S	Fecha y hora de caducidad del token.
Revoked	BOOLEAN	S	Indica si la credencial ha sido invalidada, con valor por defecto false.
Created_at	DATETIME	S	Fecha de creación, con valor por defecto CURRENT_TIMESTAMP.
Clave primaria			
Nombre	Columnas	Secuencia	
refresh_tokens_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
refresh_tokens_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
refresh_tokens_user_id_not_null	User_id	Valor no nulo: todo token debe asociarse a una cuenta válida.	
refresh_tokens_token_hash_not_null	Token_hash	Valor no nulo: solo se persiste el hash del token, nunca su valor original.	
refresh_tokens_expires_at_not_n ull	Expires_at	Valor no nulo: todo token debe declarar su fecha de caducidad.	

Tabla x - TB-04: refresh_tokens

El valor original del token de refresco nunca llega a la base de datos; solo se conserva su hash, de modo que una exposición de los datos no comprometa directamente las sesiones. Durante una renovación, el servicio de autenticación calcula el hash del valor recibido y busca una fila que coincida, no esté revocada y no haya caducado; la rotación invalida la fila anterior y crea una nueva. Si se detecta el uso de un token revocado fuera del periodo de tolerancia, el servicio revoca todos los tokens del usuario. La relación con users declara ON DELETE CASCADE, de modo que al eliminar la cuenta desaparecen sus credenciales de refresco, sin sustituir la revocación explícita que realiza el flujo de cierre de sesión.

Nombre	training_jobs		
Descripción	Representación estructurada prevista de los trabajos de entrenamiento de modelos. Definida en el esquema como parte de la inicialización de la base de datos.		
Atributos			
Campo	Tipo	Obligatorio	Descripción
Id	INT	S	Identificador interno del entrenamiento. Clave primaria autoincremental.
User_id	INT	S	Cuenta propietaria del entrenamiento. Clave ajena a users.
Dataset_path	VARCHAR(500)	N	Ruta del dataset utilizado.
Model_name	VARCHAR(100)	N	Arquitectura entrenada.
Status	VARCHAR(50)	S	Estado del entrenamiento, con valor por defecto In Progress.
Progress	FLOAT	S	Porcentaje de progreso, con valor por defecto 0.0.
Metrics_json	TEXT	N	Métricas serializadas del entrenamiento.
Started_at	DATETIME	S	Fecha de inicio, con valor por defecto CURRENT_TIMESTAMP.
Finished_at	DATETIME	N	Fecha de finalización.
Clave primaria			
Nombre	Columnas	Secuencia	
training_jobs_pkey	Id	Auto incremental de MySQL (AUTO_INCREMENT)	
Claves ajenas			
Nombre	Destino	Columnas	
training_jobs_user_id_fkey	users	User_id	
Claves únicas			
Nombre	Columnas		
N/A	N/A		
Restricciones			
Nombre	Columnas	Restricción	
training_jobs_user_id_not_null	User_id	Valor no nulo: todo entrenamiento debe asociarse a una cuenta válida.	

Tabla x - TB-05: training_jobs

Esta ficha debe leerse con una salvedad importante. La tabla training_jobs está definida en el esquema y se crea durante la inicialización, pero no se utiliza operativamente en la implementación actual: el flujo del laboratorio no inserta filas ni las consulta. La persistencia real del entrenamiento se realiza mediante el sistema de ficheros, en el directorio training_results, y el motor que lanza los entrenamientos (mlops_engine.py) no accede a la base de datos. Existen restos de lógica de actualización sobre la tabla en trainer_engine.py, un módulo que no participa en el flujo productivo, y un volcado histórico contiene filas antiguas, lo que sugiere un uso en versiones previas del proyecto. En consecuencia, esta tabla se documenta como una previsión del esquema cuya materialización se ha trasladado a los ficheros, sin atribuirle un uso que el código actual no realiza.

Los valores de los campos de dominio se validan en la capa de servicios y no mediante restricciones CHECK del esquema: el estado de la cola admite `queued`, `running`, `completed`, `failed` y `cancelled`; el tipo de trabajo admite `diagnosis`, `training` y `external_validation`; y la etiqueta de predicción admite `PNEUMONIA` y `NORMAL`. Esta decisión centraliza la lógica de validación en la aplicación y evita acoplar el esquema a la evolución de esos valores. La consistencia del esquema se apoya en tres niveles: MySQL aplica claves, obligatoriedad, unicidad y referencias; los routers y servicios aplican propiedad, roles y comprobaciones de entrada; y el worker coordina la relación temporal entre un trabajo encolado, sus artefactos y su resultado.

El esquema actual prioriza la trazabilidad directa con el código y la facilidad de puesta en marcha. Utiliza identificadores enteros, columnas JSON para parámetros variables y rutas de ficheros para los artefactos. Esta solución es suficiente para el alcance del TFG, pero tiene límites conocidos: no incorpora índices específicos para todas las consultas, no versiona automáticamente el esquema, no normaliza las métricas MLOps y no ofrece una entidad física que vincule cada fila de `job_queue` con una consulta o sesión concreta. Si el volumen de datos creciera, la evolución debería comenzar por los índices de las consultas frecuentes, una estrategia de migraciones versionadas y una política de retención, conservando la compatibilidad con el resto del sistema. Con ello, el esquema queda alineado con el estado real del código: MySQL sostiene las entidades y los estados relacionales, el sistema de ficheros conserva los artefactos, y la aplicación mantiene las reglas que no pueden expresarse solo con claves y tipos SQL.

19.2. Acceso a la capa de persistencia

El esquema descrito en el apartado 20.1 define dónde y cómo se almacenan los datos, pero no basta con ello: es necesario fijar también la forma en que los componentes del sistema leen y escriben esa información, porque de esa forma dependen tanto la corrección del comportamiento como la capacidad de evolucionar el sistema sin romper lo existente. En vitalXAI, el acceso a la capa de persistencia no pasa por una capa intermedia de mapeo objeto-relacional ni por un patrón de repositorios que abstraiga las consultas en clases independientes. La aplicación trabaja directamente con SQL parametrizado sobre conexiones obtenidas del pool de MySQLConnectionPool, que el módulo database.py construye de forma diferida y entrega a través de la función `get_db_connection()`. Esta elección responde a la simplicidad y a la trazabilidad: el código que consulta la base de datos es explícito, y la relación entre una consulta y la tabla a la que accede es inmediata.

La configuración del pool, su ciclo de vida y el proceso de inicialización del esquema se describen con detalle en el capítulo 19 dentro del subsistema SSOP-001. Este apartado se centra, por tanto, en dos aspectos complementarios: cómo se distribuye el acceso a las tablas entre los componentes del sistema, y qué consecuencias tiene esa distribución para la concurrencia, el rendimiento y la integridad de los datos. Ambos aspectos son necesarios para comprender el esquema no solo como una estructura estática, sino como un recurso que la aplicación utiliza de forma continuada.

Esta forma de trabajo tiene dos consecuencias directas. La primera es que el contrato entre la lógica de aplicación y la base de datos es pequeño y explícito: cualquier componente que necesite persistir obtiene una conexión, ejecuta sus consultas y la cierra, sin depender de una capa intermedia que abstraiga el SQL ni de clases de repositorio que oculten dónde se realiza cada operación. La segunda es que la correspondencia entre el esquema y el código es directa y trazable: cada tabla del modelo físico tiene asociados unos componentes concretos que la leen y la escriben, y esa asociación permite localizar con rapidez dónde se introduce o se modifica cada dato.

Tabla de persistencia	Operaciones principales	Componentes de acceso
<i>users</i>	Insertar cuenta, consultar credenciales y perfil, consultar rol.	routers/auth.py, routers/admin.py, routers/history.py, routers/trainer.py.
<i>consultations</i>	Insertar resultado de diagnóstico, consultar historial, renombrar y eliminar.	services/queue_worker.py, routers/history.py, routers/admin.py.
<i>job_queue</i>	Encolar trabajo, reclamarlo, actualizar su estado, cancelarlo.	routers/inference.py, routers/trainer.py, services/queue_worker.py, routers/queue.py.
<i>refresh_tokens</i>	Crear token, verificarlo, revocarlo.	services/auth_service.py, routers/auth.py.
<i>training_jobs</i>	Tabla prevista; sin acceso operativo en la implementación actual.	Ninguno (persistencia real por ficheros en training_results).

Tabla x - Distribución del acceso a las tablas del esquema

La tabla anterior permite verificar que cada tabla tiene un responsable claro en la capa de aplicación, y que la lógica de negocio que rodea a cada entidad reside en los componentes que la utilizan. Por ejemplo, la escritura de consultations es competencia del worker de la cola, que la realiza al finalizar un diagnóstico, mientras que la lectura y la gestión del historial corresponden al router de historial y, en el ámbito administrativo, al router de administración. La cola de trabajos concentra la mayor diversidad de operaciones, porque es el punto de encuentro entre las peticiones que encolan tareas y el worker que las ejecuta. Esta separación evita que la responsabilidad sobre una misma tabla se disperse sin criterio y facilita localizar dónde se introduce o se modifica cada dato durante la depuración y el mantenimiento.

Todas las operaciones de acceso siguen un ciclo de vida homogéneo, que conviene describir con precisión porque constituye el patrón que se repite en todos los componentes. En una lectura, el componente obtiene una conexión del pool, crea un cursor —normal o de diccionario, según necesite los resultados como tuplas o como objetos con nombres de columna—, ejecuta una consulta parametrizada en la que los valores viajan separados de la instrucción, procesa el resultado para transformarlo en la estructura que necesita la capa HTTP o de servicio, y cierra la conexión. En una escritura, la secuencia añade la confirmación explícita mediante `commit()` antes de cerrar, de modo que los cambios quedan persistidos de forma atómica para la operación. El uso de parámetros evita construir sentencias concatenando datos recibidos del usuario, una garantía que se aplica de forma uniforme en todas las consultas del sistema, desde la autenticación hasta la cola de trabajos, y que constituye la defensa principal frente a la inyección SQL.

Esta uniformidad tiene una implicación práctica sobre la concurrencia. Cada petición solicita una conexión del pool durante el tiempo estrictamente necesario para su operación y la devuelve al finalizar, lo que limita el número de conexiones simultáneas al tamaño configurado en `DB_POOL_SIZE`. Las operaciones que combinan acceso a la base de datos con procesamiento costoso —como la generación de artefactos, la ejecución de un modelo o la espera de servicios externos— deben ejecutar su consulta sin retener la conexión durante todo el cálculo, porque de lo contrario agotarían rápidamente el pool y provocarían esperas en el resto de las peticiones. El diseño del worker, que reclama un trabajo, ejecuta el procesamiento pesado fuera del bloque de la consulta y solo vuelve a la base de datos para actualizar el estado o persistir el resultado, respeta este principio.

El aislamiento de datos entre usuarios, por su parte, no depende solo de las claves ajenas del esquema. Las consultas del historial y de la cola filtran por el `user_id` obtenido de la sesión, de modo que un usuario no puede leer filas de otro aunque técnicamente la tabla las contenga. Las operaciones administrativas, que sí acceden a datos de terceros, comprueban previamente el rol del actor y se limitan a las funciones de supervisión definidas en el análisis. Esta doble comprobación —la clave ajena que garantiza la existencia del propietario y el filtro de aplicación que garantiza la autorización— es la que materializa en la práctica el requisito RF-005, y conviene tenerla presente porque la primera sin la segunda no bastaría para proteger los datos.

En cuanto al rendimiento, el esquema se apoya en los índices que MySQL/InnoDB crea de forma implícita para las claves primarias, las restricciones de unicidad y las claves ajenas. Estos índices cubren las consultas más habituales, que en su mayoría filtran por el identificador propio de cada tabla y por `user_id`: el historial se consulta por el usuario y se ordena por fecha; los trabajos se filtran por propietario y, en el worker, por estado; y los tokens se localizan por su hash. La cola de trabajos accede por estado, por tipo y por fecha de encolado al reclamar y actualizar trabajos; el volumen actual no exige un índice adicional, pero la evolución del sistema debería considerar índices sobre `job_queue.status` y sobre las columnas de fecha del historial si el número de trabajos y de consultas creciera de forma significativa. La carga de los modelos de inferencia, que no es una operación de base de datos, se resuelve en memoria dentro del motor de aprendizaje automático y no interviene en este apartado.

Conviene precisar, además, la relación entre el acceso a la base de datos y el acceso a los artefactos del sistema de ficheros. Cuando una consulta se completa, el worker escribe los ficheros en el sistema de ficheros y, a continuación, inserta en `consultations` las rutas que los referencian. La operación inversa, la lectura, recupera primero la fila y después entrega los ficheros desde las rutas almacenadas. Esta secuencia introduce una coordinación que debe ser correcta: si el fichero se genera pero la inserción falla, la fila no existe y el resultado no aparece en el historial; si la inserción se completa pero el fichero no llegó a escribirse, la fila quedaría referenciando un artefacto inexistente. El worker gestiona esta coordinación registrando el estado del trabajo y, en caso de error, dejando constancia en `job_queue` y evitando presentar como completada una consulta cuyos artefactos no están disponibles.

El acceso a la capa de persistencia queda, así, alineado con la naturaleza híbrida del almacenamiento descrita en el apartado 20.1. Las tablas conservan los datos estructurados que necesitan consultas y relaciones, y se accede a ellas mediante conexiones del pool y consultas parametrizadas; los artefactos grandes se escriben y se leen directamente en el sistema de ficheros, y la base de datos únicamente guarda las rutas que los referencian. Esta combinación mantiene la correspondencia entre el esquema físico y el código que lo utiliza, y deja identificadas las operaciones y los componentes responsables de cada tabla, de modo que la evolución del sistema pueda partir de un conocimiento preciso de dónde reside cada dato y de quién lo gestiona.

20. Diseño de los casos de uso

El diseño de los casos de uso constituye la etapa de la parte de diseño en la que las interacciones descritas en el capítulo 12 dejan de ser especificaciones de comportamiento y se convierten en decisiones técnicas concretas de implementación. El análisis explica qué debe hacer el sistema desde la perspectiva del usuario, mientras que el diseño de los casos de uso determina cómo lo hace internamente: qué componentes participan, qué servicios invocan, qué validaciones aplican, qué operaciones de persistencia realizan y cómo se comunican con la interfaz. Este capítulo recoge esas determinaciones para cada uno de los casos de uso de vitalXAI, de modo que la transición del análisis al código quede documentada de forma inequívoca y que un futuro mantenedor pueda localizar con rapidez dónde y cómo se materializa cada interacción (Jacobson, Booch, & Rumbaugh, 1999; Larman, 2004).

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño definida en el capítulo 18. Cada subsistema de diseño (SD-001 a SD-006) agrupa los casos de uso que comparten una responsabilidad funcional común, de acuerdo con la correspondencia establecida entre los subsistemas de análisis y los casos de uso en los capítulos 13 y 15. Para cada subsistema se presentan tres elementos: el diagrama de casos de uso, que reproduce la estructura de interacciones del análisis; los casos de uso reales, que recogen las decisiones técnicas de diseño de cada interacción; y los diagramas de interacción entre objetos, que muestran cómo colaboran los componentes del sistema para llevar a cabo los flujos más significativos de cada subsistema. Esta estructura es coherente con la organización que el propio diseño adoptó para los subsistemas y facilita la lectura transversal de la parte de diseño de la memoria.

El contenido de este capítulo se apoya en dos fuentes complementarias. Por un lado, las especificaciones de los casos de uso del capítulo 12, que definen el comportamiento observable y las condiciones que deben cumplirse. Por otro lado, la arquitectura de los subsistemas de diseño del capítulo 18 y los mecanismos de soporte del capítulo 19, que fijan los componentes, los servicios y las infraestructuras disponibles para materializar ese comportamiento. El diseño de cada caso de uso no introduce funcionalidades nuevas ni contradice el análisis: concreta, con las decisiones técnicas del proyecto, aquello que el análisis ya declaró, y lo hace de forma coherente con la implementación descrita en los capítulos precedentes.

20.1. Subsistema de diseño SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa el subsistema de análisis SS-001 y se ocupa de controlar el acceso a la plataforma y de gestionar la identidad de los usuarios registrados. Cubre el registro de nuevas cuentas, el inicio y el cierre de sesión, la renovación de credenciales y la identificación del usuario en las áreas privadas. Su responsabilidad no se limita a validar el formulario de acceso: establece la identidad que utilizarán el resto de los subsistemas para aplicar el aislamiento de datos entre cuentas y las comprobaciones de rol, por lo que constituye una condición transversal para el funcionamiento de SD-002, SD-003, SD-004, SD-005 y SD-006. Esta característica, ya señalada en el capítulo 18, explica que los demás subsistemas no reproduzcan la lógica de autenticación, sino que consuman la identidad que SD-001 establece.

El subsistema se apoya principalmente en `routers/auth.py`, que sirve las páginas de acceso y procesa los formularios recibidos desde la interfaz, y en `services/auth_service.py`, que concentra la lógica criptográfica y el ciclo de vida de las credenciales. Junto a ellos intervienen los mecanismos transversales registrados en `main.py`, como la protección CSRF, las cabeceras de seguridad y la limitación del endpoint de inicio de sesión. Los detalles de su arquitectura y de las decisiones de implementación se describen con profundidad en el capítulo 18; aquí se presentan, en los apartados siguientes, el diagrama de casos de uso del subsistema, los casos de uso reales con sus determinaciones técnicas y los diagramas de interacción entre objetos. Los casos de uso que agrupa son CU-001 a CU-004, tal y como se estableció en la especificación del capítulo 12 y en la correspondencia del capítulo 13.

20.1.1. Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-001 recoge las cuatro interacciones que el subsistema pone a disposición de los actores y es el mismo que se definió en el análisis para el módulo de gestión del acceso y de la cuenta, adaptado aquí al ámbito del subsistema de diseño. Dos de las interacciones —la creación de la cuenta (CU-001) y el inicio de sesión (CU-002)— las realiza el visitante, que accede a la plataforma sin autenticarse; el cierre de sesión (CU-003) lo realiza el usuario autenticado; y el cambio del idioma de la interfaz (CU-004) está disponible tanto para el visitante como para el usuario autenticado, porque el idioma puede adaptarse desde el área pública y desde el área privada. El diagrama se representa a continuación.

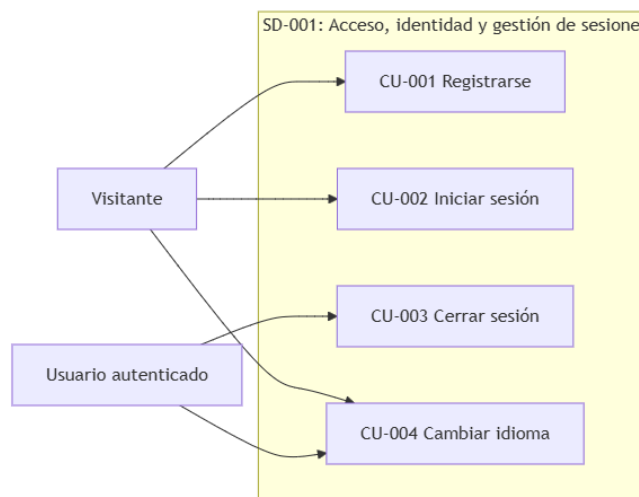


Ilustración x - Diagrama de casos de uso del subsistema SD-001

El diagrama distingue dos actores, en coherencia con la definición de actores del sistema del capítulo 12 (Jacobson, Booch, & Rumbaugh, 1999; Cockburn, 2001). El visitante representa a toda persona que accede sin haber iniciado sesión, y sus interacciones se limitan al registro, al inicio de sesión y al cambio de idioma. El usuario autenticado representa a todos los perfiles con cuenta registrada, incluido el administrador, porque las operaciones de gobierno de este último se recogen en el subsistema SD-005 y no en este diagrama. Esta decisión, ya adoptada en el análisis, evita que el administrador aparezca como un actor diferenciado en un subsistema donde no desempeña ninguna operación específica, y mantiene la coherencia entre el ámbito de SD-001 y el ámbito de SD-005.

Los cuatro casos de uso se representan como interacciones directas e independientes del actor con el sistema. No existe una relación de inclusión entre ellos, porque ninguno incorpora pasos obligatorios definidos por otro caso de uso, ni una relación de extensión, porque ninguno amplía de forma opcional el flujo de otro. La creación de la cuenta y el inicio de sesión comparten la necesidad de una identidad, pero se modelan por separado: el registro crea la identidad y el acceso la valida, sin que el segundo dependa del primero en la misma sesión. Esta estructura, idéntica a la del análisis, se mantiene en el diseño porque el flujo real de la plataforma no introduce relaciones nuevas entre estos casos de uso, y porque cada uno de ellos se implementa como un flujo propio e independiente en el subsistema.

Desde la perspectiva del diseño, el diagrama refleja una característica relevante del subsistema SD-001: actúa como puerta de entrada al resto de la plataforma, pero sus casos de uso no invocan servicios de otros subsistemas. Cuando el visitante se registra o inicia sesión, el subsistema opera exclusivamente sobre la tabla de usuarios y sobre las credenciales de sesión; cuando el usuario autenticado cierra sesión, revoca sus credenciales y elimina las cookies. El resto de los subsistemas consumen la identidad resultante mediante el mecanismo común de identificación descrito en el capítulo 18, pero no participan en estos flujos. Esta separación mantiene la cohesión del subsistema y facilita que las decisiones técnicas de SD-001 se describan de forma aislada en los apartados siguientes, sin dependencias circulares con el resto de la plataforma.

20.1.2. Casos de Uso Reales

Los casos de uso reales concretan cada interacción del diagrama en decisiones técnicas de implementación: qué componentes del subsistema intervienen, qué validaciones aplican, qué operaciones de persistencia realizan, qué respuesta devuelven al cliente y qué medidas de seguridad incorporan. Cada caso de uso real corresponde a un caso de uso del análisis y no añade funcionalidades nuevas, pero sí precisa determinaciones que el análisis dejaba abiertas, como el algoritmo de cifrado, el formato de las credenciales o los códigos de respuesta. Los cuatro casos de uso reales de SD-001 se describen a continuación.

CU-001 Registrarse

El registro se materializa en el endpoint POST /api/register del router routers/auth.py, que procesa los datos del formulario y traduce el flujo del análisis en una secuencia de validaciones y operaciones sobre la tabla users. El diseño mantiene la puerta de entrada autónoma del análisis: el alta no requiere la intervención de un administrador, y el proceso completo se resuelve en una única petición. Las decisiones técnicas del caso de uso real son las siguientes:

- **Validación de formato:** la función `_validate_register_inputs()` comprueba el nombre de usuario mediante una expresión regular de correo electrónico, exige una contraseña de al menos 8 caracteres y la presencia de nombre y apellidos. Ante cualquier incumplimiento, el sistema responde HTTP 400 con el código `validation_error` e indica el campo afectado, sin crear el registro.
- **Comprobación de unicidad:** antes de insertar, el sistema ejecuta una consulta de existencia sobre el nombre de usuario (`SELECT id FROM users WHERE username = %s`). Si el usuario ya está registrado, responde HTTP 400 con el código `user_exists` y abandona el proceso.
- **Cifrado de la contraseña:** la contraseña se convierte con `hash_password()` de `services/auth_service.py`, que aplica el algoritmo `bcrypt` con un salt generado en cada operación. La contraseña original no se transmite a ningún componente de persistencia tras completarse el hash, en coherencia con la postcondición del análisis de que nunca quede en texto plano.
- **Persistencia:** la cuenta se almacena mediante `INSERT INTO users (username, password_hash, first_name, last_name, role)`, que materializa la creación del registro con el rol indicado en el formulario.
- **Sesión tras el registro:** una vez creada la cuenta, el sistema genera el token de acceso y el token de refresco y los establece en las mismas cookies de sesión que el inicio de sesión, de modo que el visitante que acaba de registrarse accede directamente al panel. Esta determinación, que el análisis no fijaba —preveía redirigir al inicio de sesión—, se adopta en el diseño para eliminar un paso redundante al usuario que acaba de crear su cuenta.
- **Gestión de errores:** si la operación de persistencia falla, el sistema responde HTTP 500 con el código `server_error`, de modo que un fallo interno no se confunde con una condición de validación del formulario.

CU-002 Iniciar sesión

El inicio de sesión se materializa en el endpoint POST /login, protegido con la limitación @limiter.limit("5/minute") del mecanismo slowapi. El diseño conserva la protección descrita en el análisis: la verificación no distingue entre nombre de usuario y contraseña incorrectos, y el límite de peticiones bloquea temporalmente los intentos reiterados desde una misma dirección. Las decisiones técnicas del caso de uso real son las siguientes:

- **Verificación de credenciales:** el sistema recupera el registro mediante SELECT id, password_hash FROM users WHERE username = %s y comprueba la contraseña con verify_password(), que compara el valor introducido contra el hash almacenado con bcrypt.checkpw. Si la verificación falla, responde con una redirección a la página de inicio con un mensaje genérico que no revela qué parte de las credenciales era incorrecta.
- **Token de acceso:** create_access_token() de services/auth_service.py firma un JWT con el algoritmo HS256 que contiene el identificador del usuario (sub) y su fecha de expiración (exp), utilizando la clave JWT_SECRET_KEY del entorno y la duración JWT_ACCESS_EXPIRE_MINUTES.
- **Token de refresco:** create_refresh_token() genera un valor aleatorio mediante secrets.token_urlsafe, y en la tabla refresh_tokens solo se conserva su hash SHA-256 junto con el identificador del usuario, la fecha de expiración y el indicador de revocación. Esta decisión permite renovar la sesión sin guardar una credencial reutilizable en texto plano.
- **Cookies de sesión:** el token de acceso se establece en una cookie HttpOnly con SameSite=Lax, y el token de refresco en una cookie HttpOnly con SameSite=Lax y ruta restringida a /api/token/refresh. La restricción de ruta limita el envío del token de refresco al único endpoint que debe consumirlo.
- **Renovación de la sesión:** el endpoint POST /api/token/refresh ejecuta rotate_refresh_token(), que verifica el token actual, revoca el anterior y emite uno nuevo junto con un token de acceso renovado. El parámetro REFRESH_ROTATION_GRACE_SECONDS (60 segundos por defecto) tolera las peticiones concurrentes de renovación, y el uso de un token revocado fuera de ese periodo de gracia se interpreta como un posible robo: el servicio invalida todos los tokens del usuario.
- **Respuesta al cliente:** si las credenciales son válidas, el sistema redirige con HTTP 303 al panel de diagnóstico; el propio registro de la sesión se completa en el navegador mediante las cookies.

CU-003 Cerrar sesión

El cierre de sesión se materializa en el endpoint GET /logout, y su diseño refleja la doble naturaleza de las credenciales de sesión: el token de refresco se revoca de forma definitiva en la base de datos, mientras que el token de acceso se descarta del navegador y caduca por sí mismo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Revocación del token de refresco:** si existe una cookie de refresco, el sistema ejecuta `revoke_refresh_token()`, que actualiza la fila correspondiente de `refresh_tokens` marcando el indicador de revocación. A partir de ese momento, el endpoint de renovación rechaza el token, por lo que la sesión no puede prolongarse.
- **Eliminación de cookies:** el sistema elimina del navegador tanto la cookie del token de acceso como la del token de refresco, con la ruta restringida correspondiente, mediante las operaciones de borrado de cookies de la respuesta.
- **Respuesta al cliente:** el sistema redirige con HTTP 303 a la página de inicio de sesión. El acceso a las áreas privadas queda bloqueado de inmediato porque el token de refresco ya no es válido y el token de acceso, aunque aún no haya expirado, no se conserva en el navegador.

CU-004 Cambiar el idioma de la interfaz

El cambio de idioma se materializa en el selector de idioma que está presente tanto en las páginas públicas (inicio de sesión y registro) como en las privadas (panel de diagnóstico y laboratorio de entrenamiento), y combina un mecanismo de cliente con un servicio de traducciones del lado del servidor. El diseño conserva las postcondiciones del análisis: la preferencia persiste durante la sesión y el cambio no interrumpe el estado de navegación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Selección en el navegador:** el script `static/js/i18n.js`, mediante la función `changeLanguage()`, lee el idioma elegido en el selector, lo guarda en `localStorage['appLang']` y actualiza dinámicamente los textos de la interfaz marcados con `data-i18n`, sin recargar la página. Esta decisión hace que el cambio sea inmediato y preserve el estado de la vista actual.
- **Servicio de traducciones del servidor:** `services/lang.py` centraliza el diccionario de mensajes que genera el backend. La función `get_lang_from_cookie()` lee la cookie `appLang`, valida que el idioma esté dentro del conjunto permitido —español, inglés, chino e hindú— y utiliza el español como valor por defecto si la preferencia es inválida o está ausente.
- **Aplicación en los resultados generados:** los servicios que producen contenido traducible leen la misma cookie para alinear sus salidas con el idioma de la interfaz: las etiquetas de predicción del motor de diagnóstico, los títulos de los mapas de explicabilidad, los textos del informe PDF y los mensajes del asistente conversacional. De este modo, el idioma seleccionado por el actor se aplica de forma coherente en toda la plataforma.
- **Coherencia entre cliente y servidor:** el mecanismo separa la responsabilidad de la presentación, que pertenece al navegador, de la de los mensajes generados por el backend, que se resuelven con el diccionario de `services/lang.py`; ambos consultan la misma preferencia de idioma, lo que evita que la interfaz muestre un idioma distinto del que utiliza el sistema en sus respuestas.

20.1.3. Diagramas de interacción entre objetos

Los diagramas de interacción muestran cómo colaboran los componentes del subsistema para realizar cada caso de uso real. Siguiendo la notación de los diagramas de secuencia del UML (Larman, 2004), se representan los participantes reales del diseño —la interfaz del navegador, el router `routers/auth.py`, el servicio `services/auth_service.py` y la base de datos MySQL—, los mensajes que intercambian en el tiempo y las alternativas que se resuelven durante cada flujo. Las cajas de activación indican el periodo en que cada participante ejecuta una operación, y los bloques alternativos reflejan las decisiones condicionales descritas en los casos de uso reales. Los diagramas son fieles a la implementación: cada mensaje corresponde a una invocación real de los componentes, de modo que la secuencia puede contrastarse directamente con el código. La numeración automática de los mensajes facilita la correspondencia entre el diagrama y los pasos de cada flujo.

CU-001 Registrarse

El diagrama de la figura muestra el flujo completo del registro. La interfaz envía los datos del formulario al router, que ejecuta la validación de formato y, según su resultado, responde con el error correspondiente o continúa con la comprobación de unicidad. Si el usuario no existe, el router encarga al servicio el cálculo del hash con `bcrypt`, inserta el registro en la tabla `users` y, a continuación, solicita la creación de los dos tokens de sesión; el servicio conserva en la base de datos solo el hash del token de refresco. El flujo culmina con la respuesta de éxito y el establecimiento de las dos cookies, lo que materializa la decisión de diseño de que el visitante quede autenticado al completar el registro. Las respuestas de error se mantienen diferenciadas: la validación de formato devuelve el campo afectado, mientras que la existencia previa devuelve la condición de duplicidad, lo que permite a la interfaz mostrar mensajes precisos sin revelar información sensible.

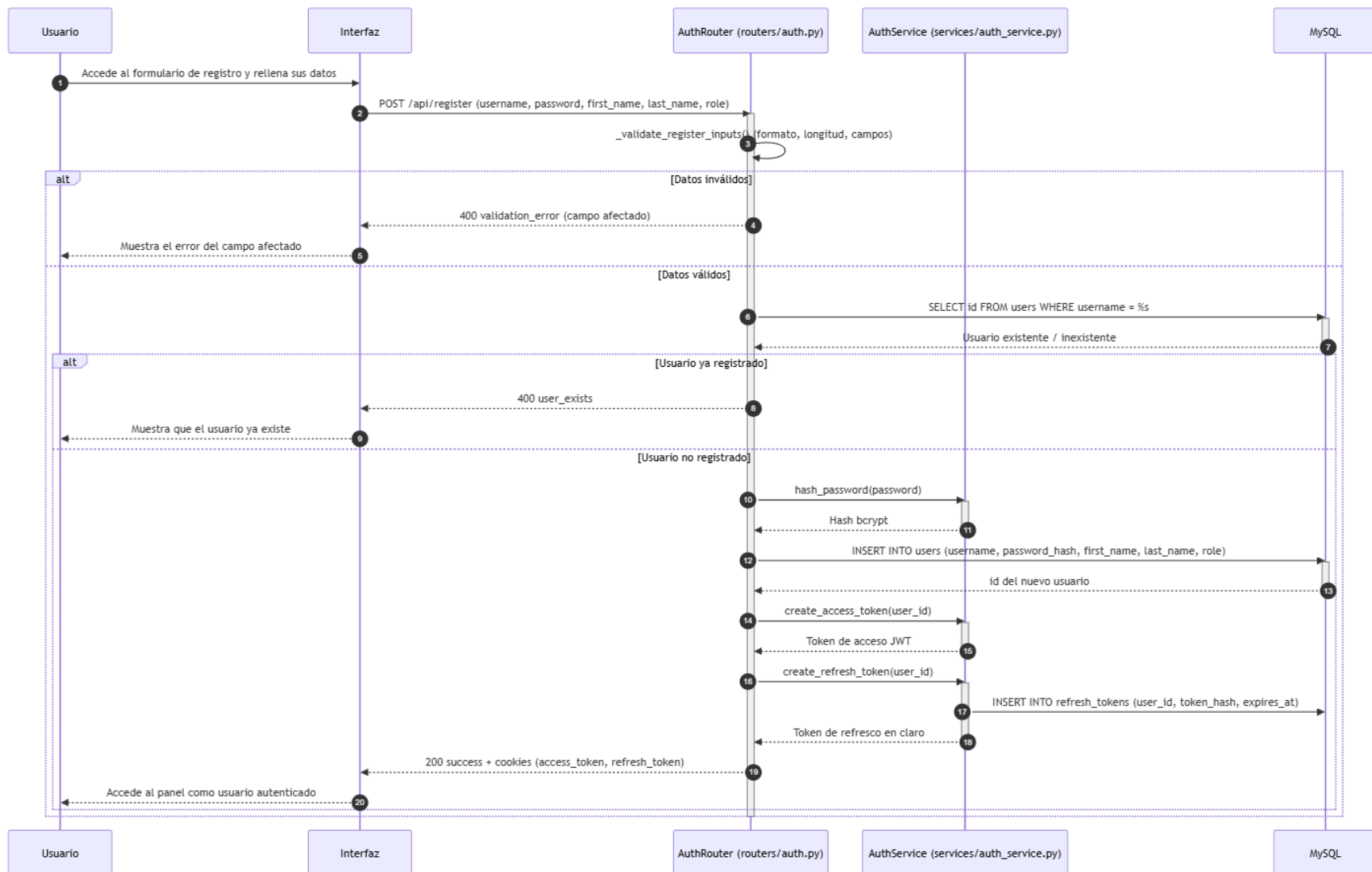


Ilustración x - Diagrama de secuencia del CU-001 Registrarse

CU-002 Iniciar sesión

El diagrama de la figura muestra el inicio de sesión. El router recupera el registro del usuario, y las dos alternativas que se suceden —usuario inexistente y contraseña incorrecta— responden con el mismo mensaje genérico, sin revelar qué parte de las credenciales falló, en coherencia con el análisis. Cuando la verificación con `bcrypt.checkpw` es correcta, el servicio genera el token de acceso y el token de refresco, y el router establece las cookies y redirige al panel. La secuencia refleja la separación de responsabilidades: el router se ocupa de la orquestación y de la respuesta HTTP, mientras que el servicio concentra la verificación criptográfica y la generación de las credenciales. El límite de peticiones del endpoint, aplicado por `slowapi` antes de procesar la solicitud, no se representa en el diagrama porque es una condición transversal que actúa en el punto de entrada del flujo.

El mecanismo de renovación de la sesión, que complementa al CU-002 y permite prolongar la sesión sin que el usuario vuelva a introducir sus credenciales, se muestra en la figura. Se activa cuando el token de acceso se acerca a su expiración: la interfaz envía el token de refresco al endpoint de rotación, y el servicio verifica su estado en la base de datos. Si el token está activo o fue revocado dentro del periodo de gracia, el servicio revoca el anterior, emite uno nuevo y lo devuelve; si fue revocado fuera del periodo de gracia, el sistema interpreta un posible robo de sesión y revoca todos los tokens del usuario, respondiendo la rotación como inválida. El diseño tolera así las renovaciones concurrentes sin tratar cada repetición como un ataque, pero invalida la sesión completa ante el uso real de una credencial revocada. Si la rotación es válida, el router emite un nuevo token de acceso y renueva las dos cookies; en caso contrario, responde con HTTP 401.

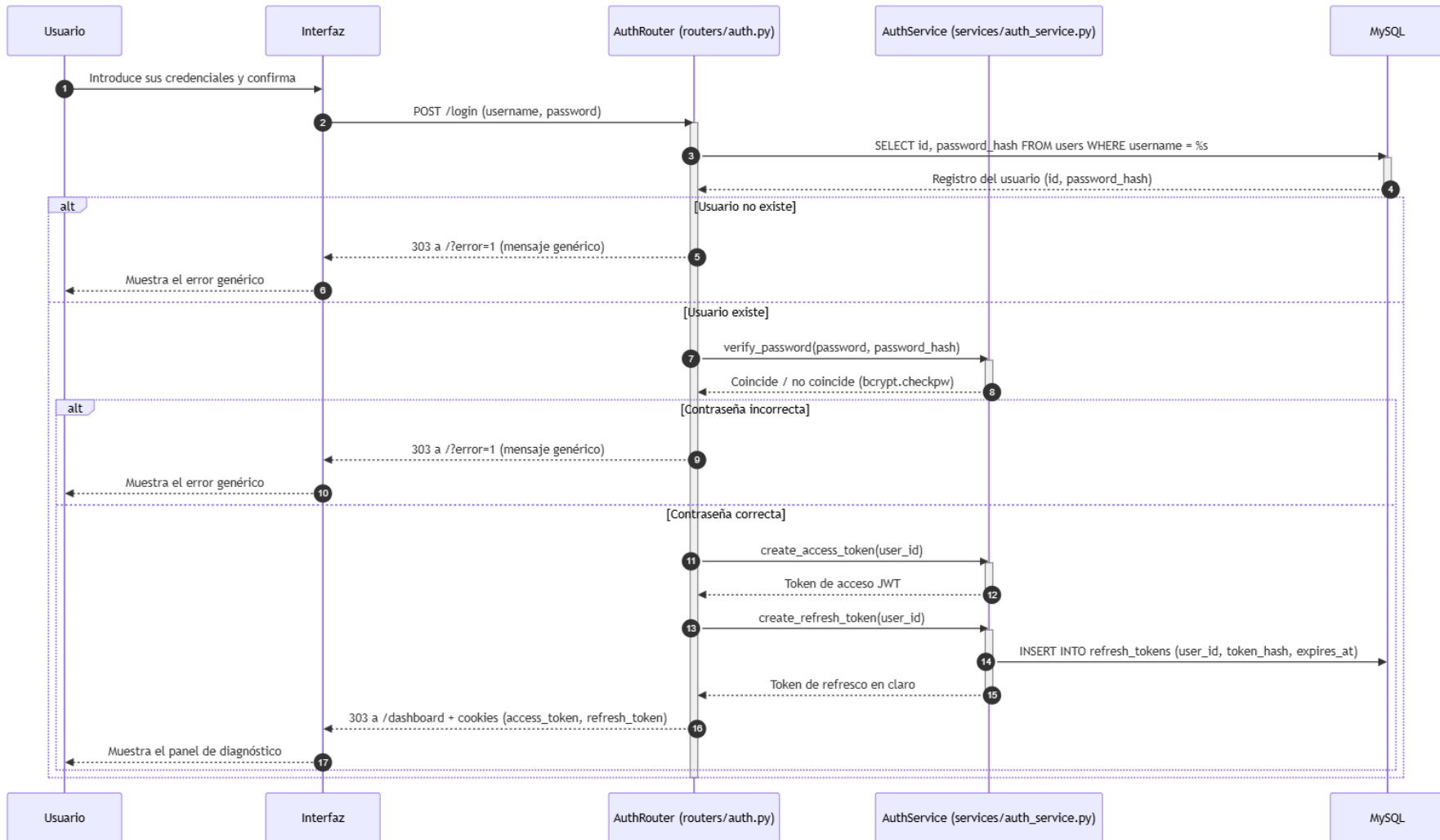


Ilustración x - Diagrama de secuencia del CU-001 Registrarse

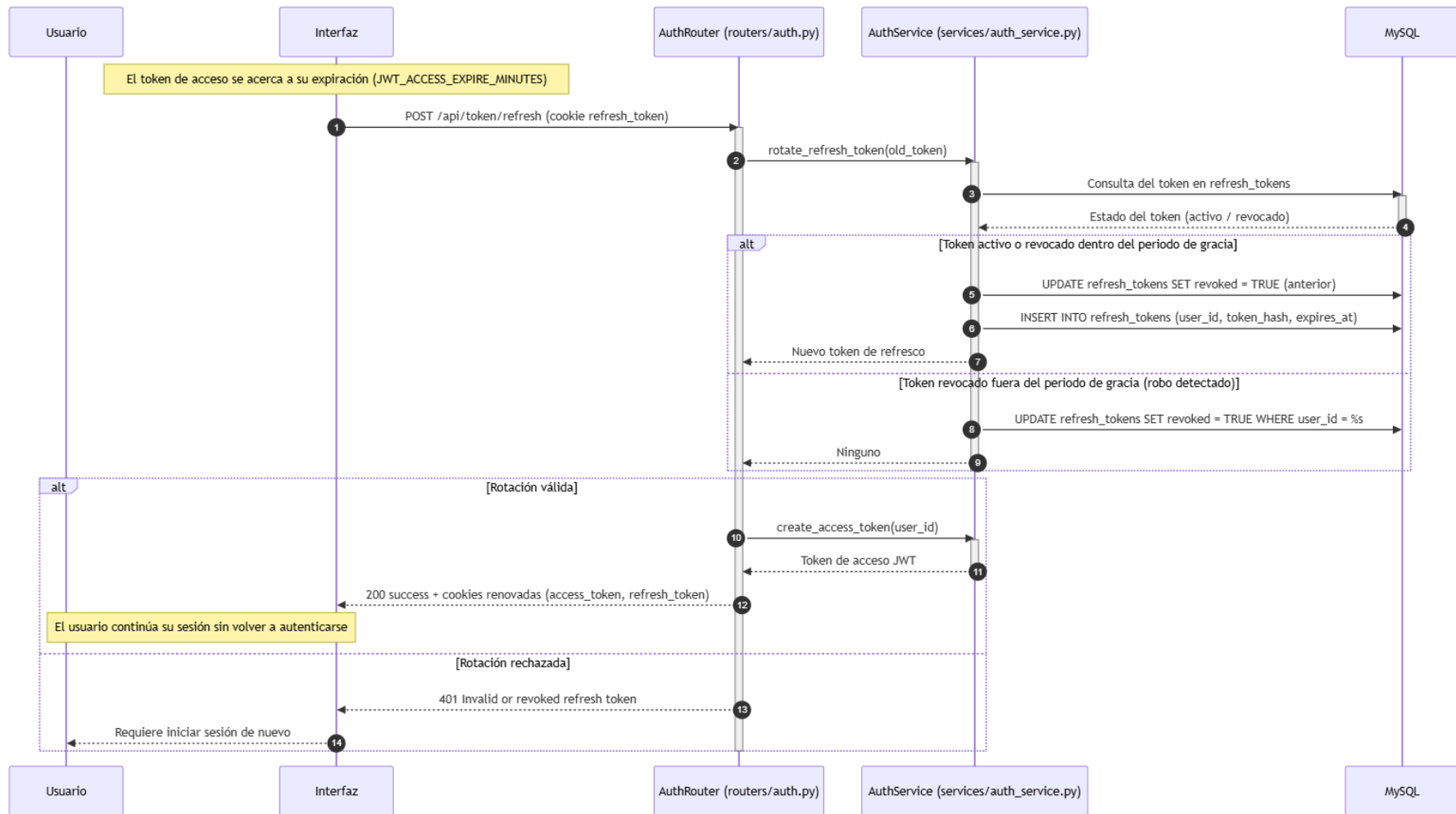


Ilustración x - Diagrama de secuencia de la renovación de la sesión

CU-003 Cerrar sesión

El diagrama de la figura muestra el cierre de sesión. El router solicita al servicio la revocación del token de refresco, que marca la fila correspondiente en la tabla `refresh_tokens`, y a continuación elimina las dos cookies de la respuesta y redirige a la página de inicio de sesión. La secuencia refleja la doble naturaleza de las credenciales: la revocación en la base de datos impide prolongar la sesión mediante la rotación, mientras que el borrado de las cookies descarta el token de acceso, que al ser un JWT autónomo no se puede revocar directamente y debe retirarse del navegador. La condición del diagrama cubre el caso en que no exista cookie de refresco —por ejemplo, una sesión sin rotación previa—, en cuyo caso la revocación se omite y la operación se limita al borrado de las cookies.

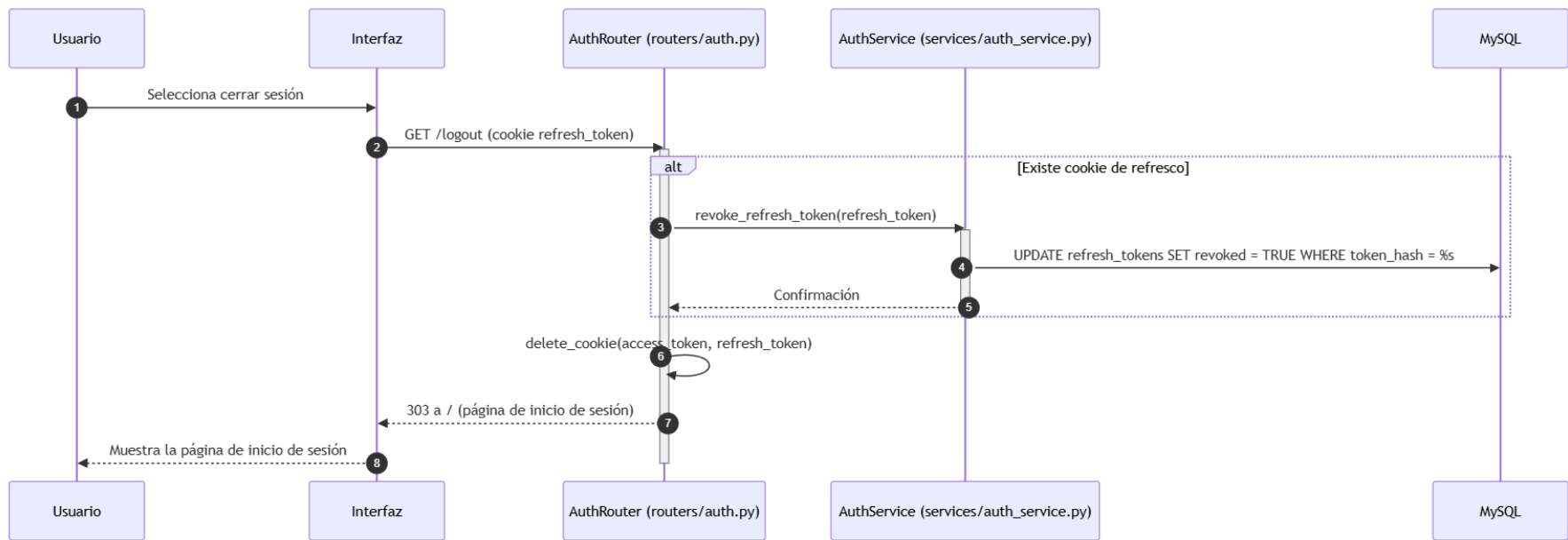


Ilustración x - Diagrama de secuencia del CU-003 Cerrar sesión

CU-004 Cambiar el idioma de la interfaz

El diagrama de la figura muestra el cambio de idioma. En el navegador, el script `i18n.js` lee la opción seleccionada, la guarda como preferencia y actualiza los textos de la interfaz sin recargar la página; este tramo se representa con auto-mensajes porque no interviene el servidor. En el tramo del servidor, los routers y servicios consultan al `LangService` el idioma aplicable mediante `get_lang_from_cookie()`, que valida la preferencia contra el conjunto permitido de idiomas y aplica el español por defecto cuando la preferencia es inválida o está ausente. La secuencia muestra la separación entre la presentación, que se resuelve en el navegador, y los mensajes generados por el backend, que se traducen con el diccionario centralizado de `services/lang.py`, de modo que las salidas del sistema —mensajes de error, etiquetas de predicción, títulos de los mapas de explicabilidad, informes PDF y mensajes del asistente— se mantienen alineadas con el idioma elegido por el actor.

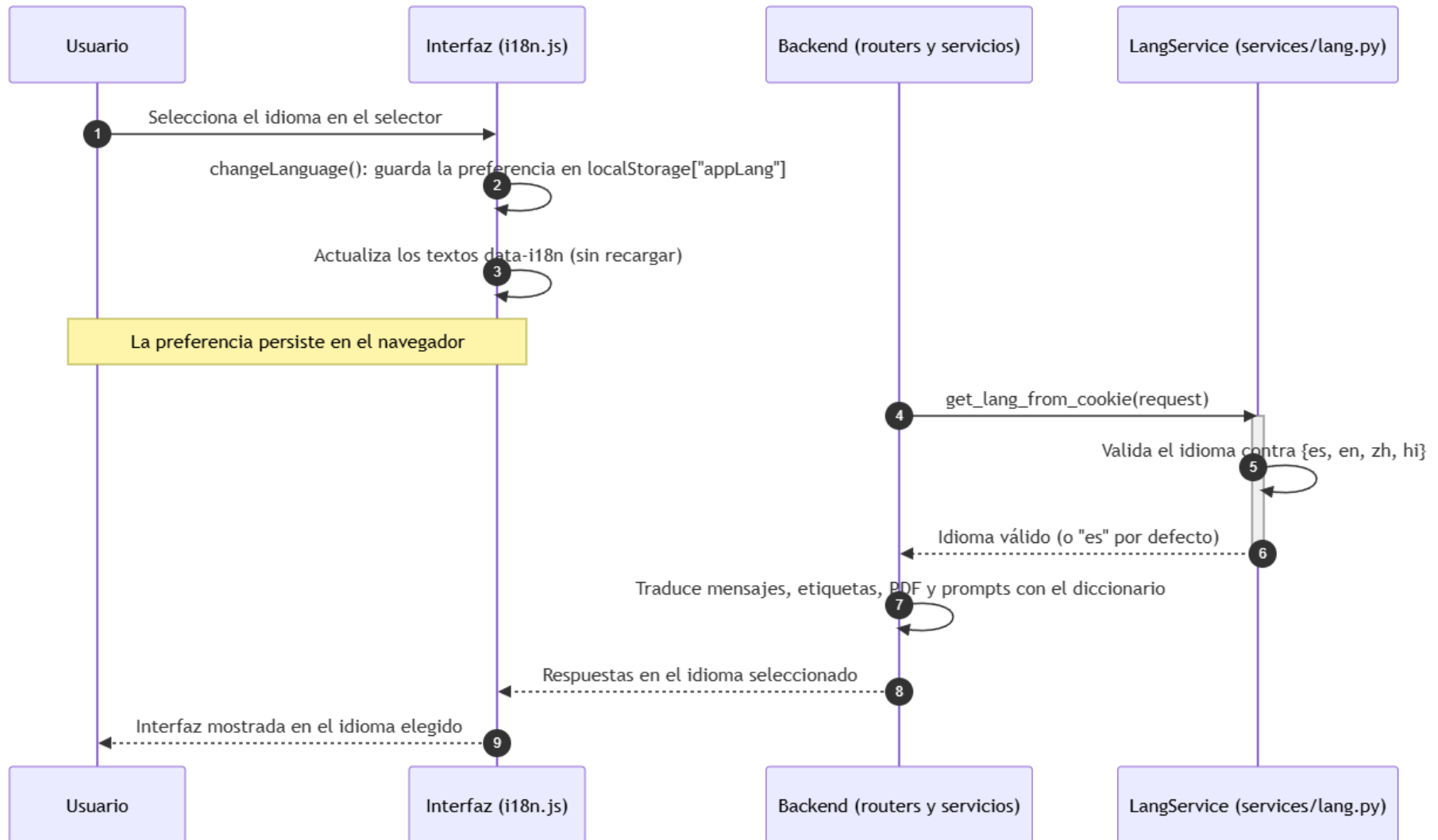


Ilustración x - Diagrama de secuencia del CU-004 Cambiar el idioma de la interfaz

20.2. Subsistema de diseño SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 corresponde al subsistema de análisis SS-002 y concentra el flujo clínico de la plataforma. Su función es recibir una imagen válida, asociarla al usuario autenticado, registrar una solicitud de diagnóstico, ejecutar la inferencia con la arquitectura seleccionada y conservar los artefactos que permiten consultar y justificar el resultado. Agrupa los casos de uso CU-005 a CU-010 y CU-037, que materializan el ciclo completo de una consulta clínica: el acceso al panel, la subida de la radiografía, la selección de la arquitectura, la solicitud del diagnóstico, la visualización del resultado y de los mapas de explicabilidad, y la generación del informe PDF. La gestión del historial de consultas (CU-011 a CU-014) no pertenece a este subsistema, sino a SD-003, que se describe en el apartado siguiente.

El subsistema se apoya en `routers/inference.py`, que actúa como fachada HTTP, y en un conjunto de servicios que se ejecutan fuera del ciclo de la petición: `services/ml_engine.py`, que prepara la imagen y produce la etiqueta y la confianza; `services/xai_generator.py`, que genera el mapa de explicabilidad adecuado al modelo; y `services/pdf_generator.py`, que construye el informe descargable. La ejecución de la inferencia se delega en la cola de trabajos y en su worker, que procesa el diagnóstico en segundo plano y persiste el resultado en la tabla `consultations`, de modo que la interfaz permanece operativa durante el análisis (RNF-020). El router, por tanto, no carga el modelo ni ejecuta la predicción: valida la petición, conserva la imagen y crea el registro de trabajo con un payload serializable que solo contiene el modelo, la ruta de la imagen y el idioma.

SD-002 participa en el sistema con tres dependencias claras. Se apoya en SD-001 para obtener la identidad del usuario a través del mecanismo común de identificación; utiliza la cola de trabajos del subsistema de ejecución asíncrona (SD-006) para procesar los diagnósticos sin bloquear la interfaz; y entrega a SD-003 los registros de `consultations` que este último consulta en el historial. Además, actúa como frontera de validación de ficheros: el tipo MIME y el tamaño se comprueban antes de escribir en disco, mientras que la validez científica de la imagen y la interpretación clínica del resultado quedan fuera del ámbito del subsistema, que no sustituye la valoración del profesional sanitario.

20.2.1. Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-002 recoge las siete interacciones que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de interfaz de diagnóstico asistido definido en el análisis. Se conservan únicamente los casos de uso del flujo clínico y del informe (CU-005 a CU-010 y CU-037), porque la gestión del historial (CU-011 a CU-014) pertenece a SD-003 y se representa en su propio diagrama. El diagrama no dibuja relaciones de extensión entre estos casos de uso: la cadena de navegación por la que el informe se genera desde el detalle de una consulta cruza la frontera del subsistema —CU-037 extiende CU-012, que pertenece a SD-003—, por lo que esa relación se describe en el texto y se refleja en los diagramas de interacción de ambos subsistemas. El diagrama se representa a continuación.

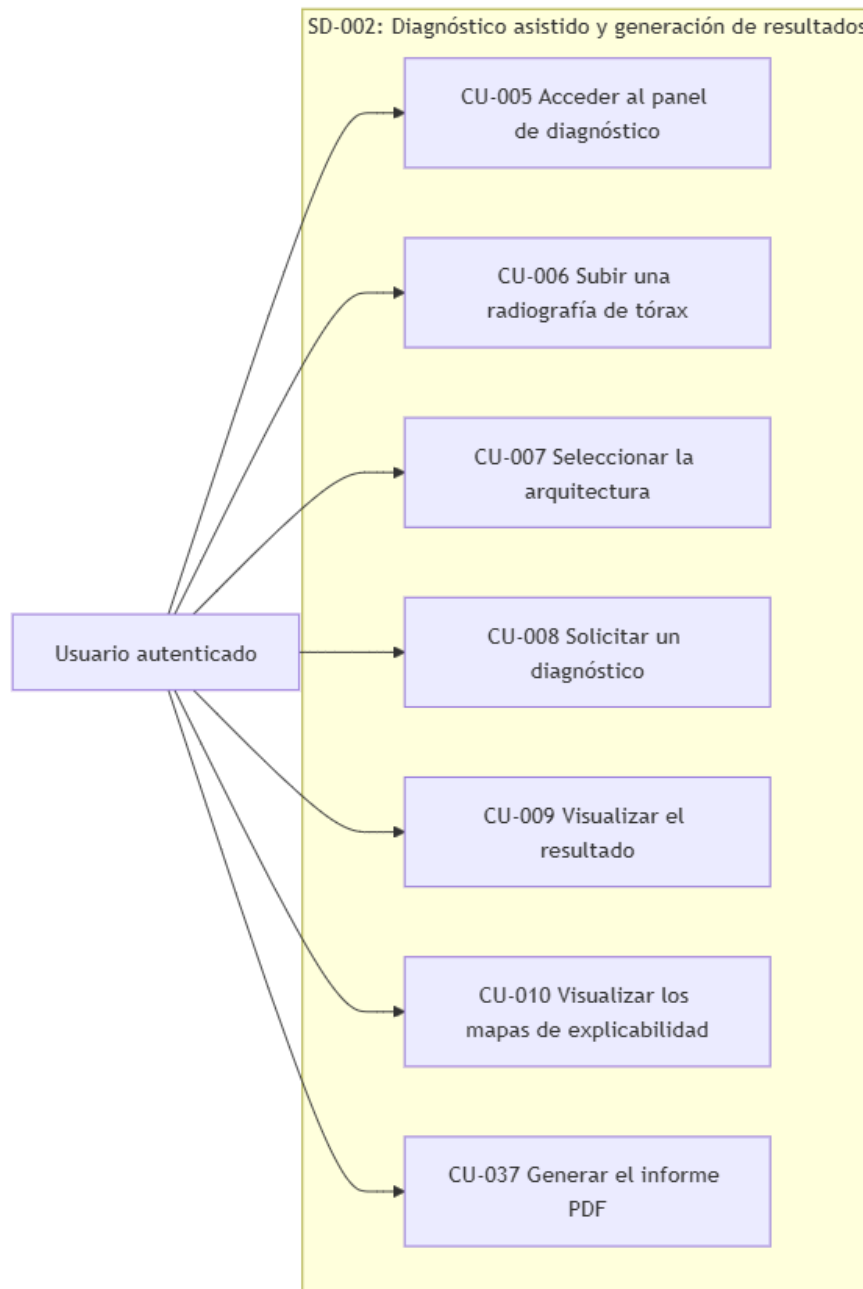


Ilustración x - Diagrama de casos de uso del subsistema SD-002

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los profesionales con cuenta registrada. El administrador no aparece como actor diferenciado porque sus operaciones de supervisión sobre las consultas se recogen en el subsistema SD-005. Los siete casos de uso se representan como interacciones directas e independientes del actor con el sistema. La subida de la imagen (CU-006), la selección de la arquitectura (CU-007) y la solicitud del diagnóstico (CU-008) forman la cadena de entrada del flujo clínico; la visualización del resultado (CU-009) y de los mapas de explicabilidad (CU-010) constituyen su salida; y el informe PDF (CU-037) su materialización documental. Los casos de uso CU-006, CU-007 y CU-008 se representan por separado, como en el análisis, aunque en la implementación se procesan en una única petición HTTP, tal y como se detalla en el apartado siguiente.

20.2.2. Casos de uso reales del subsistema

Los casos de uso reales de SD-002 concretan cada interacción del diagrama en decisiones técnicas de implementación. A diferencia de SD-001, cuyos flujos son síncronos, la mayor parte de este subsistema se organiza en torno al procesamiento asíncrono: la validación y el encolado se resuelven en el ciclo HTTP, mientras que la predicción, la generación de los mapas y del informe se ejecutan en el worker. Cada caso de uso real se describe a continuación, indicando los componentes que intervienen, las validaciones, las operaciones de persistencia y las respuestas al cliente.

CU-005 Acceder al panel de diagnóstico

El acceso al panel se materializa en el endpoint GET /dashboard del router routers/auth.py, que es la puerta de entrada al entorno clínico de la plataforma. El diseño aplica la condición transversal de SD-001: solo el usuario con una sesión válida alcanza el panel, y cualquier intento sin sesión se redirige al punto de entrada del sistema. Las decisiones técnicas del caso de uso real son las siguientes:

- **Verificación de la sesión:** el router obtiene el identificador del usuario mediante `get_user_id_from_token()` de SD-001, a partir de la cookie `access_token`. Si no hay sesión, responde con una redirección HTTP 303 a la página de inicio de sesión.
- **Carga del perfil:** con el identificador resuelto, se consultan el nombre y el rol del usuario en la tabla `users`, de modo que el panel se personaliza con el nombre completo y con el indicador de administrador cuando corresponde.
- **Control de caché:** la respuesta del panel incluye las cabeceras `Cache-Control: no-store`, que impiden que el navegador o los intermediarios sirvan la página privada desde caché tras un cierre de sesión.

CU-006 Subir una radiografía de tórax

La subida de la radiografía se materializa en la selección del archivo desde el panel de diagnóstico y en la validación que el router realiza al recibir el formulario. El diseño mantiene una doble frontera: la interfaz descarta de forma temprana los archivos claramente no válidos, y el router vuelve a comprobar el tipo y el tamaño antes de escribir nada en disco. Las decisiones técnicas del caso de uso real son las siguientes:

- **Selección del archivo:** la zona de carga del panel (`file-input`) permite adjuntar el fichero, y la interfaz habilita el envío únicamente cuando hay una imagen seleccionada.
- **Validación del tipo MIME:** el router comprueba que `file.content_type` pertenezca al conjunto permitido `{image/jpeg, image/png, image/jpg}`. Ante cualquier otro tipo, responde HTTP 400 con el mensaje de solo imágenes, sin escribir el fichero.
- **Validación del tamaño:** el router mide el archivo y rechaza con HTTP 400 los que superen los 10 MB, en coherencia con el límite declarado en el análisis.
- **Conservación de la imagen:** el fichero se guarda en el área `static/uploads` con un nombre basado en la fecha y hora de la recepción, lo que evita colisiones y no depende del nombre original del archivo.
- **Integración con la solicitud:** la imagen se envía en el mismo formulario que la solicitud del diagnóstico (CU-008), de modo que subida y solicitud se materializan en una única petición POST `/predict`.

CU-007 Seleccionar la arquitectura para el diagnóstico

La selección de la arquitectura se materializa en el selector de modelos del panel, que presenta las arquitecturas disponibles del proyecto —arquitecturas convolucionales como InceptionV3 o Xception y arquitecturas Transformer como DeiT, Swin y ViT—. El diseño reparte la responsabilidad de la selección entre la interfaz y el motor de aprendizaje, sin duplicar la lista de modelos en el router. Las decisiones técnicas del caso de uso real son las siguientes:

- **Elección en el panel:** el usuario elige la arquitectura en model-selector y el identificador del modelo viaja como campo del formulario de la solicitud (model_name).
- **Validación diferida:** el router no comprueba el modelo contra una lista, porque esa verificación pertenece al motor de inferencia: si los pesos de la arquitectura no existen, el worker marca el trabajo como fallido con un mensaje claro. Esta decisión evita mantener la lista de modelos en dos capas distintas.
- **Reutilización de modelos en memoria:** el motor conserva los modelos cargados en un diccionario global, de modo que la primera consulta con una arquitectura asume el coste de cargar los pesos, mientras que las posteriores reutilizan el modelo en memoria (RNF-019). Esta optimización pertenece al motor y no al router.

CU-008 Solicitar un diagnóstico

La solicitud del diagnóstico se materializa en el endpoint POST /predict de routers/inference.py, que traduce la petición del usuario en la creación de un trabajo de la cola. El diseño sigue el orden del análisis —validar, encolar y notificar—, pero traslada la ejecución de la inferencia al worker para que la interfaz no se bloquee. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación:** el router resuelve la identidad con SD-001 y responde HTTP 401 con el mensaje de no autenticado si no hay sesión válida.
- **Validación antes de encolar:** el tipo MIME y el tamaño se comprueban antes de crear el trabajo, de modo que una petición inválida no ocupa una posición en la cola y no consume recursos del worker.
- **Conservación de la imagen:** la imagen se guarda en static/uploads y la ruta resultante se incorpora al payload del trabajo.
- **Encolado:** se inserta un registro en job_queue con el tipo diagnosis, el identificador del usuario y un payload JSON serializable que solo contiene model_name, image_path y lang. Ni el modelo ni la imagen completa se serializan en la base de datos.
- **Posición en la cola:** el router calcula la posición del trabajo mediante la consulta de posición, que ordena los diagnósticos antes que el resto de tipos de trabajo.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el estado queued, el identificador del trabajo, su posición y un mensaje localizado, de modo que la interfaz informa al usuario de que el diagnóstico ha sido encolado.

CU-009 Visualizar el resultado del diagnóstico

La visualización del resultado se materializa en la consulta periódica del estado del trabajo y en el render del panel cuando este pasa a completado. El diseño separa la obtención del estado, que se resuelve con el router de la cola, de la presentación del resultado, que se apoya en el registro persistente por el worker. Las decisiones técnicas del caso de uso real son las siguientes:

- **Consulta del estado:** la interfaz ejecuta un sondeo de GET `/api/queue/status` cada dos segundos, con un límite de intentos, que refleja la ejecución en segundo plano del diagnóstico. El router de la cola devuelve los últimos trabajos del usuario, su estado y su posición.
- **Presentación del resultado:** cuando el trabajo pasa a completed, el panel muestra la predicción (Neumonía o Normal), el nivel de confianza y el modelo empleado, y refresca el historial de consultas.
- **Persistencia sin reinferencia:** el resultado queda registrado por el worker en la tabla consultations con su etiqueta, su confianza y las rutas de los artefactos, de modo que las visualizaciones posteriores no repiten la inferencia.
- **Localización:** la etiqueta de la predicción se adapta al idioma de la sesión mediante el servicio de idioma, en coherencia con el CU-004 de SD-001.

CU-010 Visualizar los mapas de explicabilidad

La visualización de los mapas se materializa en la imagen explicativa que el worker genera junto con la predicción y que la interfaz muestra en el detalle de la consulta. El diseño separa la generación, que pertenece al motor de explicabilidad, de la entrega, que se resuelve con los recursos estáticos de la aplicación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Generación del mapa:** `services/xai_generator.py` produce una figura con la radiografía original, el mapa de Saliency, el mapa de SmoothGrad y la superposición del Grad-CAM (para arquitecturas convolucionales) o del mapa de atención (para arquitecturas Transformer), en función de la arquitectura utilizada.
- **Conservación del artefacto:** la imagen se guarda en `static/results` con un nombre derivado del de la radiografía, y su ruta se persiste en la columna `xai_image_path` de la consulta.
- **Entrega al usuario:** la imagen se sirve como recurso estático y se muestra en el detalle de la consulta junto a la radiografía original, con la comprobación de propiedad que impone el historial: solo el usuario propietario accede a sus consultas y a sus artefactos (RF-005).
- **Inspección visual:** la interfaz permite ampliar la radiografía y el mapa en un visor, de modo que el profesional pueda inspeccionar con detalle las regiones que justifican la predicción.

CU-037 Generar el informe PDF del diagnóstico

El informe PDF se materializa durante el procesamiento del trabajo, mediante `services/pdf_generator.py`, que construye el documento descargable de la consulta. El diseño adelanta la generación del informe al momento del diagnóstico: cuando la consulta se completa, el documento ya existe, de modo que la descarga posterior no depende de una nueva operación del sistema. Las decisiones técnicas del caso de uso real son las siguientes:

- **Construcción del documento:** el generador produce un informe A4 con la fecha, el modelo utilizado, el diagnóstico, el nivel de confianza, la radiografía original y el mapa de explicabilidad, aplicando un color de diagnóstico acorde con el resultado.
- **Conservación del informe:** el PDF se guarda en `static/reports` con un nombre basado en la fecha, y su ruta se persiste en la columna `pdf_path` de la consulta.
- **Entrega al usuario:** el informe se sirve como recurso estático descargable desde el detalle de la consulta, de modo que el actor descarga el documento a su equipo tal y como declara el análisis.
- **Protección de la información:** el informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial (RGPD).

20.2.3. Diagramas de interacción entre objetos

Los diagramas de interacción de SD-002 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. A diferencia de SD-001, cuyas interacciones son síncronas, la colaboración del diagnóstico se divide en dos planos: el ciclo de petición HTTP, en el que participan la interfaz, el router y la base de datos, y el procesamiento asíncrono, en el que el worker reclama el trabajo de la cola y orquesta el motor de predicción, el generador de mapas y el generador de informes. Esta división es deliberada: el worker no acepta órdenes directamente del navegador, sino que reclama condicionalmente los trabajos que se encuentran en estado `queued`, lo que impide el doble procesamiento y mantiene la interfaz desbloqueada durante la inferencia. Los diagramas se presentan a continuación, con la misma notación de secuencia del UML empleada en el 21.1.3.

CU-005 Acceder al panel de diagnóstico

El diagrama de la figura muestra el acceso al panel. La interfaz solicita el panel y el router consulta a SD-001 la identidad contenida en la cookie de acceso. Si hay sesión válida, el router recupera el perfil del usuario y devuelve la vista del panel con el control de caché; si no la hay, redirige al punto de entrada del sistema. La secuencia refleja la condición transversal de la autenticación: el panel no se sirve en ningún caso sin identidad resuelta.

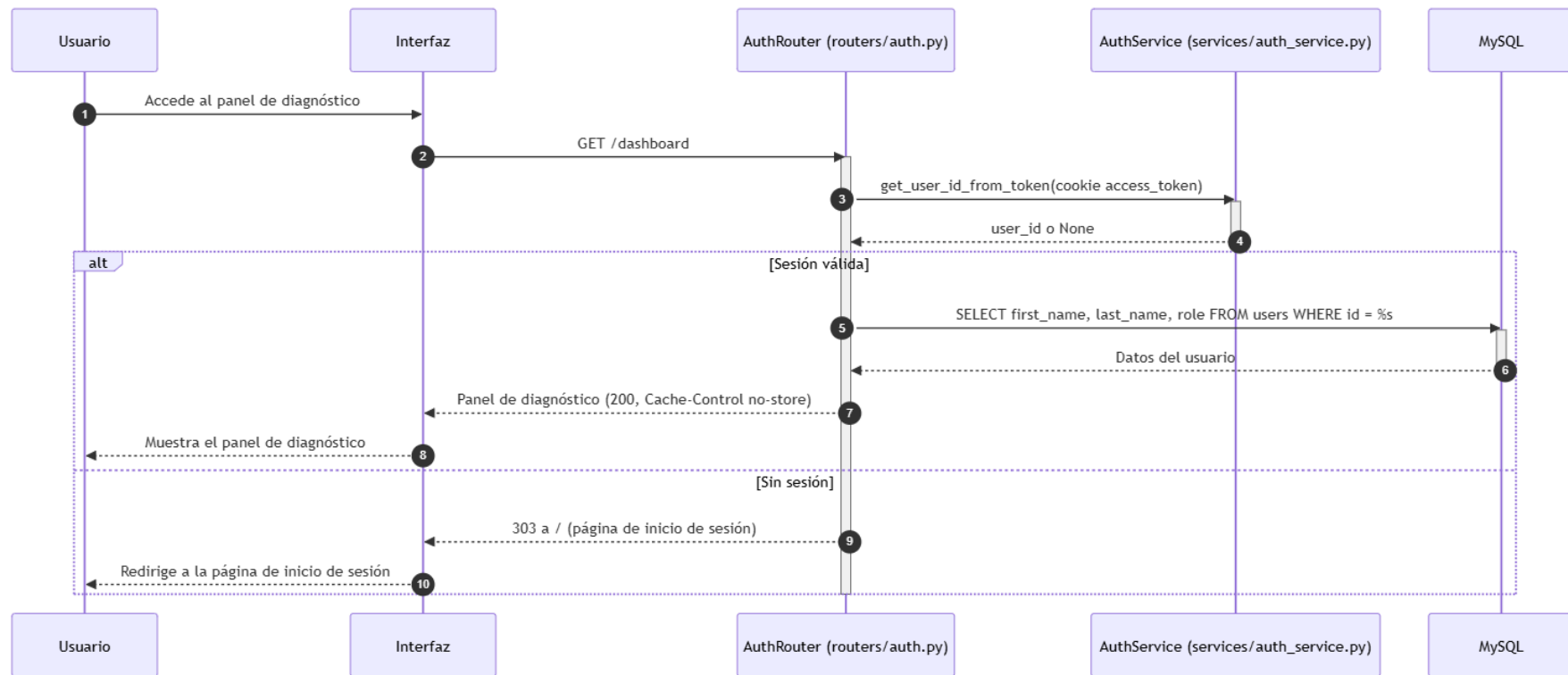


Ilustración x - Diagrama de secuencia del CU-005 Acceder al panel de diagnóstico

CU-008 Solicitar un diagnóstico

El diagrama de la figura muestra la solicitud del diagnóstico, que materializa también la subida de la radiografía (CU-006) y la selección de la arquitectura (CU-007), porque ambas se procesan en la misma petición. La interfaz adjunta la imagen y el modelo seleccionado, y el router resuelve la identidad. A continuación se suceden las validaciones de la frontera: sin sesión responde 401, y ante un tipo o un tamaño no permitidos responde 400 sin tocar la cola. Solo un fichero válido se conserva en disco y se encola: el router inserta el trabajo de tipo diagnosis, calcula su posición y devuelve el estado queued. El diagrama deja fuera la ejecución de la inferencia, que corresponde al procesamiento asíncrono de la figura 58.

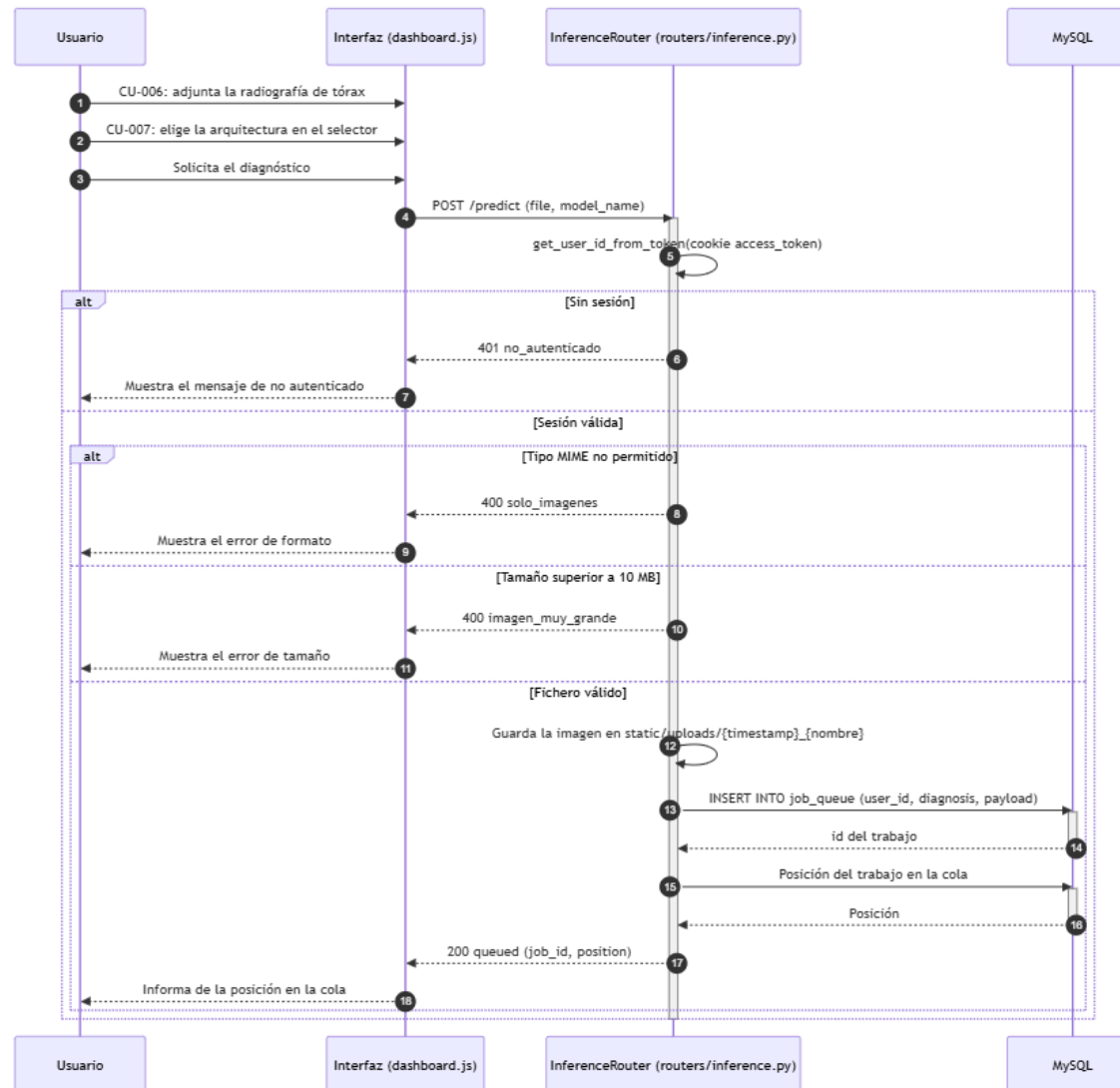


Ilustración x - Diagrama de secuencia del CU-008 Solicitar un diagnóstico

Procesamiento asíncrono del diagnóstico

El diagrama de la figura muestra la colaboración del worker con los servicios del subsistema. El bucle de la cola reclama el primer trabajo queued, con prioridad para los diagnósticos, y lo marca como running mediante una actualización condicional que solo afecta a una fila si el trabajo seguía en cola; de este modo, dos instancias no pueden procesar el mismo trabajo. A continuación el worker encarga al motor de predicción la inferencia, al generador de mapas la explicación visual y al generador de informes el documento PDF, y persiste el resultado en consultations con todas las rutas de los artefactos. Finalmente, marca el trabajo como completed con el resultado en formato JSON. Si cualquier paso falla, el trabajo pasa a failed con el mensaje de error limitado, de modo que un fallo del motor, de la imagen o del informe no se transforma en una consulta completada con datos ambiguos.

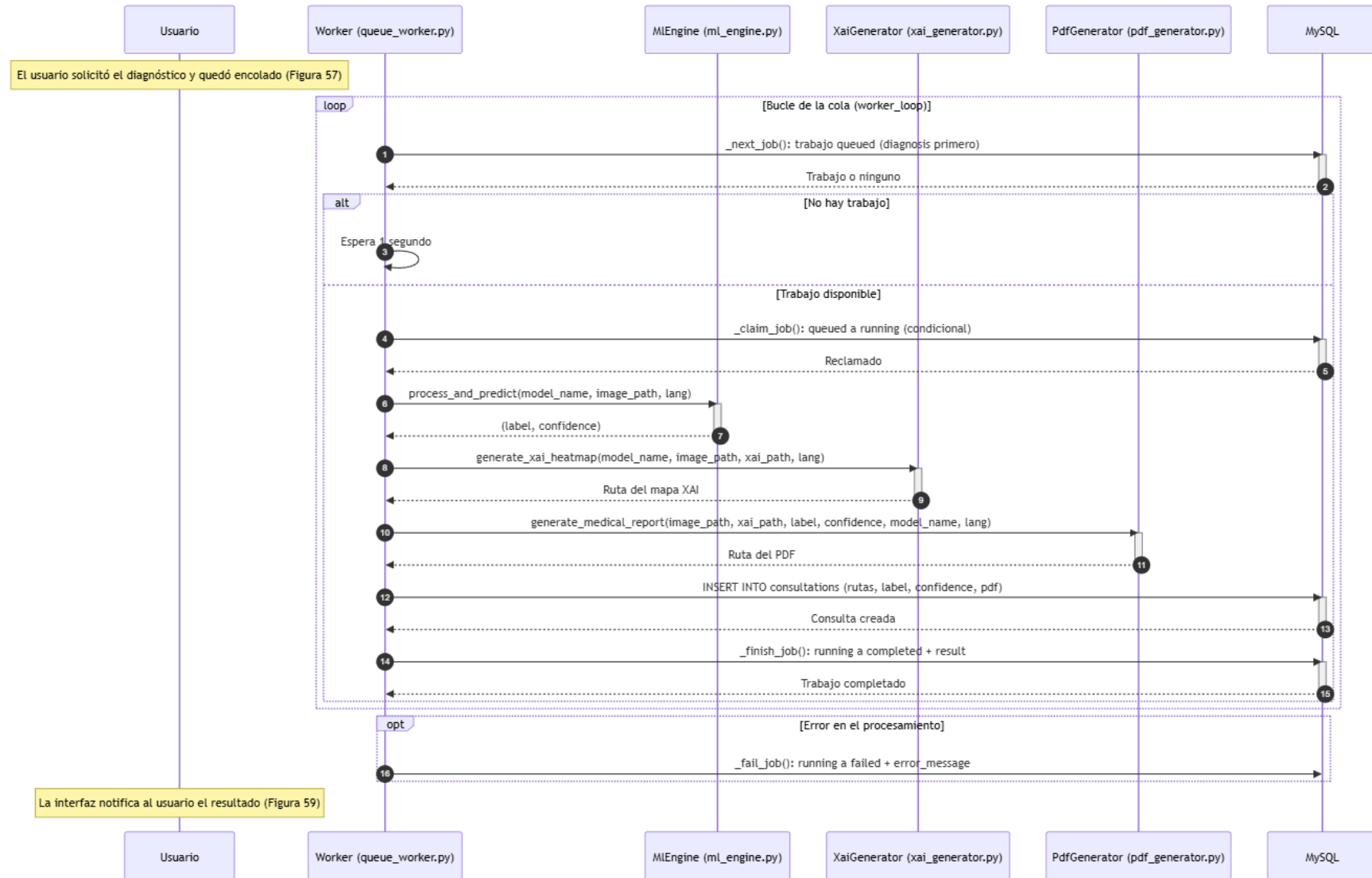


Ilustración x - Diagrama de secuencia del procesamiento asíncrono del diagnóstico

CU-009 y CU-010 Visualizar el resultado y los mapas de explicabilidad

El diagrama de la figura muestra la visualización del resultado y de los mapas. La interfaz sondea el estado del trabajo con el router de la cola; cuando el trabajo pasa a completed, muestra la predicción, la confianza y el modelo (CU-009) y refresca el historial. Desde el detalle de la consulta, la interfaz solicita la radiografía original y el mapa de explicabilidad al almacenamiento estático (CU-010). Si el trabajo terminó en failed, la interfaz muestra el error recibido. La secuencia refleja que el resultado no se calcula en la visualización, sino que se recupera del trabajo completado y de los artefactos persistidos.

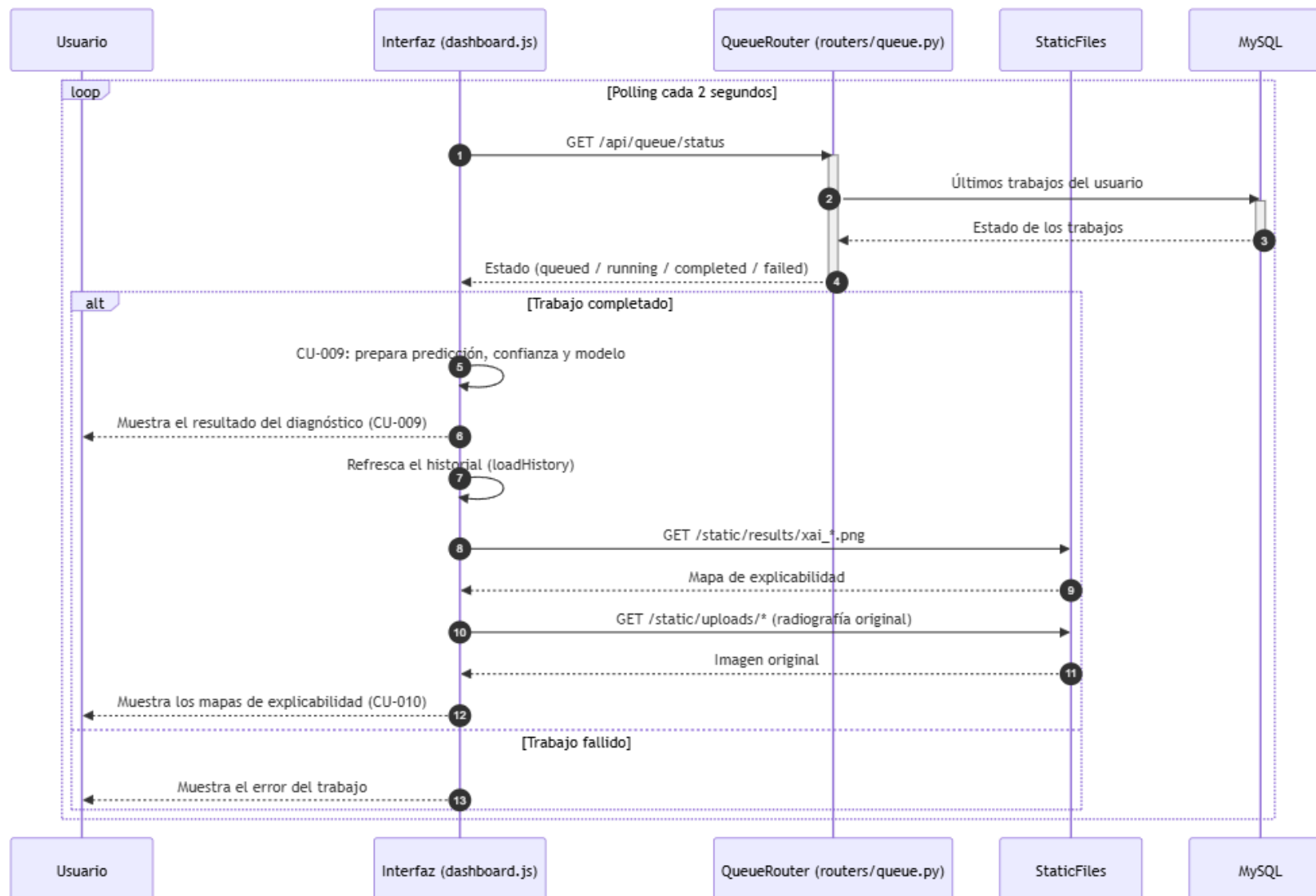


Ilustración x - Diagrama de secuencia del procesamiento asíncrono del diagnóstico

CU-037 Generar el informe PDF del diagnóstico

El diagrama de la figura muestra la entrega del informe PDF. La generación del documento ya ocurrió durante el procesamiento asíncrono, por lo que la interacción del actor se limita a recuperar la ruta del informe desde el historial y a descargar el recurso estático. La interfaz consulta la consulta en el router del historial, obtiene la ruta del PDF persistida y solicita el documento al almacenamiento estático, que lo entrega como archivo descargable. El diseño garantiza que la descarga siempre disponga del documento, porque el informe existe desde el momento en que la consulta se completa.

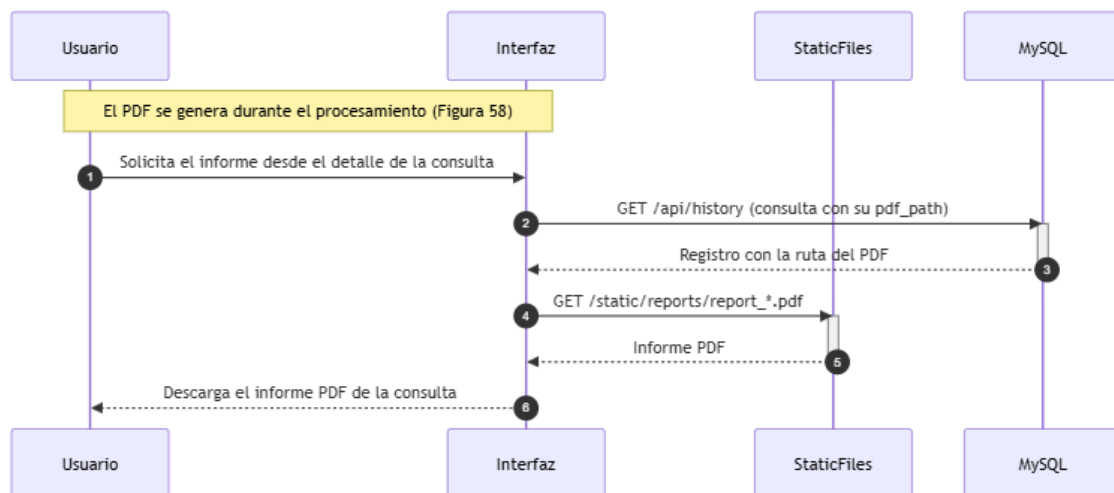


Ilustración x - Diagrama de secuencia del CU-037 Generar el informe PDF

20.3. *Subsistema de diseño SD-003: Historial y gestión de consultas*

El subsistema SD-003 corresponde al subsistema de análisis SS-003 y proporciona la recuperación y la gestión de las consultas ya realizadas. Su entidad persistente principal es la tabla consultations, que almacena las rutas de los artefactos, la arquitectura utilizada, la predicción, la confianza, el nombre mostrado al usuario y la fecha de la operación. Agrupa los casos de uso CU-011 a CU-014: la consulta del historial, la visualización del detalle, el renombrado y la eliminación. La relación con SD-002 es de productor-consumidor de resultados: SD-002 crea el registro de consultations cuando finaliza un diagnóstico, y SD-003 ofrece sobre él operaciones de consulta y organización, sin modificar en ningún caso la predicción ni sus artefactos.

El subsistema se apoya en routers/history.py, que expone la consulta del listado y las operaciones de actualización y eliminación, y en la interfaz del historial integrada en el panel de diagnóstico, que utiliza JavaScript para mostrar los resultados, cambiar el nombre visible de una consulta y solicitar su eliminación. La entidad de persistencia y la no ejecución del modelo condicionan el diseño: el subsistema recupera desde MySQL los metadatos y devuelve las rutas de la imagen original, del mapa XAI y del informe PDF, de modo que la visualización de una consulta anterior no repite la inferencia. El nombre mostrado al usuario se actualiza sobre patient_name, que funciona como etiqueta de organización de la consulta y no como identificación clínica del paciente, y la eliminación se implementa actualmente como una eliminación física de la fila, por lo que el subsistema no realiza un borrado lógico ni conserva automáticamente una auditoría de cada eliminación.

20.3.1. Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-003 recoge las cuatro interacciones que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de interfaz de diagnóstico asistido definido en el análisis (figura 4), limitada a los casos de uso de la gestión del historial. El diagrama conserva las relaciones de extensión de la cadena de navegación que el análisis declaró para este bloque: el detalle (CU-012) se alcanza de forma opcional desde el listado (CU-011), y desde el detalle pueden ejecutarse, de forma opcional, el renombrado (CU-013) o la eliminación (CU-014). El informe PDF (CU-037) también extiende CU-012, pero pertenece a SD-002, que se ocupa de su generación, por lo que esa relación se materializa en la frontera entre subsistemas y no se dibuja aquí. El diagrama se representa a continuación.

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los profesionales con cuenta registrada. El administrador no aparece como actor diferenciado, aunque las operaciones de este subsistema le afectan: la comprobación de propiedad contempla una excepción controlada para el rol administrativo, que permite supervisar consultas desde SD-005, pero esa excepción es una decisión de diseño del caso de uso real y no una interacción distinta del actor. Los cuatro casos de uso se representan como interacciones del actor con el sistema, con las relaciones de extensión que ordenan la navegación entre el listado, el detalle y las operaciones que se ejecutan desde este último.

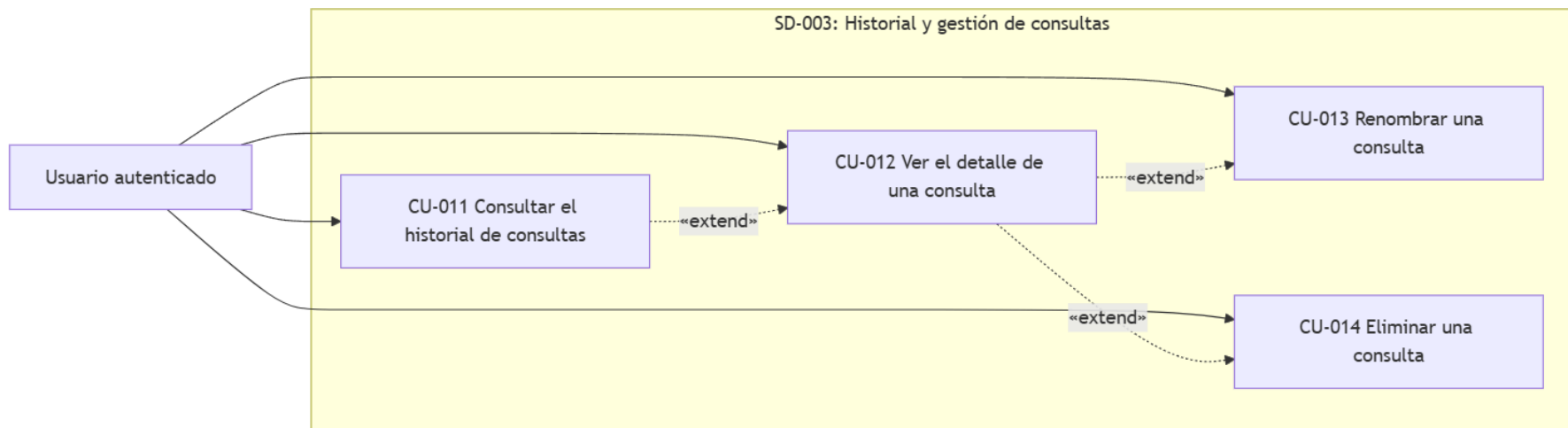


Ilustración x - Diagrama de casos de uso del subsistema SD-003

20.3.2. Casos de uso reales del subsistema

Los casos de uso reales de SD-003 concretan cada interacción del diagrama en decisiones técnicas de implementación. A diferencia de SD-002, las operaciones de este subsistema son síncronas y no requieren la cola de trabajos: recuperan o modifican registros ya persistidos, sin volver a ejecutar el modelo. Cada caso de uso real se describe a continuación, indicando los componentes que intervienen, las comprobaciones de propiedad, las operaciones de persistencia y las respuestas al cliente.

CU-011 Consultar el historial de consultas

La consulta del historial se materializa en el endpoint GET /api/history de routers/history.py, que devuelve exclusivamente las consultas del usuario autenticado. El diseño impone el aislamiento de datos en la propia consulta de persistencia: el filtro por user_id se aplica en el servidor y no se confía en la interfaz. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación:** el router resuelve la identidad con SD-001 y responde HTTP 401 con el mensaje de no autenticado si no hay sesión válida.
- **Filtrado por propietario:** la consulta recupera las columnas del historial desde consultations filtrando por WHERE user_id = %s, de modo que ningún usuario puede recibir las consultas de otro (RF-005).
- **Ordenación:** el listado se ordena por fecha descendente, de modo que las consultas más recientes aparecen en primer lugar.
- **Formato de los datos:** las fechas se convierten a una representación textual antes de formar la respuesta JSON, de modo que el navegador no necesita conocer el tipo de fecha de MySQL.
- **Presentación en la interfaz:** la vista del historial agrupa las consultas por modelo y construye las tarjetas con la imagen, la fecha, la etiqueta y la confianza, enlazando cada tarjeta con el detalle de la consulta.
- **Sin reinferencia:** la consulta no ejecuta el modelo; recupera los metadatos y las rutas de los artefactos ya persistidos, lo que reduce el coste de acceso al historial.

CU-012 Ver el detalle de una consulta del historial

La visualización del detalle se materializa en la interfaz del panel de diagnóstico, que construye la vista de detalle a partir de los datos que ya recibió en la consulta del historial. El diseño evita un endpoint específico de detalle para el usuario normal: como el listado ya está filtrado por propietario, los datos del detalle provienen de una respuesta que el servidor restringió al usuario, y los artefactos se sirven mediante las rutas estáticas configuradas en la aplicación. Las decisiones técnicas del caso de uso real son las siguientes:

- **Origen de los datos:** la interfaz conserva en la tarjeta del historial el identificador, las rutas de la imagen y del mapa, la etiqueta, la confianza, el modelo, el nombre y la fecha; al abrir el detalle, presenta estos campos sin necesidad de una nueva petición al servidor.
- **Artefactos visuales:** la vista muestra la radiografía original y el mapa de explicabilidad servidos como recursos estáticos, junto con el resultado del diagnóstico, su nivel de confianza, el modelo empleado y los metadatos de la consulta.
- **Comprobación de propiedad implícita:** la propiedad queda garantizada por la cadena de datos: el detalle solo puede construirse a partir de una consulta que el servidor ya devolvió al usuario propietario.
- **Acciones desde el detalle:** desde la vista de detalle se ejecutan, de forma opcional, las operaciones de renombrado (CU-013), eliminación (CU-014) y descarga del informe PDF (CU-037, de SD-002), en coherencia con las relaciones de extensión del diagrama.

CU-013 Renombrar una consulta del historial

El renombrado se materializa en el endpoint POST `/api/history/update_name` de `routers/history.py`, precedido de la solicitud del nuevo nombre en la interfaz. El diseño antepone la comprobación de propiedad a la modificación y valida el nuevo nombre antes de enviarlo al servidor. Las decisiones técnicas del caso de uso real son las siguientes:

- **Solicitud del nombre:** la interfaz muestra un diálogo con el nombre actual de la consulta y recoge el nuevo valor.
- **Validación del nombre:** la interfaz rechaza el envío si el nuevo nombre está vacío o no ha cambiado, de modo que no se envían peticiones innecesarias.
- **Comprobación de propiedad:** el router invoca `_check_consultation_ownership()`, que verifica que la consulta existe y pertenece al usuario solicitante, con la excepción controlada del rol admin que permite la supervisión desde SD-005. La comprobación distingue tres resultados: consulta inexistente, consulta de otro usuario y propiedad confirmada.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 cuando la consulta no existe y HTTP 403 cuando pertenece a otro usuario, de modo que la interfaz informa de la situación sin exponer detalles internos de la base de datos.
- **Actualización:** si la propiedad se confirma, el sistema ejecuta `UPDATE consultations SET patient_name = %s WHERE id = %s` y confirma la transacción, actualizando únicamente la etiqueta de organización de la consulta, sin tocar los datos técnicos de la inferencia.
- **Refresco de la vista:** la interfaz actualiza el nombre en la tarjeta del historial y en el detalle abierto, y recarga el listado para mantener la coherencia.

CU-014 Eliminar una consulta del historial

La eliminación se materializa en el endpoint POST /api/history/delete de routers/history.py, precedido de la confirmación del usuario en la interfaz. El diseño sigue el requisito del análisis de solicitar confirmación previa, en línea con el derecho de supresión del RGPD, y ejecuta la eliminación como un borrado físico de la fila. Las decisiones técnicas del caso de uso real son las siguientes:

- **Confirmación previa:** la interfaz solicita al usuario la confirmación de la eliminación antes de enviar la petición; si no se confirma, la operación se abandona y la consulta permanece intacta.
- **Comprobación de propiedad:** antes del borrado, el router invoca `_check_consultation_ownership()` con la misma lógica de los tres resultados —inexistente, ajena y propia— y con la excepción controlada del rol admin.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 y HTTP 403 en los casos de inexistencia y falta de permisos, y HTTP 200 cuando la eliminación se confirma.
- **Borrado físico:** la eliminación se ejecuta con `DELETE FROM consultations WHERE id = %s` y se confirma la transacción solo después de que MySQL la acepte. El diseño no conserva automáticamente una auditoría de la eliminación, tal y como se declaró en el capítulo 18.
- **Refresco de la vista:** la interfaz cierra el detalle abierto y recarga el historial, de modo que la consulta eliminada desaparece del listado junto con el acceso a sus artefactos.

20.3.3. Diagramas de interacción entre objetos

Los diagramas de interacción de SD-003 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. A diferencia de SD-002, todas las interacciones de este subsistema son síncronas: no interviene la cola de trabajos, porque las operaciones recuperan o modifican registros ya persistidos. El control de propiedad constituye el elemento central de los diagramas de renombrado y eliminación, donde la comprobación se resuelve antes de cualquier modificación y condiciona la respuesta al cliente. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-011 Consultar el historial de consultas

El diagrama de la figura muestra la consulta del historial. El usuario accede a su historial y la interfaz solicita el listado al router, que resuelve la identidad. Con sesión válida, el router recupera únicamente las consultas del usuario ordenadas por fecha descendente, formatea las fechas y devuelve el listado; la interfaz lo presenta agrupado por modelo. Si no hay sesión, el router responde 401 y la interfaz conduce al inicio de sesión. La secuencia refleja que el aislamiento de datos se aplica en la consulta de persistencia y no en la presentación.

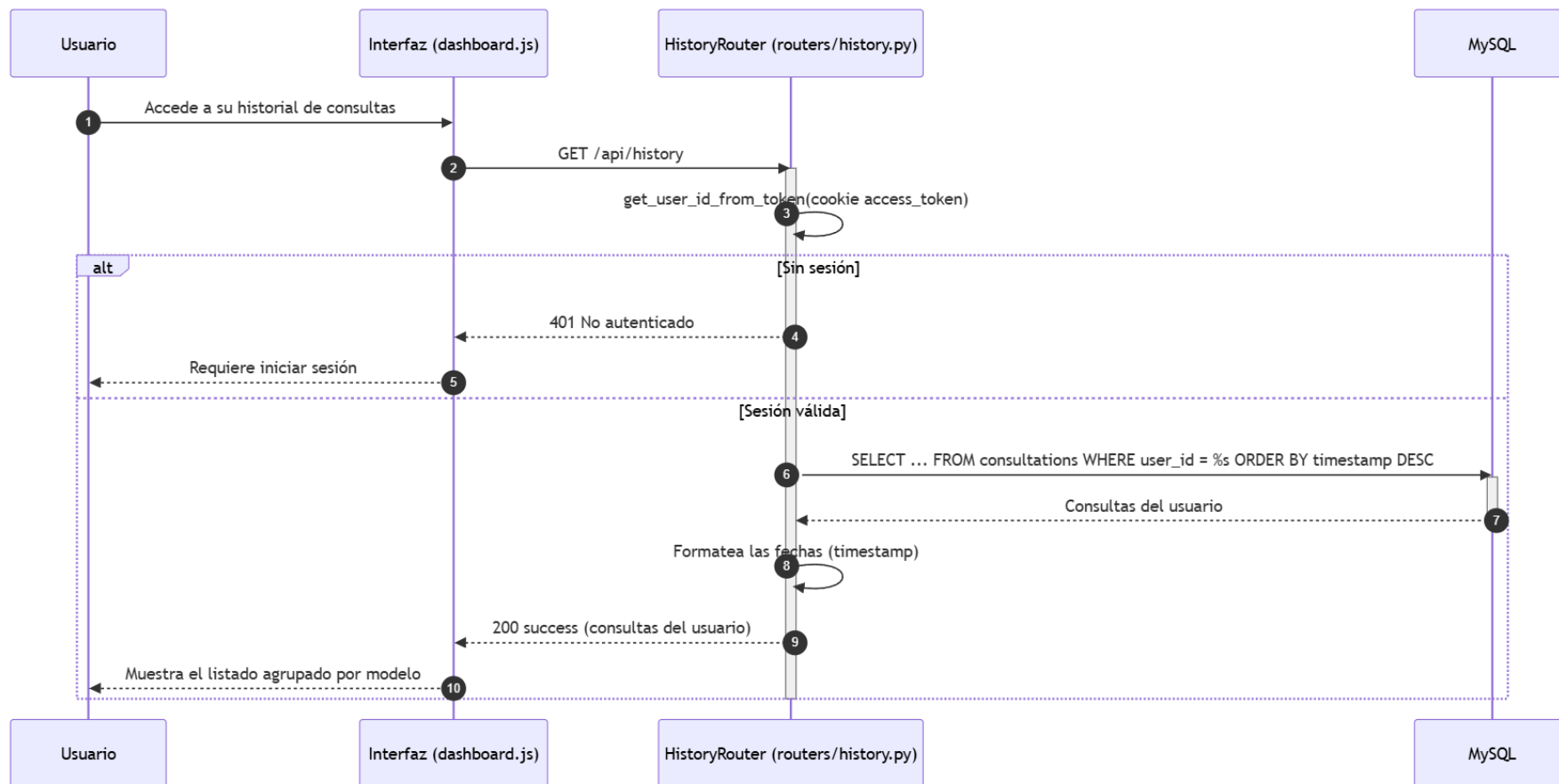


Ilustración x - Diagrama de secuencia del CU-011 Consultar el historial de consultas

CU-012 Ver el detalle de una consulta del historial

El diagrama de la figura muestra la visualización del detalle. El usuario selecciona una consulta del listado y la interfaz construye el detalle con los datos que ya recibió del historial, sin realizar una petición adicional al servidor. A continuación solicita al almacenamiento estático la radiografía original y el mapa de explicabilidad, y presenta la consulta completa al usuario. La secuencia refleja la decisión de diseño de no disponer de un endpoint de detalle para el usuario normal: la propiedad queda garantizada por la cadena de datos del listado, ya restringido al propietario.

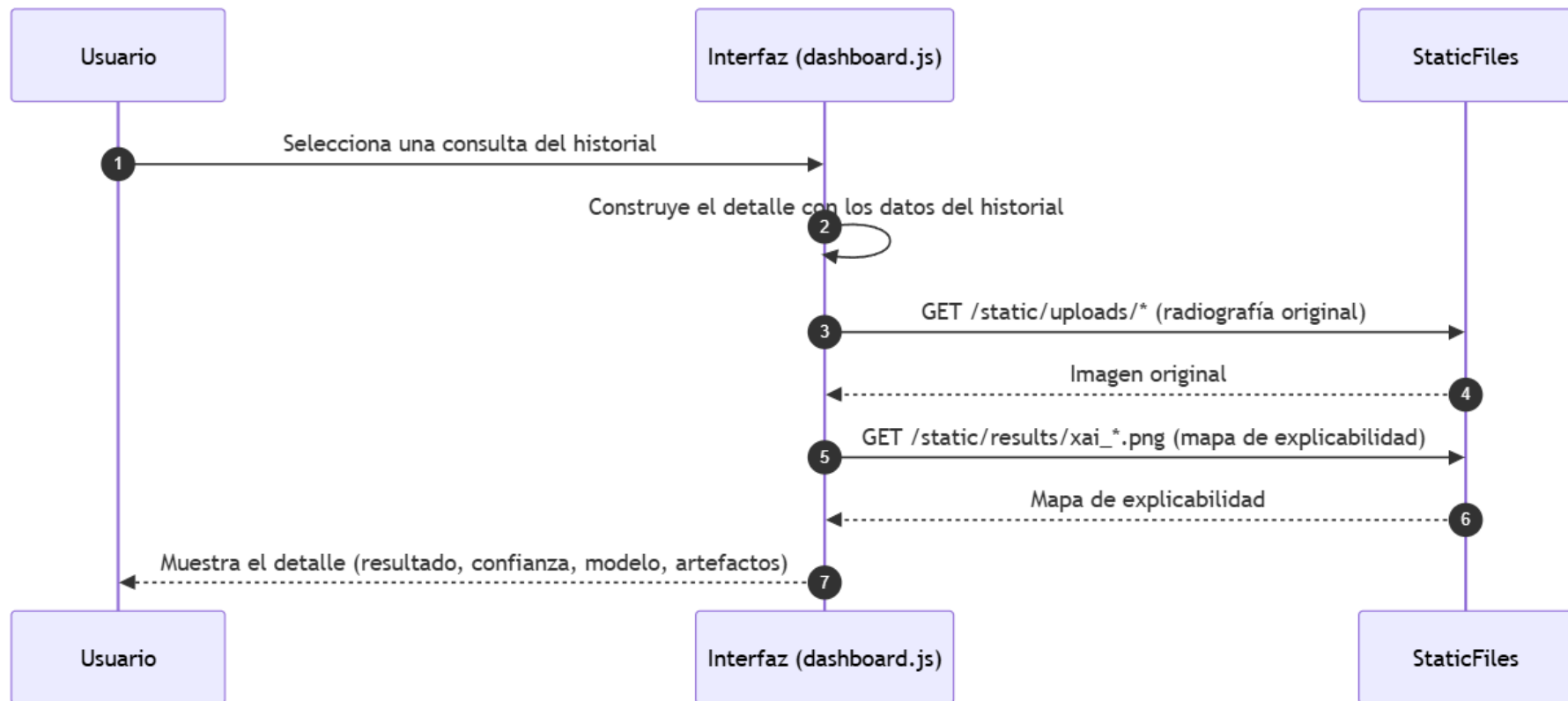


Ilustración x -Diagrama de secuencia del CU-012 Ver el detalle de una consulta del historial

CU-013 Renombrar una consulta del historial

El diagrama de la figura muestra el renombrado. El usuario indica el nuevo nombre en el diálogo de la interfaz, que valida que no esté vacío y envía la petición al router. El router resuelve la identidad y comprueba la propiedad de la consulta mediante `_check_consultation_ownership()`. Según el resultado, responde 404 si la consulta no existe, 403 si pertenece a otro usuario, o ejecuta la actualización del nombre cuando la propiedad se confirma y devuelve éxito; la interfaz muestra el nuevo nombre. La secuencia muestra que la modificación queda condicionada a la comprobación previa de propiedad.

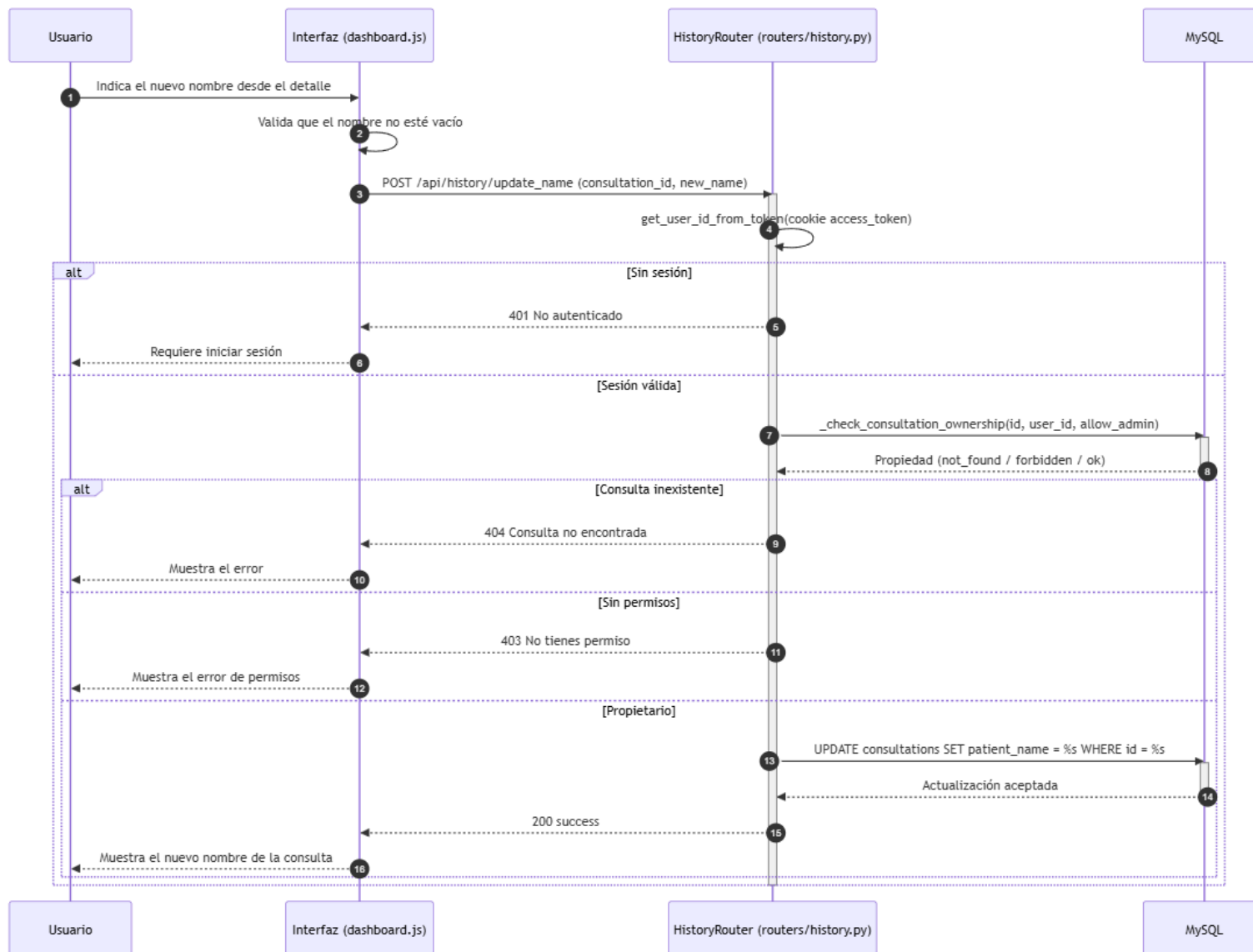


Ilustración x - Diagrama de secuencia del CU-013 Renombrar una consulta del historial

CU-014 Eliminar una consulta del historial

El diagrama de la figura muestra la eliminación. El usuario solicita eliminar la consulta desde el detalle y la interfaz pide confirmación; únicamente tras la confirmación se envía la petición al router. El router resuelve la identidad y comprueba la propiedad; con propiedad confirmada, ejecuta el borrado físico y confirma la transacción, y la interfaz cierra el detalle y refresca el historial. La secuencia refleja tanto la confirmación previa del análisis como el control de propiedad previo a la destrucción del registro.

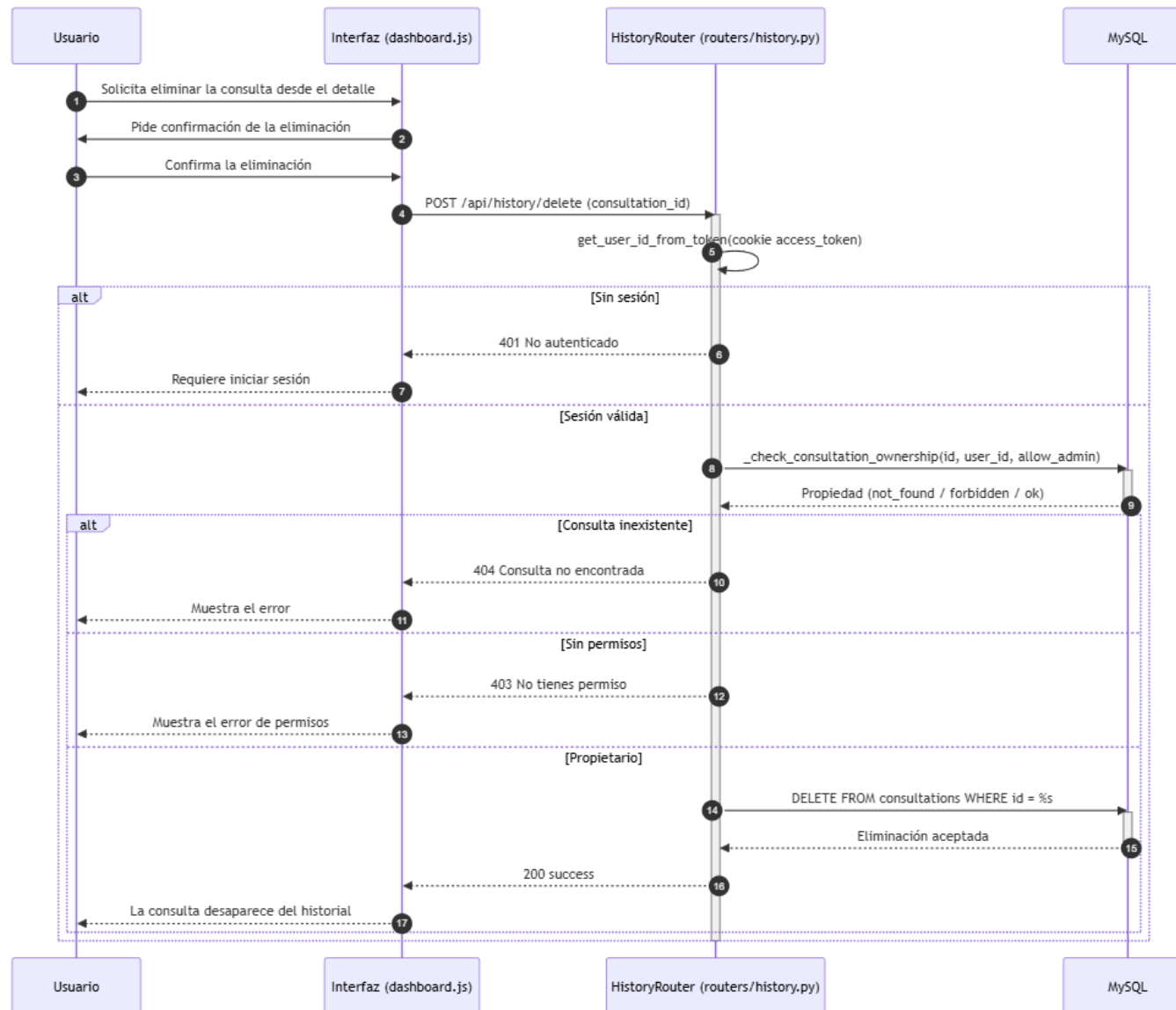


Ilustración x - Diagrama de secuencia del CU-014 Eliminar una consulta del historial

20.4. Subsistema de diseño SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 corresponde al subsistema de análisis SS-004 y constituye el bloque de mayor complejidad funcional de la plataforma. Gestiona la configuración conversacional del experimento, la selección del dataset, el entrenamiento de los modelos, la ejecución de los análisis de explicabilidad, la comparación estadística, la validación externa, la consulta de resultados y la generación de informes. Agrupa los casos de uso CU-015 a CU-030 y CU-039, que materializan los diecisiete requisitos del laboratorio: la configuración y el lanzamiento de los experimentos, la consulta de las sesiones y de sus resultados, el análisis XAI, la comparación estadística, la validación externa, la generación de informes y la gestión de las sesiones.

El subsistema se apoya en `routers/trainer.py`, que actúa como fachada ligera, y en un conjunto de servicios especializados: `services/chatbot_service.py`, que resuelve la configuración conversacional del experimento mediante el asistente externo; `services/ml_ops_engine.py`, que organiza las sesiones del laboratorio y resuelve la lectura y escritura de los resultados en el directorio `training_results`; los scripts de entrenamiento de `pneumoniacnn-main/code`, que ejecutan el pipeline de preparación, entrenamiento, análisis XAI y comparación estadística; y `services/pdf_generator_ml_ops.py`, que genera los informes consolidados de la sesión. La ejecución de las tareas de larga duración se delega en la cola de trabajos y en su worker, de modo que el laboratorio conserva una interfaz interactiva aunque el entrenamiento tarde mucho más que una petición HTTP normal.

El diseño de SD-004 introduce dos decisiones que condicionan su arquitectura. En primer lugar, la persistencia es híbrida: MySQL conserva la cola de trabajos y el estado, mientras que el sistema de ficheros alberga los artefactos de cada sesión, de modo que el identificador de la sesión se conserva tanto en la fila del trabajo como en el directorio de resultados. En segundo lugar, existen dos canales de ejecución diferenciados: el entrenamiento y la validación externa se procesan a través de la cola de trabajos, mientras que la comparación estadística se programa con las tareas en segundo plano de FastAPI (`BackgroundTasks`). La regla central del subsistema es la propiedad de las sesiones: antes de mostrar resultados, eliminar o renombrar una sesión, el router invoca la comprobación de propiedad de `ml_ops_engine`, con la excepción controlada del administrador cuando la ruta lo permite.

20.4.1. Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-004 recoge las diecisiete interacciones que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de laboratorio de experimentación MLOps definido en el análisis (figura 5), limitada al ámbito del subsistema de diseño. El diagrama conserva las relaciones del análisis: la inclusión de la configuración conversacional en el lanzamiento del experimento (CU-018 incluye CU-016, porque la ejecución no comienza hasta que el router dispone de una configuración completa y válida), y las extensiones que ordenan la navegación entre la consulta de sesiones y las vistas de resultados, ranking, comparativa e informe. El diagrama se representa a continuación.

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los profesionales con cuenta registrada. La estructura de los casos de uso refleja el ciclo de vida del laboratorio. La entrada se compone de CU-015 a CU-018: el acceso al laboratorio, la configuración conversacional con el asistente, la selección de la carpeta del dataset y el lanzamiento del experimento, que incorpora la configuración conversacional como paso obligatorio mediante la inclusión. La consulta se organiza desde las sesiones (CU-019), desde las que se accede de forma opcional a los resultados de un modelo (CU-020), al ranking (CU-022), a la comparativa estadística (CU-023) y al informe PDF (CU-028); desde los resultados de un modelo se alcanzan sus mapas de explicabilidad (CU-021), y desde la comparativa se solicita su recálculo (CU-024). El resto de operaciones —la ejecución del análisis XAI (CU-025), la validación externa (CU-026 y CU-027), el renombrado y la eliminación de sesiones (CU-029 y CU-030) y la comprobación de la limitación de entrenamientos (CU-039)— son interacciones directas e independientes del actor con el sistema.

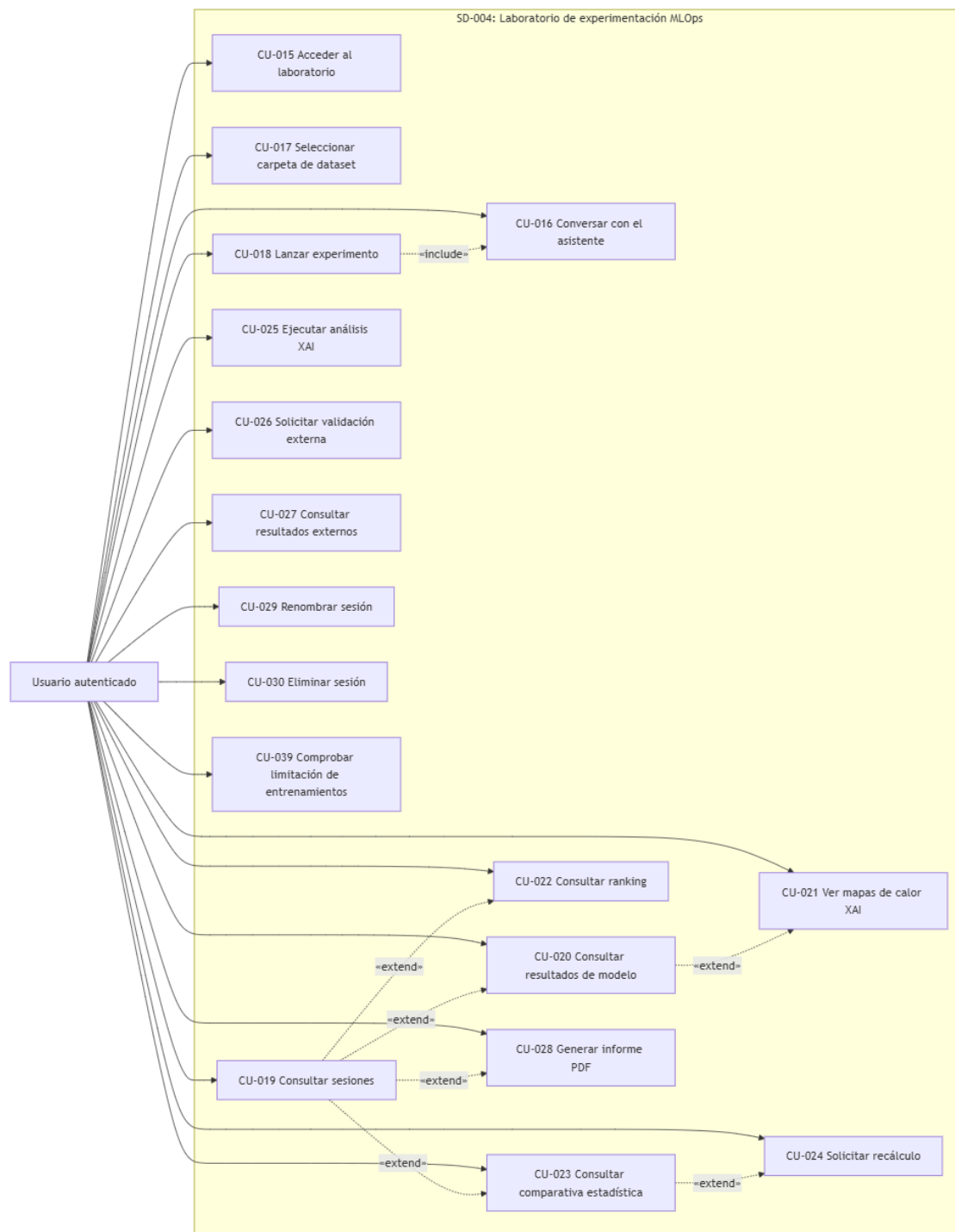


Ilustración x - Diagrama de casos de uso del subsistema SD-004

20.4.2. Casos de uso reales del subsistema

Los casos de uso reales de SD-004 concretan cada interacción del diagrama en decisiones técnicas de implementación. El subsistema combina operaciones síncronas de configuración y consulta con operaciones asíncronas de ejecución, repartidas entre la cola de trabajos y las tareas en segundo plano. Para facilitar la lectura, los casos de uso reales se presentan agrupados en cuatro bloques: la entrada del laboratorio, la consulta de resultados, el análisis y la validación, y la gestión e informes.

CU-015 Acceder al laboratorio de entrenamiento

El acceso al laboratorio se materializa en el endpoint GET /training del router routers/auth.py, que aplica la misma política de sesión que el panel de diagnóstico. Las decisiones técnicas del caso de uso real son las siguientes:

- **Verificación de la sesión:** el router obtiene el identificador del usuario mediante `get_user_id_from_token()` de SD-001; sin sesión válida, responde con una redirección HTTP 303 a la página de inicio de sesión.
- **Carga del perfil:** con el identificador resuelto, se consultan el nombre y el rol del usuario en la tabla users para personalizar la vista del laboratorio.
- **Control de caché:** la respuesta incluye las cabeceras Cache-Control: no-store, de modo que la página del laboratorio no se sirve desde caché tras un cierre de sesión.

CU-016 Conversar con el asistente para configurar un experimento

La configuración conversacional se materializa en el endpoint POST /api/chat, que delega en services/chatbot_service.py. El diseño trata al proveedor del asistente como una frontera externa: los errores del proveedor no se confunden con los de persistencia o entrenamiento. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comunicación con el asistente:** el servicio envía el mensaje del usuario al asistente externo (Groq) y devuelve una configuración estructurada del experimento, solicitando los parámetros que falten (arquitecturas, épocas, lote, tasa de aprendizaje).
- **Frontera externa:** el único componente que necesita la clave de la API del asistente es chatbot_service.py; los routers no la incluyen en las respuestas ni en los registros.
- **Localización de los prompts:** el servicio selecciona el prompt del asistente según el idioma de la sesión, en coherencia con el mecanismo de internacionalización de SD-001.
- **Condición del lanzamiento:** la configuración conversacional constituye el paso obligatorio que el lanzamiento del experimento (CU-018) incluye, de modo que la ejecución no comienza hasta que la configuración es completa y válida.

CU-017 Seleccionar la carpeta del dataset

La selección de la carpeta del dataset se materializa en el endpoint GET /api/train/browse, que delega en la exploración de mlops_engine. El diseño confina la selección dentro del directorio raíz permitido, evitando que la ruta escape del ámbito de datos del sistema. Las decisiones técnicas del caso de uso real son las siguientes:

- **Exploración del sistema de ficheros:** la función browse_folder() devuelve la ruta del dataset configurada mediante la variable de entorno TFG_DEMO_DATASET (o TFG_DEMO_EXTERNAL_DATASET para la validación externa) o, en su defecto, abre el selector de carpetas del sistema.
- **Selección confinada:** la ruta seleccionada se restringe al directorio permitido, de modo que el usuario no puede indicar una ruta arbitraria fuera del ámbito de datos.
- **Respuesta al cliente:** el router devuelve la ruta seleccionada para que la interfaz la inserte en la configuración del experimento, o responde HTTP 500 si la exploración no pudo completarse.

CU-018 Lanzar un experimento de entrenamiento

El lanzamiento del experimento se materializa en el endpoint POST /api/train/start, que crea la sesión de entrenamiento y encola su ejecución asíncrona. El diseño exige una configuración completa y válida antes de encolar, en coherencia con la inclusión de la configuración conversacional (CU-016). Las decisiones técnicas del caso de uso real son las siguientes:

- **Validación de la ruta del dataset:** el router comprueba que la ruta del dataset exista en el sistema de ficheros; en caso contrario, responde HTTP 400 con el mensaje de ruta inexistente y no crea la sesión.
- **Creación de la sesión:** create_training_session() crea el directorio de la sesión en training_results y escribe el config.json con la configuración del experimento, los modelos, la ruta del dataset y el identificador del usuario propietario.
- **Encolado del entrenamiento:** el router inserta un registro en job_queue con el tipo training y un payload JSON serializable que contiene el identificador de la sesión, los modelos, la ruta del dataset y los hiperparámetros; ni el modelo ni los datos se serializan en la base de datos.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el estado queued, el identificador del trabajo y el de la sesión, de modo que la interfaz informa del encolado y comienza a consultar el estado.

CU-039 Comprobar la limitación de entrenamientos simultáneos y encolados

La limitación de los entrenamientos se materializa en el mecanismo de ejecución de la cola de trabajos, que procesa un único trabajo de entrenamiento cada vez y prioriza de forma diferenciada los tipos de trabajo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Procesamiento secuencial:** el worker reclama el primer trabajo queued mediante una actualización condicional que solo tiene efecto si el trabajo seguía en cola, de modo que dos instancias no pueden ejecutar simultáneamente el mismo entrenamiento.
- **Prioridad de la cola:** la selección del siguiente trabajo ordena por tipo, de modo que el entrenamiento no compite por la misma prioridad que el diagnóstico; la interfaz de la cola refleja la posición de cada trabajo y los recuentos de pendientes.
- **Consulta del estado:** el endpoint de estado de la cola devuelve los trabajos del usuario con su estado y posición, de modo que el usuario puede comprobar el encolado y la progresión de sus experimentos.

CU-019 Consultar las sesiones de entrenamiento

La consulta de sesiones se materializa en el endpoint GET /api/train/models, que devuelve únicamente las sesiones del usuario autenticado. El diseño aplica el aislamiento de datos en el propio motor: la enumeración de las sesiones filtra por propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Enumeración de sesiones:** get_trained_sessions() lista los directorios de training_results que contienen al menos un modelo con resultados, en orden descendente, y filtra por el identificador del usuario mediante la configuración de cada sesión.
- **Aislamiento de datos:** cada usuario recibe únicamente sus propias sesiones, en coherencia con el requisito de aislamiento de datos entre cuentas (RF-005).
- **Respuesta al cliente:** el router devuelve la lista de sesiones con sus modelos, y la interfaz las muestra en el panel lateral del laboratorio con su fecha de creación.

CU-020 Consultar los resultados de un modelo de la sesión

La consulta de los resultados de un modelo se materializa en el endpoint GET /api/train/results/{session_id}/{model_name}, precedida de la comprobación de propiedad de la sesión. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad:** el router invoca _require_ownership(), que delega en la verificación de mlops_engine; si la sesión no pertenece al usuario, responde HTTP 403 con el mensaje de permiso denegado.
- **Lectura de resultados:** get_model_results_data() lee las métricas de validación cruzada del kfold_results.csv del modelo, las métricas de calibración, las métricas XAI cuantitativas y las rutas de los artefactos visuales.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 si el modelo no dispone de resultados y HTTP 200 con las métricas y las rutas de los artefactos cuando la consulta es correcta.
- **Servicio de artefactos:** las imágenes del modelo se sirven mediante las rutas estáticas del directorio training_results, sin un endpoint de cálculo adicional.

CU-021 Visualizar los mapas de calor de explicabilidad de un modelo

La visualización de los mapas de calor se materializa en la galería de imágenes que la consulta de resultados devuelve, correspondientes al análisis XAI cualitativo del modelo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Artefactos de la sesión:** los mapas de calor se generan durante el entrenamiento o mediante la ejecución manual del análisis XAI, y se conservan en el directorio del modelo con el prefijo `xai_example_`.
- **Presentación:** la interfaz muestra la galería de mapas de calor sobre imágenes de ejemplo, servidos desde el almacenamiento estático de `training_results`.
- **Origen de los datos:** la visualización no repite el análisis; recupera los artefactos ya persistidos por el pipeline de explicabilidad.

CU-022 Consultar el ranking de modelos de la sesión

La consulta del ranking se materializa en el endpoint `GET /api/train/session/{session_id}/ranking`, precedida de la comprobación de propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Lectura del ranking:** `get_session_ranking_data()` lee el `session_ranking.csv`, que ordena los modelos por la media del AUC de la validación cruzada con su desviación típica, junto con la configuración de la sesión y el mapa de calor del test de Wilcoxon.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 con el mensaje de sesión no comparada si la sesión aún no dispone de ranking, y HTTP 200 con el ranking, el heatmap y la configuración cuando existe.
- **Presentación:** la interfaz muestra la tabla de ranking con los mejores modelos y la matriz de significancia estadística.

CU-023 Consultar la comparativa estadística de la sesión

La consulta de la comparativa estadística se materializa en el mismo endpoint del ranking, que devuelve la matriz de significación junto con el ranking. Las decisiones técnicas del caso de uso real son las siguientes:

- **Matriz de significación:** la comparativa incluye los p-valores del test de Wilcoxon entre los modelos y, si la sesión dispone de validación externa, los p-valores del test de DeLong sobre las curvas ROC.
- **Artefacto visual:** el heatmap de significancia se sirve como imagen estática junto con los datos del ranking.
- **Condición de disponibilidad:** la comparativa solo existe cuando el pipeline de estadística se ha ejecutado sobre la sesión; en caso contrario, la consulta responde con la condición de sesión no comparada.

CU-024 Solicitar el recálculo de la comparativa estadística

El recálculo se materializa en el endpoint POST `/api/train/session/compare`, que programa la comparación estadística en segundo plano mediante las tareas de fondo de FastAPI, en lugar de la cola de trabajos. Esta decisión diferencia el canal de ejecución de la comparativa del canal del entrenamiento. Las decisiones técnicas del caso de uso real son las siguientes:

- **Programación asíncrona:** el router registra `run_statistical_comparison()` como tarea de fondo, de modo que el recálculo se ejecuta sin bloquear la petición y sin ocupar la cola de trabajos.
- **Regeneración de resultados:** la tarea ejecuta el script de estadística que regenera el ranking y la matriz de Wilcoxon, y registra la finalización mediante un marcador de estado en la sesión.
- **Consulta del estado:** la interfaz sondea el endpoint de estado del recálculo, que devuelve `running` o `completed`, y recarga la vista cuando la comparativa queda regenerada.

CU-025 Ejecutar el análisis de explicabilidad de un modelo

La ejecución del análisis XAI se materializa en el endpoint POST `/api/train/run_eval`, que permite regenerar los artefactos de explicabilidad de un modelo. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad y de ruta:** el router verifica la propiedad de la sesión y resuelve la ruta del dataset; si no existe, responde HTTP 400 con el mensaje de dataset no encontrado.
- **Ejecución del análisis:** el motor lanza los scripts de análisis XAI cualitativo y cuantitativo sobre el modelo, regenerando los mapas de calor y las métricas de fidelidad.
- **Respuesta al cliente:** el sistema responde HTTP 200 con un mensaje de generación completada, y la interfaz recarga la vista de resultados para mostrar los nuevos artefactos.

CU-026 Solicitar la validación externa de la sesión

La solicitud de validación externa se materializa en el endpoint POST `/api/train/session/external_validation`, que encola la evaluación sobre la cohorte externa. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobaciones previas:** el router verifica la propiedad de la sesión y que la ruta del dataset externo exista; en caso contrario, responde HTTP 403 o HTTP 400 con el mensaje de ruta externa inválida.
- **Encolado:** se inserta un registro en `job_queue` con el tipo `external_validation` y un payload con el identificador de la sesión y la ruta del dataset externo.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el estado `queued` y el identificador del trabajo, de modo que la interfaz informa del encolado de la validación.
- **Separación de la configuración:** la validación externa se ejecuta sobre los modelos disponibles de la sesión sin modificar la configuración original del experimento.

CU-027 Consultar los resultados de la validación externa

La consulta de los resultados externos se materializa en el endpoint GET `/api/train/session/{session_id}/external_results`, precedida de la comprobación de propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Lectura de resultados:** `get_external_results_data()` lee las métricas sobre la cohorte externa, la curva ROC y la matriz de significación de DeLong.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 con el mensaje de resultados externos no disponibles si la validación aún no ha producido resultados, y HTTP 200 con las métricas, la ROC y la matriz de DeLong cuando existen.
- **Presentación:** la interfaz muestra las métricas de la validación externa junto con los artefactos visuales de la sesión.

CU-028 Generar el informe PDF de la sesión

La generación del informe PDF se materializa en el endpoint GET `/api/train/session/{session_id}/report`, que delega en `services/pdf_generator_mlops.py`. El diseño separa el conocimiento del formato del informe del router: el generador recibe la información preparada por el motor y construye el documento. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad:** el router verifica la propiedad de la sesión antes de generar el informe, respondiendo HTTP 403 ante una sesión ajena.
- **Construcción del documento:** `generate_pdf_report()` compone un informe consolidado con la configuración del experimento, el ranking de modelos con el heatmap de Wilcoxon, los resultados de la validación externa con la ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor.
- **Conservación del informe:** el documento se guarda en el directorio de la sesión y se sirve como descarga mediante `FileResponse`.
- **Fase posterior:** el informe se genera cuando la sesión dispone de los datos necesarios; si la sesión no existe, el generador responde HTTP 404.

CU-029 Renombrar una sesión de entrenamiento

El renombrado se materializa en el endpoint POST `/api/train/session/rename`, precedido de la comprobación de propiedad y de la validación del nuevo nombre en el motor. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de propiedad:** el router responde HTTP 403 con el mensaje de permiso de renombrado si la sesión no pertenece al usuario.
- **Sanitización del nombre:** `safe_rename()` valida el nuevo nombre —caracteres alfanuméricos, espacio, guion y guion bajo— y lo rechaza con HTTP 400 si es inválido o ya existe.
- **Renombrado en disco:** el motor ejecuta el renombrado del directorio de la sesión y devuelve el resultado al router, que lo refleja en la respuesta.
- **Refresco de la vista:** la interfaz actualiza el título de la sesión y recarga el panel lateral del laboratorio.

CU-030 Eliminar una sesión de entrenamiento

La eliminación se materializa en el endpoint DELETE /api/train/session/{session_id}, precedido de la confirmación del usuario y de la comprobación de propiedad. Las decisiones técnicas del caso de uso real son las siguientes:

- **Confirmación previa:** la interfaz solicita la confirmación de la eliminación antes de enviar la petición; si no se confirma, la operación se abandona.
- **Comprobación de propiedad:** el router responde HTTP 403 con el mensaje de permiso de eliminación ante una sesión ajena.
- **Eliminación de los resultados:** delete_session() elimina el directorio de la sesión y sus artefactos mediante el borrado recursivo del sistema de ficheros.
- **Refresco de la vista:** la interfaz oculta los paneles de la sesión y recarga el listado, de modo que la sesión desaparece del laboratorio junto con sus resultados.

20.4.3. Diagramas de interacción entre objetos

Los diagramas de interacción de SD-004 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. El subsistema combina dos canales de ejecución: la cola de trabajos, por la que transitan el entrenamiento y la validación externa, y las tareas en segundo plano, por las que se ejecuta la comparación estadística. Todos los diagramas reflejan la persistencia híbrida —MySQL para la cola y el sistema de ficheros para los resultados— y la regla de propiedad de las sesiones, que se aplica antes de consultar, modificar o eliminar cualquier sesión. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-015 Acceder al laboratorio de entrenamiento

El diagrama de la figura muestra el acceso al laboratorio, cuya secuencia es equivalente a la del panel de diagnóstico: la interfaz solicita la página y el router consulta a SD-001 la identidad contenida en la cookie de acceso. Con sesión válida, recupera el perfil del usuario y devuelve la vista del laboratorio con el control de caché; sin sesión, redirige al punto de entrada del sistema.

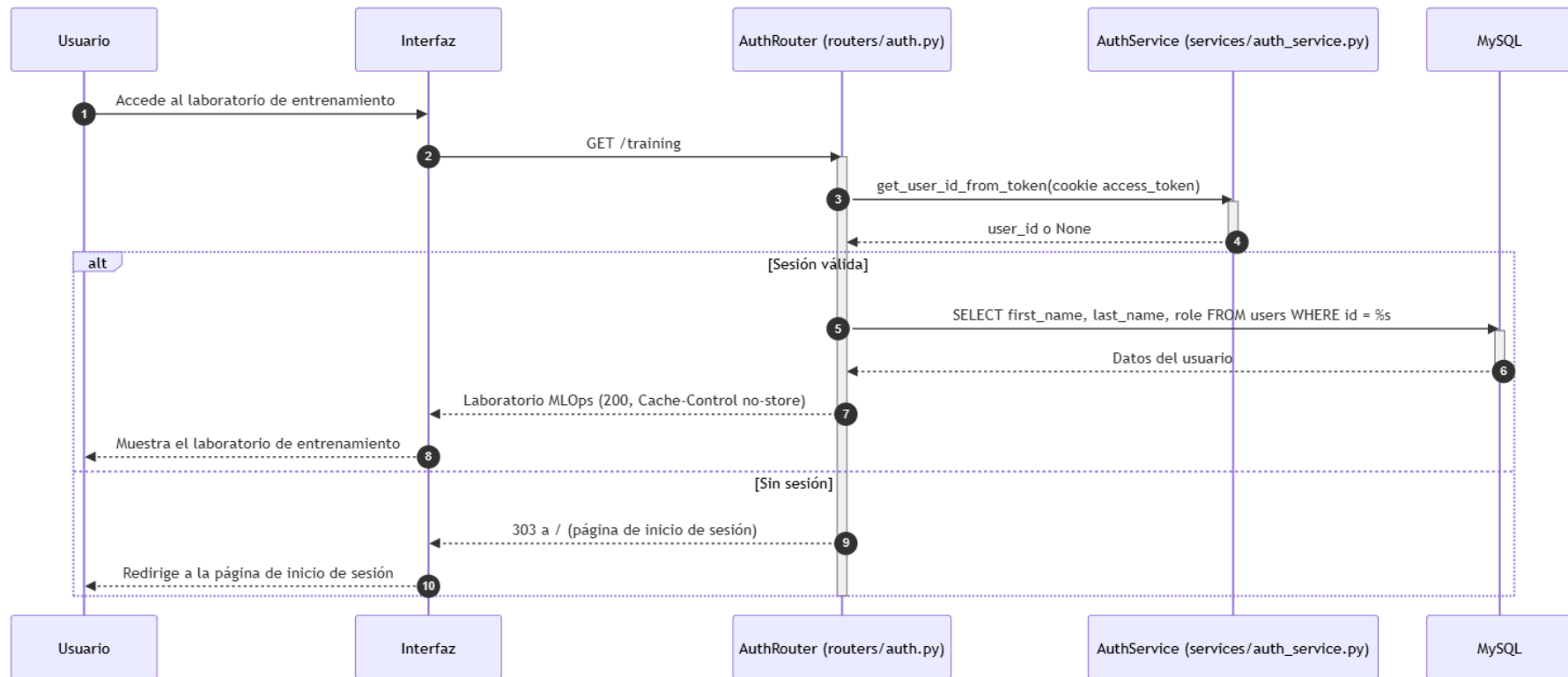


Ilustración x - Diagrama de secuencia del CU-015 Acceder al laboratorio de entrenamiento

CU-018 Lanzar un experimento de entrenamiento

El diagrama de la figura muestra el lanzamiento de un experimento, que materializa también la configuración conversacional (CU-016) y la selección de la carpeta del dataset (CU-017). El usuario describe el experimento en lenguaje natural y el servicio de chat solicita la configuración estructurada al asistente externo; a continuación selecciona la carpeta del dataset mediante la exploración del motor. Cuando la configuración está completa, el router valida la ruta del dataset, crea la sesión con su configuración y encola el entrenamiento, respondiendo el estado `queued`. La secuencia refleja la inclusión de CU-016 en CU-018: el lanzamiento solo se produce con una configuración completa y válida.

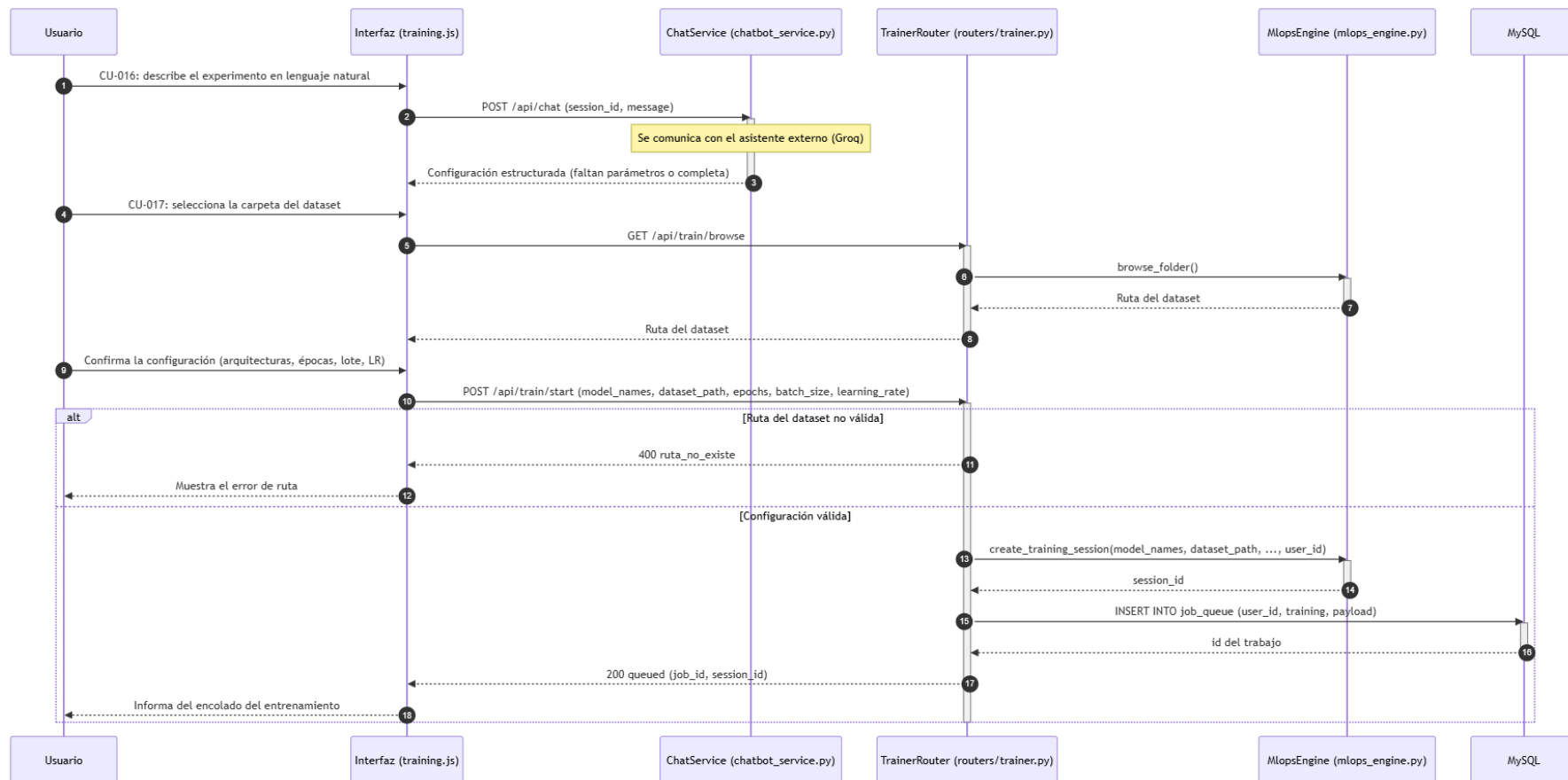


Ilustración x - Diagrama de secuencia del CU-018 Lanzar un experimento de entrenamiento

Procesamiento del entrenamiento

El diagrama de la figura muestra el procesamiento asíncrono del entrenamiento. El worker reclama el trabajo de tipo training de la cola, lo marca como running y encarga al motor el entrenamiento completo. El motor lanza secuencialmente, para cada modelo de la sesión, los scripts de entrenamiento K-fold y de análisis XAI, y finalmente el script de estadística que genera el ranking y la matriz de Wilcoxon; los scripts escriben los artefactos en el directorio de la sesión. Al terminar, el worker marca el trabajo como completed con el resultado. Si cualquier paso falla, el trabajo pasa a failed con el mensaje de error, de modo que un fallo del entrenamiento no se confunde con un estado completado.

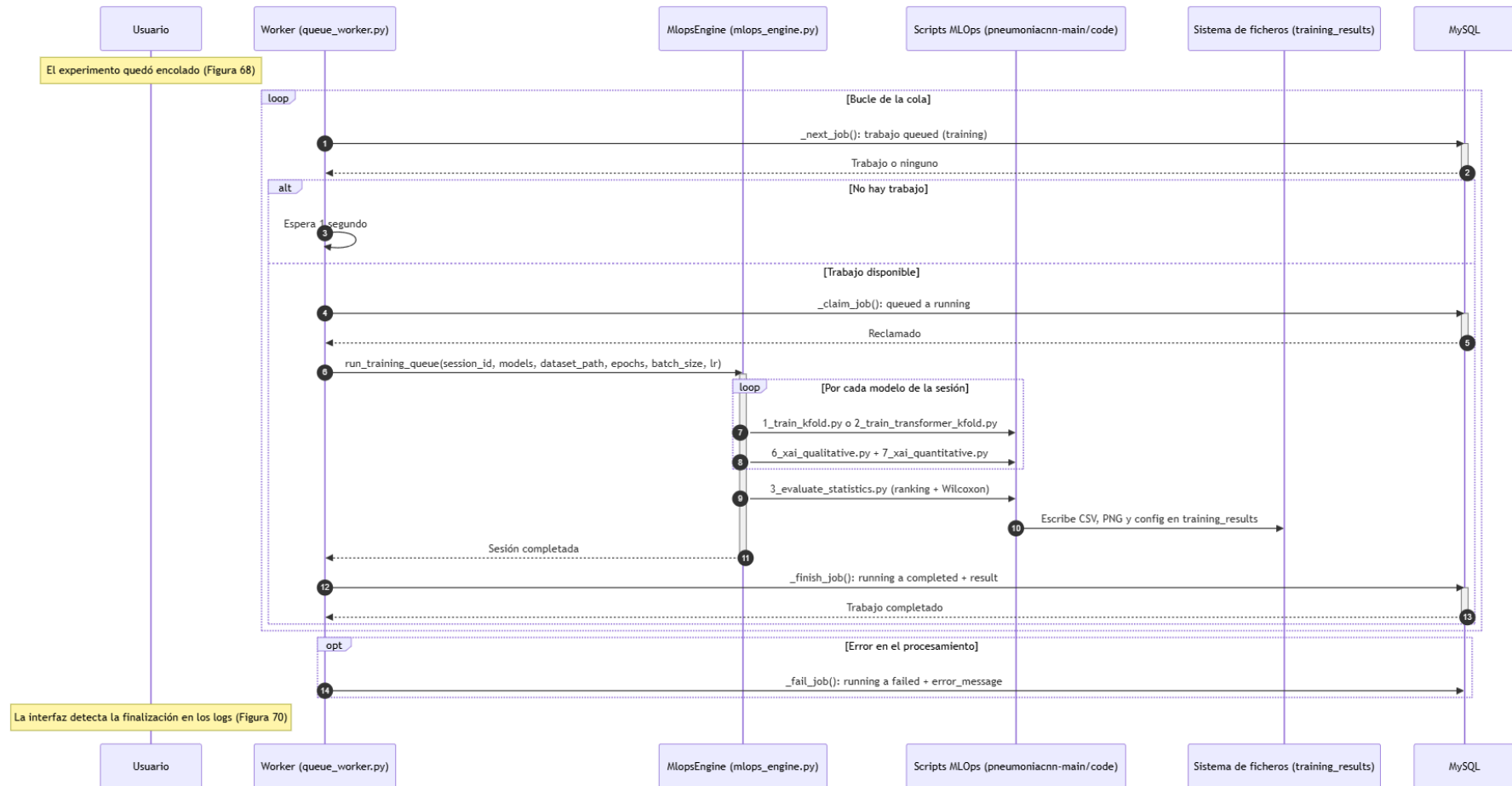


Ilustración x - Diagrama de secuencia del procesamiento del entrenamiento

CU-019, CU-020 y CU-022 Consultar sesiones y resultados

El diagrama de la figura muestra la consulta de las sesiones y de los resultados de un modelo. El usuario consulta sus sesiones y el motor enumera los directorios de `training_results` filtrando por propiedad. Desde el listado, el usuario consulta los resultados de un modelo: el router comprueba la propiedad y el motor lee las métricas K-fold y los artefactos del modelo, devolviendo los datos para su presentación. La secuencia refleja que la consulta no ejecuta el modelo: lee los resultados ya persistidos durante el entrenamiento.

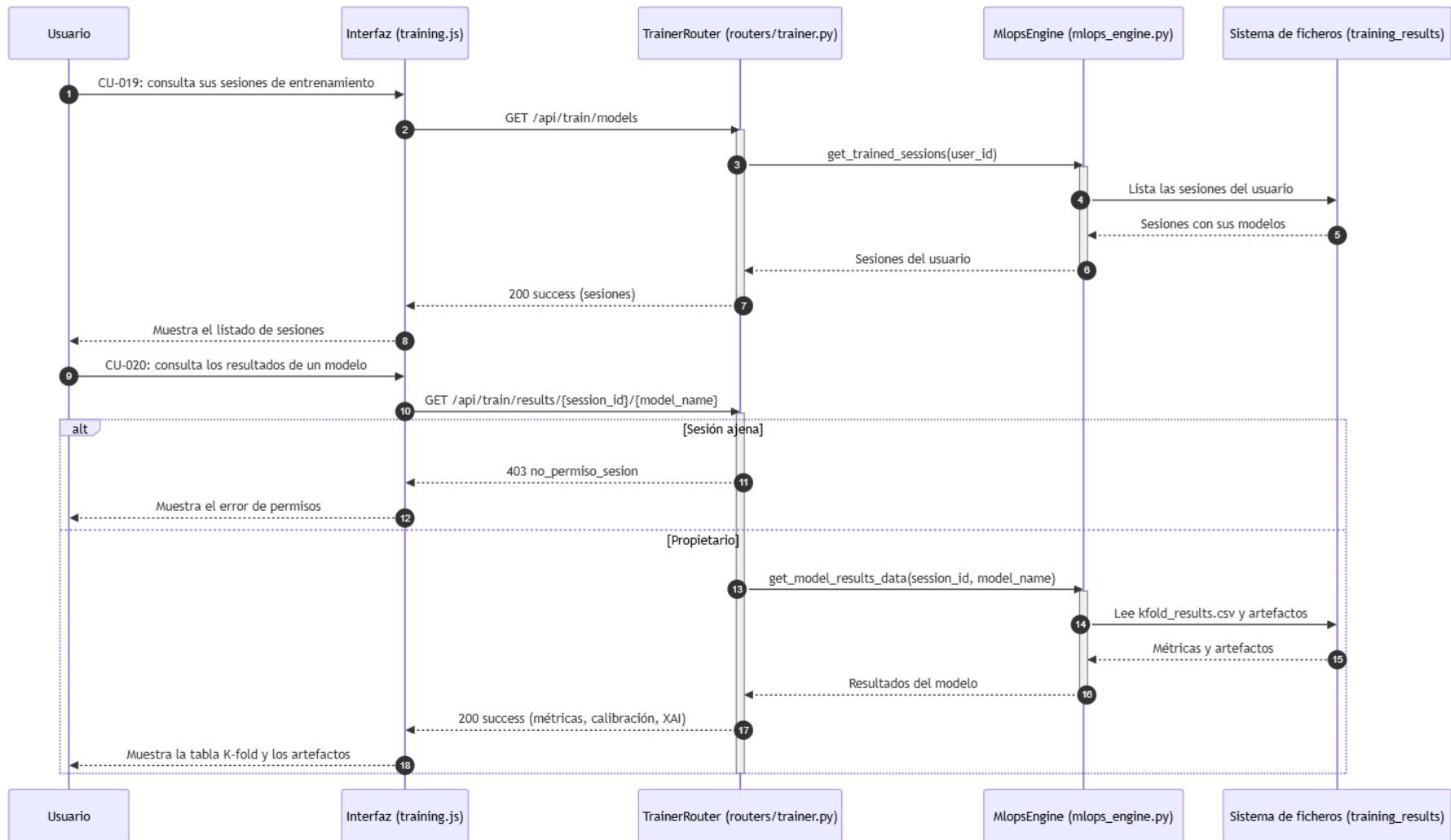


Ilustración x - Diagrama de secuencia de los CU-019, CU-020 y CU-022 Consultar sesiones y resultados

CU-023 y CU-024 Comparativa estadística y recálculo

El diagrama de la figura muestra la consulta de la comparativa estadística y su recálculo. El usuario consulta el ranking de la sesión y el motor lee el ranking y el heatmap de Wilcoxon. Cuando solicita el recálculo, el router programa la comparación estadística como tarea de fondo en lugar de encolarla, y responde el inicio del recálculo; la interfaz sondea el estado hasta que la comparativa queda regenerada. La secuencia refleja el canal de ejecución diferenciado de la comparativa, distinto de la cola de trabajos del entrenamiento.

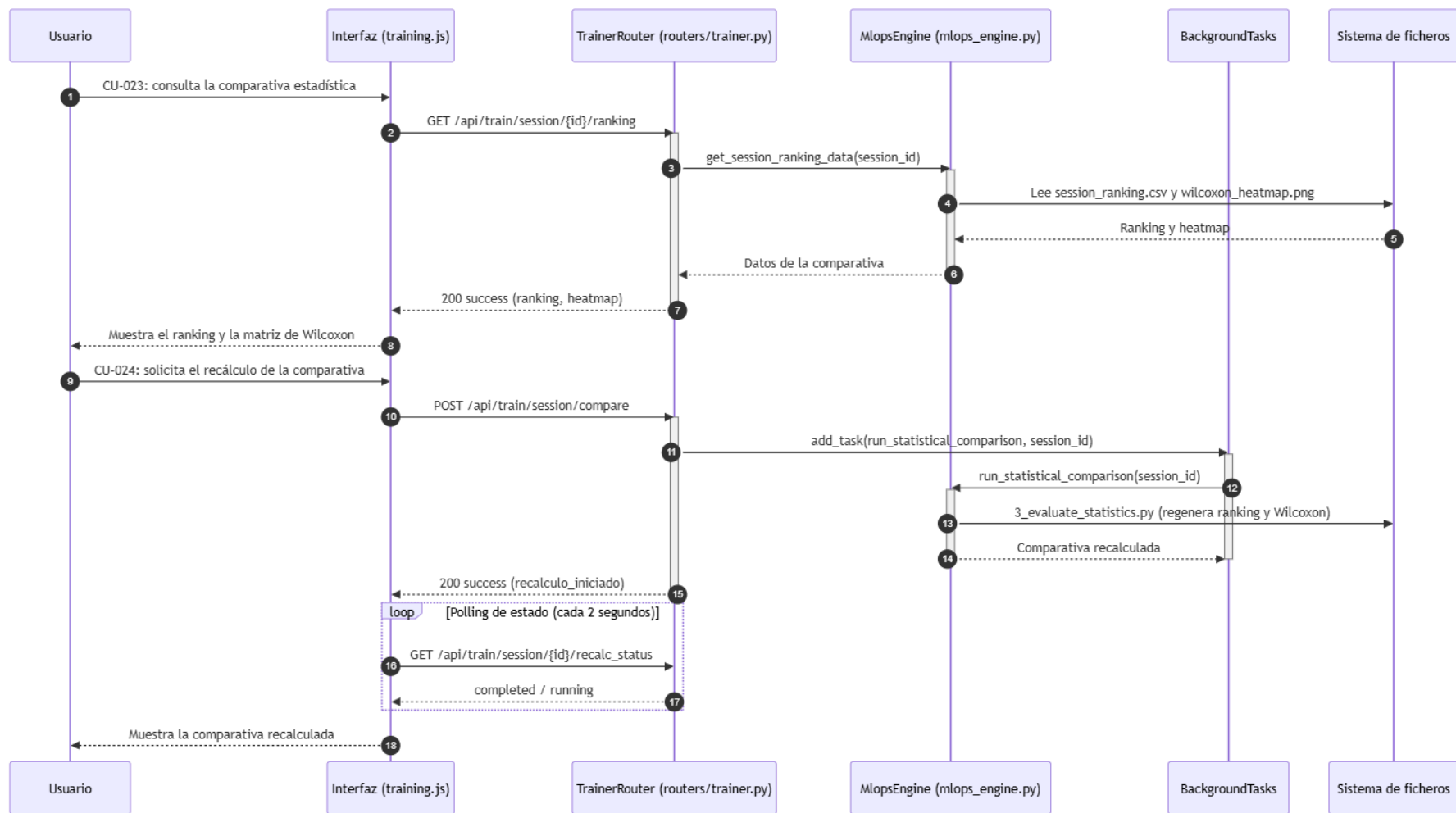


Ilustración x - Diagrama de secuencia de los CU-023 y CU-024 Comparativa estadística y recálculo

CU-026 y CU-027 Validación externa

El diagrama de la figura muestra la solicitud de la validación externa y la consulta de sus resultados. El usuario solicita la validación con la ruta del dataset externo; el router valida la propiedad y la ruta y encola el trabajo de tipo `external_validation`. El worker reclama el trabajo y el motor ejecuta la validación sobre los modelos disponibles, escribiendo las métricas, la curva ROC y la matriz de DeLong. Cuando la validación termina, el usuario consulta los resultados: el router comprueba la propiedad y el motor lee los artefactos de la validación externa. La secuencia refleja la separación de la validación externa respecto de la configuración original del experimento.

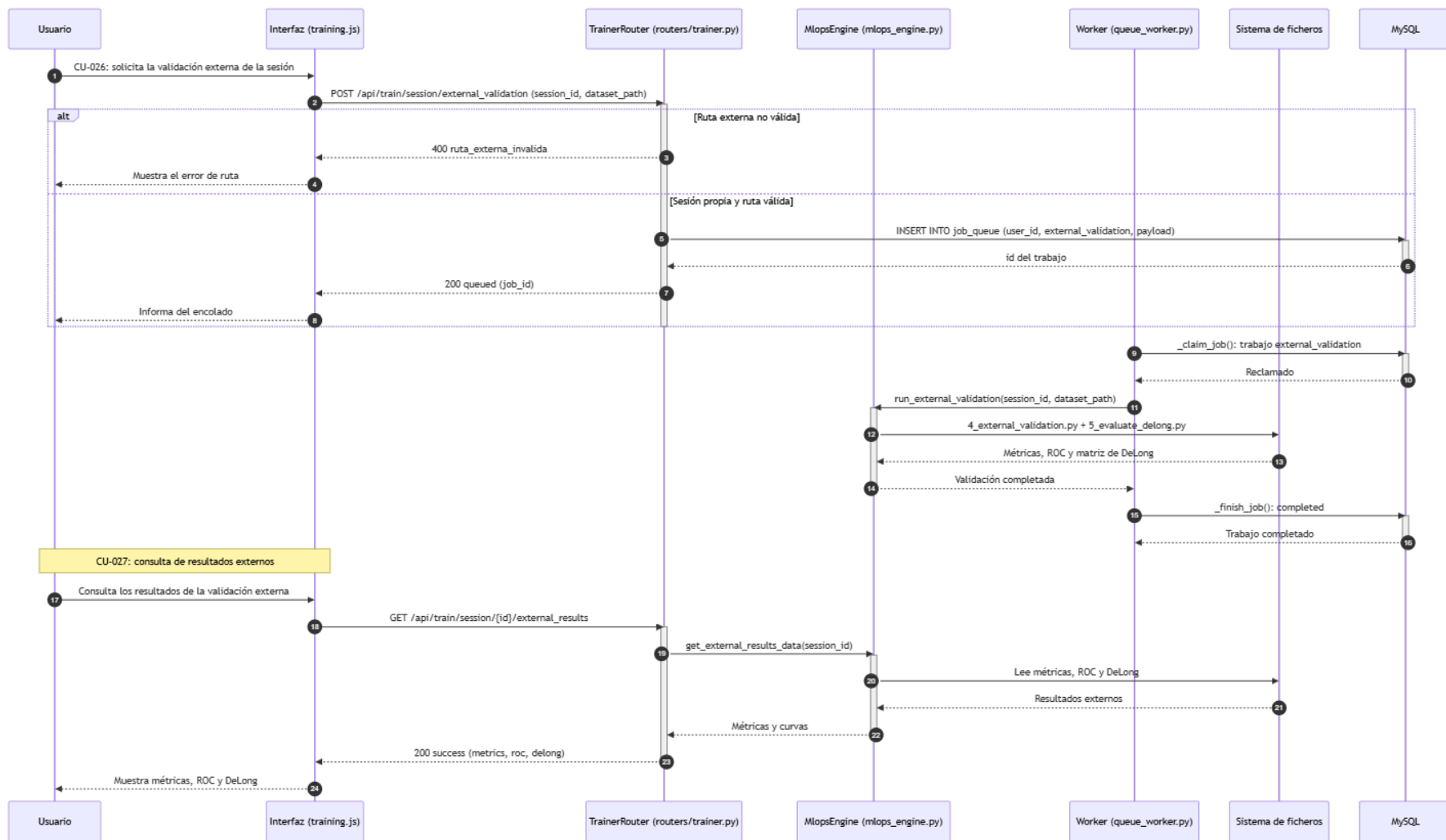


Ilustración x - Diagrama de secuencia de los CU-026 y CU-027 Validación externa

CU-028 Generar el informe PDF de la sesión

El diagrama de la figura muestra la generación y descarga del informe PDF. El usuario solicita el informe de la sesión y el router verifica la propiedad; el generador de informes lee la configuración, el ranking, el heatmap de Wilcoxon, la validación externa y las métricas XAI de cada modelo desde el directorio de la sesión, compone el documento y lo escribe como artefacto. El router sirve el PDF como descarga. La secuencia refleja que el informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios.

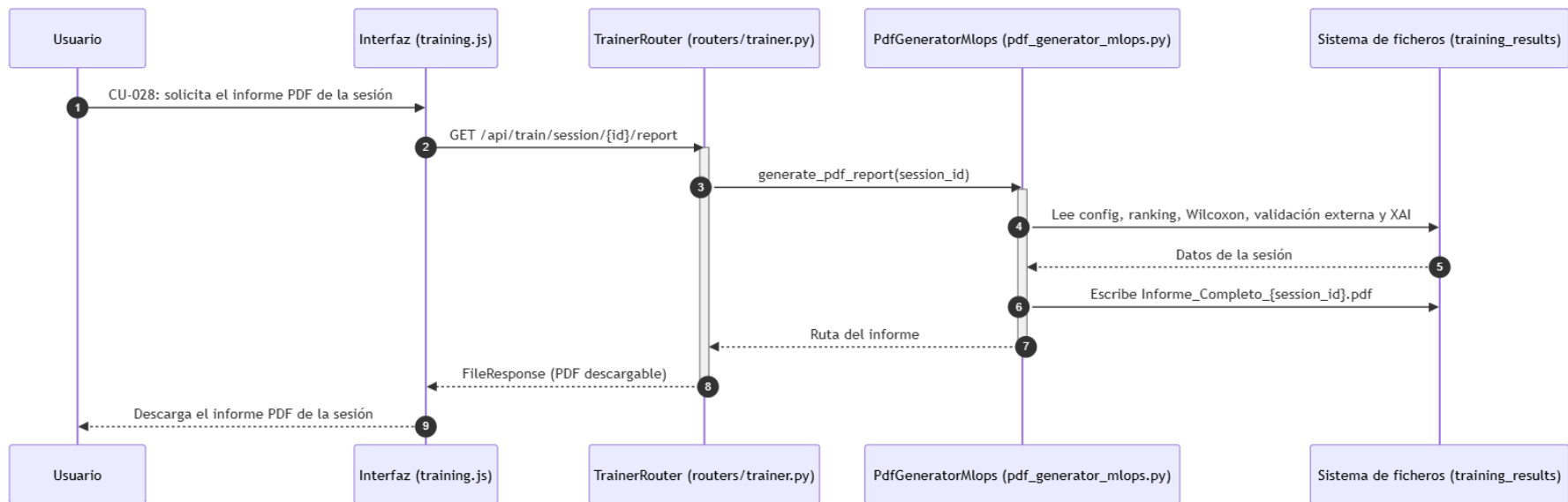


Ilustración x - Diagrama de secuencia del CU-028 Generar el informe PDF de la sesión

CU-029 y CU-030 Renombrar y eliminar una sesión

El diagrama de la figura muestra el renombrado y la eliminación de una sesión. Para renombrar, el usuario indica el nuevo nombre, el router comprueba la propiedad y el motor valida y ejecuta el renombrado del directorio. Para eliminar, la interfaz pide confirmación y, tras ella, el router comprueba la propiedad y el motor borra el directorio de la sesión y sus artefactos. La secuencia refleja que ambas operaciones quedan condicionadas a la propiedad de la sesión, que se resuelve antes de cualquier modificación del sistema de ficheros.

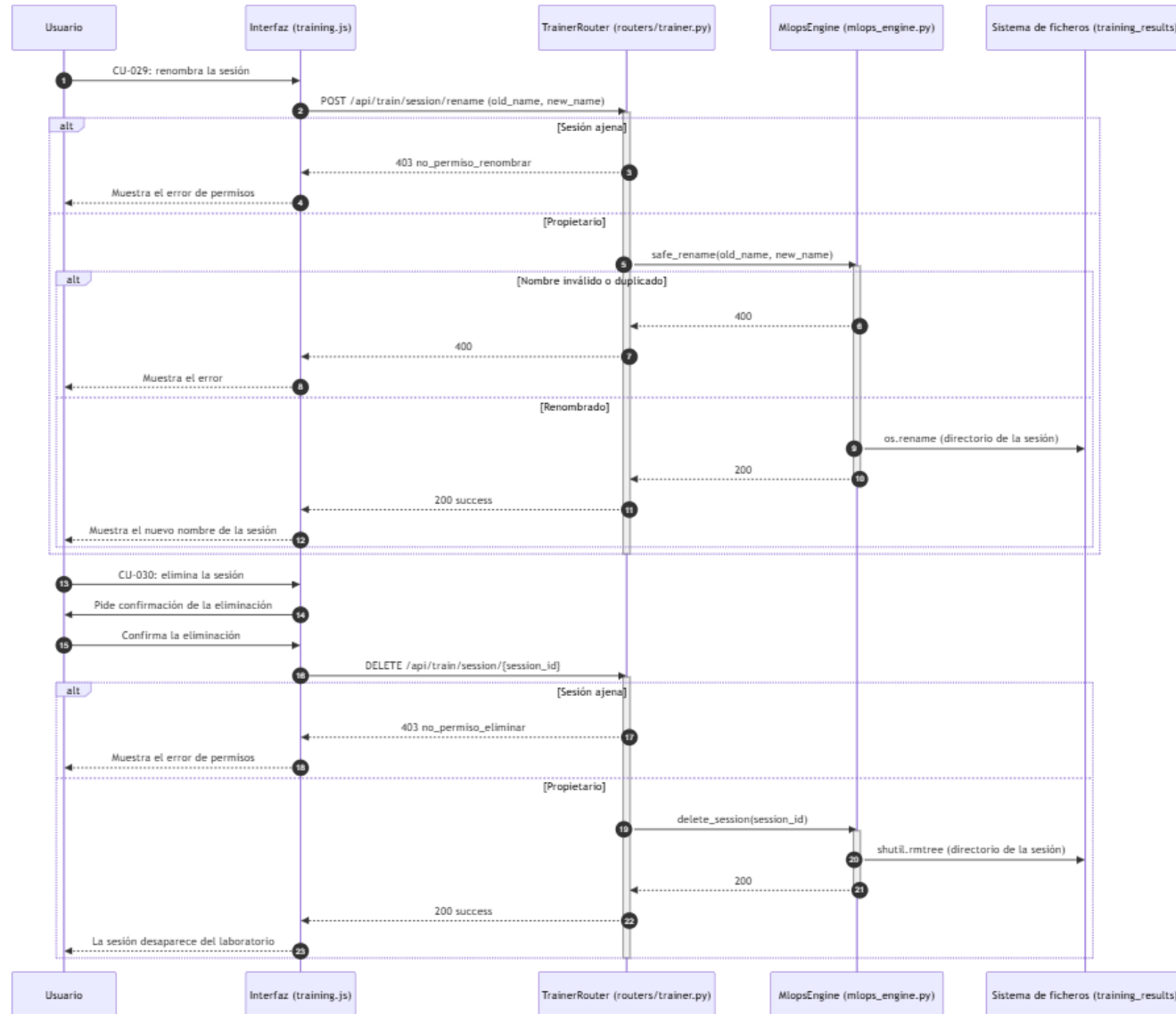


Ilustración x - Diagrama de secuencia de los CU-029 y CU-030 Renombrar y eliminar una sesión

20.5. *Subsistema de diseño SD-005: Supervisión y administración*

El subsistema SD-005 deriva del subsistema de análisis SS-005 y reúne las operaciones que solo puede realizar un usuario con rol de administrador. Su objetivo es proporcionar una visión supervisada del uso de la plataforma sin mezclar las operaciones administrativas con las rutas que utiliza un usuario ordinario. Agrupa los casos de uso CU-031 a CU-033, que materializan la consulta del listado de usuarios, la consulta de las consultas de un usuario concreto y la visualización del detalle de una de esas consultas. El análisis asignó a SS-005 también la gestión de cuentas de usuario (CU-038), pero esa operación no se materializa en el diseño actual: la responsabilidad presente de SD-005 es de consulta y supervisión, tal y como se declaró en el capítulo 18, de modo que no se describen aquí casos de uso reales para una funcionalidad no implementada.

El subsistema se apoya en `routers/admin.py`, que centraliza la comprobación de permisos mediante `_require_admin()`. La función obtiene primero la identidad desde el token de acceso de SD-001 y consulta el rol en la tabla `users`, devolviendo en la práctica un resultado ternario: no hay identidad, existe identidad sin privilegios o existe identidad con rol de administrador. Cada endpoint interpreta ese resultado antes de abrir la consulta global correspondiente, de modo que una ruta administrativa nunca ejecuta una consulta de usuarios o de consultas antes de haber comprobado el permiso. El router diferencia así una petición no autenticada (HTTP 401), una petición autenticada sin permisos (HTTP 403) y una petición administrativa válida.

SD-005 no duplica la lógica de propiedad del historial ni del laboratorio. Su responsabilidad es establecer la autorización administrativa y coordinar las consultas globales; la lectura de los datos sigue utilizando las mismas tablas y servicios que el resto de la aplicación. La vista global combina además las dos fuentes de persistencia de la plataforma: los usuarios y el número de diagnósticos se obtienen mediante una consulta agregada sobre `users` y `consultations`, mientras que el número de sesiones del laboratorio se calcula inspeccionando las configuraciones disponibles en el directorio `training_results`. La protección del subsistema reutiliza la identidad de SD-001 y no implementa una segunda autenticación ni una gestión de tokens propia, y el registro de auditoría administrativa que el análisis contempla (RNF-006) debe considerarse una condición pendiente del modelo operativo, porque el subsistema no escribe actualmente una traza independiente en cada operación.

20.5.1. Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-005 recoge las tres interacciones de supervisión que el subsistema pone a disposición del administrador, y es una adaptación del diagrama del módulo de supervisión y administración definido en el análisis (figura 6), limitada a los casos de uso que se materializan en el diseño. El diagrama conserva las relaciones de extensión de la cadena de navegación del análisis: desde el listado de usuarios (CU-031) se accede a las consultas de un usuario (CU-032), y desde ellas al detalle completo de una consulta (CU-033), de modo que el administrador puede profundizar progresivamente en la información que necesita. La gestión de cuentas (CU-038), que el análisis representaba como operación independiente, no se dibuja aquí por ser una condición pendiente del diseño. El diagrama se representa a continuación.

El diagrama distingue un único actor, el administrador, que es un usuario autenticado con el rol de administración. Los tres casos de uso se encadenan mediante relaciones de extensión que reflejan la navegación del administrador desde el listado general hasta el caso concreto: partiendo de la visión global de las cuentas, puede descender a la actividad de una cuenta y, si lo necesita, al detalle de una consulta individual con su imagen, su resultado y sus metadatos. Esta progresión, idéntica a la del análisis, se conserva en el diseño porque las tres operaciones comparten la misma política de autorización administrativa y la misma frontera de consulta, sin operaciones de modificación general.

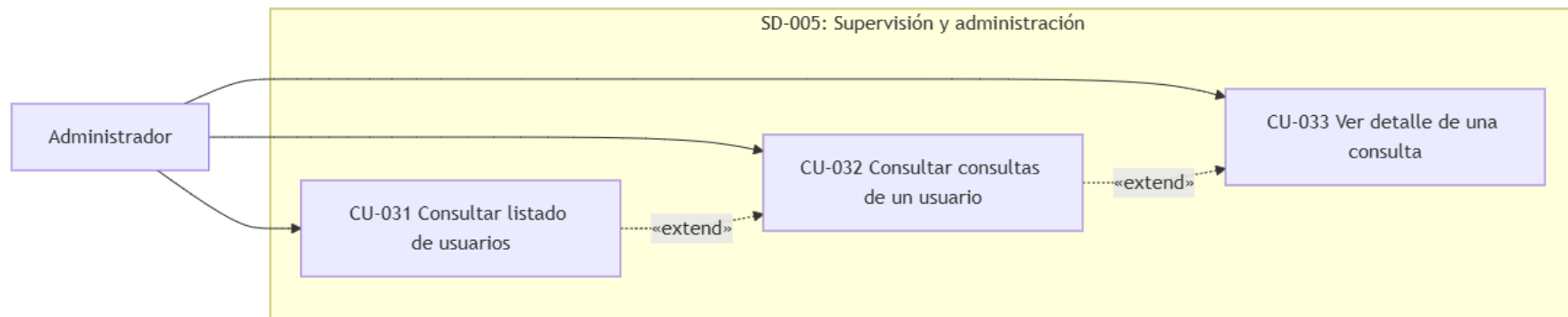


Ilustración x - Diagrama de casos de uso del subsistema SD-005

20.5.2. Casos de uso reales del subsistema

Los casos de uso reales de SD-005 concretan cada interacción del diagrama en decisiones técnicas de implementación. El elemento común de los tres es la comprobación de permisos que abre cada endpoint: ninguna consulta administrativa se ejecuta sin haber resuelto antes la identidad y el rol del solicitante. Cada caso de uso real se describe a continuación.

CU-031 Consultar el listado de usuarios

La consulta del listado se materializa en el endpoint GET /api/admin/users, que combina la información relacional de los usuarios con la información del sistema de ficheros sobre las sesiones del laboratorio. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de permisos:** el router invoca `_require_admin()`, que resuelve la identidad con SD-001 y consulta el rol en la tabla users. Sin identidad responde HTTP 401, y con identidad sin rol de administrador responde HTTP 403; el listado solo se abre con rol administrativo.
- **Consulta agregada:** el router ejecuta una consulta que combina users con consultations mediante una unión izquierda, obteniendo el número de diagnósticos de cada usuario mediante un recuento distinto sobre las consultas, y ordena el resultado por nombre de usuario.
- **Conteo de sesiones del laboratorio:** el router inspecciona el directorio training_results y, a partir de las configuraciones de cada sesión, calcula el número de sesiones por usuario, incorporándolo a cada fila del listado. Esta decisión refleja la persistencia híbrida de la plataforma: la información relacional se consulta en MySQL y parte de los artefactos de las sesiones se conserva en el sistema de ficheros.
- **Respuesta al cliente:** el sistema responde HTTP 200 con el listado de usuarios, sus recuentos de diagnósticos y de sesiones del laboratorio, y la interfaz administrativa lo presenta como el punto de partida de la supervisión.

CU-032 Consultar las consultas de un usuario

La consulta de las consultas de un usuario se materializa en el endpoint `GET /api/admin/users/{user_id}/consultations`, que recupera la actividad clínica de una cuenta concreta junto con sus sesiones del laboratorio. El diseño conserva la granularidad de la interfaz administrativa: una ruta por cada nivel de supervisión. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de permisos:** el router aplica la misma `_require_admin()` de CU-031, con las mismas respuestas HTTP 401 y 403.
- **Existencia del usuario objetivo:** antes de recuperar la actividad, el router comprueba que el usuario consultado exista; en caso contrario, responde HTTP 404 con el mensaje de usuario no encontrado.
- **Recuperación de las consultas:** el router selecciona las consultas del usuario desde `consultations`, filtradas por el identificador del usuario y ordenadas por fecha descendente, y formatea las fechas antes de formar la respuesta JSON.
- **Recuperación de las sesiones del laboratorio:** la misma operación recupera las sesiones de entrenamiento del usuario mediante `get_trained_sessions()` de `mlops_engine`, de modo que la vista administrativa ofrece el panorama completo de la actividad de la cuenta.
- **Respuesta al cliente:** el sistema responde HTTP 200 con las consultas y las sesiones del usuario, y la interfaz las muestra como el segundo nivel de la supervisión, desde el que se alcanza el detalle de una consulta.

CU-033 Ver el detalle de una consulta de un usuario

La visualización del detalle se materializa en el endpoint `GET /api/admin/consultations/{consultation_id}`, que recupera la consulta completa para la auditoría de un caso concreto. El diseño aprovecha la misma política de autorización administrativa, sin duplicar la comprobación de propiedad del historial: el administrador accede por su rol, no por ser propietario de la consulta. Las decisiones técnicas del caso de uso real son las siguientes:

- **Comprobación de permisos:** el router aplica `_require_admin()`, con las mismas respuestas HTTP 401 y 403 que el resto de las operaciones administrativas.
- **Recuperación de la consulta:** el router selecciona la consulta completa desde `consultations` por su identificador, incluidas las rutas de la imagen, del mapa XAI y del informe PDF, la predicción, la confianza, el nombre y la fecha.
- **Respuestas diferenciadas:** el sistema responde HTTP 404 si la consulta no existe y HTTP 200 con el detalle cuando existe, de modo que la interfaz distingue la ausencia del registro de un fallo de permisos.
- **Presentación del detalle:** la interfaz muestra el detalle con la radiografía original, el mapa de explicabilidad, el resultado, la confianza, el modelo y los metadatos, servidos mediante las rutas estáticas de la aplicación, completando así la cadena de supervisión desde el listado general hasta la consulta individual.

20.5.3. Diagramas de interacción entre objetos

Los diagramas de interacción de SD-005 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. Los tres comparten el mismo punto de partida: el router resuelve la identidad y el rol del solicitante mediante `_require_admin()` antes de abrir cualquier consulta global, de modo que las rutas administrativas no ejecutan operaciones de lectura sobre usuarios o consultas sin la autorización correspondiente. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-031 Consultar el listado de usuarios

El diagrama de la figura muestra la consulta del listado de usuarios. La interfaz solicita el listado y el router comprueba la autorización, con las alternativas de ausencia de identidad (401) y de identidad sin privilegios (403). Con rol administrativo, el router consulta los usuarios con el recuento de diagnósticos en la base de datos, inspecciona el sistema de ficheros para contar las sesiones del laboratorio por usuario y devuelve el listado completo. La secuencia refleja la doble fuente de información de la vista global: MySQL para los datos relacionales y el sistema de ficheros para las sesiones.

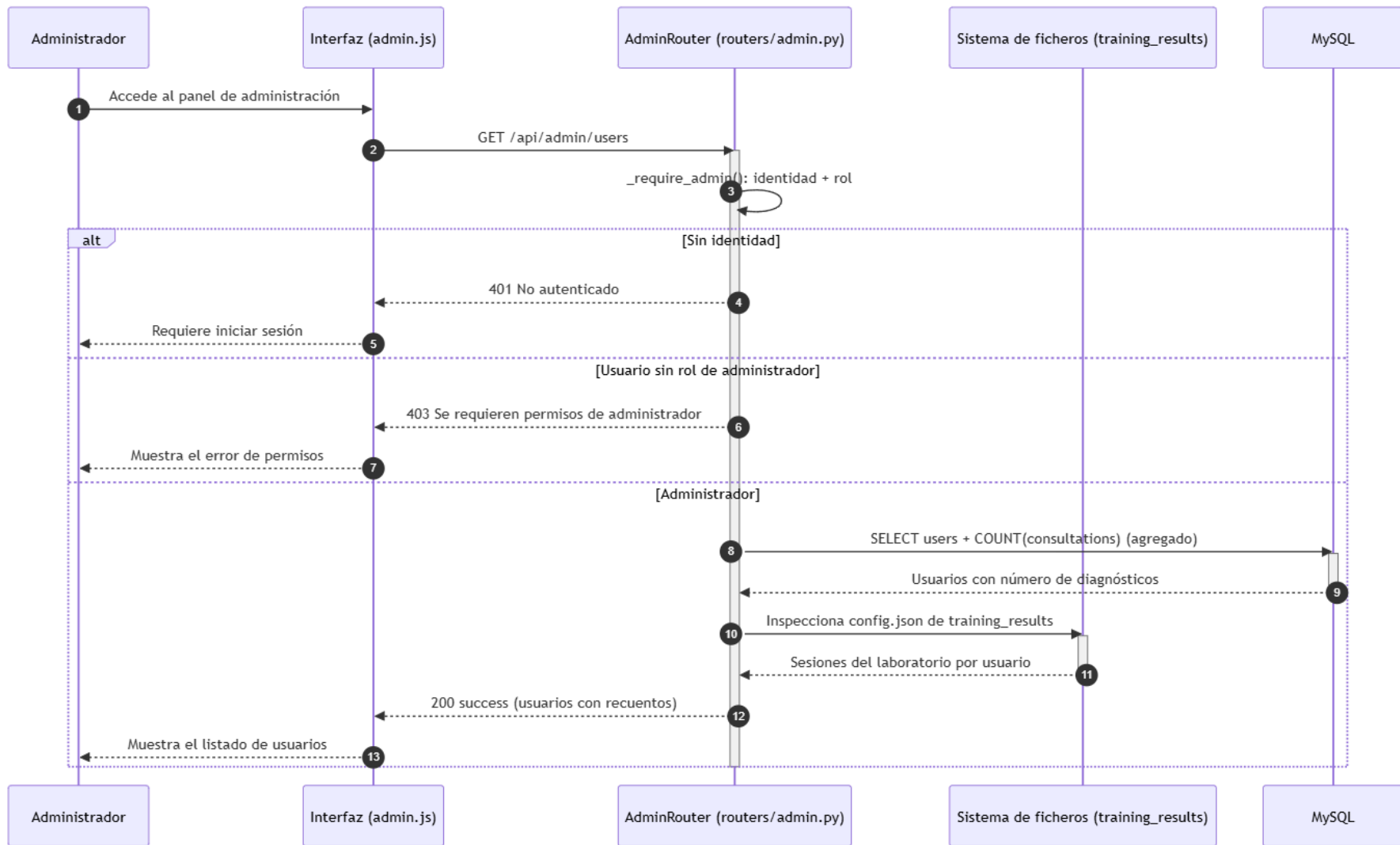


Ilustración x - Diagrama de secuencia del CU-031 Consultar el listado de usuarios

CU-032 Consultar las consultas de un usuario

El diagrama de la figura muestra la consulta de las consultas de un usuario. El administrador selecciona un usuario del listado y el router comprueba la autorización. Con rol administrativo, comprueba que el usuario exista (404 en caso contrario) y recupera sus consultas desde la base de datos, además de sus sesiones del laboratorio mediante el motor MLOps. La secuencia refleja que SD-005 coordina la consulta global reutilizando los mismos servicios del resto de la aplicación, sin duplicar la lógica de acceso a los datos.

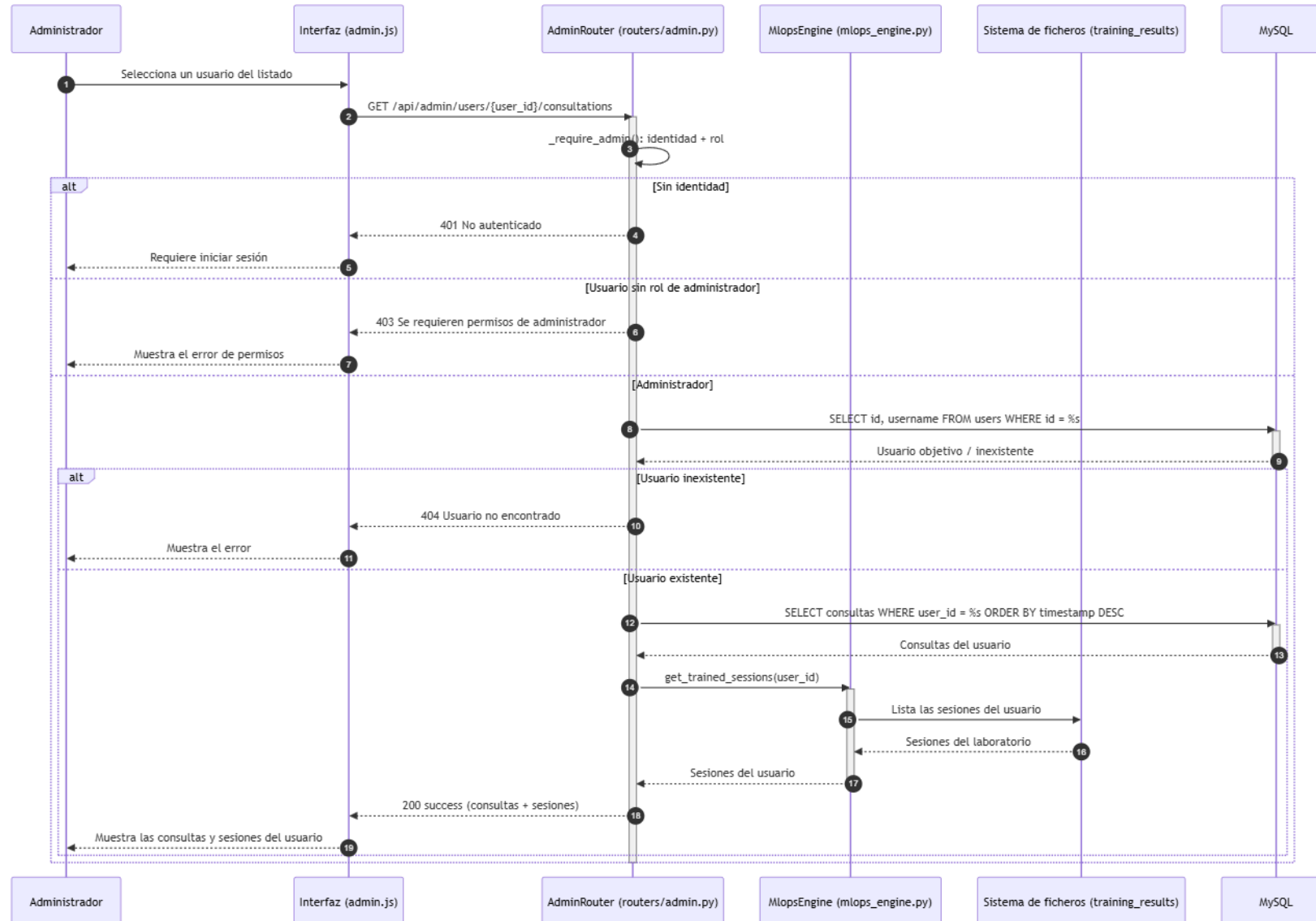


Ilustración x - Diagrama de secuencia del CU-032 Consultar las consultas de un usuario

CU-033 Ver el detalle de una consulta de un usuario

El diagrama de la figura muestra la visualización del detalle de una consulta. El administrador selecciona una consulta de las del usuario y el router comprueba la autorización; con rol administrativo, recupera la consulta por su identificador (404 si no existe) y la devuelve a la interfaz, que carga la imagen y el mapa desde el almacenamiento estático. La secuencia refleja que el acceso administrativo se concede por el rol y no por la propiedad, sin duplicar la comprobación de propiedad del historial.

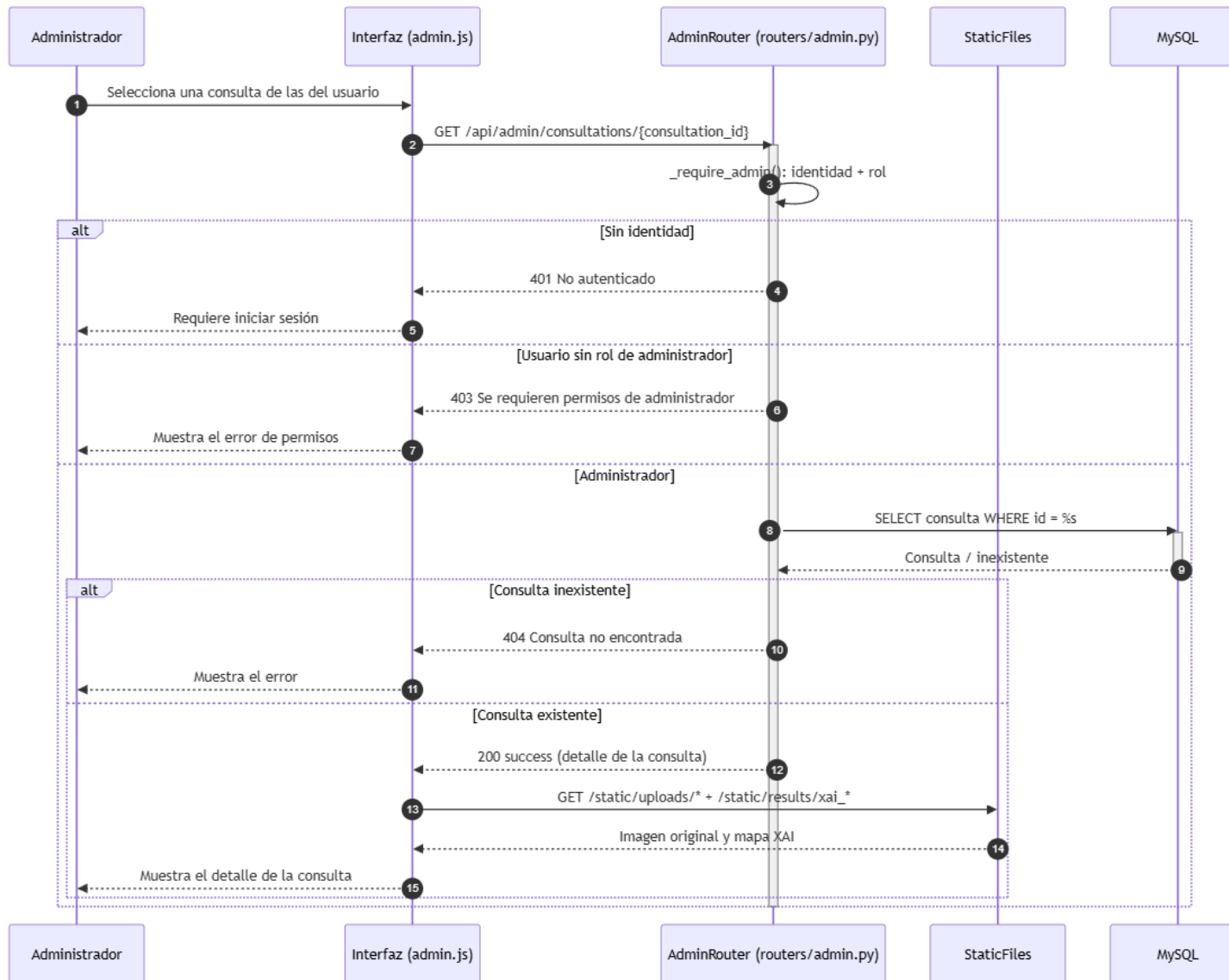


Ilustración x - Diagrama de secuencia del CU-033 Ver el detalle de una consulta

20.6. Subsistema de diseño SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa el subsistema de análisis SS-006 y reúne las capacidades que sirven de apoyo a varios flujos funcionales. Su núcleo es la cola persistente de trabajos, utilizada por los diagnósticos (SD-002) y por los entrenamientos y las validaciones externas (SD-004). Junto a la cola, el subsistema incluye las capacidades transversales de la interfaz: la consulta del estado de los trabajos, la cancelación de un trabajo pendiente y la alternancia del tema visual. Agrupa los casos de uso CU-034 a CU-036, disponibles para todo usuario autenticado con independencia del núcleo funcional en el que trabaje, y constituye el servicio común de ejecución asíncrona sobre el que se apoyan los subsistemas funcionales.

El subsistema se apoya en `routers/queue.py`, que expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente, y en `services/queue_worker.py`, que actúa como consumidor de la cola: al iniciar la aplicación restablece los trabajos que quedaron en estado `running`, selecciona el siguiente trabajo pendiente y lo reclama mediante una actualización condicionada a que siga en `queued`, para después ejecutar el flujo correspondiente en el executor y marcar el resultado como completado o fallido. El worker comparte la persistencia con los routers, pero no comparte con ellos el ciclo de petición: esta es la frontera que permite mantener disponible la interfaz durante las tareas de larga duración. La parte de internacionalización se concentra en `services/lang.py`, y los recursos JavaScript aplican el idioma y el tema visual en el navegador.

La cola define una máquina de estados compartida por los subsistemas que generan trabajos. El estado `queued` indica que la petición ha sido aceptada y espera procesamiento; `running` indica que el worker la ha reclamado; `completed` conserva un resultado; `failed` conserva el error; y `cancelled` identifica una cancelación solicitada por el usuario antes del inicio. No todos los estados se alcanzan desde todas las rutas: la cancelación solo opera sobre los trabajos en `queued`, y un trabajo en `running` no se interrumpe mediante una actualización administrativa. La cola aplica además una política de ordenación que prioriza los trabajos de diagnóstico frente a los de entrenamiento, de modo que la posición que observa el usuario se calcula desde el router y se adapta al tipo de trabajo.

20.6.1. Diagrama de casos de uso del subsistema

El diagrama de casos de uso de SD-006 recoge las tres interacciones transversales que el subsistema pone a disposición del usuario autenticado, y es una adaptación del diagrama del módulo de capacidades transversales definido en el análisis (figura 7), limitada al ámbito del subsistema de diseño. El diagrama no presenta relaciones de inclusión o extensión: la consulta de la cola, la cancelación de trabajos y el cambio de tema se ejecutan de forma autónoma, sin delegar pasos obligatorios en otros casos ni ampliar ningún flujo base. La cancelación de un trabajo (CU-035) se describe de forma coherente con la consulta de la cola (CU-034), porque es desde el panel de la cola donde el usuario identifica el trabajo que desea cancelar, aunque ambas operaciones se representan como casos de uso independientes. El diagrama se representa a continuación.

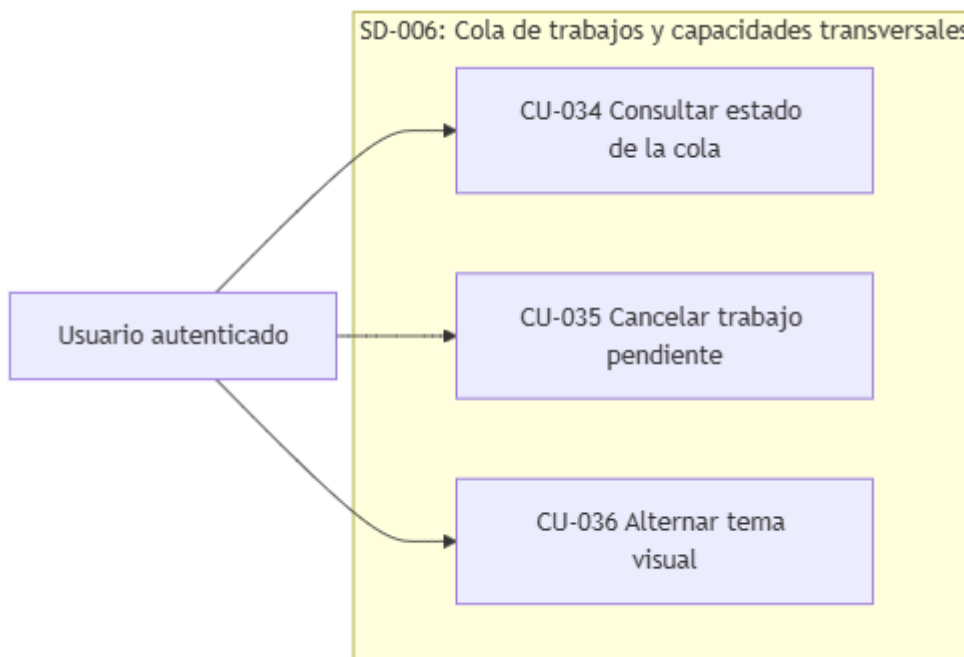


Ilustración x - Diagrama de casos de uso del subsistema SD-006

El diagrama distingue un único actor, el usuario autenticado, que reúne a todos los perfiles con cuenta registrada, incluido el administrador. Los tres casos de uso se representan como interacciones directas e independientes del actor con el sistema, en coherencia con la naturaleza transversal del subsistema: cualquiera de ellos puede ejecutarse desde cualquier núcleo funcional, sin relaciones de dependencia entre sí. El panel de la cola, que materializa la consulta del estado (CU-034), es el punto de partida natural de la cancelación (CU-035), pero el análisis mantiene ambas operaciones separadas porque la cancelación no forma parte obligatoria del flujo de consulta.

20.6.2. Casos de uso reales del subsistema

Los casos de uso reales de SD-006 concretan cada interacción del diagrama en decisiones técnicas de implementación. Los dos primeros se apoyan en el router de la cola y reflejan la máquina de estados del subsistema; el tercero se resuelve íntegramente en el navegador, sin intervención del servidor. Cada caso de uso real se describe a continuación.

CU-034 Consultar el estado de la cola de trabajos

La consulta del estado de la cola se materializa en el endpoint GET /api/queue/status, que devuelve los trabajos recientes del usuario autenticado y su estado. El diseño interpreta el payload de cada trabajo según su tipo, de modo que la interfaz muestra el nombre del modelo o el identificador de la sesión sin recibir el contenido interno completo del payload. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación:** el router resuelve la identidad con SD-001 y responde HTTP 401 si no hay sesión válida.
- **Filtrado por usuario:** la consulta recupera los últimos veinte trabajos del usuario, de modo que el panel solo refleja la actividad de la cuenta que lo consulta (RF-005).
- **Interpretación del payload:** el router deserializa el payload y expone los campos relevantes según el tipo: el nombre del modelo para los diagnósticos, el identificador de la sesión y los modelos para los entrenamientos, y el identificador de la sesión con los modelos leídos de su configuración para las validaciones externas. El payload completo no se envía al navegador.
- **Posición en la cola:** para los trabajos en queued, el router calcula su posición mediante la consulta de posición, que respeta la prioridad de la cola según el tipo de trabajo.
- **Respuesta al cliente:** el sistema responde HTTP 200 con los trabajos, su estado, el indicador de trabajos pendientes y el recuento de encolados; la interfaz actualiza el panel de forma periódica para reflejar la progresión de los diagnósticos, los entrenamientos y las validaciones externas.

CU-035 Cancelar un trabajo de la cola

La cancelación se materializa en el endpoint DELETE `/api/queue/cancel/{job_id}`, que aplica una actualización condicional sobre el trabajo. El diseño adopta una restricción deliberada: solo puede cancelarse un trabajo que pertenezca al usuario y que continúe en estado `queued`. Si el worker ya lo ha reclamado, la operación no lo interrumpe de forma abrupta, de modo que la cancelación no produce inconsistencias entre el estado persistido y el cálculo que ya se está ejecutando. Las decisiones técnicas del caso de uso real son las siguientes:

- **Autenticación y propiedad:** el router resuelve la identidad con SD-001 y condiciona la actualización al identificador del usuario, de modo que un trabajo ajeno no puede cancelarse.
- **Actualización condicional:** la cancelación ejecuta una actualización que exige que el trabajo continúe en `queued`; solo si la actualización afecta a una fila se considera cancelado el trabajo.
- **Respuestas diferenciadas:** si la actualización afectó a una fila, el sistema responde HTTP 200 con el trabajo cancelado; en caso contrario, responde HTTP 404 indicando que el trabajo no fue encontrado o ya no está en cola.
- **Alcance de la cancelación:** la operación cubre los trabajos pendientes de diagnóstico, entrenamiento y validación externa, pero no interrumpe los trabajos en ejecución. Esta decisión difiere del flujo del análisis, que preveía interrumpir los trabajos en ejecución cuando resultara técnicamente posible; el diseño actual documenta esa limitación y mantiene el estado `running` intacto.

CU-036 Alternar el tema visual de la interfaz

La alternancia del tema se materializa en el navegador, mediante los recursos JavaScript de la interfaz, sin intervención del servidor. El diseño trata el tema como una preferencia de presentación que se aplica de forma inmediata y persiste durante la navegación. Las decisiones técnicas del caso de uso real son las siguientes:

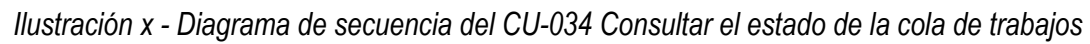
- **Cambio en el navegador:** la función `toggleTheme()` de los recursos JavaScript aplica o elimina la clase del tema oscuro en el elemento raíz de la página, de modo que el cambio se propaga a toda la interfaz sin recargar.
- **Persistencia de la preferencia:** el tema seleccionado se guarda en el almacenamiento local del navegador, de modo que se restaura en las sucesivas vistas del usuario.
- **Restauración automática:** al cargar la interfaz, la preferencia guardada se aplica y, en ausencia de preferencia, se respeta la configuración del sistema operativo del usuario mediante la consulta de preferencia de color.
- **Independencia del servidor:** la operación no requiere una nueva ruta ni una modificación de estado persistida en el servidor; se resuelve como un mecanismo transversal de presentación de la interfaz.

20.6.3. Diagramas de interacción entre objetos

Los diagramas de interacción de SD-006 muestran cómo colaboran los componentes del subsistema para realizar los casos de uso reales. Los dos primeros reflejan la máquina de estados de la cola y su frontera con el ciclo de petición, mientras que el tercero se resuelve íntegramente en el navegador. Se emplea la misma notación de secuencia del UML utilizada en los apartados anteriores.

CU-034 Consultar el estado de la cola de trabajos

El diagrama de la figura muestra la consulta del estado de la cola. La interfaz consulta periódicamente el router de la cola, que recupera los últimos trabajos del usuario. Para los trabajos de validación externa, el router lee la configuración de la sesión en el sistema de ficheros para obtener los modelos; a continuación interpreta el payload según el tipo y calcula la posición de los trabajos encolados, devolviendo el estado al panel. La secuencia refleja que el panel se actualiza de forma periódica y que el payload completo de los trabajos no llega al navegador.



CU-035 Cancelar un trabajo de la cola

El diagrama de la figura muestra la cancelación de un trabajo. El usuario solicita la cancelación desde el panel y el router resuelve la identidad; con sesión válida, ejecuta la actualización condicionada a que el trabajo pertenezca al usuario y continúe en queued. Si la actualización afecta a una fila, el trabajo queda cancelado y desaparece del panel; si no, el sistema responde 404 informando de que el trabajo ya no está en cola, lo que cubre los trabajos en ejecución y los finalizados. La secuencia refleja la restricción deliberada del diseño: un trabajo reclamado por el worker no se interrumpe.

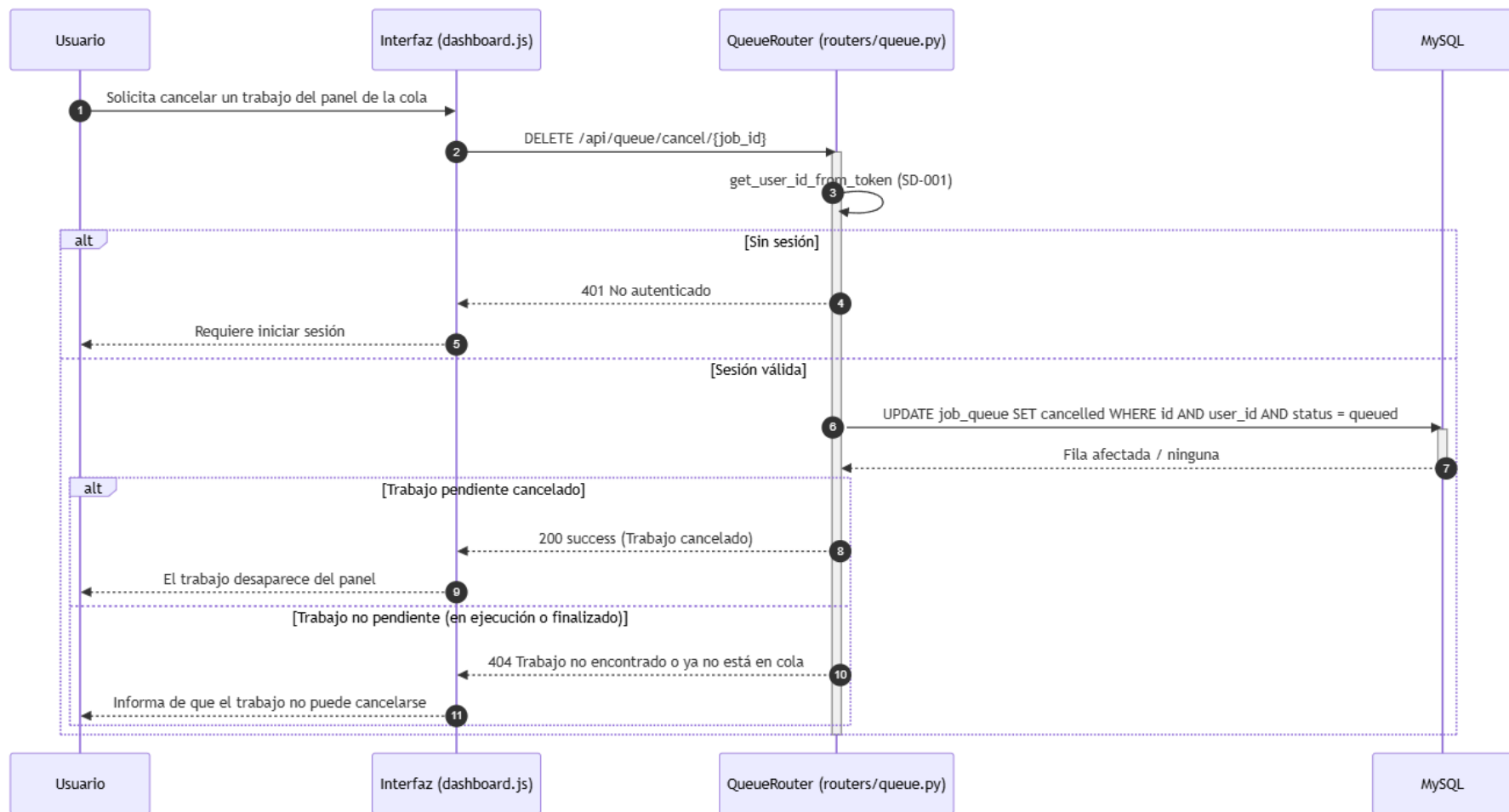


Ilustración x - Diagrama de secuencia del CU-035 Cancelar un trabajo de la cola

CU-036 Alternar el tema visual de la interfaz

El diagrama de la figura muestra la alternancia del tema visual. El usuario activa el cambio y la interfaz aplica o elimina la clase del tema oscuro en el elemento raíz, guarda la preferencia en el almacenamiento local y presenta la interfaz con el tema seleccionado. La secuencia se resuelve íntegramente en el navegador: no interviene el servidor, de modo que el cambio es inmediato y no interrumpe la navegación ni el estado del trabajo en curso.

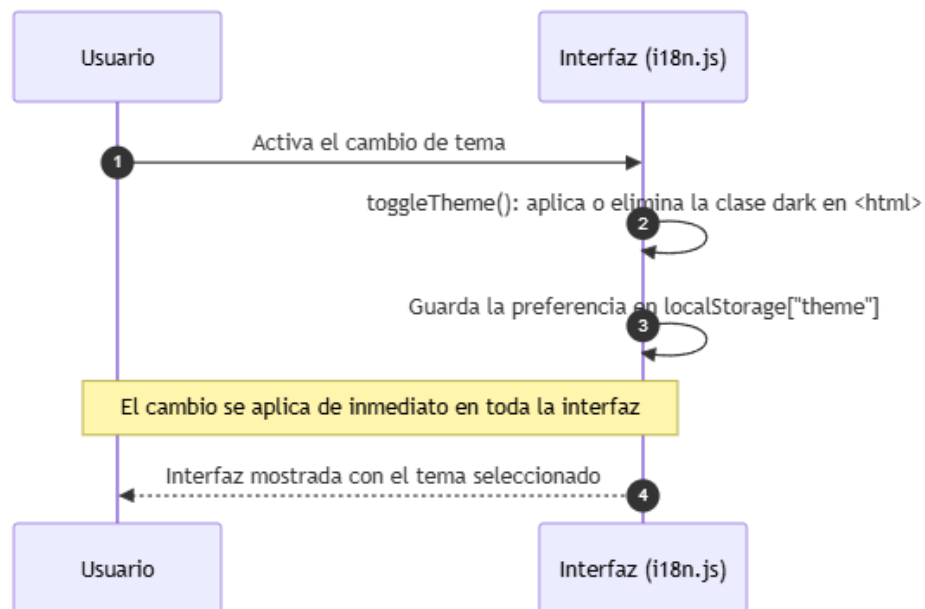


Ilustración x - Diagrama de secuencia del CU-036 Alternar el tema visual de la interfaz

21. Estructura de clases del diseño

El diseño de clases constituye la etapa del diseño que concreta la estructura estática de los subsistemas definidos en el capítulo 18: determina qué clases forman parte de cada subsistema de diseño, qué responsabilidades asume cada una y cómo colaboran entre sí para materializar los casos de uso reales descritos en el capítulo 21. Mientras que los diagramas de interacción muestran el comportamiento de los objetos en el tiempo, el modelo de clases captura la estructura persistente de ese comportamiento: los atributos que conserva cada componente, las operaciones que ofrece y las asociaciones que lo vinculan con el resto (Larman, 2004). El diseño de clases refina así los componentes identificados en la arquitectura y los presenta con el nivel de detalle necesario para comprender y mantener la implementación.

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño del capítulo 18: cada apartado corresponde a un subsistema (SD-001 a SD-006) y agrupa las clases que materializan su responsabilidad. Para cada subsistema se presentan dos elementos, conforme a la guía de diseño de la memoria (punto 5): el diagrama de clases, que representa gráficamente las clases del subsistema con sus atributos, sus operaciones y sus relaciones —incluidas las clases abstractas, las herencias y las asociaciones cuando existen—, y la especificación de las clases, que detalla cada clase mediante la ficha formal de clase CL-NNNN con sus atributos, sus operaciones y sus comentarios. Para las clases más complejas se añade, cuando procede, un diagrama de transición de estados que permite comprender la funcionalidad soportada por dicha clase.

Las clases de diseño de vitalXAI se corresponden con los componentes reales del sistema descritos en el capítulo 18. Aunque la implementación está orientada a módulos funcionales —routers, servicios y el worker—, cada módulo se modela aquí como una clase de diseño que agrupa sus responsabilidades, de modo que la notación UML permite expresar de forma homogénea la estructura de todos los subsistemas y su colaboración. La trazabilidad con el diseño de casos de uso se mantiene en cada ficha: las operaciones de cada clase materializan los mensajes que los diagramas de secuencia del capítulo 21 asignaron al componente correspondiente, y las asociaciones del modelo de clases reflejan las dependencias de colaboración entre routers y servicios.

21.1. *Subsistema SD-001: Acceso, identidad y gestión de sesiones*

El subsistema SD-001 materializa las responsabilidades de acceso, identidad y gestión de sesiones descritas en el capítulo 18 y agrupa dos clases de diseño propias. La clase AuthRouter se corresponde con el router routers/auth.py y actúa como fachada HTTP del subsistema: sirve las páginas de inicio de sesión y registro, procesa los formularios y gestiona las cookies de sesión. La clase AuthService se corresponde con el servicio services/auth_service.py y concentra la lógica criptográfica y el ciclo de vida de las credenciales: el hash de las contraseñas, la firma y verificación de los tokens de acceso, la gestión de los tokens de refresco y la rotación con detección de robo. El subsistema depende además del servicio de idioma LangService, que pertenece a SD-006 y proporciona los mensajes localizados que el router emplea en sus respuestas.

21.1.1. Diagrama de clases del subsistema

El modelo de clases de SD-001 se representa en la figura 83. El diagrama muestra las dos clases propias del subsistema con sus atributos y sus operaciones, y la dependencia hacia el servicio de idioma, que se representa como una clase de otro subsistema para reflejar la colaboración sin atribuirle la propiedad. La clase AuthRouter presenta atributos de configuración interna —la expresión regular de validación del correo electrónico y el motor de plantillas— y las operaciones que sirven las páginas y procesan los formularios; la clase AuthService presenta los parámetros de configuración de las credenciales y las operaciones criptográficas y de gestión de tokens.



Ilustración x - Diagrama de clases del subsistema SD-001

El diagrama identifica la asociación principal del subsistema: la dependencia de AuthRouter hacia AuthService, que materializa la delegación de toda la lógica criptográfica y de tokens del router al servicio. Esta separación de responsabilidades, ya declarada en el capítulo 18, mantiene al router como una fachada ligera que orquesta las peticiones HTTP y delega en el servicio las operaciones que requieren configuración sensible. La dependencia punteada hacia LangService refleja que el router consume los mensajes localizados del servicio de idioma sin que este forme parte del subsistema. El modelo no presenta herencias ni clases abstractas: las dos clases son clases concretas de diseño, y la colaboración se expresa exclusivamente mediante asociaciones de dependencia.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases se corresponden con los mensajes de los diagramas de secuencia del capítulo 21. Las operaciones login(), process_register(), logout() y token_refresh() de AuthRouter materializan los puntos de entrada de los CU-001, CU-002, CU-003 y de la renovación de la sesión, y las operaciones de AuthService—hash_password(), verify_password(), create_access_token(), create_refresh_token(), verify_refresh_token(), revoke_refresh_token() y rotate_refresh_token()—implementan las decisiones criptográficas que los diagramas de secuencia encargaron al servicio.

21.1.2. Especificación de las clases

A continuación se definen las dos clases propias de SD-001.

CL-0001	AuthRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/auth.py. Actúa como fachada HTTP del subsistema de acceso: sirve las páginas de inicio de sesión, registro, panel de diagnóstico y laboratorio de entrenamiento; procesa los formularios de registro e inicio de sesión; gestiona el cierre de sesión y la renovación de credenciales, y establece las cookies de sesión en las respuestas. Valida los datos de entrada del registro y delega toda la lógica criptográfica en AuthService.		
Atributos	Nombre	Tipo	Descripción
	_EMAIL_RE	re.Pattern	Expresión regular privada que valida el formato del nombre de usuario como dirección de correo electrónico en el registro.
	templates	Jinja2Templates	Motor de plantillas que compone las páginas HTML servidas por el router (inicio de sesión, registro, panel y laboratorio).
Operaciones	Nombre	Descripción	
	login_page(request, error)	Sirve la página de inicio de sesión, mostrando el error genérico cuando la autenticación previa falló.	
	login(request, username, password)	Procesa el formulario de inicio de sesión: verifica las credenciales mediante AuthService, establece las cookies de sesión y redirige al panel, o redirige al inicio con un mensaje genérico si la verificación falla. Aplicada la limitación de cinco peticiones por minuto.	
	register_page(request)	Sirve el formulario de registro de nuevas cuentas.	
	process_register(username, password, first_name, last_name, role)	Procesa el formulario de registro: valida el formato de los campos, comprueba la unicidad del usuario, solicita el hash de la contraseña a AuthService, crea la cuenta y establece las cookies de sesión.	

	logout(request)	Cierra la sesión: revoca el token de refresco mediante AuthService, elimina las cookies y redirige a la página de inicio de sesión.
	token_refresh(refresh_token)	Renueva la sesión: rota el token de refresco mediante AuthService y emite un nuevo token de acceso, actualizando las cookies.
	dashboard(request)	Sirve el panel de diagnóstico del usuario autenticado, redirigiendo a la página de inicio si no existe sesión válida.
	training_lab(request)	Sirve el laboratorio de entrenamiento del usuario autenticado, con la misma comprobación de sesión que el panel.
	_validate_register_inputs(username, password, first_name, last_name)	Operación privada que valida el formato del correo electrónico, la longitud mínima de la contraseña y la presencia del nombre y los apellidos, devolviendo el campo inválido o None.
Comentarios	La clase no mantiene estado de sesión propio: las credenciales se conservan en las cookies HttpOnly y SameSite=Lax, y la identidad se resuelve mediante AuthService. La operación token_refresh() cubre la renovación de credenciales descrita en el capítulo 18 y materializada en el 21.1.3. El rol de administrador no se valida en esta clase, sino en las operaciones administrativas de SD-005.	

Tabla x - Especificación de clase CL-0001

CL-0002	AuthService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/auth_service.py. Concentra la lógica criptográfica y el ciclo de vida de las credenciales del subsistema: genera y verifica el hash de las contraseñas con bcrypt, firma y valida los tokens de acceso JWT, gestiona los tokens de refresco en la base de datos mediante su hash SHA-256 y aplica la rotación con detección de uso de credenciales revocadas.		
Atributos	Nombre	Tipo	Descripción
	JWT_SECRET_KEY	str	Clave secreta de firma de los tokens de acceso JWT, obtenida del entorno; se emite una advertencia cuando se utiliza la clave de desarrollo por defecto.
	JWT_ACCESS_EXPIRE_MINUTES	int	Duración en minutos del token de acceso, configurada mediante variable de entorno.
	JWT_REFRESH_EXPIRE_DAYS	int	Duración en días del token de refresco, configurada mediante variable de entorno.
	REFRESH_ROTATION_GRACE_SECONDS	int	Periodo de gracia en segundos que tolera las renovaciones concurrentes del token de refresco sin interpretarlas como un robo.
Operaciones	Nombre	Descripción	
	hash_password(password)	Genera el hash bcrypt de la contraseña con un salt aleatorio en cada operación.	
	verify_password(password, password_hash)	Compara la contraseña introducida con el hash almacenado mediante bcrypt.checkpw.	
	create_access_token(user_id)	Firma un token de acceso JWT con el identificador del usuario y su fecha de expiración, usando la clave secreta del entorno.	
	verify_access_token(token)	Valida el token de acceso y devuelve el identificador del usuario, o None si el token es inválido o ha expirado.	
	create_refresh_token(user_id)	Genera un token de refresco aleatorio y persiste solo su hash SHA-256 en la tabla refresh_tokens, con la fecha de expiración y el indicador de revocación.	
	verify_refresh_token(raw_token)	Verifica el token de refresco: acepta el token activo, tolera el token revocado dentro del periodo de gracia y revoca todos los tokens del usuario si se detecta el uso de una credencial revocada fuera de él.	
	revoke_refresh_token(raw_token)	Marca como revocado el token de refresco en la base de datos.	
	rotate_refresh_token(old_raw_token)	Rota el token de refresco: verifica el token actual, revoca el anterior y emite uno nuevo; devuelve None si la rotación no es válida.	

	get_user_id_from_token(token)	Extrae el identificador del usuario de un token de acceso, devolviendo None si el token falta o no es válido; es el mecanismo común de identificación que consumen el resto de los subsistemas.
Comentarios	La clase no conserva la contraseña original en ningún atributo: una vez completado el hash, la credencial no se transmite a ningún componente de persistencia. La configuración sensible permanece fuera del código y se obtiene del entorno en tiempo de ejecución. Las operaciones de verificación y rotación de tokens de refresco implementan la política de seguridad de sesiones descrita en el capítulo 18 y materializada en los diagramas de secuencia del capítulo 21.	

Tabla x - Especificación de clase CL-0002

21.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 materializa el flujo clínico de la plataforma y agrupa cinco clases de diseño propias. La clase `InferenceRouter` se corresponde con el router `routers/inference.py` y actúa como fachada HTTP del diagnóstico: valida la imagen recibida, la conserva en el área de cargas y encola el trabajo de diagnóstico. Las clases `MIEngine`, `XaiGenerator` y `PdfGenerator` se corresponden con los servicios `services/ml_engine.py`, `services/xai_generator.py` y `services/pdf_generator.py`, y concentran respectivamente la predicción, la generación de los mapas de explicabilidad y la construcción del informe PDF. La clase `PDFReport` es la clase concreta del generador de informes que hereda del generador de la librería `fpdf`, lo que introduce la única herencia real del subsistema. El procesamiento de los trabajos encolados no pertenece a SD-002: lo ejecuta el worker de la cola de SD-006, que invoca las clases de predicción, explicabilidad e informe del presente subsistema.

21.2.1. Diagrama de clases del subsistema

El modelo de clases de SD-002 se representa en la figura 84. El diagrama muestra las cinco clases propias del subsistema con sus atributos y sus operaciones, la herencia de la clase `PDFReport` sobre la clase `FPDF` de la librería, y las dependencias hacia el servicio de idioma `LangService`, que pertenece a SD-006 y proporciona las etiquetas y los mensajes localizados. Las asociaciones entre las clases del subsistema reflejan la colaboración del flujo de diagnóstico: el generador de explicabilidad depende del motor de predicción para obtener el modelo cargado, y el generador de informes utiliza la clase concreta del documento PDF.

El diagrama identifica la única herencia del subsistema: la clase `PDFReport` especializa la clase `FPDF` de la librería de generación de documentos, redefiniendo la cabecera y el pie de página del informe médico. La clase `FPDF` se representa como una clase externa de la librería, marcada con el estereotipo de librería y con las operaciones básicas de generación que el informe hereda y utiliza —añadir página, configurar la fuente y el color, escribir celdas, insertar imágenes y producir el documento—, sin documentar su implementación interna por ser responsabilidad de la dependencia externa. El resto de las relaciones son asociaciones de dependencia. La dependencia de `XaiGenerator` hacia `MIEngine` refleja que la generación de los mapas de explicabilidad requiere el modelo cargado por el motor; las dependencias hacia `LangService` reflejan la localización de las etiquetas, los títulos y los mensajes; y la dependencia de `PdfGenerator` hacia `PDFReport` materializa la construcción del documento concreto. La clase `InferenceRouter` no presenta dependencias con el resto de las clases del subsistema porque su colaboración se limita al encolado del trabajo: el procesamiento de la predicción, de la explicabilidad y del informe lo orquesta el worker de SD-006, que invoca estas clases desde el ciclo asíncrono.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `predict()` de `InferenceRouter` implementa el CU-008 y las validaciones del CU-006, las operaciones de `MIEngine`, `XaiGenerator` y `PdfGenerator` materializan los pasos del procesamiento asíncrono de la figura y la operación `generate_medical_report()` de `PdfGenerator` materializa la generación del informe del CU-037.

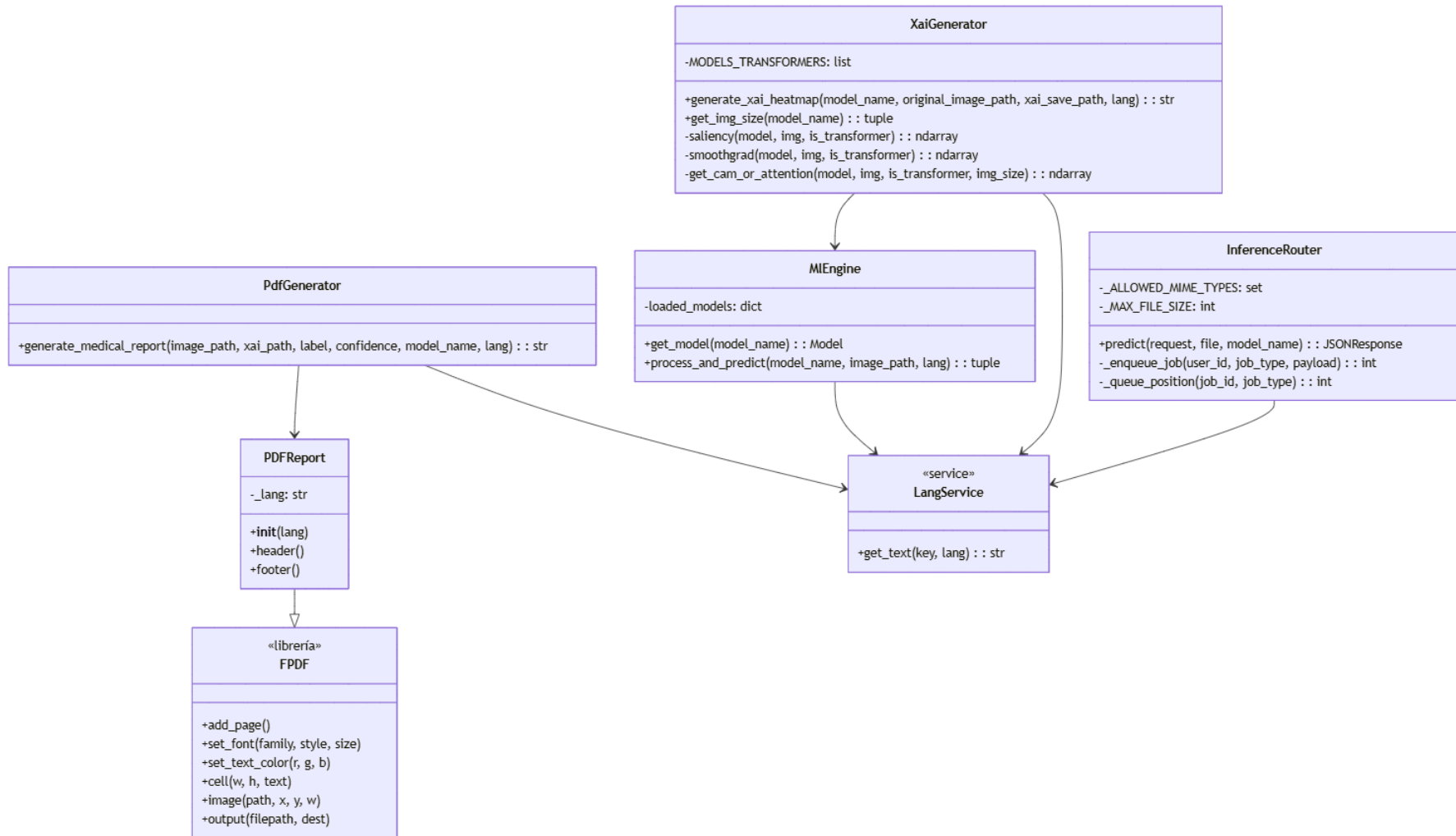


Ilustración x - Diagrama de clases del subsistema SD-002

21.2.2. Especificación de las clases

A continuación se definen las cinco clases propias de SD-002.

CL-0003	InferenceRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/inference.py. Actúa como fachada HTTP de la solicitud de diagnóstico: resuelve la identidad del usuario mediante AuthService, valida el tipo MIME y el tamaño de la imagen, la conserva en el área de cargas y crea el trabajo de diagnóstico en la cola con un payload serializable. No ejecuta la inferencia ni carga modelos.		
Atributos	Nombre	Tipo	Descripción
	_ALLOWED_MIME_TYPES	set	Conjunto de tipos MIME permitidos para la radiografía: JPEG y PNG.
	_MAX_FILE_SIZE	int	Tamaño máximo de la imagen en bytes: 10 MB.
Operaciones	Nombre	Descripción	
	predict(request, file, model_name)	Procesa la solicitud de diagnóstico: autentica al usuario (401 sin sesión), valida el tipo y el tamaño de la imagen (400 si no se permiten), guarda el fichero con un nombre basado en la fecha, encola el trabajo de tipo diagnosis y responde el estado queued con el identificador y la posición.	
	_enqueue_job(user_id, job_type, payload)	Operación privada que inserta un trabajo en la tabla job_queue con el tipo y el payload indicados, devolviendo el identificador del trabajo creado.	
	_queue_position(job_id, job_type)	Operación privada que calcula la posición del trabajo en la cola, respetando la prioridad de los diagnósticos frente al resto de tipos.	
Comentarios	La validación del fichero se realiza antes de encolar, de modo que una petición inválida no ocupa una posición en la cola ni consume recursos del worker. El payload del trabajo contiene únicamente el modelo, la ruta de la imagen y el idioma; ni el modelo ni la imagen completa se serializan en la base de datos.		

Tabla x - Especificación de clase CL-0003

CL-0004	MIEngine		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/ml_engine.py. Concentra la inferencia del diagnóstico: carga y reutiliza los modelos de aprendizaje profundo en memoria, prepara la imagen recibida y produce la etiqueta de la predicción y su nivel de confianza. Combina arquitecturas convolucionales y arquitecturas Transformer, y localiza las etiquetas mediante el servicio de idioma.		
Atributos	Nombre	Tipo	Descripción
	loaded_models	dict	Diccionario global que conserva los modelos cargados en memoria, indexado por nombre de arquitectura, de modo que la primera consulta paga la carga de los pesos y las posteriores reutilizan el modelo.
Operaciones	Nombre	Descripción	
	get_model(model_name)	Carga el modelo de la arquitectura indicada y lo conserva en la caché de modelos; si el modelo ya está cargado, lo devuelve sin volver a leer los pesos.	
	process_and_predict(model_name, image_path, lang)	Preprocesa la imagen según la arquitectura, ejecuta la inferencia y devuelve la etiqueta de la predicción (Neumonía o Normal) y el nivel de confianza, localizados mediante LangService.	
Comentarios	La reutilización de los modelos en memoria implementa el requisito de tiempo de respuesta de la inferencia (RNF-019): la optimización pertenece al motor y no al router. Un modelo no reconocido falla antes de producir un resultado ambiguo, de modo que la validación de la arquitectura queda diferida al motor.		

Tabla x - Especificación de clase CL-0004

CL-0005	XaiGenerator		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/xai_generator.py. Genera el mapa de explicabilidad adecuado a la arquitectura utilizada: los métodos basados en gradientes para las arquitecturas convolucionales y los mapas de atención para las arquitecturas Transformer, y compone la figura con la radiografía original y los tres mapas superpuestos.		
Atributos	Nombre	Tipo	Descripción
	MODELS_TRANSFORMERS	list	Lista de las arquitecturas Transformer del proyecto, utilizada para bifurcar el método de explicabilidad.
Operaciones	Nombre	Descripción	
	generate_xai_heatmap(model_name, original_image_path, xai_save_path, lang)	Genera la figura de explicabilidad (original, Saliency, SmoothGrad y superposición de Grad-CAM o atención), la guarda en la ruta indicada y devuelve dicha ruta.	
	get_img_size(model_name)	Devuelve el tamaño de entrada de la imagen según la arquitectura (299×299, 384×384 o 224×224).	
	saliency(model, img, is_transformer)	Operación privada que calcula el mapa de Saliency mediante los gradientes de la puntuación respecto a la imagen.	
	smoothgrad(model, img, is_transformer)	Operación privada que promedia los mapas de Saliency con ruido gaussiano y normaliza el resultado.	
	get_cam_or_attention(model, img, is_transformer, img_size)	Operación privada que calcula el Grad-CAM sobre la última capa convolucional para las arquitecturas convolucionales o el mapa de atención para las Transformer, con un fallback basado en gradientes.	
Comentarios	La generación de los mapas se realiza sobre el modelo cargado por MIEngine, de modo que la clase depende del motor de predicción. La figura resultante se sirve como recurso estático y su ruta se persiste en la consulta, sin repetir el cálculo en las visualizaciones posteriores.		

Tabla x - Especificación de clase CL-0005

CL-0006	PdfGenerator		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/pdf_generator.py. Construye el informe PDF del diagnóstico a partir de la imagen original, el mapa de explicabilidad, la etiqueta, la confianza y el modelo utilizado, empleando la clase concreta PDFReport y los textos localizados del servicio de idioma.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: construye el informe a partir de los parámetros recibidos en la operación.
Operaciones	Nombre	Descripción	
	generate_medical_report(image_path, xai_path, label, confidence, model_name, lang)	Compone el informe A4 con la fecha, el modelo, el diagnóstico con su color según el resultado, la confianza y las dos imágenes (original y mapa), lo guarda en static/reports con un nombre basado en la fecha y devuelve la ruta del documento.	
Comentarios	La generación del informe se produce durante el procesamiento del diagnóstico, de modo que el documento está disponible cuando la consulta se completa y la descarga posterior no depende de una nueva operación del sistema. El informe se trata como parte de la información protegida de la consulta.		

Tabla x - Especificación de clase CL-0006

CL-0007	PDFReport		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase concreta del informe PDF de diagnóstico, correspondiente a la clase PDFReport del módulo services/pdf_generator.py. Especializa la clase FPDF de la librería de generación de documentos y redefine la cabecera y el pie de página del informe, además de conservar el idioma de los textos.		
Atributos	Nombre	Tipo	Descripción
	_lang	str	Idioma utilizado para localizar los textos del informe (cabecera, pie y etiquetas).
Operaciones	Nombre	Descripción	
	__init__(lang)	Inicializa el documento de la librería base y conserva el idioma para localizar los textos.	
	header()	Redefine la cabecera de cada página del informe con el título y el color corporativo.	
	footer()	Redefine el pie de página con el número de página.	
Comentarios	Esta clase constituye la herencia real del subsistema: su relación con FPDF se representa en el modelo de clases como una especialización. El generador de informes la instancia para componer el documento, de modo que el conocimiento del formato queda encapsulado en la clase y fuera del router.		

Tabla x - Especificación de clase CL-0007

21.3. *Subsistema SD-003: Historial y gestión de consultas*

El subsistema SD-003 materializa la recuperación y la gestión de las consultas ya realizadas y agrupa una única clase de diseño propia. La clase HistoryRouter se corresponde con el router routers/history.py y concentra las operaciones del historial: la consulta del listado, la actualización del nombre y la eliminación de una consulta, todas ellas precedidas de la comprobación de propiedad. El subsistema no dispone de clases de servicio propias, porque su lógica se limita a la recuperación de los registros persistidos por SD-002 y a la actualización de la etiqueta de organización; la comprobación de propiedad se resuelve dentro de la propia clase. HistoryRouter depende de AuthService, perteneciente a SD-001, para resolver la identidad del usuario, y de LangService, perteneciente a SD-006, para los mensajes localizados de las respuestas.

21.3.1. Diagrama de clases del subsistema

El modelo de clases de SD-003 se representa en la figura 85. El diagrama muestra la clase propia del subsistema con sus operaciones, y las dependencias hacia las clases AuthService y LangService, que pertenecen a otros subsistemas y se representan con las operaciones que el historial consume. El modelo no presenta herencias ni clases abstractas: la clase HistoryRouter es una clase concreta que orquesta las operaciones del historial y delega la identidad en el subsistema de acceso.

El diagrama refleja la simplicidad estructural del subsistema: una sola clase concentra la consulta del listado, el renombrado y la eliminación, y la operación privada `_check_consultation_ownership()` que protege las dos operaciones de modificación. La dependencia hacia AuthService materializa el mecanismo común de identificación de SD-001, del que el historial obtiene el usuario para filtrar y comprobar la propiedad de las consultas. La dependencia hacia LangService localiza los mensajes de error y de respuesta. La relación de SD-003 con SD-002 es de productor-consumidor de datos y no se representa como una asociación entre clases: los registros de consultations que el historial lee son creados por el worker de SD-002, y la colaboración se produce a través de la persistencia compartida.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `get_history()` implementa el CU-011; la operación `update_patient_name()` implementa el CU-013; la operación `delete_history_record()` implementa el CU-014; y la operación privada `_check_consultation_ownership()` materializa la comprobación de propiedad que condiciona las respuestas HTTP 404 y 403 de los diagramas de secuencia.

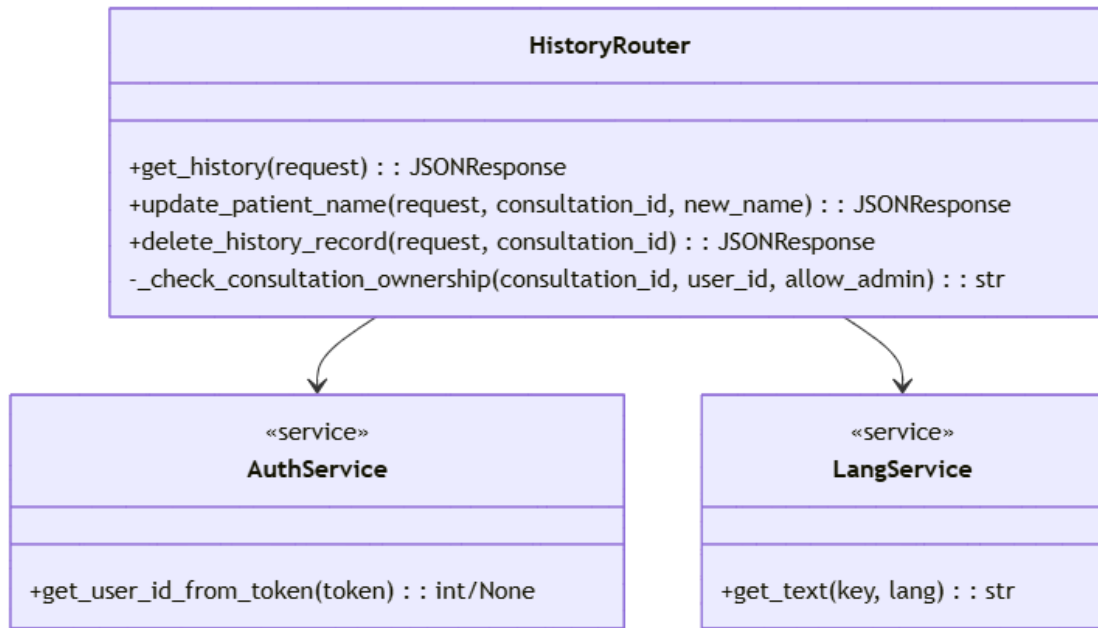


Ilustración x - Diagrama de clases del subsistema SD-003

21.3.2. Especificación de las clases

A continuación se define la clase propia de SD-003.

CL-0008	HistoryRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/history.py. Proporciona la recuperación y la gestión de las consultas de diagnóstico de un usuario: devuelve el listado del historial filtrado por el propietario, actualiza el nombre mostrado de una consulta y elimina una consulta de forma física, aplicando en las operaciones de modificación la comprobación de propiedad con la excepción controlada del administrador.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: cada operación resuelve la identidad y accede a la persistencia a partir de los parámetros recibidos.
Operaciones	Nombre	Descripción	
	get_history(request)	Devuelve el listado de consultas del usuario autenticado, filtrado por user_id y ordenado por fecha descendente; responde HTTP 401 sin sesión y formatea las fechas antes de formar la respuesta.	
	update_patient_name(request, consultation_id, new_name)	Actualiza el nombre mostrado de una consulta: comprueba la propiedad, responde HTTP 404 si la consulta no existe y HTTP 403 si pertenece a otro usuario, y ejecuta la actualización del nombre cuando la propiedad se confirma.	
	delete_history_record(request, consultation_id)	Elimina de forma física una consulta: comprueba la propiedad, responde HTTP 404 o HTTP 403 en los casos de inexistencia o falta de permisos, y ejecuta el borrado cuando la propiedad se confirma.	
	_check_consultation_ownership(consultation_id, user_id, allow_admin)	Operación privada que comprueba que la consulta existe y pertenece al usuario solicitante, permitiendo la excepción del rol admin; devuelve not_found, forbidden u ok.	
Comentarios	La clase no vuelve a ejecutar el modelo para mostrar una consulta anterior: recupera los metadatos y las rutas de los artefactos ya persistidos. El nombre actualizado se aplica sobre la columna patient_name, que funciona como etiqueta de organización y no como identificación clínica del paciente. La eliminación es un borrado físico, sin auditoría automática, tal y como se declaró en el capítulo 18.		

Tabla x - Especificación de clase CL-0008

21.4. Subsistema SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 materializa el laboratorio de experimentación MLOps y agrupa cinco clases de diseño propias. La clase `TrainerRouter` se corresponde con el router `routers/trainer.py` y actúa como fachada ligera del laboratorio: expone los endpoints de configuración, lanzamiento, consulta y gestión de las sesiones, y delega la lógica en los servicios. La clase `ChatbotService` se corresponde con `services/chatbot_service.py` y resuelve la configuración conversacional del experimento mediante el asistente externo. La clase `MlopsEngine` se corresponde con `services/mlops_engine.py` y concentra la organización de las sesiones, la ejecución de los pipelines de entrenamiento y la lectura de los resultados. Las clases `PdfGeneratorMlops` y `MedicalReport`, correspondientes a `services/pdf_generator_mlops.py`, generan el informe consolidado de la sesión; `MedicalReport` es la clase concreta que hereda de la librería `FPDF`, segunda herencia real del diseño de clases de `vitalXAI`.

21.4.1. Diagrama de clases del subsistema

El modelo de clases de SD-004 se representa en la figura 86. El diagrama muestra las cinco clases propias del subsistema, la herencia de `MedicalReport` sobre la clase `FPDF` de la librería, la dependencia del servicio conversacional hacia el proveedor externo de inteligencia artificial, y las dependencias del router hacia las clases propias y hacia los servicios transversales `AuthService` y `LangService`. El router concentra las asociaciones del subsistema: delega la exploración, el entrenamiento y la consulta de resultados en `MlopsEngine`, la conversación en `ChatbotService` y la generación del informe en `PdfGeneratorMlops`.

El diagrama identifica la herencia de `MedicalReport` sobre la clase `FPDF` de la librería de generación de documentos, que se representa con las operaciones básicas que el informe hereda y utiliza. El proveedor externo de inteligencia artificial se modela como una clase externa con el estereotipo de proveedor, con la operación de generación de respuestas que el servicio conversacional invoca; la clave de la API no forma parte de la interfaz de la clase, sino del atributo de configuración de `ChatbotService`. Las dependencias hacia `AuthService` y `LangService` reflejan los servicios transversales de SD-001 y SD-006 que el router consume. La clase `MlopsEngine` constituye el núcleo del subsistema: orquesta los scripts de entrenamiento, el análisis XAI y la comparación estadística, y organiza la lectura y escritura de los resultados en el sistema de ficheros, sin depender del servicio de idioma porque sus mensajes de registro son textos fijos del pipeline.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los casos de uso reales del capítulo 21. Las operaciones de `TrainerRouter` exponen los endpoints de los CU-015 a CU-030 y CU-039; las operaciones de `MlopsEngine` implementan los flujos del procesamiento del entrenamiento y de la validación externa; la operación `generate_pdf_report()` de `PdfGeneratorMlops` materializa el CU-028; y las operaciones de `ChatbotService` materializan la configuración conversacional del CU-016.

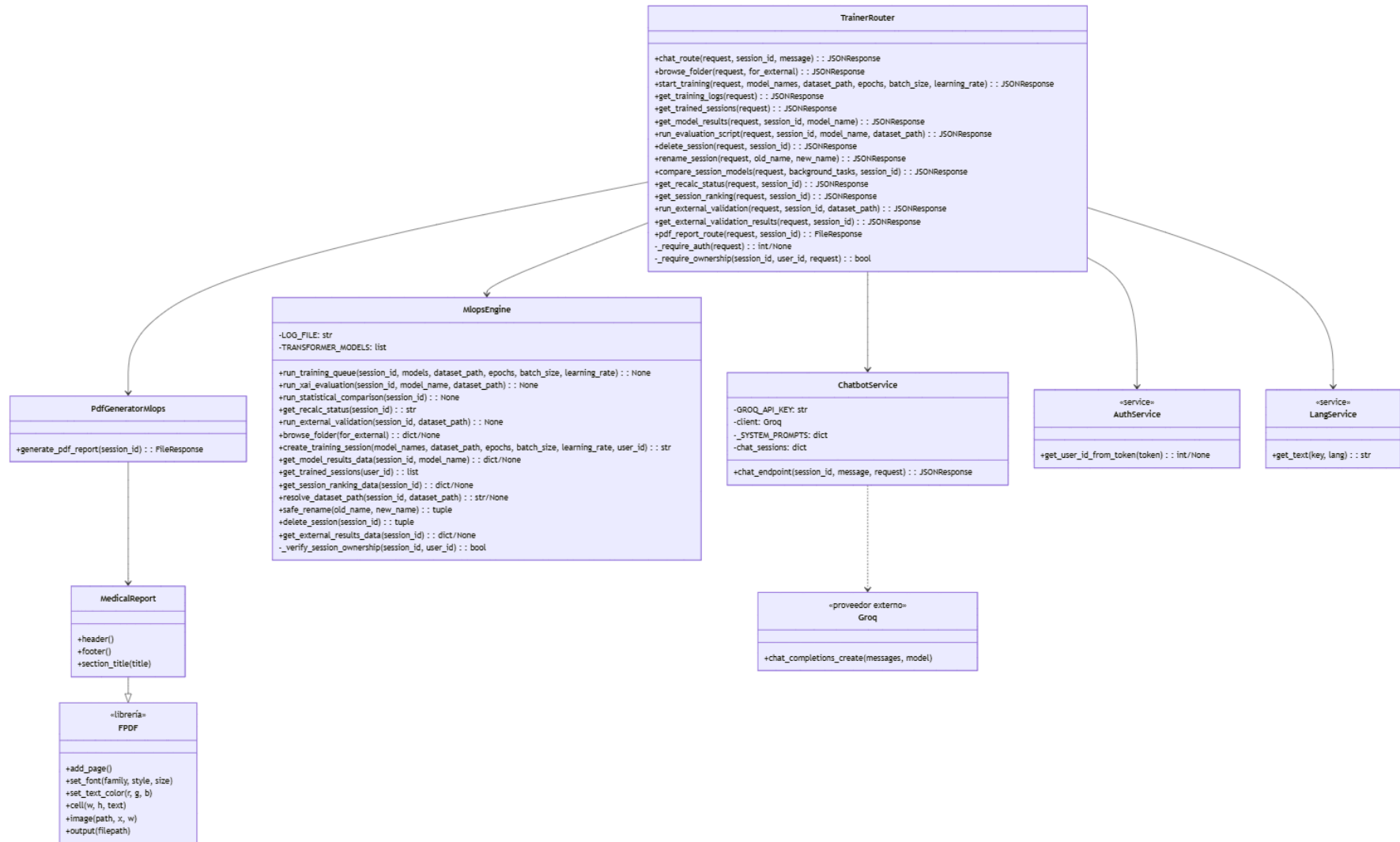


Ilustración x - Diagrama de clases del subsistema SD-004

21.4.2. Especificación de las clases

A continuación se definen las cinco clases propias de SD-004.

CL-0009	TrainerRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/trainer.py. Actúa como fachada ligera del laboratorio MLOps: expone los endpoints de conversación, exploración del dataset, lanzamiento de entrenamientos, consulta de logs, sesiones y resultados, ejecución del análisis XAI, comparativa estadística, validación externa, informes, renombrado y eliminación de sesiones. Comprueba la autenticación y la propiedad de las sesiones y delega la lógica en los servicios del subsistema.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: reexporta constantes del motor para compatibilidad y delega el resto en los servicios.
Operaciones	Nombre	Descripción	
	chat_route(request, session_id, message)	Gestiona la conversación con el asistente de configuración, delegando en ChatbotService.	
	browse_folder(request, for_external)	Explora la carpeta del dataset de entrenamiento o de validación externa mediante MlopsEngine.	
	start_training(request, model_names, dataset_path, epochs, batch_size, learning_rate)	Lanza un experimento: valida la ruta del dataset, crea la sesión mediante MlopsEngine y encola el trabajo de tipo training.	
	get_training_logs(request)	Devuelve las últimas líneas del registro de entrenamiento.	
	get_trained_sessions(request)	Devuelve las sesiones de entrenamiento del usuario mediante MlopsEngine.	
	get_model_results(request, session_id, model_name)	Devuelve los resultados de un modelo de la sesión, con las métricas K-fold, la calibración y las métricas XAI.	
	update_patient_name(request, session_id, model_name, dataset_path, consultation_id, new_name)	Ejecuta el análisis XAI manual de un modelo mediante MlopsEngine.	

	<code>delete_session(request, session_id)</code>	Elimina una sesión y sus artefactos mediante MlopsEngine, tras comprobar la propiedad.
	<code>rename_session(request, old_name, new_name)</code>	Renombra una sesión mediante MlopsEngine, tras comprobar la propiedad.
	<code>compare_session_models(request, background_tasks, session_id)</code>	Programa el recálculo de la comparativa estadística como tarea en segundo plano.
	<code>get_recalc_status(request, session_id)</code>	Devuelve el estado del recálculo de la comparativa (running o completed).
	<code>get_session_ranking(request, session_id)</code>	Devuelve el ranking de modelos y el heatmap de Wilcoxon mediante MlopsEngine.
	<code>run_external_validation(request, session_id, dataset_path)</code>	Encola la validación externa de la sesión con el tipo external_validation.
	<code>get_external_validation_results(request, session_id)</code>	Devuelve las métricas, la curva ROC y la matriz de DeLong de la validación externa.
	<code>pdf_report_route(request, session_id)</code>	Genera y sirve el informe PDF de la sesión mediante PdfGeneratorMlops.
	<code>_require_auth(request)</code>	Operación privada que resuelve la identidad del usuario mediante AuthService, devolviendo None sin sesión.
	<code>_require_ownership(session_id, user_id, request)</code>	Operación privada que comprueba la propiedad de la sesión mediante MlopsEngine, con la excepción del rol admin cuando la ruta lo permite.
Comentarios	La clase concentra un gran número de operaciones porque el laboratorio expone numerosos puntos de consulta y gestión; todas ellas mantienen la frontera de autorización: la comprobación de propiedad se resuelve antes de abrir cualquier operación sobre una sesión. Las operaciones de lanzamiento de entrenamiento y de validación externa encolan trabajos en la cola de SD-006.	

Tabla x - Especificación de clase CL-0009

CL-0010	ChatbotService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/chatbot_service.py. Gestiona la configuración conversacional del experimento: recibe los mensajes del usuario en lenguaje natural, los envía al asistente externo y devuelve una configuración estructurada del entrenamiento, solicitando los parámetros que falten. Trata al proveedor de inteligencia artificial como una frontera externa.		
Atributos	Nombre	Tipo	Descripción
	GROQ_API_KEY	str	Clave de la API del proveedor del asistente, obtenida del entorno; el único atributo del subsistema que conserva la credencial del proveedor.
	client	Groq	Cliente del proveedor externo, instanciado con la clave de la API; es None si la clave no está configurada.
	_SYSTEM_PROMPTS	dict	Diccionario de prompts del sistema por idioma, que guían la configuración conversacional.
	chat_sessions	dict	Diccionario que conserva el historial de mensajes de cada conversación, indexado por el identificador de la sesión de chat.
Operaciones	Nombre	Descripción	
	chat_endpoint(session_id, message, request)	Procesa un mensaje de la conversación: selecciona el prompt del sistema según el idioma, mantiene el historial de la sesión, solicita la configuración al proveedor externo y devuelve la respuesta con los parámetros de la configuración o la solicitud de los faltantes.	
Comentarios	La clase es el único componente del sistema que necesita la clave de la API del asistente; los routers no la incluyen en las respuestas ni en los registros. Los errores del proveedor externo se tratan como condiciones de la frontera y no se confunden con los errores de persistencia o de entrenamiento.		

Tabla x - Especificación de clase CL-0010

CL-0011	MlopsEngine		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/mlops_engine.py. Constituye el núcleo del laboratorio: organiza las sesiones de entrenamiento en el sistema de ficheros, orquesta la ejecución de los scripts del pipeline —entrenamiento K-fold, análisis XAI, comparación estadística y validación externa— y resuelve la lectura de los resultados y la comprobación de propiedad de las sesiones.		
Atributos	Nombre	Tipo	Descripción
	LOG_FILE	str	Ruta del archivo de registro del entrenamiento, donde el motor escribe la progresión del pipeline.
	TRANSFORMER_MODELS	list	Lista de las arquitecturas Transformer del proyecto, utilizada para bifurcar el script de entrenamiento.
Operaciones	Nombre	Descripción	
	run_training_queue(session_id, models, dataset_path, epochs, batch_size, learning_rate)	Orquesta el entrenamiento completo de la sesión: por cada modelo lanza el script de entrenamiento CNN o Transformer y los scripts de explicabilidad, y al finalizar ejecuta la comparación estadística que genera el ranking y la matriz de Wilcoxon.	
	run_xai_evaluation(session_id, model_name, dataset_path)	Ejecuta el análisis XAI cualitativo y cuantitativo de un modelo en modo manual.	
	run_statistical_comparison(session_id)	Regenera la comparativa estadística de la sesión, gestionando el marcador de estado del recálculo.	
	get_recalc_status(session_id)	Devuelve completed o running según el marcador de estado del recálculo.	
	run_external_validation(session_id, dataset_path)	Ejecuta la validación externa de la sesión y el test estadístico de DeLong sobre la cohorte independiente.	
	browse_folder(for_external)	Devuelve la ruta del dataset configurada por entorno o abre el selector de carpetas; distingue el dataset de entrenamiento del de validación externa.	
	create_training_session(model_names, dataset_path, epochs, batch_size, learning_rate, user_id)	Crea el directorio de la sesión en training_results y escribe su configuración, con el identificador del usuario propietario; devuelve el identificador de la sesión.	
	get_model_results_data(session_id, model_name)	Lee las métricas K-fold, la calibración y las métricas XAI de un modelo, junto con las rutas de sus artefactos; devuelve None si el modelo no dispone de resultados.	
	get_trained_sessions(user_id)	Enumera las sesiones con resultados que pertenecen al usuario, en orden descendente, con sus modelos.	

	get_session_ranking_data(session_id)	Lee el ranking de modelos, el heatmap de Wilcoxon y la configuración de la sesión; devuelve None si no hay ranking.
	resolve_dataset_path(session_id, dataset_path)	Devuelve la ruta del dataset indicada o la conservada en la sesión.
	safe_rename(old_name, new_name)	Valida y sanitiza el nuevo nombre de la sesión y ejecuta el renombrado del directorio, devolviendo el estado y el contenido de la respuesta.
	delete_session(session_id)	Elimina el directorio de la sesión y sus artefactos, devolviendo el estado y el contenido de la respuesta.
	get_external_results_data(session_id)	Lee las métricas, la curva ROC y la matriz de DeLong de la validación externa; devuelve None si no existen resultados.
	_verify_session_ownership(session_id, user_id)	Operación privada que comprueba que la configuración de la sesión contiene el identificador del usuario, como regla de propiedad del laboratorio.
Comentarios	La clase combina la persistencia híbrida de la plataforma: coordina la escritura de los artefactos en el sistema de ficheros y conserva en MySQL únicamente los trabajos de la cola. La comprobación de propiedad de las sesiones se resuelve en esta clase, que la comparte con el router sin duplicarla. Los scripts del pipeline se invocan como procesos externos que leen la configuración desde variables de entorno.	

Tabla x - Especificación de clase CL-0011

CL-0012	PdfGeneratorMlops		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/pdf_generator_mlops.py. Construye el informe consolidado de una sesión de entrenamiento: recopila la configuración del experimento, el ranking con el heatmap de Wilcoxon, la validación externa con la curva ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor, empleando la clase concreta MedicalReport.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	generate_pdf_report(session_id)	Compone el informe de la sesión, lo guarda en el directorio de la sesión con el nombre Informe_Completo_{session_id}.pdf y lo devuelve como descarga; responde HTTP 404 si la sesión no existe.	
Comentarios	La generación del informe separa el conocimiento del formato del router: el generador recibe la información preparada por el motor y construye el documento, de modo que el router no conoce el formato interno del informe. El informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios.		

Tabla x - Especificación de clase CL-0012

CL-0013	PdfGeneratorMlops		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase concreta del informe MLOps, correspondiente a la clase MedicalReport del módulo services/pdf_generator_mlops.py. Especializa la clase FPDF de la librería de generación de documentos y redefine la cabecera corporativa, el pie de página y el formato de los títulos de sección del informe.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	header()	Redefine la cabecera de cada página del informe con el título corporativo y el encabezado del protocolo MLOps.	
	footer()	Redefine el pie de página con el número de página y la indicación de generación automática.	
	section_title(title)	Escribe un título de sección con el formato propio del informe.	
Comentarios	Esta clase constituye la segunda herencia real del diseño de clases de vitalXAI: su relación con FPDF se representa en el modelo de clases como una especialización. El generador de informes la instancia para componer el documento consolidado de la sesión.		

Tabla x - Especificación de clase CL-0013

21.5. *Subsistema SD-005: Supervisión y administración*

El subsistema SD-005 materializa las operaciones de supervisión y administración reservadas al rol de administrador y agrupa una única clase de diseño propia. La clase AdminRouter se corresponde con el router routers/admin.py y concentra las tres operaciones de consulta administrativa: el listado de usuarios, las consultas de un usuario concreto y el detalle de una consulta, todas ellas precedidas de la comprobación de la autorización administrativa. El subsistema no dispone de clases de servicio propias, porque su responsabilidad es establecer la autorización y coordinar las consultas globales reutilizando los servicios existentes. AdminRouter depende de AuthService, perteneciente a SD-001, para resolver la identidad y consultar el rol, y de MlopsEngine, perteneciente a SD-004, para recuperar las sesiones del laboratorio de un usuario en la operación de consulta de su actividad.

21.5.1. Diagrama de clases del subsistema

El modelo de clases de SD-005 se representa en la figura 87. El diagrama muestra la clase propia del subsistema con sus operaciones, y las dependencias hacia las clases AuthService y MlopsEngine, pertenecientes a SD-001 y SD-004, representadas con las operaciones que la supervisión consume. El modelo no presenta herencias ni clases abstractas: la clase AdminRouter es una clase concreta que centraliza la comprobación de permisos y coordina las consultas globales.

El diagrama refleja la simplicidad estructural del subsistema y su decisión de diseño central: la operación privada `_require_admin()` se antepone a todas las consultas administrativas, de modo que ninguna ruta ejecuta una operación de lectura sobre usuarios o consultas sin haber resuelto antes la identidad y el rol del solicitante. La dependencia hacia AuthService materializa el mecanismo de identificación de SD-001, del que el router obtiene el usuario y consulta su rol; la dependencia hacia MlopsEngine materializa la recuperación de las sesiones del laboratorio en la consulta de la actividad de un usuario. El subsistema no depende del servicio de idioma porque sus mensajes de autorización son textos fijos, y no duplica la lógica de propiedad del historial ni del laboratorio: accede a los datos con las mismas tablas y servicios que el resto de la aplicación.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación `admin_users()` implementa el CU-031; la operación `admin_user_consultations()` implementa el CU-032; la operación `admin_get_consultation()` implementa el CU-033; y la operación privada `_require_admin()` materializa la comprobación de permisos que condiciona las respuestas HTTP 401 y 403 de los diagramas de secuencia.

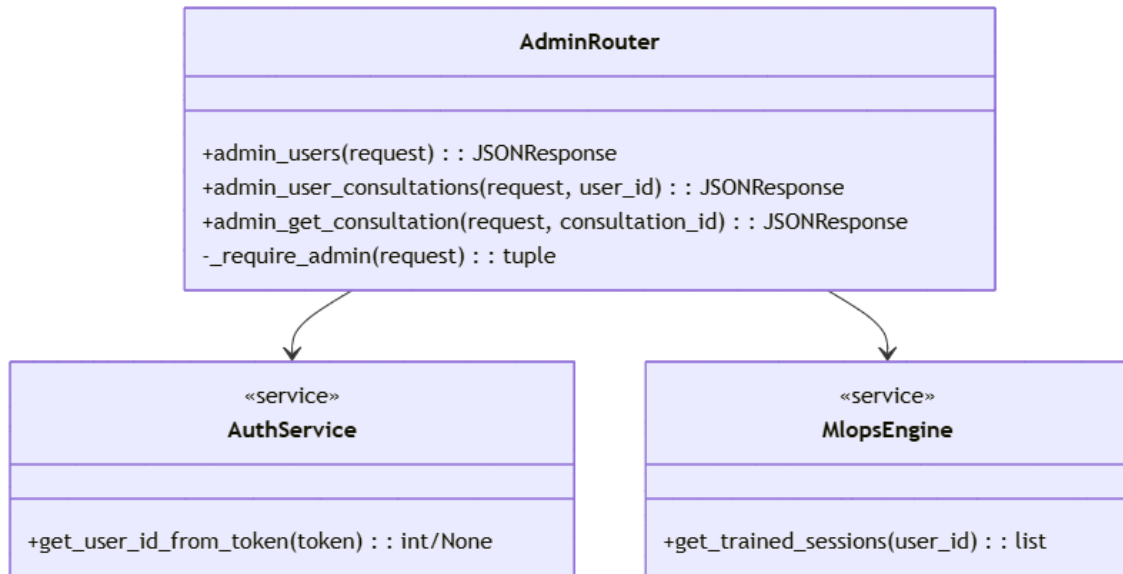


Ilustración x - Diagrama de clases del subsistema SD-005

21.5.2. Especificación de las clases

A continuación se define la clase propia de SD-005.

CL-0014	AdminRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/admin.py. Proporciona las operaciones de supervisión administrativa: el listado de usuarios con sus recuentos de diagnósticos y de sesiones del laboratorio, las consultas y las sesiones de un usuario concreto, y el detalle de una consulta. Centraliza la comprobación de la autorización administrativa en la operación privada _require_admin() y reutiliza los servicios existentes sin duplicar la lógica de propiedad.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	admin_users(request)	Devuelve el listado de usuarios con el número de diagnósticos, obtenido mediante una consulta agregada sobre users y consultations, y con el número de sesiones del laboratorio, calculado inspeccionando las configuraciones del sistema de ficheros.	
	admin_user_consultations(request, user_id)	Devuelve las consultas de diagnóstico de un usuario concreto, ordenadas por fecha descendente, junto con sus sesiones de entrenamiento recuperadas mediante MlopsEngine; responde HTTP 404 si el usuario no existe.	
	admin_get_consultation(request, consultation_id)	Devuelve el detalle completo de una consulta por su identificador, con las rutas de los artefactos, la predicción, la confianza y los metadatos; responde HTTP 404 si la consulta no existe.	
	_require_admin(request)	Operación privada que resuelve la identidad mediante AuthService y consulta el rol en la tabla users, devolviendo una tupla con el identificador y un indicador de rol administrativo; distingue la ausencia de identidad, la identidad sin privilegios y la identidad administrativa.	
Comentarios	La clase mantiene la frontera entre el control de permisos y la representación de la información: no duplica la lógica de propiedad del historial ni del laboratorio, y la autorización administrativa se comprueba en el servidor, no se infiere de parámetros del navegador. La protección reutiliza la identidad de SD-001 sin una segunda autenticación, y la traza de auditoría administrativa permanece como condición pendiente, tal y como se declaró en el capítulo 18.		

Tabla x - Especificación de clase CL-0014

21.6. *Subsistema SD-006: Cola de trabajos y capacidades transversales*

El subsistema SD-006 materializa la cola persistente de trabajos y las capacidades transversales de la plataforma, y agrupa cuatro clases de diseño propias. La clase QueueRouter se corresponde con el router routers/queue.py y expone la consulta del estado de los trabajos y la cancelación de un trabajo pendiente. La clase QueueWorker se corresponde con el servicio services/queue_worker.py y actúa como consumidor de la cola: reclama los trabajos pendientes, ejecuta el flujo correspondiente en el executor y actualiza el estado del trabajo. La clase LangService se corresponde con el servicio services/lang.py y concentra la internacionalización de la plataforma. La clase QueueJob modela la entidad persistente de la cola, la fila de la tabla job_queue, que constituye la clase más compleja del subsistema por su máquina de estados, compartida por los subsistemas que generan trabajos.

21.6.1. Diagrama de clases del subsistema

El modelo de clases de SD-006 se representa en la figura 88. El diagrama muestra las cuatro clases propias del subsistema, la dependencia del router hacia el servicio de identidad de SD-001, y las dependencias del worker hacia las clases MIEngine y MlopsEngine, pertenecientes a SD-002 y SD-004, que representan los flujos que el worker ejecuta según el tipo de trabajo. La clase QueueJob se representa con los atributos de su estado persistido y aparece asociada tanto al router, que consulta y cancela los trabajos, como al worker, que los reclama y actualiza.

El diagrama refleja la frontera entre el ciclo de petición y la ejecución asíncrona: el router y el worker comparten la persistencia de la cola, materializada en la clase QueueJob, pero no comparten el ciclo de petición. El worker despacha el trabajo según su tipo: para los diagnósticos invoca la clase MIEngine de SD-002, y para los entrenamientos y las validaciones externas invoca la clase MlopsEngine de SD-004; esas clases se representan con las operaciones que el worker utiliza. La clase LangService no presenta dependencias de colaboración en el diagrama porque sus consumidores pertenecen a otros subsistemas: el mecanismo de internacionalización se ofrece como un servicio transversal que cualquier router o servicio invoca. El modelo no presenta herencias ni clases abstractas.

Desde el punto de vista de la trazabilidad, las operaciones del modelo de clases materializan los mensajes de los diagramas de secuencia del capítulo 21. La operación queue_status() de QueueRouter implementa el CU-034; la operación cancel_job() implementa el CU-035; y las operaciones del worker materializan el procesamiento asíncrono que orquesta los flujos de SD-002 y SD-004. La clase QueueJob concentra la máquina de estados de la cola, que se describe en su definición.

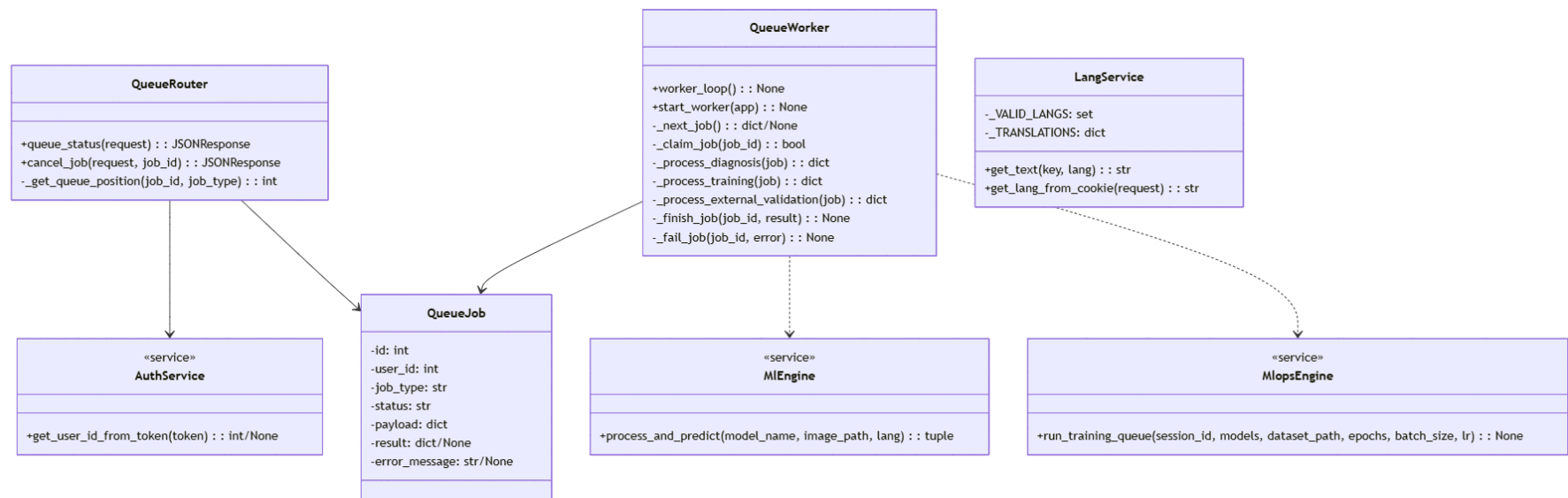


Ilustración x - Diagrama de clases del subsistema SD-006

21.6.2. Especificación de las clases

A continuación se definen las cuatro clases propias de SD-006.

CL-0015	QueueRouter		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo routers/queue.py. Expone la consulta del estado de los trabajos de la cola y la cancelación de un trabajo pendiente: devuelve los trabajos recientes del usuario con su estado y su posición, interpreta el payload según el tipo de trabajo y aplica la cancelación mediante una actualización condicional.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	queue_status(request)	Devuelve los últimos veinte trabajos del usuario con su estado, sus fechas y la posición de los trabajos encolados; interpreta el payload por tipo (modelo del diagnóstico, sesión y modelos del entrenamiento, o modelos de la validación externa leídos de su configuración) sin enviar el payload completo al navegador.	
	cancel_job(request, job_id)	Cancela un trabajo mediante una actualización condicionada a que pertenezca al usuario y continúe en queued; responde HTTP 200 si la actualización afectó a una fila y HTTP 404 si el trabajo no está en cola.	
	_get_queue_position(job_id, job_type)	Operación privada que calcula la posición de un trabajo encolado, respetando la prioridad de la cola según el tipo de trabajo.	
Comentarios	La cancelación solo opera sobre trabajos pendientes: un trabajo reclamado por el worker no se interrumpe, en coherencia con la máquina de estados de la cola. La consulta del estado filtra siempre por el usuario autenticado, aplicando el aislamiento de datos entre cuentas.		

Tabla x - Especificación de clase CL-0015

CL-0016	QueueWorker		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/queue_worker.py. Actúa como consumidor de la cola de trabajos: al iniciar la aplicación restablece los trabajos que quedaron en estado running, selecciona el siguiente trabajo pendiente, lo reclama mediante una actualización condicionada a que siga en queued y ejecuta el flujo correspondiente en el executor, marcando el resultado como completado o fallido. No comparte el ciclo de petición con los routers.		
Atributos	Nombre	Tipo	Descripción
	-	-	La clase no presenta atributos de estado propios: compone el informe a partir de los artefactos persistidos en la sesión.
Operaciones	Nombre	Descripción	
	worker_loop()	Bucle infinito del worker: selecciona el siguiente trabajo pendiente, lo reclama y ejecuta el flujo correspondiente en el executor, esperando cuando la cola está vacía.	
	start_worker(app)	Restablece los trabajos en running a queued y crea la tarea asíncrona del bucle del worker, conservándola en el estado de la aplicación.	
	_next_job()	Operación privada que selecciona el primer trabajo queued, con prioridad para los diagnósticos frente a los entrenamientos.	
	_claim_job(job_id)	Operación privada que reclama un trabajo mediante una actualización condicionada a que siga en queued, devolviendo True solo si la actualización afectó a una fila.	
	_process_diagnosis(job)	Operación privada que procesa un trabajo de diagnóstico: invoca MIEngine, XaiGenerator y PdfGenerator de SD-002 y persiste la consulta con sus artefactos.	
	_process_training(job)	Operación privada que procesa un trabajo de entrenamiento, delegando en MlopsEngine.	
	_process_external_validation(job)	Operación privada que procesa una validación externa, delegando en MlopsEngine.	
	_finish_job(job_id, result)	Operación privada que marca el trabajo como completed con su resultado en formato JSON.	
	_fail_job(job_id, error)	Operación privada que marca el trabajo como failed con el mensaje de error limitado.	
Comentarios	La reclamación condicional evita que dos iteraciones del worker procesen el mismo registro, lo que introduce una garantía de consistencia útil si la ejecución evoluciona hacia más de un consumidor. Los flujos de procesamiento se ejecutan en el executor para no bloquear el bucle de eventos de la aplicación.		

Tabla x - Especificación de clase CL-0016

CL-0017	LangService		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño correspondiente al módulo services/lang.py. Concentra la internacionalización de la plataforma: mantiene el diccionario de mensajes del backend en los cuatro idiomas soportados, obtiene el idioma seleccionado por el usuario y devuelve los textos de la interfaz y de las respuestas. No requiere un servicio externo ni una base de datos adicional.		
Atributos	Nombre	Tipo	Descripción
	_VALID_LANGS	set	Conjunto de los idiomas válidos de la plataforma: español, inglés, chino e hindú.
	_TRANSLATIONS	dict	Diccionario de traducciones de los mensajes del backend, indexado por idioma y por clave de mensaje.
Operaciones	Nombre	Descripción	
	get_text(key, lang)	Devuelve el mensaje correspondiente a la clave en el idioma indicado, aplicando el español como valor por defecto cuando el idioma o la clave no están disponibles.	
	get_lang_from_cookie(request)	Lee la cookie de idioma de la petición, valida que pertenezca al conjunto permitido y devuelve el idioma aplicable, con el español como valor por defecto.	
Comentarios	La clase constituye el mecanismo transversal de presentación del idioma: los routers y los servicios de los subsistemas la invocan para localizar sus mensajes, las etiquetas de predicción, los títulos de los mapas de explicabilidad, los textos de los informes y los mensajes del asistente, en coherencia con el caso de uso CU-004.		

Tabla x - Especificación de clase CL-0017

CL-0018	QueueJob		
Versión	1.0		
Autores	Luis Carmona Berdugo		
Descripción	Clase de diseño que modela la entidad persistente de la cola de trabajos, correspondiente a la fila de la tabla job_queue. Conserva el estado de cada tarea asíncrona —diagnóstico, entrenamiento o validación externa— junto con el usuario propietario, el tipo, el payload serializable y el resultado o el error. Constituye la clase más compleja del subsistema por su máquina de estados, que determina las transiciones permitidas entre los estados de un trabajo.		
Atributos	Nombre	Tipo	Descripción
	id	int	Identificador del trabajo, asignado automáticamente por la persistencia.
	user_id	int	Identificador del usuario propietario del trabajo, que condiciona la consulta y la cancelación.
	job_type	str	Tipo del trabajo: diagnosis, training o external_validation.
	status	str	Estado del trabajo en la máquina de estados: queued, running, completed, failed o cancelled.
	payload	dict	Configuración serializable del trabajo, que contiene el modelo y la ruta de la imagen, o la sesión y los hiperparámetros.
	result	dict	Resultado del trabajo cuando se completa, en formato JSON; None en el resto de los estados.
	error_message	str	Mensaje de error cuando el trabajo falla, limitado en longitud; None en el resto de los estados.
Operaciones	Nombre	Descripción	
	-	La clase no expone operaciones propias: las transiciones de su estado las ejecutan el router (cancelación) y el worker (reclamación, finalización y fallo), de modo que el comportamiento de la máquina de estados se describe mediante el diagrama de transición de estados siguiente.	
Comentarios	La máquina de estados de la clase es compartida por los subsistemas que generan trabajos: SD-002 crea los diagnósticos, SD-004 crea los entrenamientos y las validaciones externas, y el worker de SD-006 aplica las transiciones. La cancelación solo opera sobre queued, y un trabajo en running no se interrumpe mediante una actualización administrativa, de modo que la reclamación condicional del worker mantiene la coherencia entre el estado persistido y la ejecución en curso.		

Tabla x - Especificación de clase CL-0018

22. Diseño de interfaces de los subsistemas

El diseño de interfaces constituye la etapa del diseño que concreta las ventanas del sistema y la navegación entre ellas, refinando la estructura de interfaz definida en el análisis. Mientras que el capítulo 21 determinó el comportamiento de los casos de uso y el capítulo 22 la estructura de las clases, este capítulo establece cómo se presenta esa funcionalidad al usuario: qué ventanas componen la aplicación, qué campos y acciones contiene cada una, cómo se navega entre ellas y qué informes se generan (Sharp, Rogers, & Preece, 2019). El diseño de interfaces aplica los principios de usabilidad que orientan la interacción del profesional sanitario con la plataforma: la claridad de las acciones, la consistencia entre ventanas y la adecuación de los mensajes de error al contexto (Nielsen, 1994).

El capítulo se organiza siguiendo la misma estructura de subsistemas de diseño del capítulo 18: cada apartado corresponde a un subsistema (SD-001 a SD-006) y describe las interfaces que materializan su responsabilidad. Para cada subsistema se presentan tres elementos, conforme a la guía de diseño de la memoria (punto 6): el flujo de navegación, que define la navegación definitiva entre las ventanas del subsistema; la especificación de las interfaces, que documenta cada ventana mediante la ficha formal de interfaz IU-NNNN con sus campos, sus botones y sus enlaces; y los informes del subsistema, que describen cada documento generado mediante la ficha formal IF-NNNN con sus datos y sus agrupaciones. Las ventanas de la aplicación corresponden a las plantillas reales del sistema: la página de inicio de sesión, el registro, el panel de diagnóstico y el laboratorio de entrenamiento, junto con las vistas que se integran en ellas.

El diagrama de la figura representa la navegación global de la aplicación. El visitante entra por la página de inicio de sesión (IU-0001), desde la que accede al registro (IU-0002); una vez autenticado o registrado, el sistema conduce al panel de diagnóstico (IU-0003), que constituye el centro de la aplicación. Desde el panel se alcanzan las vistas integradas del historial y el detalle (IU-0004), la administración (IU-0006, solo para el rol administrador), la cola de trabajos (IU-0007) y el laboratorio de entrenamiento (IU-0005); desde el laboratorio también se alcanza la cola de trabajos.

La navegación global refleja la estructura funcional de la plataforma: las ventanas públicas de acceso y registro (SD-001) preceden al resto, y el panel de diagnóstico se sitúa como punto central desde el que se accede al laboratorio y a las vistas integradas. Las vistas del historial, la administración y la cola de trabajos no constituyen ventanas independientes de la aplicación, sino secciones y diálogos integrados en el panel de diagnóstico o en el laboratorio, que se describen en los apartados de sus subsistemas correspondientes.

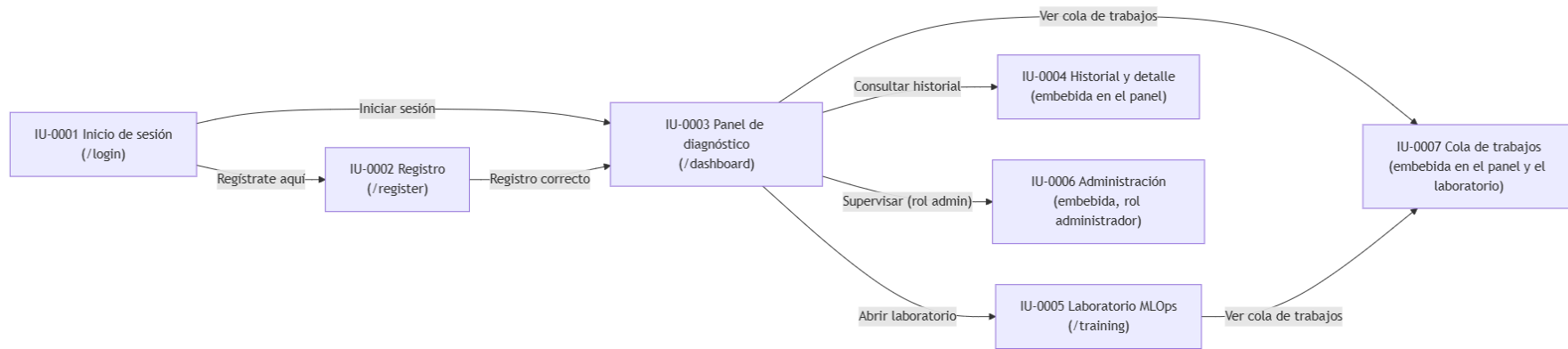


Ilustración x - Diagrama de navegación global de la aplicación

22.1. Subsistema SD-001: Acceso, identidad y gestión de sesiones

El subsistema SD-001 materializa las interfaces de acceso de la plataforma: la página de inicio de sesión y la página de registro. Ambas constituyen las únicas ventanas públicas de la aplicación, accesibles sin autenticación, y comparten los mecanismos transversales de presentación: el selector de idioma y el cambio del tema visual. La página de inicio de sesión permite al visitante autenticarse con sus credenciales, y la página de registro permite crear una cuenta; en ambos casos, la autenticación correcta conduce al panel de diagnóstico del subsistema SD-002.

22.1.1. Flujo de navegación

El flujo de navegación del subsistema SD-001, representado en la figura, refleja la progresión del visitante hasta la entrada al área privada. El visitante llega a la página de inicio de sesión, desde la que puede desplazarse a la página de registro mediante el enlace correspondiente y volver a la página de inicio desde el registro. Tras completar el inicio de sesión o el registro, el sistema conduce al usuario al panel de diagnóstico, que pertenece a SD-002; la navegación entre SD-001 y SD-002 materializa la frontera entre el área pública y el área privada.

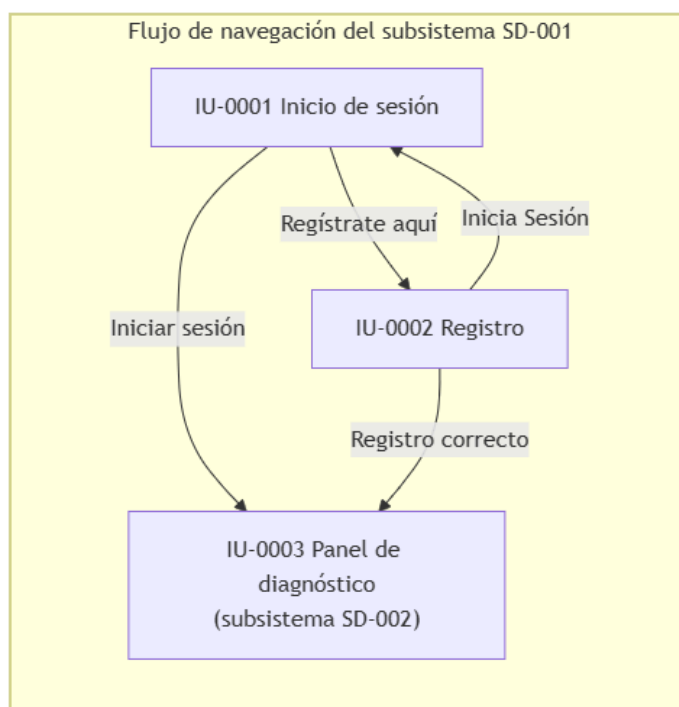


Ilustración x - Flujo de navegación del subsistema SD-001

El flujo no presenta rutas de navegación a otras ventanas del subsistema: las dos ventanas se enlazan mutuamente y ambas desembocan en el panel de diagnóstico. La página de inicio de sesión recibe el parámetro de error en la URL cuando una autenticación falla, de modo que muestra el mensaje genérico sin revelar qué parte de las credenciales era incorrecta, en coherencia con el CU-002 del análisis.

22.1.2. Especificación de las interfaces

A continuación se especifican las dos interfaces propias de SD-001.

IU-0001: Inicio de sesión					
Descripción	Ventana de acceso a la plataforma, correspondiente a la plantilla login.html (ruta /). Presenta el formulario de inicio de sesión con el nombre de usuario y la contraseña, el mensaje de error genérico, el selector de idioma y el cambio del tema visual. Ante una autenticación correcta, redirige al panel de diagnóstico; ante un fallo, muestra un mensaje genérico sin revelar la causa.				
Campos	Nombre	Tipo Datos	Editable/ Consulta	Oblig.	Descripción
	username	Texto	Editable	Sí	Nombre de usuario o correo electrónico con el que se identifica la cuenta.
	password	Contraseña	Editable	Sí	Contraseña de la cuenta; se introduce enmascarada.
Botones/Enlaces	Nombre	Acción			
	Entrar al Sistema	Envía el formulario al endpoint POST /login; con credenciales válidas, el sistema establece las cookies de sesión y redirige al panel de diagnóstico; con credenciales inválidas, redirige a la página con el parámetro de error para mostrar el mensaje genérico.			
	Regístrate aquí	Navega a la página de registro (/register).			
	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.			
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.			

Tabla x - Especificación de interfaz IU-0001

La página constituye la puerta de entrada del sistema: todos los flujos privados comienzan en ella. El mensaje de error es genérico para no revelar si el fallo corresponde al nombre de usuario o a la contraseña, y el endpoint de acceso está limitado a cinco peticiones por minuto. La ventana se sirve sin almacenamiento de caché, de modo que no se reutiliza una sesión anterior.

IU-0002: Registro					
Descripción	Ventana de creación de cuenta, correspondiente a la plantilla register.html (ruta /register). Presenta el formulario de registro con el nombre, el apellido, la especialidad, el nombre de usuario y la contraseña, junto con el selector de idioma y el cambio del tema visual. Envía los datos al endpoint de registro de forma asíncrona; ante un registro correcto muestra la confirmación y conduce al panel de diagnóstico, y ante un error muestra el mensaje correspondiente.				
Campos	Nombre	Tipo Datos	Editable/ Consulta	Oblig.	Descripción
	first_name	Texto	Editable	Sí	Nombre del profesional.
	last_name	Texto	Editable	Sí	Apellido del profesional.
	role	Lista de selección	Editable	Sí	Especialidad del profesional: Médico Residente, Radiólogo Especialista, Médico de Familia o Neumólogo.
	username	Texto	Editable	Sí	Nombre de usuario o correo electrónico con el que se identificará la cuenta.
	password	Contraseña	Editable	Sí	Contraseña de la cuenta; el servidor exige una longitud mínima de ocho caracteres.
Botones/Enlaces	Nombre	Acción			
	Registrarme	Envía el formulario al endpoint POST /api/register de forma asíncrona; ante un registro correcto muestra la confirmación y redirige al panel de diagnóstico, y ante un error muestra el mensaje del campo afectado o de la duplicidad.			
	Inicia Sesión	Navega a la página de inicio de sesión (/).			
	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.			
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.			

Tabla x - Especificación de interfaz IU-0002

La ventana mantiene la puerta de entrada autónoma del análisis: el alta no requiere la intervención de un administrador. La validación del formato del correo, de la longitud de la contraseña y de la unicidad del usuario se resuelve en el servidor, y la interfaz muestra los errores devueltos sin interrumpir la sesión de navegación. Tras el registro correcto, el sistema establece las cookies de sesión y el visitante queda autenticado, en coherencia con el CU-001 y con la decisión de diseño del capítulo 21.

22.1.3. Informes del subsistema

El subsistema SD-001 no genera informes: sus ventanas se limitan a la autenticación y a la creación de cuentas, y no producen documentos descargables. Los informes de la plataforma corresponden a los subsistemas funcionales que generan resultados: el informe PDF del diagnóstico, que se describe en el apartado de SD-002, y el informe consolidado de las sesiones de entrenamiento, que se describe en el apartado de SD-004. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.2. Subsistema SD-002: Diagnóstico asistido y generación de resultados

El subsistema SD-002 materializa la interfaz clínica de la plataforma: el panel de diagnóstico, correspondiente a la plantilla `dashboard.html` (ruta `/dashboard`), que constituye el entorno central de la aplicación y sobre el que se realiza el ciclo completo de una consulta clínica. La ventana integra además las vistas del historial y el detalle de las consultas, la cola de trabajos y el panel de administración, que pertenecen a los subsistemas SD-003, SD-006 y SD-005, de modo que la ficha de interfaz se centra en las funciones de diagnóstico y se remite a los apartados correspondientes para el resto de las vistas. El subsistema genera además el informe PDF del diagnóstico (IF-0001), que se produce durante el procesamiento de la consulta.

22.2.1. Flujo de navegación

El flujo de navegación del subsistema SD-002, representado en la figura, sitúa el panel de diagnóstico como el punto central de la navegación del usuario autenticado. El usuario accede al panel tras el inicio de sesión o el registro, y desde él realiza el ciclo de diagnóstico sin abandonar la ventana: carga la radiografía, selecciona el modelo, solicita el análisis y visualiza el resultado y los mapas de explicabilidad en la misma vista. Desde el panel se alcanzan el historial y el detalle de las consultas (SD-003), el laboratorio de entrenamiento (SD-004) y la cola de trabajos (SD-006).

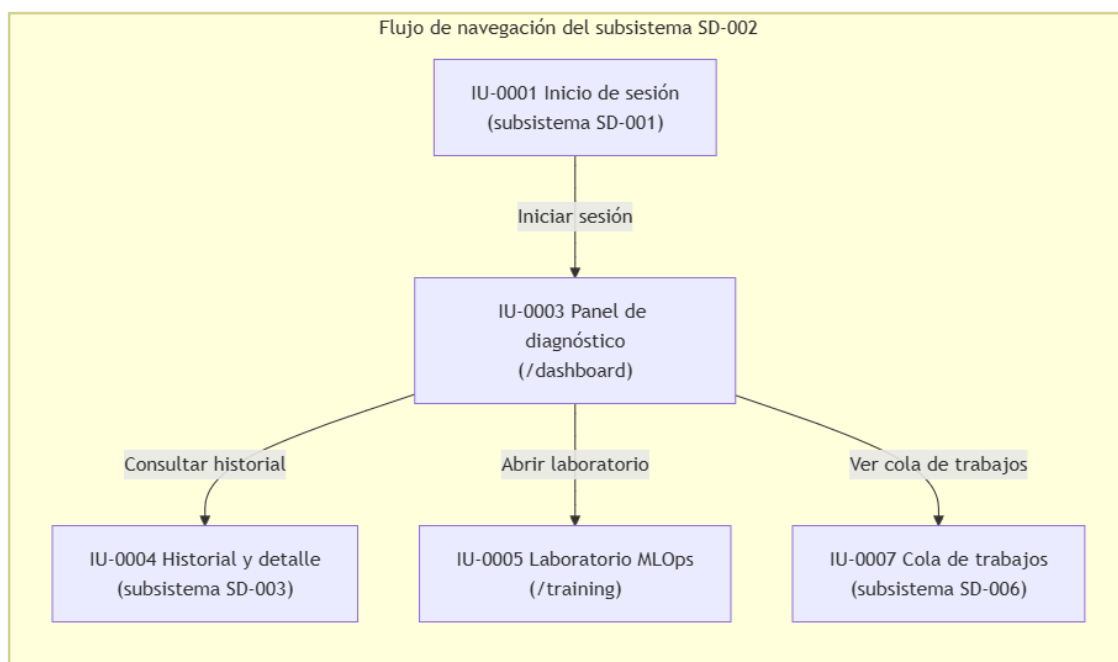


Ilustración x - Flujo de navegación del subsistema SD-002

El flujo refleja la naturaleza central del panel de diagnóstico: el ciclo clínico se resuelve en una única ventana, sin transiciones intermedias, y el resto de los subsistemas se alcanzan desde el panel mediante la barra lateral. La navegación hacia el historial y el detalle, hacia el laboratorio y hacia la cola de trabajos materializa las relaciones de colaboración entre SD-002 y los subsistemas SD-003, SD-004 y SD-006 descritas en el capítulo 18.

22.2.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-002, centrada en las funciones de diagnóstico; las vistas integradas del historial, la cola y la administración se especifican en los apartados de sus subsistemas.

IU-0003: Panel de diagnóstico					
Descripción	Ventana central de la aplicación, correspondiente a la plantilla dashboard.html (ruta /dashboard). Presenta el entorno de diagnóstico rápido: la zona de carga de la radiografía, el selector del modelo de inteligencia artificial, el área de conversación donde se muestra el resultado del análisis y los mapas de explicabilidad, y el visor de imágenes. Integra en su barra lateral la navegación al laboratorio, el historial de consultas, el panel de la cola de trabajos, el acceso a la administración (solo para el rol administrador) y el cierre de sesión. Solo es accesible para usuarios autenticados.				
Campos	Nombre	Tipo Datos	Editable/ Consulta	Oblig.	Descripción
	model-selector	Lista de selección	Editable	Sí	Arquitectura de inteligencia artificial para el diagnóstico, entre las CNN Clásicas disponibles (DenseNet121, ResNet50, MobileNetV2, EfficientNetB0 y ConvNeXtTiny).
	file-input	Archivo	Editable	Sí	Radiografía de tórax en formato JPEG o PNG, de hasta 10 MB, que se adjunta mediante la zona de carga.
	Área de resultado	Contenido	Consulta	-	Zona de la ventana que muestra la imagen enviada, el resultado del diagnóstico (etiqueta y confianza), el modelo empleado y el mapa de explicabilidad generado.
Botones/Enlaces	Nombre	Acción			
	Zona de carga	Permite seleccionar o arrastrar la radiografía de tórax; al adjuntarla habilita el envío.			
	Enviar	Envía la solicitud de diagnóstico al endpoint POST /predict, muestra la posición del trabajo en la cola y sondea su estado hasta que se completa.			
	Diagnóstico Rápido	Recarga el panel de diagnóstico (GET /dashboard).			
	Laboratorio de Entrenamiento	Navega al laboratorio de experimentación MLOps (GET /training), del subsistema SD-004.			
	Historial	Muestra el listado de consultas del usuario en la barra lateral, correspondiente a SD-003.			
	Cerrar Sesión	Cierra la sesión del usuario (GET /logout), del subsistema SD-001.			

	Selector de idioma	Cambia el idioma de la interfaz, materializando el CU-004.
	Botón de tema	Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.

Tabla x - Especificación de interfaz IU-0003

La ventana constituye el entorno de trabajo del profesional sanitario: el diagnóstico se solicita y se visualiza sin abandonar la vista, y la interfaz permanece operativa durante el procesamiento asíncrono mediante el sondeo del estado de la cola. La respuesta del sistema localiza las etiquetas según el idioma de la sesión. La ventana integra las vistas del historial, la cola y la administración, que se especifican en los apartados de SD-003, SD-006 y SD-005 respectivamente.

22.2.3. Informes del subsistema

A continuación se especifica el informe propio de SD-002.

IF- 0001: Informe del diagnóstico				
Descripción	Informe PDF de una consulta de diagnóstico, generado durante el procesamiento de la consulta mediante services/pdf_generator.py. Recoge la identificación de la consulta con la fecha y el modelo empleado, el diagnóstico con su nivel de confianza, la radiografía original y el mapa de explicabilidad, y se entrega como documento descargable desde el detalle de la consulta.			
Módulo de Interfaz	IU-0003 Panel de diagnóstico.			
Datos	Campo	Ordenación	Tipo Datos	Descripción
	Fecha	—	Fecha/hora	Fecha y hora de la consulta.
	Modelo	—	Texto	Arquitectura de inteligencia artificial empleada en el diagnóstico.
	Diagnóstico	—	Texto	Etiqueta de la predicción (Neumonía o Normal), con el color de diagnóstico acorde con el resultado.
	Confianza	—	Numérico	Nivel de confianza de la predicción.
	Radiografía original	—	Imagen	Radiografía de tórax enviada por el usuario.
	Mapa de explicabilidad	—	Imagen	Mapa de calor XAI que justifica la predicción.
Resumen/Acumulado	Resumen			Campos del Resumen
	-			N/A

Tabla x - Especificación de informe IF-0001

El informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial. La generación del documento se adelanta al momento del diagnóstico, de modo que la descarga posterior no depende de una nueva operación del sistema.

22.3. Subsistema SD-003: Historial y gestión de consultas

El subsistema SD-003 materializa la interfaz del historial y el detalle de las consultas, integrada en el panel de diagnóstico. La vista del historial ocupa la barra lateral del panel, donde se muestran las consultas del usuario agrupadas por modelo, y la vista del detalle ocupa el área principal al abrir una consulta, donde se presentan los artefactos y los metadatos de la consulta junto con las acciones de renombrado y eliminación. Ambas vistas constituyen la interfaz IU-0004, correspondiente a las secciones del historial y del detalle de la plantilla dashboard.html, y materializan los casos de uso CU-011 a CU-014.

22.3.1. Flujo de navegación

El flujo de navegación del subsistema SD-003, representado en la figura, refleja la cadena de navegación del análisis: desde el panel de diagnóstico se accede al historial, desde el listado se abre el detalle de una consulta, y desde el detalle se ejecutan, de forma opcional, el renombrado o la eliminación, además de la vuelta al listado. La navegación entre el listado, el detalle y las operaciones de gestión reproduce las relaciones de extensión de los casos de uso del subsistema.

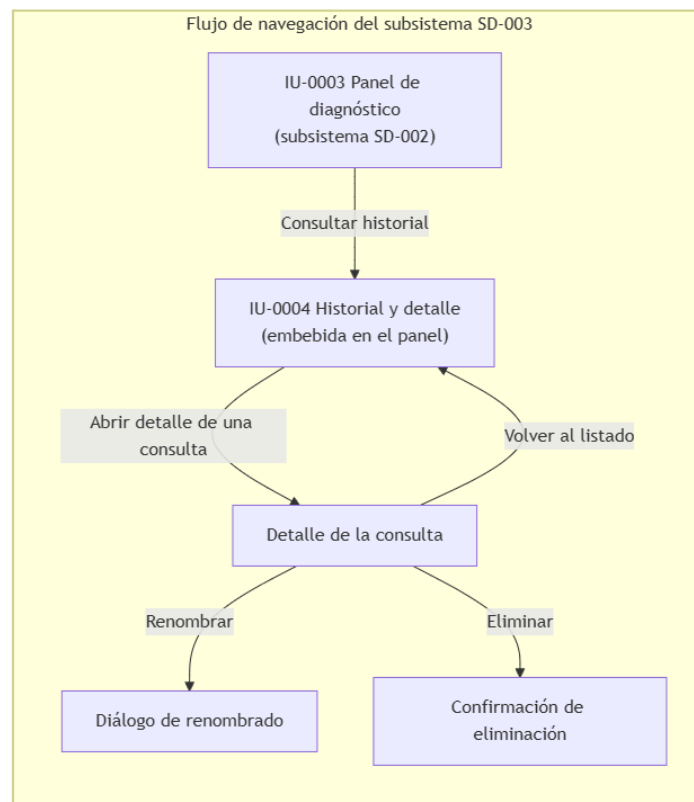


Ilustración x - Flujo de navegación del subsistema SD-003

El flujo refleja la progresión del usuario por el historial: el listado conduce al detalle de una consulta, y desde el detalle se alcanzan las operaciones opcionales de gestión. La navegación hacia el detalle y hacia las operaciones es de ida y vuelta: el usuario puede volver al listado o al panel en cualquier momento, sin que las acciones de gestión interrompan la navegación principal.

22.3.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-003.

IU-0004: Historial y detalle					
Descripción	Vista integrada del historial y el detalle de las consultas, correspondiente a las secciones del historial y del detalle de la plantilla dashboard.html. La vista del historial ocupa la barra lateral del panel de diagnóstico y muestra las consultas del usuario agrupadas por modelo, con la imagen, la fecha, la etiqueta y la confianza de cada una. La vista del detalle ocupa el área principal al abrir una consulta y muestra la radiografía original, el mapa de explicabilidad, el diagnóstico, la confianza, el modelo, el nombre visible, la fecha y las acciones de renombrado y eliminación.				
Campos	Nombre	Tipo Datos	Editable/Consulta	Oblig.	Descripción
	Listado de consultas	Contenido	Consulta	—	Listado de las consultas del usuario, agrupado por modelo, con la imagen, la fecha, la etiqueta y la confianza de cada consulta.
	cd-original	Imagen	Consulta	—	Radiografía original de la consulta seleccionada.
	cd-xai	Imagen	Consulta	—	Mapa de explicabilidad XAI de la consulta seleccionada.
	cd-label	Texto	Consulta	—	Etiqueta de la predicción (Neumonía o Normal) de la consulta.
	cd-confi-dence	N Numérico	Consulta	—	Nivel de confianza de la predicción.
	cd-model	Texto	Consulta	—	Arquitectura del modelo empleado en la consulta.
	cd-patient	Texto	Consulta	—	Nombre visible de la consulta, que funciona como etiqueta de organización y no como identificación clínica.
	cd-timesta-mp	Fecha/hora	Consulta	—	Fecha de la consulta.
Botones/Enlaces	Nombre		Acción		
	Tarjeta del historial		Abre el detalle de la consulta seleccionada, materializando el CU-012.		

	Volver	Cierra el detalle y regresa al listado del historial.
	Renombrar	Abre el diálogo para indicar el nuevo nombre visible de la consulta y lo actualiza, materializando el CU-013.
	Eliminar	Solicita la confirmación de la eliminación y, si se confirma, elimina la consulta y sus artefactos, materializando el CU-014.

Tabla x - Especificación de interfaz IU-0004

La vista del historial solo muestra las consultas del usuario propietario, de modo que el acceso al detalle y a sus artefactos queda restringido al propietario mediante la cadena de datos del listado. La vista del detalle no requiere una petición adicional al servidor: se construye con los datos recibidos en el listado y las imágenes se sirven desde el almacenamiento estático. El informe del diagnóstico se descarga desde el detalle, aunque la generación del documento pertenece a SD-002.

22.3.3. Informes del subsistema

El subsistema SD-003 no genera informes propios: su responsabilidad es la recuperación y la gestión de las consultas ya realizadas, y no produce documentos descargables. El informe del diagnóstico de cada consulta (IF-0001) pertenece al subsistema SD-002, que lo genera durante el procesamiento, de modo que el historial únicamente conserva y sirve la ruta del documento ya generado. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.4. Subsistema SD-004: Laboratorio de experimentación MLOps

El subsistema SD-004 materializa la interfaz del laboratorio de experimentación, correspondiente a la plantilla `training.html` (ruta `/training`). La ventana integra el asistente de configuración conversacional, la exploración de la carpeta del dataset, el listado de los experimentos guardados, la consola de entrenamiento, la vista de resultados de la sesión —con el ranking, la matriz de Wilcoxon y la validación externa— y la vista de resultados de un modelo, con las métricas de validación cruzada, la calibración y las métricas XAI. La ventana integra además la cola de trabajos y el acceso a la administración, pertenecientes a SD-006 y SD-005. El subsistema genera el informe consolidado de la sesión (IF-0002).

22.4.1. Flujo de navegación

El flujo de navegación del subsistema SD-004, representado en la figura, se organiza en torno al laboratorio como ventana principal. El usuario accede al laboratorio desde el panel de diagnóstico, y dentro de la ventana navega entre el asistente de configuración, la vista de resultados de la sesión y la vista de resultados de un modelo, además de la consola de entrenamiento que se muestra durante la ejecución. Desde el laboratorio se alcanzan la cola de trabajos y, en sentido inverso, el panel de diagnóstico.

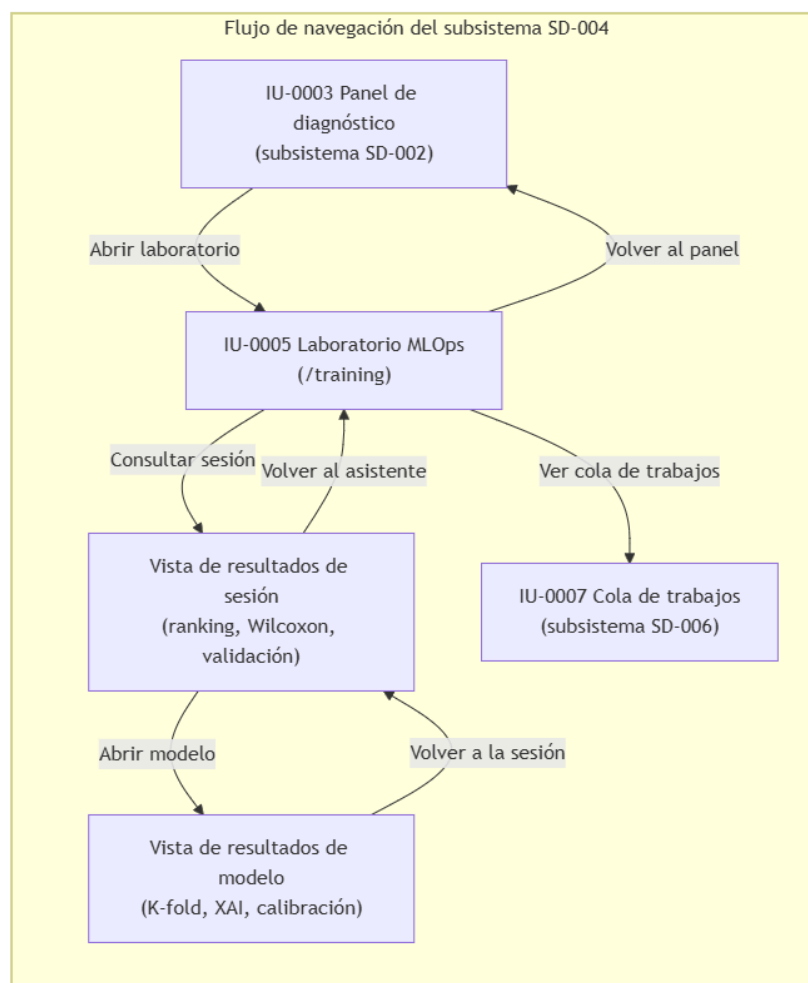


Ilustración x - Flujo de navegación del subsistema SD-004

El flujo refleja la progresión del investigador por el laboratorio: desde el asistente de configuración se lanzan los experimentos, desde el listado de experimentos se consulta la sesión, y desde la vista de la sesión se desciende a los resultados de un modelo concreto. La navegación entre las vistas es de ida y vuelta, y el usuario puede volver al asistente o al panel en cualquier momento sin interrumpir el estado del laboratorio.

22.4.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-004.

IU-0005: Laboratorio de entrenamiento					
Descripción	Ventana del laboratorio de experimentación MLOps, correspondiente a la plantilla training.html (ruta /training). Presenta el asistente de configuración conversacional, la exploración de la carpeta del dataset, el listado de los experimentos guardados, la consola del servidor de entrenamiento, la vista de resultados de la sesión (ranking por AUC, matriz de Wilcoxon, validación externa con curvas ROC y matriz de DeLong, y exploración de modelos) y la vista de resultados de un modelo (validación cruzada K-fold, calibración y métricas XAI). Solo es accesible para usuarios autenticados.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	chat-input	Texto	Editab le	Sí	Mensaje en lenguaje natural dirigido al asistente de configuración del experimento.
	Listado de experime ntos	Conteni do	Consu lta	—	Listado de las sesiones de entrenamiento del usuario, con sus modelos, en la barra lateral.
	session-r anking-ta ble	Tabla	Consu lta	—	Ranking de los modelos de la sesión por la media del AUC de la validación cruzada, con su desviación.
	session-heatmap -img	Imagen	Consu lta	—	Matriz de significancia estadística del test de Wilcoxon.
	external-ranking-t able	Tabla	Consu lta	—	Rendimiento de la validación externa por modelo: exactitud, F1 y AUC.
	external-roc-img	Imagen	Consu lta	—	Curvas ROC de la validación externa.
	external-delong-i mg	Imagen	Consu lta	—	Matriz de significancia de DeLong.
	res-table -body	Tabla	Consu lta	—	Resultados de la validación cruzada K-fold de un modelo (exactitud, precisión, sensibilidad, F1 y AUC por pliegue).

	res-xai-table	Tabla	Consulta	—	Métricas cuantitativas de explicabilidad del modelo (deletion, insertion, sparsity, entropy y stability por método).
	console-logs	Contenido	Consulta	—	Registro de la ejecución del entrenamiento, actualizado durante el procesamiento.
Botones/Enlaces	Nombre		Acción		
	Explorar Carpeta		Abre el explorador del dataset y inserta la ruta seleccionada en la conversación, materializando el CU-017.		
	Enviar		Envía el mensaje al asistente de configuración, materializando el CU-016; si la configuración está completa, lanza el entrenamiento (CU-018).		
	Actualizar		Refresca el listado de experimentos guardados.		
	Volver al Asistente		Oculta la vista de resultados de la sesión y regresa al asistente de configuración.		
	Renombrar		Cambia el nombre visible de la sesión, materializando el CU-029.		
	Reutilizar Configuración		Reutiliza la configuración de la sesión actual en una nueva conversación.		
	Validación Externa		Solicita la validación externa de la sesión, materializando el CU-026.		
	Generar Reporte PDF		Descarga el informe consolidado de la sesión, materializando el CU-028.		
	Recalcular Wilcoxon		Solicita el recálculo de la comparativa estadística de la sesión, materializando el CU-024.		
	Volver a la Sesión		Regresa desde los resultados de un modelo a la vista de la sesión.		
	Generar XAI y Métricas		Ejecuta el análisis de explicabilidad de un modelo, materializando el CU-025.		
	Diagnóstico Rápido		Navega al panel de diagnóstico (GET /dashboard), del subsistema SD-002.		
	Cerrar Sesión		Cierra la sesión del usuario (GET /logout), del subsistema SD-001.		
	Selector de idioma		Cambia el idioma de la interfaz, materializando el CU-004.		
	Botón de tema		Alterna el tema claro y oscuro de la interfaz, materializando el CU-036 de SD-006.		

Tabla x - Especificación de interfaz IU-0005

El informe se genera bajo demanda, cuando la sesión dispone de los datos necesarios, y separa el conocimiento del formato del router: el generador recibe la información preparada por el motor y compone el documento. El informe consolida el análisis estadístico de la sesión y se considera parte de los artefactos de experimentación del laboratorio.

22.4.3. Informes del subsistema

A continuación se especifica el informe propio de SD-004.

IF- 0002: Informe de la sesión de entrenamiento				
Descripción	Informe PDF consolidado de una sesión de entrenamiento, generado bajo demanda mediante <code>services/pdf_generator_mlops.py</code> . Recoge la configuración del experimento, el ranking de modelos con la matriz de Wilcoxon, los resultados de la validación externa con las curvas ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas de explicabilidad y sus mapas de calor. Se entrega como documento descargable desde la vista de la sesión.			
Módulo de Interfaz	IU-0005 Laboratorio de entrenamiento.			
Datos	Campo	Ordenación	Tipo Datos	Descripción
	ID de sesión	—	Texto	Identificador de la sesión de entrenamiento.
	Fecha	—	Fecha/hora	Fecha de generación del informe.
	Dataset	—	Texto	Ruta del dataset de entrenamiento de la sesión.
	Modelos	—	Texto	Arquitecturas entrenadas en la sesión.
	Hiperparámetros	—	Texto	Épocas, tamaño de lote y tasa de aprendizaje de la sesión.
	Ranking	1	Tabla	Modelos ordenados por la media del AUC de la validación cruzada, con su desviación típica.
	Matriz de Wilcoxon	—	Imagen	Matriz de significancia estadística entre los modelos.
	Validación externa	—	Tabla	Exactitud, F1 y AUC de cada modelo sobre la cohorte externa.
	Curvas ROC	—	Imagen	Curvas ROC de la validación externa.
	Matriz de DeLong	—	Imagen	Matriz de comparación estadística de las curvas ROC.

	Métricas XAI	—	Tabla	Métricas de fidelidad de la explicabilidad por modelo y método.
	Mapas de calor XAI	—	Imagen	Mapas de explicabilidad de cada modelo sobre imágenes de ejemplo.
Resumen/Acumulado	Resumen			Campos del Resumen
	Ranking de modelos			Modelo, Media AUC, Desviación

Tabla x - Especificación de informe IF-0002

El informe se trata como parte de la información protegida de la consulta: se genera exclusivamente a partir de los datos del usuario propietario y su acceso queda sujeto al control de propiedad del historial. La generación del documento se adelanta al momento del diagnóstico, de modo que la descarga posterior no depende de una nueva operación del sistema.

22.5. Subsistema SD-005: Supervisión y administración

El subsistema SD-005 materializa la interfaz de administración, integrada en las ventanas del panel de diagnóstico y del laboratorio de entrenamiento mediante los diálogos de administración de las plantillas dashboard.html y training.html. La vista de administración es accesible únicamente para el usuario con rol de administrador y presenta el listado de usuarios con sus recuentos de actividad, las consultas de un usuario concreto junto con sus sesiones del laboratorio, y el detalle de una consulta seleccionada. Constituye la interfaz IU-0006 y materializa los casos de uso CU-031 a CU-033.

22.5.1. Flujo de navegación

El flujo de navegación del subsistema SD-005, representado en la figura 95, refleja la progresión del administrador desde el listado general hasta el caso concreto, en coherencia con las relaciones de extensión del análisis. Desde el panel de diagnóstico o el laboratorio, el administrador abre el diálogo de administración con el listado de usuarios; al seleccionar un usuario se muestran sus consultas y sus sesiones del laboratorio, y desde el listado de consultas se abre el detalle completo de una consulta, con la posibilidad de volver en cada paso.

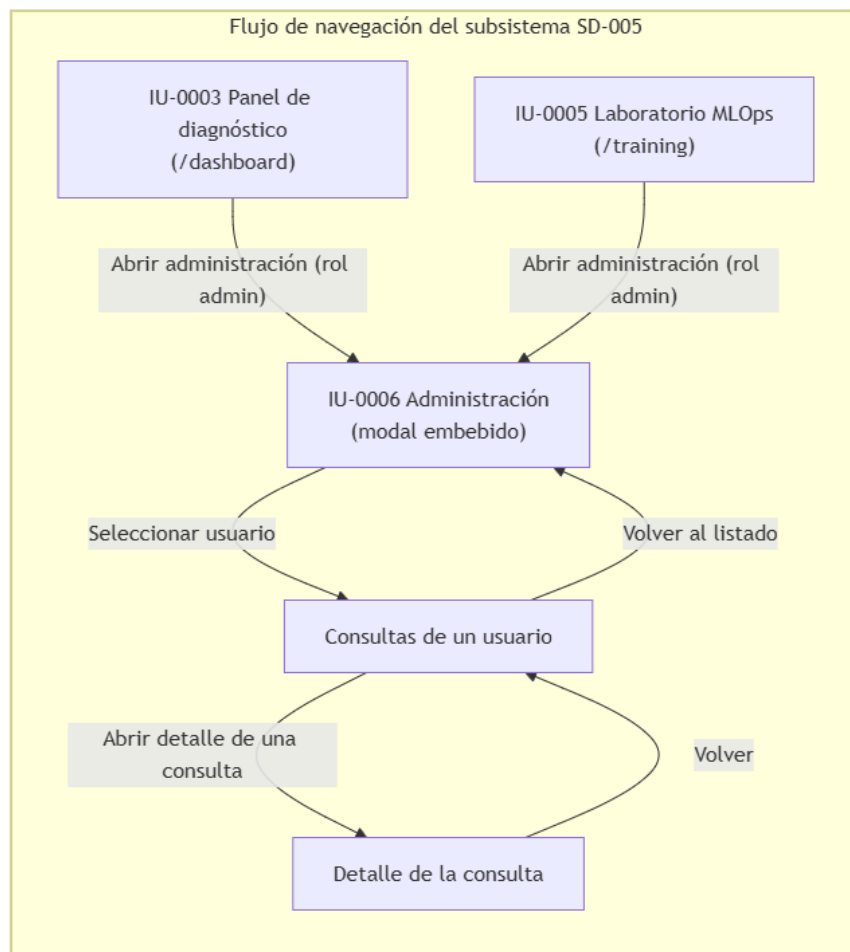


Ilustración x - Flujo de navegación del subsistema SD-005

El flujo refleja la cadena de navegación de la supervisión: el listado de usuarios conduce a la actividad de una cuenta, y desde la actividad se desciende al detalle de una consulta individual. La navegación es de ida y vuelta en cada nivel, de modo que el administrador puede profundizar progresivamente en la información y regresar al nivel anterior sin perder el contexto de la supervisión. El acceso al diálogo de administración queda restringido al rol administrador, y su presencia en la barra lateral depende de la comprobación de rol que realiza el servidor.

22.5.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-005.

IU-0006: Laboratorio de entrenamiento					
Descripción	Vista de supervisión y administración, integrada mediante los diálogos de administración de las plantillas dashboard.html y training.html. Presenta el listado de usuarios con el número de diagnósticos y de sesiones del laboratorio de cada uno, las consultas de un usuario concreto junto con sus sesiones de entrenamiento, y el detalle de una consulta con su imagen, su mapa de explicabilidad, su resultado, su confianza y sus metadatos. Solo es accesible para el usuario con rol de administrador.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	admin-us ers-list	Conteni do	Consu lta	—	Listado de usuarios registrados con el número de diagnósticos y de sesiones del laboratorio de cada uno.
	admin-us er-consul tations-li st	Conteni do	Consu lta	—	Consultas de diagnóstico del usuario seleccionado, con sus sesiones de entrenamiento del laboratorio.
	Detalle de consulta	Conteni do	Consu lta	—	Detalle completo de la consulta seleccionada: radiografía original, mapa de explicabilidad, diagnóstico, confianza, modelo, nombre y fecha.
Botones/Enla ces	Nombre		Acción		
	Panel de Administración		Abre el diálogo de administración con el listado de usuarios; solo se muestra al usuario con rol administrador.		
	Usuario del listado		Abre las consultas y las sesiones del laboratorio del usuario seleccionado, materializando el CU-032.		
	Consulta del listado		Abre el detalle completo de la consulta seleccionada, materializando el CU-033.		
	Cerrar		Cierra el diálogo de administración y regresa a la ventana desde la que se abrió.		

Tabla x - Especificación de interfaz IU-0006

La vista de administración reutiliza el detalle de la consulta del panel de diagnóstico para presentar la consulta supervisada, de modo que la interfaz administrativa no duplica la vista del historial. La autorización administrativa se comprueba en el servidor antes de servir los datos, y no se infiere de la interfaz. El acceso del administrador a los datos de otros usuarios constituye la excepción al aislamiento de datos, acotada a la función de supervisión.

22.5.3. Informes del subsistema

El subsistema SD-005 no genera informes: su responsabilidad es la consulta y la supervisión de la actividad de la plataforma, y no produce documentos descargables. Los informes de la plataforma corresponden a los subsistemas que generan resultados, SD-002 y SD-004, y el administrador puede consultarlos a través de las consultas y las sesiones que supervisa, sin generar informes propios. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

22.6. Subsistema SD-006: Cola de trabajos y capacidades transversales

El subsistema SD-006 materializa la interfaz de la cola de trabajos, integrada como panel lateral en las ventanas del panel de diagnóstico y del laboratorio de entrenamiento. La vista de la cola muestra los trabajos asíncronos del usuario —diagnósticos, entrenamientos y validaciones externas— con su tipo, su estado y su posición, y permite cancelar los trabajos pendientes, materializando los casos de uso CU-034 y CU-035. Además del panel de la cola, el subsistema comprende los mecanismos transversales de presentación de la interfaz: el selector de idioma y el cambio del tema visual, que ya se describieron como acciones comunes en las fichas de interfaz de los subsistemas anteriores.

22.6.1. Flujo de navegación

El flujo de navegación del subsistema SD-006, representado en la figura, refleja el carácter transversal de la cola de trabajos: el panel se integra en el panel de diagnóstico y en el laboratorio de entrenamiento, de modo que el usuario puede consultar el estado de sus trabajos desde cualquiera de los dos núcleos funcionales sin abandonar su ventana. Desde el panel de la cola se ejecuta la cancelación de un trabajo pendiente, que elimina el trabajo del estado encolado.

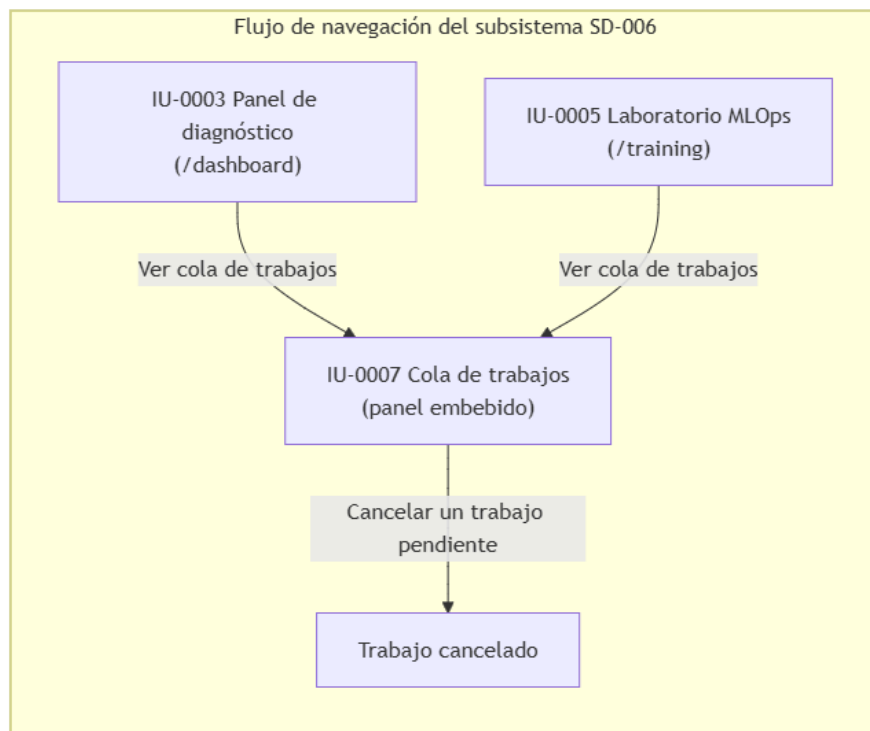


Ilustración x - Flujo de navegación del subsistema SD-006

El flujo refleja la integración transversal del panel de la cola: no constituye una ventana independiente, sino una sección que se muestra en las ventanas de los subsistemas que generan trabajos. La actualización del panel es periódica, de modo que el estado de los trabajos se refleja sin necesidad de navegación adicional, y la cancelación de un trabajo pendiente se resuelve desde el propio panel, sin transiciones intermedias.

22.6.2. Especificación de las interfaces

A continuación se especifica la interfaz propia de SD-006.

IU-0007: Cola de trabajos					
Descripción	Panel lateral de la cola de trabajos, integrado en las ventanas del panel de diagnóstico (dashboard.html) y del laboratorio de entrenamiento (training.html). Muestra los trabajos asíncronos del usuario con su tipo (diagnóstico, entrenamiento o validación externa), su estado (encolado, en ejecución, completado o fallido) y la posición de los trabajos encolados, y permite cancelar los trabajos pendientes. El panel se actualiza de forma periódica, de modo que refleja la progresión de los trabajos sin recargar la ventana.				
Campos	Nombre	Tipo Datos	Editabl e/ Consult a	Obli g.	Descripción
	queue-items	Contenido	Consulta	—	Listado de los trabajos del usuario con el tipo del trabajo, el estado (encolado, en ejecución, completado o fallido) y la posición de los trabajos encolados.
	Estado del trabajo	Texto	Consulta	—	Indicación del estado del trabajo: en ejecución o la posición en la cola para los encolados.
Botones/Enlaces	Nombre	Acción			
	Cancelar	Cancela un trabajo pendiente de la cola, tras la confirmación del usuario, mediante el endpoint DELETE /api/queue/cancel/{job_id}; solo se muestra para los trabajos en estado encolado, materializando el CU-035.			

Tabla x - Especificación de interfaz IU-0006

El panel de la cola refleja la máquina de estados de los trabajos y solo ofrece la cancelación para los trabajos pendientes: un trabajo en ejecución no se interrumpe, en coherencia con el diseño de SD-006. La actualización periódica del panel permite al usuario conocer en qué momento estará disponible un resultado o si un trabajo ha fallado, materializando el CU-034. Los mecanismos transversales de idioma y de tema visual se integran en las cabeceras de las ventanas, tal y como se describió en las fichas de interfaz de los subsistemas anteriores.

22.6.3. Informes del subsistema

El subsistema SD-006 no genera informes: su responsabilidad es la ejecución asíncrona de los trabajos y las capacidades transversales de presentación, y no produce documentos descargables. Los informes de la plataforma corresponden a los subsistemas que generan resultados, SD-002 y SD-004, y la cola de trabajos únicamente refleja su estado de procesamiento. Por esta razón, no se definen fichas IF-NNNN para este subsistema.

23. Especificación de la construcción del sistema

La especificación de la construcción constituye la etapa final del diseño que determina cómo se materializa el sistema diseñado en un software ejecutable. Los capítulos anteriores definieron la arquitectura, los casos de uso, las clases y las interfaces; este capítulo concreta el entorno tecnológico en el que se construye la aplicación, los paquetes y componentes que la forman, el procedimiento de construcción y puesta en marcha, y la generación del esquema de la base de datos. La especificación cierra la parte de diseño de la memoria y establece la base sobre la que se definen posteriormente las pruebas y la implantación del sistema (Sharp, Rogers, & Preece, 2019).

El capítulo se organiza en cuatro apartados, conforme a la guía de diseño de la memoria (punto 7): el entorno tecnológico de construcción, que describe las herramientas, las dependencias y las restricciones del entorno; los paquetes y componentes de construcción, que representan la estructura del software y sus dependencias; el proceso de construcción, que especifica cómo se prepara el entorno y se lanza la aplicación; y la generación del esquema de base de datos, que describe cómo se materializa el modelo físico definido en el capítulo 20. El contenido se apoya en la arquitectura de soporte del capítulo 19, que establece los subsistemas de soporte sobre los que se asienta la construcción.

La construcción de vitalXAI no produce ejecutables compilados: la aplicación es un sistema Python interpretado, servido por un proceso ASGI y apoyado en un conjunto de dependencias declaradas. Por esta razón, el proceso de construcción consiste en preparar el entorno virtual, instalar las dependencias, configurar las variables de entorno y lanzar el servidor y el worker de la cola, sin una fase de compilación ni de generación de paquetes instalables. Esta naturaleza condiciona las especificaciones de los apartados siguientes, que se centran en la reproducibilidad del entorno y en la configuración necesaria para que la aplicación funcione.

23.1. Entorno tecnológico de construcción

El entorno de construcción de vitalXAI reúne las herramientas necesarias para ejecutar y mantener la aplicación. El sistema se construye sobre Python 3.11, que constituye el lenguaje principal y alberga tanto el servidor web como el motor de aprendizaje profundo, y emplea el framework FastAPI para la capa HTTP, servido por el servidor ASGI Uvicorn (FastAPI, 2024; Uvicorn, 2024). La presentación se compone con el motor de plantillas Jinja2 y los recursos estáticos HTML, CSS y JavaScript; el almacenamiento relacional se apoya en MySQL, accedido a través del conector oficial de Python, y en el entorno de desarrollo se ejecuta mediante MariaDB a través de XAMPP (Oracle, 2024). El aprendizaje profundo se apoya en TensorFlow y en la librería Transformers de Hugging Face para las arquitecturas Transformer (TensorFlow, 2024; Hugging Face, 2024), junto con OpenCV para el preprocesamiento de las imágenes y matplotlib para la generación de los mapas de explicabilidad. Las credenciales de sesión se gestionan con bcrypt y python-jose, la limitación de peticiones con slowapi, los informes PDF con FPDF (FPDF2, 2024) y la configuración conversacional con el cliente del proveedor Groq.

La gestión de las dependencias se resuelve mediante el archivo requirements.txt, que fija las versiones de las librerías de la aplicación, de modo que el entorno de construcción se reproduce de forma determinista. La instalación se realiza sobre un entorno virtual de Python, y las dependencias se organizan en tres grupos: las del servidor web (FastAPI, Uvicorn, Jinja2, slowapi, python-multipart, bcrypt, python-jose), las del aprendizaje automático y el análisis (TensorFlow, Keras, Transformers, OpenCV, matplotlib, pandas, scikit-learn, seaborn) y las de calidad y verificación (pytest, pytest-cov, ruff, mypy), tal y como se describen en el capítulo 19. El peso de estas dependencias condiciona el entorno: la carga de TensorFlow y de los modelos de aprendizaje requiere recursos de memoria y, en su caso, de cómputo acelerado por GPU.

La configuración sensible y operativa permanece fuera del código y se suministra mediante variables de entorno, que se cargan desde un archivo .env en el arranque de la aplicación. La tabla siguiente resume las variables de entorno del sistema, agrupadas por ámbito.

Variable	Variable
JWT_SECRET_KEY	Clave secreta de firma de los tokens de acceso; en producción debe configurarse y no usar la clave de desarrollo.
JWT_ACCESS_EXPIRE_MINUTES	Duración en minutos del token de acceso.
JWT_REFRESH_EXPIRE_DAYS	Duración en días del token de refresco.
REFRESH_ROTATION_GRACE_SECONDS	Periodo de gracia de la rotación del token de refresco.
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	Parámetros de conexión a MySQL (por defecto localhost, root, sin contraseña y base tfg_pneumonia).
DB_POOL_SIZE	Tamaño del pool de conexiones a la base de datos.
GROQ_API_KEY	Clave de la API del proveedor del asistente conversacional.
TFG_DEMO_DATASET, TFG_DEMO_EXTERNAL_DATASET	Rutas por defecto de los datasets de entrenamiento y de validación externa.
TFG_SESSION_ID, TFG_MODEL_NAME, TFG_DATASET_DIR, TFG_EXTERNAL_DATASET_DIR, TFG_EPOCHS, TFG_BATCH_SIZE, TFG_LEARNING_RATE	Variables de entorno que el pipeline MLOps transmite a los scripts de entrenamiento y de validación externa.

Tabla x - Entorno tecnológico de construcción

El entorno impone además una serie de restricciones que condicionan la construcción y la ejecución. La aplicación requiere una instancia de MySQL accesible y con el esquema inicializable; en el entorno de desarrollo se utiliza XAMPP con el servicio de MariaDB activo, y el arranque de la aplicación crea las tablas si no existen. El motor de diagnóstico necesita los pesos de los modelos entrenados, que se conservan en el directorio pneumoniaccn-main/results y se cargan bajo demanda por la primera consulta con cada arquitectura; sin esos pesos, la solicitud de un diagnóstico falla en el procesamiento. El pipeline de experimentación MLOps requiere el directorio pneumoniaccn-main/code con los scripts de entrenamiento, análisis XAI, estadística y validación externa, que se invocan como procesos externos y escriben sus resultados en el directorio training_results. La generación de los mapas de explicabilidad emplea el backend no interactivo de matplotlib (Agg), que evita la dependencia de un entorno gráfico en el servidor.

El diagrama de despliegue representa la disposición física de los componentes del sistema en su entorno de ejecución. El navegador del cliente accede al servidor de aplicación mediante HTTP; el servidor Uvicorn aloja la aplicación FastAPI, que orquesta los routers, los servicios y el middleware, atiende las consultas a la base de datos MySQL y sirve los recursos estáticos; el worker asíncrono comparte la persistencia con la aplicación y ejecuta los scripts del pipeline MLOps, que leen los modelos de Hugging Face y escriben los artefactos en el sistema de ficheros; la aplicación invoca además al proveedor externo Groq para el asistente conversacional.

El diagrama de despliegue refleja la separación entre el ciclo de petición y la ejecución asíncrona: la aplicación atiende las peticiones del navegador y encola los trabajos, mientras que el worker reclama y procesa los diagnósticos y los entrenamientos fuera del ciclo de petición. El almacenamiento de ficheros alberga las plantillas, los recursos estáticos, las imágenes subidas, los informes y los resultados de los entrenamientos, y se expone parcialmente al navegador mediante los montajes de recursos estáticos. Los proveedores externos —el asistente conversacional y el repositorio de modelos— se integran como fronteras del sistema, de modo que los fallos de esos servicios no se confunden con los de la aplicación. Los requisitos no funcionales con impacto sobre el entorno de construcción, como el tiempo de respuesta de la inferencia (RNF-019) o la ejecución sin bloqueo de la interfaz (RNF-020), se satisfacen mediante la reutilización de los modelos en memoria y la cola de trabajos, que no requieren componentes adicionales de construcción.

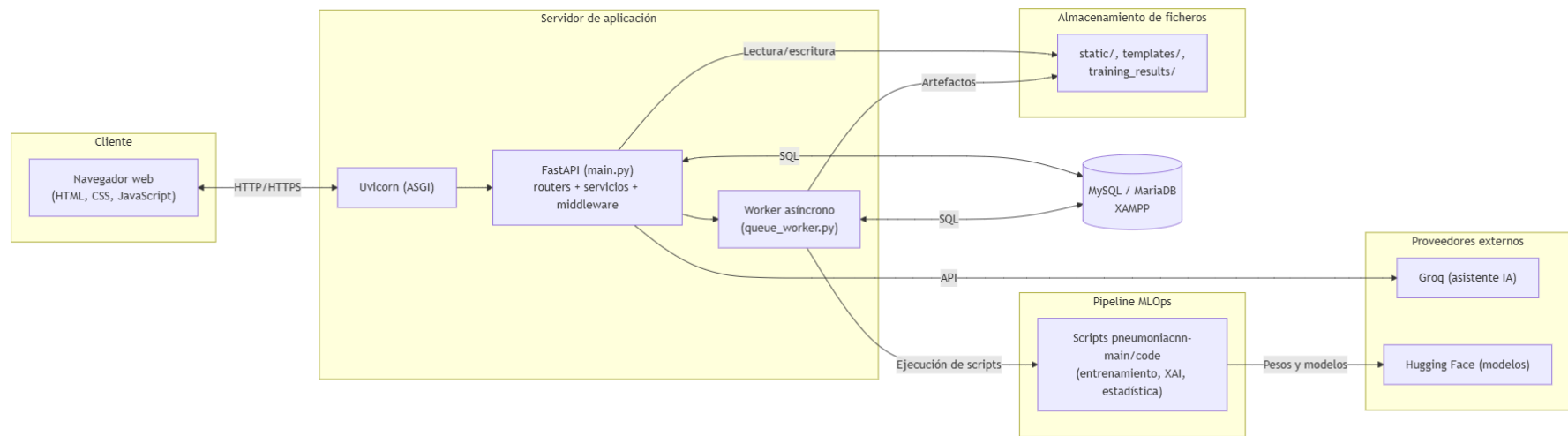


Ilustración x - Diagrama de despliegue del sistema

23.2. Paquetes y componentes de construcción

Los paquetes de construcción de vitalXAI se corresponden con las agrupaciones funcionales del código fuente, representadas en la figura. La estructura distingue la capa HTTP (routers/), la capa de servicios (services/), el núcleo de la aplicación (core), la presentación (templates/ y static/), el pipeline de experimentación (pneumoniaccnn-main/) y las pruebas (tests/). El diagrama de paquetes representa estas agrupaciones y las dependencias entre ellas, de modo que la construcción respeta la jerarquía de acoplamiento del sistema.

El diagrama refleja la dependencia principal del sistema: el núcleo de la aplicación compone la aplicación FastAPI a partir de los routers y los servicios, y monta los paquetes de presentación; los routers dependen de los servicios, que a su vez acceden a la persistencia relacional y al pipeline de experimentación. El paquete de pruebas depende de los routers, los servicios y el núcleo, de modo que su construcción no forma parte de la aplicación ejecutable. El paquete pneumoniaccnn-main/ constituye un componente externo al código de la aplicación: contiene los scripts del pipeline MLOps y los pesos de los modelos entrenados, y los servicios lo invocan como procesos externos, sin acoplamiento de importación.

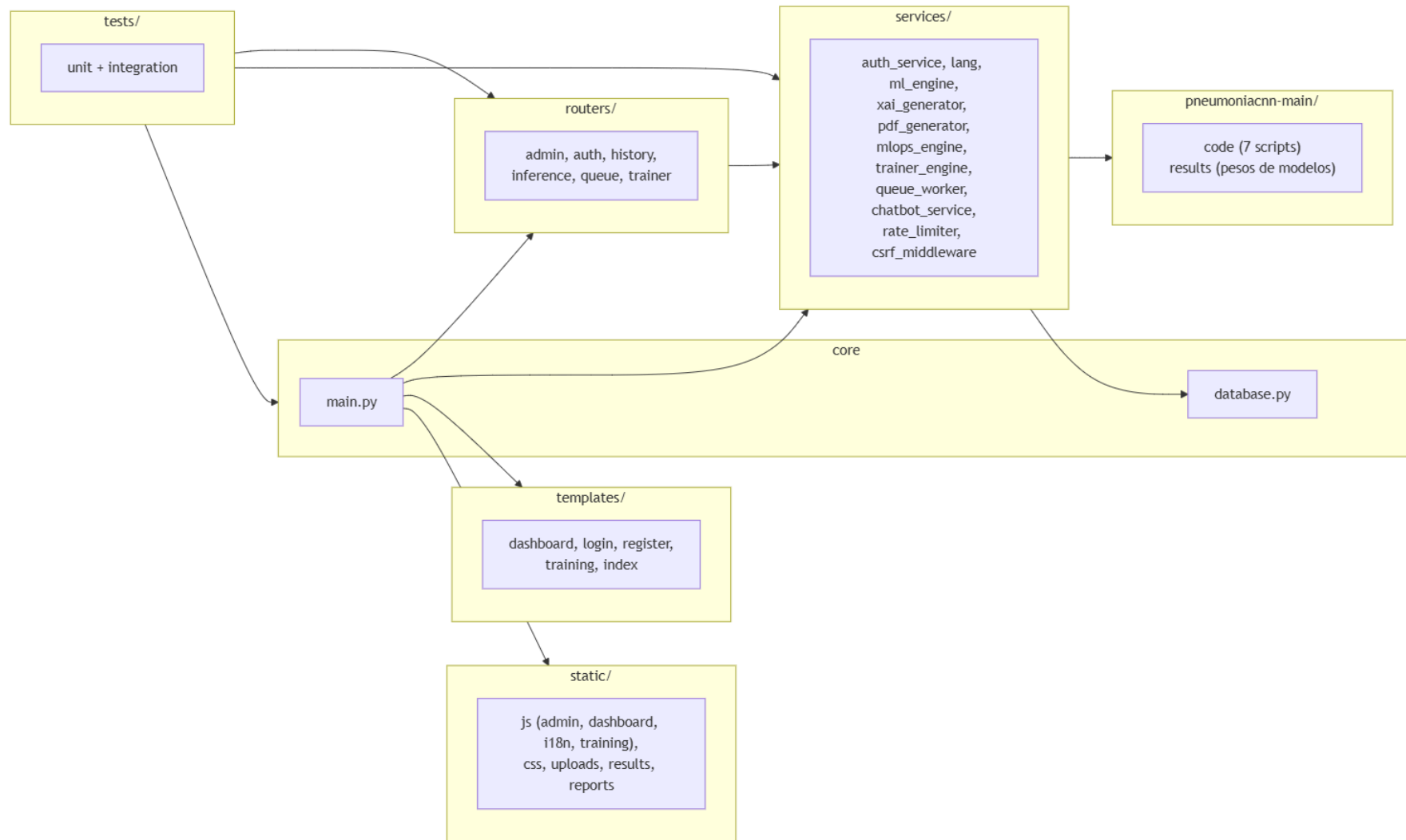


Ilustración x - Diagrama de paquetes de construcción

La tabla siguiente especifica los componentes que forman cada paquete de construcción y su responsabilidad.

Paquete	Componentes	Responsabilidad
core	main.py, database.py	Composición de la aplicación FastAPI, configuración de middleware y montajes, inicialización de la base de datos y pool de conexiones.
routers/	admin.py, auth.py, history.py, inference.py, queue.py, trainer.py	Capa HTTP: definición de los endpoints de la aplicación, autenticación, diagnóstico, historial, cola, laboratorio y administración.
services/	auth_service.py, lang.py, ml_engine.py, xai_generator.py, pdf_generator.py, mlops_engine.py, trainer_engine.py, queue_worker.py, chatbot_service.py, rate_limiter.py, csrf_middleware.py	Lógica de aplicación: criptografía y sesiones, internacionalización, predicción, explicabilidad, informes, laboratorio MLOps, worker de la cola, asistente conversacional, limitación y seguridad.
templates/	login.html, register.html, dashboard.html, training.html, index.html	Plantillas Jinja2 de las páginas de la aplicación.
static/	js/admin.js, js/dashboard.js, js/i18n.js, js/training.js, css, uploads/, results/, reports/	Recursos de presentación: scripts de la interfaz, estilos y directorios de artefactos generados.
pneumoniaccn-main/	code/ (siete scripts), results/ (pesos de modelos)	Pipeline MLOps: entrenamiento K-fold, entrenamiento Transformer, estadística, validación externa, test de DeLong y análisis XAI.
tests/	unit/, integration/, conftest.py	Pruebas unitarias e integración de la aplicación, con sus accesorios y mocks.

Tabla x - Paquetes y componentes de construcción

La estructura de paquetes se corresponde con los subsistemas de soporte definidos en el capítulo 19. El paquete core, con la persistencia relacional, materializa el subsistema de soporte SSOP-001; los paquetes templates/, static/ y el directorio training_results/ materializan el almacenamiento de ficheros y recursos (SSOP-002); el paquete core con la integración de los proveedores externos y el entorno de ejecución materializa el subsistema SSOP-003; y el paquete tests/ materializa la calidad y verificación (SSOP-004). Esta correspondencia mantiene la coherencia entre los paquetes de construcción y los subsistemas de soporte del diseño.

23.3. Proceso de construcción del software

El proceso de construcción de vitalXAI establece los pasos necesarios para preparar el entorno y lanzar la aplicación. Como se señaló en la introducción del capítulo, la construcción no incluye una fase de compilación ni de generación de ejecutables: consiste en preparar el entorno virtual, instalar las dependencias declaradas, configurar las variables de entorno, disponer de los artefactos externos y lanzar el servidor. El proceso se describe en la tabla siguiente, con el orden de ejecución y la finalidad de cada paso.

Orden	Paso	Procedimiento
1	Preparar el entorno	Instalar Python 3.11 y crear un entorno virtual para el proyecto.
2	Instalar dependencias	Ejecutar <code>pip install -r requirements.txt</code> dentro del entorno virtual, lo que instala las versiones fijadas de las librerías de la aplicación.
3	Configurar el entorno	Crear el archivo <code>.env</code> con las variables de entorno del sistema (credenciales de MySQL, claves de sesión y del asistente, y rutas de los datasets).
4	Preparar la base de datos	Iniciar el servicio de MySQL (en el entorno de desarrollo, mediante XAMPP); la aplicación crea automáticamente las tablas del esquema en el arranque.
5	Verificar los artefactos	Comprobar que el directorio <code>pneumoniacnn-main/code</code> contiene los scripts del pipeline y que <code>pneumoniacnn-main/results</code> contiene los pesos de los modelos entrenados.
6	Lanzar la aplicación	Ejecutar <code>python main.py</code> , que inicia el servidor Uvicorn, inicializa la base de datos y arranca el worker de la cola.

Tabla x - Proceso de construcción del software

El proceso de construcción depende de dos familias de requisitos adicionales. En primer lugar, los artefactos del aprendizaje automático: los pesos de los modelos entrenados, conservados en `pneumoniacnn-main/results`, son necesarios para el diagnóstico, de modo que la primera consulta con cada arquitectura carga los pesos y las siguientes reutilizan el modelo en memoria. En segundo lugar, las variables de entorno del pipeline MLOps: los scripts de entrenamiento y de validación externa leen la configuración de cada trabajo desde las variables `TFG_*`, que el motor de experimentación establece al invocarlos, y escriben sus resultados en el directorio `training_results`. Sin estas condiciones, el entorno no dispone de la funcionalidad de diagnóstico ni de experimentación completa.

La figura 99 representa el diagrama de componentes del sistema, que detalla la colaboración entre los componentes construidos. La aplicación compone los routers, que delegan en los servicios; los servicios acceden a la persistencia, invocan al asistente externo y orquestan el worker; el worker procesa los diagnósticos con el motor de predicción, el generador de mapas y el generador de informes, y los entrenamientos con el motor MLOps, que ejecuta los scripts del pipeline; la aplicación sirve la presentación desde las plantillas y los recursos estáticos.

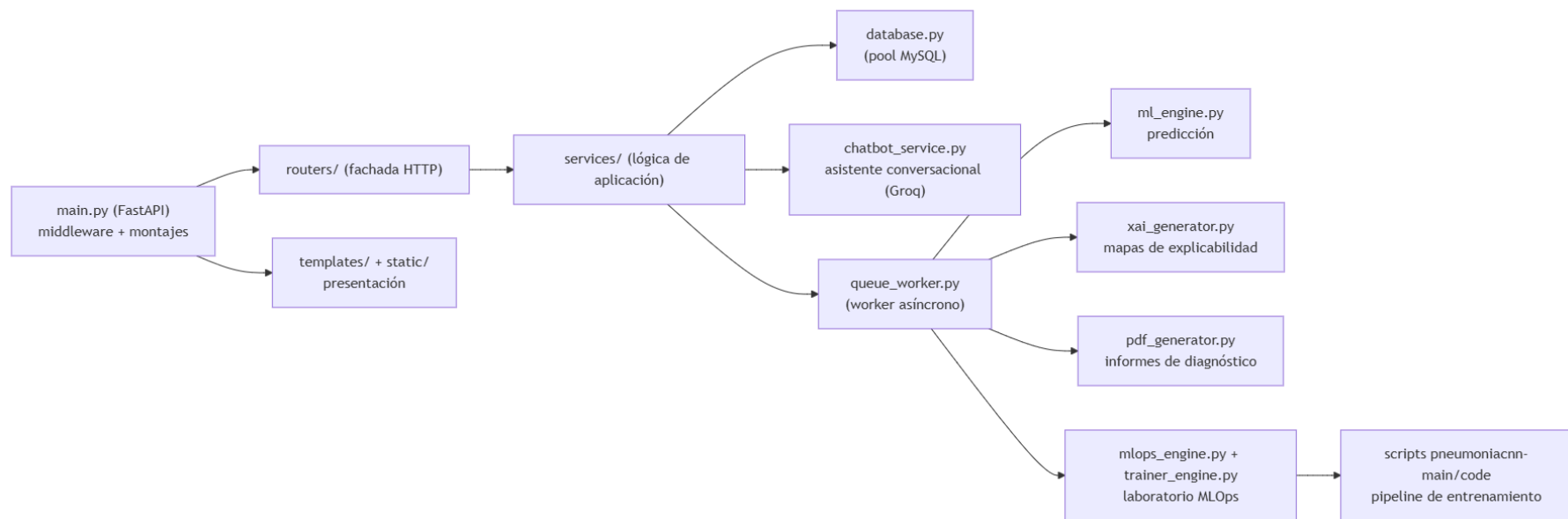


Ilustración x - Diagrama de componentes del sistema

La especificación detallada de los componentes del sistema se resume en la tabla siguiente, indicando qué requiere cada componente para su construcción y cómo se procede.

Componente	Dependencias y requisitos	Procedimiento de construcción
Aplicación web (main.py)	FastAPI, Uvicorn y las dependencias del servidor	Se construye al instalar las dependencias; compone la aplicación, los routers, el middleware y los montajes.
Routers (routers/)	Los módulos de los routers y los servicios	Se incluyen en la aplicación mediante la composición de main.py; no requieren construcción adicional.
Servicios (services/)	Las librerías de seguridad, aprendizaje y generación	Se instalan con requirements.txt; los servicios se importan desde los routers y el worker.
Persistencia (database.py)	MySQL y el conector	Se requiere una instancia de MySQL accesible; el esquema se crea automáticamente en el arranque.
Worker (queue_worker.py)	Las librerías de la aplicación	Se arranca automáticamente en el ciclo de vida de la aplicación.
Motor de diagnóstico (ml_engine.py, xai_generator.py, pdf_generator.py)	TensorFlow, Transformers, OpenCV, matplotlib y FPDF	Se instalan con requirements.txt; requiere los pesos de los modelos para la predicción.
Laboratorio MLOps (mlops_engine.py, trainer_engine.py)	Los scripts del pipeline y los datasets	Requiere pneumoniacnn-main/code y las variables TFG_*; los scripts se invocan como procesos.
Presentación (templates/, static/)	Jinja2 y el navegador	Se sirven mediante los montajes de main.py; no requieren compilación.
Asistente conversacional (chatbot_service.py)	La librería del proveedor y la clave de la API	Se instala con requirements.txt; requiere GROQ_API_KEY configurada.

Tabla x - Especificación detallada de los componentes del sistema

La construcción de la aplicación queda así especificada de forma reproducible: con las dependencias instaladas, la configuración suministrada y los artefactos del aprendizaje automático disponibles, la ejecución de main.py produce el sistema en funcionamiento, con la base de datos inicializada y el worker de la cola activo.

23.4. Elaboración de Especificaciones del Modelo Físico de Datos

La generación del esquema de la base de datos materializa el modelo físico de datos definido en el capítulo 20 en las tablas del sistema. vitalXAI no conserva un script SQL externo de instalación: el esquema se define en el módulo `database.py` y se ejecuta en el arranque de la aplicación mediante la función `init_db()`, que se invoca en el ciclo de vida de la aplicación antes de iniciar el worker. La función utiliza sentencias `CREATE TABLE IF NOT EXISTS`, de modo que la base de datos se crea y se verifica automáticamente en cada arranque, sin necesidad de un procedimiento manual de instalación ni de un script de migración adicional.

La tabla siguiente especifica las tablas del esquema, generadas por `init_db()` a partir del modelo físico, con su propósito y sus columnas principales.

Tabla	Propósito	Columnas principales
users	Cuentas de usuario de la plataforma y credenciales de acceso.	id (clave primaria), username (única), password_hash, first_name, last_name, role.
consultations	Consultas de diagnóstico realizadas por los usuarios, con sus rutas de artefactos y su resultado.	id, user_id (clave ajena a users), model_name, original_image_path, xai_image_path, prediction_label, confidence_score, patient_name, pdf_path, timestamp.
training_jobs	Registro histórico de los trabajos de entrenamiento, con su estado y su progreso.	id, user_id (clave ajena), dataset_path, model_name, status, progress, metrics_json, started_at, finished_at.
job_queue	Cola persistente de trabajos asíncronos: diagnósticos, entrenamientos y validaciones externas.	id, user_id (clave ajena), job_type, status, payload (JSON), result (JSON), error_message, created_at, started_at, finished_at.
refresh_tokens	Tokens de refresco de sesión, conservados mediante su hash y su estado de revocación.	id, user_id (clave ajena), token_hash, expires_at, revoked, created_at.

Tabla x - Especificación de las tablas del esquema

Las relaciones entre las tablas se establecen mediante las claves ajenas definidas en las sentencias de creación. Las tablas `consultations`, `training_jobs`, `job_queue` y `refresh_tokens` referencian la tabla `users` a través de la columna `user_id`, de modo que cada registro de actividad pertenece a una cuenta. Las claves ajenas de `training_jobs`, `job_queue` y `refresh_tokens` se declaran con la política `ON DELETE CASCADE`, de modo que la eliminación de un usuario elimina sus trabajos y sus credenciales de sesión; la clave ajena de `consultations` se declara sin esa política, de modo que el borrado de una cuenta deja los registros de diagnóstico conservados, en coherencia con la gestión del historial descrita en el capítulo 18.

La generación del esquema mantiene la trazabilidad con el modelo físico del capítulo 20: las columnas y los tipos de datos de las sentencias de creación corresponden a las entidades y los atributos definidos en el modelo, con las adaptaciones necesarias para el motor MySQL. La sentencia `CREATE TABLE IF NOT EXISTS` garantiza la idempotencia de la inicialización: si la tabla ya existe, la ejecución no la altera ni elimina datos, de modo que el arranque de la aplicación puede repetirse sin riesgo para los datos persistidos. Cuando el modelo físico evolucione con nuevas tablas o columnas, la generación del esquema deberá actualizarse en `database.py` y registrarse en el registro de cambios de la base de datos, conforme a las prácticas de persistencia del proyecto.

24. Preparación inicial de los datos del sistema

La preparación inicial de los datos constituye la etapa del diseño que determina cómo se disponen los datos necesarios para que el sistema funcione por primera vez y cómo evoluciona su esquema de persistencia. El capítulo 24 definió el entorno y el proceso de construcción de la aplicación; este capítulo especifica la carga inicial de datos: qué información debe estar disponible antes del primer uso, en qué entorno se prepara y en qué orden se ejecutan los procedimientos que la materializan (Elmasri & Navathe, 2016). La especificación cierra el diseño de la persistencia y establece la base sobre la que se definen las pruebas y la implantación.

El capítulo se organiza en dos apartados, conforme a la guía de diseño de la memoria (punto 8): el entorno de preparación de los datos, que describe el entorno tecnológico en el que se dispone la carga inicial, y los procedimientos de preparación y evolución del esquema, que definen el proceso, el orden de lanzamiento de los procedimientos y el diseño detallado de cada uno. El contenido se apoya en el modelo físico de datos del capítulo 20 y en la guía de despliegue del sistema, que fija los requisitos operativos de la puesta en marcha.

La naturaleza de la carga inicial de vitalXAI condiciona su especificación. El sistema no dispone de un mecanismo de carga de datos semilla ni de un sistema de migraciones automáticas: el esquema relacional se crea y se verifica automáticamente en el arranque de la aplicación, y las cuentas de usuario se crean mediante el registro, sin datos de prueba precargados en la base de datos. La carga inicial de datos consiste, por tanto, en disponer del esquema inicializado, de los datos de demostración —los conjuntos de imágenes y los pesos de los modelos entrenados— y de la configuración del entorno que permite acceder a ellos. Los cambios futuros del esquema se gestionan de forma manual y se registran en el registro de cambios de la base de datos, sin un proceso de migración formal.

24.1. Entorno de Carga Inicial o Migración

El entorno de preparación de los datos de vitalXAI reúne los elementos necesarios para la puesta en marcha del sistema. La preparación se realiza sobre el mismo equipo que ejecuta la aplicación: el entorno de desarrollo y de demostración del proyecto, que aloja el servidor, la base de datos, los conjuntos de imágenes y los pesos de los modelos. La tabla siguiente resume los elementos del entorno y su papel en la carga inicial.

Elemento de carga	Función en la preparación inicial
MySQL (MariaDB en XAMPP)	Almacén relacional donde la aplicación crea y verifica el esquema en el arranque.
Datasets de demostración	Conjuntos de imágenes de entrenamiento y de validación externa, necesarios para el diagnóstico y la experimentación.
Pesos de los modelos	Pesos de las arquitecturas entrenadas, necesarios para ejecutar la inferencia del diagnóstico.
Configuración del entorno	Variables de entorno con las credenciales y las rutas de los datos, suministradas mediante el archivo .env.

Tabla x - Especificación de entorno de Carga Inicial

La base de datos relacional se prepara mediante el servicio de MySQL del entorno, que en el desarrollo se ejecuta a través de XAMPP. La aplicación no requiere un script de instalación del esquema: la función `init_db()` de `database.py`, invocada en el arranque, ejecuta las sentencias `CREATE TABLE IF NOT EXISTS` que materializan el modelo físico de datos definido en el capítulo 20, creando las tablas `users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens` cuando no existen (Oracle, 2024). La verificación de la conexión se realiza al iniciar la aplicación, que informa de la imposibilidad de conectar si el servicio no está activo.

Los datos de demostración comprenden los conjuntos de imágenes y los pesos de los modelos. Los datasets se conservan en el directorio `pneumoniaccnn-main/Images` (conjunto de entrenamiento y diagnóstico) y en `pneumoniaccnn-main/ExternalDataset` (conjunto independiente para la validación externa), organizados en subcarpetas `NORMAL/` y `PNEUMONIA/`. Los pesos de los modelos entrenados se conservan en el directorio `pneumoniaccnn-main/results`, con los archivos de pesos de cada arquitectura, y se cargan bajo demanda en la primera consulta de diagnóstico. La disponibilidad de estos artefactos es condición necesaria para el funcionamiento del diagnóstico y del laboratorio de experimentación.

La configuración del entorno se suministra mediante el archivo `.env`, que la aplicación carga en el arranque. La configuración incluye las credenciales de la base de datos (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_POOL_SIZE`), las claves de sesión (`JWT_SECRET_KEY` y los parámetros de expiración), la clave del asistente conversacional (`GROQ_API_KEY`) y las rutas de los datasets de demostración (`TFG_DEMO_DATASET` y `TFG_DEMO_EXTERNAL_DATASET`), que permiten lanzar los entrenamientos y la validación externa sin recurrir al selector de carpetas. Las cuentas de usuario no se precargan: se crean mediante el registro en la aplicación, de modo que la carga inicial de cuentas se realiza operativamente, conforme al flujo del subsistema de acceso.

El diagrama de despliegue de la figura representa el entorno de preparación de los datos. La aplicación, en el arranque, inicializa el esquema en la base de datos MySQL, carga los pesos de los modelos bajo demanda, accede a los datasets de demostración para el diagnóstico y la experimentación, y lee la configuración del entorno; el asistente conversacional se integra como proveedor externo.

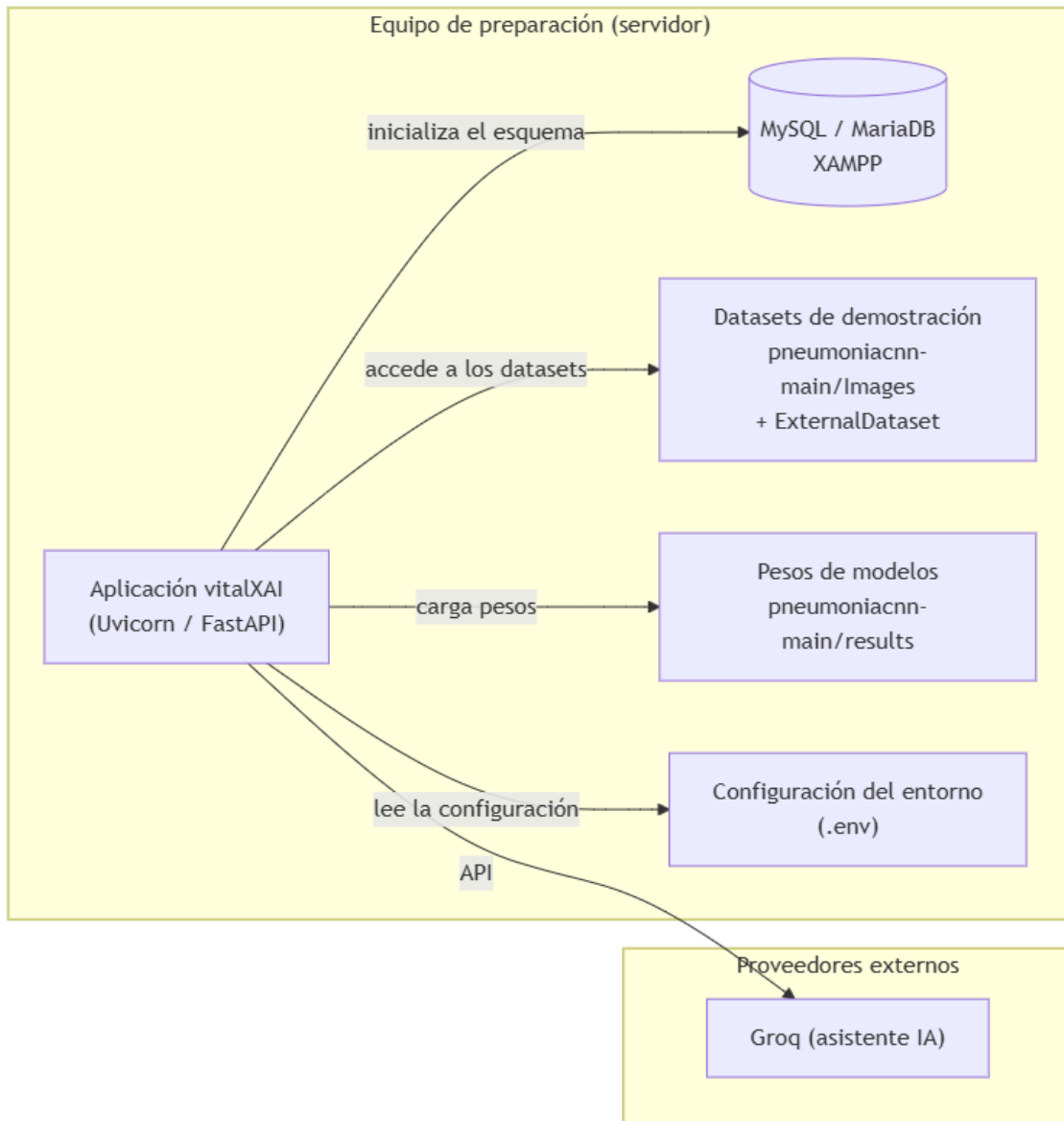


Ilustración x - Diagrama de despliegue del entorno de preparación de los datos

El diagrama refleja que la preparación inicial se resuelve en el equipo que aloja la aplicación, sin componentes de carga adicionales: la base de datos se inicializa desde la propia aplicación, y los datos de demostración y la configuración se disponen como artefactos del sistema de ficheros y variables de entorno. La verificación de los requisitos del entorno —el servicio de MySQL activo, la presencia de los datasets y de los pesos, y la configuración correcta— constituye el punto de partida de los procedimientos de preparación descritos en el apartado siguiente.

24.2. Procedimientos de Carga Inicial o Migración

Los procedimientos de preparación de los datos definen el proceso por el que el sistema queda disponible para su primer uso, el orden o la jerarquía de lanzamiento de cada procedimiento y el diseño detallado de cada uno. El proceso se representa mediante el diagrama de actividad de la figura, que refleja la secuencia de preparación: la verificación de los requisitos del entorno, la inicialización del esquema, la disposición de los datos de demostración y de la configuración, y la creación de las cuentas de acceso.

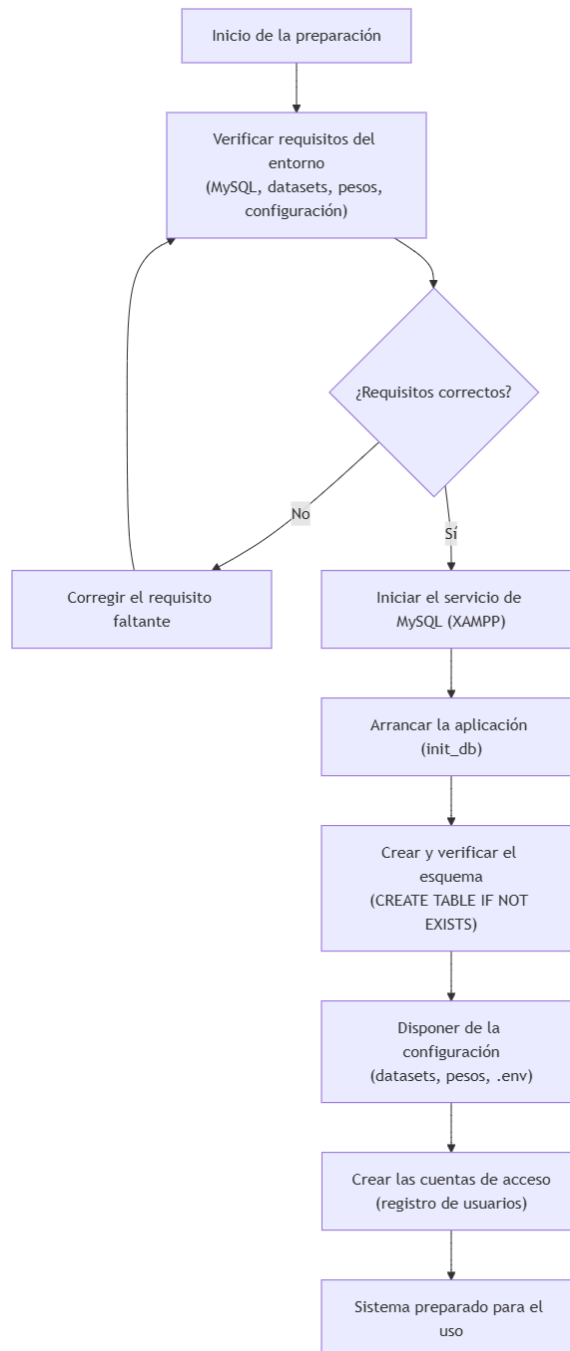


Ilustración x - Diagrama de actividad del proceso de preparación inicial de los datos

El diagrama de actividad refleja la jerarquía de lanzamiento de los procedimientos: la preparación comienza por la comprobación de los requisitos del entorno, que condiciona la corrección de los elementos faltantes; una vez verificados, se inicia el servicio de la base de datos y se arranca la aplicación, que crea y verifica el esquema; a continuación se dispone de los datos de demostración y de la configuración, y finalmente se crean las cuentas de acceso mediante el registro. Esta jerarquía garantiza que cada procedimiento cuenta con las condiciones del anterior antes de ejecutarse.

El diseño detallado de cada procedimiento que participa en la preparación inicial se especifica en la tabla siguiente, con su orden de lanzamiento, su descripción y la verificación de su resultado.

Procedimiento	Orden	Descripción	Verificación
Verificación de requisitos	1	Comprobar que el servicio de MySQL está disponible, que existen los datasets y los pesos de los modelos y que la configuración del entorno es correcta.	El entorno informa de los requisitos faltantes y permite corregirlos antes de continuar.
Inicialización del esquema	2	Ejecutar la aplicación, que invoca <code>init_db()</code> y crea las tablas del esquema con <code>CREATE TABLE IF NOT EXISTS</code> cuando no existen.	El arranque informa de la conexión a la base de datos y de la verificación de las tablas.
Preparación de los datos de demostración	3	Disponer de los datasets de entrenamiento y de validación externa y de los pesos de los modelos en sus directorios correspondientes.	La solicitud de un diagnóstico carga los pesos del modelo; el laboratorio accede a las rutas de los datasets.
Configuración del entorno	3	Suministrar las variables de entorno del sistema mediante el archivo <code>.env</code> , con las credenciales, las claves y las rutas de los datasets.	La aplicación carga la configuración en el arranque y la utiliza en las operaciones de sesión, diagnóstico y experimentación.
Creación de cuentas	4	Crear las cuentas de acceso mediante el registro de usuarios en la aplicación, sin datos de prueba precargados.	El nuevo usuario inicia sesión y accede al panel de diagnóstico.

Tabla x - Especificación del Procedimientos de Carga Inicial

Los procedimientos de preparación de los datos de demostración y de configuración se ejecutan en el mismo orden, ya que ambos deben estar completos antes del uso operativo del sistema; la tabla los ordena de forma conjunta en el paso tercero. La creación de cuentas constituye un procedimiento operativo y continuo, que no forma parte de una carga única sino del funcionamiento normal del subsistema de acceso.

La evolución del esquema de la base de datos se gestiona de forma manual, sin un sistema de migraciones automáticas. Cuando el modelo físico de datos cambia —por la incorporación de una tabla, una columna o una restricción—, la modificación se aplica actualizando las sentencias de creación de `database.py`, que en el siguiente arranque adapta el esquema a las tablas existentes mediante la política `CREATE TABLE IF NOT EXISTS`, y se registra en el registro de cambios de la base de datos del proyecto, con las sentencias SQL del cambio y las notas de reversión. Esta decisión, coherente con la simplicidad del esquema y con el estado del proyecto, evita la complejidad de un sistema de migraciones mientras la base de datos se encuentra en una fase estable; si el esquema evolucionara hacia cambios destructivos o una base de datos con datos persistidos críticos, la incorporación de un mecanismo de migraciones formal debería evaluarse conforme al registro de decisiones del proyecto.

25. Estrategia de verificación del sistema

La estrategia de verificación del sistema constituye la etapa del diseño que determina cómo se comprueba que el sistema construido se comporta conforme a lo especificado. El capítulo 17 definió el plan de pruebas de la plataforma desde la perspectiva del análisis, con las cinco categorías de verificación y sus criterios de éxito; este capítulo, conforme a la guía de diseño de la memoria (punto 9), especifica el entorno en el que se ejecutan las pruebas y la definición de los niveles de verificación, sin reproducir el detalle de cada prueba que ya documenta el capítulo 17. La estrategia distingue así el qué se verifica, que pertenece al plan de pruebas, del dónde y del cómo se verifica, que pertenece a este capítulo (Myers, Sandler, & Badgett, 2011).

El capítulo se organiza en dos apartados: los entornos de verificación, que describen el entorno tecnológico en el que se ejecutan las pruebas, las herramientas utilizadas, el origen de los datos de prueba y las restricciones operativas; y los niveles de verificación y criterios de aceptación, que definen los niveles de prueba, la naturaleza de las pruebas de integración y de sistema con una carga parecida a la de explotación, las validaciones funcionales y no funcionales que cubren las excepciones y los criterios que deben satisfacerse para aceptar cada nivel. El contenido se apoya en la guía de pruebas del proyecto, que fija los comandos, los niveles y el umbral de cobertura, y en el plan de pruebas del capítulo 17.

La verificación de vitalXAI combina la automatización con la verificación sobre el entorno real. Las pruebas unitarias y de integración se ejecutan de forma automatizada, integradas en el flujo de integración continua, y proporcionan una verificación repetible de la lógica de negocio y de los flujos críticos; las pruebas de rendimiento y de sistema se ejecutan sobre el despliegue de la plataforma, de modo que sus criterios se miden sobre el sistema real. Esta combinación, coherente con la metodología de desarrollo dirigida por pruebas del proyecto, garantiza que cualquier cambio posterior detecte las regresiones de forma inmediata y que la verificación final se realice en condiciones próximas a las de explotación.

25.1. Entornos de verificación

El entorno de verificación de vitalXAI se organiza en tres entornos que atienden a distintos niveles de aislamiento y de fidelidad con respecto al sistema real: el entorno unitario, el entorno de integración y el entorno de sistema. Cada entorno emplea sus propios dobles de prueba y su propio origen de datos, de modo que la verificación progresa desde el aislamiento total de los componentes hasta la ejecución sobre el despliegue real. La tabla siguiente resume los tres entornos y sus características.

Entorno	Alcance	Aislamiento	Origen de los datos de prueba
Unitario	Verificación de componentes individuales (lógica de negocio, funciones puras, componentes aislados).	Sin red, sin base de datos y sin operaciones de entrada y salida; todos los servicios externos se sustituyen por dobles.	Fixtures de tests/conftest.py: base de datos simulada, modelo TensorFlow simulado y cliente externo simulado.
Integración	Verificación de la interacción entre capas (API y persistencia) y entre subsistemas.	Base de datos en memoria (SQLite) y servicios externos simulados.	Datos de siembra del directorio de integración y mocks de las APIs externas (Groq y Hugging Face).
Sistema	Verificación integral sobre el despliegue real de la plataforma.	Sin aislamiento: entorno real con la base de datos MySQL, los pesos de los modelos y los datasets.	Datos de demostración del entorno de despliegue (imágenes, pesos y cuentas).

Tabla x - Especificación de Entornos de Pruebas

Las herramientas de verificación del proyecto se especifican conforme a la guía de pruebas. La batería automatizada se ejecuta con el marco de pruebas pytest, con la extensión de cobertura pytest-cov para medir la cobertura de los módulos de la aplicación y la extensión pytest-mock para la simulación de los servicios externos (pytest, 2024). La calidad estática se comprueba con ruff, que verifica el estilo y las reglas del código, y con mypy, que verifica los tipos de los módulos configurados (Ruff, 2024; Mypy, 2024). La verificación se integra en el flujo de integración continua, que ejecuta la batería de pruebas y las comprobaciones estáticas en cada integración.

El origen de los datos de prueba se resuelve mediante las fixtures y los datos de siembra del directorio de pruebas. Las fixtures del módulo conftest.py proporcionan la base de datos simulada, el modelo de TensorFlow simulado y el cliente externo simulado para las pruebas unitarias, de modo que la ejecución no depende de la red, de la base de datos ni de los pesos reales de los modelos. Las pruebas de integración utilizan una base de datos SQLite en memoria con datos de siembra y simulan las APIs externas, de modo que la verificación de los flujos entre subsistemas se centra en la colaboración de las capas internas. Los datos del entorno de sistema proceden del despliegue real: las imágenes de demostración, los pesos de los modelos y las cuentas de acceso, en coherencia con la preparación inicial de los datos del capítulo 25.

Las restricciones operativas del entorno de verificación son las siguientes. Los servicios externos —el proveedor del asistente conversacional, los modelos de Hugging Face y los modelos de TensorFlow—

se simulan siempre en las pruebas unitarias y de integración, de modo que la batería automatizada es determinista y no depende de la disponibilidad de esos servicios. El umbral de cobertura del código está fijado en el setenta por ciento, con un valor actual superior a ese umbral, y la batería falla si no se alcanza. Las pruebas unitarias y de integración se ejecutan con los comandos definidos en la guía de pruebas del proyecto; las pruebas de rendimiento y de sistema se ejecutan manual o semiautomatizada sobre el despliegue, con la carga de trabajo parecida a la de explotación que se detalla en el apartado siguiente.

El diagrama de despliegue de la figura representa el entorno de verificación del sistema. Las pruebas unitarias se ejecutan sobre las fixtures aisladas, las pruebas de integración sobre la base de datos en memoria con los servicios externos simulados, y las pruebas de sistema sobre el despliegue real con la base de datos MySQL y los artefactos del aprendizaje automático; las comprobaciones de calidad estática acompañan a la batería automatizada.

El diagrama refleja la progresión de la verificación por entornos: la batería automatizada parte del aislamiento total del entorno unitario, avanza a la interacción entre capas en el entorno de integración y culmina en la verificación sobre el despliegue real en el entorno de sistema. La calidad estática se aplica sobre la batería automatizada, de modo que el estilo y los tipos del código se verifican en cada integración. Los criterios de aceptación de cada nivel se definen en el apartado siguiente, completando la estrategia de verificación del sistema.

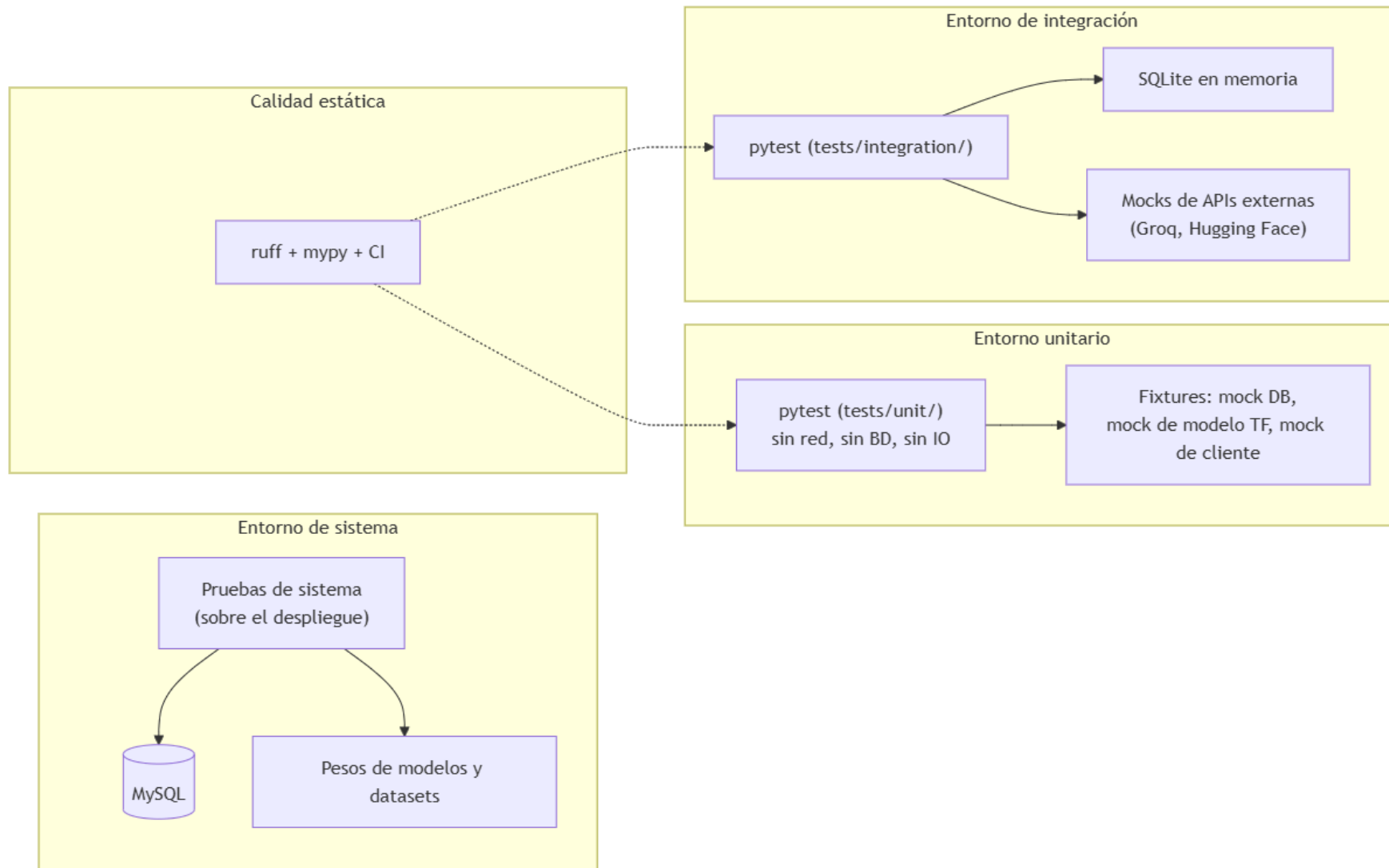


Ilustración x - Diagrama de despliegue del entorno de verificación

25.2. *Niveles de verificación y criterios de aceptación*

Los niveles de verificación definen la profundidad de la comprobación del sistema, en correspondencia con los entornos descritos en el apartado anterior. Cada nivel agrupa las verificaciones que se ejecutan en su entorno y establece los criterios que deben satisfacerse para aceptarlo, de modo que la superación de un nivel es condición para considerar verificada la parte del sistema que abarca. La definición de los niveles se mantiene coherente con el plan de pruebas del capítulo 17, que detalla las pruebas concretas de cada categoría; en este apartado se especifican los niveles, su alcance y sus criterios de aceptación, sin reproducir el detalle de cada prueba. Los tres niveles se presentan a continuación, con la tabla de criterios de aceptación de cada uno.

25.2.1. Nivel unitario

El nivel unitario verifica el comportamiento correcto de los componentes del sistema de manera aislada, en el entorno unitario, sin dependencia de la red, de la base de datos ni de los servicios externos, que se sustituyen por dobles de prueba. Cubre la lógica de negocio de mayor criticidad funcional: la gestión de cuentas y sesiones, la validación de las entradas, el motor de inferencia con modelos simulados, la generación de las explicaciones, la gestión del historial, el laboratorio de entrenamiento, la cola de trabajos, la internacionalización y el acceso a datos. La verificación unitaria recorre tanto los caminos normales como las excepciones —entradas inválidas, credenciales incorrectas, usuarios duplicados, tokens revocados, modelos ausentes, sesiones ajenas y trabajos no interrumpibles—, de modo que cada componente se comprueba en las condiciones esperadas y en sus desviaciones. Este nivel se corresponde con la categoría de verificación de componentes del capítulo 17.

Criterio	Descripción
Batería unitaria superada	Todos los tests unitarios del directorio tests/unit/ se ejecutan y superan sin fallos.
Cobertura de código	La cobertura de los módulos de la aplicación alcanza al menos el umbral del setenta por ciento.
Calidad estática	La verificación con ruff no reporta errores de estilo y la verificación con mypy no reporta errores de tipos en los módulos configurados.
Determinismo	La batería unitaria se ejecuta sin dependencia de la red, de la base de datos ni de los servicios externos.
Cobertura de excepciones	Los caminos alternativos y de error de los componentes verificados están cubiertos por las pruebas.

Tabla x - Especificación de Nivel unitario

La superación del nivel unitario se verifica en cada ciclo de desarrollo mediante los comandos de la guía de pruebas del proyecto, de modo que la calidad de los componentes se comprueba de forma continua y las regresiones se detectan de inmediato.

25.2.2. Nivel de integración

El nivel de integración verifica la colaboración entre las capas del sistema y entre los subsistemas, en el entorno de integración, superando el aislamiento del nivel unitario. Las pruebas ejercitan la interacción entre la API, la capa de negocio y la persistencia, sustituyendo la base de datos real por una instancia en memoria y los servicios externos por simulaciones, de modo que la verificación se centra en la colaboración interna sin depender del entorno de producción. Los flujos críticos se ejercitan con una carga de trabajo parecida a la de explotación: el flujo de acceso a la plataforma, el flujo de un diagnóstico desde la subida hasta el resultado, el aislamiento de datos entre usuarios, la supervisión administrativa y el lanzamiento de un experimento desde el asistente hasta la cola. La verificación de integración confirma que los subsistemas colaboran correctamente y que las condiciones de autorización y de aislamiento se respetan en los flujos combinados. Este nivel se corresponde con la categoría de verificación de flujos entre subsistemas del capítulo 17.

Criterio	Descripción
Batería de integración superada	Todos los tests de integración del directorio tests/integration/ se ejecutan y superan sin fallos.
Flujos críticos	Los flujos de extremo a extremo entre la API, la capa de negocio y la persistencia se verifican correctamente.
Aislamiento y autorización	Las condiciones de aislamiento de datos entre usuarios y de autorización administrativa se respetan en los flujos combinados.
Carga parecida a la de explotación	Los flujos se ejercitan con una carga de trabajo representativa del uso real de la plataforma.
Servicios externos simulados	La batería de integración no depende de la disponibilidad de los servicios externos, que se simulan en las pruebas.

Tabla x - Especificación de Nivel de integración

La superación del nivel de integración se verifica antes de la integración de los cambios en el flujo de trabajo del proyecto, de modo que la colaboración entre subsistemas queda confirmada antes de la verificación sobre el entorno real.

25.2.3. Nivel de sistema

El nivel de sistema verifica integralmente la plataforma sobre el despliegue real, en el entorno de sistema, sin aislamiento de la base de datos ni de los artefactos del aprendizaje automático. Las pruebas ejercitan los flujos completos de uso con la base de datos MySQL, los pesos de los modelos y los datasets del entorno de despliegue, con una carga de trabajo parecida a la de explotación. La verificación comprende las validaciones funcionales de los flujos completos —el diagnóstico asistido, la gestión del historial, el laboratorio de entrenamiento, la ejecución asíncrona, la supervisión administrativa y las capacidades transversales— y las validaciones no funcionales de rendimiento y capacidad de respuesta, de protección y control de acceso y de usabilidad, cubriendo las excepciones en ambos casos. Este nivel se corresponde con las categorías de protección, de rendimiento y de verificación integral de los flujos completos de uso del capítulo 17.

Criterio	Descripción
Flujos completos	Los flujos de uso de la plataforma se realizan correctamente sobre el despliegue real: diagnóstico, historial, laboratorio, cola, administración y capacidades transversales.
Validaciones funcionales	El comportamiento de los flujos coincide con el especificado en los casos de uso, incluidos los escenarios alternativos y de error.
Seguridad	Los mecanismos de protección (CSRF, cabeceras de seguridad, limitación de peticiones y gestión de tokens) se verifican sobre el sistema real.
Rendimiento	Los tiempos de respuesta de la inferencia y la ejecución sin bloqueo de la interfaz satisfacen los umbrales definidos para el entorno de despliegue.
Artefactos	Los informes PDF del diagnóstico y de la sesión, y los mapas de explicabilidad, se generan correctamente y se entregan al usuario.
Carga de explotación	La verificación se realiza con una carga de trabajo parecida a la de explotación y cubre las excepciones previstas.

Tabla x - Especificación de Nivel de sistema

La superación del nivel de sistema se verifica sobre el despliegue de la plataforma antes de su presentación, y constituye la confirmación final de que el sistema construido satisface los requisitos funcionales y no funcionales del análisis. Con la superación de los tres niveles, la estrategia de verificación del sistema queda completa y garantiza que las evoluciones futuras de la plataforma podrán incorporarse con un riesgo de regresión controlado y medible.

26. Implantación del sistema

La implantación del sistema constituye la etapa final del diseño que determina cómo se pone en operación la plataforma y qué documentación se necesita para operar con ella. Los capítulos 24 y 25 especificaron la construcción del sistema y la preparación inicial de sus datos; este capítulo, conforme a la guía de diseño de la memoria (punto 10), define los requisitos de documentación necesarios para la operación y los requisitos de implantación en el entorno de trabajo, incluidas las necesidades de formación y la infraestructura de instalación. La especificación establece así las condiciones bajo las que el sistema pasa a estar disponible para sus usuarios finales (Sharp, Rogers, & Preece, 2019).

El capítulo se organiza en dos apartados: la documentación necesaria para la operación, que especifica los manuales requeridos y sus características; y la implantación en el entorno de operación, que define las necesidades de formación, los requisitos de infraestructura e instalación y el proceso de puesta en marcha. El contenido se apoya en la guía de despliegue del proyecto y en los capítulos precedentes de diseño, de modo que la implantación mantiene la coherencia con la construcción y con la preparación de los datos.

La implantación de vitalXAI se orienta a dos escenarios de operación. El escenario principal es el despliegue de la plataforma en un equipo que aloja el servidor, la base de datos y los artefactos del aprendizaje automático, al que acceden los usuarios finales a través de la red; en la demostración del proyecto, el acceso se realiza mediante un túnel HTTPS que expone el servidor a través de una URL pública. La documentación y los requisitos de implantación se especifican para ambos escenarios, de modo que la operación del sistema queda cubierta tanto en el uso local como en el acceso remoto.

26.1. Documentación necesaria para la operación

La operación de vitalXAI requiere tres manuales, dirigidos a los distintos roles que intervienen en el uso y el mantenimiento de la plataforma: el manual de usuario, el manual de explotación y administración, y el manual de despliegue e implantación. Cada manual define su destinatario, su contenido y sus características, de modo que toda la documentación necesaria para operar con el sistema queda especificada. La tabla siguiente resume los manuales requeridos.

Documento técnico	Perfil destinatario	Alcance del contenido
Manual de usuario	Profesionales de la plataforma: facultativos e investigadores.	Acceso y registro; diagnóstico asistido (subida de la radiografía, selección del modelo, solicitud del diagnóstico, visualización del resultado y de los mapas de explicabilidad, e informe PDF); gestión del historial (listado, detalle, renombrado y eliminación); laboratorio de entrenamiento (configuración con el asistente, lanzamiento de experimentos, consulta de sesiones y resultados e informe de la sesión); y capacidades transversales (idioma, tema visual y cola de trabajos).
Manual de explotación y administración	Administrador de la plataforma y responsable de la operación.	Supervisión de la plataforma (listado de usuarios, consultas de un usuario y detalle de una consulta); gestión operativa de la actividad; monitorización de la cola de trabajos; y resolución de incidencias operativas.
Manual de despliegue e implantación	Equipo técnico responsable de la puesta en marcha.	Entorno y construcción del sistema; preparación inicial de los datos; despliegue en el entorno de operación; configuración del acceso remoto mediante el túnel HTTPS; y procedimientos de verificación. Se apoya en la guía de despliegue del proyecto y en los capítulos 24 y 25.

Tabla x - Especificación de documentación necesaria para la operación

Las características de los manuales se especifican conforme a la guía de diseño de la memoria. Los documentos se elaboran en formato digital, en el mismo formato de documentación del proyecto —Markdown, convertible a PDF—, de modo que puedan revisarse, distribuirse y consultarse en cualquier entorno. La estructura de cada manual se organiza por rol y por flujo funcional, con secciones dedicadas a cada operación del sistema y a sus condiciones de error, de modo que el lector localiza con rapidez el procedimiento que necesita. El contenido combina la descripción de los pasos de cada operación, las condiciones que deben cumplirse y los resultados esperados, junto con los escenarios alternativos y de error. El control de versiones de los manuales se alinea con la gestión de versiones del proyecto, de modo que cada versión de la documentación corresponde a una versión del sistema y refleja sus cambios. Finalmente, la distribución se dirige a los destinatarios indicados en la tabla: el manual de usuario a los profesionales, el de explotación y administración al administrador y al responsable de operación, y el de despliegue e implantación al equipo técnico.

La documentación del sistema se completa con la guía de despliegue del proyecto, que recoge los requisitos operativos y el procedimiento de la demostración, y con la documentación de diseño de la memoria, que constituye la referencia técnica de la arquitectura, el diseño de casos de uso, de clases y de interfaces, y de la construcción y preparación de los datos. Estos documentos, junto con los tres manuales, conforman el conjunto de documentación necesario para operar con el sistema en su entorno de trabajo.

26.2. Requisitos de Implantación

La implantación en el entorno de operación define las necesidades de formación de los usuarios, los requisitos de infraestructura e instalación y el proceso de puesta en marcha de la plataforma. Los requisitos se apoyan en los capítulos de construcción y de preparación de los datos: la implantación dispone el sistema sobre el entorno descrito en el capítulo 24 y aplica la preparación inicial definida en el capítulo 25, y añade las condiciones de formación y de acceso propias de la operación.

Las necesidades de formación se orientan a los tres roles que operan con la plataforma. La formación de los facultativos se centra en la operación clínica: el diagnóstico asistido, desde la subida de la radiografía hasta el informe PDF, y la gestión del historial de consultas. La formación de los investigadores se centra en el laboratorio de entrenamiento: la configuración conversacional de los experimentos, el lanzamiento de los entrenamientos, la consulta de los resultados y la interpretación de las comparativas estadísticas. La formación del administrador se centra en la administración del sistema: la supervisión de los usuarios y de su actividad, la monitorización de la cola de trabajos y la resolución de las incidencias operativas. La formación se realiza mediante sesiones guiadas sobre la plataforma, apoyadas en el manual de usuario y en el manual de explotación y administración, de modo que cada rol aprende sobre el flujo real que va a utilizar.

Los requisitos de infraestructura e instalación se especifican para el entorno de operación, atendiendo al software, al hardware y a las comunicaciones. En cuanto al software, el sistema requiere Python en su versión 3.11 o superior, el servicio de MySQL (en el entorno de desarrollo, MariaDB mediante XAMPP) y las dependencias declaradas en el archivo requirements.txt (FastAPI, 2024; Oracle, 2024). En cuanto al hardware, la plataforma requiere recursos de memoria y de cómputo suficientes para la carga de los modelos de TensorFlow y la ejecución del pipeline MLOps, además de espacio de almacenamiento para los datasets, los pesos de los modelos y los resultados de los entrenamientos; la disponibilidad de una unidad de cómputo acelerado por GPU reduce de forma significativa el tiempo de los entrenamientos. En cuanto a las comunicaciones, la operación local accede al servidor mediante la dirección local del equipo, mientras que el acceso remoto de la demostración se expone mediante un túnel HTTPS que publica el servidor a través de una URL pública; en un despliegue de producción, la comunicación debe cifrarse con HTTPS mediante un certificado válido.

El diagrama de implantación de la figura representa el sistema en su entorno de operación. Los usuarios finales —el facultativo, el investigador y el administrador— acceden mediante su navegador a través de las comunicaciones, que en el escenario de demostración se establecen mediante el túnel HTTPS; el servidor de operación aloja la aplicación, la base de datos MySQL, los pesos de los modelos y los datasets, y el worker asíncrono.

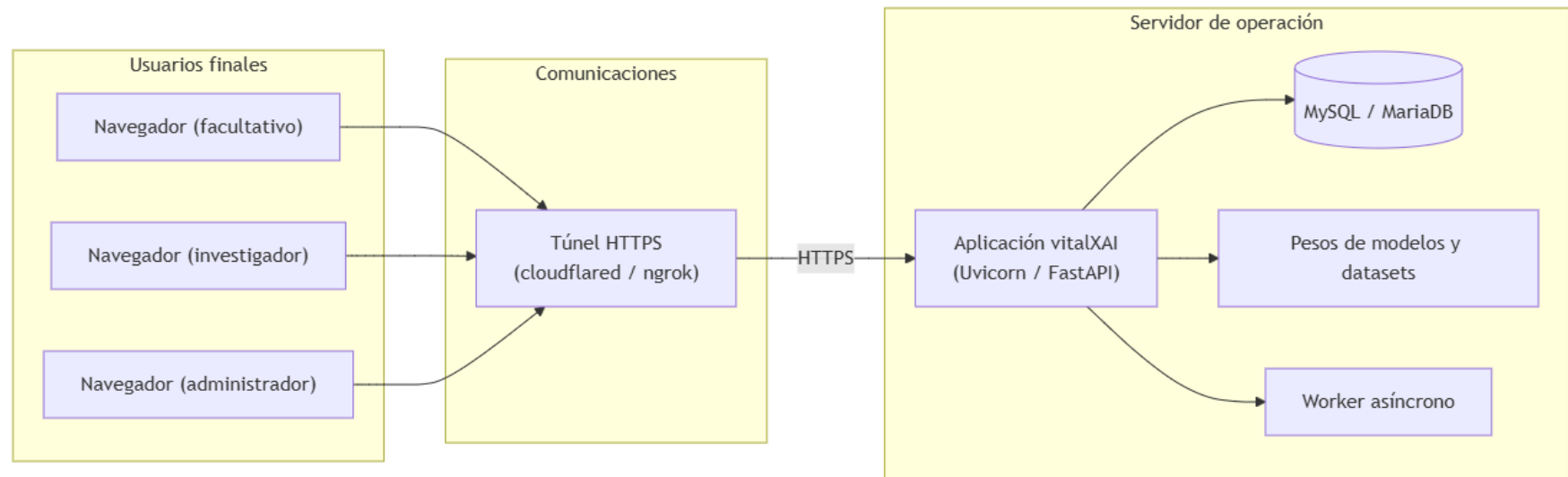


Ilustración x - Diagrama de implantación del sistema en el entorno de operación

El diagrama refleja la disposición de la implantación: los usuarios finales no acceden directamente al equipo que aloja el servidor, sino a través de la comunicación establecida, de modo que los datos de la plataforma permanecen en el servidor de operación. El servidor agrupa la aplicación, la persistencia y los artefactos del aprendizaje automático, y el worker atiende las tareas asíncronas de la cola.

El proceso de implantación se resuelve aplicando los procedimientos definidos en los capítulos precedentes. En primer lugar, se construye el sistema conforme al capítulo 24: se prepara el entorno virtual, se instalan las dependencias y se lanza la aplicación, que inicializa la base de datos y arranca el worker. A continuación, se dispone la preparación inicial de los datos conforme al capítulo 25: se verifican los requisitos del entorno, se inicializa el esquema, se disponen los datasets y los pesos de los modelos, se configura el entorno y se crean las cuentas de acceso. Finalmente, se establece el acceso a los usuarios finales, local o mediante el túnel, y se verifica el sistema conforme a la estrategia de verificación del capítulo 26. Con la formación de los roles completada y la infraestructura dispuesta, la plataforma queda implantada y operativa para sus usuarios.

ANEXO IV – CODIFICACIÓN

27. Introducción a la codificación

La parte de codificación de la memoria describe la implementación del sistema vitalXAI: cómo se materializó en código ejecutable el diseño especificado en los capítulos 18 a 27. Mientras que el diseño determina la arquitectura, los casos de uso, las clases, las interfaces, la construcción, la preparación de los datos, la verificación y la implantación, esta parte describe las decisiones de implementación concretas que hicieron realidad el sistema: la organización del repositorio, las convenciones de código, la estructura de cada capa tecnológica y los fragmentos de código más representativos de cada módulo. La codificación es, por tanto, la traducción fiel del diseño a un sistema Python servido por FastAPI y apoyado en MySQL, TensorFlow y los servicios del laboratorio MLOps.

La parte se organiza en seis capítulos. Este capítulo introductorio presenta la visión general de la implementación, la estructura del repositorio, las convenciones de código y las herramientas de calidad; los cinco capítulos siguientes describen la codificación de cada capa tecnológica: el backend (capítulo 29), la ejecución asíncrona (capítulo 30), el motor de diagnóstico y de explicabilidad (capítulo 31), el laboratorio de experimentación MLOps (capítulo 32) y el frontend (capítulo 33). Cada capítulo de codificación presenta los componentes implementados, los fragmentos de código representativos con su explicación y la coherencia con el diseño del sistema.

La codificación de vitalXAI siguió la metodología de desarrollo dirigida por pruebas (TDD) declarada en el plan de pruebas del capítulo 17, de modo que cada capacidad funcional se implementó después de escribir la prueba que verifica su comportamiento. Esta metodología dio lugar a una batería de pruebas automatizadas —más de ciento ochenta pruebas distribuidas entre unitarias y de integración— que acompaña a la implementación y que se integra en el flujo de integración continua (Myers, Sandler, & Badgett, 2011). Los capítulos siguientes describen la implementación real del sistema, referenciando los módulos, los ficheros de pruebas y las decisiones técnicas que la sustentan.

27.1. *Visión general de la implementación*

La implementación de vitalXAI se organiza en torno a las capas definidas en el diseño: la capa HTTP, la capa de servicios, la persistencia relacional, la ejecución asíncrona, el motor de inteligencia artificial y la capa de presentación. La capa HTTP agrupa los routers que exponen los endpoints de la API; la capa de servicios concentra la lógica de aplicación —criptografía, sesiones, predicción, explicabilidad, informes, laboratorio y cola—; la persistencia relacional se resuelve con MySQL a través de un pool de conexiones; la ejecución asíncrona se materializa en la cola de trabajos y en el worker; el motor de inteligencia artificial agrupa la carga de los modelos y la generación de las explicaciones; y la capa de presentación se compone de las plantillas Jinja2 y los recursos JavaScript del navegador. Cada capa se implementa en un paquete del repositorio, de modo que la estructura del código refleja la arquitectura del sistema.

El lenguaje de implementación es Python en su versión 3.11, que alberga tanto el servidor web como el motor de aprendizaje profundo. El servidor se construye con FastAPI y se sirve mediante Uvicorn; las plantillas se componen con Jinja2; la persistencia se accede mediante el conector MySQL de Python; el aprendizaje profundo se apoya en TensorFlow y en la librería Transformers de Hugging Face; y la seguridad de las sesiones se resuelve con bcrypt, python-jose y slowapi. Estas tecnologías, junto con el resto de las dependencias del sistema, se describieron en el entorno de construcción del capítulo 24, y su implementación se detalla en los capítulos de codificación de esta parte.

La coherencia entre el diseño y la implementación se mantiene mediante la trazabilidad de los módulos: cada router y cada servicio del código se corresponde con los componentes descritos en el capítulo 18 y con las clases de diseño del capítulo 22, y cada endpoint materializa los casos de uso del capítulo 21. Esta correspondencia permite verificar que la implementación satisface el diseño y que los capítulos de codificación pueden leerse como la materialización concreta de las decisiones técnicas ya documentadas.

27.2. Estructura del repositorio

La estructura del repositorio del proyecto refleja la organización por capas de la implementación. El árbol de directorios de la siguiente ilustración representa la estructura del repositorio de vitalXAI y los ficheros principales de cada paquete, con sus agrupaciones funcionales.

La estructura distingue los ficheros raíz de la aplicación, los paquetes de la implementación y los directorios de artefactos. Los ficheros raíz incluyen el punto de entrada `main.py`, la persistencia `database.py`, las dependencias `requirements.txt`, la configuración de calidad `pyproject.toml`, la configuración de la integración continua en `.github/workflows/` y las variables de entorno. Los paquetes `routers/`, `services/`, `templates/` y `static/` contienen la implementación de cada capa; el directorio `pneumoniacnn-main/` alberga el pipeline de experimentación con sus scripts y los pesos de los modelos; el directorio `tests/` contiene las pruebas unitarias y de integración; el directorio `scripts/` reúne los scripts operativos de arranque y de migración; y los directorios `sql/`, `training_results/` y los subdirectorios de `static/` conservan los artefactos generados y los recursos de la aplicación.

```

vitalXAI/
├── .github/
│   └── workflows/
│       └── ci.yml
├── routers/
│   ├── admin.py
│   ├── auth.py
│   ├── history.py
│   ├── inference.py
│   ├── queue.py
│   └── trainer.py
├── services/
│   ├── auth_service.py
│   ├── chatbot_service.py
│   ├── csrf_middleware.py
│   ├── lang.py
│   ├── ml_engine.py
│   ├── mlops_engine.py
│   ├── pdf_generator.py
│   ├── pdf_generator_mlops.py
│   ├── queue_worker.py
│   ├── rate_limiter.py
│   ├── trainer_engine.py
│   └── xai_generator.py
├── static/
│   ├── js/
│   │   ├── admin.js
│   │   ├── dashboard.js
│   │   ├── i18n.js
│   │   └── training.js
│   ├── uploads/
│   ├── results/
│   └── reports/
├── templates/
│   ├── dashboard.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   └── training.html
├── tests/
│   ├── unit/
│   └── integration/
├── pneumoniaccnn-main/
│   ├── code/
│   │   ├── 1_train_kfold.py
│   │   ├── 2_train_transformer_kfold.py
│   │   ├── 3_evaluate_statistics.py
│   │   ├── 4_external_validation.py
│   │   ├── 5_evaluate_delong.py
│   │   ├── 6_xai_qualitative.py
│   │   └── 7_xai_quantitative.py
│   └── results/
├── scripts/
│   ├── demo_start.bat
│   ├── demo_start.ps1
│   ├── migrate_passwords.py
│   └── migrate_roles.py
├── sql/
├── training_results/
├── .env.example
├── .gitignore
├── database.py
├── main.py
├── pyproject.toml
├── README.md
└── requirements.txt

```

Ilustración x - Estructura del repositorio de vitalXAI

27.3. Convenciones de código y decisiones transversales

La codificación de vitalXAI sigue las convenciones de estilo de Python definidas en la guía de estilo PEP 8 (van Rossum, Warsaw, & Coghlan, 2001), aplicadas de forma automática por la herramienta de análisis estático ruff. La configuración del proyecto fija una longitud máxima de línea de 140 caracteres y la selección de un conjunto de reglas que abarca los errores sintácticos, las advertencias, las importaciones, las convenciones de nombres, las actualizaciones de sintaxis, las comprobaciones de seguridad y las mejoras de simplificación. La aplicación de estas reglas se verifica de forma continua en el flujo de integración, de modo que el estilo del código se mantiene consistente en todo el repositorio.

Las decisiones transversales de codificación incluyen el uso de tipos estáticos, la política de comentarios y la gestión de los recursos. El proyecto emplea anotaciones de tipos en los módulos de mayor criticidad —los servicios de autenticación y seguridad—, verificadas con la herramienta mypy, de modo que las funciones documentan sus parámetros y sus valores de retorno y los errores de tipos se detectan en tiempo de verificación. Los comentarios se reservan para explicar la intención y las decisiones no evidentes de cada bloque, sin narrar la mecánica obvia del código, en línea con las prácticas de calidad del proyecto. Las variables de entorno se leen mediante la función de carga del entorno, de modo que la configuración sensible permanece fuera del código y se suministra en tiempo de ejecución. Los identificadores de los módulos y de las funciones siguen las convenciones de nombres de Python, con módulos en minúsculas separados por guiones bajos y funciones con la misma convención.

27.4. Herramientas y calidad del código

La calidad del código de vitalXAI se garantiza mediante un conjunto de herramientas de verificación integradas en el flujo de trabajo. El marco de pruebas pytest ejecuta la batería de pruebas unitarias y de integración, con la extensión pytest-cov para medir la cobertura de los módulos de la aplicación y la extensión pytest-mock para la simulación de los servicios externos (pytest, 2024). La cobertura de los módulos de la aplicación está sujeta a un umbral mínimo del setenta por ciento, de modo que la batería falla si la cobertura desciende por debajo del umbral. El análisis estático del estilo y de las reglas se resuelve con ruff (Ruff, 2024), y la verificación de tipos se resuelve con mypy en los módulos de seguridad configurados (Mypy, 2024).

La verificación se integra en la integración continua mediante el flujo de trabajo del repositorio, que se ejecuta en cada integración de la rama principal y en cada petición de cambios. El flujo instala las dependencias, ejecuta el análisis estático con ruff y ejecuta la batería de pruebas con la medición de cobertura, de modo que cualquier regresión de comportamiento, de estilo o de tipos se detecta de forma inmediata. Esta estrategia, coherente con la definida en la estrategia de verificación del sistema del capítulo 26, garantiza que la codificación se mantiene estable a lo largo del desarrollo y que los capítulos siguientes describen una implementación verificada y mantenible.

28. Codificación del backend

El backend de vitalXAI materializa las capas de la aplicación descritas en el capítulo 24: la capa HTTP, la capa de servicios, la persistencia relacional y los mecanismos de seguridad transversales. Este capítulo describe la codificación de estas capas, presentando los componentes implementados y los fragmentos de código más representativos de cada uno, con la explicación de las decisiones de implementación que los sustentan. La codificación del backend se organiza en cinco apartados: el arranque y la configuración de la aplicación, la capa HTTP de los routers, los servicios de aplicación, la persistencia y el acceso a los datos, y la seguridad transversal.

El backend se construye sobre FastAPI, servido por Uvicorn, y se apoya en MySQL para la persistencia (FastAPI, 2024; Uvicorn, 2024; Oracle, 2024). La implementación sigue el patrón de fachada ligera descrito en el diseño: los routers se ocupan de la recepción y la serialización de las peticiones HTTP, y delegan la lógica en los servicios, que concentran las operaciones de aplicación y acceden a la persistencia. Esta separación, ya declarada en el capítulo 18, se refleja en la estructura de los paquetes routers/ y services/ descrita en el capítulo 28, y permite verificar cada componente de forma aislada mediante las pruebas unitarias del capítulo 17.

28.1. *Arranque y configuración de la aplicación*

El punto de entrada de la aplicación se implementa en el módulo main.py, que compone la aplicación FastAPI, configura los mecanismos de seguridad transversales, monta los recursos estáticos e integra los routers de los subsistemas. El arranque se gestiona mediante el ciclo de vida de la aplicación: al iniciarse, se inicializa la base de datos y se arranca el worker de la cola; al finalizar, el worker queda detenido con la aplicación. El fragmento siguiente muestra la implementación del ciclo de vida y de la composición de la aplicación.

La implementación del arranque refleja las decisiones del diseño. La inicialización de la base de datos se realiza mediante la función `init_db()` de `database.py`, que crea las tablas del esquema si no existen, de modo que el arranque no requiere un script de instalación previo; el fallo de conexión se captura y se informa sin interrumpir el arranque, en coherencia con el entorno de construcción del capítulo 24. El worker de la cola se arranca mediante `start_worker()`, que crea la tarea asíncrona del procesamiento en segundo plano. Los mecanismos de seguridad se añaden como middleware —las cabeceras de seguridad y la protección CSRF— y el limitador de peticiones se registra en el estado de la aplicación con su manejador de excepciones. Los recursos estáticos y el directorio de resultados de entrenamiento se montan como directorios servidos por la aplicación, y los routers de los seis subsistemas se integran mediante la inclusión de sus enrutadores.

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        init_db()
        print("Base de datos conectada e inicializada correctamente.")
    except Exception:
        print("ATENCIÓN: No se pudo conectar a la base de datos MySQL.")
    start_worker(app)
    print("Worker de cola iniciado.")
    yield

app = FastAPI(title="X-Ray AI Consultant", lifespan=lifespan)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
app.add_middleware(SecurityHeadersMiddleware)
app.add_middleware(CSRFMiddleware)

app.mount("/static", StaticFiles(directory="static"), name="static")
os.makedirs("training_results", exist_ok=True)
app.mount("/training_results", StaticFiles(directory="training_results"), name="training_results")

app.include_router(admin.router)
app.include_router(auth.router)
app.include_router(history.router)
app.include_router(inference.router)
app.include_router(queue.router)
app.include_router(trainer.router)

```

Código 1 - Arranque y composición de la aplicación (main.py)

28.2. Capa HTTP: los routers

La capa HTTP se implementa en el paquete `routers/`, con un módulo por subsistema: `auth.py` para el acceso y las sesiones, `inference.py` para el diagnóstico, `history.py` para el historial, `queue.py` para la cola de trabajos, `trainer.py` para el laboratorio MLOps y `admin.py` para la administración. Cada router sigue el patrón de fachada ligera del diseño: resuelve la identidad del usuario, valida los datos de entrada, delega en los servicios y devuelve las respuestas HTTP con los códigos diferenciados. El fragmento siguiente muestra la implementación del endpoint de solicitud de diagnóstico del router `inference.py`, que materializa la subida de la radiografía y el encolado del trabajo.

La implementación del endpoint refleja las decisiones del diseño del subsistema de diagnóstico. La identidad se resuelve mediante el mecanismo común de identificación de SD-001, y las validaciones del tipo MIME y del tamaño se realizan antes de escribir el fichero en disco, de modo que una petición inválida no ocupa espacio ni una posición en la cola. La imagen se conserva en el área de cargas con un nombre basado en la fecha, y el trabajo se encola con un payload serializable que solo contiene el modelo, la ruta de la imagen y el idioma. La respuesta devuelve el estado `queued` con la posición, y el mensaje se localiza mediante el servicio de idioma. Este patrón —resolver la identidad, validar, delegar y responder con códigos diferenciados— se repite en los routers de los demás subsistemas, con las variaciones propias de cada flujo.

```

@router.post("/predict")
async def predict(request: Request, file: UploadFile = File(...), model_name: str = Form(...)):
    user_id = get_user_id_from_token(request.cookies.get("access_token"))
    if not user_id:
        return JSONResponse(status_code=401, content={"error": get_text("no_autenticado")})

    if file.content_type not in _ALLOWED_MIME_TYPES:
        return JSONResponse(status_code=400, content={"status": "error", "message": get_text("solo_imagenes")})

    file.file.seek(0, 2)
    file_size = file.file.tell()
    file.file.seek(0)
    if file_size > _MAX_FILE_SIZE:
        return JSONResponse(status_code=400, content={"status": "error", "message": get_text("imagen_muy_grande")})

    timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"{timestamp}_{file.filename}"
    upload_path = os.path.join("static", "uploads", filename)
    with open(upload_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    job_id = _enqueue_job(user_id, "diagnosis", {
        "model_name": model_name, "image_path": upload_path,
        "lang": get_lang_from_cookie(request),
    })
    position = _queue_position(job_id, "diagnosis")
    return JSONResponse(content={
        "status": "queued", "job_id": job_id, "position": position,
        "message": get_text("diagnostico_encolado").format(position=position),
    })

```

Código 2 - Solicitud de diagnóstico (routers/inference.py)

28.3. Servicios de aplicación

La capa de servicios se implementa en el paquete `services/` y concentra la lógica de aplicación del sistema. Los servicios se organizan por responsabilidad: `auth_service.py` gestiona la criptografía y las sesiones, `lang.py` la internacionalización, `ml_engine.py` y `xai_generator.py` la predicción y la explicabilidad, `pdf_generator.py` los informes del diagnóstico, `mlops_engine.py` y `trainer_engine.py` el laboratorio, `queue_worker.py` el procesamiento asíncrono, `chatbot_service.py` el asistente conversacional y `rate_limiter.py` y `csrf_middleware.py` los mecanismos de seguridad. El fragmento siguiente muestra la implementación de la creación y la verificación de los tokens de sesión del servicio de autenticación.

La implementación de los tokens refleja la política de seguridad de sesiones del diseño. El token de acceso se firma con JWT mediante la clave secreta del entorno e incluye el identificador del usuario y la fecha de expiración; el token de refresco se genera como un valor aleatorio con `secrets.token_urlsafe`, y en la base de datos solo se conserva su hash SHA-256 junto con la expiración, de modo que una credencial reutilizable no se almacena en texto plano. La gestión de la rotación y de la detección de robo, descrita en el capítulo 18, se implementa en las funciones de verificación del mismo servicio, que distinguen el token activo, el revocado dentro del periodo de gracia y el uso de una credencial revocada, revocando todos los tokens del usuario en este último caso.

```

def create_access_token(user_id: int) -> str:
    expire = datetime.datetime.now(datetime.UTC) + datetime.timedelta(minutes=JWT_ACCESS_EXPIRE_MINUTES)
    payload = {"sub": str(user_id), "exp": expire}
    return jwt.encode(payload, JWT_SECRET_KEY, algorithm="HS256")

def create_refresh_token(user_id: int) -> str:
    raw_token = secrets.token_urlsafe(48)
    token_hash = _hash_token(raw_token)
    expires_at = datetime.datetime.now(datetime.UTC) + datetime.timedelta(days=JWT_REFRESH_EXPIRE_DAYS)
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO refresh_tokens (user_id, token_hash, expires_at) VALUES (%s, %s, %s)",
        (user_id, token_hash, expires_at),
    )
    conn.commit()
    conn.close()
    return raw_token

```

Código 3 - Creación de los tokens de sesión (services/auth_service.py)

28.4. Persistencia y acceso a los datos

La persistencia se implementa en el módulo `database.py`, que gestiona el pool de conexiones a MySQL y la inicialización del esquema. El acceso a los datos se resuelve mediante un pool de conexiones configurado con las variables de entorno, de modo que las peticiones concurrentes reutilizan las conexiones del pool sin abrir una conexión por operación. El fragmento siguiente muestra la configuración del pool y la inicialización del esquema.

La implementación de la persistencia refleja el modelo físico del capítulo 20 y la generación del esquema del capítulo 24. El pool se configura con los parámetros del entorno y se crea perezosamente en la primera conexión; la inicialización del esquema ejecuta las sentencias `CREATE TABLE IF NOT EXISTS` de las cinco tablas del sistema —`users`, `consultations`, `training_jobs`, `job_queue` y `refresh_tokens`— de modo que el arranque crea el esquema sin un script de migración manual. Los routers y los servicios obtienen las conexiones mediante la función común, aplican las operaciones y las cierran tras confirmar la transacción, manteniendo la integridad de los datos.

```

def _get_pool():
    global _pool
    if _pool is None:
        _pool = MySQLConnectionPool(
            pool_name="mypool",
            pool_size=int(os.getenv("DB_POOL_SIZE", "5")),
            host=os.getenv("DB_HOST", "localhost"),
            user=os.getenv("DB_USER", "root"),
            password=os.getenv("DB_PASSWORD", ""),
            database=os.getenv("DB_NAME", "tfg_pneumonia"),
        )
    return _pool

def get_db_connection():
    return _get_pool().get_connection()

def init_db():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            username VARCHAR(255) NOT NULL UNIQUE,
            password_hash VARCHAR(255) NOT NULL,
            first_name VARCHAR(255) NOT NULL,
            last_name VARCHAR(255) NOT NULL,
            role VARCHAR(255) NOT NULL
        )
    """)
    conn.commit()
    conn.close()

```

Código 4 - Pool de conexiones e inicialización del esquema (database.py)

28.5. Seguridad transversal

La seguridad transversal del backend se implementa en los servicios `csrf_middleware.py` y `rate_limiter.py`, que se aplican a toda la aplicación mediante el middleware y el limitador configurados en el arranque. La protección CSRF se implementa como un middleware que establece una cookie de token en las peticiones seguras y exige su correspondencia en las peticiones que modifican el estado; las cabeceras de seguridad se añaden a todas las respuestas. El fragmento siguiente muestra la implementación de la protección CSRF.

La implementación de la protección CSRF refleja los requisitos de seguridad del análisis. Las peticiones seguras —GET, HEAD y OPTIONS— se eximen de la comprobación y establecen la cookie del token si no existe; el endpoint de inicio de sesión se exige porque se limita en frecuencia de forma independiente. Las peticiones que modifican el estado exigen que el token de la cookie coincida con el de la cabecera `x-csrf-token`, que la interfaz envía de forma automática en las peticiones asíncronas; si no coinciden, el middleware responde HTTP 403 sin procesar la petición. Las cabeceras de seguridad se añaden en todas las respuestas, y el limitador de peticiones aplica un límite global de sesenta peticiones por minuto y un límite específico de cinco peticiones por minuto al endpoint de inicio de sesión, en coherencia con las pruebas de protección del capítulo 17.

El backend de vitalXAI queda así codificado de forma coherente con el diseño: la aplicación se compone en el arranque con los mecanismos de seguridad y los routers de los seis subsistemas, la capa HTTP sigue el patrón de fachada ligera, los servicios concentran la lógica de aplicación, la persistencia se resuelve con el pool de conexiones y la inicialización del esquema, y la seguridad transversal protege las peticiones y las respuestas de toda la aplicación. La implementación de la ejecución asíncrona, que constituye el soporte del procesamiento en segundo plano, se describe en el capítulo siguiente.

```

class CSRFMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint) -> Response:
        if request.method in SAFE_METHODS or request.url.path in CSRF_EXEMPT_PATHS:
            response = await call_next(request)
            if CSRF_COOKIE_NAME not in request.cookies:
                token = secrets.token_urlsafe(32)
                response.set_cookie(key=CSRF_COOKIE_NAME, value=token, httponly=False, samesite="lax")
            return response

        csrf_cookie = request.cookies.get(CSRF_COOKIE_NAME)
        csrf_token = request.headers.get(CSRF_HEADER_NAME)
        if not csrf_cookie or not csrf_token or csrf_cookie != csrf_token:
            return JSONResponse(status_code=403, content={"status": "error", "message": "CSRF validation failed"})

        response = await call_next(request)
        return response

```

Código 5 - Protección CSRF (services/csrf_middleware.py)

29. Codificación de la ejecución asíncrona

La ejecución asíncrona constituye el soporte del procesamiento en segundo plano de vitalXAI: los diagnósticos, los entrenamientos y las validaciones externas se procesan fuera del ciclo de petición HTTP, de modo que la interfaz permanece operativa durante las tareas de larga duración. Este capítulo describe la codificación de este mecanismo, que se materializa en la cola persistente de trabajos y en el worker de la cola. La implementación se organiza en cinco apartados: la cola de trabajos y su máquina de estados, el bucle del worker, la reclamación y el despacho de los trabajos, la tolerancia a fallos y la recuperación, y la consulta y la cancelación desde la API.

La implementación de la ejecución asíncrona se apoya en la persistencia relacional descrita en el capítulo 29: la cola se materializa en la tabla `job_queue` de MySQL, que conserva los trabajos con su estado, su payload serializable, su resultado y su error, y el worker comparte esa persistencia con los routers, pero no comparte el ciclo de petición. Esta frontera, ya declarada en el diseño del subsistema SD-006, se implementa en el módulo `services/queue_worker.py`, que contiene el bucle del worker y las operaciones de transición de estado, y en el router `routers/queue.py`, que expone la consulta del estado y la cancelación de los trabajos.

29.1. *La cola de trabajos y su máquina de estados*

La cola de trabajos se implementa en la tabla `job_queue`, cuyas filas representan los trabajos asíncronos del sistema. Cada trabajo conserva el usuario propietario, el tipo (diagnosis, training o external_validation), el estado, el payload serializable, el resultado y el mensaje de error, de modo que la tabla materializa la clase `QueueJob` definida en el diseño de clases del capítulo 22 y su máquina de estados, representada en la figura 89. Las transiciones de estado se implementan en el worker mediante operaciones que actualizan la fila del trabajo: la reclamación, la finalización y el fallo. El fragmento siguiente muestra la implementación de estas transiciones.

La implementación de las transiciones refleja la máquina de estados de la cola. La reclamación utiliza una actualización condicionada a que el trabajo continúe en `queued`, de modo que solo afecta a una fila si el trabajo seguía pendiente; el resultado de la actualización determina si el worker ha adquirido el trabajo, lo que impide que dos iteraciones procesen el mismo registro. La finalización marca el trabajo como `completed` con el resultado en formato JSON, y el fallo lo marca como `failed` con el mensaje de error limitado a quinientos caracteres, de modo que el estado persistido nunca conserva un error desbordado. La cancelación desde la API se implementa con la misma técnica condicionada, limitada a los trabajos en `queued`, tal y como se describe en el apartado 30.5.

```

def _claim_job(job_id: int):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'running', started_at = %s WHERE id = %s AND status = 'queued'",
        (datetime.now(), job_id),
    )
    conn.commit()
    affected = cursor.rowcount
    conn.close()
    return affected > 0

def _finish_job(job_id: int, result: dict | None = None):
    conn = get_db_connection()
    cursor = conn.cursor()
    result_json = json.dumps(result) if result else None
    cursor.execute(
        "UPDATE job_queue SET status = 'completed', finished_at = %s, result = %s WHERE id = %s",
        (datetime.now(), result_json, job_id),
    )
    conn.commit()
    conn.close()

def _fail_job(job_id: int, error: str):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'failed', finished_at = %s, error_message = %s WHERE id = %s",
        (datetime.now(), error[:500], job_id),
    )
    conn.commit()
    conn.close()

```

Código 6 - Transiciones de estado del trabajo (services/queue_worker.py)

29.2. El bucle del worker

El worker se implementa como un bucle asíncrono que selecciona el siguiente trabajo pendiente, lo reclama, procesa el flujo correspondiente y actualiza su estado, apoyado en el modelo de concurrencia asíncrona de Python (Python Software Foundation, 2024). El bucle se crea en el arranque de la aplicación mediante `start_worker()`, que restablece los trabajos en ejecución y lanza la tarea asíncrona del procesamiento. El fragmento siguiente muestra la implementación del bucle del worker.

La implementación del bucle refleja las decisiones de diseño del procesamiento asíncrono. El worker selecciona el primer trabajo pendiente mediante la consulta de prioridad, que ordena los diagnósticos antes que los entrenamientos; si no hay trabajos, espera un segundo y vuelve a consultar. Cuando hay un trabajo, lo reclama con la actualización condicionada; si la reclamación no afecta a una fila, otro consumidor ya lo tomó y el worker continúa con el siguiente. La ejecución del flujo se delega en el executor de eventos mediante `run_in_executor`, de modo que el procesamiento intensivo no bloquea el bucle de eventos de la aplicación; al terminar, el trabajo se marca como completado con el resultado, y ante cualquier excepción se marca como fallido con el mensaje de error. El arranque del worker se integra en el ciclo de vida de la aplicación descrito en el capítulo 29, y conserva la tarea en el estado de la aplicación.

```

async def worker_loop():
    loop = asyncio.get_running_loop()
    while True:
        try:
            job = _next_job()
            if job is None:
                await asyncio.sleep(1)
                continue

            claimed = _claim_job(job["id"])
            if not claimed:
                continue

            try:
                result = await loop.run_in_executor(None, _execute_job, job)
                _finish_job(job["id"], result)
            except Exception as e:
                _fail_job(job["id"], str(e))
        except Exception:
            await asyncio.sleep(1)

def start_worker(app):
    _reset_running_jobs()
    task = asyncio.create_task(worker_loop())
    app.state.queue_worker = task

```

Código 7 - Bucle del worker y su arranque (services/queue_worker.py)

29.3. *Reclamación, despacho y procesamiento de los trabajos*

El procesamiento de cada trabajo se resuelve mediante el despacho por tipo, que selecciona el flujo correspondiente según el tipo del trabajo. La reclamación condicionada, descrita en el apartado 30.1, garantiza que el despacho solo se ejecuta sobre trabajos que el worker ha adquirido. El fragmento siguiente muestra la implementación del despacho y del procesamiento de un diagnóstico.

La implementación del procesamiento de un diagnóstico refleja el flujo asíncrono definido en el diseño del subsistema SD-002. El worker deserializa el payload del trabajo, invoca el motor de predicción, genera el mapa de explicabilidad y construye el informe PDF, y persiste la consulta en la tabla consultations con las rutas de los artefactos y el resultado. El procesamiento se ejecuta en el executor, de modo que la predicción y la generación de los artefactos no bloquean la interfaz. Los procesamientos de los entrenamientos y de las validaciones externas siguen el mismo patrón y delegan en el motor del laboratorio MLOps, que se describe en el capítulo 32. El resultado devuelto se conserva en el campo result del trabajo completado.

```

def _execute_job(job):
    if job["job_type"] == "diagnosis":
        return _process_diagnosis(job)
    elif job["job_type"] == "training":
        return _process_training(job)
    elif job["job_type"] == "external_validation":
        return _process_external_validation(job)
    else:
        raise ValueError(f"Unknown job type: {job['job_type']}")

def _process_diagnosis(job):
    payload = _get_payload(job)
    label, confidence = process_and_predict(payload["model_name"], payload["image_path"], lang=payload.get("lang", "es"))

    xai_path = os.path.join("static", "results", "xai_" + os.path.basename(payload["image_path"]))
    generate_xai_heatmap(payload["model_name"], payload["image_path"], xai_path, lang=payload.get("lang", "es"))

    pdf_path = generate_medical_report(payload["image_path"], xai_path, label, confidence, payload["model_name"], lang=payload.get("lang", "es"))

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO consultations (user_id, model_name, original_image_path, xai_image_path, prediction_label, confidence_score, patient_name, pdf_path) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
        (job["user_id"], payload["model_name"], payload["image_path"], xai_path, label, confidence, default_patient_name, pdf_path),
    )
    conn.commit()
    conn.close()
    return {"label": label, "confidence": confidence, "original_image": f"/{payload['image_path']}",
            "xai_image": f"/{xai_path}", "pdf_report": f"/{pdf_path}", "model_used": payload["model_name"]}

```

Código 8 - Despacho por tipo y procesamiento de un diagnóstico (services/queue_worker.py)

29.4. Tolerancia a fallos y recuperación

La ejecución asíncrona incorpora mecanismos de tolerancia a fallos y de recuperación que garantizan la coherencia del sistema ante errores y reinicios. Ante un fallo del procesamiento, el worker captura la excepción y marca el trabajo como fallido con el mensaje de error, de modo que un fallo del motor, de la imagen o del informe no se transforma en una consulta completada con datos ambiguos. Ante el reinicio de la aplicación, los trabajos que quedaron en estado running se restablecen a queued, de modo que el procesamiento interrumpido vuelve a la cola y puede ser retomado. El fragmento siguiente muestra la implementación del restablecimiento de los trabajos en ejecución.

La implementación del restablecimiento refleja la decisión de recuperación del diseño. Al arrancar la aplicación, `start_worker()` ejecuta `_reset_running_jobs()`, que devuelve a la cola los trabajos que quedaron en running cuando el proceso anterior se detuvo, con su fecha de inicio restablecida; la reclamación condicionada evita que esos trabajos se procesen dos veces, porque solo un consumidor puede adquirir cada trabajo. Esta técnica, junto con la captura de excepciones del bucle, garantiza que el estado persistido de la cola permanece coherente con la ejecución real, incluso ante fallos del proceso o de los componentes.

```
def _reset_running_jobs():  
    conn = get_db_connection()  
    cursor = conn.cursor()  
    cursor.execute("UPDATE job_queue SET status = 'queued', started_at = NULL WHERE status = 'running'")  
    conn.commit()  
    conn.close()
```

Código 9 - Restablecimiento de los trabajos en ejecución (services/queue_worker.py)

29.5. *La cola desde la API*

La consulta del estado y la cancelación de los trabajos se exponen en el router `routers/queue.py`, que materializa los casos de uso CU-034 y CU-035 del análisis. La consulta del estado devuelve los trabajos del usuario con su tipo, su estado, su posición y el nombre del modelo o de la sesión, interpretando el payload según el tipo; la cancelación aplica una actualización condicionada que solo afecta a los trabajos en `queued` pertenecientes al usuario. El fragmento siguiente muestra la implementación de la cancelación.

La implementación de la cancelación refleja la restricción del diseño: solo un trabajo en estado `queued` y perteneciente al usuario puede cancelarse. La actualización condicionada a esos tres criterios afecta a una fila únicamente cuando el trabajo sigue pendiente; si el worker ya lo reclamó o el trabajo finalizó, la actualización no afecta a ninguna fila y el router responde HTTP 404 informando de que el trabajo no está en cola. De este modo, un trabajo en ejecución no se interrumpe de forma abrupta, en coherencia con la máquina de estados de la cola. La consulta del estado, por su parte, interpreta el payload de cada tipo de trabajo para mostrar el nombre del modelo o de la sesión sin exponer el contenido interno completo del payload.

La ejecución asíncrona de `vitalXAI` queda así codificada de forma completa: la cola persistente materializa la máquina de estados de los trabajos, el worker reclama y procesa los trabajos fuera del ciclo de petición, el despacho por tipo orquesta los flujos de los subsistemas, los mecanismos de tolerancia a fallos y de recuperación mantienen la coherencia del estado, y la API expone la consulta y la cancelación a los usuarios. La implementación de los motores que el worker invoca —la predicción y la explicabilidad del diagnóstico y el laboratorio `MLOps`— se describe en los capítulos siguientes.

```

@router.delete("/api/queue/cancel/{job_id}")
async def cancel_job(request: Request, job_id: int):
    user_id = get_user_id_from_token(request.cookies.get("access_token"))
    if not user_id:
        return JSONResponse(status_code=401, content={"error": "No autenticado"})

    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE job_queue SET status = 'cancelled', finished_at = NOW() WHERE id = %s AND user_id = %s AND status = 'queued'",
        (job_id, user_id),
    )
    affected = cursor.rowcount
    conn.commit()
    conn.close()

    if affected == 0:
        return JSONResponse(status_code=404, content={"error": "Trabajo no encontrado o ya no está en cola"})
    return JSONResponse(content={"status": "success", "message": "Trabajo cancelado"})

```

Código 10 - Cancelación de un trabajo desde la API (routers/queue.py)

30. Codificación del motor de diagnóstico y XAI

El motor de diagnóstico y de explicabilidad constituye el núcleo clínico de vitalXAI: recibe una radiografía de tórax, produce la predicción de neumonía con su nivel de confianza y genera los mapas de calor que explican la decisión del modelo, además del informe PDF de la consulta. Este capítulo describe la codificación de este motor, que se materializa en tres servicios: `ml_engine.py`, que carga los modelos y ejecuta la predicción; `xai_generator.py`, que genera los mapas de explicabilidad; y `pdf_generator.py`, que construye el informe del diagnóstico. La implementación se organiza en cuatro apartados: la carga y la caché de los modelos, el preprocesado y la predicción, la generación de los mapas de explicabilidad y la generación del informe PDF.

El motor se apoya en las librerías de aprendizaje profundo y de visión por computador descritas en el entorno de construcción del capítulo 24: TensorFlow y la librería Transformers de Hugging Face para la carga de las arquitecturas, OpenCV para el preprocesado de las imágenes y FPDF para la generación del informe (TensorFlow, 2024; Hugging Face, 2024). La implementación distingue dos familias de arquitecturas —las convolucionales (CNN) y las Transformer—, cuyas decisiones de carga, preprocesado e inferencia se bifurcan en cada servicio, en coherencia con el diseño del subsistema SD-002 descrito en el capítulo 18.

30.1. *Carga y caché de los modelos*

La carga de los modelos se implementa en el servicio `ml_engine.py`, que diferencia el procedimiento según la familia de la arquitectura. Los modelos convolucionales se cargan desde sus pesos entrenados en formato Keras; los modelos Transformer se cargan desde su arquitectura preentrenada de Hugging Face y, a continuación, se aplican los pesos entrenados del proyecto. En ambos casos, el modelo se conserva en un diccionario global de caché, de modo que la primera consulta con cada arquitectura paga la carga de los pesos y las posteriores reutilizan el modelo en memoria. El fragmento siguiente muestra la implementación de la carga.

La implementación de la carga refleja las decisiones del diseño del motor. La diferenciación por familia permite cargar cada arquitectura con el procedimiento adecuado: las CNN se cargan directamente con el cargador de modelos de Keras, y los Transformer se construyen sobre la arquitectura de Hugging Face, con la salida de atención habilitada para la generación de los mapas de explicabilidad, y reciben los pesos entrenados del proyecto. La caché en memoria implementa el requisito de tiempo de respuesta de la inferencia (RNF-019): la primera consulta con cada arquitectura asume el coste de la carga, mientras que las posteriores reutilizan el modelo. La ausencia de los pesos se resuelve con un error explícito, de modo que un modelo no disponible falla antes de producir un resultado ambiguo, tal y como se declaró en el diseño.

```

loaded_models = {}
MODELS_TRANSFORMERS = ["deit", "swin_base", "vit_384"]
HF_MODEL_IDS = {
    "deit": "facebook/deit-base-distilled-patch16-224",
    "swin_base": "microsoft/swin-base-patch4-window7-224",
    "vit_384": "google/vit-base-patch16-384",
}

def get_model(model_name: str):
    if model_name not in loaded_models:
        is_transformer = model_name in MODELS_TRANSFORMERS

        if is_transformer:
            from transformers import TFAutoModelForImageClassification
            model_path = os.path.join("pneumoniaccn-main", "results", model_name, "best_fold1.h5")
            if not os.path.exists(model_path):
                model_path = os.path.join("pneumoniaccn-main", "results", model_name, "best_fold1.weights.h5")
            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Pesos de Transformer no encontrados en: {model_path}")

            model = TFAutoModelForImageClassification.from_pretrained(
                HF_MODEL_IDS[model_name], num_labels=1, ignore_mismatched_sizes=True, output_attentions=True
            )
            model.load_weights(model_path)
            loaded_models[model_name] = model
        else:
            model_path = os.path.join("pneumoniaccn-main", "results", model_name, "best_fold1.keras")
            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Modelo CNN no encontrado en: {model_path}")
            loaded_models[model_name] = load_model(model_path)

    return loaded_models[model_name]

```

Código 11 - Carga y caché de los modelos (services/ml_engine.py)

30.2. *Preprocesado y predicción*

La predicción se implementa en la función `process_and_predict`, que prepara la imagen según la arquitectura y ejecuta la inferencia. El preprocesado ajusta la resolución de entrada a la arquitectura seleccionada —299×299 para InceptionV3 y Xception, 384×384 para ViT-384 y 224×224 para el resto—, convierte la imagen al espacio RGB y la normaliza. La inferencia se bifurca por familia: los Transformer procesan la imagen con los `pixel_values` y aplican una sigmoide sobre el logit, mientras que las CNN utilizan la predicción directa del modelo. El fragmento siguiente muestra la implementación del preprocesado y de la inferencia.

La implementación de la predicción refleja las decisiones del diseño clínico. La normalización divide los píxeles entre 255 para escalarlos al intervalo de entrada de los modelos; la inferencia devuelve la probabilidad de la clase neumonía, y el umbral de 0.5 decide la etiqueta de la predicción. La confianza se expresa en el intervalo 0-100: para la clase neumonía coincide con la probabilidad, y para la clase normal se calcula como su complementaria, de modo que la confianza siempre refleja la seguridad de la predicción emitida. Las etiquetas se localizan mediante el servicio de idioma, en coherencia con la internacionalización de la plataforma.

```

def process_and_predict(model_name: str, image_path: str, lang: str = "es"):
    model = get_model(model_name)
    is_transformer = model_name in MODELS_TRANSFORMERS

    if model_name in ["InceptionV3", "Xception"]:
        img_size = (299, 299)
    elif model_name == "vit_384":
        img_size = (384, 384)
    else:
        img_size = (224, 224)

    img = cv2.imread(image_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img_rgb, img_size)
    img_array = img_resized / 255.0
    img_tensor = np.expand_dims(img_array, axis=0)

    if is_transformer:
        img_tensor_tf = tf.convert_to_tensor(img_tensor, dtype=tf.float32)
        logits = model(pixel_values=img_tensor_tf).logits
        prediction = float(tf.nn.sigmoid(logits)[0][0])
    else:
        prediction = float(model.predict(img_tensor, verbose=0)[0][0])

    is_pneumonia = prediction > 0.5
    label = get_text("label_pneumonia", lang) if is_pneumonia else get_text("label_normal", lang)
    confidence = prediction if is_pneumonia else (1 - prediction)
    confidence_percent = round(confidence * 100, 2)

    return label, confidence_percent

```

Código 12 - Preprocesado y predicción (services/ml_engine.py)

30.3. *Generación de los mapas de explicabilidad*

La generación de los mapas de explicabilidad se implementa en el servicio `xai_generator.py`, que calcula los tres mapas de la explicación visual y compone la figura con la radiografía original. Para las arquitecturas convolucionales se aplican los métodos basados en gradientes —Saliency, SmoothGrad y Grad-CAM—, mientras que para las Transformer se utiliza el mapa de atención de la última capa, con un respaldo basado en gradientes ante un fallo del cálculo de la atención. El fragmento siguiente muestra la implementación de la generación del mapa de explicabilidad.

La implementación de la explicabilidad refleja las decisiones del diseño del subsistema de diagnóstico. El mapa de Saliency se calcula mediante los gradientes de la puntuación de la clase respecto a la imagen, aplicados con una cinta de gradientes de TensorFlow; el mapa de SmoothGrad promedia treinta mapas de Saliency con ruido gaussiano y normaliza el resultado; y el mapa de Grad-CAM pondera las activaciones de la última capa convolucional por los gradientes, mientras que las Transformer producen un mapa de atención a partir de la última capa de atención. La figura resultante combina la radiografía original, el Saliency, el SmoothGrad y la superposición del mapa principal, se guarda con el backend no interactivo de matplotlib y se persiste su ruta en la consulta, de modo que las visualizaciones posteriores no repiten el cálculo.

```

def generate_xai_heatmap(model_name: str, original_image_path: str, xai_save_path: str, lang: str = "es"):
    model = get_model(model_name)
    is_transformer = model_name in MODELS_TRANSFORMERS
    img_size = get_img_size(model_name)
    xai_method_name = "Attention Map" if is_transformer else "Grad-CAM"

    img = load_img_tf(original_image_path, img_size)
    sal = saliency(model, img, is_transformer)
    sm = smoothgrad(model, img, is_transformer)
    cam = get_cam_or_attention(model, img, is_transformer, img_size)

    plt.figure(figsize=(15, 5))
    plt.subplot(1, 4, 1)
    plt.imshow(img)
    plt.title(get_text("xai_original", lang), fontsize=12)
    plt.axis("off")
    # ... subplots de Saliency, SmoothGrad y la superposición del Grad-CAM o la atención ...
    plt.tight_layout()
    plt.savefig(xai_save_path, bbox_inches="tight", dpi=150)
    plt.close()
    return xai_save_path

```

Código 13 - Generación del mapa de explicabilidad (services/xai_generator.py)

30.4. *Generación del informe PDF*

La generación del informe PDF se implementa en el servicio `pdf_generator.py`, que compone el documento de la consulta mediante la clase `PDFReport`, especialización de la librería `FPDF`. El informe incluye la fecha, el modelo utilizado, el diagnóstico con su color según el resultado, el nivel de confianza y las dos imágenes —la radiografía original y el mapa de explicabilidad—. El fragmento siguiente muestra la implementación de la generación del informe.

La implementación del informe refleja las decisiones del diseño del CU-037. El documento se compone con la clase `PDFReport`, que hereda de la librería y redefine la cabecera y el pie de página; el color del diagnóstico se determina según el resultado —rojo para la neumonía y verde para el resultado normal—, considerando las etiquetas en los idiomas soportados. Las imágenes se insertan en paralelo y, si la carga de alguna falla, se muestra un texto de error en su lugar, de modo que el documento no queda incompleto. El informe se guarda en el área de informes con un nombre basado en la fecha y se devuelve la ruta, que se persiste en la consulta para su descarga posterior.

El motor de diagnóstico y de explicabilidad de `vitalXAI` queda así codificado de forma completa: la carga y la caché de los modelos atienden la inferencia con las dos familias de arquitecturas, el preprocesado y la predicción producen la etiqueta y la confianza, la generación de los mapas de explicabilidad proporciona la justificación visual de la decisión y la generación del informe PDF materializa el documento de la consulta. Estos servicios, invocados por el worker de la cola descrito en el capítulo 30, constituyen el núcleo clínico de la plataforma; la codificación del laboratorio de experimentación `MLOps` se describe en el capítulo siguiente.

```

def generate_medical_report(image_path, xai_path, label, confidence, model_name, lang="es"):
    pdf = PDFReport(lang=lang)
    pdf.add_page()
    pdf.set_font("helvetica", "", 12)

    pdf.cell(0, 10, f" {get_text('pdf_date', lang)} {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    pdf.cell(0, 10, f" {get_text('pdf_model', lang)} {model_name}")
    pdf.ln(10)

    pdf.set_font("helvetica", "B", 14)
    if "neumonía" in label.lower() or "pneumonia" in label.lower() or "肺炎" in label or "न्यूमोनिया" in label:
        pdf.set_text_color(192, 57, 43)
    else:
        pdf.set_text_color(39, 174, 96)

    pdf.cell(0, 10, get_text("pdf_diagnosis", lang).format(label=label.upper()))
    pdf.set_text_color(0)
    pdf.cell(0, 10, get_text("pdf_confidence", lang).format(confidence=confidence))
    pdf.ln(10)

    try:
        pdf.image(image_path, 20, 100, 70)
        pdf.image(xai_path, 110, 100, 70)
    except Exception:
        pdf.cell(0, 10, get_text("pdf_error_images", lang))

    filename = f"report_{datetime.now().strftime('%Y%m%d%H%M%S')}.pdf"
    filepath = os.path.join("static", "reports", filename)
    pdf.output(filepath, "F")
    return filepath

```

Código 14 - Generación del informe PDF (services/pdf_generator.py)

31. Codificación del laboratorio MLOps

El laboratorio MLOps constituye el segundo núcleo funcional de vitalXAI: permite configurar, lanzar y analizar experimentos de entrenamiento de modelos de detección de neumonía, con validación cruzada, análisis de explicabilidad, comparación estadística y validación externa sobre cohortes independientes. Este capítulo describe la codificación de este laboratorio, que se materializa en el servicio `mlops_engine.py`, que organiza las sesiones de experimentación y orquesta el pipeline, y en el servicio `pdf_generator_mlops.py`, que genera el informe consolidado de cada sesión. La implementación se organiza en cinco apartados: la orquestación del pipeline de entrenamiento, la validación externa y la comparación estadística, la gestión de las sesiones, la lectura de los resultados y la generación del informe de la sesión.

La codificación del laboratorio sigue la decisión de diseño del capítulo 24: el pipeline de experimentación se ejecuta mediante los scripts del directorio `pneumoniacnn-main/code`, que el motor invoca como procesos externos con la configuración de cada trabajo transmitida por variables de entorno, apoyándose en el módulo de ejecución de procesos de Python (Python Software Foundation, 2024). Los scripts realizan el entrenamiento con validación cruzada, el análisis XAI cualitativo y cuantitativo, la comparación estadística y la validación externa, y escriben sus resultados en el directorio `training_results` de la sesión. El motor orquesta la ejecución de los scripts y resuelve la lectura de los resultados, manteniendo la persistencia híbrida de la plataforma: la cola y el estado en MySQL, y los artefactos en el sistema de ficheros.

31.1. Orquestación del pipeline de entrenamiento

La orquestación del entrenamiento se implementa en la función `run_training_queue`, que ejecuta el pipeline completo de una sesión: por cada modelo configurado, lanza el script de entrenamiento adecuado —convolucional o Transformer— y los scripts de análisis XAI, y al finalizar ejecuta el script de comparación estadística. La configuración de cada trabajo se transmite a los scripts mediante variables de entorno, y el registro de la ejecución se escribe en el archivo de log del entrenamiento. El fragmento siguiente muestra la implementación del bucle de orquestación.

La implementación de la orquestación refleja la decisión de diseño de delegar la computación intensiva en los scripts del pipeline. El motor itera sobre los modelos de la sesión, transmite la configuración mediante las variables de entorno `TFG_*` y ejecuta el script de entrenamiento correspondiente a la familia —el script 1 para las CNN o el script 2 para las Transformer—, seguido de los scripts de análisis XAI cualitativo y cuantitativo. La salida de los scripts se dirige al archivo de registro, de modo que la consola del laboratorio refleja la progresión del entrenamiento. Al completar todos los modelos, se ejecuta el script de comparación estadística, que genera el ranking y la matriz de Wilcoxon. Este flujo se ejecuta en el worker de la cola descrito en el capítulo 30, fuera del ciclo de petición.

```

def run_training_queue(session_id, models, dataset_path, epochs, batch_size, learning_rate):
    base_path = os.getcwd()
    script_train_cnn = os.path.join(base_path, "pneumoniaccnn-main", "code", "1_train_kfold.py")
    script_train_trans = os.path.join(base_path, "pneumoniaccnn-main", "code", "2_train_transformer_kfold.py")
    script_img = os.path.join(base_path, "pneumoniaccnn-main", "code", "6_xai_qualitative.py")
    script_math = os.path.join(base_path, "pneumoniaccnn-main", "code", "7_xai_quantitative.py")

    for model_name in models:
        env_vars = os.environ.copy()
        env_vars.update({
            "TFG_SESSION_ID": session_id, "TFG_MODEL_NAME": model_name,
            "TFG_DATASET_DIR": dataset_path, "TFG_EPOCHS": str(epochs),
            "TFG_BATCH_SIZE": str(batch_size), "TFG_LEARNING_RATE": str(learning_rate),
        })
        is_trans = model_name in TRANSFORMER_MODELS
        script_to_run = script_train_trans if is_trans else script_train_cnn
        with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
            subprocess.Popen(["python", script_to_run], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
            subprocess.Popen(["python", script_img], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
            subprocess.Popen(["python", script_math], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()

    env_vars_comp = os.environ.copy()
    env_vars_comp["TFG_SESSION_ID"] = session_id
    script_comp = os.path.join(base_path, "pneumoniaccnn-main", "code", "3_evaluate_statistics.py")
    with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
        subprocess.Popen(["python", script_comp], stdout=log, stderr=subprocess.STDOUT, env=env_vars_comp, text=True, encoding="utf-8", errors="replace").wait()

```

Código 15 - Orquestación del pipeline de entrenamiento (services/ml_ops_engine.py)

31.2. Validación externa y comparación estadística

La validación externa y la comparación estadística se implementan en las funciones `run_external_validation` y `run_statistical_comparison`, que ejecutan los scripts correspondientes del pipeline. La validación externa evalúa los modelos de la sesión sobre el dataset independiente y aplica el test de DeLong; la comparación estadística regenera el ranking y la matriz de Wilcoxon, gestionando el marcador de estado del recálculo. El fragmento siguiente muestra la implementación de la validación externa.

La implementación de la validación externa refleja la separación entre la validación y la configuración original del experimento: la función recibe la sesión y la ruta del dataset externo, transmite la ruta mediante la variable de entorno `TFG_EXTERNAL_DATASET_DIR` y ejecuta los scripts de validación y del test estadístico de DeLong, que escriben las métricas, la curva ROC y la matriz de DeLong en el subdirectorio `external_validation` de la sesión. La comparación estadística, por su parte, ejecuta el script 3 con el identificador de la sesión, limpia el marcador de estado del recálculo antes de ejecutarlo y lo recrea al finalizar, de modo que la interfaz puede consultar si el recálculo está en curso o completado.

```
def run_external_validation(session_id, dataset_path):
    base_path = os.getcwd()
    env_vars = os.environ.copy()
    env_vars["TFG_SESSION_ID"] = session_id
    env_vars["TFG_EXTERNAL_DATASET_DIR"] = dataset_path
    script_val = os.path.join(base_path, "pneumoniaccnn-main", "code", "4_external_validation.py")
    script_delong = os.path.join(base_path, "pneumoniaccnn-main", "code", "5_evaluate_delong.py")
    with open(LOG_FILE, "a", encoding="utf-8", errors="replace") as log:
        subprocess.Popen(["python", script_val], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
        subprocess.Popen(["python", script_delong], stdout=log, stderr=subprocess.STDOUT, env=env_vars, text=True, encoding="utf-8", errors="replace").wait()
```

Código 16 - Validación externa (services/ml_ops_engine.py)

31.3. *Gestión de las sesiones de experimentación*

La gestión de las sesiones se implementa en el motor MLOps, que organiza el directorio de cada sesión en `training_results` y resuelve las operaciones de creación, consulta, renombrado y eliminación. La creación de una sesión materializa el identificador, la configuración y el propietario; la consulta enumera las sesiones del usuario con sus modelos; el renombrado y la eliminación operan sobre el directorio con las comprobaciones de validez y de propiedad. El fragmento siguiente muestra la implementación de la creación de la sesión.

La implementación de la creación de la sesión refleja la persistencia híbrida del laboratorio: la sesión se materializa en un directorio de `training_results` con su configuración en `config.json` y la ruta del dataset en `dataset_path.txt`. El identificador se genera a partir de la fecha y la hora, y la configuración incorpora los modelos, los hiperparámetros y el identificador del usuario propietario, que la comprobación de propiedad de las sesiones utiliza para aislar los datos entre usuarios. Las operaciones de renombrado y de eliminación operan sobre el directorio mediante el sistema de ficheros, con la sanitización del nuevo nombre y las comprobaciones de existencia y de propiedad descritas en el diseño.

```

def create_training_session(model_names, dataset_path, epochs, batch_size, learning_rate, user_id=None):
    session_id = f"RUN_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}"
    session_dir = f"training_results/{session_id}"
    os.makedirs(session_dir, exist_ok=True)
    with open(os.path.join(session_dir, "dataset_path.txt"), "w", encoding="utf-8") as f:
        f.write(dataset_path)
    config = {"dataset_path": dataset_path, "epochs": epochs, "batch_size": batch_size,
             "learning_rate": learning_rate, "models": [m.strip() for m in model_names.split(",")]}
    if user_id is not None:
        config["user_id"] = user_id
    with open(os.path.join(session_dir, "config.json"), "w", encoding="utf-8") as f:
        json.dump(config, f)
    return session_id

```

Código 17 - Creación de la sesión de entrenamiento (services/ml_ops_engine.py)

31.4. *Lectura de los resultados*

La lectura de los resultados se implementa en las funciones de consulta del motor, que transforman los artefactos persistidos en el sistema de ficheros en estructuras para la interfaz. La consulta de los resultados de un modelo lee las métricas de validación cruzada, la calibración, las métricas XAI y las rutas de los artefactos visuales; la consulta del ranking lee el ranking por AUC y el heatmap de Wilcoxon; y la consulta de la validación externa lee las métricas, la curva ROC y la matriz de DeLong. El fragmento siguiente muestra la implementación de la lectura del ranking.

La implementación de la lectura de los resultados refleja la decisión de no ejecutar el modelo en las consultas: las funciones transforman los ficheros de resultados persistidos por el pipeline —los CSV de métricas y los artefactos visuales— en las estructuras que la interfaz presenta, devolviendo None cuando los resultados no existen para que el router informe con el código correspondiente. Esta decisión reduce el coste de acceso a los resultados y conserva la separación entre el cálculo y la visualización, en coherencia con el diseño del laboratorio.

```
def get_session_ranking_data(session_id):
    session_dir = f"training_results/{session_id}"
    csv_path = f"{session_dir}/session_ranking.csv"
    if not os.path.exists(csv_path):
        return None
    data = []
    with open(csv_path, encoding="utf-8") as f:
        for row in csv.DictReader(f):
            data.append(row)
    heatmap = f"/training_results/{session_id}/wilcoxon_heatmap.png"
    return {"ranking": data, "heatmap": heatmap, "config": read_config(session_dir)}
```

Código 18 - Lectura del ranking de la sesión (services/mlops_engine.py)

31.5. *Generación del informe de la sesión*

La generación del informe de la sesión se implementa en el servicio `pdf_generator_mlops.py`, que compone el documento consolidado mediante la clase `MedicalReport`, especialización de la librería `FPDF`. El informe recoge la configuración del experimento, el ranking con el heatmap de Wilcoxon, los resultados de la validación externa con la curva ROC y la matriz de DeLong, y el detalle técnico de cada modelo con sus métricas XAI y sus mapas de calor. El fragmento siguiente muestra el inicio de la implementación del generador.

La implementación del informe refleja la decisión de diseño de separar el conocimiento del formato del router: el generador recibe la sesión, lee los artefactos persistidos en el sistema de ficheros —la configuración, el ranking, los heatmaps, las métricas de la validación externa y las métricas XAI de cada modelo— y compone el documento con la clase `MedicalReport`, que redefine la cabecera corporativa, el pie de página y los títulos de sección. El informe se guarda en el directorio de la sesión y se devuelve como documento descargable; si la sesión no existe, el generador responde HTTP 404.

El laboratorio MLOps de vitalXAI queda así codificado de forma completa: el motor orquesta el pipeline de entrenamiento y de validación externa mediante los scripts del proyecto, gestiona las sesiones en el sistema de ficheros, lee los resultados persistidos para la interfaz y genera el informe consolidado de cada sesión. Con la codificación del motor de diagnóstico y del laboratorio, los motores del sistema quedan implementados; la codificación del frontend, que presenta estas capacidades al usuario, se describe en el capítulo siguiente.

```

async def generate_pdf_report(session_id):
    session_dir = f"training_results/{session_id}"
    if not os.path.exists(session_dir):
        return JSONResponse(status_code=404, content={"message": "Sesión no encontrada"})

    pdf = MedicalReport()
    pdf.set_auto_page_break(auto=True, margin=20)
    pdf.add_page()
    pdf.section_title("1. CONFIGURACIÓN DEL SISTEMA Y PARÁMETROS")

    config_path = os.path.join(session_dir, "config.json")
    if os.path.exists(config_path):
        with open(config_path, encoding="utf-8") as f:
            cfg = json.load(f)
            # ... secciones de configuración, ranking, Wilcoxon, validación externa y detalle por modelo ...

    pdf_output_path = os.path.join(session_dir, f"Informe_Completo_{session_id}.pdf")
    pdf.output(pdf_output_path)
    return FileResponse(pdf_output_path, filename=f"Reporte_MLOps_{session_id}.pdf")

```

Código 19 - Generación del informe de la sesión (services/pdf_generator_mlops.py)

32. Codificación del frontend

El frontend de vitalXAI materializa la presentación del sistema: las ventanas descritas en el diseño de interfaces del capítulo 23, que permiten al profesional sanitario y al investigador interactuar con las capacidades del backend. El frontend se construye con las plantillas Jinja2 servidas por el backend y con los recursos JavaScript del navegador, sin un framework de componentes independiente, en coherencia con la decisión tecnológica adoptada en el análisis. Este capítulo describe la codificación del frontend, organizada en cuatro apartados: las plantillas de presentación, los recursos JavaScript por ámbito, la comunicación asíncrona con la API y la internacionalización con el tema visual.

La presentación se resuelve con las plantillas Jinja2 del directorio `templates/`, que componen las páginas en el servidor, y con los recursos estáticos del directorio `static/`, que incluyen los scripts JavaScript y los estilos. La interactividad se implementa en JavaScript nativo, con módulos por ámbito funcional, y la comunicación con la API se realiza mediante la interfaz de `fetch` del navegador, con la protección CSRF y la renovación automática de la sesión (MDN Web Docs, 2024). El estilo visual se apoya en la utilidades de Tailwind CSS, cargadas mediante CDN y configuradas con el modo oscuro mediante clase (Tailwind CSS, 2024).

32.1. *Las plantillas de presentación*

Las plantillas Jinja2 componen las páginas del sistema en el servidor, inyectando los datos del usuario y los textos de la interfaz. Las páginas principales son el inicio de sesión (`login.html`), el registro (`register.html`), el panel de diagnóstico (`dashboard.html`) y el laboratorio de entrenamiento (`training.html`). Las plantillas combinan la estructura HTML con los atributos de internacionalización `data-i18n`, que el script de idioma traduce dinámicamente, y con las variables de contexto del servidor, como el nombre y el rol del usuario. El fragmento siguiente muestra una sección de la plantilla del panel de diagnóstico.

La implementación de las plantillas refleja la decisión del diseño de interfaces: las páginas se componen en el servidor con Jinja2, que inyecta el nombre y el rol del usuario, y se sirven como respuestas HTML. Las secciones de la interfaz se identifican con atributos `data-i18n`, que el módulo de idioma traduce sin recargar la página, y la navegación entre el panel y el laboratorio se resuelve con los enlaces de la barra lateral. Las ventanas integran además las vistas del historial, la cola de trabajos y la administración, tal y como se describió en el diseño de interfaces del capítulo 23.

```

<div class="p-4 border-b ... bg-blue-50 ...">
  <div class="flex items-center gap-3">
    <div class="w-10 h-10 rounded-full bg-blue-600 ...">
      <i class="fa-solid fa-user-md text-lg"></i>
    </div>
    <div class="overflow-hidden">
      <p class="text-[10px] font-bold ...">{{ role }}</p>
      <p class="text-sm font-black ...">{{ full_name }}</p>
    </div>
  </div>
</div>

<div class="p-3 space-y-1 border-b ...">
  <a href="/dashboard" class="flex items-center gap-3 px-3 py-2 rounded-lg bg-blue-100 ...">
    <i class="fa-solid fa-stethoscope w-5 text-center"></i>
    <span id="ui-nav-diag" data-i18n="navDiag">Diagnóstico Rápido</span>
  </a>
  <a href="/training" class="flex items-center gap-3 px-3 py-2 rounded-lg ...">
    <i class="fa-solid fa-flask w-5 text-center"></i>
    <span id="ui-nav-lab" data-i18n="navLab">Laboratorio de Entrenamiento</span>
  </a>
</div>

```

Código 20 - Plantilla del panel de diagnóstico (templates/dashboard.html)

32.2. *Los recursos JavaScript por ámbito*

La interactividad del frontend se implementa en JavaScript nativo, organizado en módulos por ámbito funcional: `dashboard.js` para el panel de diagnóstico, `training.js` para el laboratorio, `admin.js` para la administración y la cola, e `i18n.js` para la internacionalización compartida. Cada módulo se encarga de la interacción de su ventana: la carga y el envío de la radiografía, el sondeo del estado de la cola, el render de los resultados, la gestión de los experimentos del laboratorio y los diálogos de la administración. Los módulos se cargan en las plantillas al final del documento, y se comunican con la API mediante las peticiones asíncronas descritas en el apartado siguiente.

La separación por ámbito refleja la organización del código del frontend: cada módulo conoce los elementos de su ventana y las operaciones de su subsistema, de modo que los cambios de un ámbito no afectan al resto. El módulo `i18n.js` es el único compartido, y se carga en todas las páginas para aplicar el idioma y el tema visual. La coordinación entre módulos se resuelve mediante la ventana global, que expone las funciones de interacción que las plantillas invocan desde los atributos de los elementos, como el envío de la imagen o la apertura del detalle de una consulta.

32.3. *Comunicación asíncrona con la API*

La comunicación con la API se realiza mediante la interfaz de `fetch` del navegador, con el envío de los formularios de forma asíncrona mediante `FormData`, de modo que las operaciones se procesan sin recargar la página. La protección CSRF se gestiona mediante un interceptor global de peticiones, que añade la cabecera `X-CSRF-Token` a las peticiones que modifican el estado y gestiona la renovación automática de la sesión ante una respuesta de no autenticado. El fragmento siguiente muestra la implementación del interceptor global.

La implementación de la comunicación asíncrona refleja las decisiones de seguridad y de experiencia del sistema. El interceptor reemplaza la función `fetch` global: añade la cabecera `X-CSRF-Token` con el token de la cookie a todas las peticiones que modifican el estado, en correspondencia con la protección CSRF del backend descrita en el capítulo 29, y ante una respuesta de no autenticado intenta renovar la sesión mediante el endpoint de refresco, reintentando la petición original si la renovación tiene éxito o redirigiendo al inicio de sesión en caso contrario. Este mecanismo, aplicado de forma transversal, mantiene la sesión del usuario sin interrupciones y evita duplicar la gestión de la autenticación en cada módulo.

```

(function() {
  const c = document.cookie.match(/csrf_token=([^\;]+)/);
  if (!c) return;
  const csrfToken = c[1];
  const origFetch = window.fetch;
  let _isRefreshing = false;
  window.fetch = async function(url, options) {
    if (options && (options.method || 'GET').toUpperCase() !== 'GET') {
      options = Object.assign({}, options);
      options.headers = Object.assign({}, options.headers || {});
      options.headers['X-CSRF-Token'] = csrfToken;
    }
    let response = await origFetch(url, options);
    if (response.status === 401 && !_isRefreshing) {
      _isRefreshing = true;
      try {
        const refreshResp = await origFetch('/api/token/refresh', { method: 'POST', headers: { 'X-CSRF-Token': csrfToken } });
        if (refreshResp.ok) {
          response = await origFetch(url, options);
        } else {
          window.location.href = '/';
          return response;
        }
      } catch (e) {
        window.location.href = '/';
      }
    }
    return response;
  };
})();

```

Código 21 - Interceptor global de peticiones con CSRF y renovación de sesión (static/js/admin.js)

32.4. Internacionalización y tema visual

La internacionalización y el tema visual se implementan en el módulo `i18n.js`, compartido por todas las páginas. La internacionalización mantiene un diccionario de traducciones en el navegador para los cuatro idiomas soportados, aplica los textos a los elementos `data-i18n` y persiste la preferencia del idioma en el almacenamiento local; el tema visual alterna entre el tema claro y el oscuro aplicando la clase correspondiente en el elemento raíz y respeta la preferencia del sistema cuando no hay una guardada. El fragmento siguiente muestra la implementación de la traducción y del cambio de idioma.

La implementación de la internacionalización refleja las decisiones del diseño del CU-004: el selector de idioma actualiza la preferencia en el almacenamiento local y traduce de forma inmediata los textos de la interfaz marcados con `data-i18n`, sin recargar la página, con el español como valor por defecto. El tema visual alterna la clase del modo oscuro en el elemento raíz, de modo que el cambio se propaga a toda la interfaz, y persiste la preferencia en el almacenamiento local; al cargar la página, se restaura la preferencia guardada y, en ausencia de esta, se respeta la preferencia de color del sistema. Los módulos de las ventanas complementan la traducción re-renderizando las vistas dinámicas, como el historial o los resultados del laboratorio, en el idioma seleccionado.

El frontend de vitalXAI queda así codificado de forma completa: las plantillas Jinja2 componen las ventanas en el servidor, los recursos JavaScript por ámbito gestionan la interacción de cada vista, la comunicación asíncrona se protege con el interceptor CSRF y la renovación de sesión, y la internacionalización con el tema visual se aplican de forma transversal. Con la codificación del backend, de la ejecución asíncrona, de los motores y del frontend, la Parte de Codificación de la memoria queda completada, describiendo la implementación integral del sistema vitalXAI.

```
function t(key) {
  const lang = currentLang || localStorage.getItem('apLang') || 'es';
  const langDict = dict[lang] || dict.es;
  return langDict[key] || dict.es[key] || key;
}

function changeLanguage() {
  const selector = document.getElementById('lang-selector');
  if (selector) currentLang = selector.value;
  localStorage.setItem('apLang', currentLang);
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.getAttribute('data-i18n');
    el.innerText = t(key);
  });
}

function toggleTheme() {
  const html = document.documentElement;
  if (html.classList.contains('dark')) {
    html.classList.remove('dark');
    localStorage.setItem('theme', 'light');
  } else {
    html.classList.add('dark');
    localStorage.setItem('theme', 'dark');
  }
}
```

Código 22 - Internacionalización y tema visual (static/js/i18n.js)

[OBJ]
[OBJ]
[OBJ]
[OBJ]

Parte V.

Pruebas

33. Estado del Arte y Contextualización

La parte de verificación de la memoria presenta los resultados de las pruebas realizadas sobre el sistema vitalXAI, describiendo cómo se comprobó que la implementación satisface los requisitos funcionales y no funcionales del análisis. El plan de pruebas del capítulo 17 definió las verificaciones previstas del sistema, con sus códigos y sus criterios de éxito, y la estrategia de verificación del capítulo 26 especificó los entornos y los niveles de prueba; esta parte recoge la ejecución de esa verificación y sus resultados, de modo que la memoria documenta tanto lo que se planificó como lo que se comprobó realmente. La verificación se apoya en la batería de pruebas automatizadas del proyecto y en las comprobaciones estáticas de calidad, cuya metodología se describió en la introducción a la codificación del capítulo 28 (Myers, Sandler, & Badgett, 2011).

La parte se organiza en cinco capítulos. Este capítulo introductorio presenta el alcance de la verificación, los entornos y las herramientas empleadas y la organización de la parte; el capítulo 35 describe las pruebas automatizadas —unitarias y de integración— con sus resultados; el capítulo 36 describe las pruebas de seguridad; el capítulo 37 describe las pruebas de rendimiento; y el capítulo 38 describe las pruebas de sistema y la validación funcional de los flujos completos de uso. Cada capítulo presenta las pruebas ejecutadas, los resultados obtenidos y la correspondencia con las categorías del plan de pruebas del capítulo 17.

La verificación de vitalXAI se ejecutó sobre la implementación descrita en la parte de codificación, siguiendo la estrategia definida en el capítulo 26: la batería automatizada se ejecutó en los entornos unitario y de integración, con los servicios externos simulados, y las pruebas de rendimiento y de sistema se definieron sobre el despliegue de la plataforma. Los resultados de la ejecución de la batería automatizada y de las comprobaciones estáticas, que se detallan en los capítulos siguientes, confirman que el sistema implementado cumple los criterios de aceptación definidos en el plan de pruebas.

33.1. Alcance de la verificación

El alcance de la verificación de vitalXAI abarca las cinco categorías del plan de pruebas del capítulo 17: la verificación de los componentes individuales, la verificación de los flujos entre subsistemas, las pruebas de protección y control de acceso, las pruebas de rendimiento y capacidad de respuesta, y la verificación integral de los flujos completos de uso. Las dos primeras categorías se materializan en la batería automatizada de pruebas unitarias y de integración; la tercera se materializa en las pruebas de seguridad automatizadas; y las dos últimas se definen sobre el entorno de despliegue, conforme a la estrategia del capítulo 26. La tabla siguiente resume el alcance de la verificación y su correspondencia con las categorías del plan.

Categoría del plan	Nivel	Materialización
Verificación de componentes individuales	Unitario	Pruebas unitarias del directorio tests/unit/, por subsistema.
Verificación de los flujos entre subsistemas	Integración	Pruebas de integración del directorio tests/integration/.
Protección y control de acceso	Seguridad	Pruebas de seguridad automatizadas (CSRF, cabeceras, limitación, tokens).
Rendimiento y capacidad de respuesta	Sistema	Pruebas de rendimiento sobre el despliegue (PR-001 a PR-005).
Verificación integral de los flujos completos	Sistema	Pruebas de sistema de los flujos de uso (PE-001 a PE-006).

Tabla x - Alcance de la verificación

El alcance se define de modo que la verificación progrese desde el aislamiento de los componentes hasta la comprobación integral del sistema: cada nivel cubre un grado de fidelidad mayor con respecto al entorno real, y la superación de cada nivel es condición para considerar verificada la parte del sistema que abarca, en coherencia con los criterios de aceptación definidos en el capítulo 26.

33.2. Entornos y herramientas de verificación

La verificación se ejecutó en los tres entornos definidos en la estrategia de verificación del capítulo 26: el entorno unitario, sin dependencia de la red, de la base de datos ni de los servicios externos, con los dobles de prueba; el entorno de integración, con la base de datos en memoria y los servicios externos simulados; y el entorno de sistema, sobre el despliegue real de la plataforma. Las herramientas de verificación empleadas son las especificadas en el capítulo 26 y en la guía de pruebas del proyecto: el marco pytest con la extensión de cobertura pytest-cov y la extensión pytest-mock para la simulación de los servicios, y las comprobaciones estáticas ruff y mypy (pytest, 2024; Ruff, 2024; Mypy, 2024).

La ejecución de la batería automatizada y de las comprobaciones estáticas sobre la implementación actual del proyecto produjo los resultados globales que se resumen en la tabla siguiente. La batería completa de pruebas unitarias y de integración se ejecutó en el entorno de verificación automatizada, la cobertura de los módulos de la aplicación se midió sobre el código implementado y las comprobaciones estáticas se aplicaron sobre el repositorio.

Verificación	Resultado
Batería de pruebas automatizadas	190 pruebas superadas, sin fallos.
Cobertura de código	73,72 % sobre los módulos de la aplicación, por encima del umbral del 70 %.
Análisis estático (ruff)	Todas las comprobaciones superadas sin errores.
Verificación de tipos (mypy)	Sin problemas en los módulos configurados.

Tabla x - Entornos y herramientas de verificación

Los resultados detallados de cada nivel —la distribución de las pruebas por subsistema, la cobertura por módulo y los resultados de las comprobaciones de seguridad— se presentan en los capítulos 35 y 36, de modo que este resumen global se desglosa en la verificación específica de cada componente y de cada mecanismo.

33.3. Organización de la parte

La parte de verificación se organiza por niveles de prueba, de modo que cada capítulo presenta las pruebas de un nivel y sus resultados. El capítulo 35 describe las pruebas automatizadas: las pruebas unitarias de los componentes de cada subsistema y las pruebas de integración de los flujos entre subsistemas, con los ficheros de pruebas, la distribución de las verificaciones y la cobertura alcanzada. El capítulo 36 describe las pruebas de seguridad automatizadas: la protección CSRF, las cabeceras de seguridad, la limitación de peticiones y la gestión de los tokens de sesión. El capítulo 37 describe las pruebas de rendimiento y capacidad de respuesta, con sus criterios de éxito sobre el entorno de despliegue. El capítulo 38 describe las pruebas de sistema y la validación funcional de los flujos completos de uso, presentando cada flujo con su criterio de éxito y el resultado de su verificación. Esta organización permite leer la verificación como una progresión desde los componentes aislados hasta el sistema completo, en coherencia con el plan y con la estrategia de verificación de la memoria.

34. Pruebas automatizadas

Las pruebas automatizadas constituyen el primer nivel de la verificación de vitalXAI: comprueban el comportamiento de los componentes y de los flujos entre subsistemas de forma repetible, sin depender del entorno real de despliegue. Este capítulo describe las pruebas automatizadas del proyecto y presenta los resultados reales de su ejecución: las pruebas unitarias de los componentes, las pruebas de integración de los flujos entre subsistemas, la ejecución de la batería con las comprobaciones estáticas y la cobertura de los módulos de la aplicación. La implementación de estas pruebas, desarrollada conforme a la metodología TDD descrita en el capítulo 28, se corresponde con las categorías de verificación de componentes y de flujos entre subsistemas del plan de pruebas del capítulo 17.

La batería automatizada se ejecuta con el marco `pytest`, con la extensión de cobertura `pytest-cov` y la extensión de simulación `pytest-mock`, y se organiza en los directorios `tests/unit/` y `tests/integration/` del repositorio (`pytest`, 2024). Las pruebas unitarias sustituyen la base de datos, los modelos de TensorFlow y los servicios externos por dobles de prueba, de modo que la batería es determinista y no depende de la red ni de la disponibilidad de los servicios; las pruebas de integración utilizan una base de datos en memoria y simulan las APIs externas, de modo que verifican la colaboración entre las capas del sistema.

34.1. Las pruebas unitarias

Las pruebas unitarias verifican el comportamiento de los componentes del sistema de manera aislada, organizadas por subsistema conforme a la estructura del capítulo 17. El conjunto comprende 186 pruebas distribuidas en 19 ficheros del directorio `tests/unit/`, que cubren los servicios y los enrutadores del backend: la gestión de cuentas y sesiones, la validación de las entradas, el motor de inferencia, la generación de las explicaciones, la gestión del historial, el laboratorio de entrenamiento, la cola de trabajos, la internacionalización y la capa de acceso a datos. La tabla siguiente desglosa las pruebas unitarias por fichero, agrupadas por subsistema.

La distribución de las pruebas refleja la complejidad funcional de cada subsistema: el laboratorio MLOps concentra el mayor número de verificaciones, por ser el bloque de mayor complejidad, seguido del acceso y la gestión de cuentas y del motor de diagnóstico. Las pruebas de cada fichero materializan las verificaciones unitarias del plan de pruebas del capítulo 17 —los códigos PU-001 a PU-037— y se ejecutan de forma aislada, con la base de datos, los modelos y los servicios externos sustituidos por dobles.

Subsistema	Fichero de pruebas	Nº de pruebas
Laboratorio MLOps	test_trainer_router.py	30
Acceso y gestión de cuentas	test_auth_router.py	15
Laboratorio MLOps	test_trainer_engine.py	14
Motor de diagnóstico	test_xai_generator.py	13
Capacidades transversales	test_lang.py	12
Laboratorio MLOps	test_mlops_engine.py	11
Motor de diagnóstico	test_ml_engine.py	11
Capacidades transversales	test_queue_worker.py	10
Supervisión y administración	test_admin_router.py	10
Acceso y gestión de cuentas	test_auth_service.py	10
Historial	test_history_router.py	9
Acceso y gestión de cuentas	test_input_validation.py	8
Seguridad	test_security_headers.py	7
Persistencia	test_database.py	6
Capacidades transversales	test_queue_router.py	6
Seguridad	test_csrf_middleware.py	5
Motor de diagnóstico	test_pdf_generator.py	4
Motor de diagnóstico	test_inference_router.py	3
Seguridad	test_rate_limiting.py	2

Tabla x - Recuento de pruebas unitarias

34.2. Las pruebas de integración

Las pruebas de integración verifican la colaboración entre las capas del sistema y entre los subsistemas, superando el aislamiento de las pruebas unitarias. El conjunto comprende 4 pruebas distribuidas en el fichero `test_auth_flow.py` del directorio `tests/integration/`, que ejercitan el flujo completo de acceso a la plataforma: el registro de un nuevo usuario, el inicio de sesión con las credenciales registradas, el acceso al panel y la redirección de un usuario no autenticado al punto de entrada, además de la verificación de que un usuario recién registrado no dispone de consultas en su historial. Las pruebas utilizan una base de datos en memoria y simulan los servicios externos, de modo que la verificación se centra en la colaboración entre la API, la capa de negocio y la persistencia.

Las pruebas de integración materializan las verificaciones PI-001 a PI-003 del plan de pruebas del capítulo 17, confirmando de extremo a extremo el comportamiento especificado en los casos de uso CU-001, CU-002 y CU-005. Las ampliaciones previstas del plan —los flujos del diagnóstico, el aislamiento de datos, la supervisión administrativa y el laboratorio— se definen como verificaciones de integración futuras, conforme a la estrategia de verificación del capítulo 26, y su ejecución está prevista sobre los mismos principios de aislamiento y de simulación.

34.3. Ejecución y resultados de la batería

La ejecución de la batería completa de pruebas automatizadas, mediante el comando de ejecución de `pytest` sobre el directorio de pruebas, produjo un resultado global de 190 pruebas superadas sin fallos, con una duración de ejecución de aproximadamente un minuto y medio. La batería comprende las 186 pruebas unitarias y las 4 pruebas de integración descritas en los apartados anteriores, y su ejecución se acompaña de las comprobaciones estáticas de calidad: el análisis estático con `ruff`, que superó todas las comprobaciones sin errores, y la verificación de tipos con `mypy`, que no encontró problemas en los módulos de seguridad configurados (Ruff, 2024; Mypy, 2024). Estos resultados confirman que la implementación del sistema satisface los criterios de aceptación del nivel unitario y del nivel de integración definidos en el capítulo 26.

La ejecución de la batería se integra en el flujo de integración continua del repositorio, de modo que las pruebas y las comprobaciones estáticas se ejecutan en cada integración de la rama principal y en cada petición de cambios, conforme al flujo de trabajo descrito en el capítulo 28. El resultado global de la batería se resume en la tabla siguiente.

Verificación	Resultado
Pruebas unitarias	186 pruebas superadas.
Pruebas de integración	4 pruebas superadas.
Total de la batería	190 pruebas superadas, sin fallos.
Análisis estático (ruff)	Todas las comprobaciones superadas.
Verificación de tipos (mypy)	Sin problemas en los módulos configurados.

Tabla x - Ejecución y resultados

34.4. Cobertura de los módulos

La cobertura de los módulos de la aplicación se midió con la extensión pytest-cov sobre los paquetes de servicios, los routers y el módulo de persistencia, obteniendo una cobertura total del 73,72 %, por encima del umbral mínimo del setenta por ciento definido en la guía de pruebas del proyecto. La tabla siguiente desglosa la cobertura alcanzada por cada módulo de la aplicación.

Módulo	Cobertura
services/csrf_middleware.py	100 %
services/lang.py	100 %
services/rate_limiter.py	100 %
routers/inference.py	96 %
services/ml_engine.py	96 %
routers/admin.py	95 %
services/auth_service.py	94 %
services/chatbot_service.py	92 %
routers/auth.py	86 %
routers/history.py	85 %
routers/trainer.py	84 %
services/xai_generator.py	80 %
services/pdf_generator.py	79 %
routers/queue.py	74 %
services/trainer_engine.py	71 %
services/pdf_generator_mlops.py	57 %
services/queue_worker.py	48 %
services/mlops_engine.py	41 %
Total	73,72 %

Tabla x - Cobertura de los módulos

La cobertura refleja la estrategia de verificación de la plataforma: los módulos de seguridad transversal alcanzan el cien por cien de cobertura, y los módulos del diagnóstico y del acceso presentan valores elevados. Los módulos del laboratorio y de la ejecución asíncrona presentan los valores más bajos, porque su lógica se resuelve en gran medida en los scripts del pipeline y en los procesos del worker, que las pruebas unitarias verifican de forma parcial mediante los dobles de prueba; las verificaciones de estos flujos se completan en los niveles de rendimiento y de sistema descritos en los capítulos 37 y 38. Con la superación de la batería automatizada y de las comprobaciones estáticas, el sistema queda verificado en los niveles unitario y de integración; las pruebas de seguridad, que comparten esta batería, se presentan con detalle en el capítulo siguiente.

35. Pruebas de seguridad

Las pruebas de seguridad verifican que la plataforma aplica correctamente los mecanismos de protección especificados en los requisitos no funcionales de seguridad del análisis, en coherencia con los vectores de ataque más habituales sobre las aplicaciones web autenticadas (OWASP, 2024). Este capítulo describe las pruebas de seguridad automatizadas del proyecto y presenta los resultados reales de su ejecución: la protección CSRF, las cabeceras de seguridad, la limitación de peticiones y la gestión de los tokens de sesión. Estas pruebas se corresponden con la categoría de protección y control de acceso del plan de pruebas del capítulo 17 —los códigos PS-001 a PS-009— y se integran en la batería automatizada descrita en el capítulo 35.

Las pruebas de seguridad se implementan en los ficheros `test_csrf_middleware.py`, `test_security_headers.py`, `test_rate_limiting.py` y `test_auth_service.py` del directorio `tests/unit/`, y se ejecutan de forma aislada, sin dependencia del entorno real, verificando los mecanismos de seguridad implementados en el backend descrito en el capítulo 29. La ejecución específica de estos ficheros produjo un resultado de 24 pruebas superadas sin fallos, como se detalla en cada apartado.

35.1. Protección CSRF

La protección CSRF se verifica con las cinco pruebas del fichero `test_csrf_middleware.py`, que comprueban el comportamiento del middleware de protección frente a las peticiones que modifican el estado. Las pruebas cubren los casos PS-001 a PS-004 del plan: el rechazo de una petición POST sin token CSRF, el procesamiento de una petición POST con un token válido, el rechazo de una petición POST con un token erróneo y la exención de las peticiones GET de la protección. La tabla siguiente desglosa estas verificaciones.

ID	Verificación	Resultado
PS-001	Una petición POST sin token CSRF es rechazada.	Superada.
PS-002	Una petición POST con token CSRF válido se procesa.	Superada.
PS-003	Una petición POST con token CSRF erróneo es rechazada.	Superada.
PS-004	Las peticiones GET están exentas de la protección CSRF.	Superada.

Tabla x - Protección CSRF

Las verificaciones confirman que el middleware implementado en `csrf_middleware.py` establece la cookie del token en las peticiones seguras y exige su correspondencia con la cabecera en las peticiones que modifican el estado, de modo que una petición de modificación sin el token o con un token erróneo responde HTTP 403 sin procesarse, tal y como se describió en la codificación del backend.

35.2. Cabeceras de seguridad

Las cabeceras de seguridad se verifican con las siete pruebas del fichero `test_security_headers.py`, que comprueban que todas las respuestas de la aplicación incluyen las cabeceras configuradas. Las pruebas cubren los casos PS-005 a PS-007 del plan y las cabeceras complementarias: la política de seguridad de contenido (Content-Security-Policy), la seguridad de transporte (Strict-Transport-Security), la opción de tipo de contenido (X-Content-Type-Options), el marco (X-Frame-Options), la protección de XSS y la política de referente. La tabla siguiente resume las verificaciones principales.

ID	Verificación	Resultado
PS-005	Las respuestas incluyen la cabecera Content-Security-Policy.	Superada.
PS-006	Las respuestas incluyen la cabecera Strict-Transport-Security.	Superada.
PS-007	Las respuestas incluyen la cabecera X-Content-Type-Options.	Superada.

Tabla x - Cabeceras de seguridad

Las verificaciones confirman que el middleware de cabeceras de seguridad añade las cabeceras configuradas a todas las respuestas de la aplicación, en coherencia con las directivas definidas en la codificación del backend, de modo que el navegador aplica las restricciones de contenido, transporte y tipo MIME en cada respuesta.

35.3. Limitación de peticiones

La limitación de peticiones se verifica con las dos pruebas del fichero `test_rate_limiting.py`, que comprueban el comportamiento del limitador configurado en el backend. Las pruebas cubren la limitación del endpoint de inicio de sesión, que permite cinco peticiones por minuto, verificando que la sexta petición en el mismo minuto es rechazada, y la aplicación de la limitación global sobre las peticiones de la aplicación. La tabla siguiente resume estas verificaciones.

ID	Verificación	Resultado
PS-008	El inicio de sesión está limitado en frecuencia.	Superada.

Tabla x - Limitación de peticiones

La verificación confirma que el limitador implementado en `rate_limiter.py` rechaza los intentos reiterados de acceso desde una misma dirección, en coherencia con el requisito de protección frente a ataques de fuerza bruta y con la codificación del backend descrita en el capítulo 29.

35.4. Gestión de los tokens de sesión

La gestión de los tokens de sesión se verifica con las diez pruebas del fichero `test_auth_service.py`, que comprueban el comportamiento del servicio de autenticación respecto a las credenciales y a los tokens. Las pruebas cubren la creación y la verificación de los tokens de acceso, la creación y la verificación de los tokens de refresco, la invalidación de los tokens revocados, la rotación del token de refresco y la detección del robo de sesión. Entre ellas se encuentra la verificación PS-009 del plan, que comprueba que el acceso con un token de refresco revocado es rechazado. La tabla siguiente resume las verificaciones principales.

ID	Verificación	Resultado
PS-009	El acceso con un token de refresco revocado es rechazado.	Superada.
—	La rotación del token de refresco invalida el anterior.	Superada.
—	La detección de robo de sesión revoca todos los tokens del usuario.	Superada.

Tabla x - Gestión de los tokens de sesión

Las verificaciones confirman que el servicio de autenticación implementado en `auth_service.py` gestiona correctamente el ciclo de vida de los tokens: la verificación distingue el token activo, el revocado dentro del periodo de gracia y el uso de una credencial revocada, y ante este último revoca todos los tokens del usuario, tal y como se describió en la codificación del backend y en el diseño del subsistema de acceso.

35.5. Resultados de la verificación de seguridad

La ejecución específica de las pruebas de seguridad produjo un resultado de 24 pruebas superadas sin fallos, distribuidas en los cuatro ficheros de verificación. La tabla siguiente resume el resultado de la verificación de seguridad.

Fichero de pruebas	Verificaciones	Resultado
test_csrf_middleware.py	5	Superadas.
test_security_headers.py	7	Superadas.
test_rate_limiting.py	2	Superadas.
test_auth_service.py	10	Superadas.
Total	24	Superadas.

Tabla x - Resultados de la verificación de seguridad

Las verificaciones de seguridad confirman que la plataforma aplica correctamente los mecanismos de protección frente a los vectores de ataque más habituales sobre las aplicaciones web: la falsificación de peticiones en sitios cruzados, la exposición de cabeceras de seguridad, los ataques de fuerza bruta sobre el acceso y la reutilización indebida de las credenciales de sesión. La verificación de la seguridad en el entorno real de despliegue, junto con las pruebas de rendimiento y de los flujos completos, se describe en los capítulos siguientes de la parte de verificación.

36. Pruebas de rendimiento

Las pruebas de rendimiento verifican que la plataforma satisface los requisitos no funcionales de rendimiento y capacidad de respuesta definidos en el análisis: el tiempo de respuesta de la inferencia (RNF-019), la ejecución sin bloqueo de la interfaz durante las tareas de larga duración (RNF-020) y la capacidad de acceso concurrente (RNF-021) (Myers, Sandler, & Badgett, 2011). Este capítulo describe las pruebas de rendimiento del plan de pruebas del capítulo 17 —los códigos PR-001 a PR-005—, sus criterios de éxito y las condiciones de medición sobre el entorno de despliegue. A diferencia de las pruebas automatizadas de los capítulos 35 y 36, estas verificaciones se ejecutan sobre el sistema real, de modo que los resultados reflejan la infraestructura de operación y no los dobles de prueba.

Las pruebas de rendimiento se corresponden con la categoría de rendimiento y capacidad de respuesta del plan de pruebas del capítulo 17, y su ejecución se define sobre el entorno de sistema de la estrategia de verificación del capítulo 26, con el despliegue completo de la plataforma: el servidor de aplicación, la base de datos MySQL, los pesos de los modelos y el worker de la cola. Los criterios de éxito se fijan en función del comportamiento esperado del sistema, y los umbrales concretos de tiempo se ajustan tras una primera campaña de medición sobre el entorno de despliegue, de modo que los criterios reflejen la infraestructura real de la plataforma.

36.1. Rendimiento de la inferencia

Las pruebas PR-001 y PR-002 verifican el rendimiento del diagnóstico asistido, asociadas al requisito RNF-019, que exige que el sistema gestione la carga de los modelos de forma eficiente. La prueba PR-001 verifica que la primera inferencia sobre una radiografía se completa en un tiempo razonable, asumiendo el coste de la carga de los pesos del modelo; la prueba PR-002 verifica que las inferencias posteriores sobre el mismo modelo son casi instantáneas, al reutilizar los pesos ya cargados en memoria. La tabla siguiente resume estas verificaciones y sus criterios.

ID	Verificación	Criterio de éxito
PR-001	La primera inferencia sobre una radiografía se completa en un tiempo razonable.	La primera consulta con cada arquitectura carga el modelo y produce el resultado en un tiempo que no hace incómodo el uso de la herramienta.
PR-002	Las inferencias posteriores son casi instantáneas.	Las consultas siguientes sobre el mismo modelo se completan de forma sensiblemente más rápida que la primera, al reutilizar los pesos ya cargados.

Tabla x - Rendimiento de la inferencia

La verificación de estas pruebas se apoya en el mecanismo de caché de modelos implementado en el motor de inferencia, descrito en la codificación del capítulo 31: la primera consulta con cada arquitectura paga la carga de los pesos, mientras que las posteriores reutilizan el modelo en memoria. La medición se realiza sobre el entorno de despliegue con radiografías de prueba, registrando el tiempo de cada consulta para comparar la primera con las siguientes.

36.2. Ejecución sin bloqueo de la interfaz

La prueba PR-003 verifica la ejecución sin bloqueo de la interfaz durante las tareas de larga duración, asociada al requisito RNF-020, que exige que los procesos intensivos se ejecuten de forma asíncrona sin bloquear la interfaz. La prueba comprueba que, durante un entrenamiento o un análisis de explicabilidad del laboratorio, la plataforma continúa respondiendo a las peticiones del usuario. La tabla siguiente resume esta verificación.

ID	Verificación	Criterio de éxito
PR-003	La interfaz permanece operativa durante una tarea de larga duración.	Durante un entrenamiento o un análisis de explicabilidad, la plataforma continúa respondiendo a las peticiones del usuario sin bloquearse.

Tabla x - Ejecución sin bloqueo de la interfaz

La verificación de esta prueba se apoya en la ejecución asíncrona implementada en el capítulo 30: el worker procesa los trabajos fuera del ciclo de petición mediante el executor de eventos, de modo que la interfaz permanece operativa mientras la cola procesa los entrenamientos. La prueba se realiza lanzando una tarea de larga duración en el laboratorio y comprobando, durante su ejecución, que la aplicación responde a otras peticiones, como la consulta del estado de la cola o la navegación entre las ventanas.

36.3. Acceso concurrente

Las pruebas PR-004 y PR-005 verifican la capacidad de la plataforma para soportar el acceso concurrente, asociadas al requisito RNF-021, que exige la gestión del acceso concurrente mediante el pool de conexiones. La prueba PR-004 comprueba que el sistema soporta una carga razonable de usuarios concurrentes sin una degradación significativa del tiempo de respuesta; la prueba PR-005 comprueba que el pool de conexiones gestiona el acceso concurrente a la base de datos sin errores de conexión ni contención excesiva. La tabla siguiente resume estas verificaciones.

ID	Verificación	Criterio de éxito
PR-004	El sistema soporta la carga de varios usuarios concurrentes.	Con un número razonable de usuarios concurrentes, el tiempo de respuesta no se degrada de forma significativa.
PR-005	El pool de conexiones gestiona el acceso concurrente a la base de datos.	Las peticiones concurrentes se atienden sin errores de conexión ni contención excesiva.

La verificación de estas pruebas se apoya en la gestión del pool de conexiones implementado en la capa de persistencia, descrito en la codificación del capítulo 29: el pool configura un número de conexiones que las peticiones concurrentes reutilizan, de modo que no se abre una conexión por operación. La prueba se realiza con un conjunto de usuarios concurrentes que ejecutan operaciones habituales de la plataforma, observando el tiempo de respuesta y la ausencia de errores de conexión.

36.4. Condiciones de medición

Las pruebas de rendimiento se ejecutan sobre el entorno de despliegue descrito en el capítulo 27, con la configuración de infraestructura del sistema en operación. Las condiciones de medición incluyen la infraestructura del servidor —el proceso de aplicación, la base de datos MySQL y los artefactos del aprendizaje automático— y la carga de trabajo de la prueba, que emplea radiografías de prueba y operaciones representativas de los flujos de la plataforma. Los umbrales concretos de tiempo para las pruebas PR-001 a PR-004 se fijan tras una primera campaña de medición sobre el entorno de despliegue, de modo que los criterios reflejen la infraestructura real y no valores especulativos; la estrategia de verificación del capítulo 26 establece que estos umbrales se ajustan con los datos medidos.

Las pruebas de rendimiento completan la verificación de los requisitos no funcionales de rendimiento y capacidad de respuesta de la plataforma, junto con las verificaciones automatizadas de los capítulos 35 y 36. Con las pruebas de rendimiento descritas, la parte de verificación aborda en el capítulo siguiente las pruebas de sistema y la validación funcional de los flujos completos de uso, que verifican el comportamiento integral de la plataforma sobre el despliegue real.

37. Pruebas de sistema y validación funcional

Las pruebas de sistema completan la verificación de vitalXAI con la comprobación integral de los flujos de uso de la plataforma, ejercitando la interacción del usuario con la interfaz a través de todos los subsistemas implicados, de manera análoga al uso real del sistema. Este capítulo describe las pruebas de sistema del plan de pruebas del capítulo 17 —los códigos PE-001 a PE-006—, presentando cada flujo completo con sus pasos, su criterio de éxito y el resultado de su verificación. Las pruebas se ejecutan manualmente sobre el despliegue real de la plataforma, con el entorno de sistema de la estrategia de verificación del capítulo 26, y cada flujo se documenta con una captura de pantalla real del sistema en funcionamiento, insertada en la ilustración correspondiente.

Las pruebas de sistema verifican los seis flujos completos de la plataforma: el flujo clínico del diagnóstico, la gestión del historial, el flujo del laboratorio de entrenamiento, la ejecución asíncrona de la cola, la supervisión administrativa y las capacidades transversales de idioma y tema. Cada prueba recorre las operaciones del flujo sobre la interfaz real, desde el registro o el acceso hasta la obtención del resultado, y comprueba que el comportamiento coincide con el especificado en los casos de uso del análisis. La ejecución de cada prueba se apoya en la documentación de operación de la plataforma y en las cuentas de acceso del entorno de despliegue.

37.1. PE-001: Flujo clínico completo

La prueba PE-001 verifica el flujo clínico completo del diagnóstico asistido: la subida de una radiografía, la selección del modelo, la solicitud del diagnóstico y la visualización del resultado y de los mapas de explicabilidad. La prueba se inicia desde el panel de diagnóstico del usuario autenticado y recorre las operaciones descritas en los casos de uso CU-005 a CU-010 y CU-037. Los pasos de la prueba son los siguientes.

1. El usuario autenticado accede al panel de diagnóstico.
2. El usuario sube una radiografía de tórax en la zona de carga.
3. El usuario selecciona una arquitectura de inteligencia artificial.
4. El usuario solicita el diagnóstico y espera el procesamiento en la cola.
5. El sistema muestra el resultado con la etiqueta, la confianza y el modelo empleado.
6. El usuario visualiza los mapas de explicabilidad.

El criterio de éxito de la prueba es que el flujo completo se realiza sin errores y el profesional obtiene el diagnóstico, su confianza, los mapas de explicabilidad y el informe de la consulta. El resultado esperado se confirma con la captura del panel de diagnóstico mostrando el resultado.

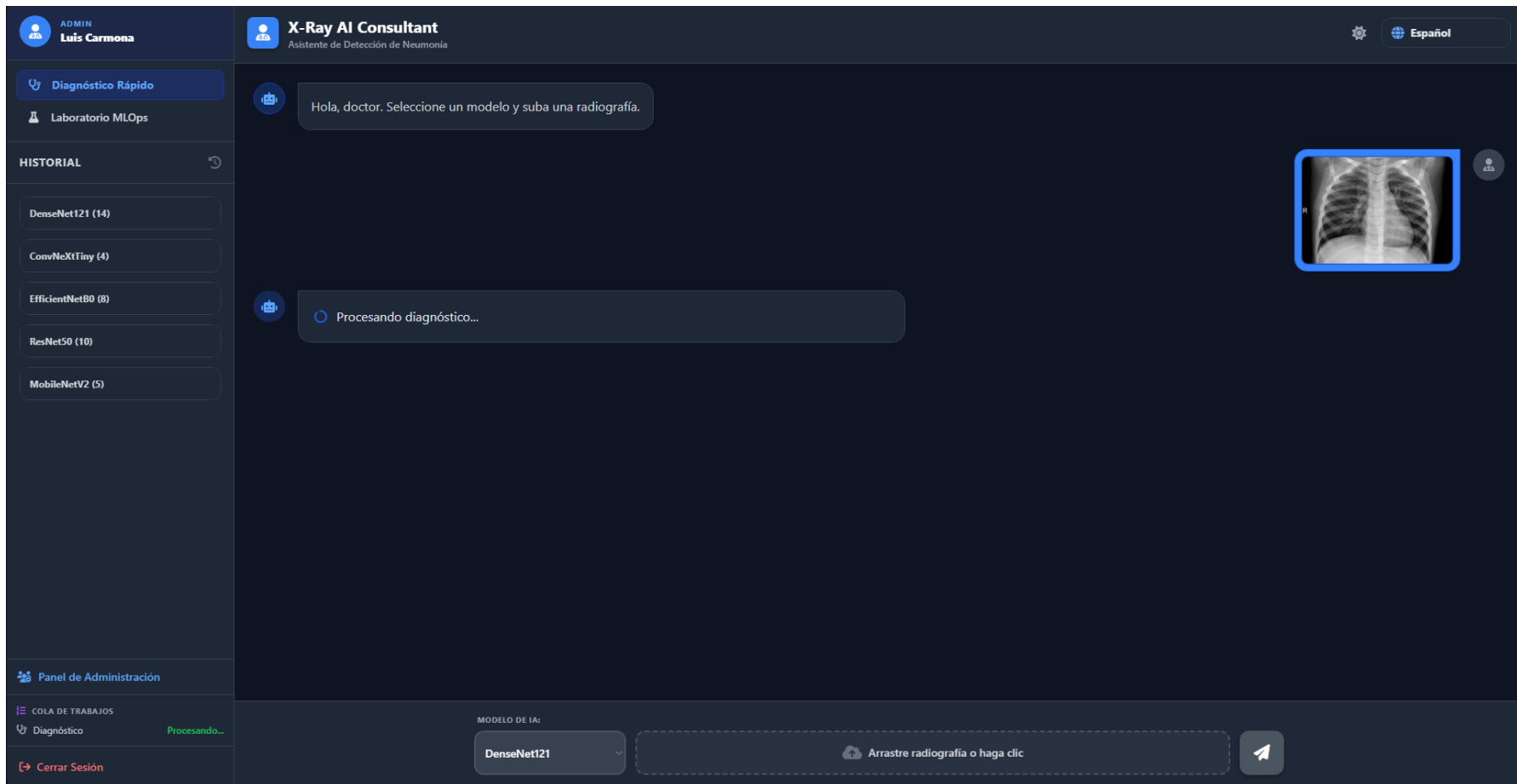


Ilustración x - Visualización de Diagnóstico Rápido y Cola de Trabajo tras el lanzamiento de un diagnóstico

ADMIN
Luis Carmona

Diagnóstico Rápido

Laboratorio MLOps

HISTORIAL

DenseNet121 (15)

ConvNeXtTiny (4)

EfficientNetB0 (8)

ResNet50 (10)

MobileNetV2 (5)

Panel de Administración

Cerrar Sesión

X-Ray AI Consultant
Asistente de Detección de Neumonía

← Volver

Detalle de Consulta
2026-08-25 20:54

RenombrarEliminar

RADIOGRAFÍA ORIGINAL

MAPAS DE CALOR XAI

DIAGNÓSTICO
Neumonía

CONFIANZA
91.78%

MODELO
DenseNet121

PACIENTE
Paciente sin nombre
DenseNet121 #14

MODELO DE IA:
DenseNet121

Arrastre radiografía o haga clic

Ilustración x - Visualización de resultados y mapas de explicabilidad

37.2. PE-002: Gestión del historial

La prueba PE-002 verifica la gestión del historial de consultas: la consulta del listado, el detalle de una consulta, el renombrado y la eliminación. La prueba se realiza desde el panel de diagnóstico, sobre las consultas del usuario, y recorre las operaciones descritas en los casos de uso CU-011 a CU-014. Los pasos de la prueba son los siguientes.

1. El usuario consulta su historial de consultas en el panel.
2. El usuario abre el detalle de una consulta del listado.
3. El usuario renombra la consulta con un nuevo nombre visible.
4. El usuario elimina una consulta, confirmando la operación.

El criterio de éxito de la prueba es que todas las operaciones del historial se ejecutan correctamente y las consultas se recuperan tras cada operación, con el aislamiento de datos entre usuarios respetado. El resultado esperado se confirma con la captura del historial y del detalle de una consulta.

ADMIN

Luis Carmona

Diagnóstico Rápido

Laboratorio MLOps

HISTORIAL

DenseNet121 (15)

ConvNeXtTiny (4)

EfficientNetB0 (8)

ResNet50 (10)

MobileNetV2 (5)

2026-07-29 22:48

Neumonía

Paciente sin nombre MobileNetV2 #5

2026-03-23 18:09

Normal

Paciente sin nombre MobileNetV2 #4

2026-03-23 18:03

Normal

Paciente sin nombre MobileNetV2 #3

2026-03-22 20:42

Neumonía

Paciente sin nombre MobileNetV2 #2

2026-03-22 20:42

Normal

Juan Manuel

Panel de Administración

Cerrar Sesión

X-Ray AI Consultant

Asistente de Detección de Neumonía

← Volver

Detalle de Consulta

2026-03-22 20:42

Renombrar

Eliminar

RADIOGRAFÍA ORIGINAL

R

MAPAS DE CALOR XAI

Importance Score

Importance Map

Relevance

Global Importance

DIAGNÓSTICO

CONFIANZA

MODELO

PACIENTE

Normal

88.04%

MobileNetV2

Juan Manuel

MODELO DE IA:

DenseNet121

Arrastre radiografía o haga clic

Ilustración x - Visualización del historial antes de la eliminación de un Diagnóstico y tras el cambio de nombre del paciente

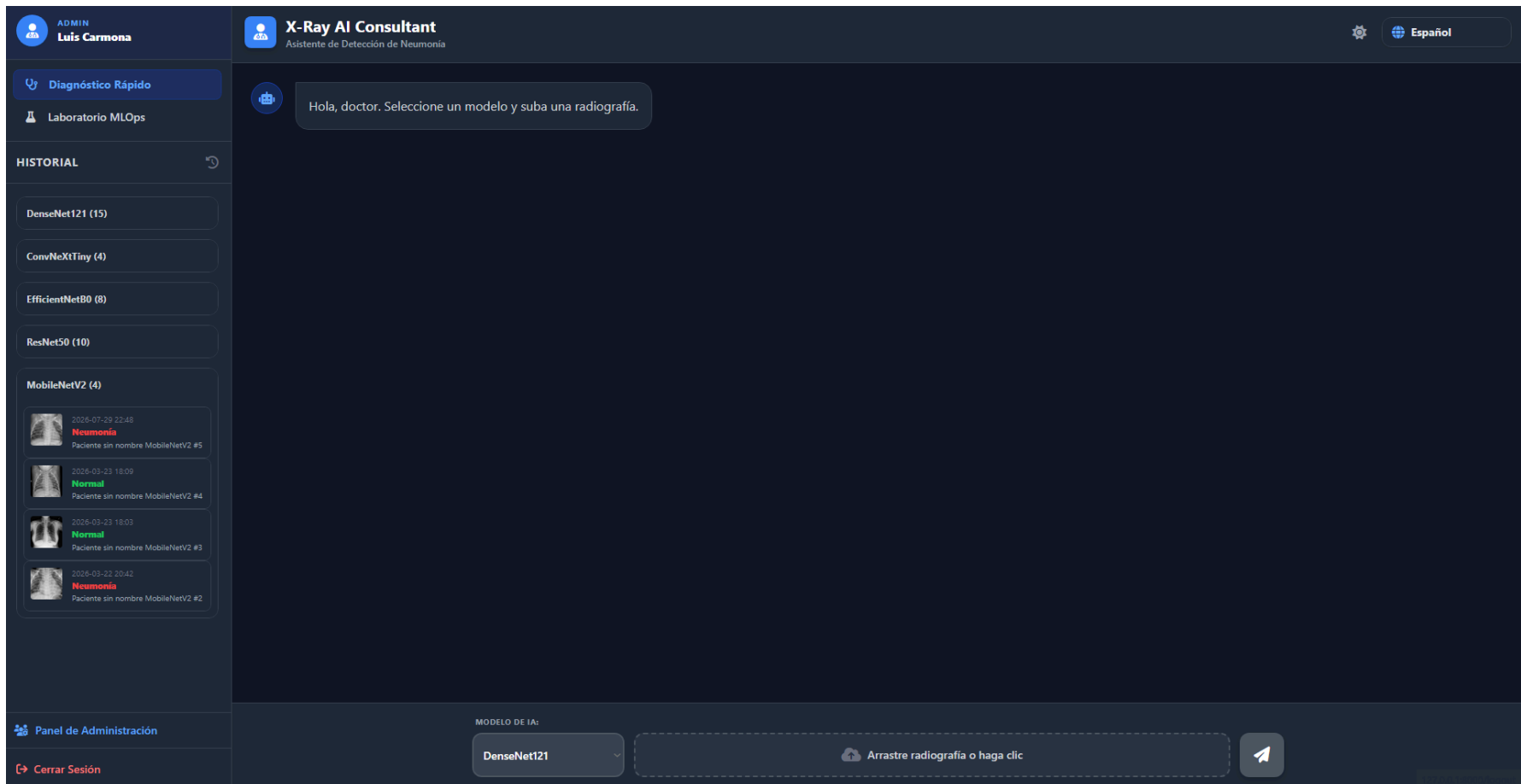


Ilustración x - Visualización del historial tras la eliminación de un Diagnóstico

37.3. PE-003: Flujo del laboratorio

La prueba PE-003 verifica el flujo del laboratorio de entrenamiento: la conversación con el asistente, el lanzamiento del experimento, la consulta de las sesiones y de los resultados, y la generación del informe PDF de la sesión. La prueba se realiza desde el laboratorio del usuario investigador y recorre las operaciones descritas en los casos de uso CU-015 a CU-030. Los pasos de la prueba son los siguientes.

1. El usuario accede al laboratorio de entrenamiento.
2. El usuario configura el experimento con el asistente conversacional y selecciona la carpeta del dataset.
3. El usuario lanza el experimento y lo encola en segundo plano.
4. El usuario consulta la sesión, el ranking de modelos y los resultados de un modelo.
5. El usuario genera el informe PDF de la sesión.

El criterio de éxito de la prueba es que el experimento se configura y lanza mediante el asistente, se ejecuta en segundo plano y sus resultados e informe se obtienen sin errores. El resultado esperado se confirma con la captura de la vista de resultados de la sesión.

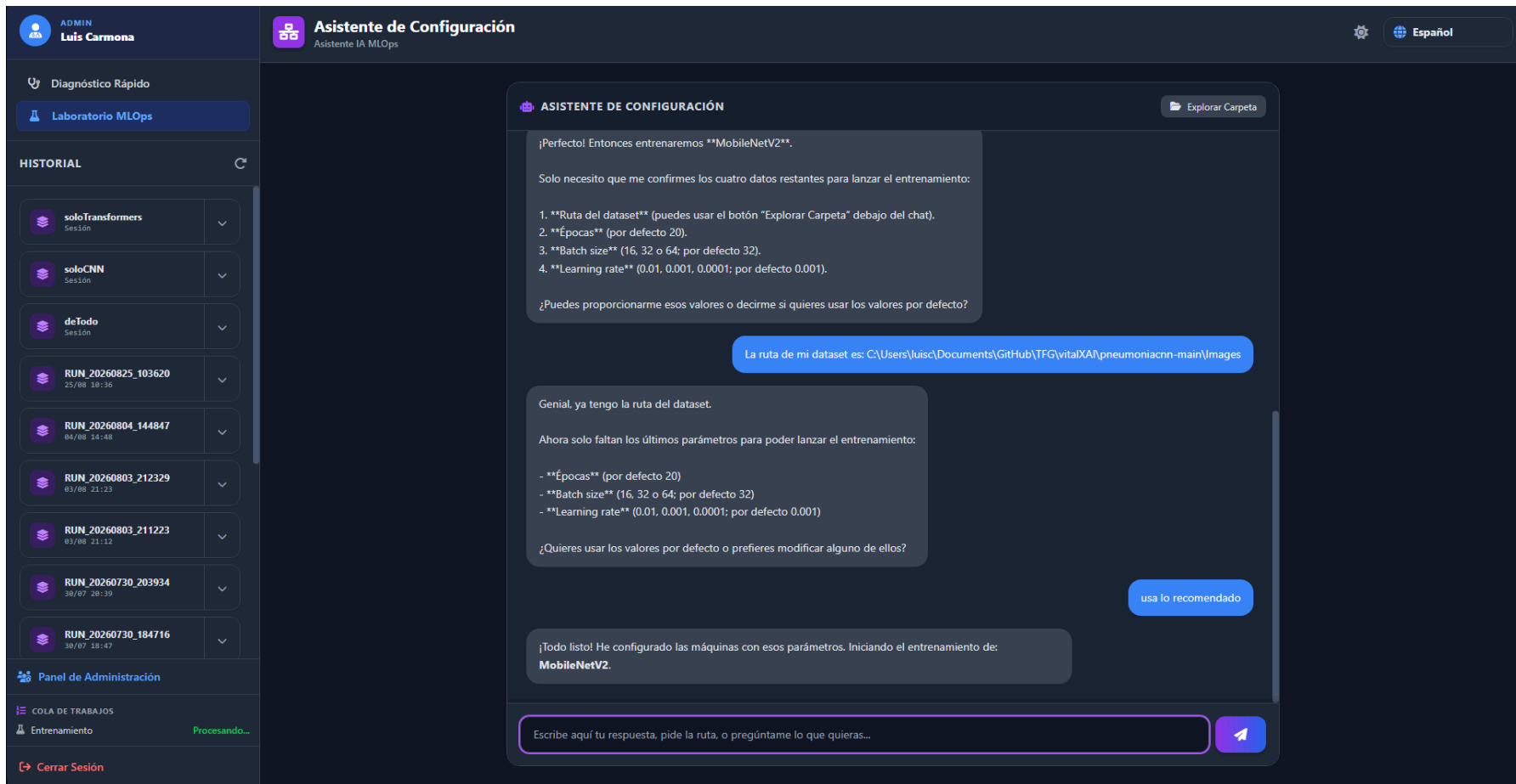


Ilustración x - Visualización del panel del Laboratorio y la Cola de Trabajo tras lanzar un entrenamiento

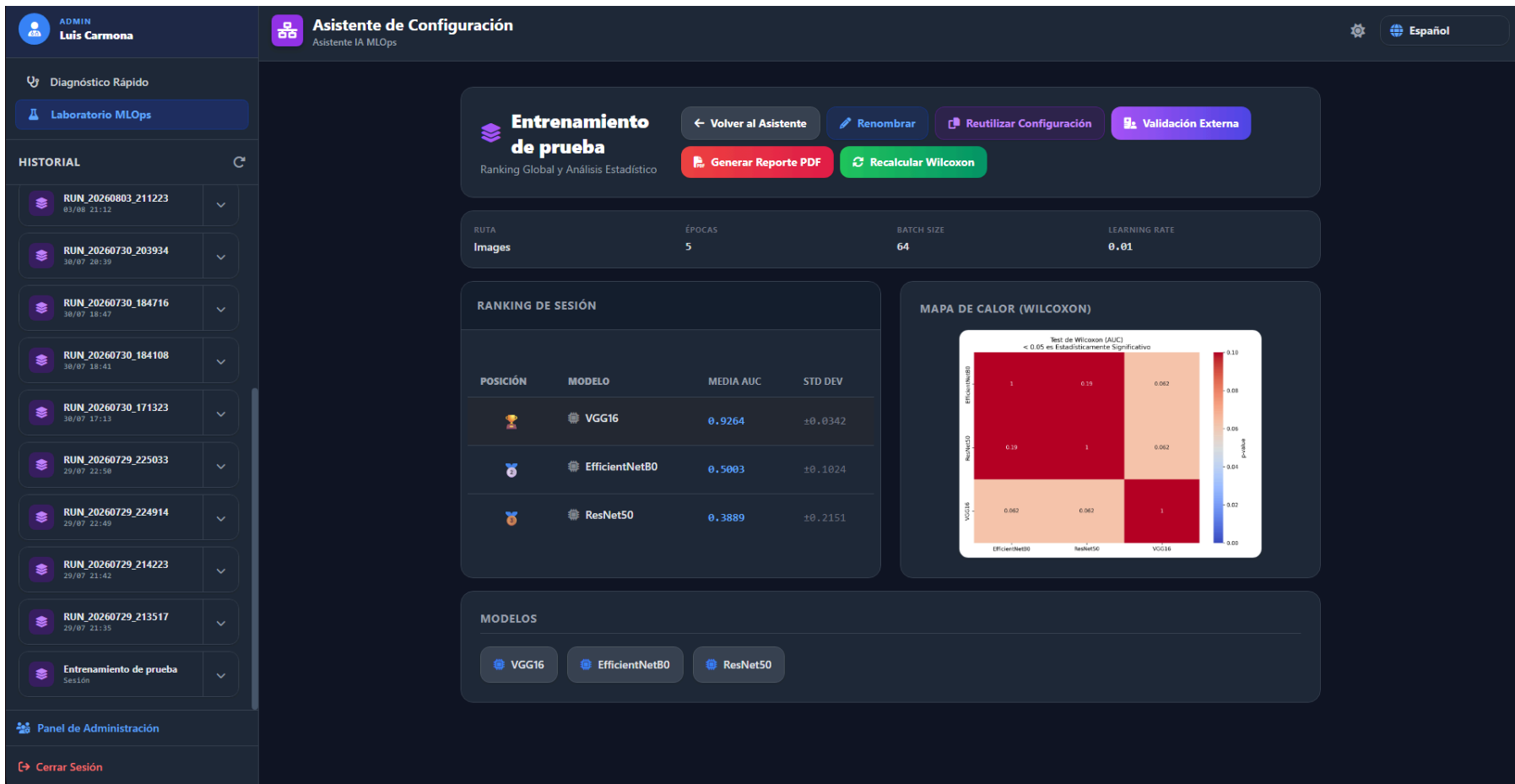


Ilustración x - Visualización de los resultados del entrenamiento

X-RAY CONSULTANT AI - MEDICAL REPORT

Protocolo MLOps: Deep Learning para Detección de Neumonía

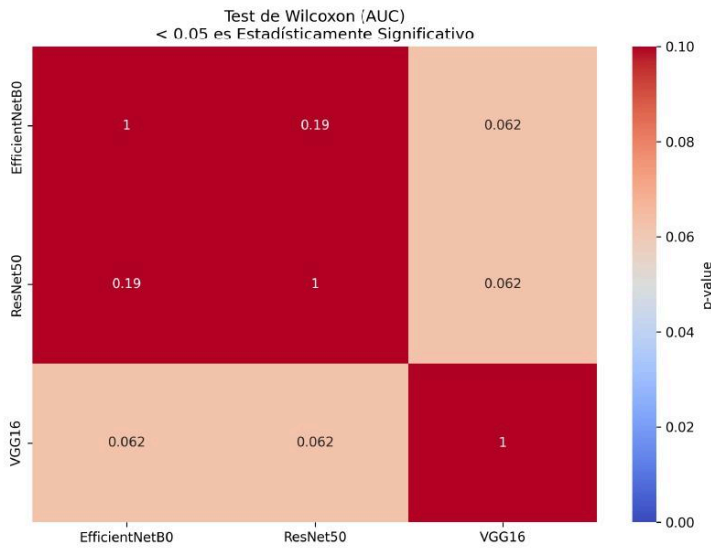
1. CONFIGURACIÓN DEL SISTEMA Y PARÁMETROS

ID Sesión:	Entrenamiento de prueba
Fecha:	25/08/2026 21:18
Dataset:	C:\\Users\\luisc\\Documents\\GitHub\\TFG\\vitalXA\\pneumoniaccn-main\\Images
Modelos:	ResNet50, EfficientNetB0, VGG16
Hiperparámetros:	Epochs: 5 Batch: 64 LR: 0.01

2. RENDIMIENTO GLOBAL (K-FOLD CROSS-VALIDATION)

Arquitectura de Modelo	Media AUC	Desviación Estándar
VGG16	0.9263888888888887	0.034207997810417584
EfficientNetB0	0.5003472222222222	0.10237698693409425
ResNet50	0.3888888888888884	0.21509008394847717

Matriz de Significancia Estadística (P-Values):



37.4. PE-004: Ejecución asíncrona de la cola

La prueba PE-004 verifica la ejecución asíncrona de la cola de trabajos: la consulta del estado de la cola y la cancelación de un trabajo pendiente. La prueba se realiza desde el panel de diagnóstico o el laboratorio, y recorre las operaciones descritas en los casos de uso CU-034 y CU-035. Los pasos de la prueba son los siguientes.

1. El usuario consulta el panel de la cola de trabajos.
2. El usuario comprueba el estado de sus trabajos (encolado, en ejecución, completado o fallido).
3. El usuario cancela un trabajo pendiente, confirmando la operación.

El criterio de éxito de la prueba es que el usuario conoce el estado de sus trabajos y puede cancelar un trabajo pendiente, mientras que un trabajo en ejecución no se interrumpe. El resultado esperado se confirma con la captura del panel de la cola de trabajos.

Diagnóstico Rápido

Laboratorio MLOps

HISTORIAL

soloTransformers

Sesión

soloCNN

Sesión

deTodo

Sesión

RUN_20260825_103620

25/08 10:36

RUN_20260804_144847

04/08 14:48

RUN_20260803_212329

03/08 21:23

RUN_20260803_211223

03/08 21:12

RUN_20260730_203934

30/07 20:39

Panel de Administración

COLA DE TRABAJOS

Validación Externa

Posición #2

Entrenamiento

Modelos: MobileNetV2, MobileNetV3Large, ConvNeXtTiny, deit, swin_base

Diagnóstico

Posición #1

Entrenamiento

Procesando...

Cerrar Sesión

ASISTENTE DE CONFIGURACIÓN

Explorar Carpeta

¡Hola! Soy tu asistente de MLOps. Para empezar un nuevo entrenamiento de modelos para detectar neumonía, necesito que me digas qué arquitecturas quieres usar y la ruta de la carpeta de tus imágenes. ¿Con qué empezamos?

Escribe aquí tu respuesta, pide la ruta, o pregúntame lo que quieras...

Ilustración x - Visualización de la Cola de Trabajo

Diagnóstico Rápido

Laboratorio MLOps

HISTORIAL

soloTransformers

Sesión

soloCNN

Sesión

deTodo

Sesión

RUN_20260825_103620

25/08 10:36

RUN_20260804_144847

04/08 14:48

RUN_20260803_212329

03/08 21:23

RUN_20260803_211223

03/08 21:12

RUN_20260730_203934

30/07 20:39

Panel de Administración

COLA DE TRABAJOS

Validación Externa

Entrenamiento

Entrenamiento

Posición #1

Posición #2

Procesando...

Cerrar Sesión

ASISTENTE DE CONFIGURACIÓN

Explorar Carpeta

¡Hola! Soy tu asistente de MLOps. Para empezar un nuevo entrenamiento de modelos para detectar neumonía, necesito que me digas qué arquitecturas quieres usar y la ruta de la carpeta de tus imágenes. ¿Con qué empezamos?

Escribe aquí tu respuesta, pide la ruta, o pregúntame lo que quieras...

Ilustración x - Visualización de la Cola de Trabajo tras la eliminación del Diagnóstico pendiente

37.5. PE-005: Supervisión administrativa

La prueba PE-005 verifica la supervisión administrativa de la plataforma: el listado de usuarios, la consulta de las consultas de un usuario y su detalle. La prueba se realiza desde el panel del usuario con rol de administrador, y recorre las operaciones descritas en los casos de uso CU-031 a CU-033. Los pasos de la prueba son los siguientes.

1. El administrador abre el panel de administración desde su ventana.
2. El administrador consulta el listado de usuarios con sus recuentos de actividad.
3. El administrador consulta las consultas de un usuario concreto y el detalle de una consulta.

El criterio de éxito de la prueba es que el administrador supervisa la actividad de un usuario concreto desde el panel de administración, con la comprobación de rol aplicada por el sistema. El resultado esperado se confirma con la captura del diálogo de administración.

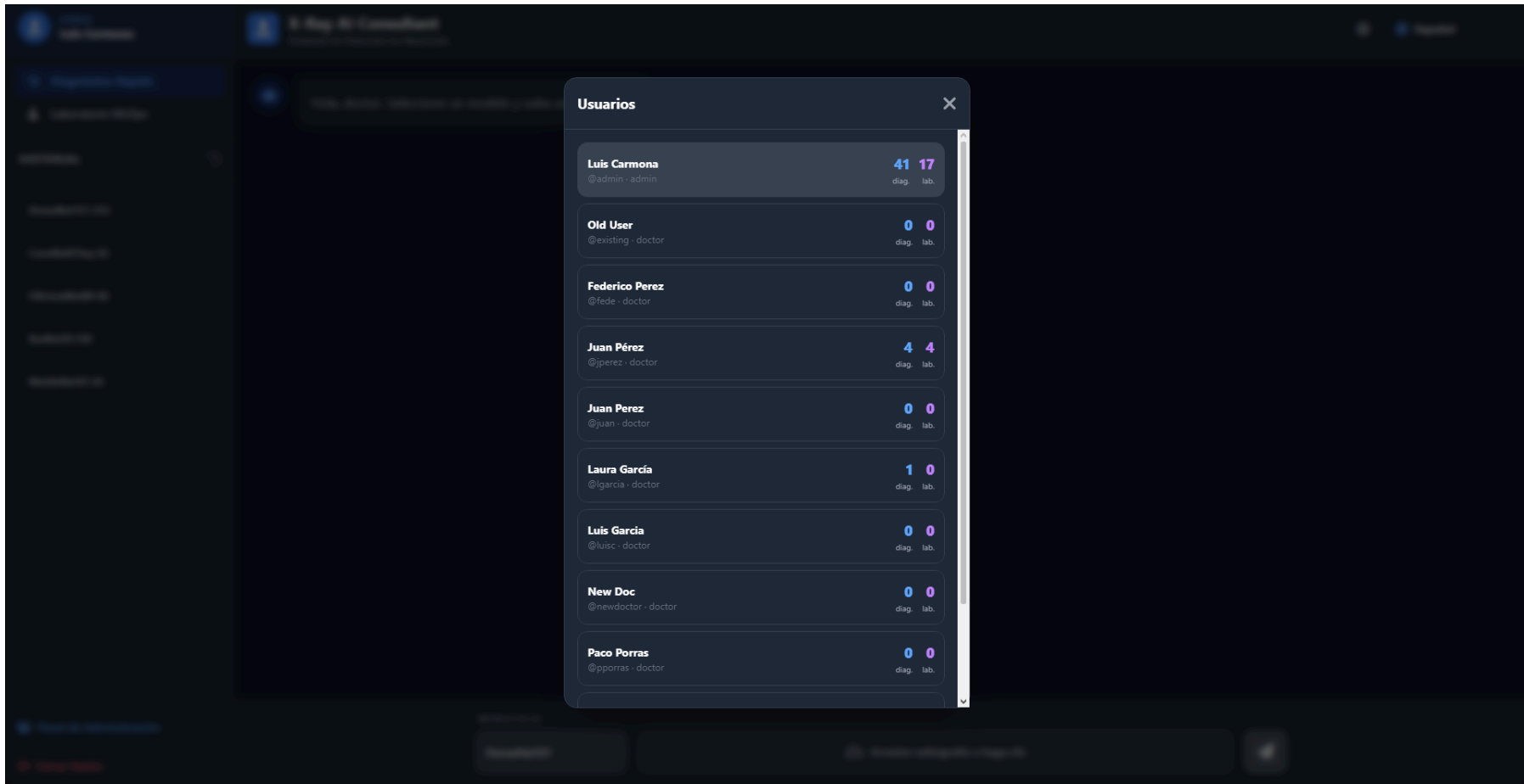


Ilustración x - Visualización del Panel de Administrador

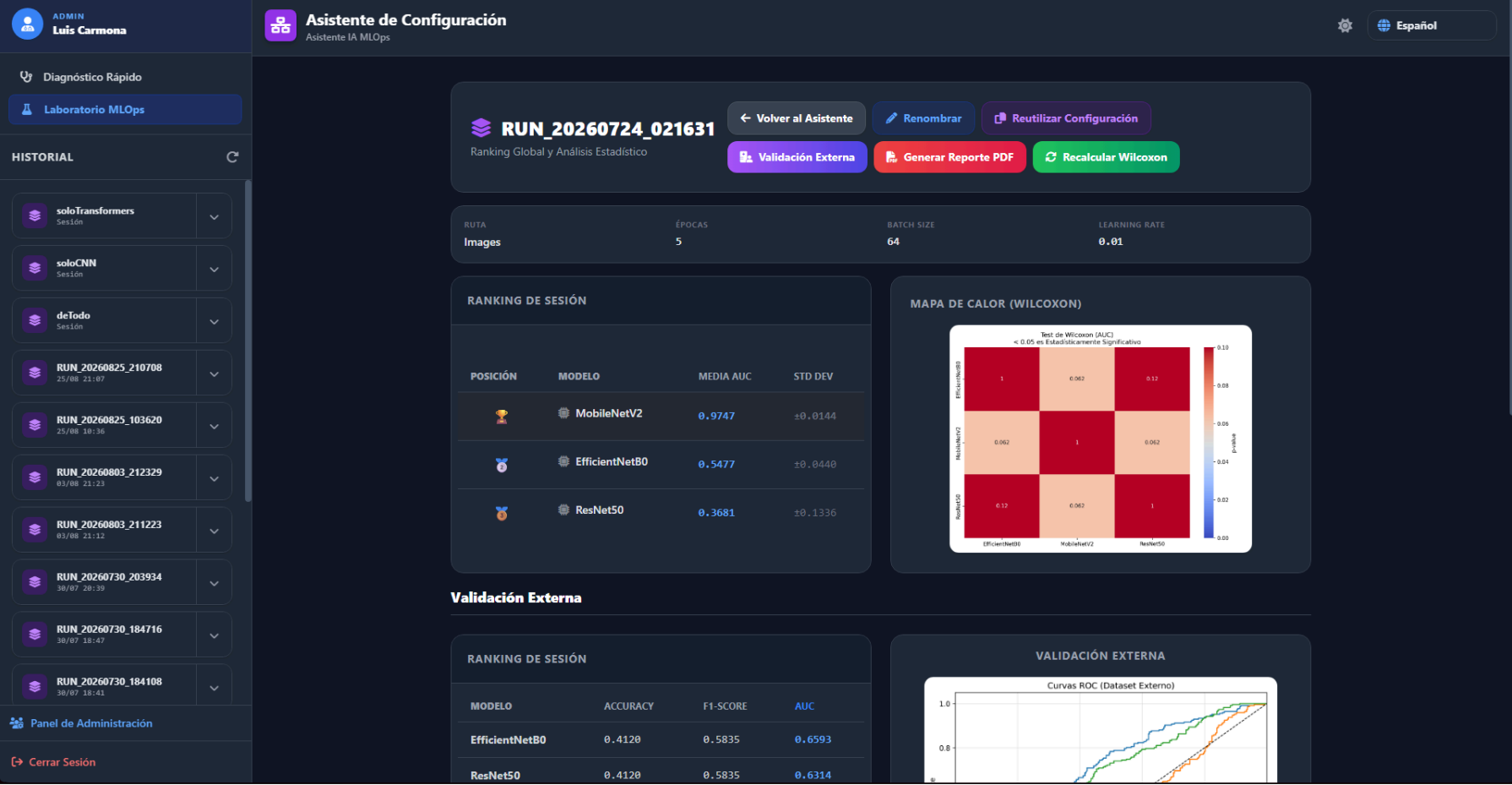


Ilustración x - Visualización de los detalles de una consulta de otro usuario

37.6. PE-006: Capacidades transversales

La prueba PE-006 verifica las capacidades transversales de la plataforma: el cambio de idioma y el cambio del tema visual de la interfaz. La prueba se realiza desde cualquiera de las ventanas del sistema y recorre las operaciones descritas en los casos de uso CU-004 y CU-036. Los pasos de la prueba son los siguientes.

1. El usuario cambia el idioma de la interfaz mediante el selector de idioma.
2. El sistema aplica las traducciones en toda la interfaz sin recargar la página.
3. El usuario alterna el tema visual entre el claro y el oscuro.
4. El sistema aplica el tema en toda la interfaz y conserva la preferencia.

El criterio de éxito de la prueba es que el idioma y el tema se aplican en toda la interfaz sin interrumpir la navegación. El resultado esperado se confirma con la captura de una ventana en el idioma y el tema seleccionados.

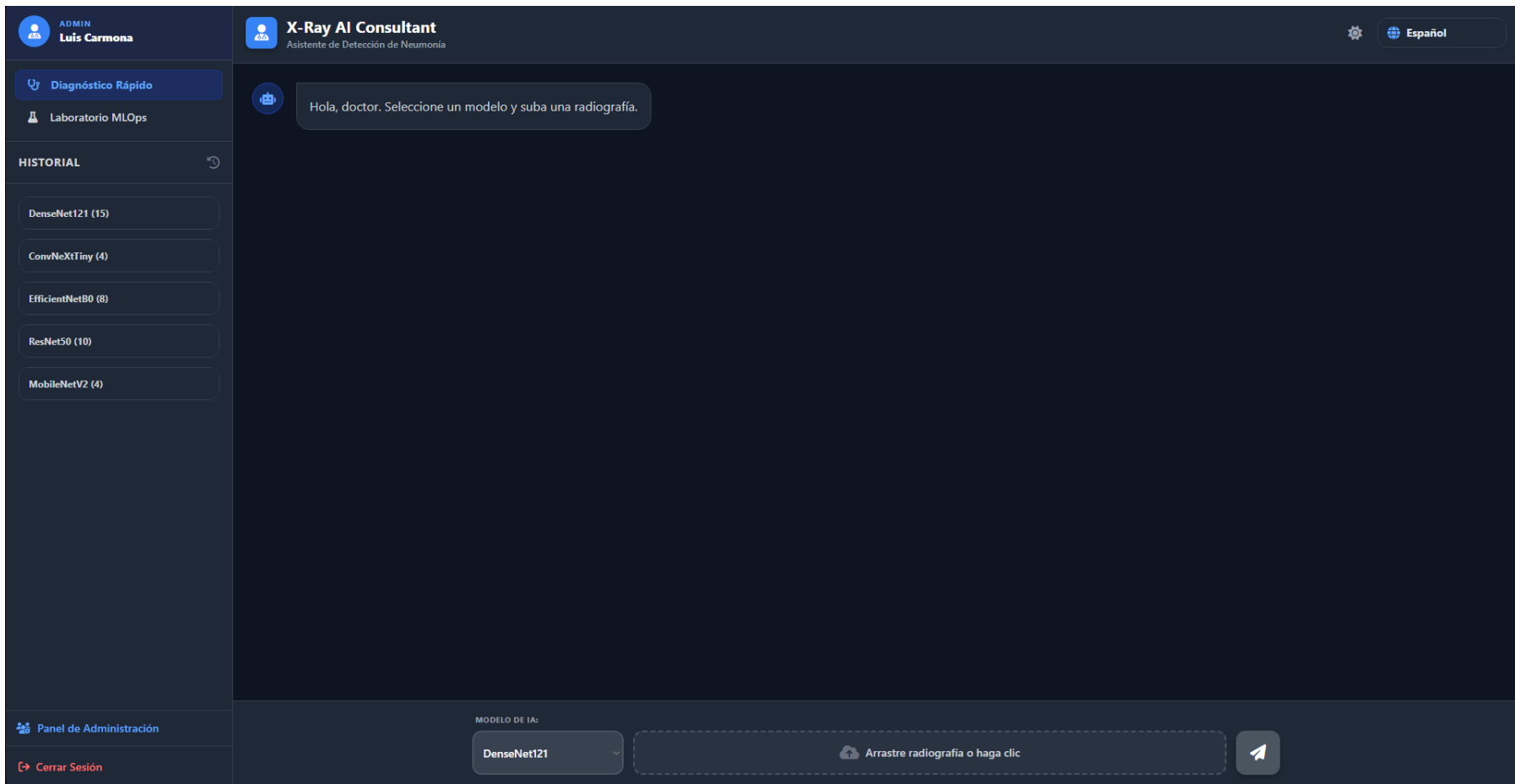


Ilustración x - Visualización de la interfaz en Español y estilo oscuro

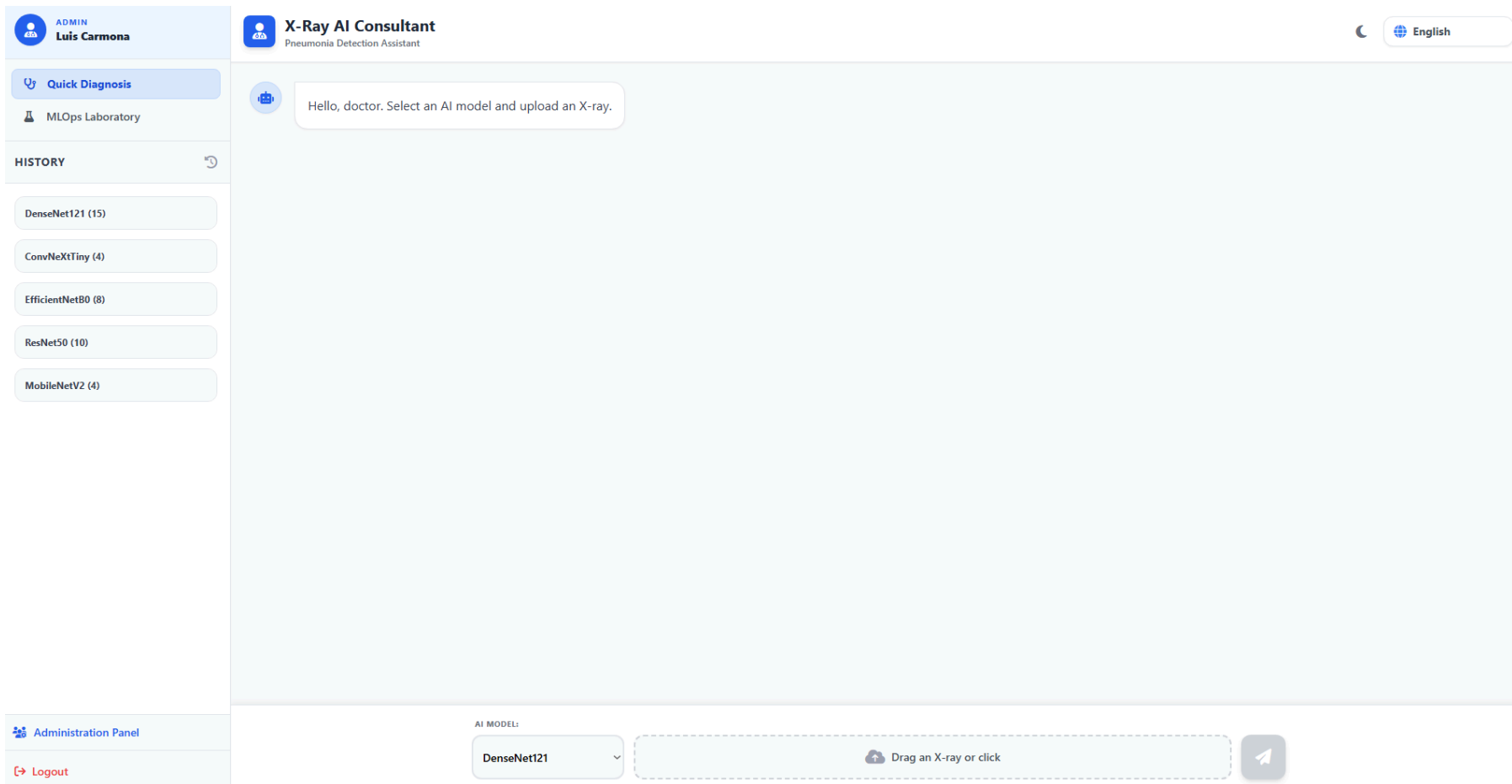


Ilustración x - Visualización de la interfaz en Inglés y estilo claro

37.7. Resultados de la validación funcional

Las seis pruebas de sistema verifican los flujos completos de uso de la plataforma sobre el despliegue real, cubriendo el diagnóstico, el historial, el laboratorio, la cola, la administración y las capacidades transversales. La tabla siguiente resume el resultado de la validación funcional.

ID	Flujo verificado	Resultado
PE-001	Flujo clínico completo del diagnóstico	Superada.
PE-002	Gestión del historial de consultas	Superada.
PE-003	Flujo del laboratorio de entrenamiento	Superada.
PE-004	Ejecución asíncrona de la cola de trabajos	Superada.
PE-005	Supervisión administrativa	Superada.
PE-006	Capacidades transversales (idioma y tema)	Superada.

Tabla x - Resultados de la validación funcional

Con la superación de las pruebas de sistema, la parte de verificación de vitalXAI queda completa: las pruebas automatizadas de los capítulos 35 y 36 verificaron los componentes, los flujos entre subsistemas y los mecanismos de seguridad; las pruebas de rendimiento del capítulo 37 verificaron los requisitos de rendimiento y capacidad de respuesta; y las pruebas de sistema de este capítulo verificaron los flujos completos de uso sobre el despliegue real. El conjunto de la verificación confirma que el sistema implementado satisface los requisitos funcionales y no funcionales del análisis y las condiciones definidas en el plan de pruebas.